

# Multimedia

## Vorlesung

Hannes Kaufmann

Virtual & Augmented Reality Group

Research Unit Computer Graphics

Institute of Visual Computing & Human-Centered Technology

Based on material by Prof. Christian Breiteneder

# Psychoacoustics

- Scientific study of sound perception and audiology: How humans perceive various sounds
- **Study on your own** the Youtube course “Psychoakustik” (German)  
[https://www.youtube.com/watch?v=8\\_TxY93L8xE&list=PLftcEygAp\\_6lk8SpKNesLim\\_omxfXMbmo&index=1](https://www.youtube.com/watch?v=8_TxY93L8xE&list=PLftcEygAp_6lk8SpKNesLim_omxfXMbmo&index=1)

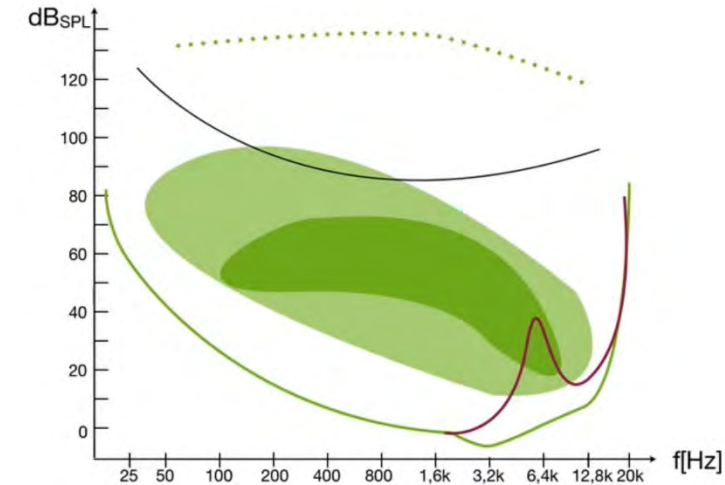
11 parts, ~ 107 min (total)

# Inverted Classroom: Psychoacoustics Questions

- How do we hear?
- What happens inside the cochlea?
- What are the different ways to judge the location & direction of sound?
- What is 1 MEL? What is 1 BARK?
- What is critical bandwidth?
- How large is a critical band?
- What is the connection between critical bands and hair cells?

# Inverted Classroom: Psychoacoustics Questions

- Explain and annotate the figure ->
- What is masking?
- What is temporal masking?
- Describe a psychoacoustic phenomenon (e.g. Shepard Scala, Tritonus Paradoxon,...)





# Multimedia

## Vorlesung

Hannes Kaufmann

Research Unit Virtual and Augmented Reality  
Institute of Visual Computing & Human-Centered Technology

Based on material by Prof. Christian Breiteneder

# Psychoacoustics

- Scientific study of sound perception and audiology: How humans perceive various sounds
- **Study on your own** the Youtube course “Psychoakustik” (German)  
[https://www.youtube.com/watch?v=8\\_TxY93L8xE&list=PLftcEygAp\\_6lk8SpKNesLim\\_omxfXMbmo&index=1](https://www.youtube.com/watch?v=8_TxY93L8xE&list=PLftcEygAp_6lk8SpKNesLim_omxfXMbmo&index=1)

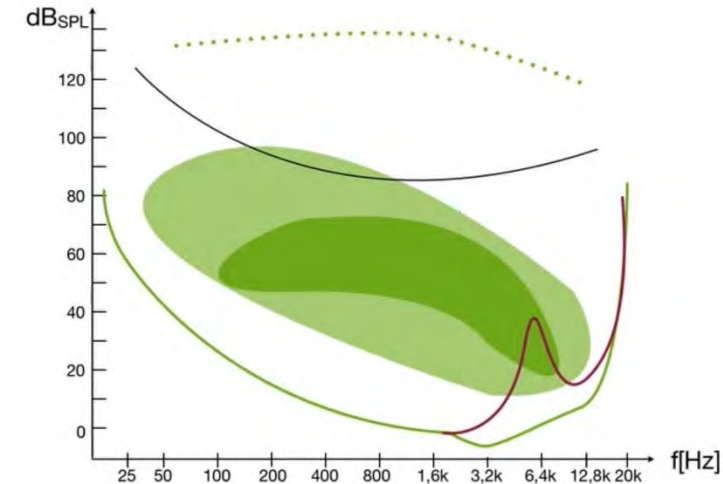
11 parts, ~ 107 min (total)

# Inverted Classroom: Psychoacoustics Questions

- How do we hear?
- What happens inside the cochlea?
- What are the different ways to judge the location & direction of sound?
- What is 1 MEL? What is 1 BARK?
- What is critical bandwidth?
- How large is a critical band?
- What is the connection between critical bands and hair cells?

# Inverted Classroom: Psychoacoustics Questions

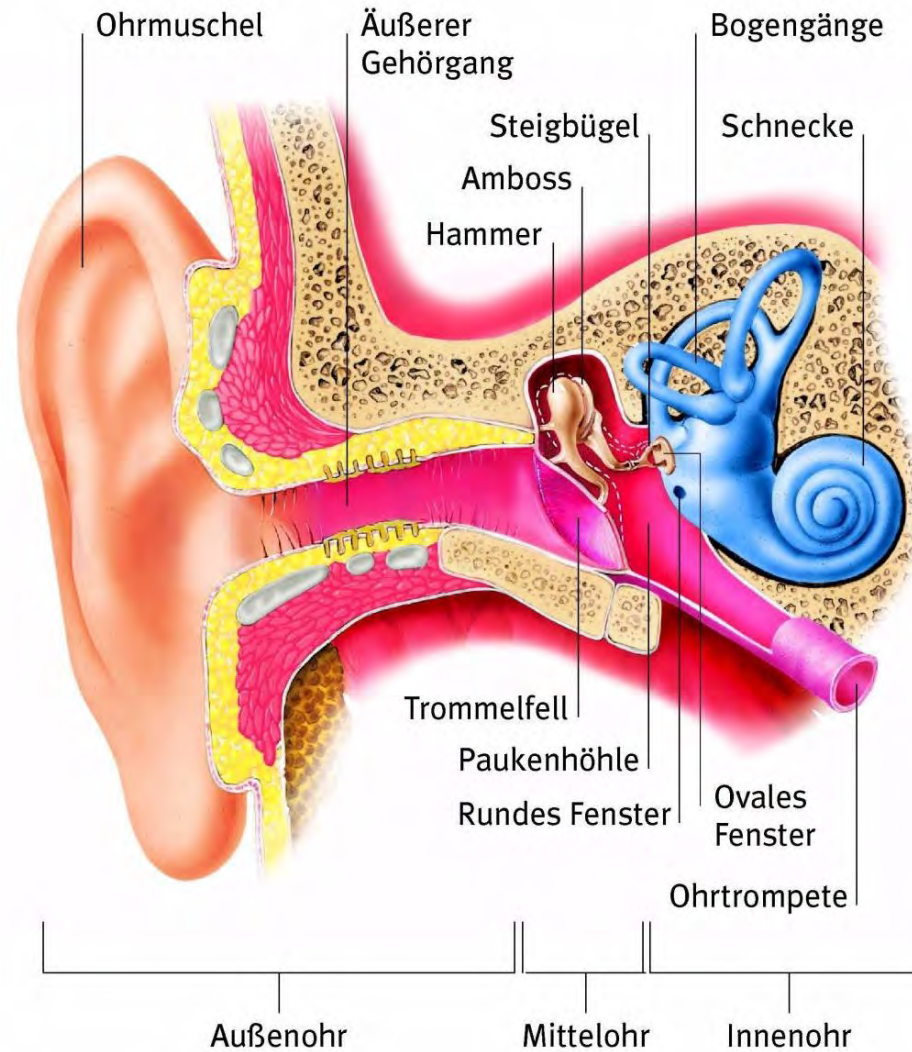
- Explain and annotate the figure ->
- What is masking?
- What is temporal masking?
- Describe a psychoacoustic phenomenon (e.g. Shepard Scala, Tritonus Paradoxon,...)





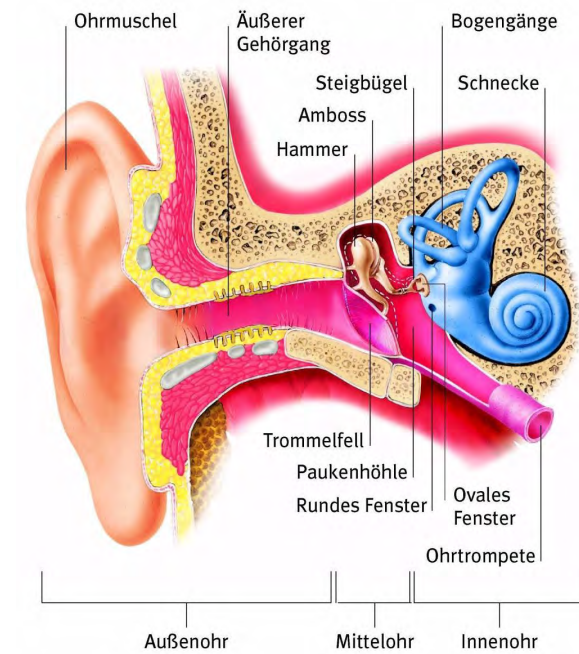
# Psychoacoustics - Summary

# The Human Ear



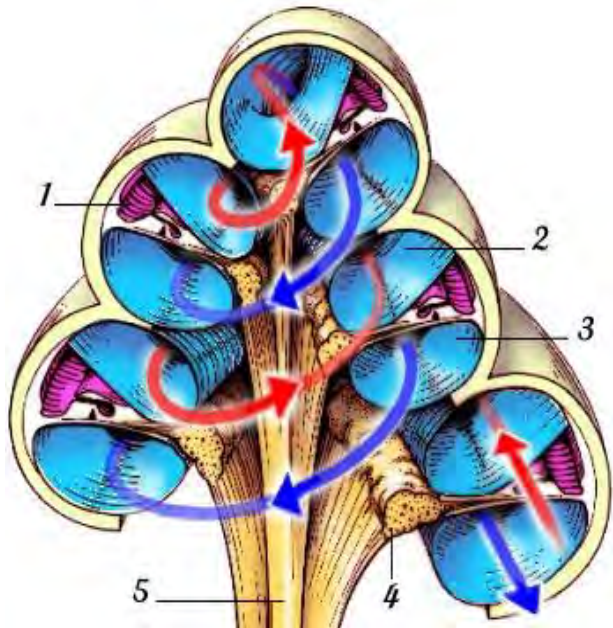
# Hearing

- Sound waves hit the eardrum
- Eardrum transmits vibration to the ossicular chain in the middle ear;
  - as the last link the foot plate (“Steigbügel”) and the
  - oval window are moved.
- Thereby oscillation of the endolymph in scala vestibuli





# Cochlea



1. cochlear duct
2. scala vestibuli
3. scala tympani
4. spiral ganglion
5. auditory nerve fibres

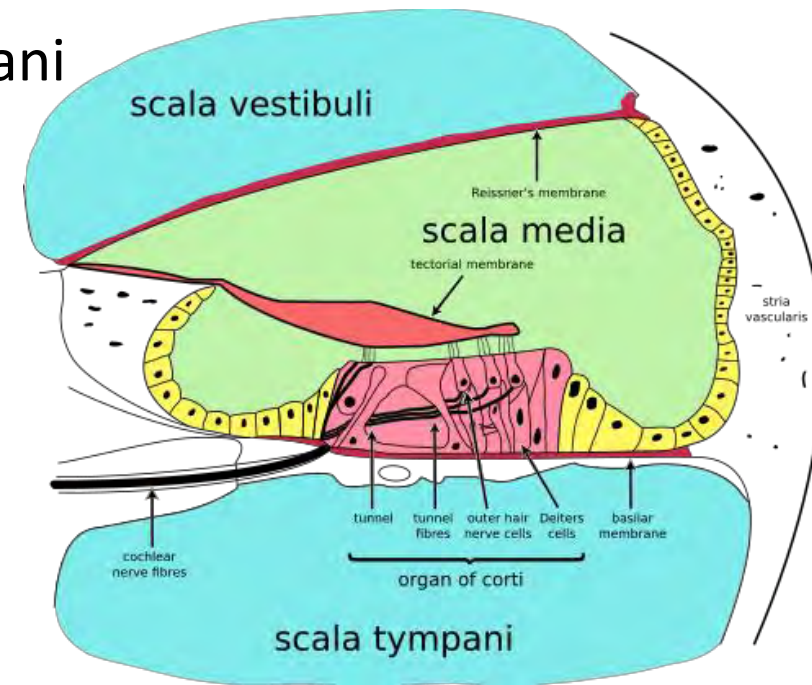
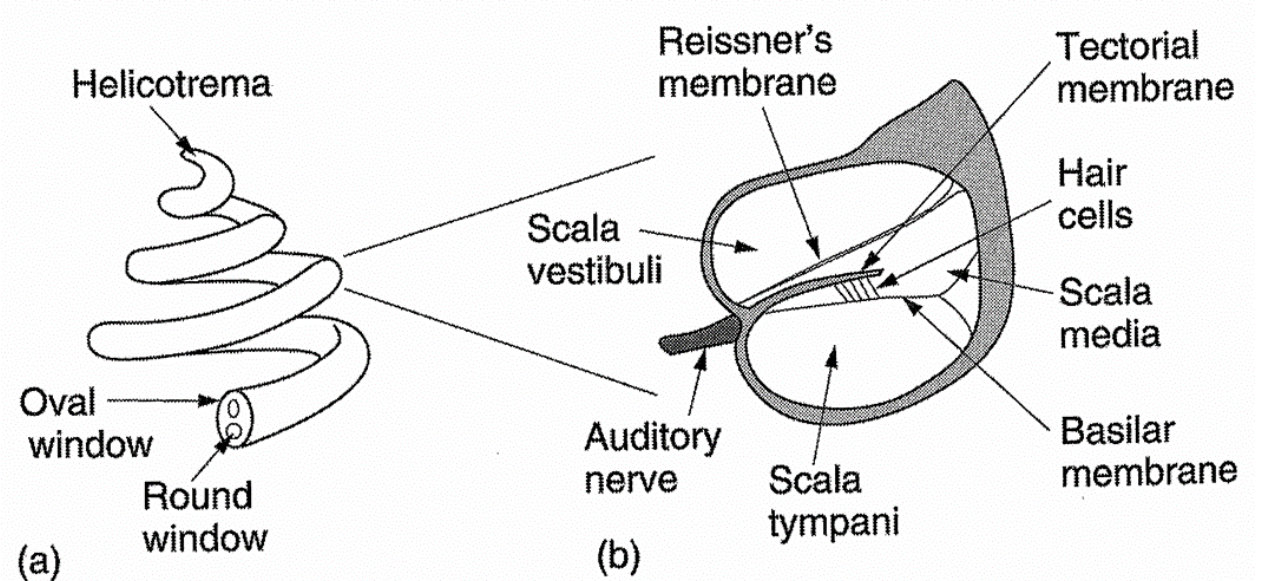


2 mm

- The red arrow is from the oval window
- The blue arrow points to the round window
- The cochlea is about 2 mm in diameter

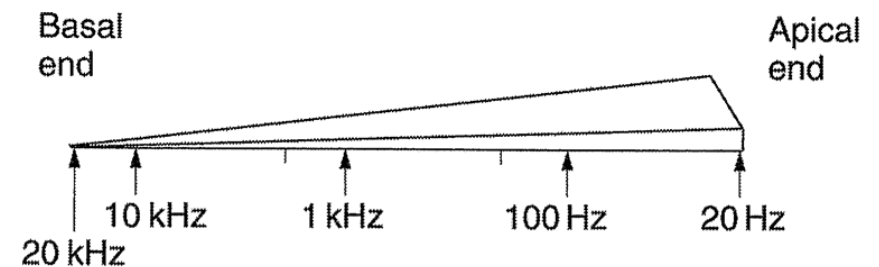
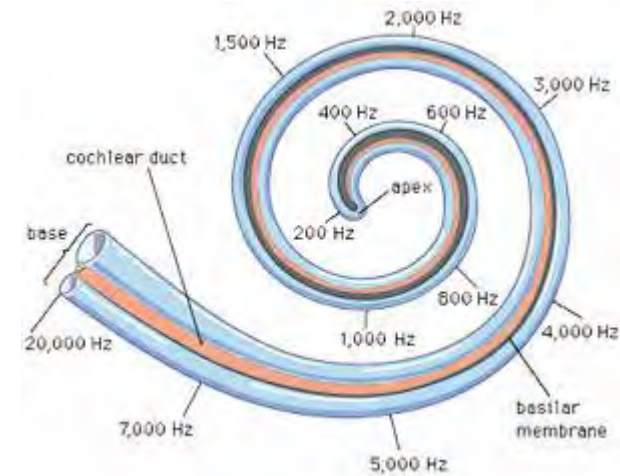
# Hearing Cochlea

- 3 superimposed fluid-filled passages:
- *Scala vestibuli* (vestibule staircase)
- *Scala media* or cochlear duct (spiral duct) and
- *Scala tympani* (tympani staircase)
- Basilar membrane separates Scala Media and Scala tympani
- On basilar membrane approx. 15,000 hair cells
- *Outer hair cells* (3 rows) serve to amplify sound traveling waves within the cochlea; prefilter.
- *Inner hair cells* (1 row) perform conversion of mechanical oscillations into nerve impulses (transduction).



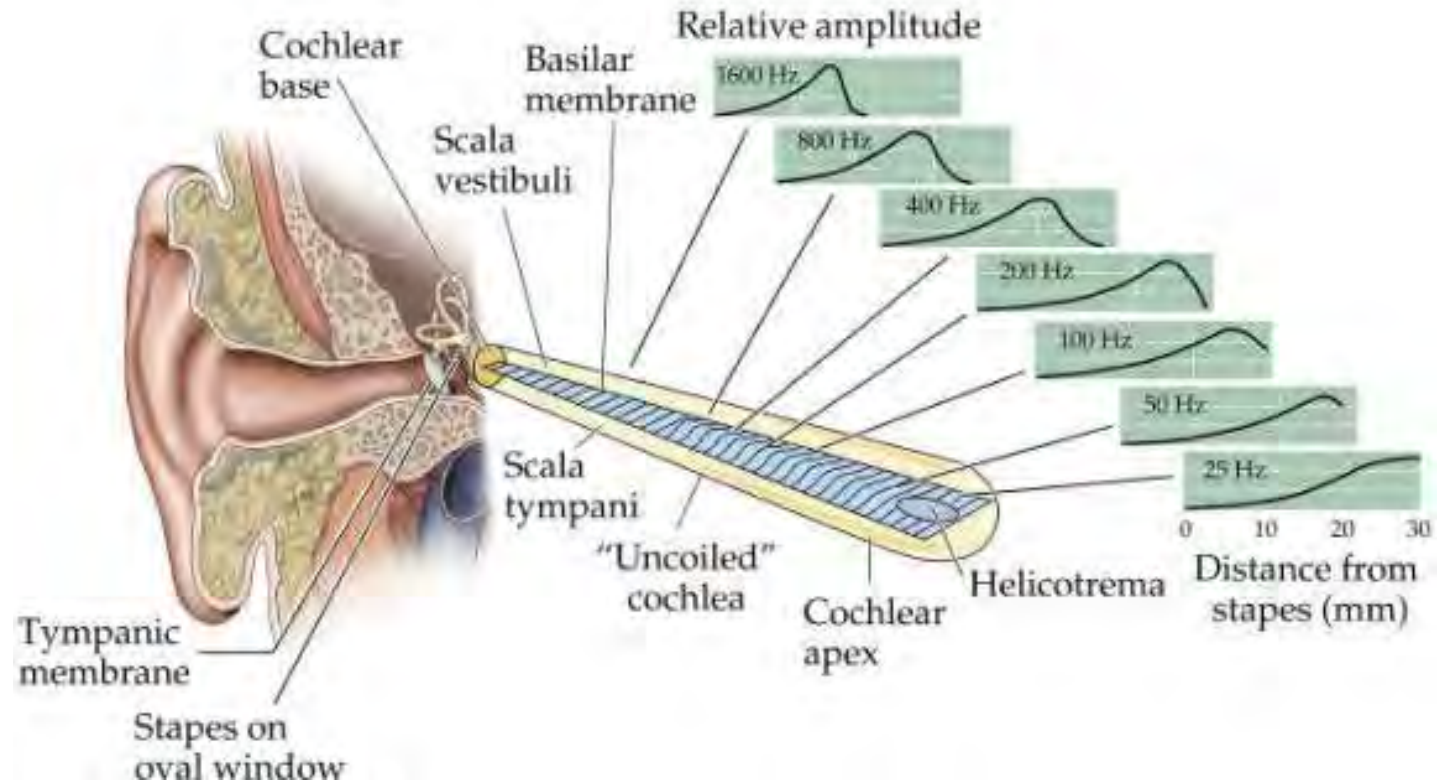
# Basilar membrane

- Width of basilar membrane increases from oval window to helicotrema.
- This changes the mechanical properties (mass loading, stiffness, damping) and thus also the vibration properties of the system as a function of the distance to the helicotrema.
- Basilar membrane resonates at different points for different frequencies.
- High frequencies have a deflection maximum near the oval window, low frequencies, on the other hand, only near the helicotrema.



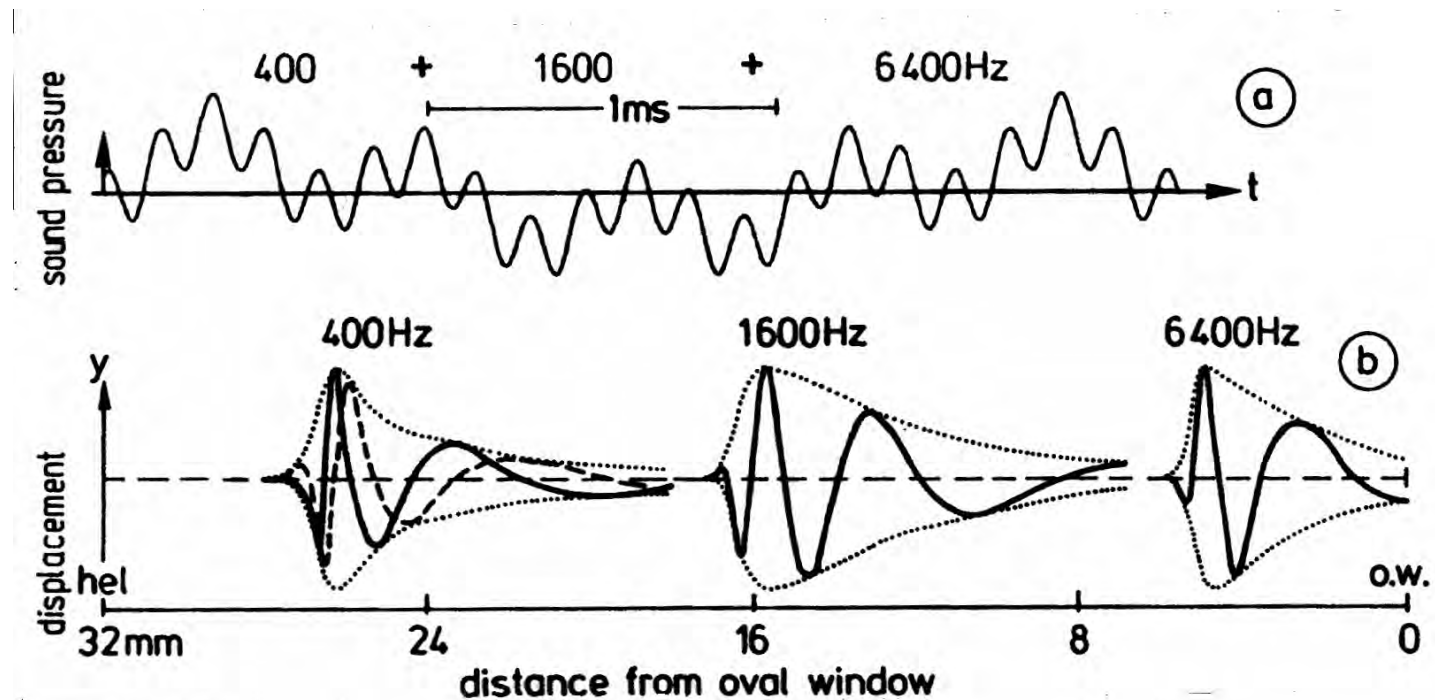


# Basilar membrane

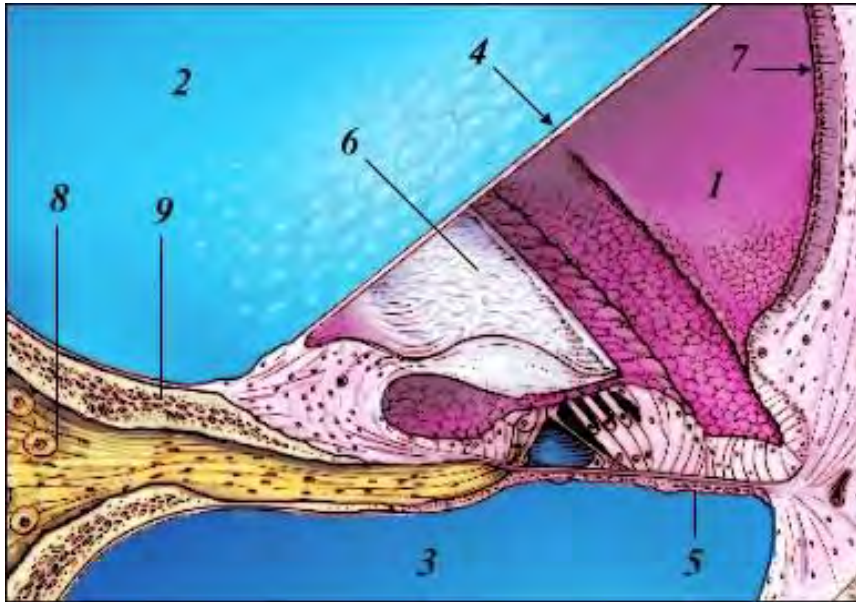


# Basilar membrane

Frequency-to-place transformation:



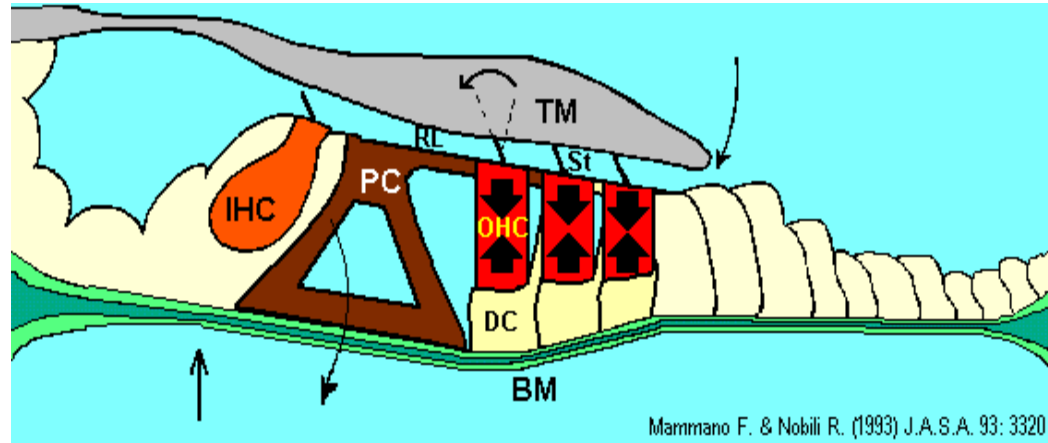
# Cholea



1. Cochlear duct
2. Scala vestibuli
3. Scala tympani
4. Reissner's membrane
5. Basilar membrane
6. Tectorial membrane
7. Stria vascularis
8. Nerve fibres
9. Bony spiral lamina

The organ of Corti is on top of the basilar membrane  
under the tectorial membrane

# Organ of Corti



- Tectorial membrane (TM)
- Basilar membrane (BM)
- Inner haircells (IHC)
- Outer haircells (OHC)
- Deiters' cells (DC)
- Reticular lamina (RL)
- Pillar cells (PC)
- Stereocilia (St)

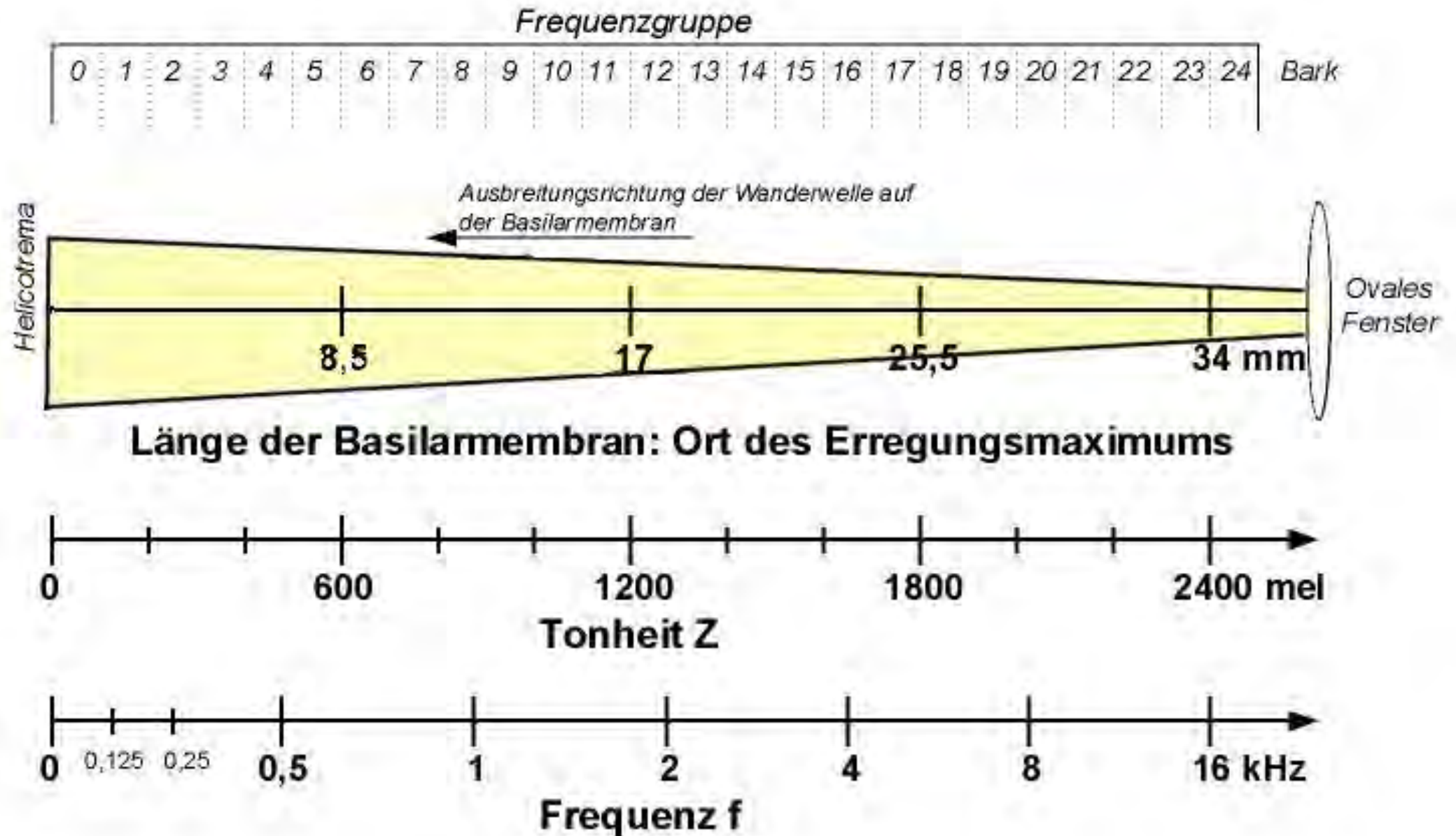
# Hair Cells

- At the resonance point, the vibrations are amplified and thus the inner hair cells are stimulated more strongly.
- Beyond the resonance point, the oscillations are strongly damped, the corresponding frequency hardly propagates further (frequency selectivity)
- High frequencies (resonance point near the oval window) do not cause excitation of the inner hair cells for low frequencies,
- low frequencies, however, also excite the hair cells responsible for high frequencies.



# Frequency Groups, Critical Bands

- Hearing divides the basilar membrane into **24 sections of equal length** = **frequency groups**.
- **Nerve impulses from a frequency group are evaluated together ([pitch], volume, sound, direction).**
- Width of a frequency group is about 100 Hz for frequencies up to 500 Hz and a ~100Hz – 4kHz above 500 Hz (minor third = “kleine Terz”). Corresponds to approx. 1 Bark = 100 Mel.
- **Critical Band / Critical Bandwidth** is not fixed (e.g. 100-200Hz) but each tone is creating a critical band around itself.
- Unlike critical bands, which are based on psychoacoustic principles, frequency groups are more of a practical construct used for organizing and processing audio signals.



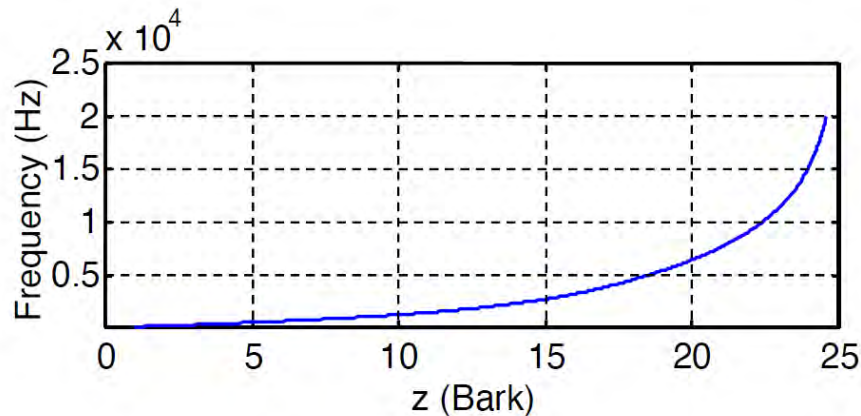
1 Bark = 100 Mel = 1,3mm on the basilar membrane = 150 hair cells = 27 JND  
Verdopplung in Mel entspricht Verdopplung der wahrgenommenen Tonhöhe.

# Bark Scale and Critical Bandwidth

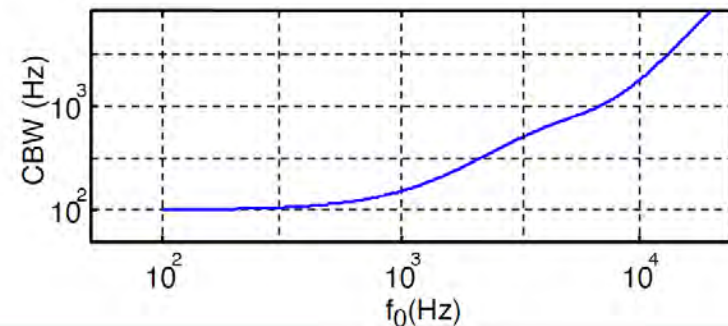
- A scale that converts frequency (Hz) into units of critical bandwidth

$$z(f) = 13 \arctan(0.00076 f) + 3.5 \arctan \left[ \left( \frac{f}{7500} \right)^2 \right] \quad (\text{Bark})$$

named after Heinrich Barkhausen (who proposed the first subjective measurements of loudness)



Bark scale



Critical bandwidth

Depending on the base frequency  $f_0$  the width of the frequency group/critical band can be seen  
About 100Hz up to 500Hz,  
for higher frequencies (20kHz) up to 4kHz bandwidth

# Frequency Groups

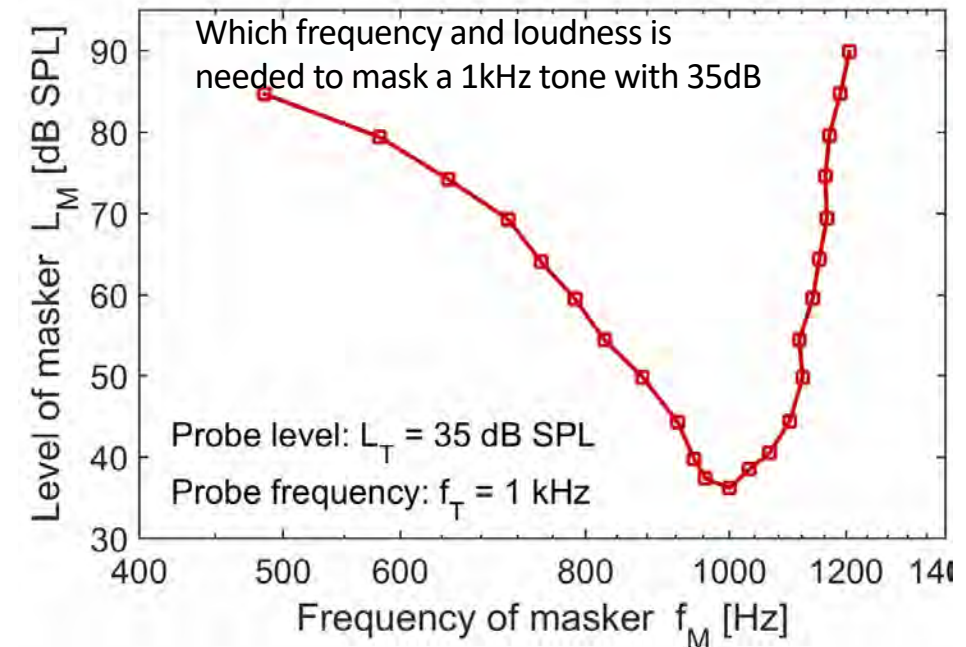
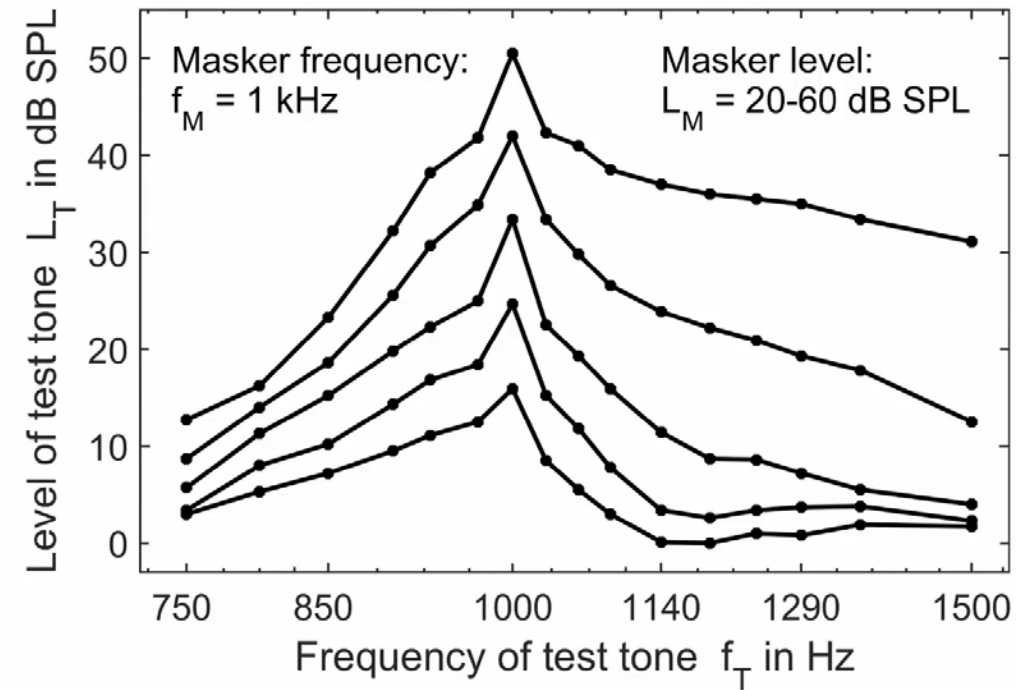
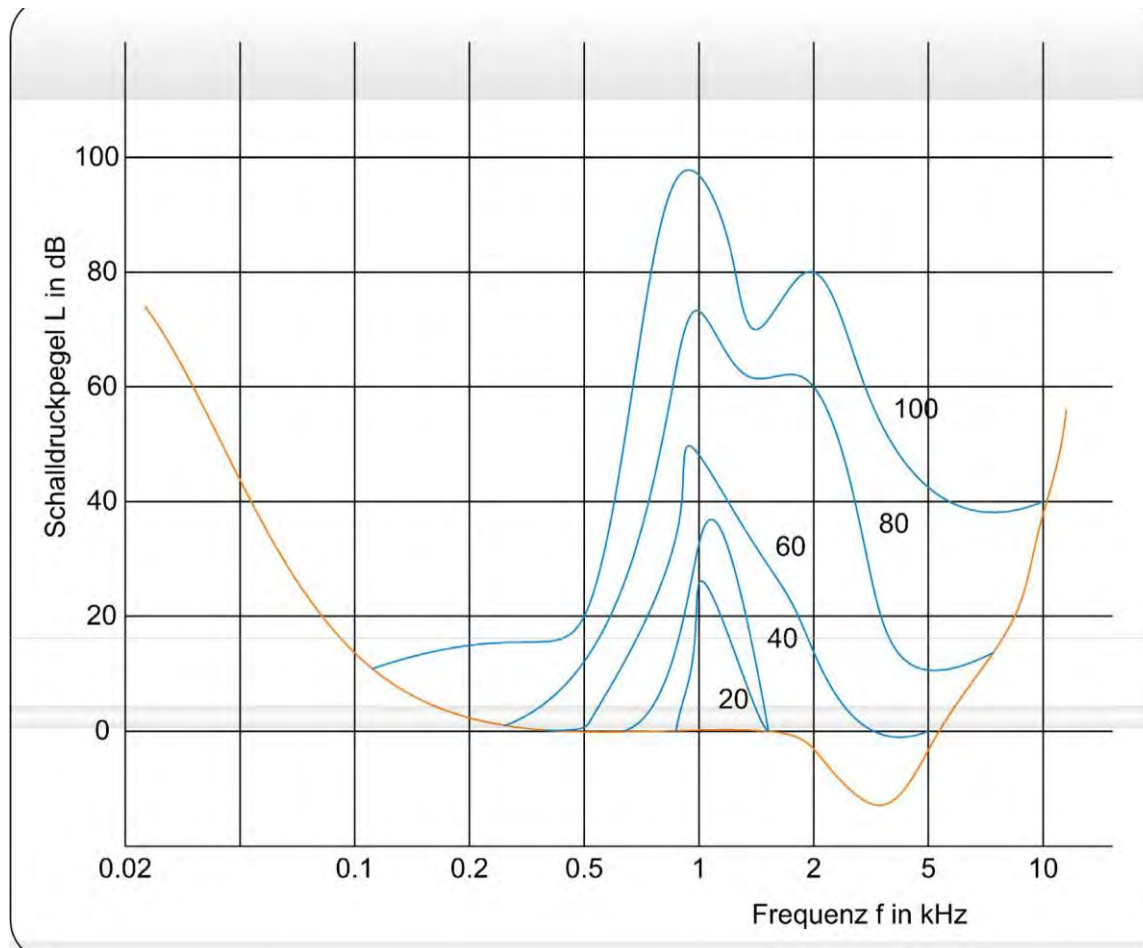
- Psychoacoustic effect of hearing
- Different perception of sounds within frequency bands:
  - Loudness
  - Frequencies of simultaneously sounding tones must have certain minimum distance to be perceived as tones of different pitch
  - Enable masking
- Frequency group comprises a frequency range within which a second tone will interfere with the perception of the first tone
  - refers only to simultaneously played tones (with sequentially played tones, frequency resolution of the auditory system is much higher).

# Masking Effect

- Two tones play simultaneously
  - with different volume and
  - "similar" frequency
- Masking effect - ear hears only the loud sound
  - Quiet sound is "masked"
  - the loud sound reduces sensitivity of hearing in the surrounding frequency range
  - More quiet sound is below new hearing threshold curve
- Masking threshold is minimum volume that the more quiet sound would have to have in order not to be masked

# Masking Effect

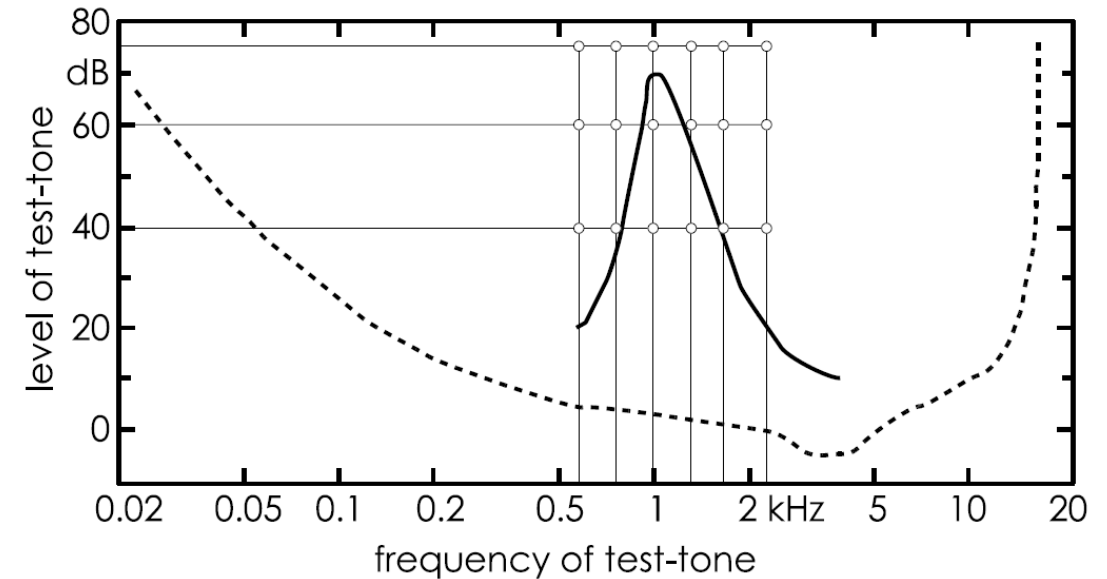
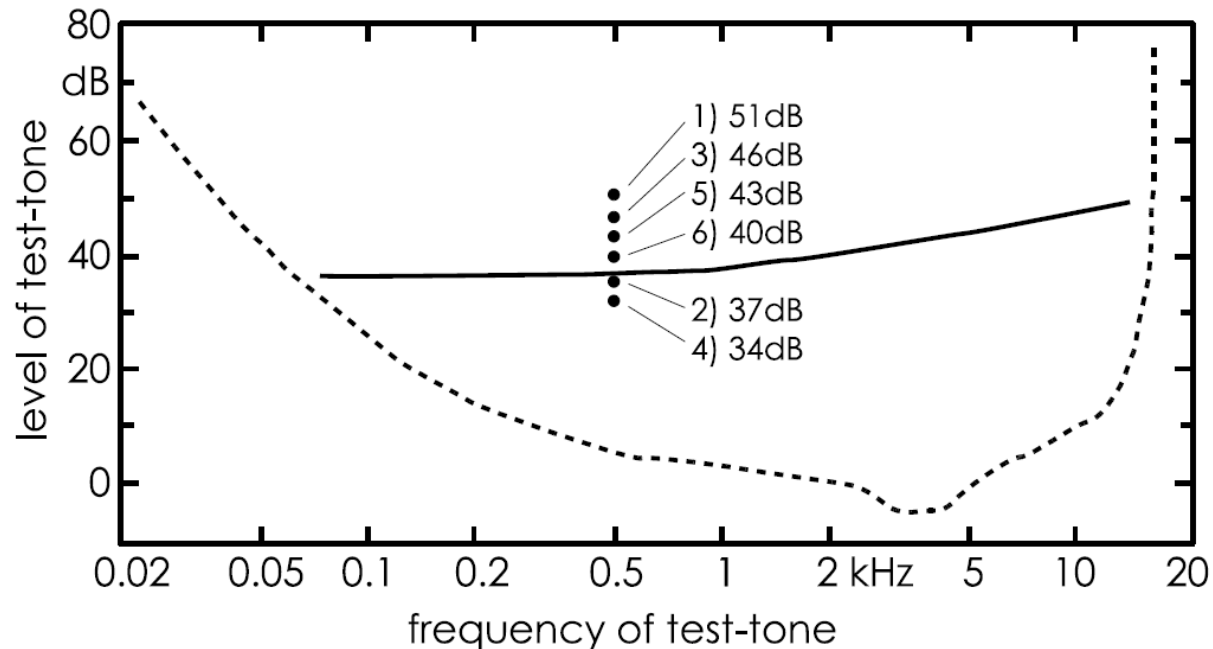
Hearing threshold curve at 1kHz maskers  
(different sound intensities)







# Masking - Examples

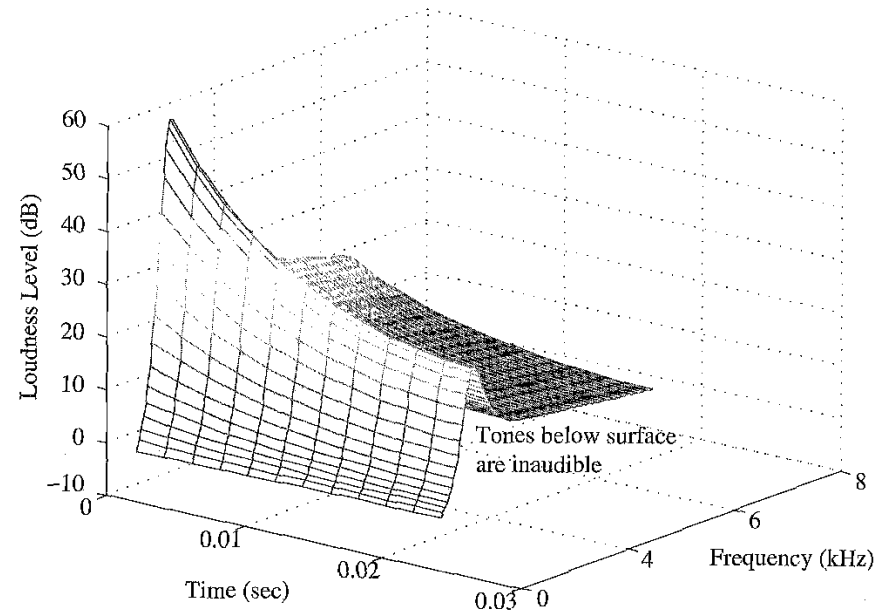


A pure tone is masked by white noise. The tone has a frequency of 50Hz, the white noise has an overall level of 63dB which corresponds to a density level of about 20dB. You will hear six presentations, each with triplets of pure tones, masked by white noise. The level of the tone triplets is (in this order) 51dB, 37dB, 46dB, 34dB, 43dB, 40dB. Except for the second (37 dB) and fourth triplet (34 dB), the tones are above masked threshold and should be audible.

The masked threshold of pure tones masked by critical-band wide noise (1 kHz, 70 dB) is illustrated. You will hear three series of tone triplets: the first series is played at a level of 75 dB, the second at a level of 60 dB, the third at a level of 40 dB. Each series consists of six tone triplets with the frequencies 600 Hz, 800 Hz, 1000 Hz, 1300 Hz, 1700 Hz, and 2300 Hz. In the second series the third tone triplet at 1000 Hz is masked by the narrow-band noise, and in the third series the third and fourth triplet at 1000 Hz and 1300 Hz (for some persons also the fifth triplet at 1700 Hz) are masked.



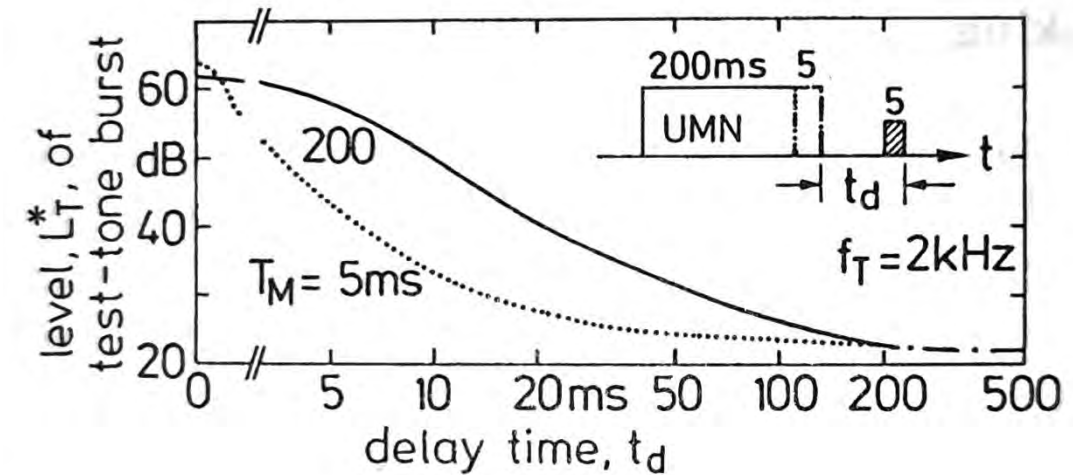
# Masking



- Masking depends on volume, frequency, time interval and duration of the masking sound ("masker").

# Temporal Masking

- Pre-masking (up to 2-20 msec)
  - Tone before a louder tone is masked
- Post-masking (50-200 msec)
  - Tone after a louder tone is masked



The level of the uniform masking noise is 60dB; duration of the 2-kHz test tone is 5ms (dotted line). Test tone of 200ms – solid line.

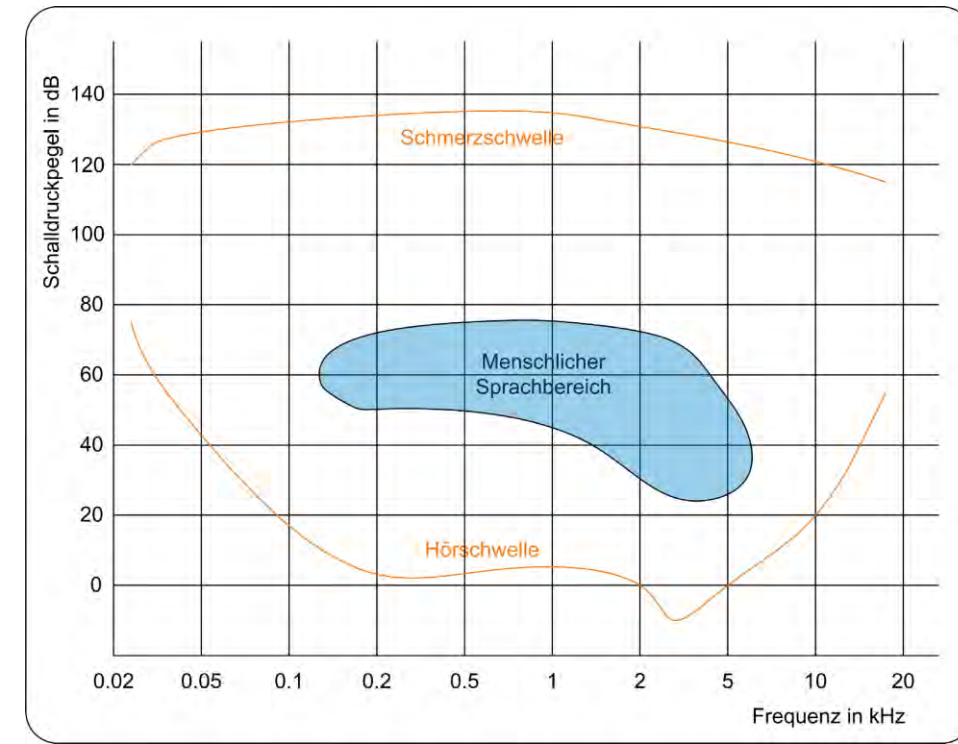
Note that delay time in this case is time between the end of the masker and the end of the test tone.

## Postmasking depends on masker duration:

Levels of just-audible test-tone bursts are indicated as a function of their delay time after the offset of maskers of 200-ms and 5-ms duration.

# Listening Range – MPEG Audio Compression

- Pain threshold and speech range are not relevant for MPEG audio coding
- Hearing threshold - essential for MPEG audio coding
  - Sound must have certain minimum sound level to be perceived
- Minimum sound level is frequency-dependent
- Hearing threshold curve: minimum sound level/frequency function
  - Orange curve is resting hearing threshold (“Ruhehörschwelle”)



# Masking and MPEG Audio

- General audio signal consists of many frequency components
  - each frequency component influences the hearing threshold curve
  - Audio signal varies with time
- Hearing threshold different at each point in time
- **Psychoacoustic model**
  - Calculates current hearing threshold curve in each case
- **Variable quantization**
  - Frequency components quantized so that quantization noise just below current hearing threshold curve
  - Where masking is used, coarse quantization, small word lengths

# Psychoacoustic Effects - Demos

- Further **Examples in PDF** (TUWEL):  
“*Psychoacoustic demonstrations*” from the book
  - Psychoacoustics: Facts and Models,  
by Hugo Fastl and Eberhard Zwicker, 3<sup>rd</sup> ed., 2007.
- Patti Software (Psycho-Acoustic Toolbox at TU Ilmenau)  
<https://sourceforge.net/projects/patti/>
- Shepard Skala  
<https://mynoise.net/NoiseMachines/shepardAudioIllusionToneGenerator.php>

# Cochlea Implant

- Short German description and hearing sample  
<https://www.youtube.com/watch?v=iFY7cmDIWqE>
- <https://www.youtube.com/watch?v=8uthhsMncDY>

# Digital Audio



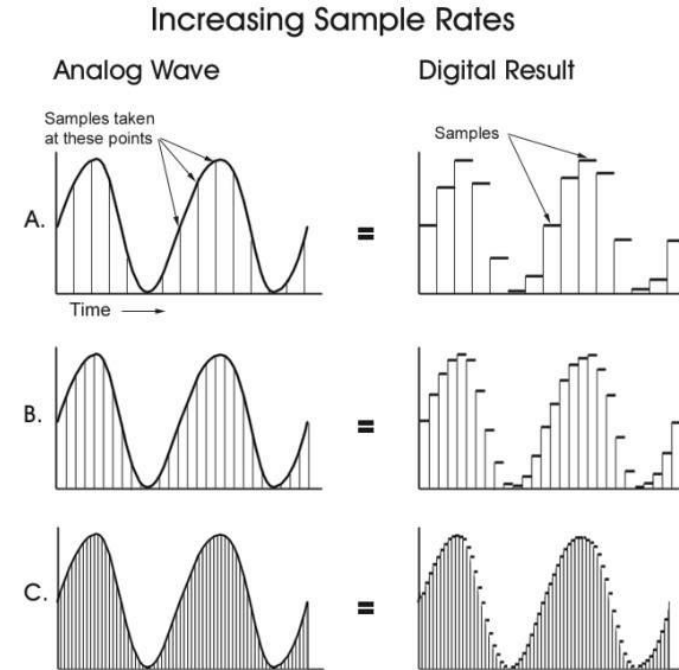
# Introduction Digital Audio

Microphone converts sound wave into electrical signal: voltage curve analogous to the pressure fluctuation curve at the diaphragm of the microphone.

- Analog audio technology:  
Analog signal stored on magnetic tape, vinyl record...  
Editing: mechanical cutting, dubbing
- Digital audio technology:  
Analog signal is converted into binary representation by A/D converter, stored, edited on computer; is output again as analog signal at loudspeaker by means of D/A converter.

# Digitization in General: Sampling & Quantization

- Microphones produce analog signals (continuous-valued voltages)
- To get audio into a computer, it must be converted into a stream of numbers;  
**discrete sampling** (both time and voltage)
- **Sampling**: divide the horizontal axis (**the time dimension**) into discrete pieces; uniform sampling is ubiquitous
- **Quantization**: divide the vertical axis (**signal amplitude**) into pieces; sometimes, a non-linear function is applied

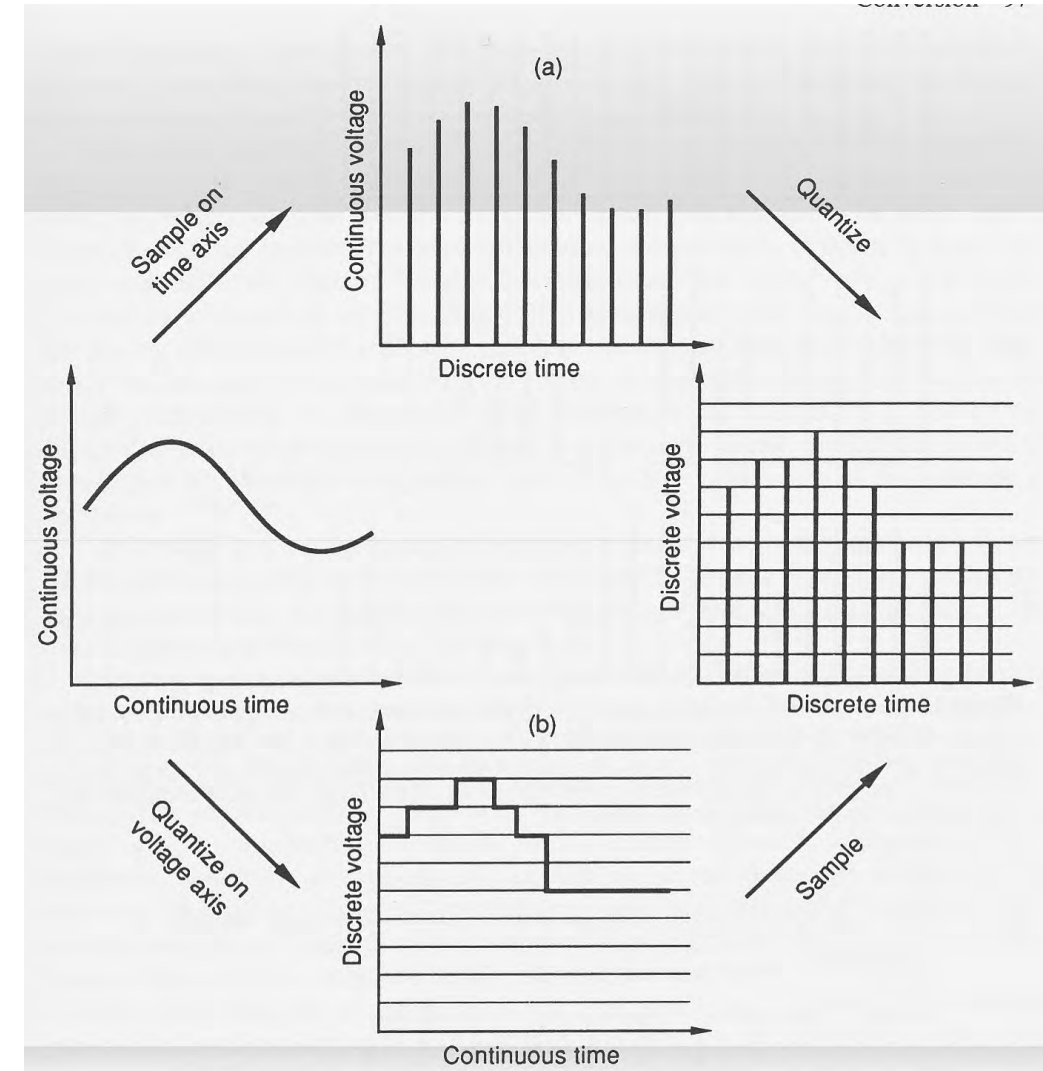


# Digitization Basics

- Input: time and voltage continuous waveform
- Sampling- Quantizing: are independent of each other temporal order does not matter
- Result: time and voltage discrete format

(a) Input signal is sampled, samples are quantized (common with audio)

(b) Input signal is quantized, then sampled (usual for video)



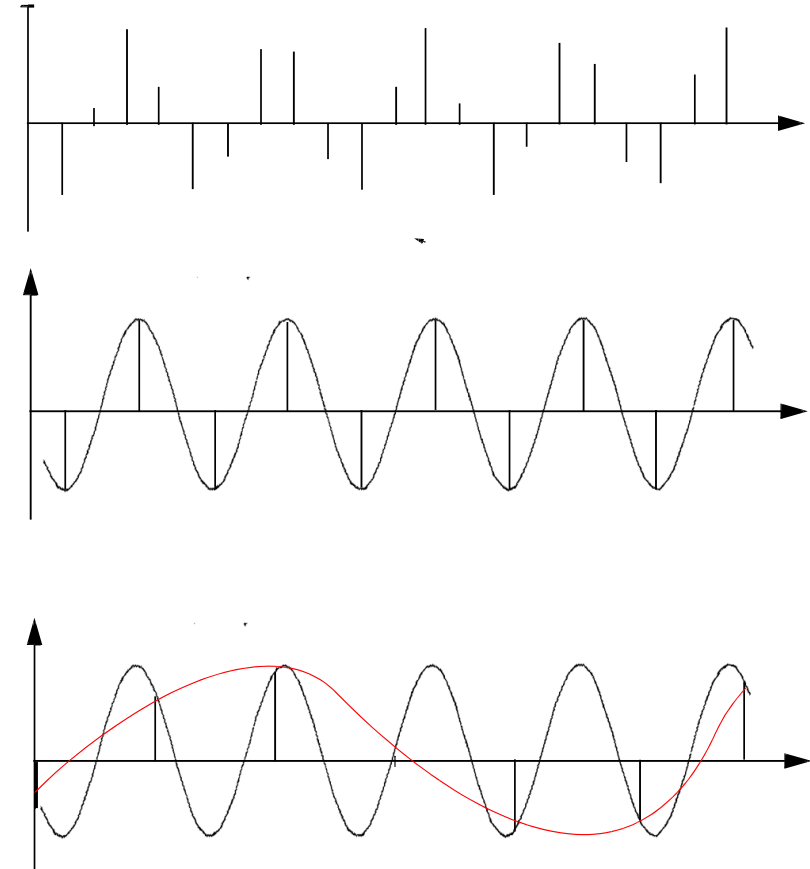
# Digitization in General

- Questions for producing digital audio (Analog-to-Digital Conversion):
  - how often do you need to sample the signal?
  - how good is the signal?
  - how is audio data formatted?
- Suppose we are sampling a sine wave.
  - How often do we need to sample it to figure out its frequency?
  - If we sample at 1 time per cycle, we can think it's a constant
  - If we sample at 1.5 times per cycle, we can think it's a lower frequency sine wave --> alias effects

# Alias component

**Aliasing** is an effect that causes different signals to become indistinguishable (or *aliases* of one another) when sampled.

Often refers to the distortion or artifact that results when a signal reconstructed from samples is different from the original continuous signal.



# Sampling: Nyquist-Shannon Theorem

- Procedure: a sample is taken from the analog input signal at periodic time intervals.
- How often must an analog signal be sampled?

## **Nyquist-Shannon Theorem:**

A sampled signal can be reconstructed without loss of information, if and only if

$$f_s \geq 2 \times f_{max}$$

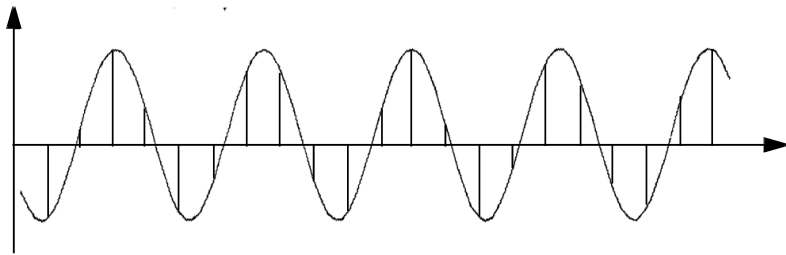
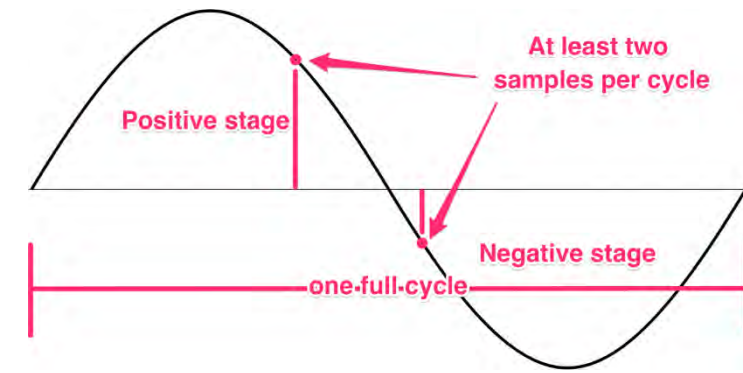
$f_s$  .....Sampling frequency

$f_{max}$  ...the highest frequency occurring in the signal

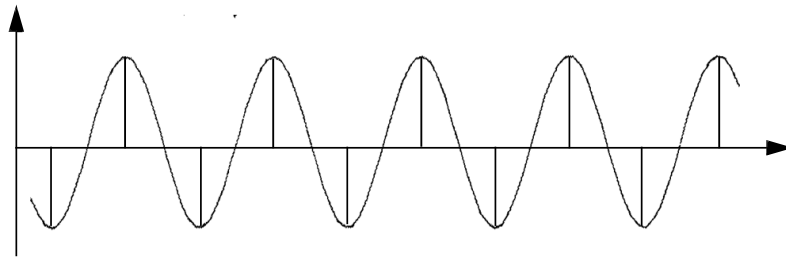
Nyquist Rate, Nyquist Frequency

# Alias Component

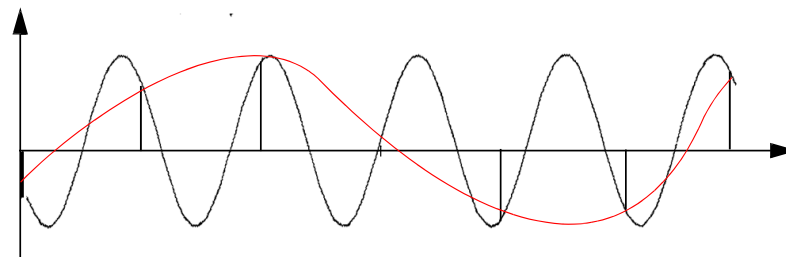
To determine the wavelength and therefore frequency of a sound wave we need at least 2 samples per cycle.



- $f_s > 2 \times f_{max}$   
Reconstructed signal identical with original



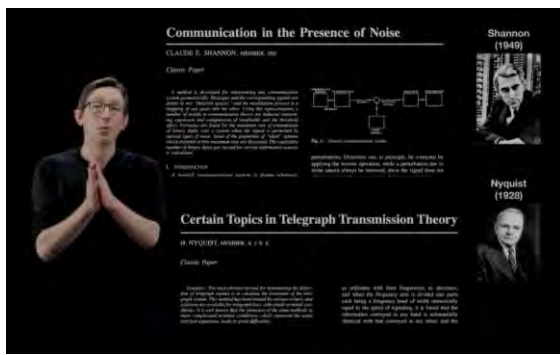
- $f_s = 2 \times f_{max}$   
Reconstructed signal same  $f$  as original



- $f_s < 2 \times f_{max}$   
Interference frequency  
"alias component"

Ex. :  $f_s = 48\text{kHz}$ ,  $f_{id} = 30\text{kHz}$ ,  $f_{alias} = 18\text{kHz}$



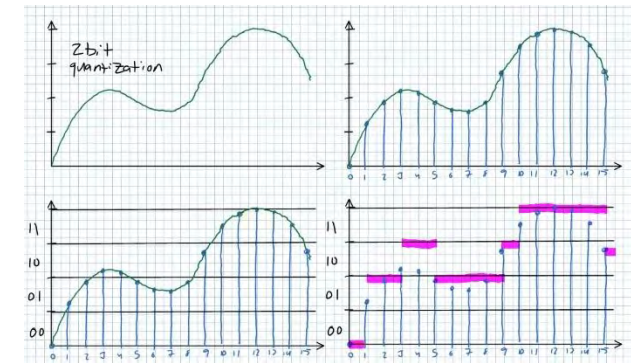


# Optional Material

Shannon Nyquist Sampling Theorem  
by Steve Brunton, History, Overview & Outlook  
<https://www.youtube.com/watch?v=FcXZ28BX-xE>



Analog to Digital Conversion Basics  
(2-4 bit quantization)  
by Jim Pytel  
[https://www.youtube.com/watch?v=sRq\\_gM8r6pl](https://www.youtube.com/watch?v=sRq_gM8r6pl)



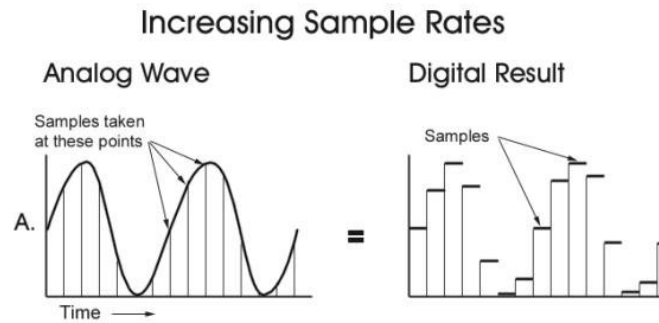
Optional (starting from middle)  
Digital Signal Processing Basics and Nyquist Sampling Theorem  
<https://www.youtube.com/watch?v=WgJMjDh0nLU>

# Examples

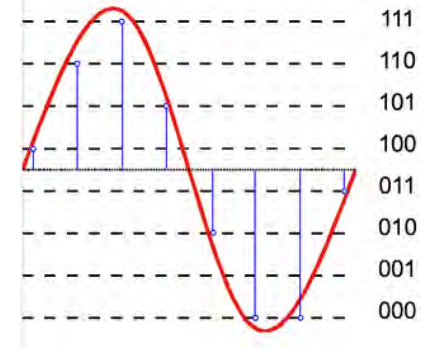
Sampling frequency and max. frequency

| Format        | Sampling frequency | Frequency range |
|---------------|--------------------|-----------------|
| Phone         | 8 kHz              | 200-3400 Hz     |
| Audio CD      | 44.1 kHz           | 20-20000 Hz     |
| Prof. Audio   | 48/96/192 kHz      | 20-20000 Hz     |
| FM/Sat- Radio | 22/32 kHz          | 20-15000 Hz     |

# Quantization



3-bit resolution  
( $2^3 = 8$  levels)



- Sampled voltage values are assigned discrete numerical values:  
Total voltage range divided into quantization intervals  $Q$
- ➔ Continuous value assigned to **nearest numerical value**.
- Quality:  $Q$  should be as small as possible – is a result of the length of the data word size:

$V_{pp}$  = peak to peak range (max range of signal) or  $2 * \text{abs}(V_{max})$

8 bit (256 intervals):  $Q = \frac{V_{pp}}{2^8} = (4 * 10^{-3}) * V_{pp} = 0.004 * V_{pp}$

16 bit (65536 intervals):  $Q = \frac{V_{pp}}{2^{16}} = (15 * 10^{-6}) * V_{pp} = 0.000015 * V_{pp}$

24 bit (16 777 216 intervals) is also used

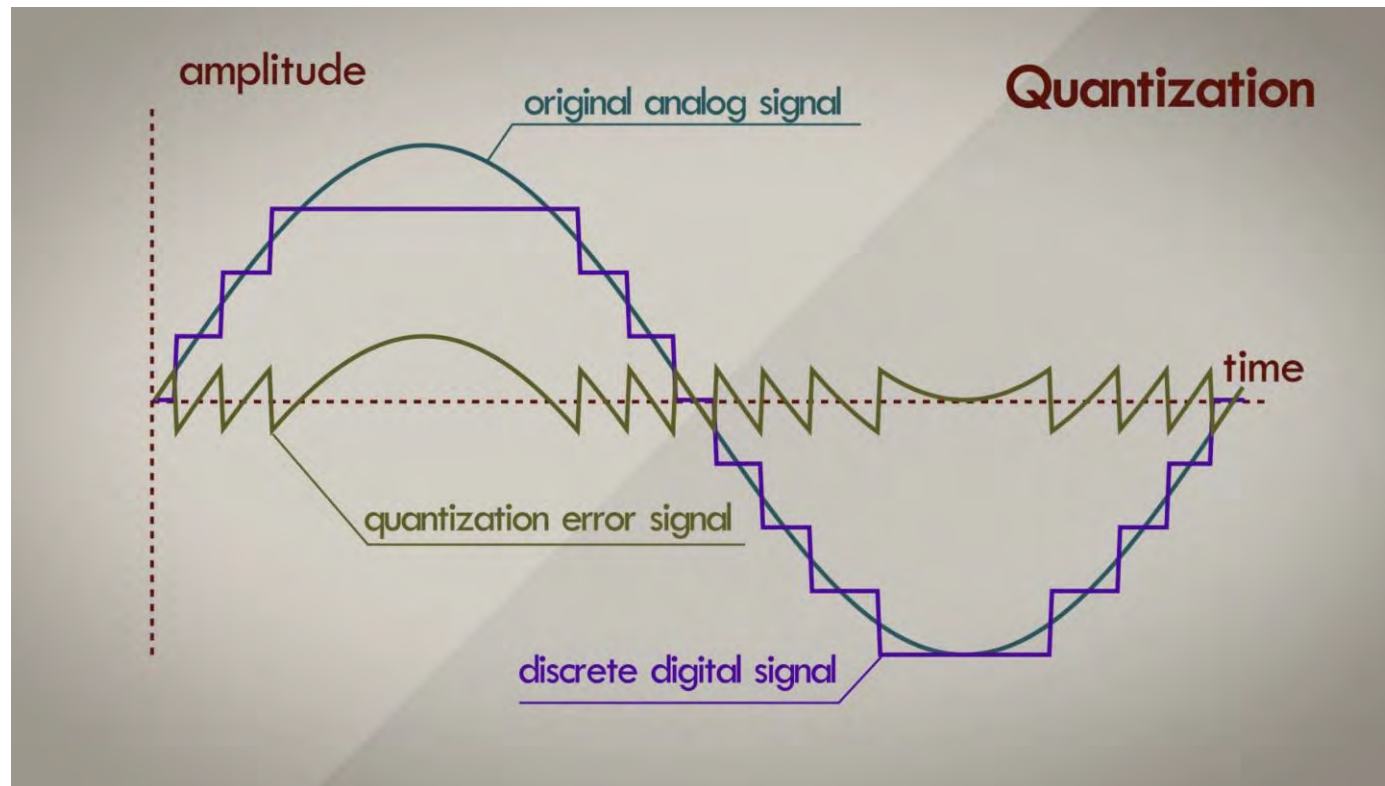
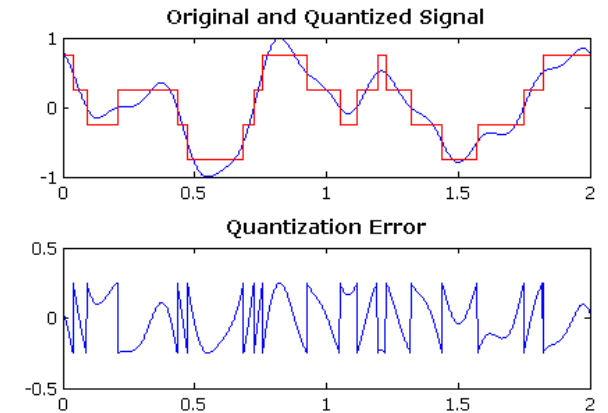
# Some Examples

- 44,000 Hz 16 bit  
110 KB
- 44,000 Hz 8 bit  
90 KB
- 22,000 Hz 16 bit  
61 KB
- 22,000 Hz 8 bit  
45 KB
- 8,000 Hz 16 bit  
16 KB
- 8,000 Hz 8 bit  
13 KB



# Quantization Error

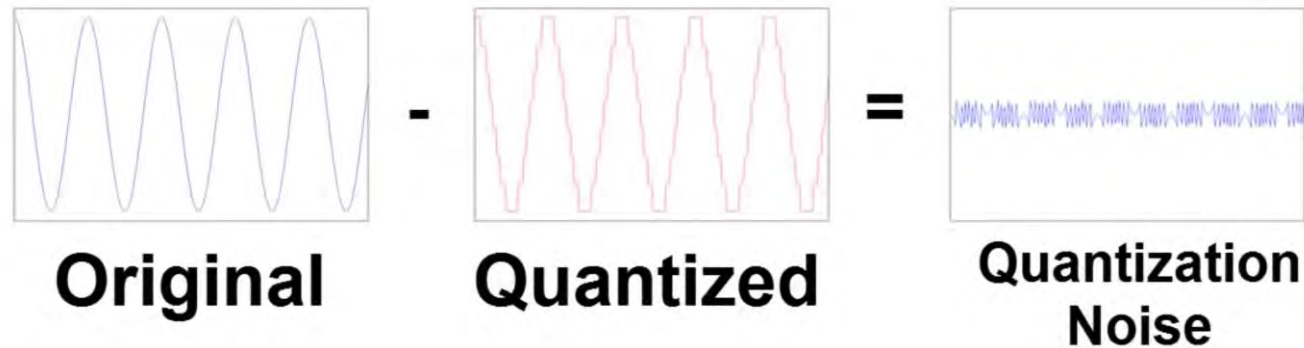
- Numerical value deviates from the sampled value by  $\leq 0.5 Q$ .  
Error is audible as "quantization noise" during reconstruction.



<https://www.youtube.com/watch?v=1KBLguIXL30>

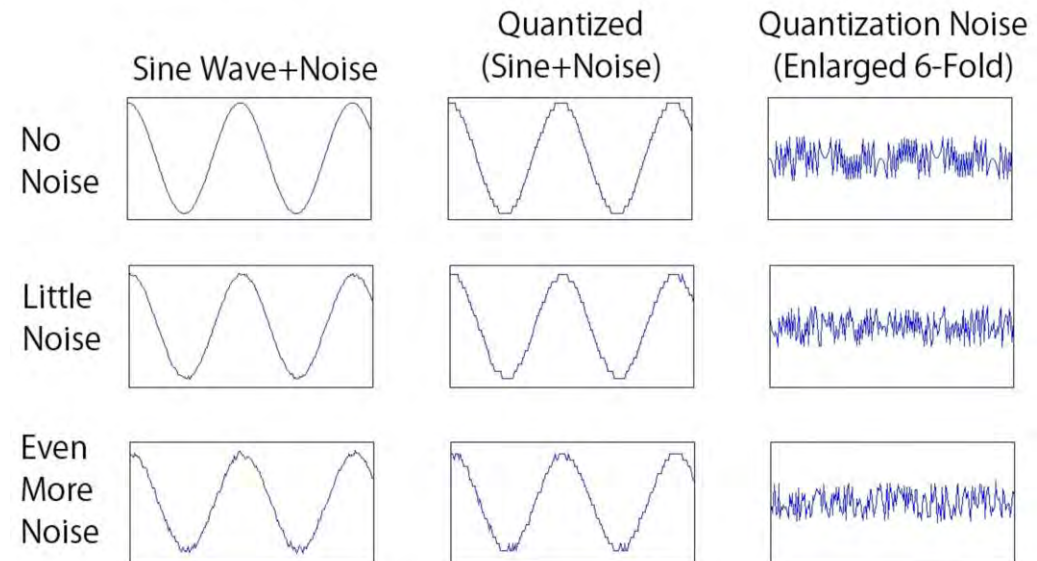
# Quantization Noise & Dithering

- Quantization Noise



<https://www.youtube.com/watch?v=U2mwXiJqAgA>

- Dithering: intentionally adding noise during quantization to preserve low level information and prevent distortion



<https://www.youtube.com/watch?v=tb6X3WZ3-Uc>

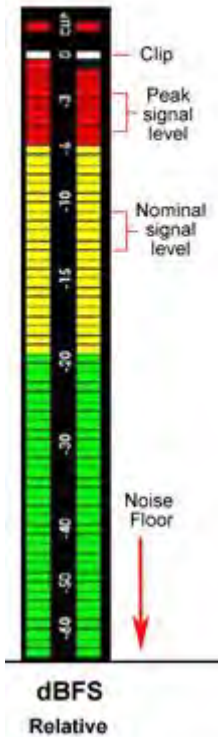
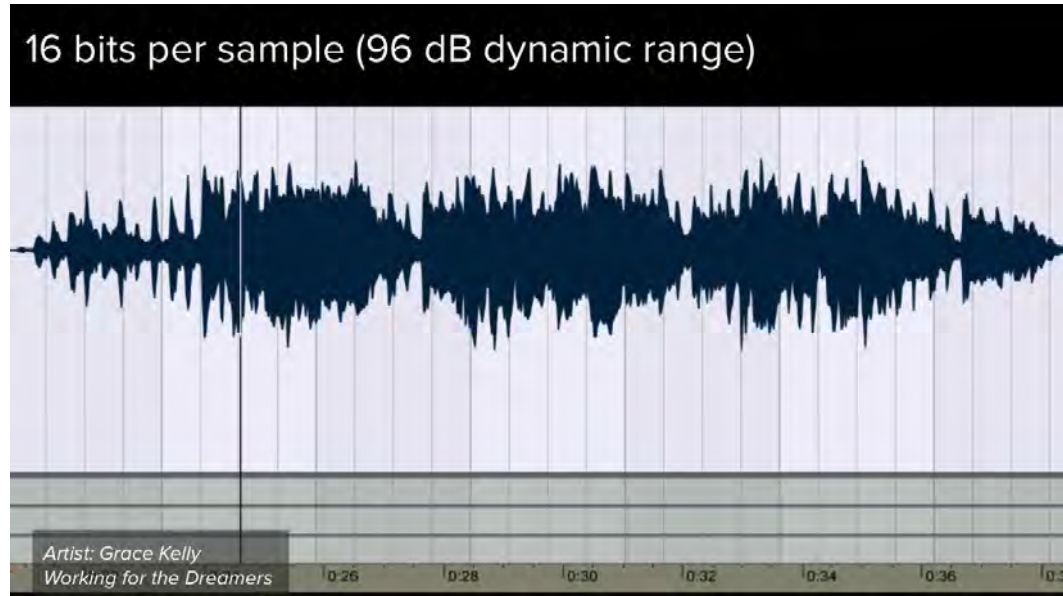
# Signal to Noise Ratio / Dynamic range

- **SNR**..Signal to noise ratio [dB] – in General:
  - Ratio of power of a signal (meaningful input) to power of background noise (unwanted input)
  - $SNR = \frac{P_{Signal}}{P_{Noise}}$
- In Audio:
  - Quantization noise generates sound pressure level  $L_{noise}$
  - Maximum usable signal Sound pressure level  $L_{max}$  (loudest possible signal)
  - $SNR = \log \left( \frac{P_{Signal}}{P_{Noise}} \right) = L_{max} - L_{noise}$
- In audio **Dynamic Range** is SNR:

| Resolution    | 8 bit   | 12 bit  | 16 bit  | 24 bit   |
|---------------|---------|---------|---------|----------|
| Dynamic Range | 49.8 dB | 73.7 dB | 96.3 dB | 144.5 dB |

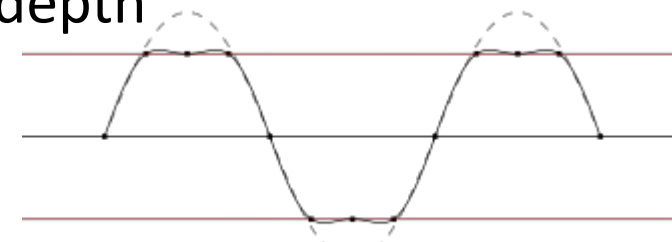


# Bit Depth – Dynamic Range



<https://www.youtube.com/watch?v=ubCMI3Jq6e4&list=PL4dISychPQEtnZkzPUfstKVYsG0e5gwbt&index=2>

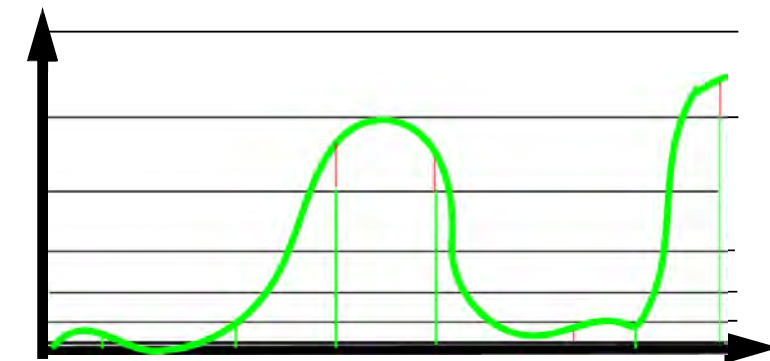
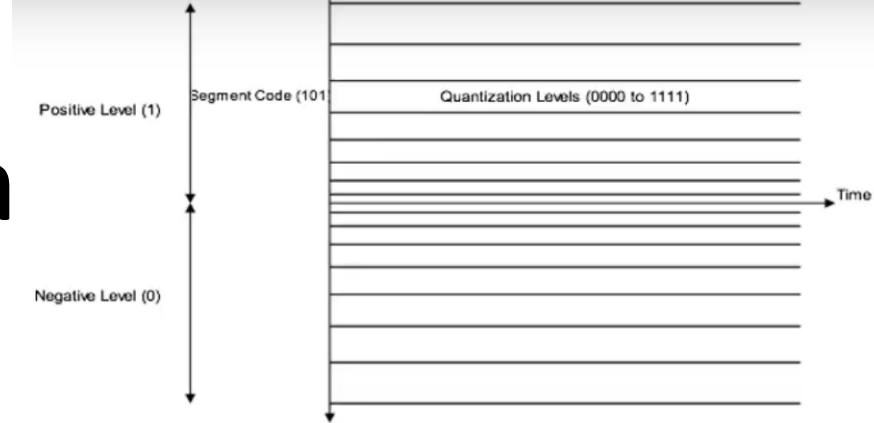
- Remark: **dBFS = Decibel relative to Full Scale**, used in audio production software  
= maximum peak level for digital signal according to bit depth
- Clipping occurs above 0 dBFS



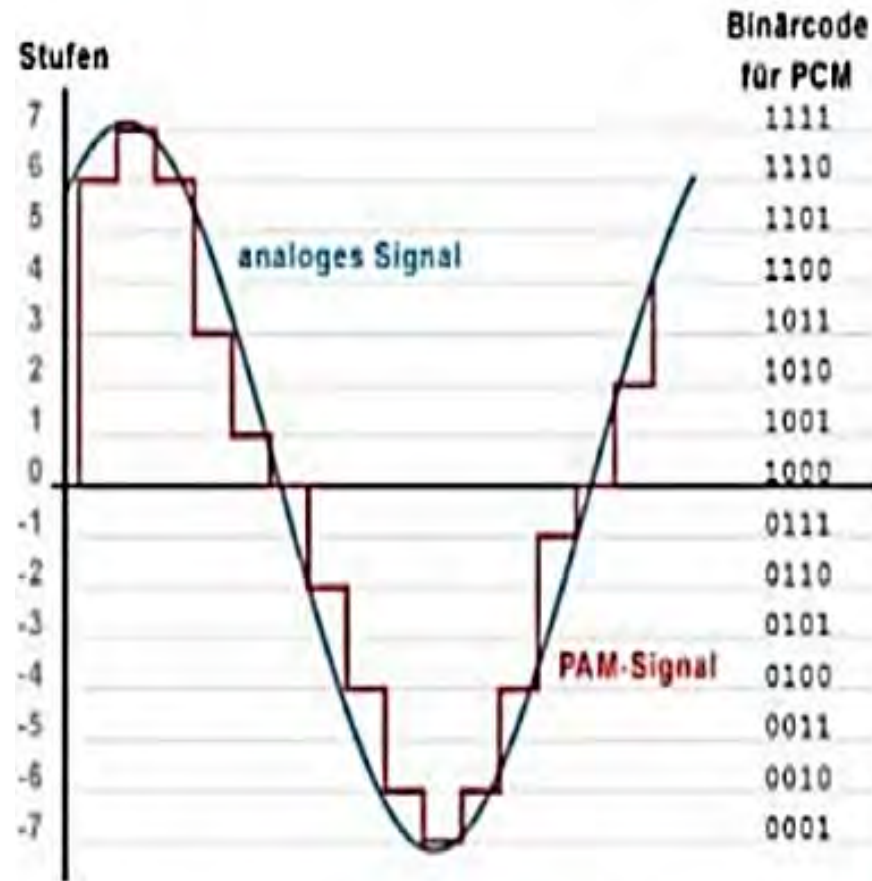


# Linear; Non-linear quantization

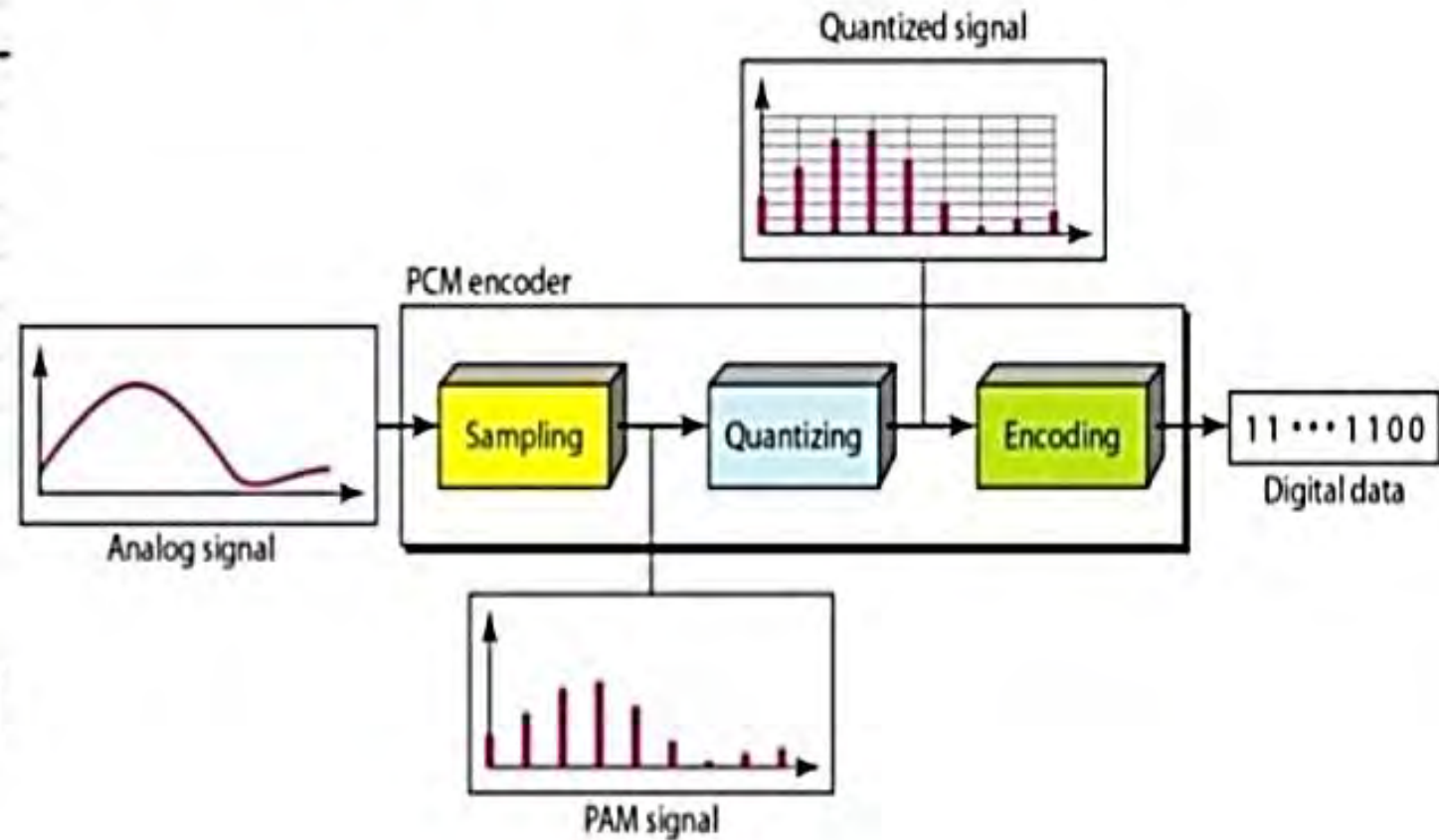
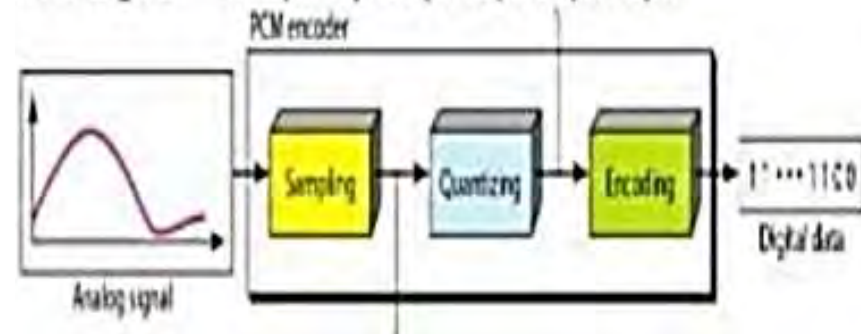
- Linear quantization:  $Q$  constant; common method in audio engineering
- Nonlinear quantization:  $Q$  of different size  
small values  $\rightarrow$  small quantization error  
large values  $\rightarrow$  large quantization error
- Noise masked if high desired signal  $\rightarrow$  passable quality even with small resolution
- Usage for systems with low transmission bandwidth



# PCM Pulse Code Modulation

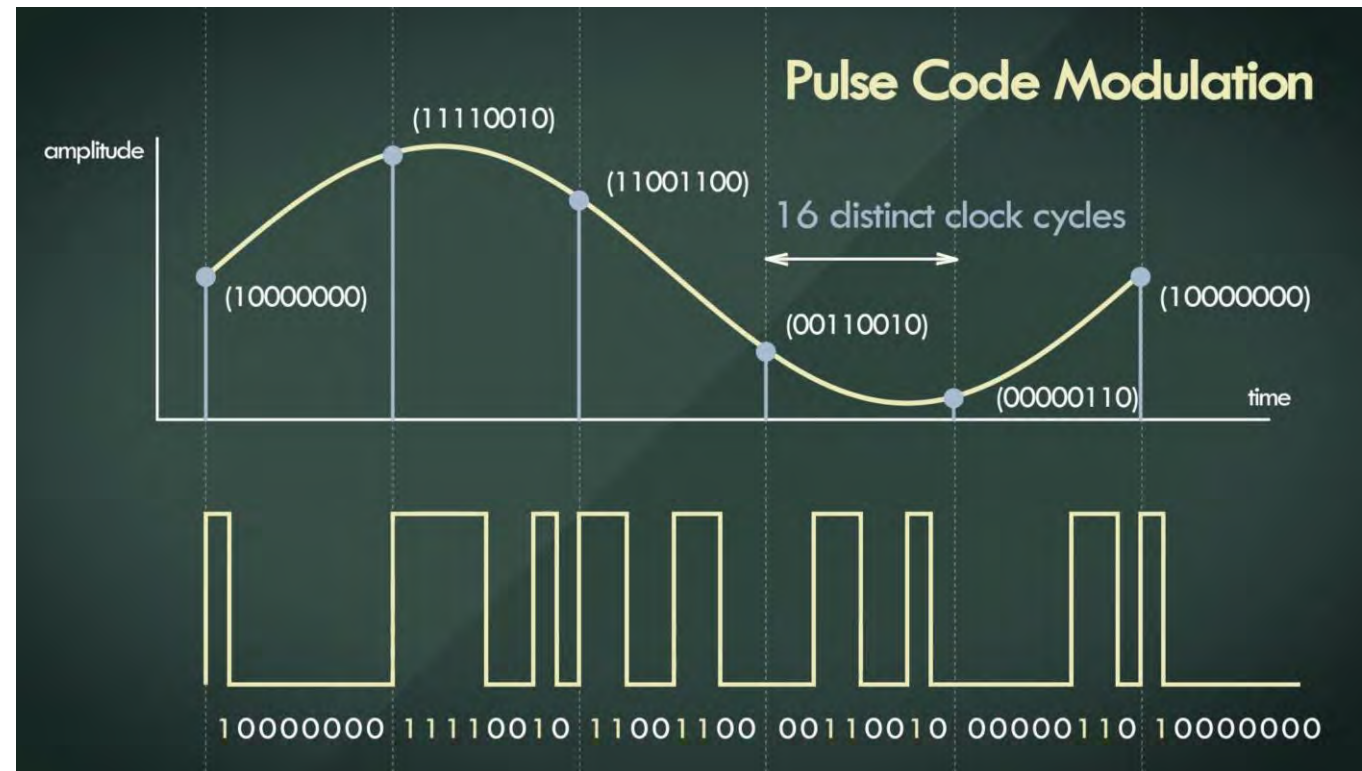


PCM-Signal 1110|1111|1110|1011|1001|1000|...



# Pulse Code Modulation

- Sampling and Quantization give discrete values at discrete intervals
- In Pulse Code Modulation a discrete value is converted into a series of binary digits



8-bit example

# Pulse Code Modulation – Lossless Encoding

- Procedure: Binary values are transmitted serially in the form of modulated voltage pulses via a single line
- Disadvantage: **High data rate**

***Sampling frequency [Hz] \* bit depth [bit] = data rate [bit/s].***

*Ex. : CD-Audio(mono):*

*44,1kHz; 16 bit -> 705600 bit/s*

**Bandwidth** bit rate / information rate

Number of bits conveyed in 1 second

**Pulse Code Modulation**

lossless encoding

44,100 samples

Generally,

sample rate

x

16 bits

bit depth

=

705.6

kilo-bits per second (Kbps)

number of channels

Bandwidth of single channel audio (mono)

Bandwidth of multi channel audio

# Characteristics of Digital Audio

- sampling frequency (rate)
- sample size / quantization
- sample representation (negative values)
- number of channels (tracks)
- **Interleaving** (e.g. 2 channels: LRLRLRLRLR)
- coding/compression

# Example Audio Formats

|                                    | CD audio  | CD-I<br>(Level B) | A-law <sup>a</sup><br>-law |
|------------------------------------|-----------|-------------------|----------------------------|
| sampling rate (kHz)                | 44.1      | 37.8              | 8                          |
| sample size (bits)                 | 16        | 4                 | 8                          |
| quantization                       | uniform   | uniform           | log                        |
| no. of channels                    | 2         | 1-8               | 1                          |
| data rate per channel<br>(bit/sec) | 705       | 174               | 64                         |
| encoding                           | PCM       | ADPCM             | PCM                        |
| quality                            | very high | "Mid Fi", "FM"    | "telephone"                |

a. A-law is used within European telephone systems, while -law is North American.

**Adaptive differential pulse-code modulation (ADPCM)** (1973) is a variant of **differential** pulse-code modulation (DPCM) that varies the size of the quantization step, to allow further reduction of the required data bandwidth. The adaptation to signal statistics in ADPCM consists simply of an **adaptive scale factor** before quantizing the difference in the DPCM encoder. Typical compression ratios are from 4:1 to 2:1.



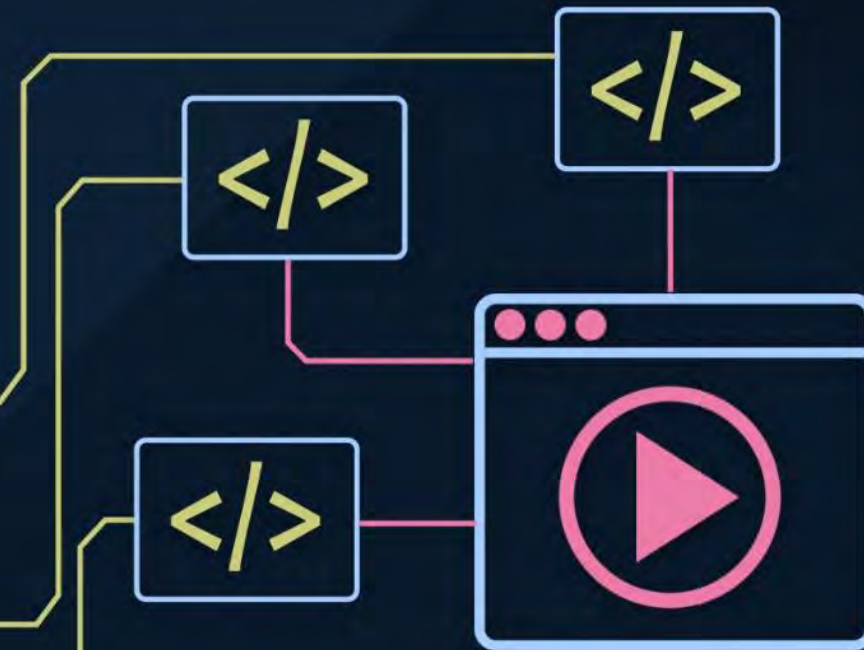
# WAV File Format, FLAC & Codecs

## Format chunk

compression type

| code   | description             |
|--------|-------------------------|
| 0      | unknown                 |
| 1      | PCM (uncompressed)      |
| 2      | Microsoft ADPCM         |
| 6      | ITU G.711 $\alpha$ -law |
| 7      | ITU G.711 $\mu$ -law    |
| 20     | ITU G.723 ADPCM         |
| 49     | GSM 6.10                |
| 80     | MPEG                    |
| 65,536 | experimental            |

coder - decoder



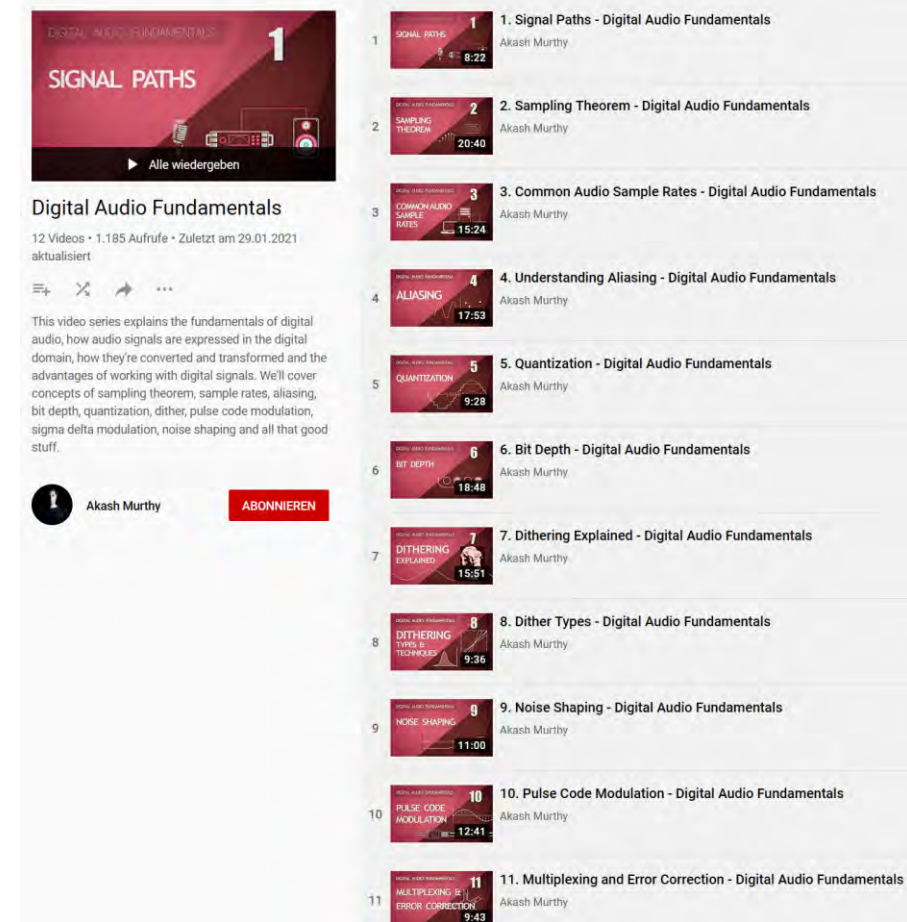


# Detailed Further Material

- Course on “**Digital Audio Fundamentals**” (12 parts)

[https://www.youtube.com/playlist?list=PLbqhA-NKGP6B6V\\_AiS-jbvSzdd7nbwwCw](https://www.youtube.com/playlist?list=PLbqhA-NKGP6B6V_AiS-jbvSzdd7nbwwCw)

by Akash Murthy



**Digital Audio Fundamentals**

12 Videos • 1.185 Aufrufe • Zuletzt am 29.01.2021 aktualisiert

This video series explains the fundamentals of digital audio, how audio signals are expressed in the digital domain, how they're converted and transformed and the advantages of working with digital signals. We'll cover concepts of sampling theorem, sample rates, aliasing, bit depth, quantization, dither, pulse code modulation, sigma delta modulation, noise shaping and all that good stuff.

Akash Murthy **ABONNIEREN**

1. Signal Paths - Digital Audio Fundamentals (8:22)
2. Sampling Theorem - Digital Audio Fundamentals (20:40)
3. Common Audio Sample Rates - Digital Audio Fundamentals (15:24)
4. Understanding Aliasing - Digital Audio Fundamentals (17:53)
5. Quantization - Digital Audio Fundamentals (9:28)
6. Bit Depth - Digital Audio Fundamentals (18:48)
7. Dithering Explained - Digital Audio Fundamentals (15:51)
8. Dither Types - Digital Audio Fundamentals (9:36)
9. Noise Shaping - Digital Audio Fundamentals (11:00)
10. Pulse Code Modulation - Digital Audio Fundamentals (12:41)
11. Multiplexing and Error Correction - Digital Audio Fundamentals (9:43)

# MIDI = Musical Instruments Digital Interface

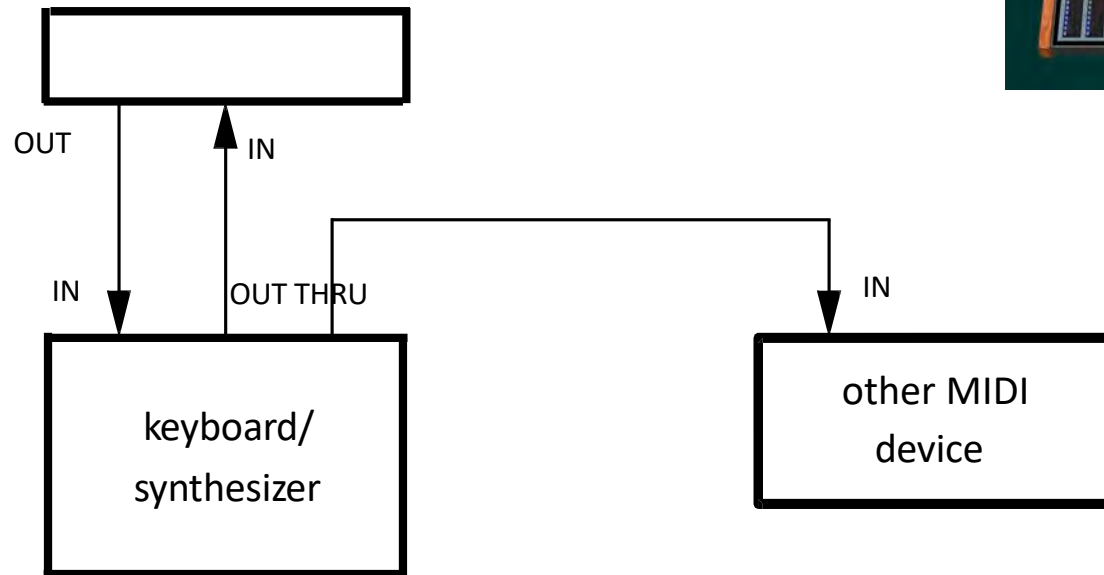
## Terminology

- MIDI - a music description language in digital (binary) form  
A protocol that enables computers, synthesizers, keyboards, and other musical devices to communicate with each other
- Digital audio workstation (DAW) - software used for recording, editing and producing audio files. It interfaces with MIDI devices (multiple MIDI INs and MIDI OUTs), can handle (edit/manipulate) MIDI data; uses plugins (e.g. based on VST standard) that provide virtual instruments and effects.
- Synthesizer - a sound generator (various pitch, loudness, tone color); usually has a microprocessor, keyboard, control panels, memory, etc.

# MIDI = Musical Instruments Digital Interface

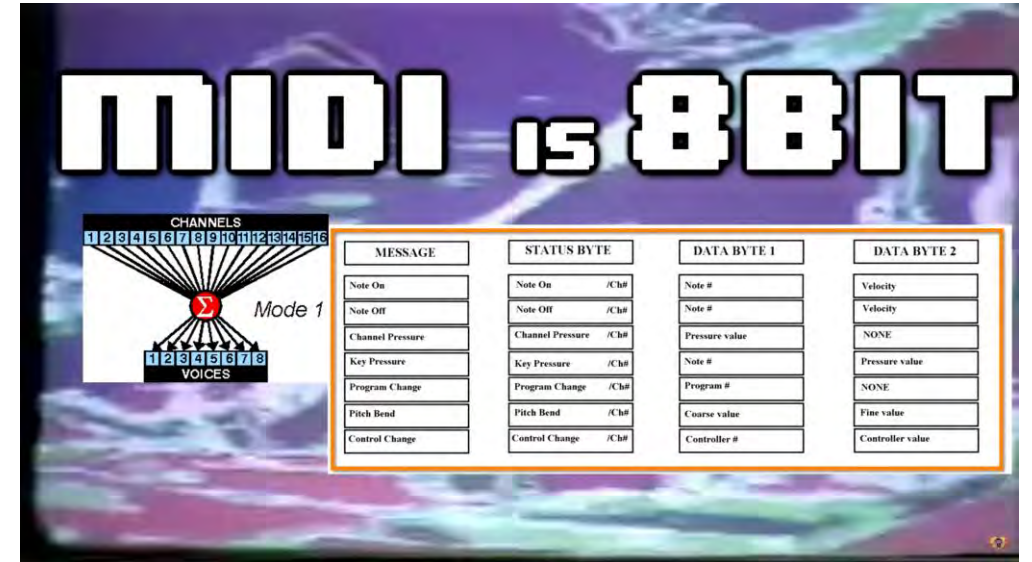
Example MIDI system configuration

Digital Audio Workstation (DAW)  
– formerly a MIDI sequencer



# MIDI – Brief History

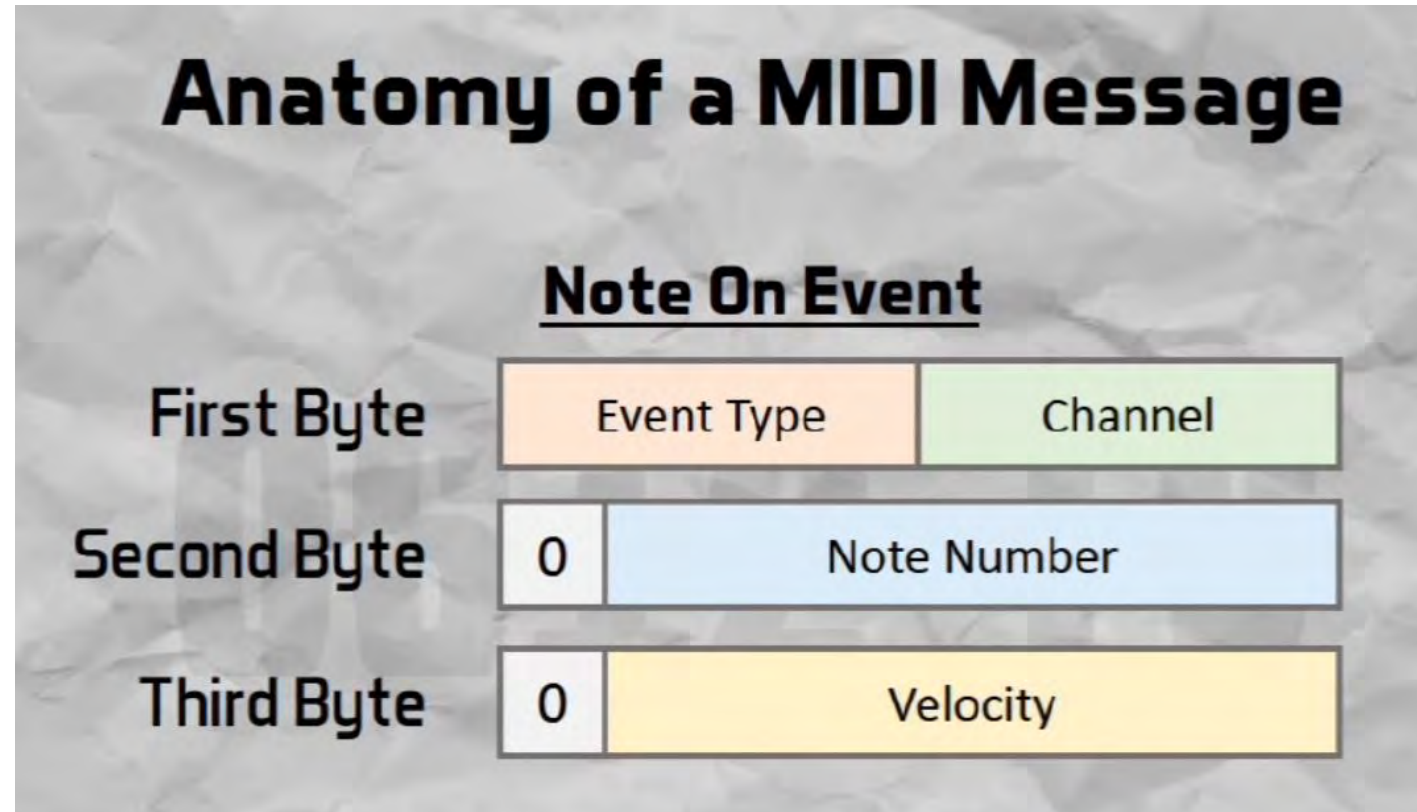
First specification in 1983



<https://www.youtube.com/watch?v=SUUxmJ84dni>



# Basics of MIDI



<https://www.youtube.com/watch?v=5ZXuXqHdHks>

# MIDI Example



# MIDI Summary & Concepts

- MIDI is control data for electronic instruments and devices
- MIDI does not include audio signals
- Instruments create audio based on MIDI data
- MIDI messages are usually 1-3 bytes, each component 8-bit

## Important MIDI Concepts:

- Timing clocks - MIDI sequencer time stamps messages, timebase measured in parts per quarter note, tempo in beats per minute
- MIDI synchronization - external or internal sync
- MIDI Time Code (MTC) - used to synchronize with film or video



# Extended MIDI Specifications

Extended specifications for

- MIDI Machine Control (MMC)
- MIDI Show Control (MSC)
- MIDI Time Code
- General MIDI
- Downloadable Sounds
- Scalable Polyphony MIDI

# Open Sound Control (OSC)

- A **protocol** for **networking** sound synthesizers, computers, and other multimedia devices for purposes such as musical performance or show control. OSC's advantages include interoperability, accuracy, flexibility and enhanced organization and documentation.
- First specified in 2002
- **Alternative to MIDI when higher resolution and a richer parameter space is desired**
- Main features, compared to MIDI, include:
  - Open-ended, dynamic, URI-style symbolic naming scheme
  - Symbolic and high-resolution numeric data
  - Pattern matching language to specify multiple recipients of a single message
  - High resolution time tags
  - "Bundles" of messages whose effects must occur simultaneously



# Multimedia

## Vorlesung

Hannes Kaufmann

Research Unit Virtual and Augmented Reality  
Institute of Visual Computing & Human-Centered Technology

Based on material by Prof. Christian Breiteneder

# Multimedia Compression Basics

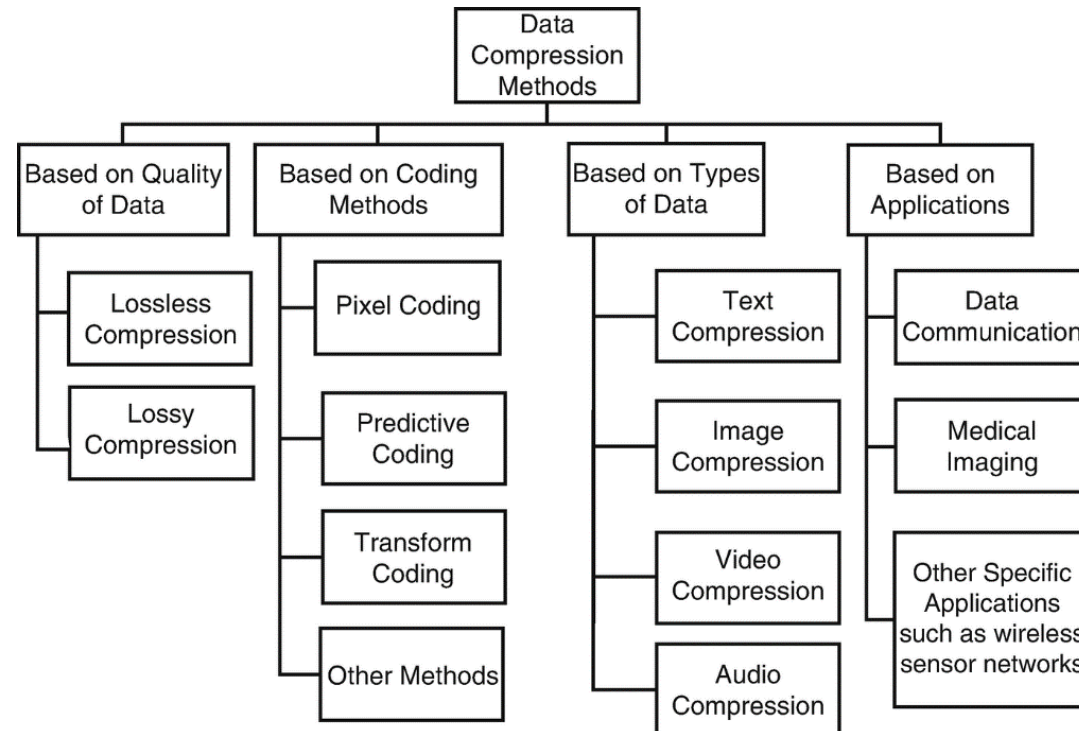
# Codec

- A codec is a device or computer program which encodes or decodes a digital data stream or signal
- Typically a pair of algorithms for encoding and decoding
- Codec is a blend of the words en**code**r – **de**code
- An audio codec converts analog audio signals into digital signals for transmission or encodes them for storage. A receiving device converts the digital signals back to analog form using an audio decoder for playback.
- A codec may also **compress** the data to reduce transmission bandwidth or storage space

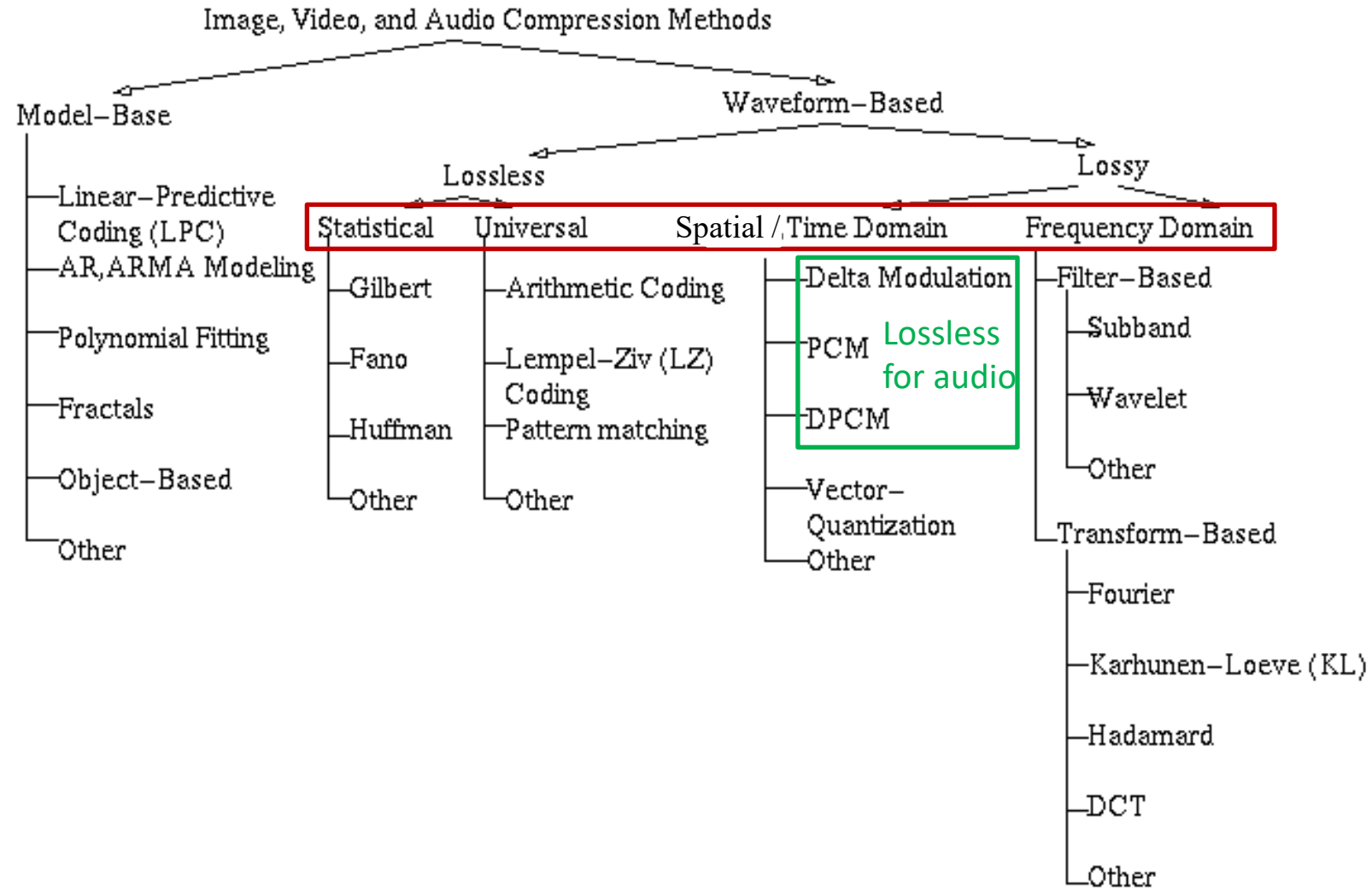


# Compression Classification

- **Lossless** Compression
  - decompressed data is identical to original data.
- **Lossy** compression (irreversible compression)
  - uses inexact approximations and partial data discarding to represent the content



# Compression Taxonomy



# Audio Codecs

## CODECS

### VIDEO

H.264 (AVC)  
H.265 (HEVC)  
VP9  
AV1

### AUDIO

MP3  
AAC

## AAC

Advanced Audio Coding



**Widely supported**



**More efficient than MP3**



**Limit on audio channels**

# Motion Picture Experts Group (MPEG)

- General Information about MPEG
- MPEG Objectives and Standards
- Audio Codecs:
  - MPEG-1 / MPEG-2 Audio Layer 3 (MP3)
  - MPEG-2 / MPEG-4 Advanced Audio Coding (AAC)

# Motion Picture Experts Group (MPEG)

About 350 members from 200 industries, universities, and research institutions from 20 countries

ISO/IEC JTC 1/SC 29 **Coding of audio, picture, multimedia and hypermedia information** (since 1991) is a standardization subcommittee (7 working groups, 3 advisory groups)

- SC 29/AG 2: MPEG Technical Coordination
- SC 29/AG 3: MPEG Liaison and Communication
- SC 29/AG 5: MPEG Visual Quality Assessment
- SC 29/WG 2: MPEG Technical requirements
- SC 29/WG 3: MPEG Systems
- SC 29/WG 4: MPEG Video Coding
- SC 29/WG 5: MPEG Joint Video Coding Team(s) with ITU-T SG 16
- SC 29/WG 6: MPEG Audio coding
- SC 29/WG 7: MPEG 3D Graphics coding
- SC 29/WG 8: MPEG Genomic coding

# MPEG Objectives

- Deliver acceptable video quality at compressed data rates (for MPEG-1 between 1.0 and 1.5 Mbps)
- Support either **symmetric** or **asymmetric** compress / decompress applications
  - symmetric = compression/decompression takes same time
- If compression takes it into account, **random-access** playback is possible
- If compression takes it into account, fast-forward, fast-reverse or normal reverse playback modes are available
- Audio/video **synchronization** will be maintained

# MPEG Objectives

- Catastrophic behavior in the presence of data errors should be avoidable
- If required, compression/decompression delay can be controlled
- Editability should be available if required by applications
- Processing requirements should allow the development of low-cost chipsets which are capable of encoding in real-time



# MPEG Standards

- Standard specifies audio, video and system layers
- Many standards defined
- MPEG-1 targeted at low data rates  
(VHS quality at 1.5 Mbits/ sec) [1992]
- MPEG-2 targeted higher quality, hence higher data rates  
(studio quality up to 15 Mbits/sec) [1994]
- MPEG-4 targeted at very low bit rates (<64 kbs) with small images [1998]

| Acronym for a group of standards ⇄ | Title ⇄                                                                                                                                                                                              | ISO/IEC standards ⇄                   | First public release date (First edition) ⇄ |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|---------------------------------------------|
| MPEG-1                             | Coding of moving pictures and associated audio for digital storage media. Commonly limited to about 1.5 Mbit/s although specification is capable of much higher bit rates <b>Audio Layer 3 = MP3</b> | ISO/IEC 11172                         | 1993                                        |
| MPEG-2                             | Generic coding of moving pictures and associated audio information <b>Audio Layer 3 = MP3</b>                                                                                                        | ISO/IEC 13818                         | 1995                                        |
| MPEG-3                             |                                                                                                                                                                                                      |                                       |                                             |
| MPEG-4                             | Coding of audio-visual objects <b>H.264 = MPEG-4/Part 10 = MPEG-4 AVC</b>                                                                                                                            | ISO/IEC 14496                         | 1999                                        |
| MPEG-7                             | Multimedia content description interface <b>AAC = MPEG-4 Part 3, MPEG-4 Audio or</b>                                                                                                                 | ISO/IEC 14496-3:1999<br>ISO/IEC 15938 | 2002                                        |
| MPEG-21                            | Multimedia framework (MPEG-21)                                                                                                                                                                       | ISO/IEC 21000                         | 2001                                        |
| MPEG-A                             | Multimedia application format (MPEG-A)                                                                                                                                                               | ISO/IEC 23000                         | 2007                                        |
| MPEG-B                             | MPEG systems technologies                                                                                                                                                                            | ISO/IEC 23001                         | 2006                                        |
| MPEG-C                             | MPEG video technologies                                                                                                                                                                              | ISO/IEC 23002                         | 2006                                        |
| MPEG-D                             | MPEG audio technologies                                                                                                                                                                              | ISO/IEC 23003                         | 2007                                        |
| MPEG-E                             | Multimedia Middleware                                                                                                                                                                                | ISO/IEC 23004                         | 2007                                        |
| MPEG-G                             | Genomic Information Representation                                                                                                                                                                   | ISO/IEC 23092                         | 2019                                        |
| (none)                             | Supplemental media technologies                                                                                                                                                                      | ISO/IEC 29116                         | 2008                                        |
| MPEG-V                             | Media context and control                                                                                                                                                                            | ISO/IEC 23005 <sup>[42]</sup>         | 2011                                        |
| MPEG-M                             | MPEG extensible middleware (MXM)                                                                                                                                                                     | ISO/IEC 23006 <sup>[47]</sup>         | 2010                                        |
| MPEG-U                             | Rich media user interfaces                                                                                                                                                                           | ISO/IEC 23007 <sup>[49]</sup>         | 2010                                        |
| MPEG-H                             | High Efficiency Coding and Media Delivery in Heterogeneous Environments <b>H.265 = High Efficiency Video Coding (HEVC), MPEG-H Part 2</b>                                                            | ISO/IEC 23008 <sup>[54]</sup>         | 2013                                        |
| MPEG-DASH                          | Information technology – DASH <b>Dynamic adaptive streaming over HTTP (DASH)</b>                                                                                                                     | ISO/IEC 23009                         | 2012                                        |
| MPEG-I                             | Coded Representation of Immersive Media <b>H.266 = Versatile Video Coding (VVC), ISO/IEC 23090-3, MPEG-I Part 3, Future Video Coding (FVC)</b>                                                       | ISO/IEC 23090                         | 2020                                        |

MPEG-H 3D Audio, Audio Objects

# MPEG General Information

- MPEG standard defines audio, video coding and system data streams with synchronization
- MPEG considers explicitly functionalities of other standards, e.g. it uses JPEG
- MPEG stream provides information on
  - aspect ratio (e.g., 1:1, 4:3, 16:9)
  - refresh frequencies (8 frequencies) encoded:  
23.976 Hz, 24 Hz, 25 Hz, 29.97 Hz, 30 Hz, 50 Hz, 59.94 Hz and 60 Hz

▼ Stream 0

- Codec: H264 - MPEG-4 AVC (part 10) (avc1)
- Type: Video
- Video resolution: 1280x720
- Buffer dimensions: 1280x720
- Frame rate: 23.976024
- Decoded format:
- Orientation: Top left
- Chroma location: Left

▼ Stream 1

- Codec: MPEG AAC Audio (mp4a)
- Type: Audio
- Channels: Stereo
- Sample rate: 48000 Hz
- Bits per sample: 32

# MPEG-1 General Information

- Audio compressor uses subband coder with psychoacoustic model to reduce bit rate
- Video compressor uses block transform coder with motion- compensated inter-frame coding
- Audio and video compression designed to operate at CD-ROM speeds (video—1.150 Mbits/sec, audio—0.256 Mbits/sec, system 0.094 Mbits/sec)

# MPEG-1 Audio Standard

- Precision: 16 Bits, sampling frequencies 32kHz, 44.1kHz, 48kHz
- Audio coding: single channel, 2 independent channels, 1 stereo channel
- 3 compression methods: Layer 1, Layer 2, Layer 3 (hierarchical decoding)
  - Layer 3: decoder is accepting Layer 2 and Layer 1 (32kbps- 320kbps, target 64 kbps)
  - Layer 2: decoder is accepting Layer 1 (32 kbps-384kbps, target 128 kbps)
  - Layer 1: 32 kbps-448kbps, target 192 kbps

# MPEG Audio

- *MPEG-1/2 Layer-3 is called MP3*
- Exponential growth of MPEG coded audio material on the internet since 1995
- “mp3” the most searched for term in early 1999
- Everybody is using MP3, not many know details of MPEG audio coding
- Compared to CD-quality digital audio, **MP3 compression can commonly achieve a 75 to 95% reduction in size**. For example, an MP3 encoded at a constant bitrate of 128 kbit/s would result in a file **approximately 9% of the size of the original CD audio**.



# Why MPEG-1/2 Layer-3?

- Open standard
  - Specified in 1993
  - specification available to everybody (patents expired 2017)
  - no single company “owns” the standard
  - public example source code available to implementers
  - format well defined
- Availability of encoders and decoders
- Widespread use and support; computers fast enough for encoding & decoding

## Anecdote:

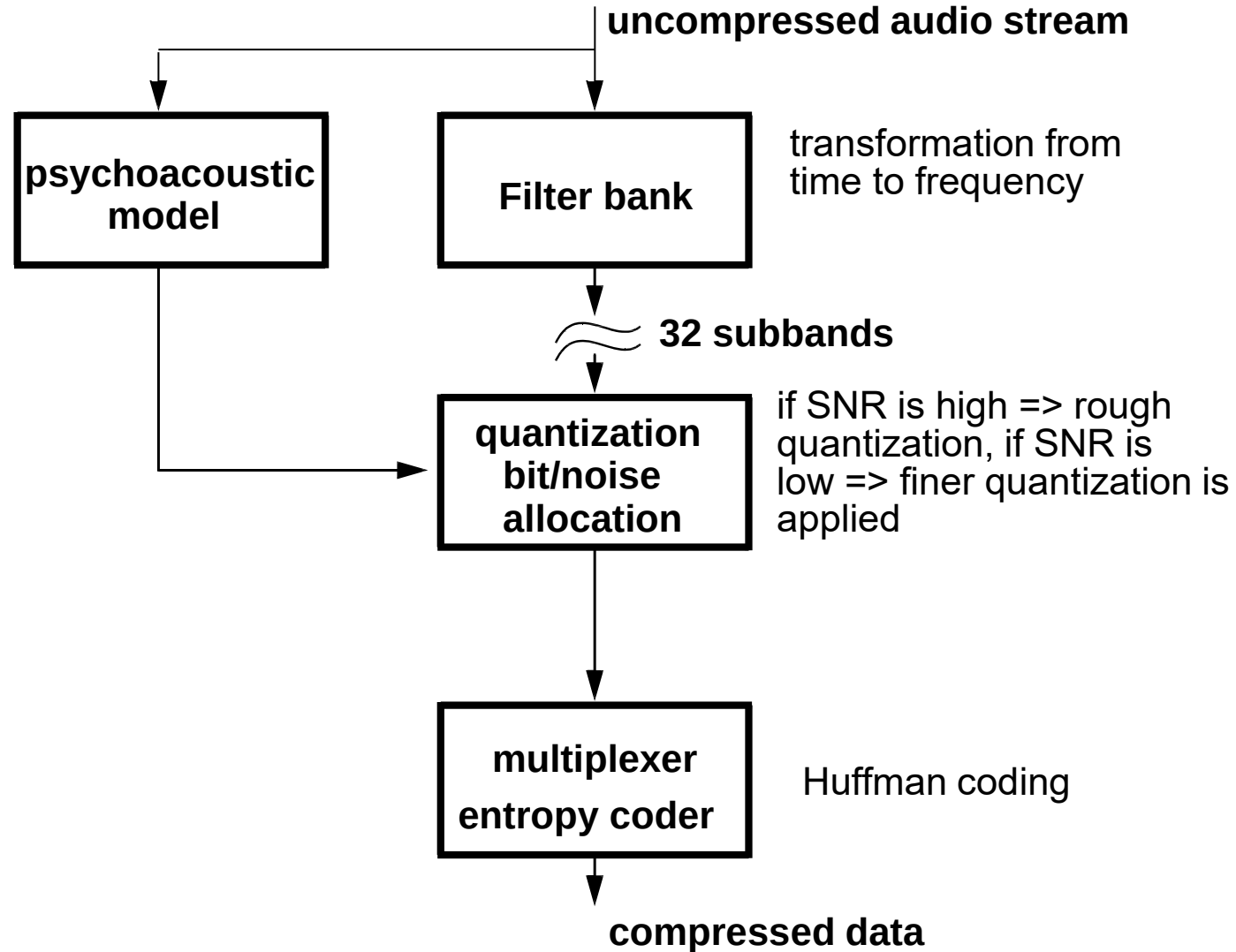
A hacker named SoloH discovered the source code of the "dist10" MPEG reference implementation shortly after the release on the servers of the University of Erlangen. He developed a higher-quality version and spread it on the internet. This code started the widespread CD ripping and digital music distribution as MP3 over the internet.



# General Approach of MPEG-1/2 Layer-3

- Encoder is not precisely specified but building blocks are described
  - Implementers of the standard were supposed to devise their own algorithms suitable for removing parts of the information from the audio input.
  - Many different MP3 encoders became available, each producing files of differing quality.
- Decoding is carefully defined in the standard. Most decoders are "bitstream compliant"
  - Only computational performance of decoders differs (depending how efficiently implemented)

# MPEG Audio Standard (Encoding)



# High Quality Audio Coding

- MPEG-1 Layer-3 has been specified 1993
- since then, research on perceptual audio coding has progressed and codes with better compression efficiency became available: MPEG-2 Advanced Audio Coding (AAC) and other proprietary compression systems.
- Basic task is to compress audio data in a way that
  - the compression is as efficient as possible (file size small)
  - the reconstructed (decoded) audio sounds exactly (or as close as possible) to the original audio data
  - requires low complexity (in software or inexpensive hardware)
  - offers flexibility for different application scenarios.

# Masking and MPEG Audio Compression

- General audio signal consists of many frequency components
  - each frequency component influences the hearing threshold curve
  - Audio signal varies with time
- Hearing threshold different at each point in time
- **Psychoacoustic model**
  - Calculates current hearing threshold curve in each case
- **Variable quantization**
  - Frequency components quantized so that quantization noise just below current hearing threshold curve
  - Where masking is used, coarse quantization, small word lengths

# A Basic Perceptual Audio Encoder

Consists of the following building blocks:

1. Filter bank—is used to decompose the input signal into subsampled spectral components (time/frequency domain)
2. Perceptual model—using either the domain input signal and/or the output of the analysis filter bank, an estimate of the actual masking threshold is computed by rules from psychoacoustics
3. Quantization and coding—the spectral components are quantized and coded; noise introduced should be kept below threshold
4. Lossless Huffman Encoding of bitstream

# A Basic Perceptual Audio Coder

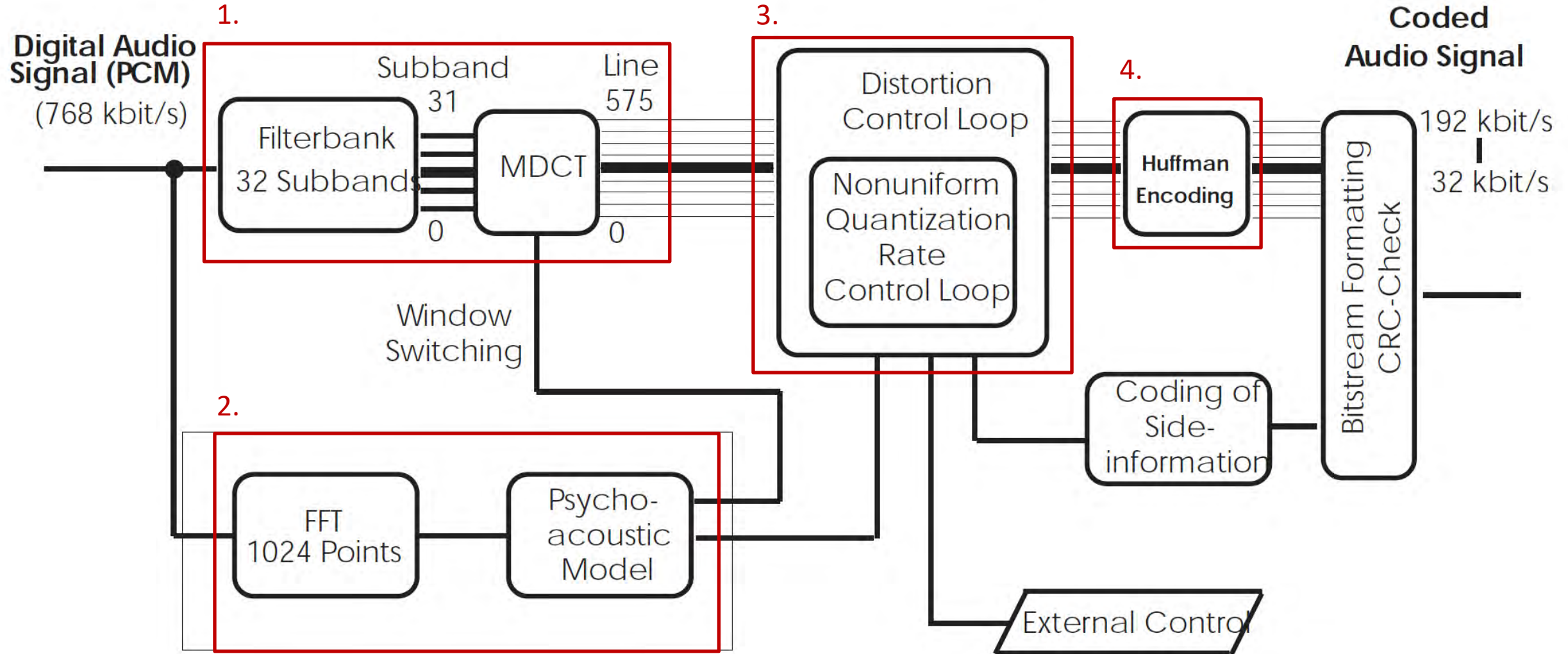


Figure 2: Block diagram of an MPEG-1 Layer-3 encoder.

# Layer-3 Encoding Algorithm

## Basic building blocks

- **Filterbank**: Actually 2 cascading filterbanks:
  - Polyphase filterbank (as in Layer-1 and 2); Cut off frequencies  $<16\text{Hz}$ ,  $>16\text{kHz}$
  - Additional Modified DCT (MDCT); the subdivision of each polyphase frequency band into **18 finer subbands** increases the potential for redundancy removal;
- **Perceptual Model**
  - Mainly determines the quality of a given encoder implementation; uses either a separate filterbank or combines the calculation of energy values (for masking calculations) and main filter bank



# Layer-3 Encoding Algorithm

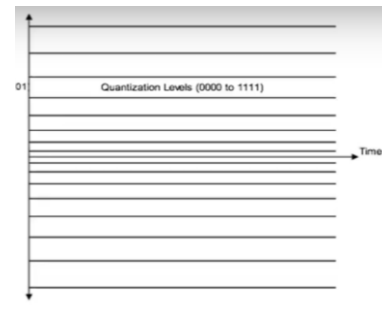
- Perceptual Model (continued)
  - Output of the perceptual model consists of values for the masking threshold or allowed noise for each coder partition (roughly equivalent to the critical bands of human hearing)
- **Quantization** and coding
  - two nested iteration loops (rate loop and noise control loop)
  - quantization is done via a power-law quantizer; larger values are automatically coded with less accuracy
  - quantized values are coded by **Huffman coding**

# Layer-3 Encoding Algorithm

- Quantization and coding (continued)
  - outer iteration loop (noise control loop)

If the quantization noise in a given band is found to exceed the masking threshold (allowed noise) as supplied by the perceptual model, the **scale factor** for this band is adjusted to reduce the quantization noise.  
(power-law quantizer = non-linear, can adjust quantization steps)  
= **adapt quantization steps per band**
  - inner iteration loop (rate loop – modifies overall coder rate)

If number of bits resulting from coding operation exceeds the number of **bits available** to code a **block** of data, a larger quantization step is applied.  
= **adjust global gain**



# Huffman Encoding—Example

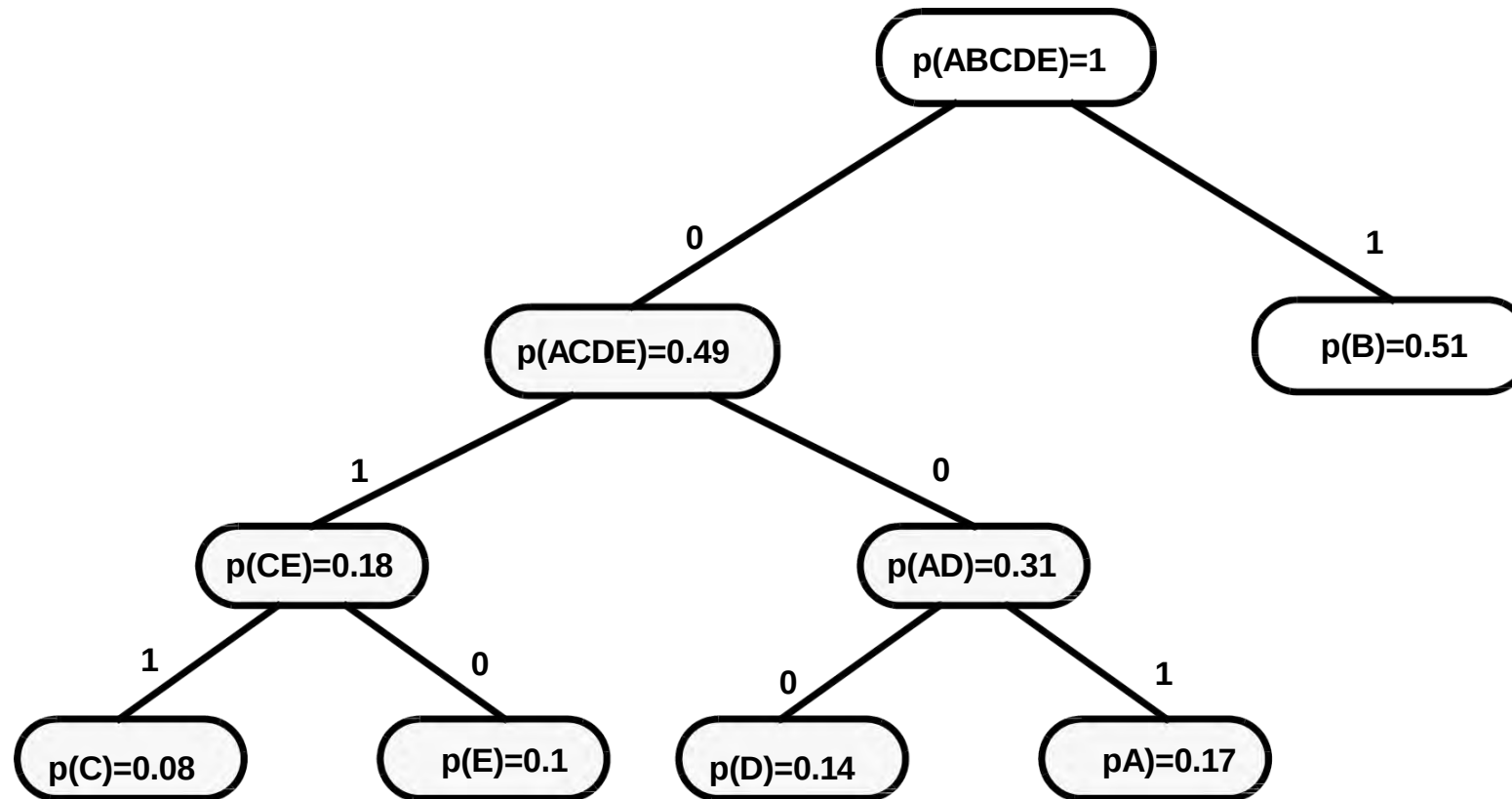
encode symbols according to tree

|   |     |
|---|-----|
| A | 001 |
| B | 1   |
| C | 011 |
| D | 000 |
| E | 010 |

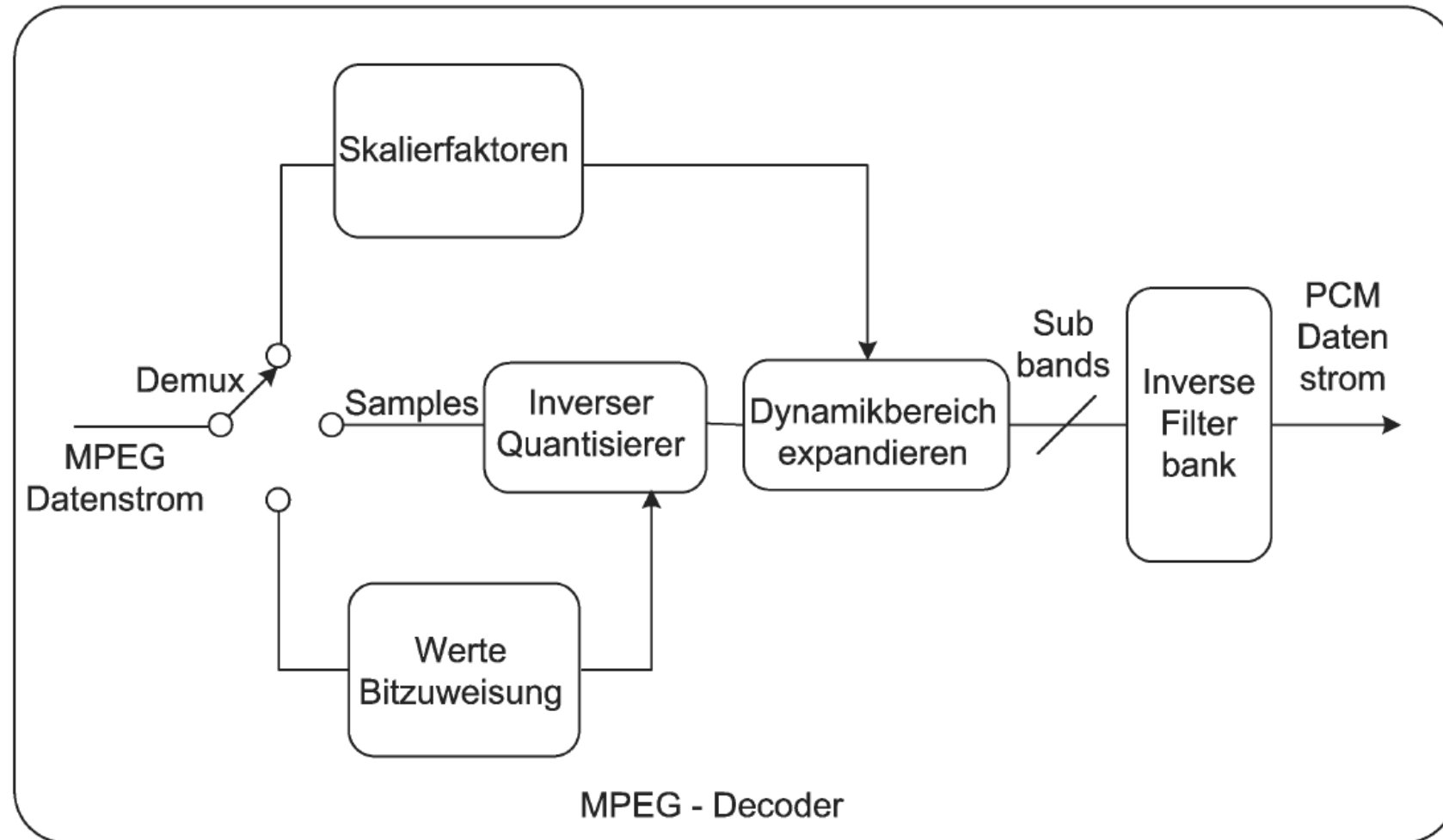
$p(C)=0.08$ ,  $p(E)=0.1$ ,  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(B)=0.51$

# Huffman Encoding—Example

last step:  $p(ACDE)=0.49$ ,  $p(B)=0.51$



# Simple MPEG decoder



# First Step: Split Audio File into “Frames”

- Split audio file into “frames”
- During encoding 576 time domain samples are taken & transformed to 576 frequency domain samples. If there is a transient 192 samples are taken.
  - This is done to limit the temporal spread of quantization noise accompanying the transient.
- The decoder expects a frame consisting of 384, 576, or 1152 samples
  - Typically in MP3 each "frame" has a length of 1152 samples = 0.026 sec (26,122449 ms)
- The following processing happens in each frame individually - with some overlap with previous/next frame to minimize temporal effects

# MP3 Algorithm – Basic Steps (Simplified)

```
algorithm MPEG-1_audio
```

```
/*Input: An audio file in the time domain
```

```
Output: The same audio file, compressed*/
```

```
{
```

```
  Divide the audio file into frames
```

```
  For each frame {
```

```
    By applying a bank of filters, separate the signal into frequency bands.
```

```
    For each frequency band {
```

```
      Perform a Fourier transform to analyze the band's frequency spectrum
```

```
      Analyze the influence of tonal and nontonal elements (i.e., transients)
```

```
      Analyze how much the frequency band is influenced by neighboring bands
```

```
      Find the masking threshold and signal-to-mask ratio (SMR) for the band,  
      and determine the bit depth in the band accordingly
```

```
      Quantize the samples from the band using the determined bit depth
```

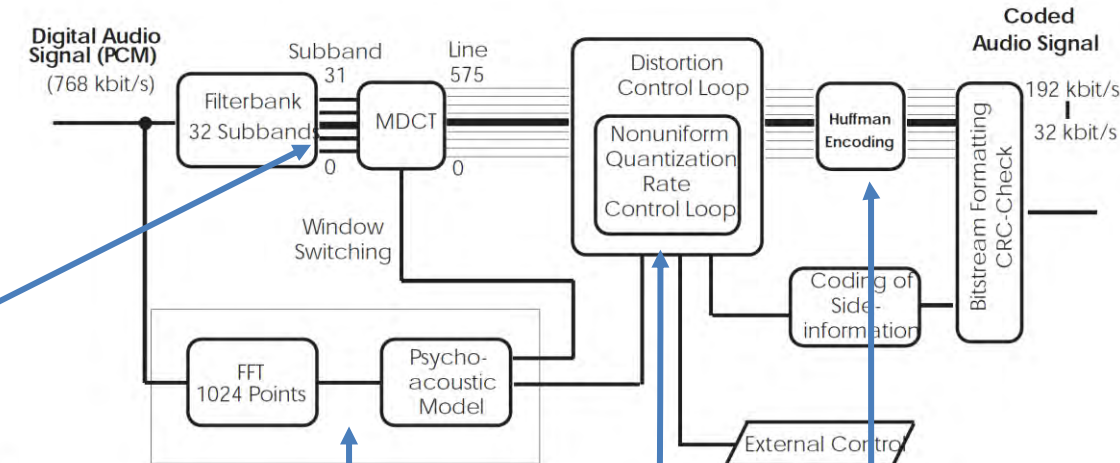
```
      Apply Huffman encoding (optional)
```

```
    }
```

```
  Create a frame with a header and encoded samples from all bands
```

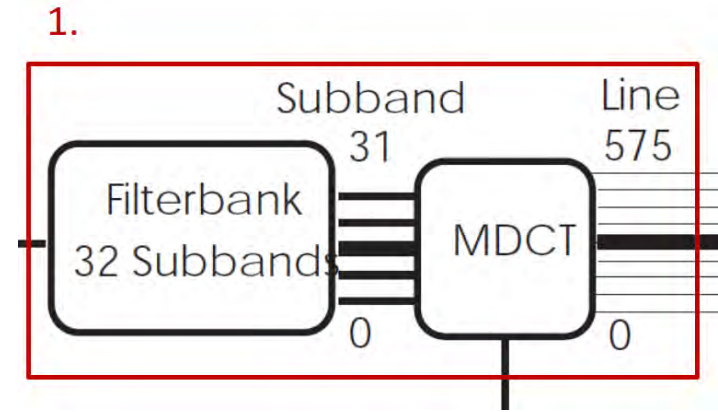
```
}
```

```
}
```





# Subband coding

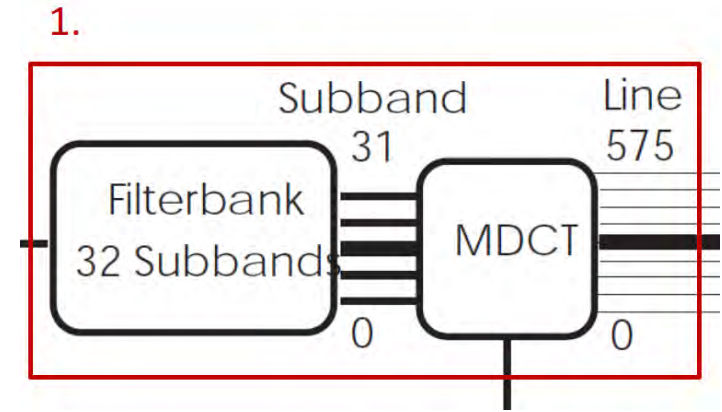


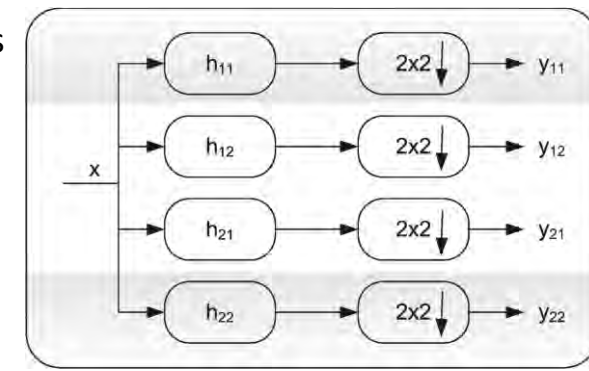
MPEG audio encodes frequency components

- Filter bank
  - divides audio signal into frequency bands (subbands)
- Ideal filter bank
  - Subbands identical to frequency groups of human hearing

# MPEG Polyphase Filter Bank

- Input: Digital PCM Audio Signal
- MPEG Polyphase Filter Bank
  - Audio signal is divided into 32 subbands
  - Each subband has equal size/bandwidth
- Advantage
  - simple structure
- Disadvantage
  - lower subbands span over several frequency groups
    - solved in MPEG-1/Layer 3 by MDCT, modeling human critical bands more closely
  - overlapping subbands lead to information loss





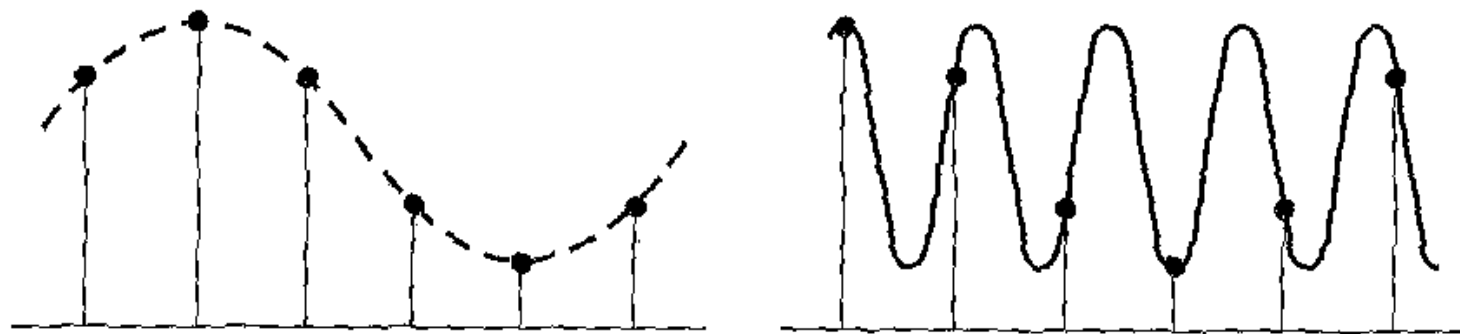
- Example filter bank with 4 subbands
- Input - 1 x PCM: frequency range 0 - 20kHz, sampling rate 48 kHz; word length 16 bit per sample
- Output - 4 x PCM
  - Subband 1: 0Hz - 5kHz; sampling rate 12kHz (!); 16 bit
  - Subband 2: 5kHz -10 kHz; sampling rate 12kHz (!); 16 bit
  - Subband 3: 10kHz - 15 kHz; sampling rate 12kHz (!); 16 bit
  - Subband 4: 15 kHz - 20 kHz; sampling rate 12kHz (!); 16 bit
- Seems to be a violation of the Nyquist-Shannon theorem BUT (see [1])
  - IF a **band does not start at zero frequency** but at some higher value and can be shifted by a linear translation, smaller sampling rate – **at least twice the width of the non-zero frequency interval** - is required.
  - For example, in order to sample the FM radio signals in the frequency range of 100–102 MHz, it is not necessary to sample at 204 MHz (twice the upper frequency), but rather it is sufficient to sample at 4 MHz (twice the width of the frequency interval).

[1]

[https://en.wikipedia.org/wiki/Nyquist%E2%80%93Shannon\\_sampling\\_theorem#Sampling\\_of\\_non-baseband\\_signals](https://en.wikipedia.org/wiki/Nyquist%E2%80%93Shannon_sampling_theorem#Sampling_of_non-baseband_signals)

# Filter Bank - Critical Sampling

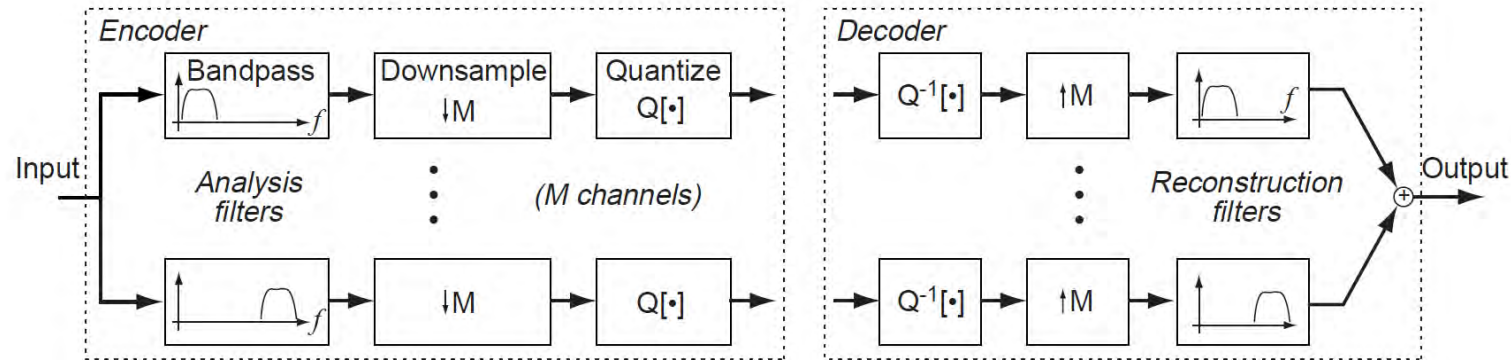
- Typically 32 subbands in MP3
- Critical sampling of digital filter banks
  - Sampling rate in subbands reciprocal to the number of subbands
    - Reduced sampling rate in subbands is called critical sampling rate
- Critical sampling of subbands ensures that **filter bank has no influence on data volume**
  - More channels are generated, but their sampling frequencies are correspondingly lower.



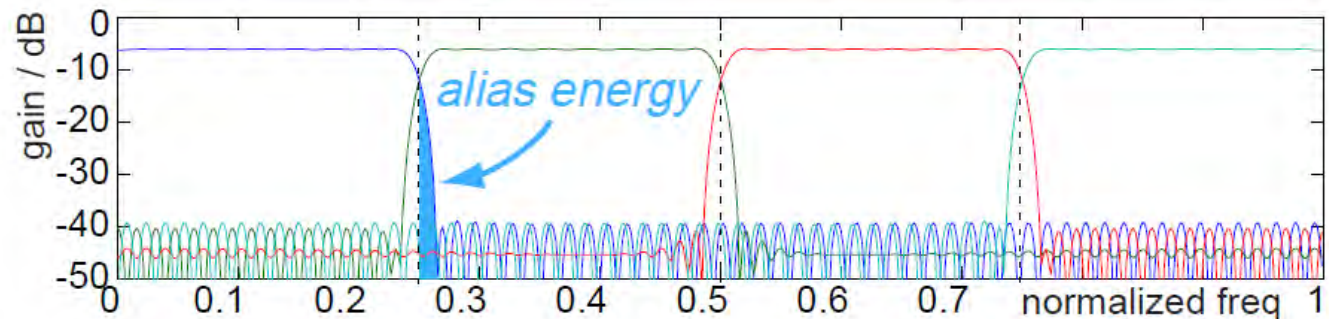
- Left: Samples / signal without specification of frequency range
- Right: Samples / signal with indication of frequency range (right)

# Subband Coding

- Idea: Quantize separately in separate bands
  - Quantization Noise stays within band, gets masked



- 'Critical sampling'  $\rightarrow 1/M$  of spectrum per band
  - some aliasing inevitable



- Trick is to cancel with alias of adjacent band
  - $\rightarrow$  'quadrature-mirror' filters / MDCT

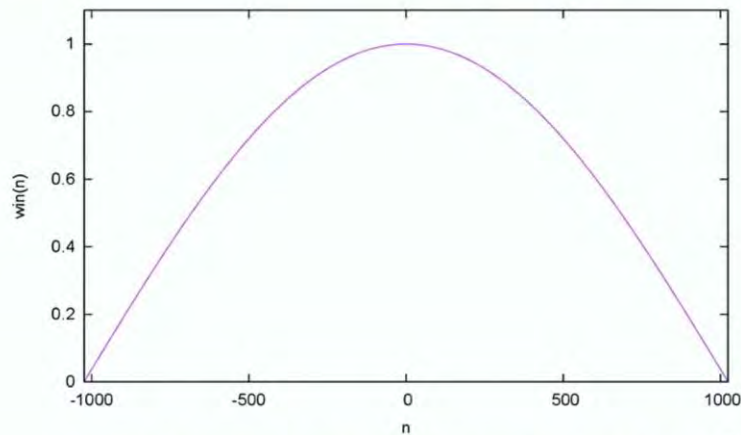
# Subbands in MPEG Standard

- MPEG-1/Layer 1 and Layer 2 - 32 Subband Filter Bank
- In MPEG-1/Layer 3 (MP3) a Hybrid filter bank is used
  - Modified Discrete Cosine Transform (MDCT) performs transformation into frequency domain
- In MPEG-2 - AAC
  - Lossless MDCT - frequency decomposition corresponds to 1024 subbands

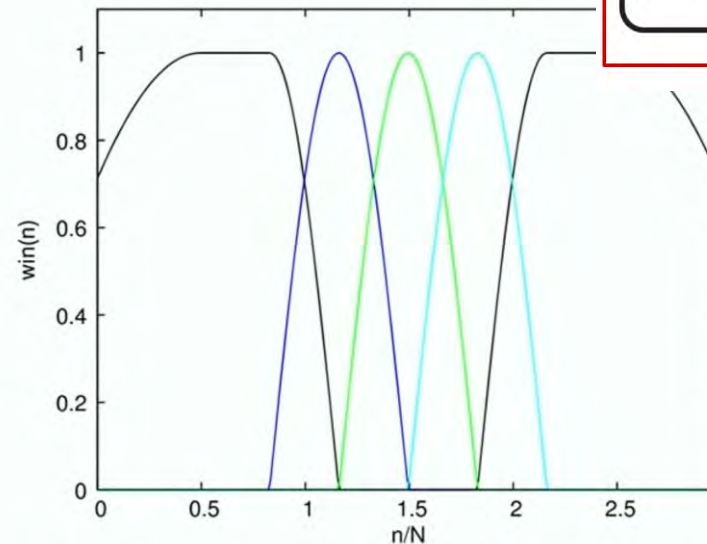
# Modified Discrete Cosine Transform (MDCT)

- Performs transformation of subbands into frequency domain
- Overlaps with neighboring frequency bands (but does not increase sample size)
- MDCT also used for avoiding temporal alias effects

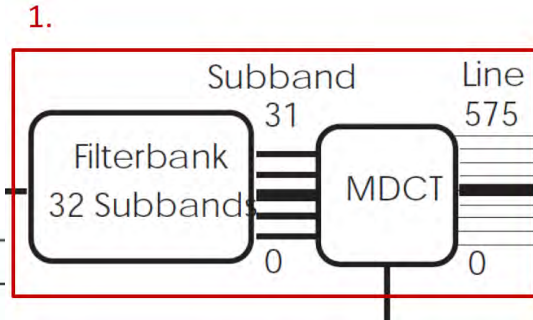
## Modified Discrete Cosine Transform (MDCT)



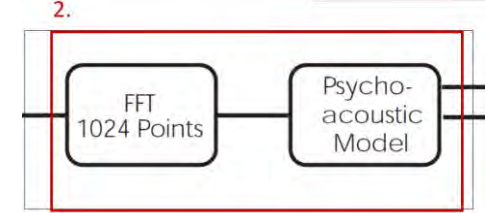
Sine window frequently used for MDCT



Adaptively switched MDCT windows

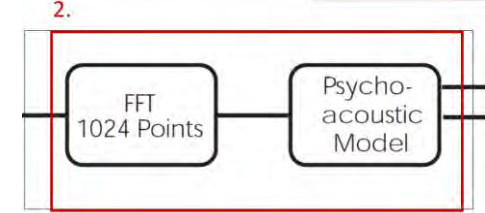






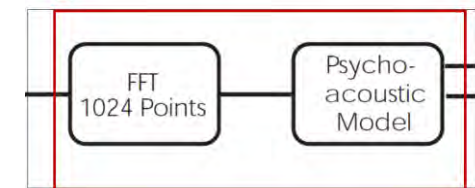
# MPEG Psychoacoustic Model: Overview

- First: Fast Fourier Transform into frequency domain
  - Psychoacoustic model operates on frequency spectrum
- Analyzes audio signal section by section for masking effects
  - Algorithms simulate human hearing
- Calculates masking threshold per subband
  - calculates maximum allowed quantization noise per subband
- Quality of the psychoacoustic model mainly responsible for overall quality of coding

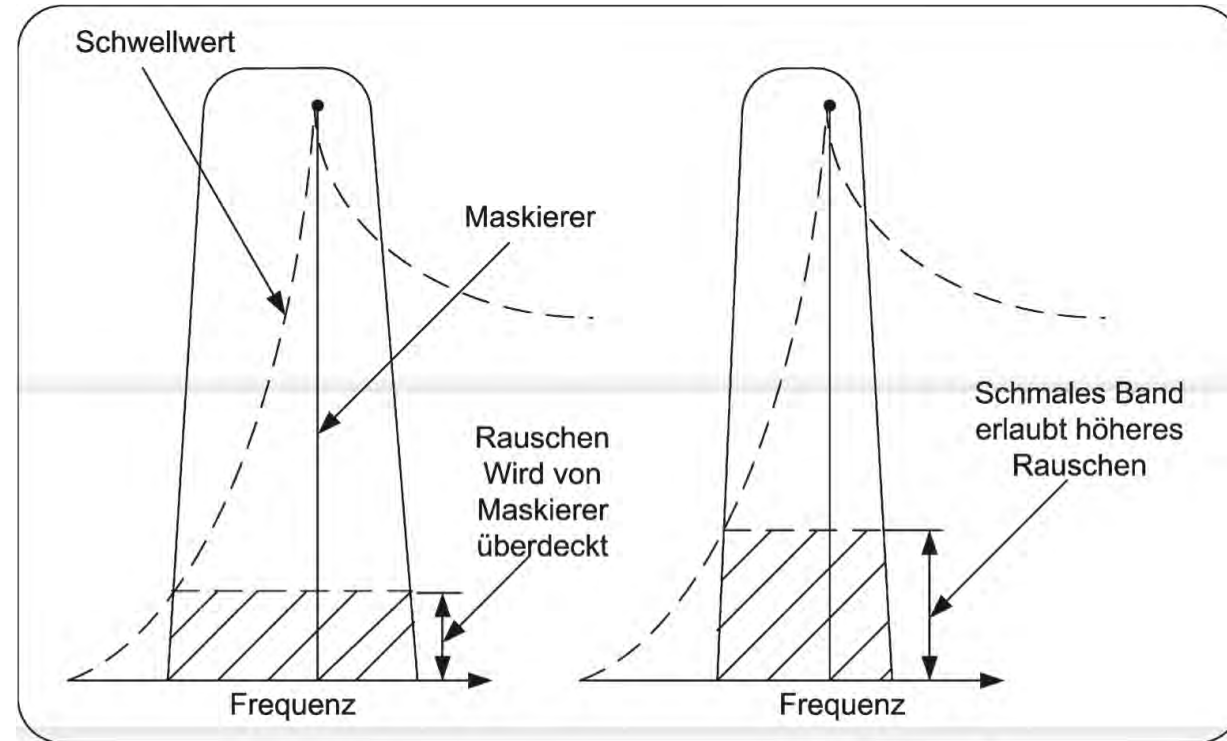


# Fast Fourier Transform

- For signal analysis FFT is particularly well suited
  - Provides fine frequency resolution of the signal
    - provides phase information of the frequency components (important for psychoacoustic model 2 (mentioned later))
  - Very accurate calculation of the masking thresholds
- ➡ high compression possible
- Technique for minimizing the computational effort
    - Bundling values in frequency groups
    - Frequency groups correspond to critical bandwidths of human hearing

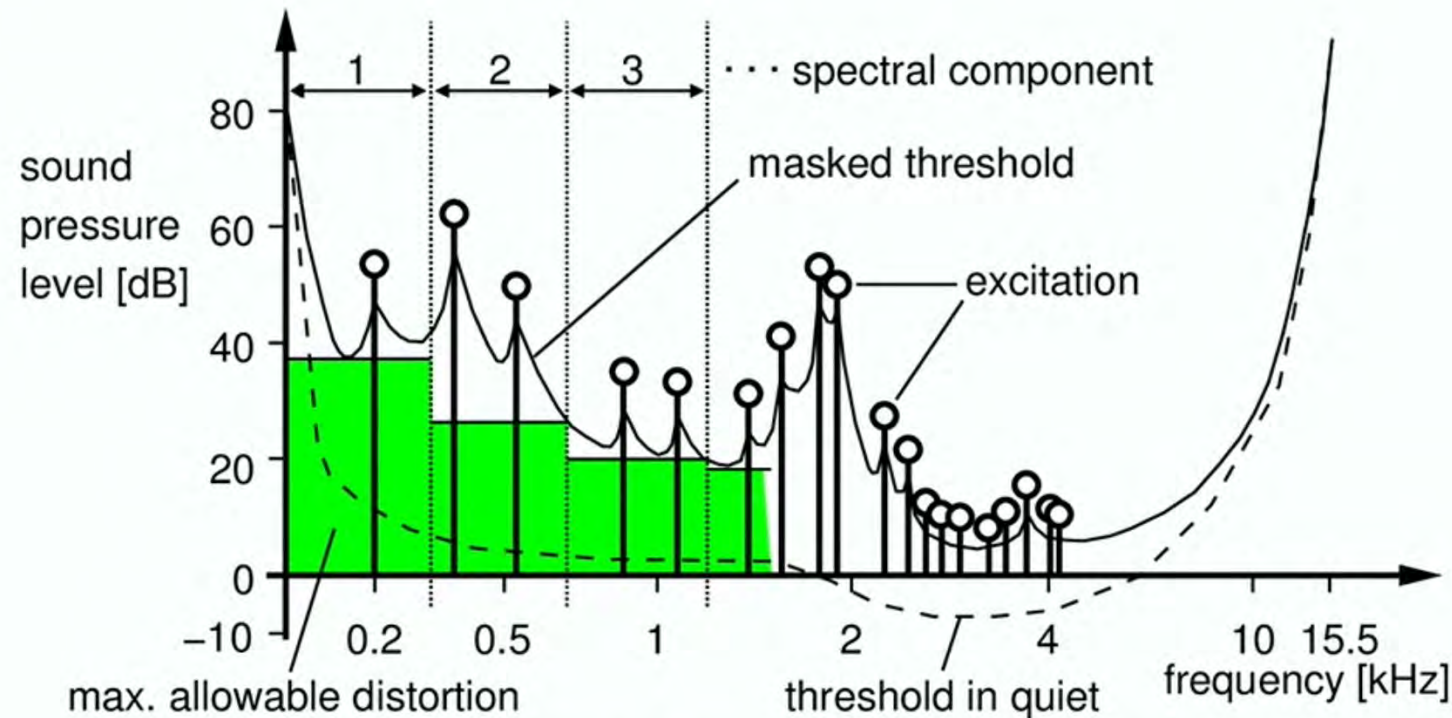


# Masking in Psychoacoustic Model

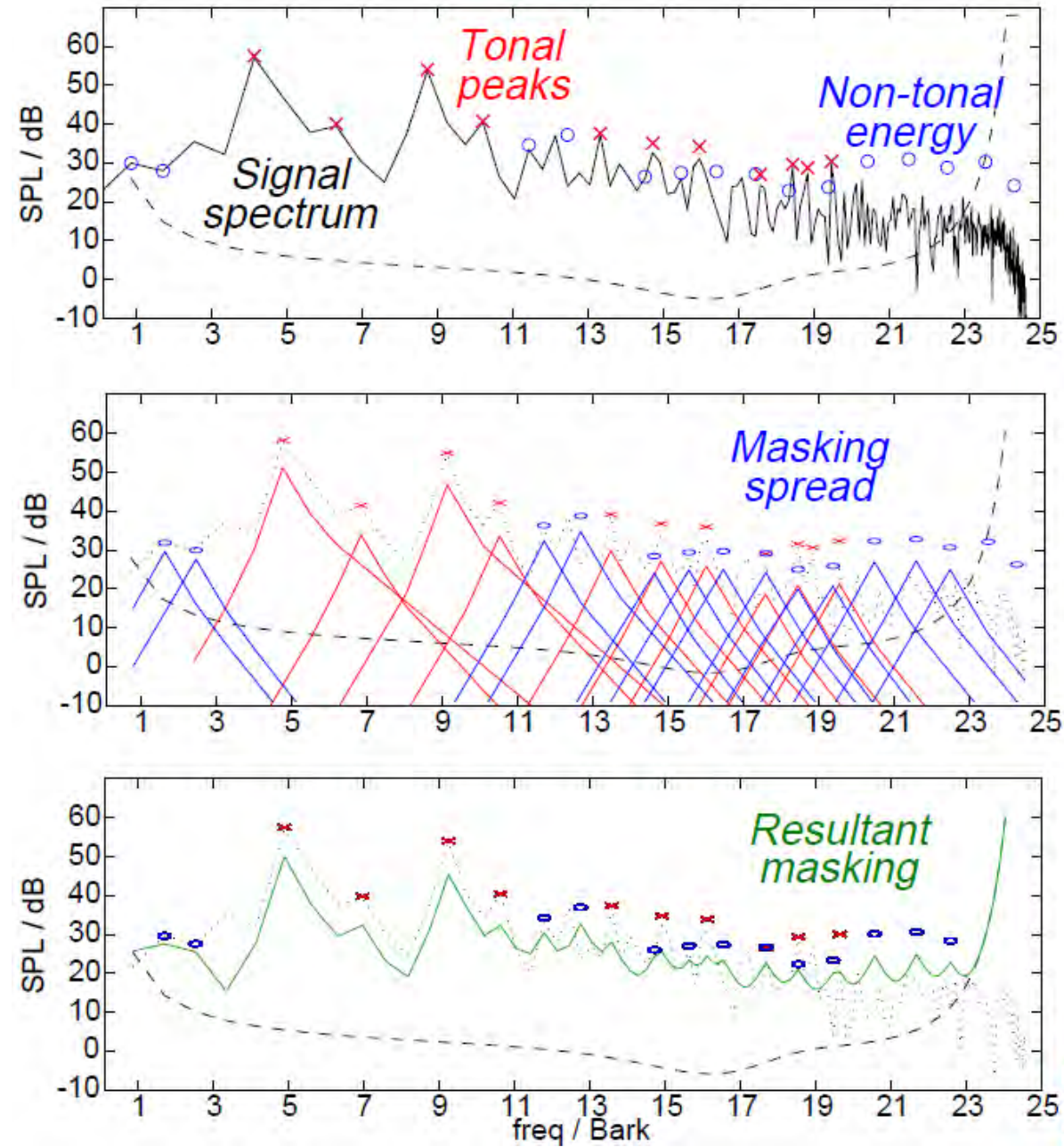


# Masking in Psychoacoustic Model

- For each subband: How high can quantization noise be, so that it is still masked?

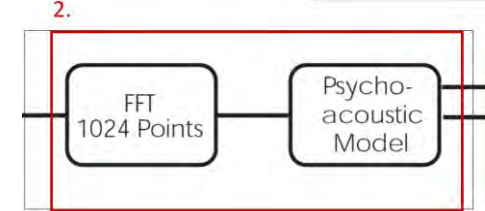


Spectral noise shaping in transform/subband coding

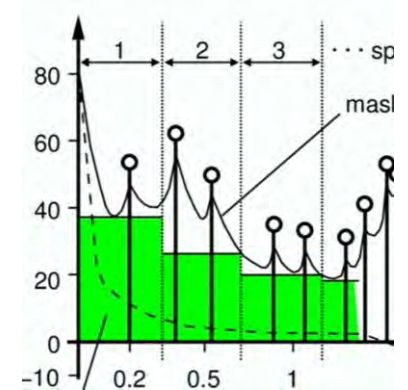


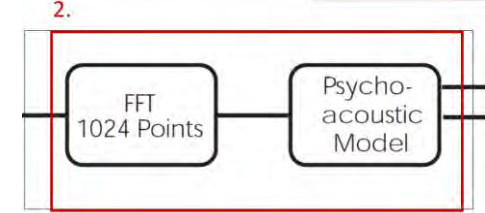


# Masking in Psychoacoustic Model



- Because of narrow subbands
  - high masking threshold for noise possible
  - frequency components require only few bits, which corresponds to high compression
- In general
  - the higher the subband number the narrower the subbands
  - the narrower the subbands the higher the compression

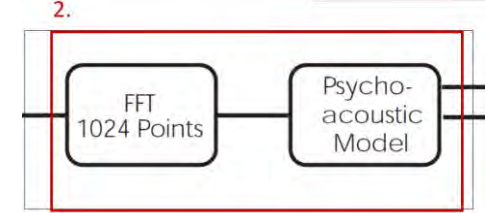




# Tonal / Atonal Components

- Tonal and noise-like acoustic events have different masking properties
- Psychoacoustic model needs algorithms to distinguish between sound and noise
- Model 1
  - strong peaks in the frequency spectrum evaluated as tonal components
  - remaining parts interpreted as noises
- Model 2
  - more complex algorithms, considering phase, etc..

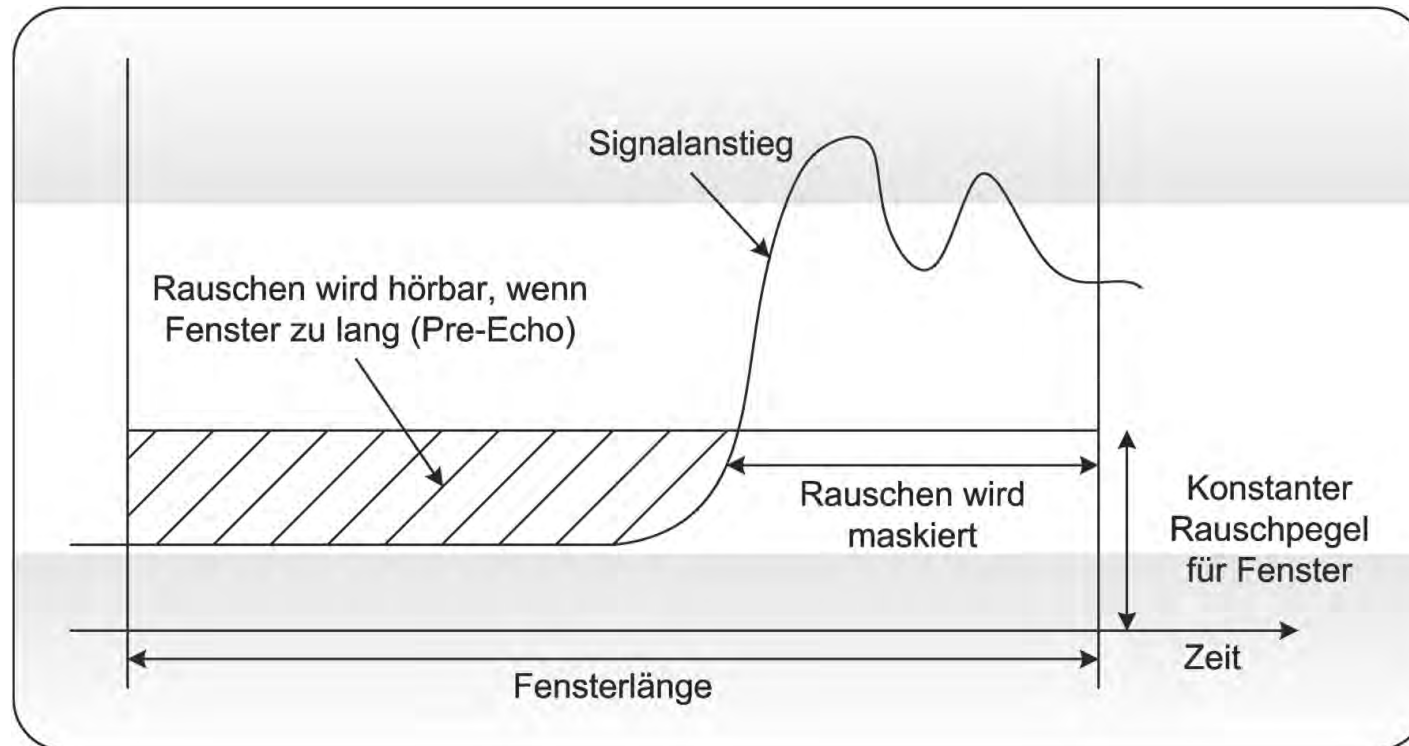




# Result Psychoacoustic Model

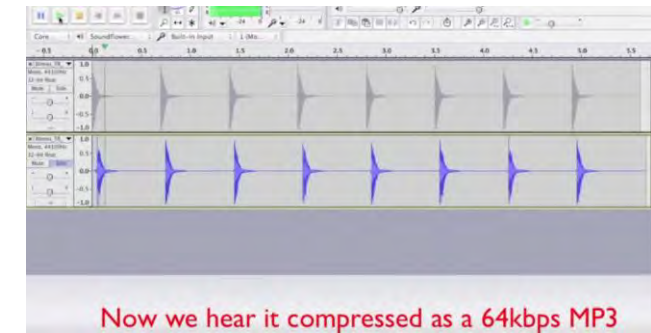
- Psychoacoustic model calculates hearing threshold curve
  - Noise below curve is not audible
  - Noise above curve is audible
  - for noise-free coding, audible noise level must not be exceeded at any point
- Calculation of the masking thresholds
  - for each subband the masking threshold is calculated
  - the smaller the subbands are, the better the fit of the step function to hearing threshold curve (high compression)

# Compression with Steep Signal Rise – Temporal Issues



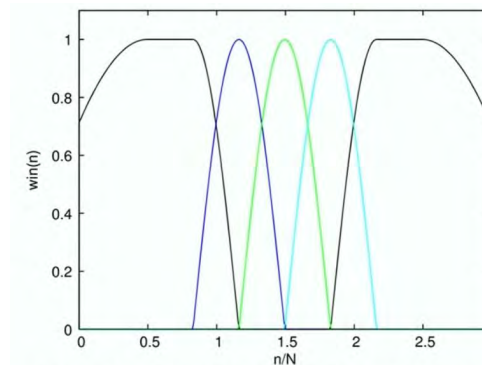
# Compression with Steep Signal Rise – Pre-Echo

- Signal sections (windows) with steep signal slopes are very difficult to compress
  - Quantization noise equal over entire window length
  - Noise before signal rise audible as **pre-echo**
    - Quantization noise is unmasked in the silence before “attack”
- MPEG Strategies
  - Layer1/Layer2 - no artifact-free compression possible
  - MP3 - uses short window for this, pre-echo effect is reduced, but moderate compression, as less subbands are achievable with short window
  - AAC - short window, Temporal Noise Shaping (TNS) module, good compression

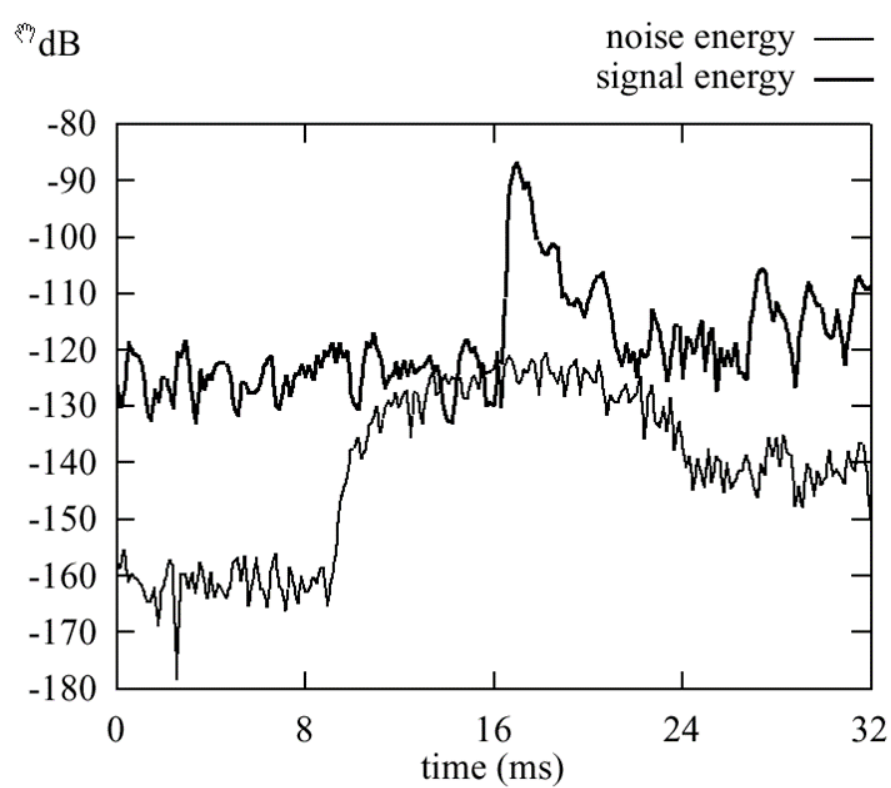


Pre-Echo

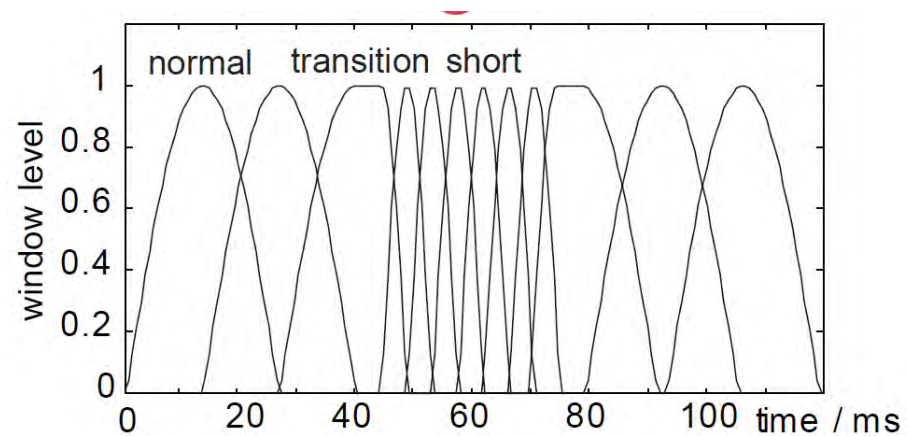
<https://www.youtube.com/watch?v=MtiRBFwKRs>



# Pre-Echo



- Adaptive switching of time-window



# Frequency Resolution / Time Window

High frequency resolution with long time window

+ : many subbands, high compression

- Subbands well adaptable to hearing threshold curve

- : Quantization noise not changeable for a long time

- bad with signal increases

Low frequency resolution with short time window

+ : Quantization noise changeable in short time intervals

- Noise can be well matched to time curve

- : Fewer subbands, moderate compression

# A Basic Perceptual Audio Coder

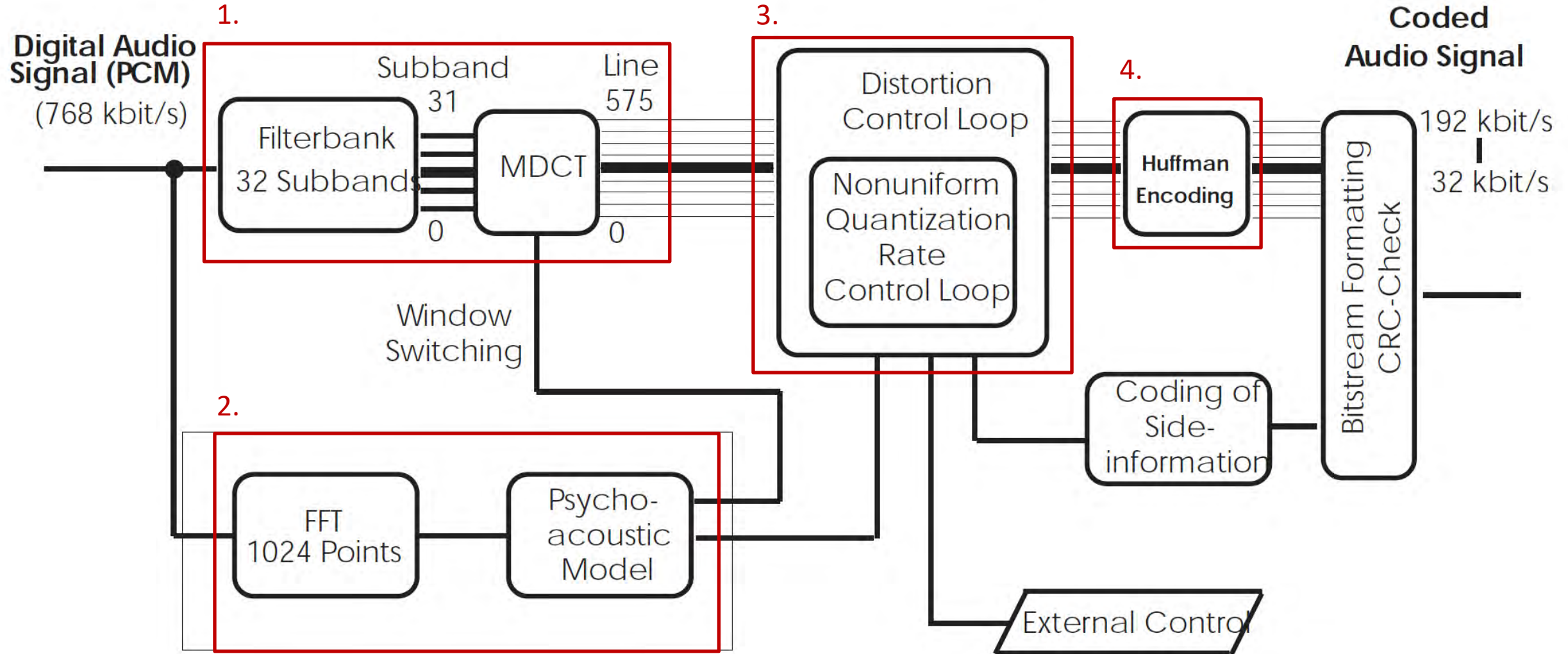


Figure 2: Block diagram of an MPEG-1 Layer-3 encoder.

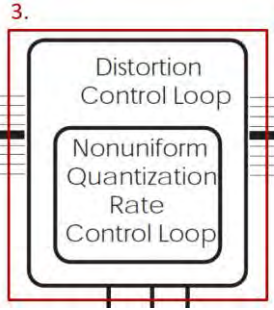


- Basic idea: Subband coding plus psychoacoustic masking model to choose dynamic Quantization levels in subbands

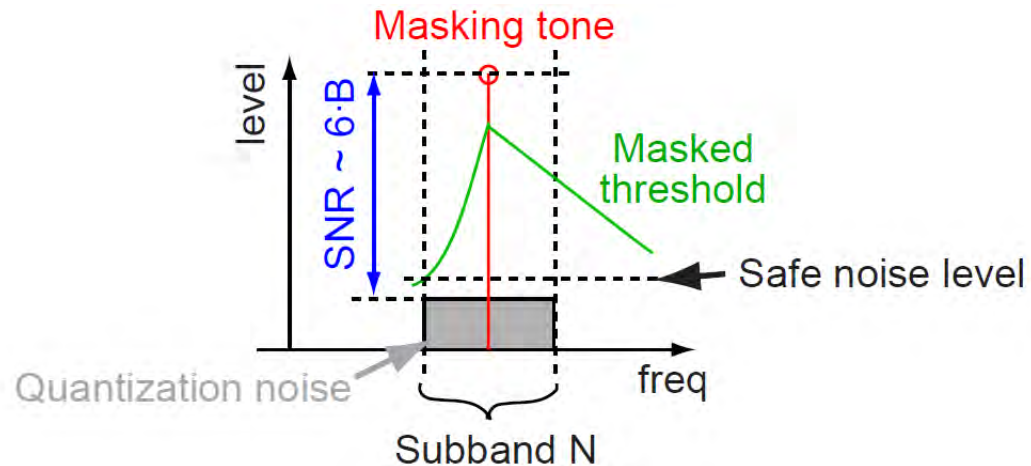




# MPEG Bit allocation



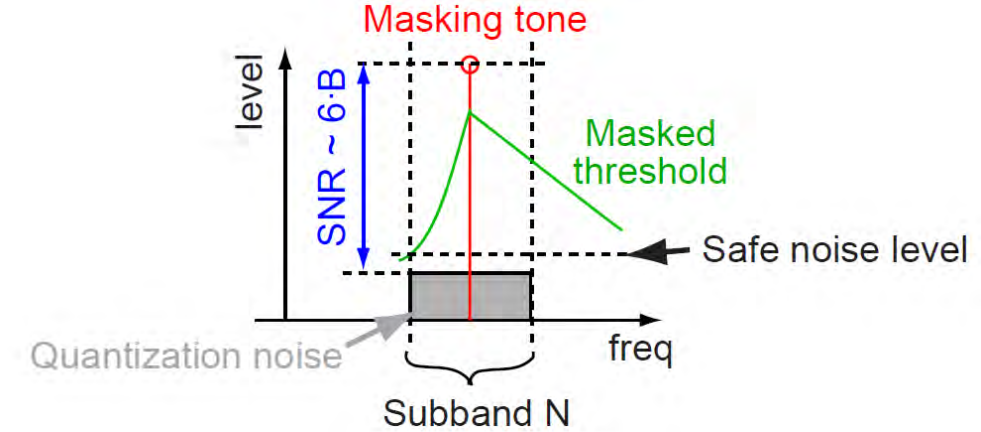
- **Result of psychoacoustic model is maximum tolerable noise per subband**



- safe noise level  $\rightarrow$  required SNR  $\rightarrow$  bits B

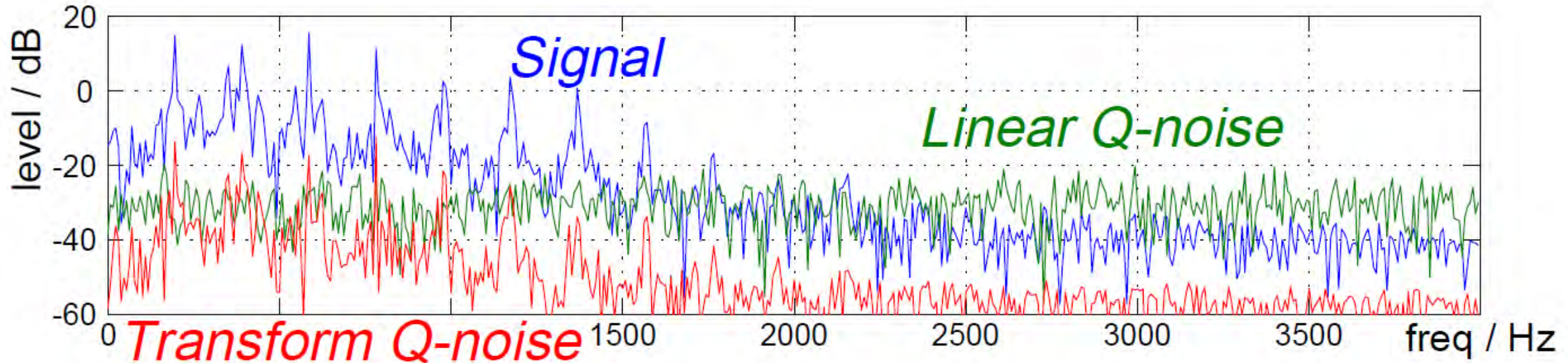
- **Bands with no signal above masking curve can be skipped entirely**
- **Bit allocation procedure (fixed bit rate):**
  - pick channel with worst noise-masker ratio
  - improve its quantization by one step
  - repeat while more bits available for this frame

# Layer-3 Quantization Algorithm



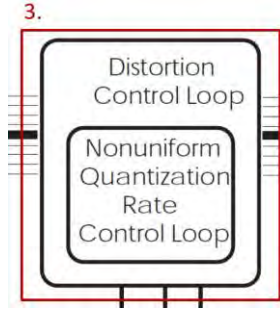
- **Quantization** and coding
  - quantization is done via a power-law quantizer; larger values are automatically coded with less accuracy
  - two nested iteration loops (rate loop and noise control loop)
- Outer iteration loop (noise control loop) – for each band
  - If the quantization noise in a given band is found to exceed the masking threshold (allowed noise) - supplied by the perceptual model - the scale factor for this band is adjusted to reduce the quantization noise.
- Inner iteration loop (rate loop – modifies overall coder rate) – for each frame
  - Check if Huffman coding is small enough
  - If number of bits resulting from coding operation exceeds the number of bits available to code a block of data, **adjust global gain** -> results in larger quantization step.

# Example

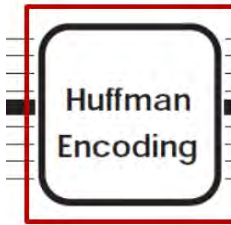


Comparison: Linear Quantization Noise over whole frequency band (green) compared to reduced quantization noise (in red) where noise is scaled in different frequency bands.

# Bit assignment



- User can select desired MPEG bit rate
  - Psychoacoustic model calculates values so that no quantization noise is audible
  - at low data rates, available bits are usually not sufficient
  - Bit allocation algorithms
    - high frequencies are removed
    - Stereo will be spatially inaccurately resolved
    - Well-adjusted introduction of quantization noise
- Algorithms are left to software developers
- MPEG Audio standard offers tables as recommendation



# Huffman Encoding

- Final quantized bands will be Huffman encoded = lossless encoding (the lossy part happened before)
- This is all still done in the frequency domain!

# Huffman Encoding

- Statistical encoding, depends on **occurrence frequency of single characters or sequences** of data bytes
- Characters stored with their probabilities
- Length (number of bits) of the coded characters differs
- Shortest code assigned to the most frequently occurring character
- To determine Code, it is useful to construct a **binary tree**
  - leaves are characters to be encoded
  - nodes carry **occurrence probabilities** of the characters belonging to the subtree

# Huffman Encoding — Algorithm

build a Huffman Tree:

$p(A)=0.17$ ,  $p(B)=0.51$ ,  $p(C)=0.08$ ,  $p(D)=0.14$ ,  $p(E)=0.1$

```
build ordered list of symbols (increasing probability)
```

```
do while list contains at least 2 elements
```

```
    construct tree using the first two elements in list
```

```
    add parent node for the union of these elements and compute probability
```

```
    mark edges by '0' and '1'
```

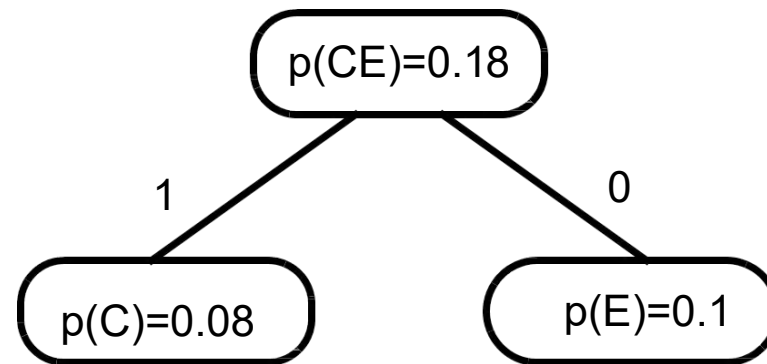
```
    delete the first 2 elements in list; insert parent into list
```

```
end
```



# Huffman Encoding—Example

step 1:  $p(C)=0.08$ ,  $p(E)=0.1$ ,  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(B)=0.51$



# Huffman Encoding—Example

step 2:  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(CE)=0.18$ ,  $p(B)=0.51$

build ordered list of symbols (increasing probability)

do while list contains at least 2 elements

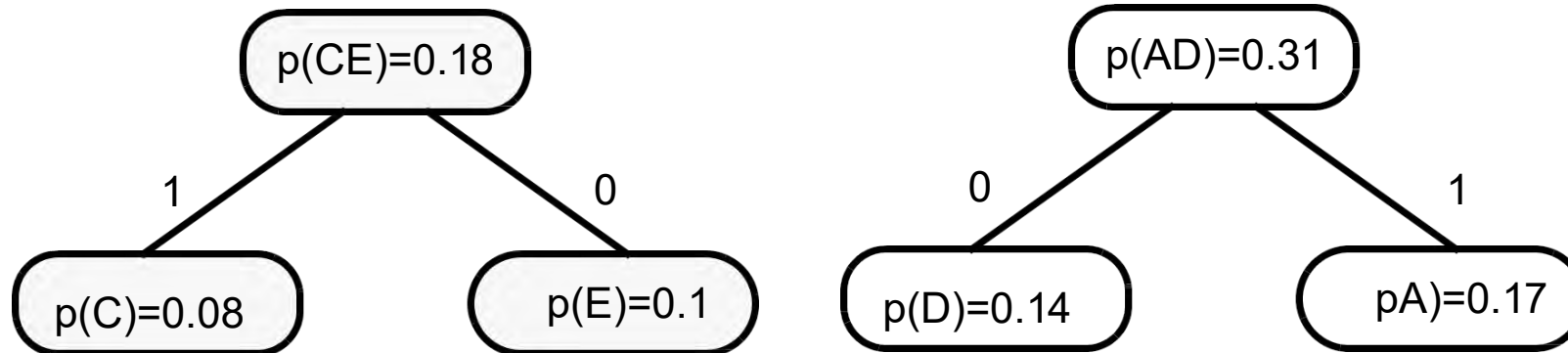
construct tree using the first two elements in list

add parent node for the union of these elements and compute probability

mark edges by '0' and '1'

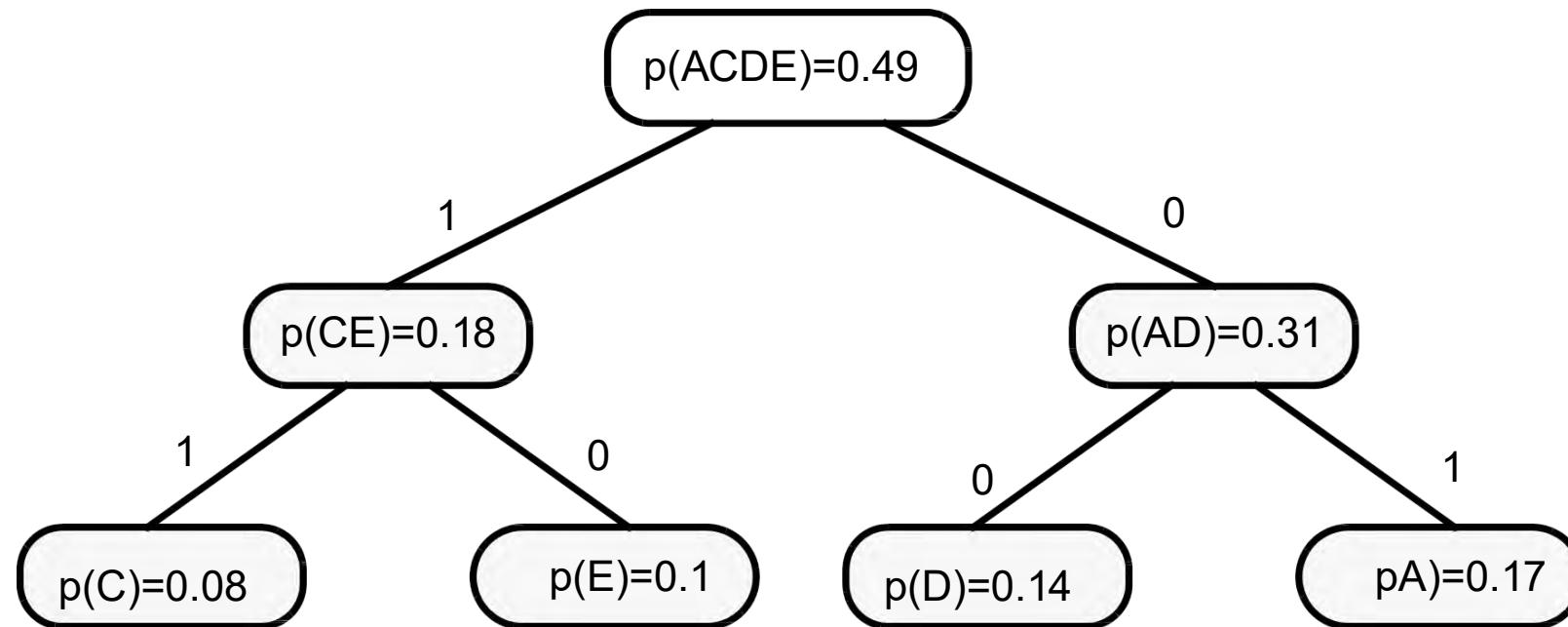
delete the first 2 elements in list;  
insert parent into list

end



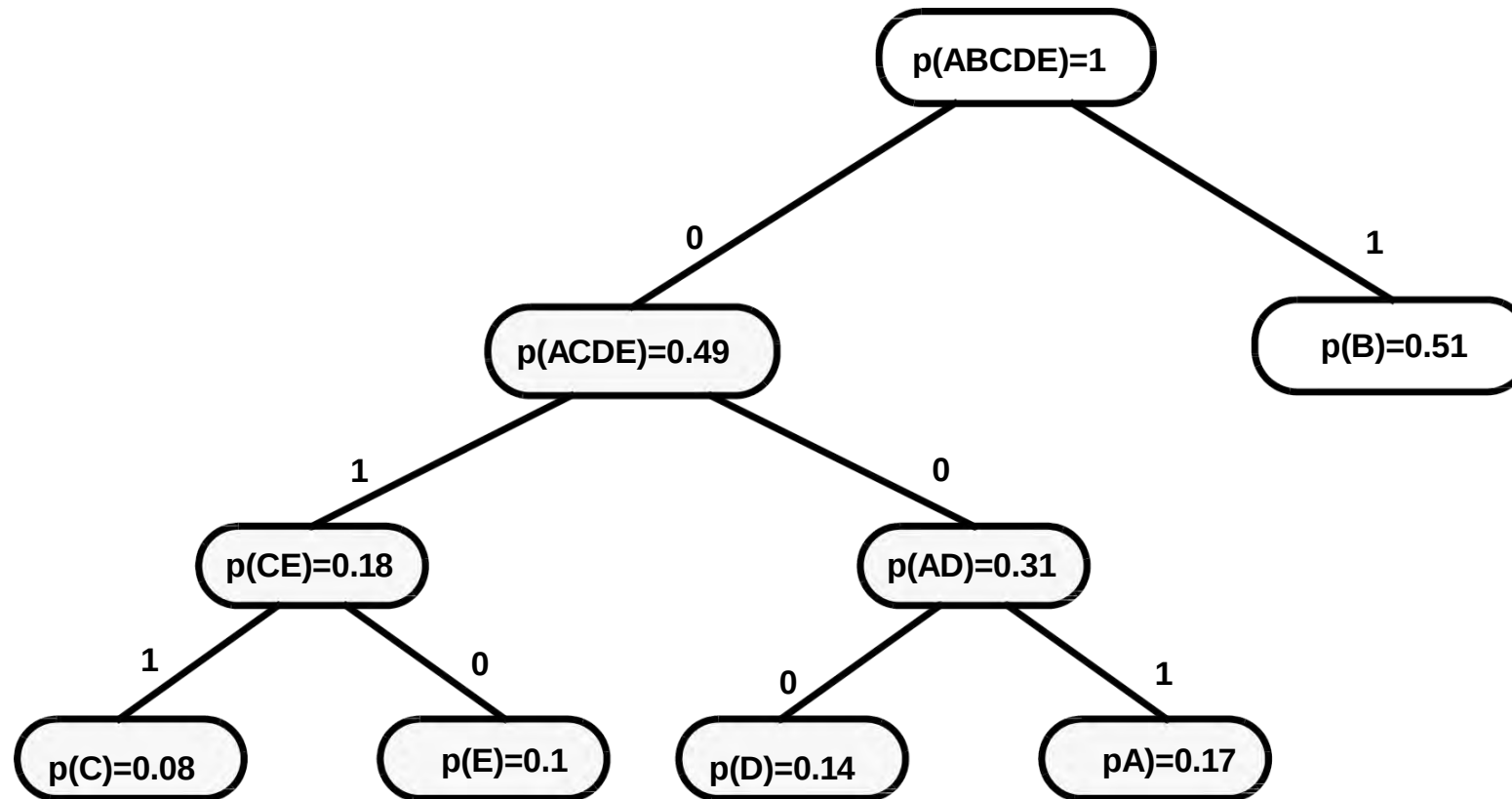
# Huffman Encoding—Example

step 3:  $p(CE)=0.18$ ,  $p(AD)=0.31$ ,  $p(B)=0.51$



# Huffman Encoding—Example

last step:  $p(ACDE)=0.49$ ,  $p(B)=0.51$



# Huffman Encoding—Example

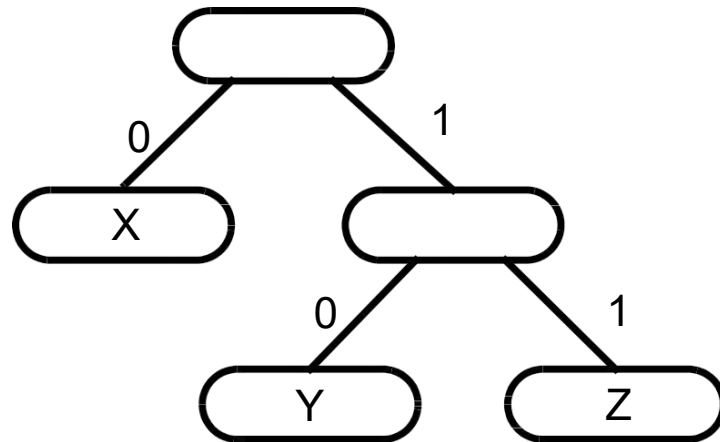
encode symbols according to tree

|   |     |
|---|-----|
| A | 001 |
| B | 1   |
| C | 011 |
| D | 000 |
| E | 010 |

$p(C)=0.08$ ,  $p(E)=0.1$ ,  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(B)=0.51$

# Huffman Decoding—Example

given: 000101011 and Huffman Tree

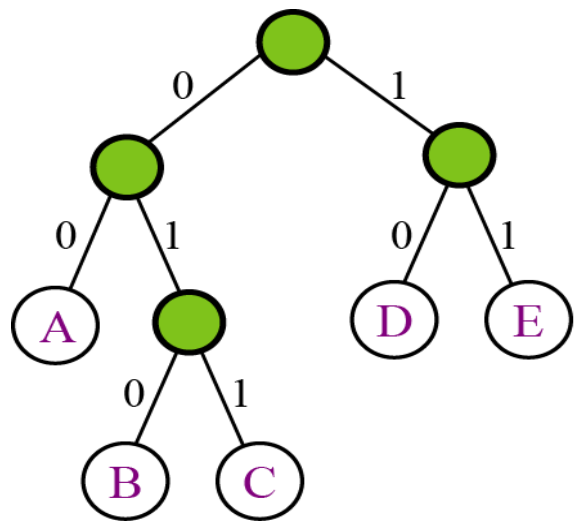


reconstruct message from bit stream using  
finite state machine

Solution: XXXYYZ

# Huffman Encoding

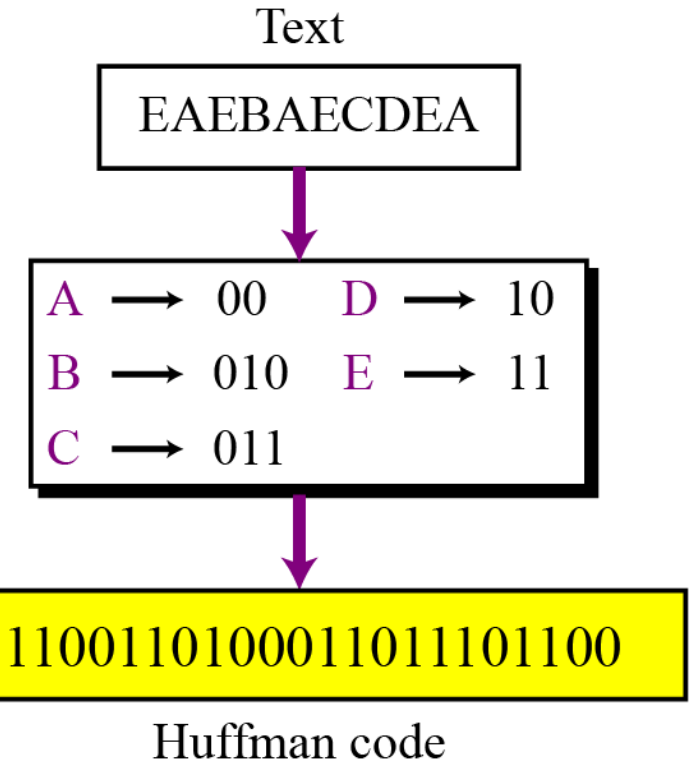
- Character code found by starting at the root and following the branches that lead to that character.
- The code itself is the bit value of each branch on the path, taken in sequence.



|        |       |
|--------|-------|
| A: 00  | D: 10 |
| B: 010 | E: 11 |
| C: 011 |       |

Code

Encoder



- Decoding: reverse process



# MP3 Bandwidth

The MP3 audio format provides lossy data compression. Audio quality improves with increasing bitrate.

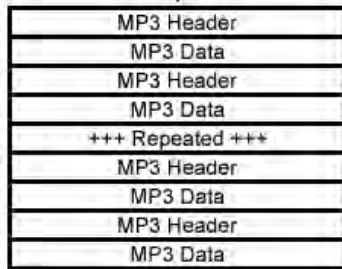
- Constant Bit Rate (CBR) – constant over whole file
- Variable Bit Rate (VBR) – a bit reservoir allows distribution of bit rate over frames
  - lower bit rate for the less complex passages and a higher one for the more complex parts
  - In MP3: Average Bitrate (ABR)
- 32 kbit/s – generally acceptable only for speech
- 96 kbit/s – generally used for speech or low-quality streaming
- 128 or 160 kbit/s – mid-range bitrate quality
- 192 kbit/s – medium quality bitrate
- 256 kbit/s – a commonly used high-quality bitrate
- 320 kbit/s – highest level supported by the MP3 standard



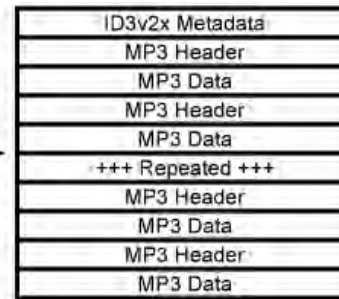
# Simple MPEG data stream

| Header | CRC | Bit Zuweisung | Skal. Faktor | Sub-band | Werte | Hilfs-daten |
|--------|-----|---------------|--------------|----------|-------|-------------|
|--------|-----|---------------|--------------|----------|-------|-------------|

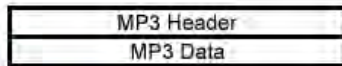
Internal Structure of an MP3 File



Note that the MP3 file structure may be 'encapsulated' within an ID3 tag.



An MP3 Frame



Example MP3 Header

FFBFA000

Colour-coding shows binary bit mapping to hex values below

Detail of an MP3 Header

| Bits    | 1             | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13       | 14           | 15 | 16 | 17               | 18 | 19 | 20 | 21 | 22         | 23 | 24 | 25 | 26 | 27            | 28                                         | 29 | 30                      | 31        | 32   |         |                   |  |  |                          |                   |                     |                            |           |  |
|---------|---------------|---|---|---|---|---|---|---|---|----|----|----|----------|--------------|----|----|------------------|----|----|----|----|------------|----|----|----|----|---------------|--------------------------------------------|----|-------------------------|-----------|------|---------|-------------------|--|--|--------------------------|-------------------|---------------------|----------------------------|-----------|--|
| Binary  | 1             | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1  | 1  | 1  | 1        | 0            | 1  | 1  | 0                | 0  | 1  | 0  | 0  | 0          | 0  | 0  | 0  | 1  | 0             | 0                                          | 0  | 0                       | 0         | 0    |         |                   |  |  |                          |                   |                     |                            |           |  |
| Hex     | F F F F       |   |   |   |   |   |   |   |   |    |    |    | B        |              |    | 2  |                  |    |    |    | 0  |            |    | 4  |    |    | 0             |                                            |    |                         |           |      |         |                   |  |  |                          |                   |                     |                            |           |  |
|         |               |   |   |   |   |   |   |   |   |    |    |    |          |              |    |    |                  |    |    |    |    |            |    |    |    |    |               | Mode Extension<br>(Used With Joint Stereo) |    |                         |           |      |         |                   |  |  |                          |                   |                     |                            |           |  |
| Meaning | MP3 Sync Word |   |   |   |   |   |   |   |   |    |    |    | Version  | Layer        |    |    | Error Protection |    |    |    |    | Bit Rate   |    |    |    |    | Frequency     |                                            |    | Pad. Bit                | Priv. Bit | Mode |         |                   |  |  | Copy                     | Original          | Emphasis            |                            |           |  |
| Value   | Sync Word     |   |   |   |   |   |   |   |   |    |    |    | 1 = MPEG | 01 = Layer 3 |    |    | 1 = No           |    |    |    |    | 1010 = 160 |    |    |    |    | 00 = 44100 Hz |                                            |    | 0 = Frame is not padded |           |      | Unknown | 01 = Joint Stereo |  |  | 0 = Intensity Stereo Off | 0 = MS Stereo Off | 0 = Not Copyrighted | 0 = Copy Of Original Media | 00 = None |  |

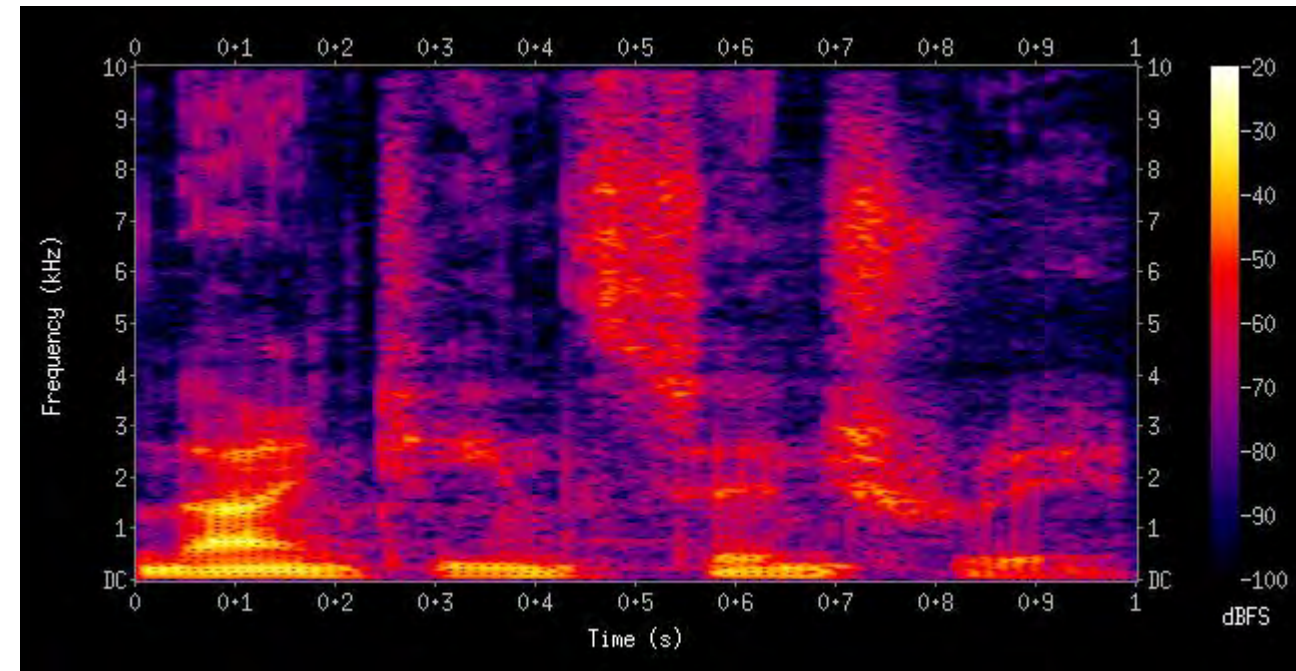
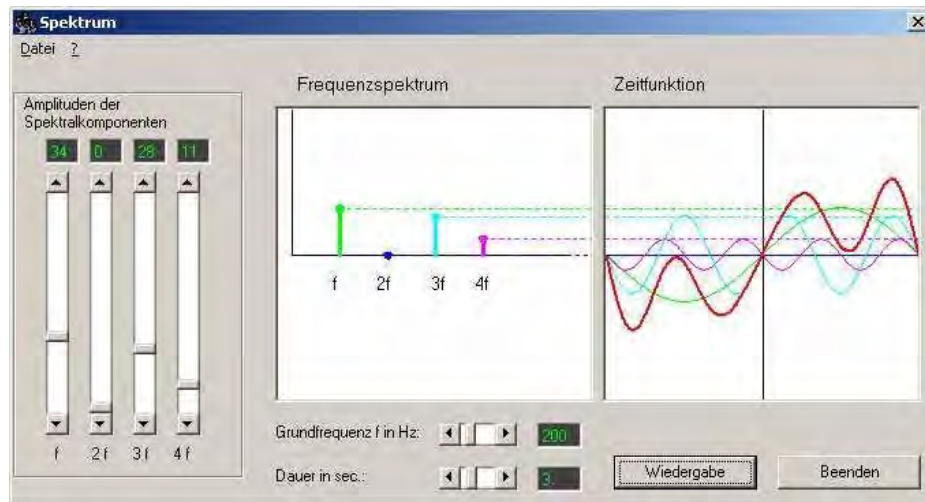
- Header - description of the data stream
- Redundancy Code (CRC) - detection of data stream errors
- Bit assignment - word length of the following subband values
- Scaling factor - 6-bit scaling factor for the following subband values
- Auxiliary data - any data can be inserted

# MP3 Further Details

- Further details the algorithm and implementation specifics (based on LAME MP3 encoder)
- <https://wiki.hydrogenaud.io/index.php?title=MP3>

# Spectrogram

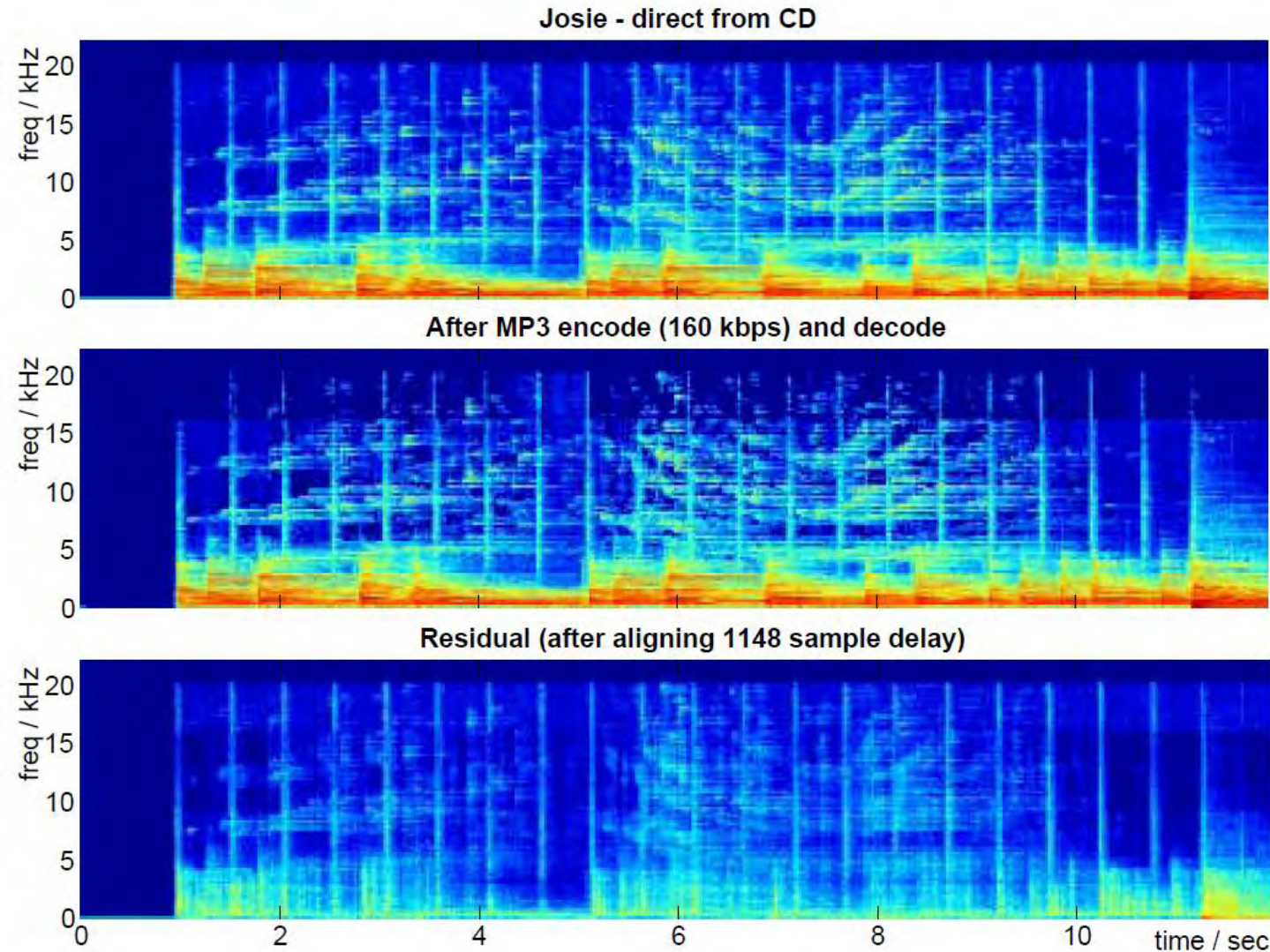
- A **spectrogram** is a visual representation of the spectrum of frequencies of a signal as it varies with time.





# The Effects of MP3

- Chop off high frequency (above 16 kHz)
- Occasional other time-frequency 'holes'
- Quantization noise under signal



# Quality Considerations

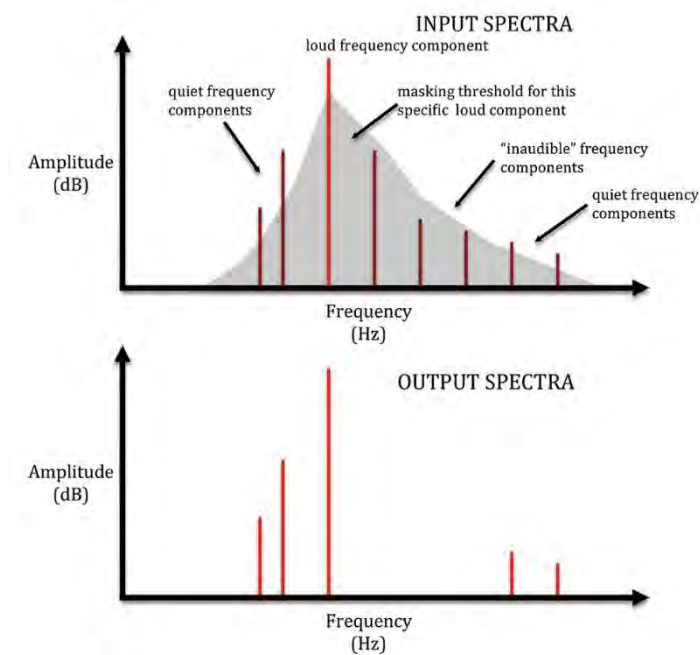
- Common types of artifacts

- loss of bandwidth (no signal >16kHz)
- pre-echoes

Name is misleading: the basic coding artifact is noise spread out over some time even before the music event causing the noise

- roughness, double-speak

at low bit-rates and lower sampling frequencies there is a mismatch between time resolution of the coder and the requirements to follow the time structure of some signals. (e.g. robot like “double-speak”)



Further examples:

<https://www.soundonsound.com/techniques/what-data-compression-does-your-music>

# How to Measure Codec Quality

- Simple objective measurement techniques
  - Are **misleading**; these methods defy the whole purpose of perceptual coding
  - Examples of such measurements: signal-to-noise-ratio, bandwidth of decoded signal



# How to Measure Codec Quality

## Listening tests

- large scale and well-controlled listening tests are still the only method available
- the aim of these tests is to stress the encoders under worst case conditions
- During the ISO-MPEG process: a number of international listening tests with trained listeners.
- tests used the "triple stimulus, hidden reference" (ABC/HR) method and the "CCIR impairment scale"

# ABC/HR Test

- listening sequence is "ABC" , with A = original, BC = pair of original / coded signal with random sequence,
- listener must first distinguish the encoded version from the original (Hidden Reference, "HR" )
- than assign a score as a subjective judgment of the quality.

# CCIR Impairment Scale

- the listener has to evaluate both B and C with a number between 1.0 and 5.0.
- meaning of these values:
  - 5.0 = transparent (this should be the original signal)
  - 4.0 = perceptible, but not annoying (first differences noticable)
  - 3.0 = slightly annoying
  - 2.0 = annoying
  - 1.0 = very annoying

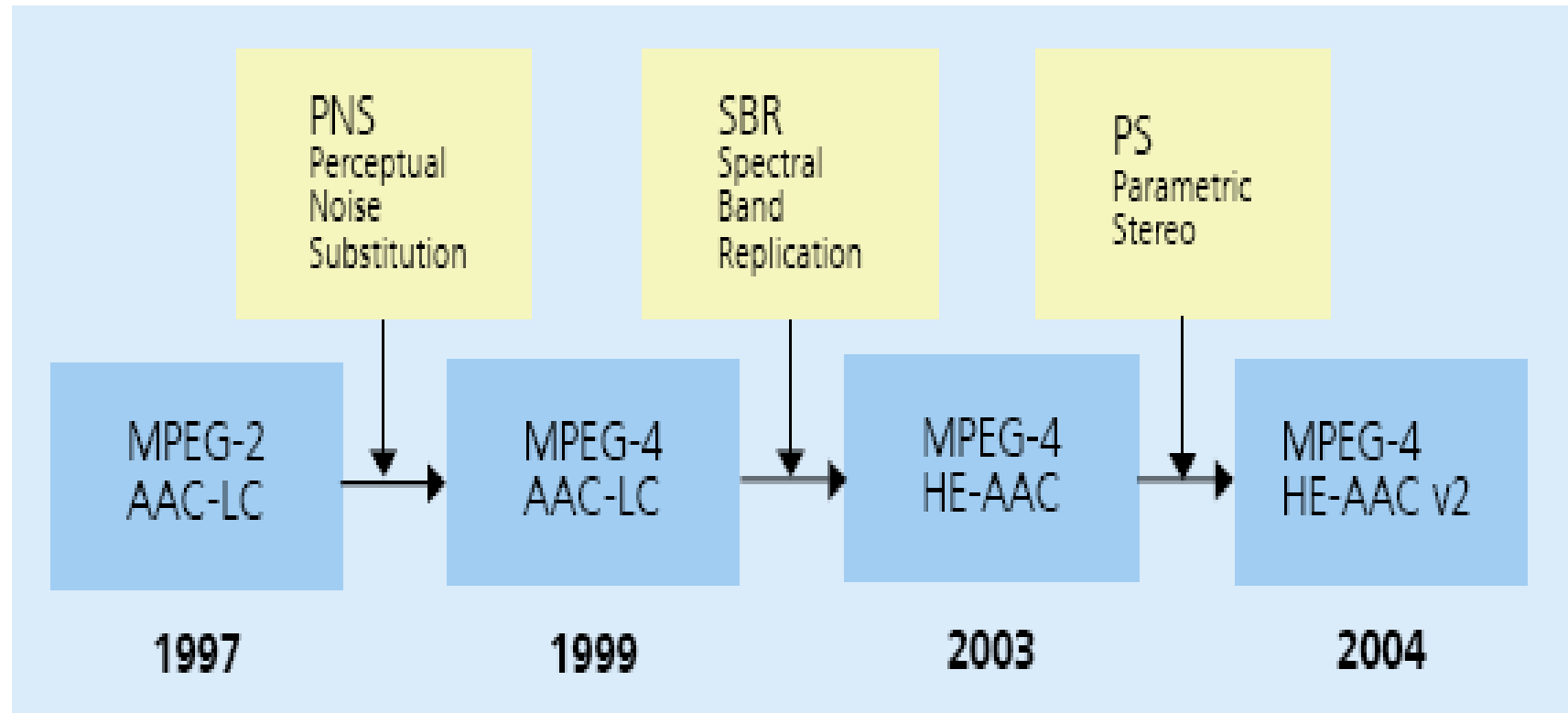
# How to Measure Codec Quality

- Perceptual Measurement Techniques (PMT)
  - state of the art not yet sufficient to make large scale and well-prepared listening tests obsolete
  - however, PMT have progressed to the point where they are a very useful supplement to listening tests and can replace them in some cases.
  - ITU-R Task Group 10/4 (Int. Telecommunications Union, Radiocommunications sector) produced a recommendation on a system called PEAQ (Perceptual Evaluation of Audio Quality)
    - a standardized algorithm for objectively measuring perceived audio quality
    - utilizes software to simulate perceptual properties of the human ear and then integrates multiple model output variables into a single metric. PEAQ characterizes the perceived audio quality as subjects would do in a listening test according to ITU-R BS.1116

# MPEG-2 Audio

- Original MPEG-2 Audio standard (1995) consists of 2 extensions to MPEG-1:
  - backwards compatible multichannel coding adds option of forward and backwards compatible coding of multichannel signals (including 5.1 channel configurations)
  - Coding at lower sampling frequencies of 16kHz, 22.05 kHz and 24 kHz
- Verification tests in 1994 showed that introducing new coding algorithms and giving up backwards compatibility promised a significant improvement in coding efficiency => AAC

# MPEG AAC FAMILY



# Advanced Audio Codec (AAC)

- successor of MP3 format
- Specified in MPEG-2 (Part 7) (1997) and MPEG-4 (Part 3) (1999)
- AAC generally achieves higher sound quality than MP3 at the same bit rate
- HE-AAC and HE-AAC v2 (in MPEG-4/Part 3)
  - extensions of the original AAC, but do not replace it
  - are optimized exclusively for low bit rates and are used, for example, for live broadcasts of digital radio and television.
  - At higher bit rates (from 96 kbit/s), they perform worse than AAC and should not be used



# AAC's Improvements over MP3 1/2

- More sample rates (from 8 to 96 kHz) than MP3 (16 to 48 kHz);
- Up to **48 channels** (MP3 supports up to two channels in MPEG-1 mode and up to 5.1 channels in MPEG-2 mode);
- Arbitrary bit rates and **variable frame length**. Standardized constant bit rate with bit reservoir;
- Higher efficiency and **simpler filter bank**. AAC uses a pure **MDCT** (modified discrete cosine transform), rather than MP3's hybrid coding (which was part MDCT and part FFT);
- Higher coding efficiency for stationary signals (AAC uses a blocksize of 1024 or 960 samples, allowing more efficient coding than MP3's 576 sample blocks);
- Higher coding accuracy for transient signals (AAC uses a blocksize of 128 or 120 samples, allowing more accurate coding than MP3's 192 sample blocks);
- Possibility to use Kaiser-Bessel derived window function to eliminate spectral leakage at the expense of widening the main lobe;

# AAC's Improvements over MP3 2/2

- Much better handling of **audio frequencies above 16 kHz**;
- More flexible joint stereo (different methods for different frequency ranges)
- Additional modules (tools) added to increase compression efficiency:  
**Temporal Noise Shaping** (TNS), backwards prediction, perceptual noise substitution (PNS), etc.
  - Modules can be combined to constitute different encoding profiles
- Overall, AAC format allows developers more flexibility to design codecs than MP3 does, and corrects many of the design choices made in the original MPEG-1 audio specification. This **increased flexibility** often leads to more concurrent encoding strategies and more efficient compression – especially at **very low bit rates** where superior stereo coding, pure MDCT, and better transform window sizes leave MP3 unable to compete.

# Advantages Over MP3

## AAC

1. Multi Channel Audio – up to 48 audio channels
2. Sample frequencies from 8KHz - 96KHz
3. Simpler filter bank (pure MDCT used)
4. Better stationary and transient response due to block sizes of 1024 and 128 samples
5. Excellent handling of high frequency signals
6. CD quality audio at 64Kbits/sec
7. Much better quality of audio at lower bit rates (down to 32Kbps)

## MP3

1. Stereo signal – maximum of only 2 channels
2. Sampling frequencies from 16KHz ~ 48KHz
3. Hybrid filter bank (requires more computational power)
4. Poorer stationary and transient response due to block sizes of 576 and 192 samples
5. Signal handling up to 15.5/15.8 KHz
6. CD quality audio at 128Kbits/sec
7. Audio quality is poorer at low bit rates and may present coding artifacts

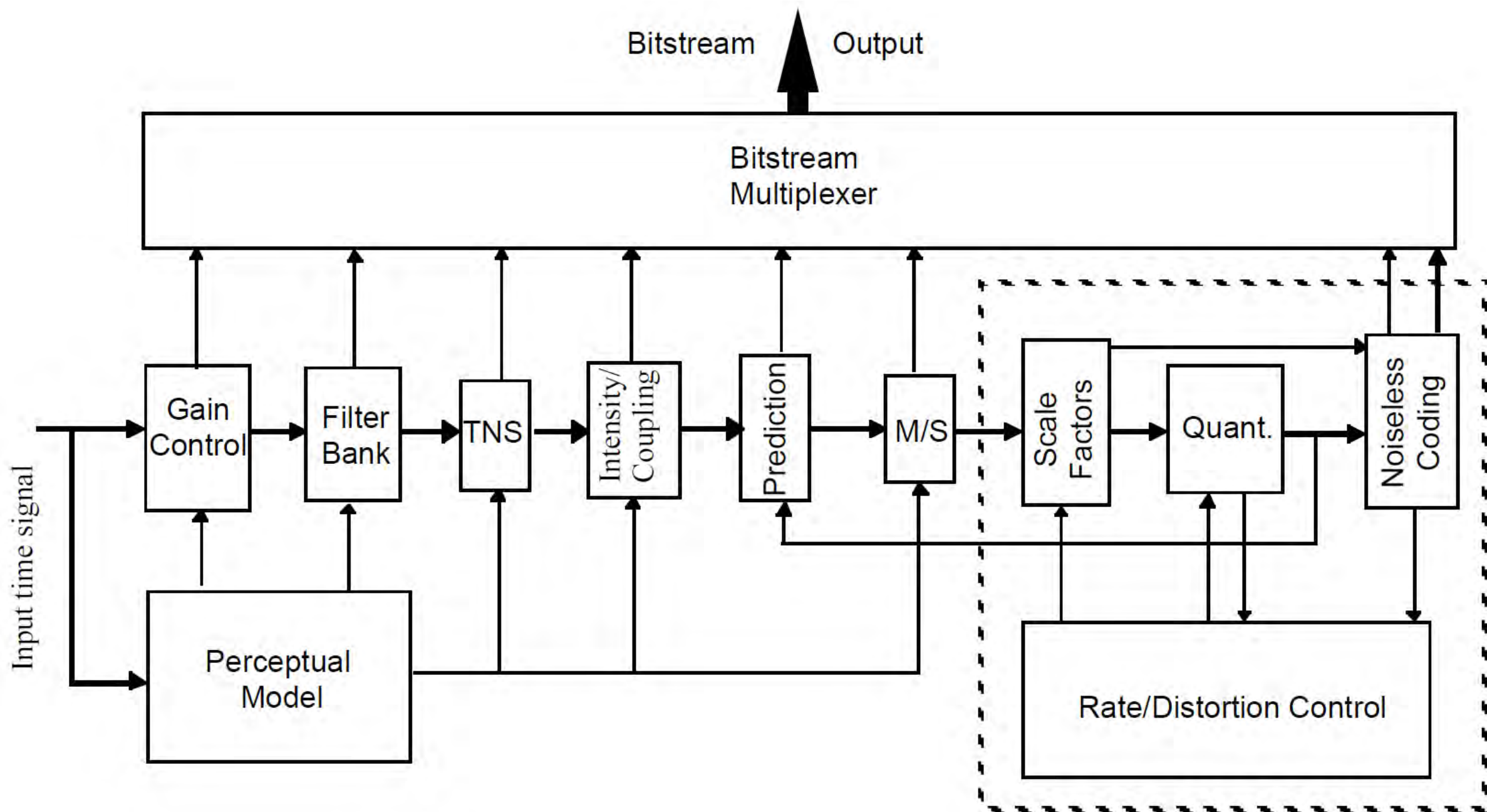


Figure 3: Block diagram of an MPEG-2 AAC encoder.

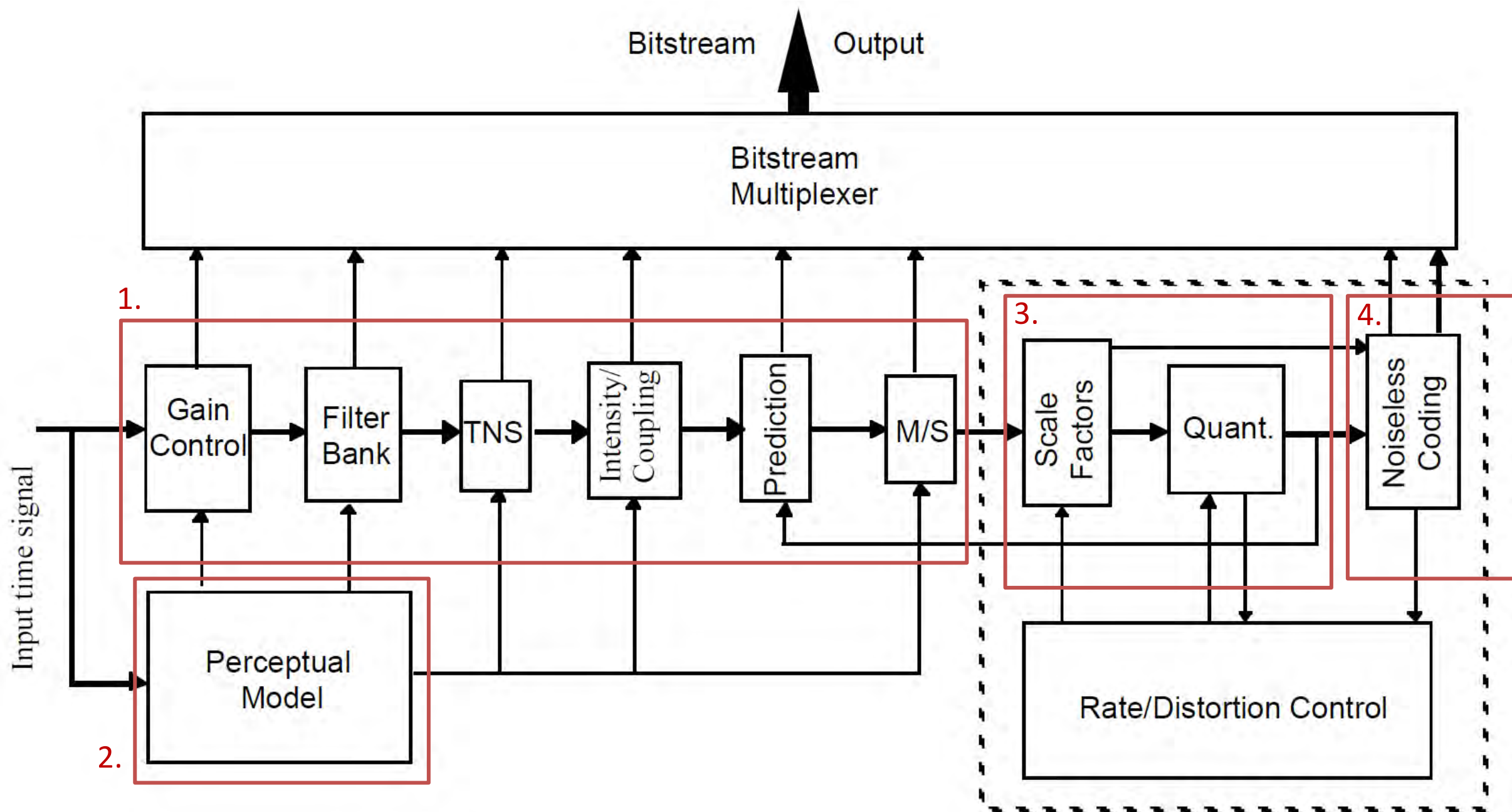
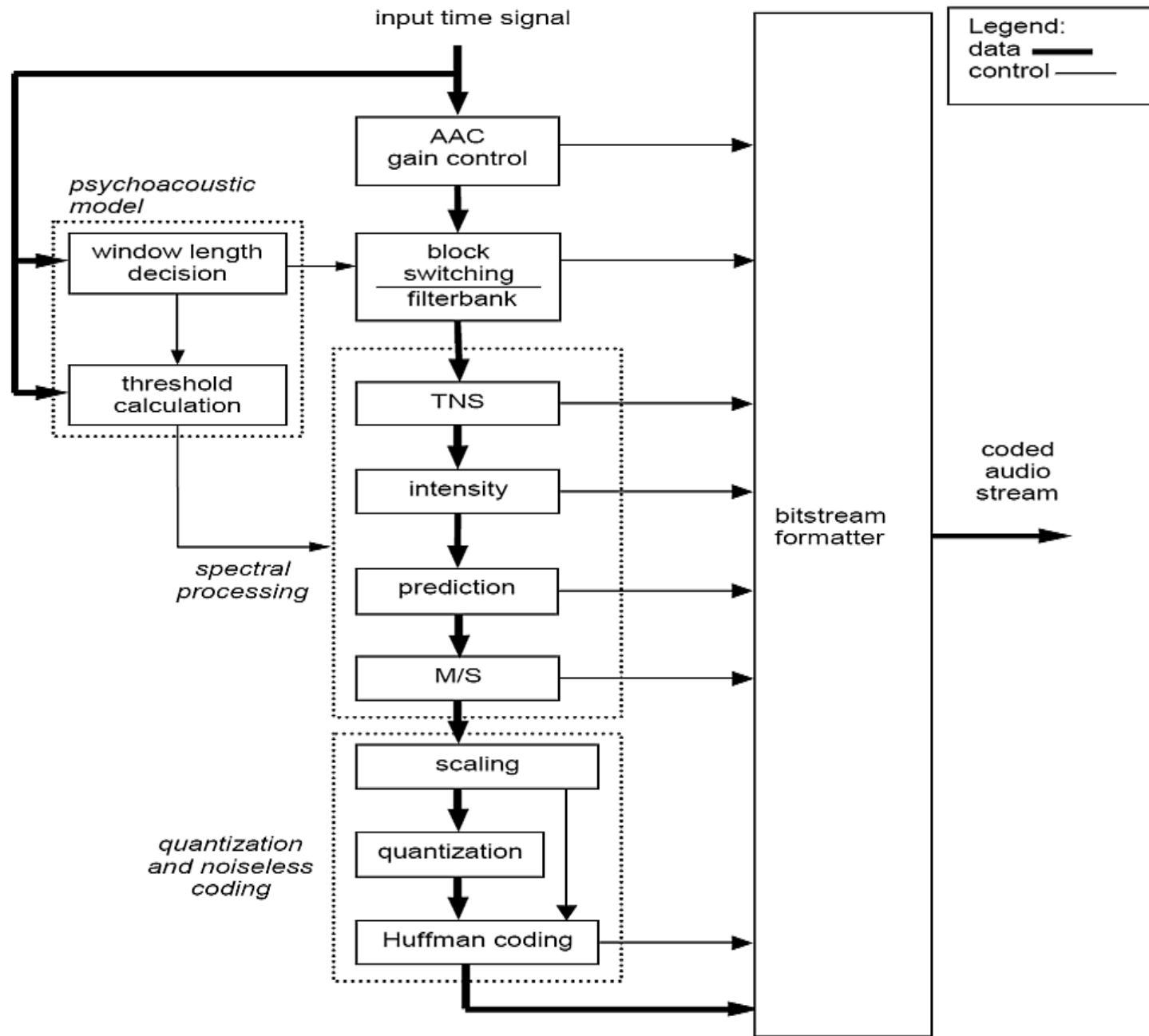
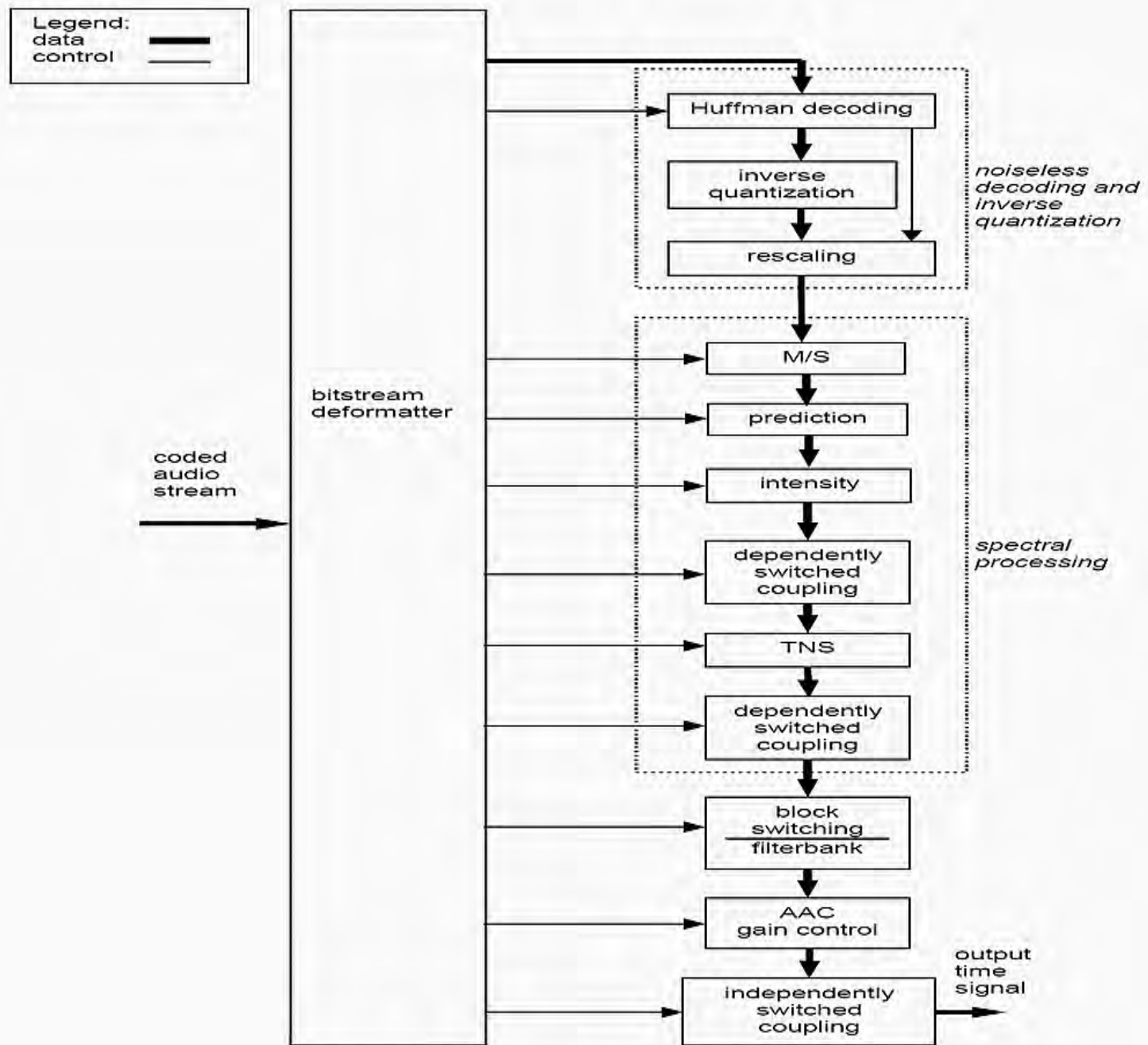


Figure 3: Block diagram of an MPEG-2 AAC encoder.



MPEG-2 AAC Encoder Block Diagram



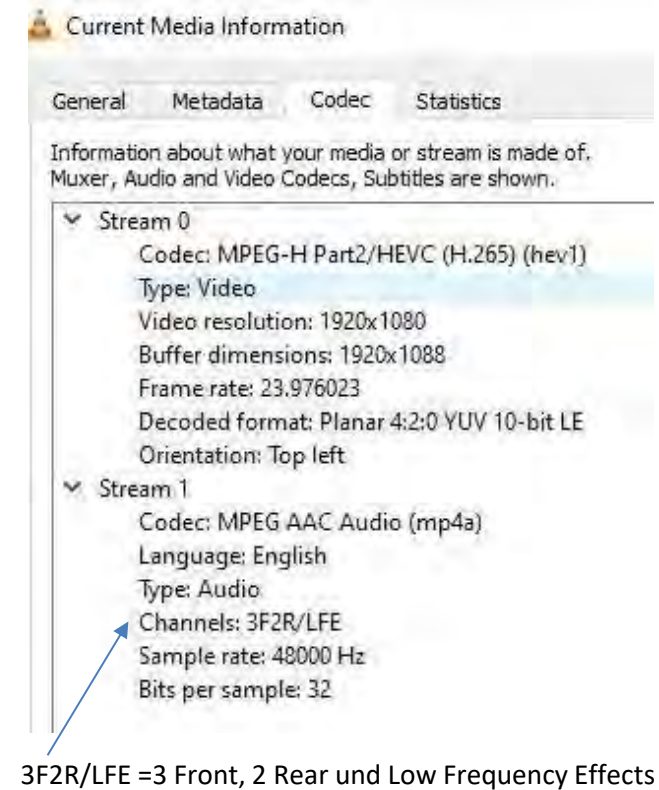


**MPEG-2 AAC Decoder Block Diagram**



# Dolby Digital 1/2

- Dolby Digital contains up to six discrete channels of sound. 5.1 mode has
  - five channels for normal-range speakers (20 Hz – 20,000 Hz) (right, center, left, right surround, left surround) and
  - one channel (20 Hz – 120 Hz allotted audio) for the subwoofer driven low-frequency effects.
  - Mono and stereo modes are also supported.
- Dolby **AC-3** (1991) multi-channel audio coding standard = *ATSC A/52* or *AC-3 Bitstream-Format (Adaptive Transform Coder 3)*
  - Basis is the modified discrete cosine transform (MDCT), a lossy audio compression algorithm. It is a modification of the discrete cosine transform (DCT) algorithm.
  - AC-3 supports audio sample-rates up to 48 kHz.
- Patents expired (since 2017), free to use



# Dolby Digital 2/2

Mögliche Kanalkonfigurationen bei Dolby Digital

| Name                | Modus                              | Vorne |                   |        | Hinten            |                    | Subwoofer<br>(LFE) | Verwendung                                                             |
|---------------------|------------------------------------|-------|-------------------|--------|-------------------|--------------------|--------------------|------------------------------------------------------------------------|
|                     |                                    | Links | Mitte<br>(Center) | Rechts | Surround<br>Links | Surround<br>Rechts |                    |                                                                        |
| Dolby Digital 1/0   | Mono                               | Nein  | Ja                | Nein   | Nein              | Nein               | Nein               | Vor allem alte restaurierte Filme.                                     |
| Dolby Digital 2/0   | Stereo                             | Ja    | Nein              | Ja     | Nein              | Nein               | Nein               | Audiokommentar. Kann auch Dolby Surround Informationen enthalten.      |
| Dolby Digital 3/0   | 3-Stereo                           | Ja    | Ja                | Ja     | Nein              | Nein               | Nein               |                                                                        |
| Dolby Digital 2/1   | Stereo mit Mono-Surround           | Ja    | Nein              | Ja     | Ja                |                    | Nein               |                                                                        |
| Dolby Digital 2/1.1 | Stereo mit Mono-Surround und LFE   | Ja    | Nein              | Ja     | Ja                |                    | Ja                 |                                                                        |
| Dolby Digital 3/1   | 3-Stereo mit Mono-Surround         | Ja    | Ja                | Ja     | Ja                |                    | Nein               |                                                                        |
| Dolby Digital 3/1.1 | 3-Stereo mit Mono-Surround und LFE | Ja    | Ja                | Ja     | Ja                |                    | Ja                 |                                                                        |
| Dolby Digital 2/2   | Quadrophonie                       | Ja    | Nein              | Ja     | Ja                | Ja                 | Nein               |                                                                        |
| Dolby Digital 2/2.1 | Quadrophonie mit LFE               | Ja    | Nein              | Ja     | Ja                | Ja                 | Ja                 |                                                                        |
| Dolby Digital 3/2   | 5-Kanal-Surround                   | Ja    | Ja                | Ja     | Ja                | Ja                 | Nein               |                                                                        |
| Dolby Digital 3/2.1 | 5-Kanal-Surround mit LFE           | Ja    | Ja                | Ja     | Ja                | Ja                 | Ja                 | Meistens als Haupttonspur bei Spielfilmen oder auch Serien eingesetzt. |



Extensions to 7.1 (2003) and 9.1 (2006) speaker configurations, resulting in Dolby Surround.

# Spatial Audio

# Binaural Audio

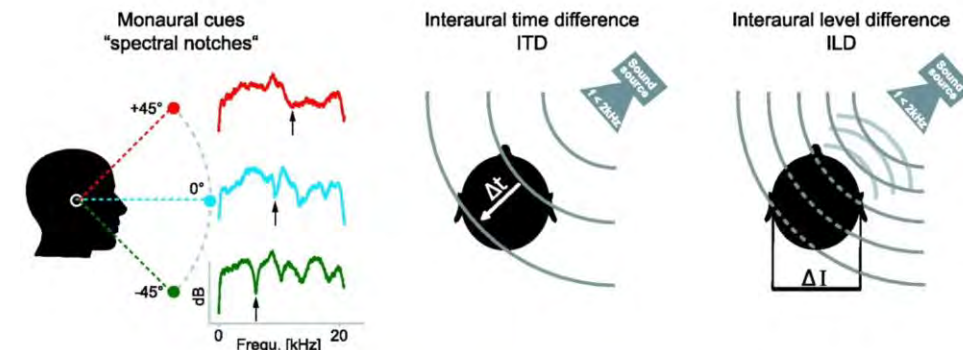
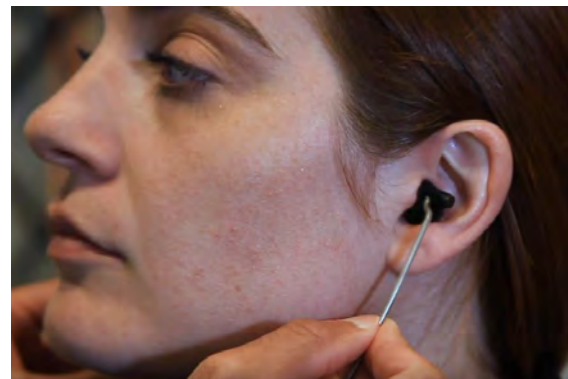
- Spatialize (=render) audio in 3D for headphones
  - more advanced processing than stereo
  - Sends individual signals to left/right ear
  - Uses psychoacoustics to spatialize sound



<https://www.youtube.com/watch?v=FMdabRXWwDE&list=PL7w4cOVVxL6FTg6i9osmlWK-w2wU4xvI2&index=5>

## Head Related Transfer Function (HRTF)

- characterizes how an ear receives a sound from a point in space
- Different for every individual

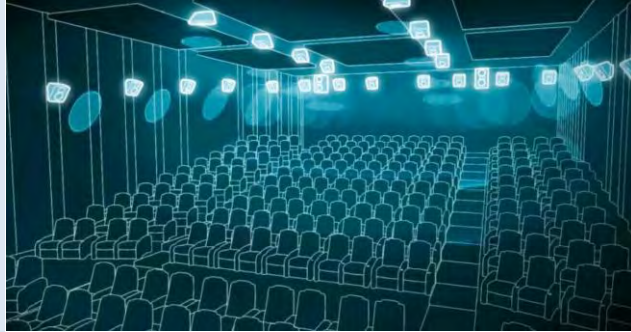


<https://www.youtube.com/watch?v=cB-IW9KKu4>

<http://physrev.physiology.org/content/90/3/983.long>

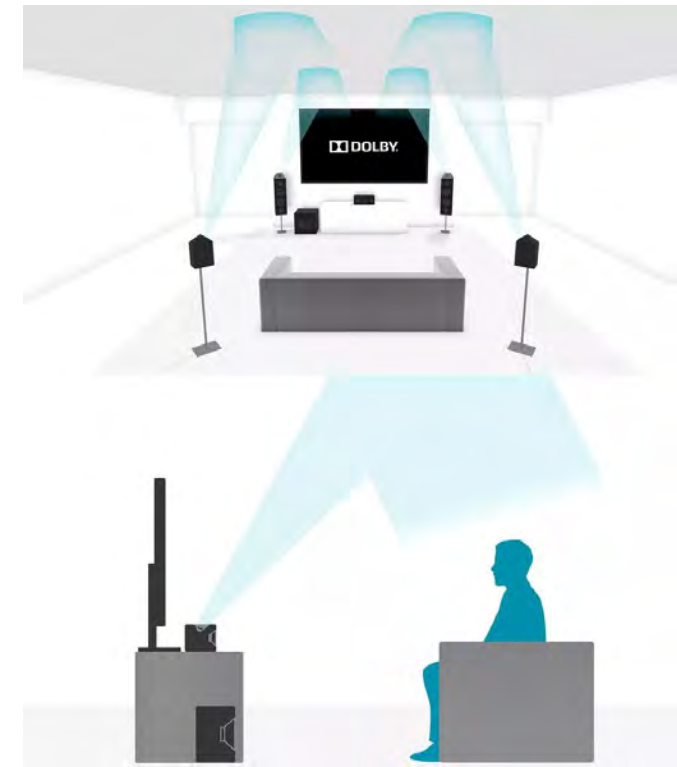
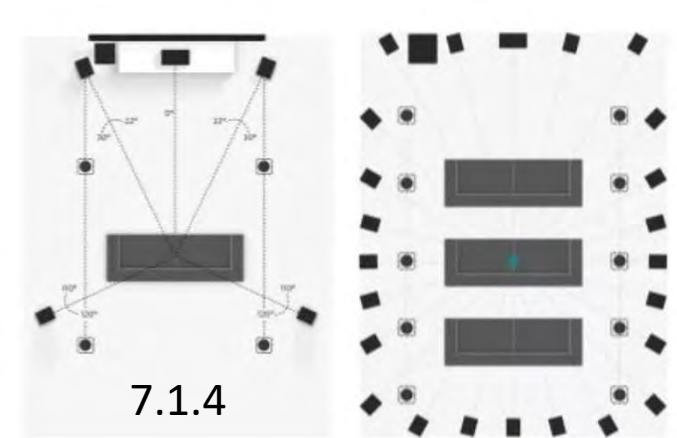
Example: Virtual Barber Shop  
<https://www.youtube.com/watch?v=IUDTlvagjJA>





# Dolby Atmos

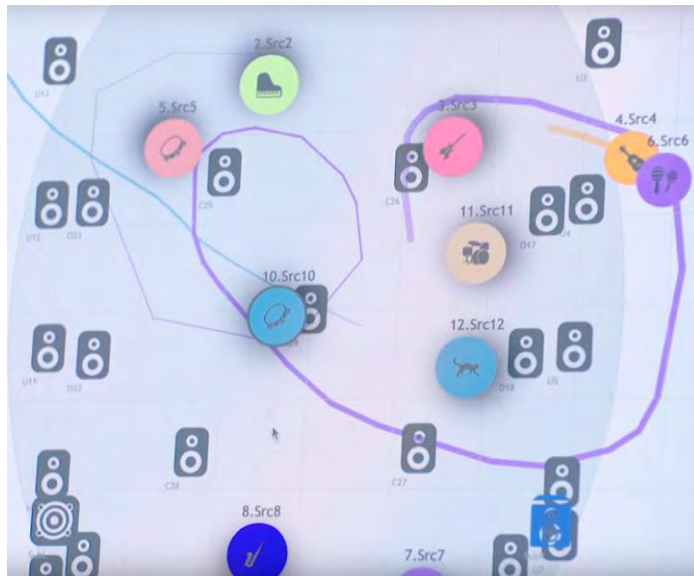
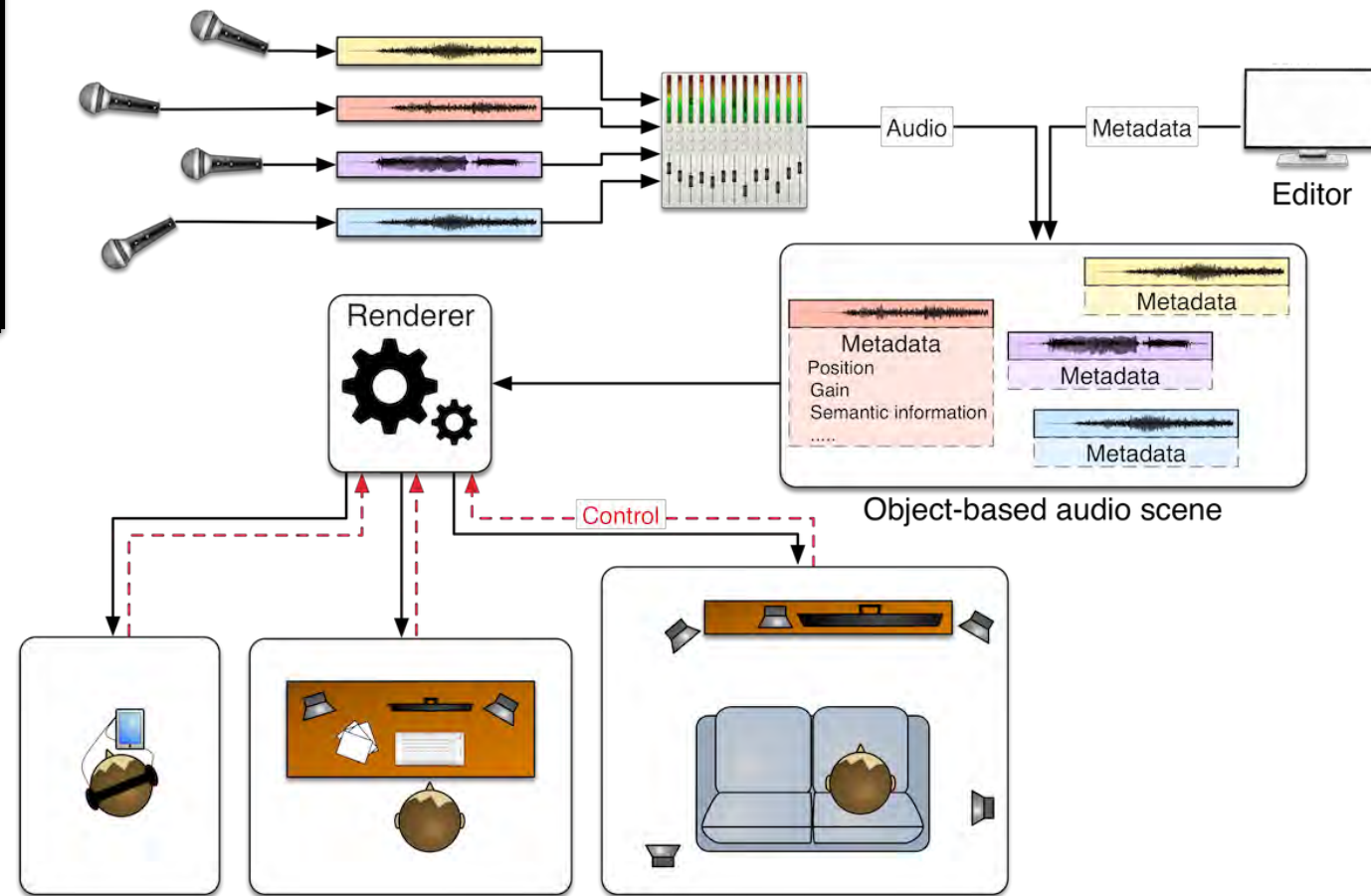
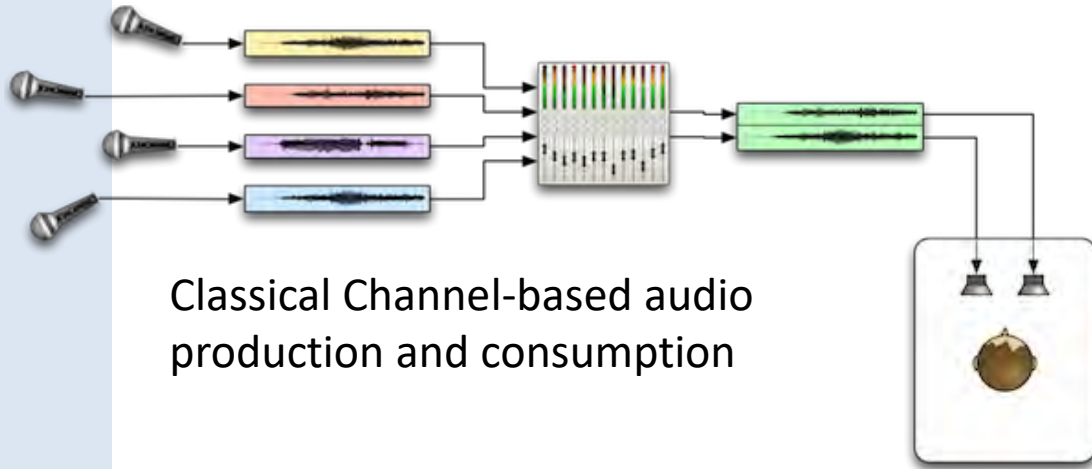
- Up to 128 **audio objects** (or channels)
- For home theaters typically 5.1 or 7.1 base channels with overhead or “Dolby Atmos speakers” (projecting to ceiling) e.g. 7.1.4 (7.1 + 4 overhead)
- Audio objects allow specification of location of sound – rendered in real time based on speaker positions
- Dolby-TrueHD-Codec – **lossless** – based on
  - Meridian Lossless Packing, also known as Packed PCM (PPCM) or Dolby Lossless for compressing PCM audio data
    - 1.5-3 : 1 compression rates (uses redundancy in multiple channels); incl. error correction up to 2ms
    - 44.1-192 kHz, up to 24 bits, 64 channels



# Audio Objects



<https://www.youtube.com/watch?v=Y79Q1sfvNmc>

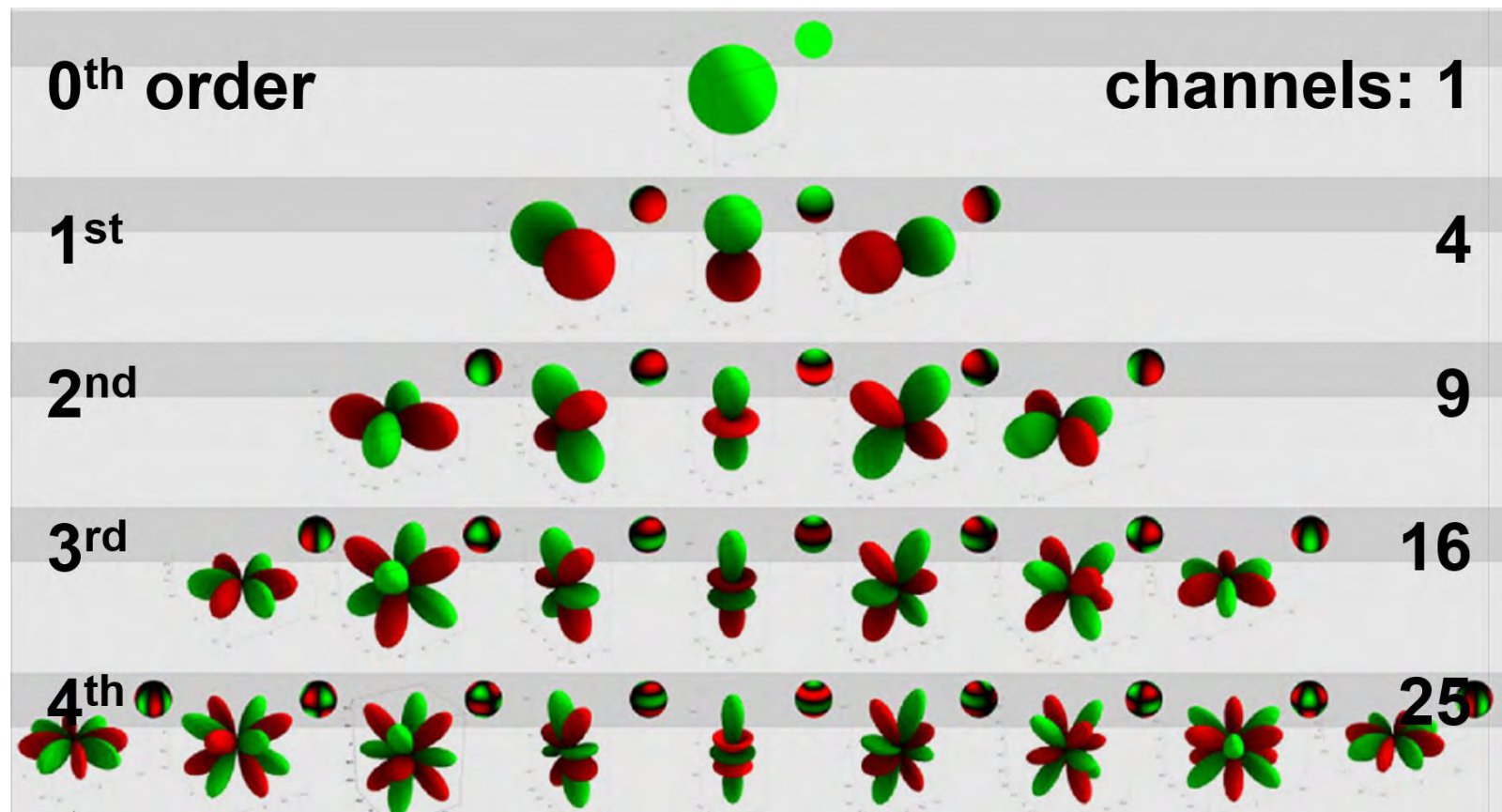


<https://www.youtube.com/watch?v=cqC1rWSxT4c>

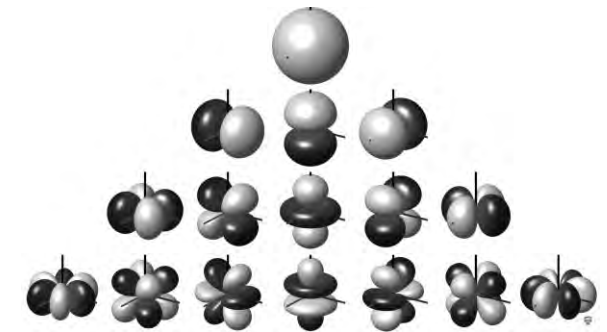


# Spatial Audio: Higher-Order Ambisonics

- Full-sphere surround sound format



3<sup>rd</sup> order Ambisonics microphone  
19 microphone omnidirectional  
digital MEMS capsules on a sphere



<https://www.youtube.com/watch?v=9m4VxFSYSEA>

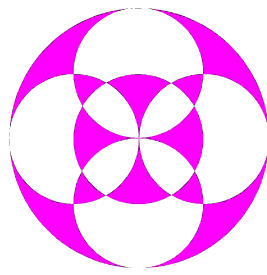


# Mono, Stereo, 3<sup>rd</sup> order Ambisonics compared

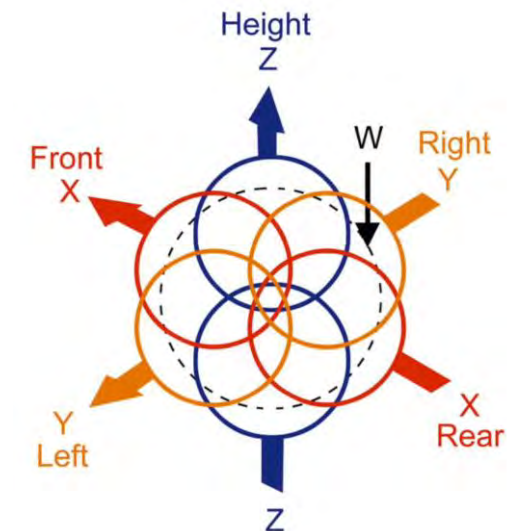
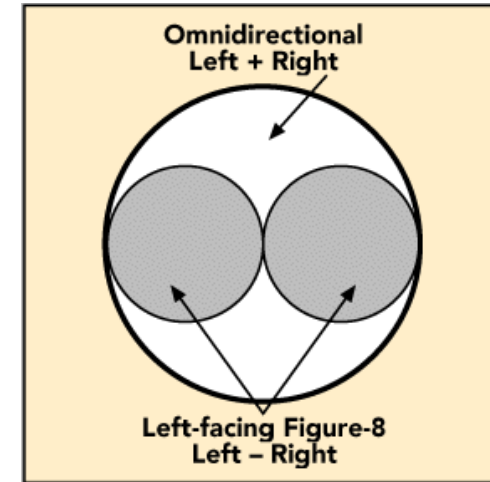


[https://www.youtube.com/watch?v=an9N4CKCO\\_o](https://www.youtube.com/watch?v=an9N4CKCO_o)

# Principles of Ambisonics



- Taking mid-side (M-S) coincident pair scheme into three dimensions
- **A-Format** Recording: All individual microphone channels
- **B-Format**: Processed individual channels e.g.
- 1<sup>st</sup> order Ambisonics **B-Format**:
  - $W = \text{Omni (F+B+L+R+U+D)}$
  - $X = \text{Front} - \text{Back}$
  - $Y = \text{Left} - \text{Right}$
  - $Z = \text{Up} - \text{Down}$
- These signals are symbolized in the Ambisonics logo
- 2<sup>nd</sup> order Ambisonics uses 9 channels
- 3<sup>rd</sup> order Ambisonics jumps up to 16 channels
- ... 6<sup>th</sup> order Ambisonics with 49 channels



# What Is Ambisonics?

- “Ambisonics” means “Surround Sound”
- Surround system developed by a team of British researchers in the 1970s, notably the late Michael Gerzon. All patents expired.
- Designed to bring the concert hall into the living room, by accurately capturing and replaying a soundfield on practical speaker arrays
- Four channels (B-Format) (in case of 1<sup>st</sup> order Ambisonics) capture entire soundfield – including height (“Periphony”)
- Transmission systems compatible with stereo (UJJ) and modern “discrete” surround (G-Format)
- Ambisonic production can use special microphone (Soundfield mic) or multitrack sources

# The Kirchhoff-Helmholtz Integral

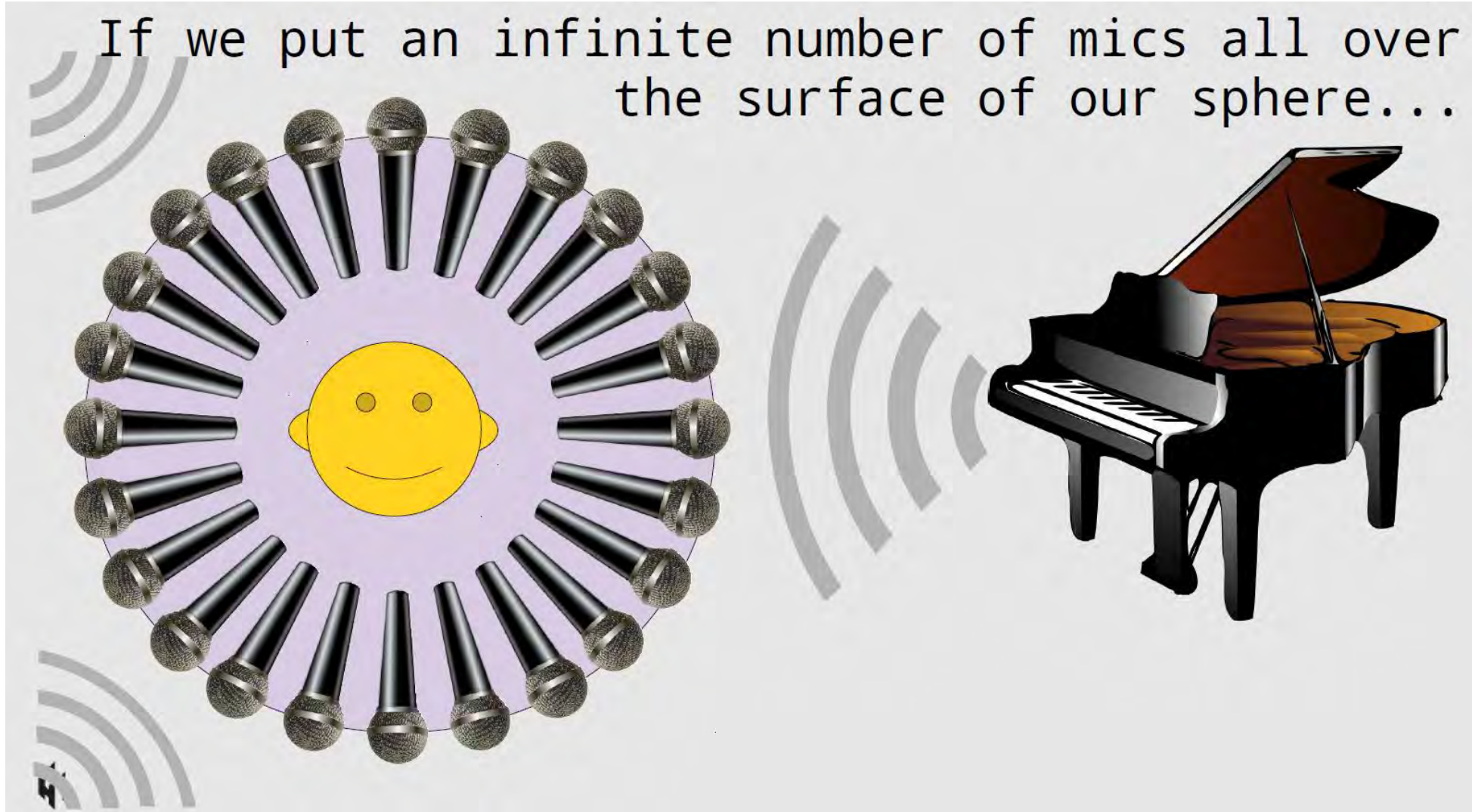
= If you can record the boundary,

"If you know the sound pressure and velocity in any point on the surface of a volume free of sources, you have complete knowledge of the sound field inside."

you can reproduce the inside.

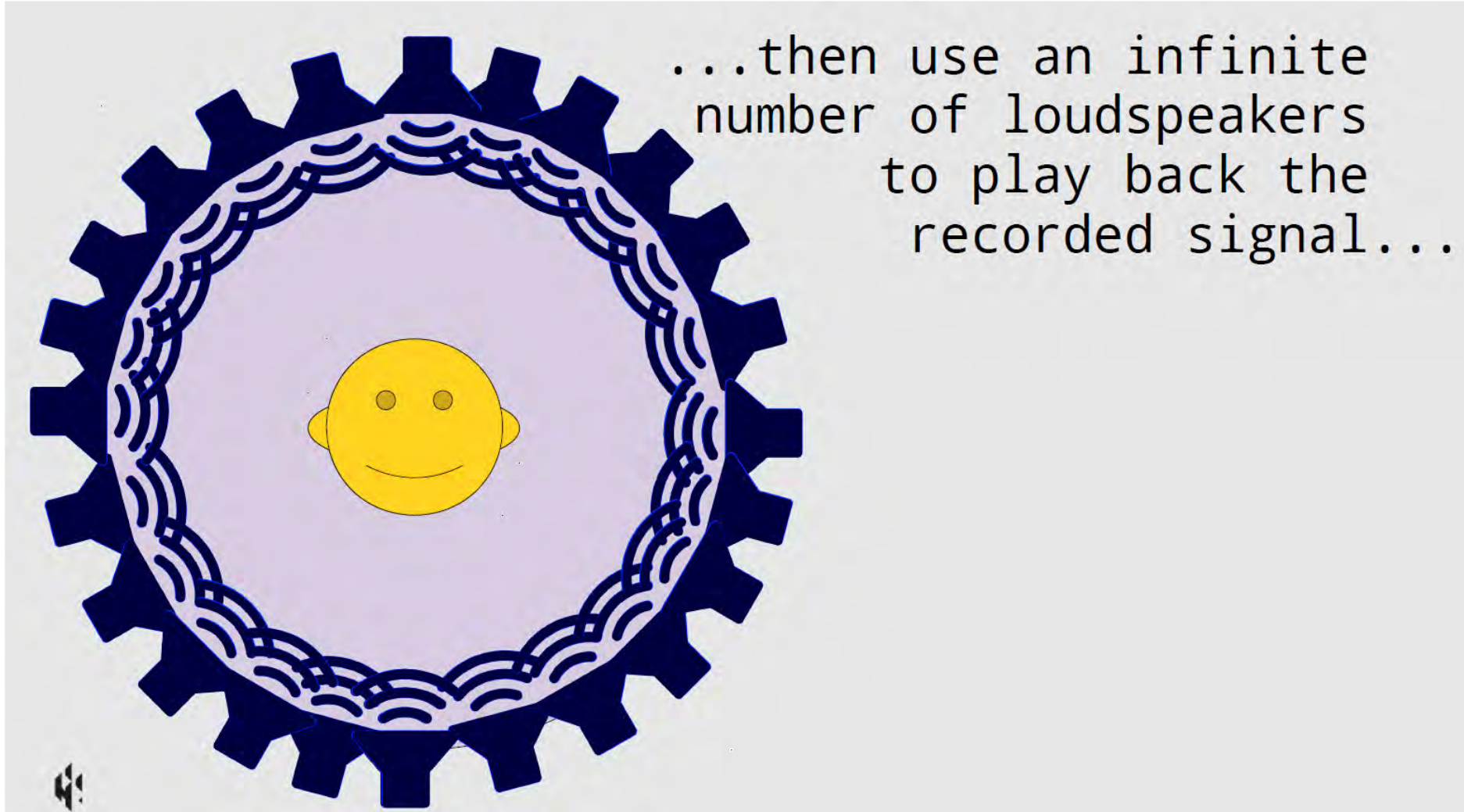
# The Kirchhoff-Helmholtz Integral

If we put an infinite number of mics all over the surface of our sphere...



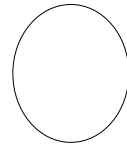
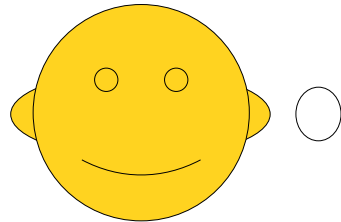


# The Kirchhoff-Helmholtz Integral



# The Kirchhoff-Helmholtz Integral

...the result will be  
*identical* to the  
original performance.





# Sampling the Sphere

The systematic approach:

Find a set of microphones that cover the sphere completely.

Avoid redundant information.

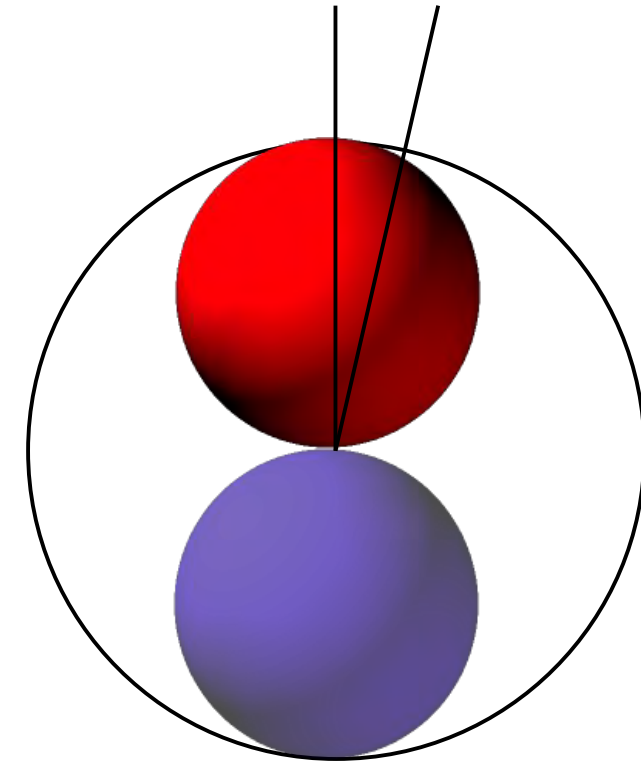
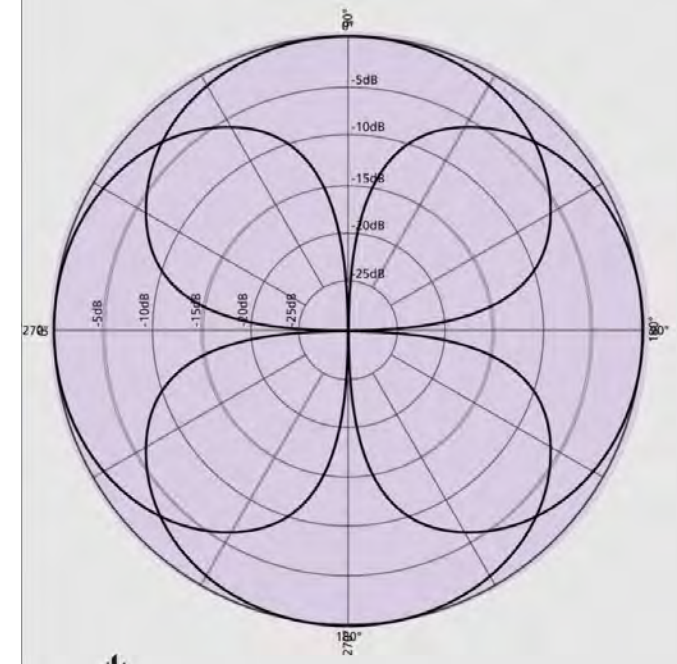
- Omnidirectional microphone = 0<sup>th</sup> order component
- **W** is the pressure component.  
Remains when Ambisonics is played back in mono.



by Jörn Nettingsmeier, <http://stackingdwarves.net>

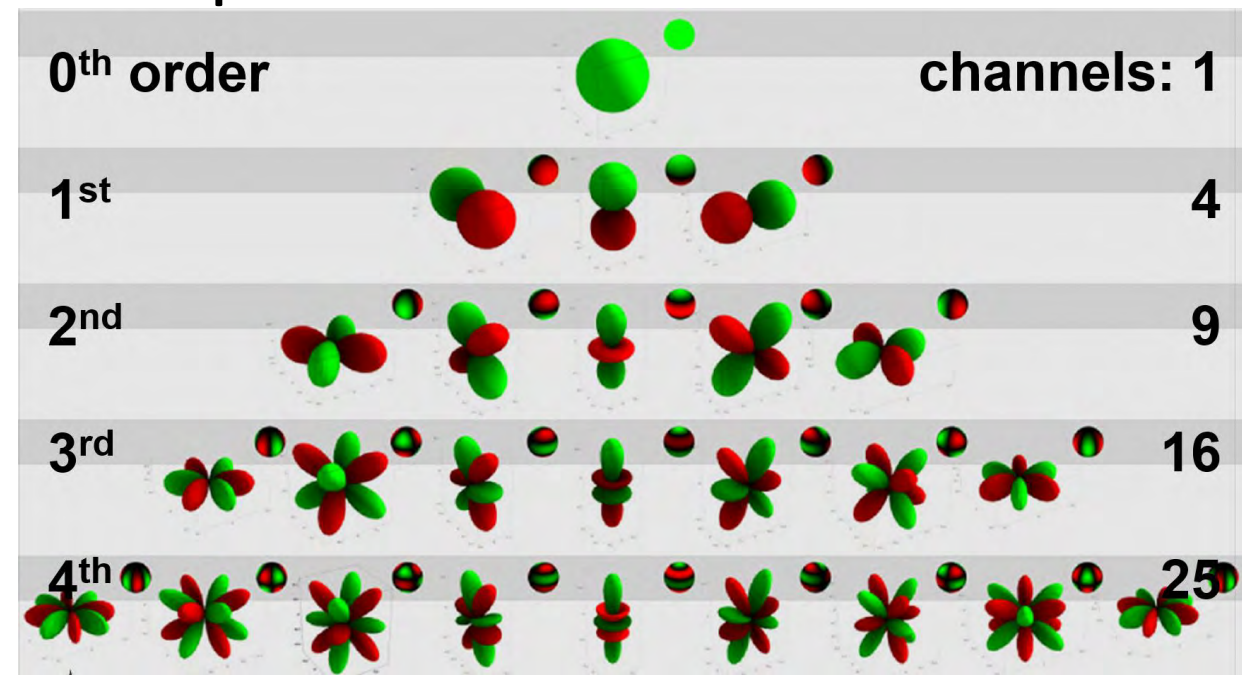
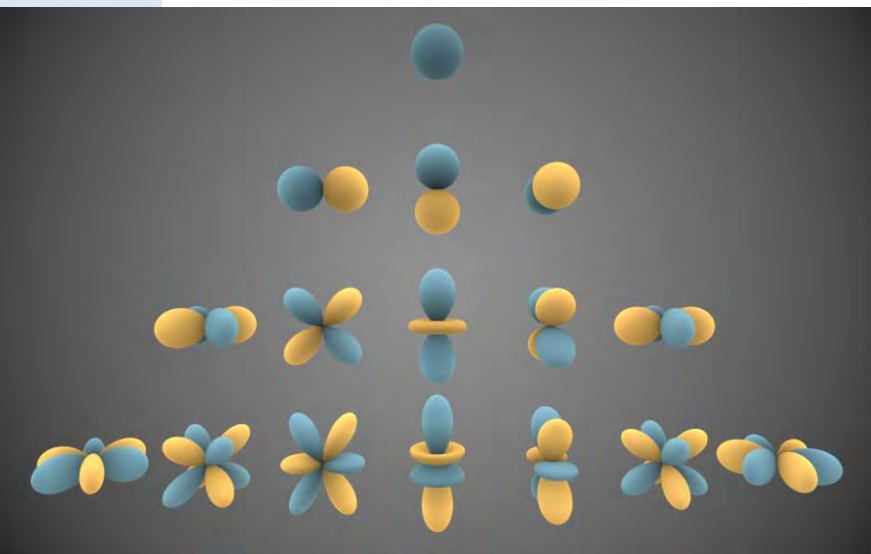
# 1<sup>st</sup> order Ambisonics

- Three figures-of-eight will also cover the sphere uniformly.
- Together with W, they provide directional information.
- In the B-format, they are denoted as X,Y,Z
- What's wrong with 1<sup>st</sup> order Ambisonics?
  - Low angular resolution:  
off-axis sources have a shallow roll-off.
  - In practice, this means a small sweet spot  
(= listening area)
- So we need narrower polar patterns which also sample the sphere uniformly and linearly independent



# Spherical Harmonics: Higher-order Ambisonics

- **Spherical harmonics** are a set of functions used to represent functions on the surface of the **sphere**.
- They are a higher-dimensional analogy of Fourier series, which form a complete basis for the set of periodic functions of a single variable.



# Ambisonics Defined

“**Ambisonics** goes further than stereo in that what it does... is to sample the acoustic field in such a way that the combination of the signals from all the loudspeakers in the array produces... a reconstruction of the original acoustic wave field (both traveling and standing wave components).

“If a listener puts their head in the sound field, because the wavefronts are similar to the original, the perception of directionality and space should correspond to the original too. It is a ‘wavefront reconstruction’ scheme in the small. The ear signals (crosstalk and all) will be correct if the reconstructed wavefronts are correct. This is just like natural hearing.

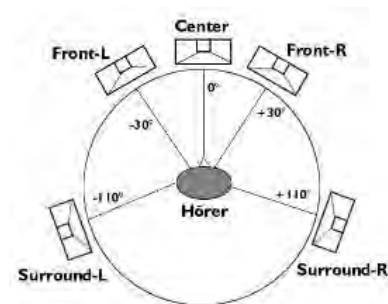
“Increasing the number of loudspeakers in Ambisonics (each fed its correctly-decoded signal) increases the accuracy of the reconstruction and the region over which it holds up. The interaural arrival-time differences also correspond to natural hearing in Ambisonics.”

—*Stanley Lipshitz*

# 5.1 and Ambisonics - Compared

## 5.1 surround

- Localization achieved solely with level
- Poor inter-speaker imaging except at front
- Localization varies with listener position
- Small sweet spot
- Horizontal surround uses six channels
- Speakers in special positions (e.g. ITU)
- One-to-one channel relationship from studio to speakers



ITU-Circle

## Ambisonics

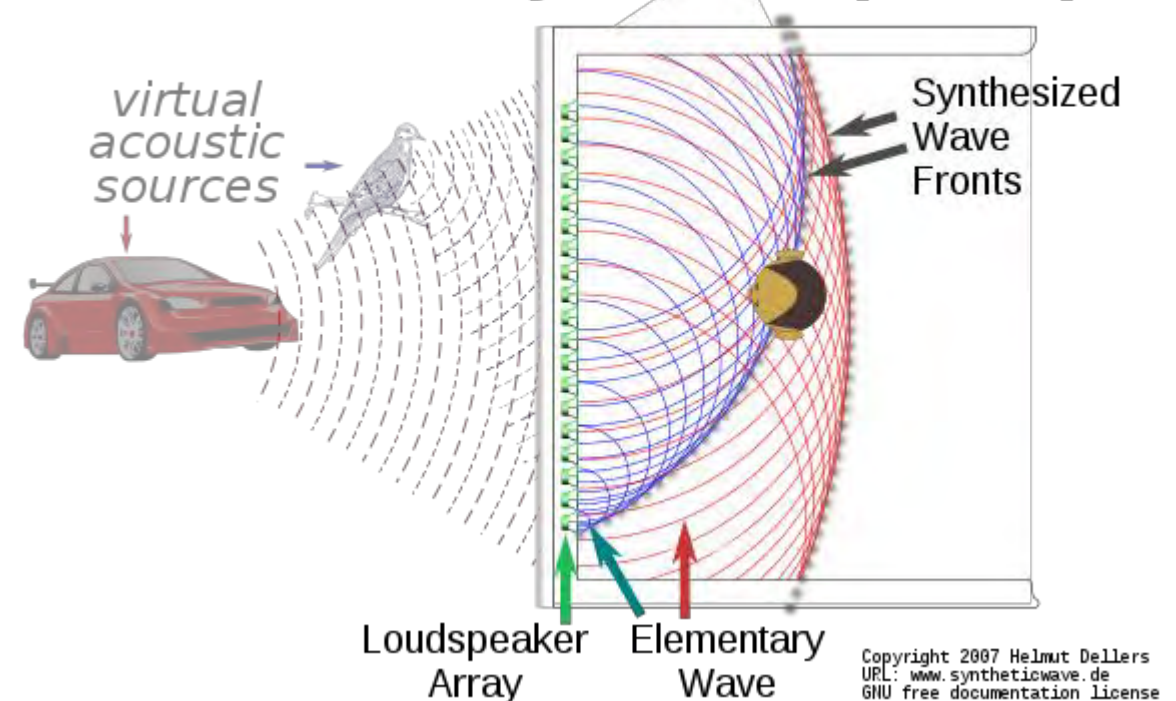
- Localization includes other mechanisms
- Good inter-speaker imaging all round
- Less variation of image with position
- Larger sweet spot
- Even with height, only four channels needed (1<sup>st</sup> order)
- Decode to 5.1 or other configurations
- Recover B-Format and decode for completely flexible speaker arrays



# Spatial Audio: Wave Field Synthesis (WFS)

- A spatial audio rendering technique, characterized by creation of virtual acoustic environments
- Produces artificial wavefronts synthesized by a large number of individually driven loudspeakers.
- The localization of virtual sources in WFS does not depend on or change with the listener's position

## Wave Field Synthesis principle

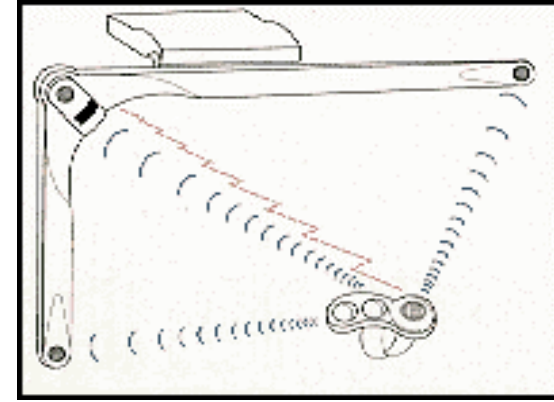


# MPEG-H 3D Audio (2013 - )

- Supports coding audio as **audio channels**, **audio objects**, or **higher order ambisonics** (HOA)
- Can support up to **64 loudspeaker channels** and **128 codec core channels**
- Audio objects may be used alone or in combination with channels or HOA components
- Use of audio objects allows **interactivity** or personalization of a program by adjusting gain or position of the objects during rendering in the decoder
- Audio is encoded using an improved modified discrete cosine transform (MDCT) algorithm
- Transmit immersive sound as well as mono, stereo, or surround sound.
- MPEG-H 3D Audio decoder renders bitstream to a number of standard speaker configurations as well as to misplaced speakers.
- **Binaural rendering** of sound for headphone listening is also supported
- Used since May 2017 in terrestrial South Korean UHD TV broadcast



# Audio Processing Examples

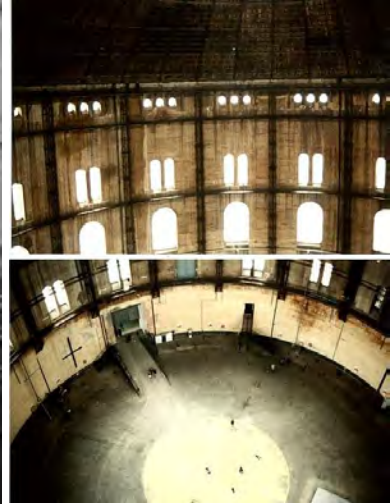


- reshape impulse response to simulate a different room
- change perceived location from which sound comes
- locate speaker in 3D space using microphone arrays
- acoustic tracking
- cover missing samples
- echo cancellation
- Spatial audio (multi-speaker setups)
  - Audio objects
  - High number of channels





# In Retrospect: Importance of Space “Delay Maschine” Gasometer





# Multimedia

## Vorlesung

Hannes Kaufmann

Research Unit Virtual and Augmented Reality  
Institute of Visual Computing & Human-Centered Technology

Based on material by Prof. Christian Breiteneder

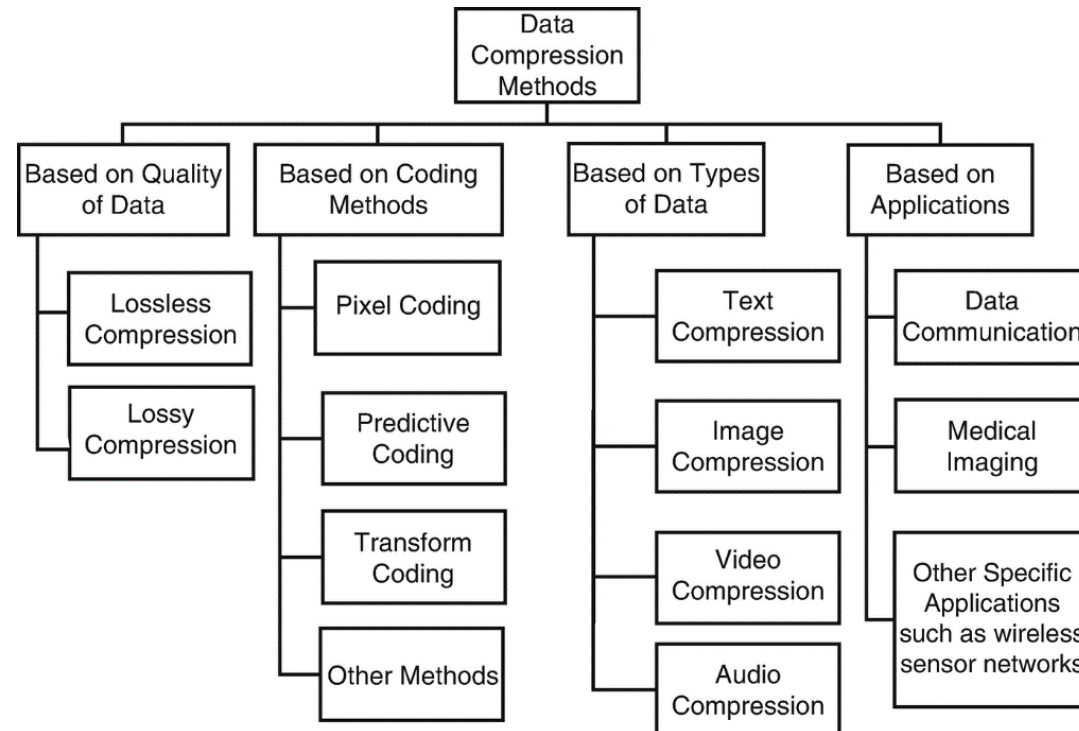
# Multimedia Compression Basics

# Codec

- A codec is a device or computer program which encodes or decodes a digital data stream or signal
- Typically a pair of algorithms for encoding and decoding
- Codec is a blend of the words en**code**r – **de**code
- An audio codec converts analog audio signals into digital signals for transmission or encodes them for storage. A receiving device converts the digital signals back to analog form using an audio decoder for playback.
- A codec may also **compress** the data to reduce transmission bandwidth or storage space

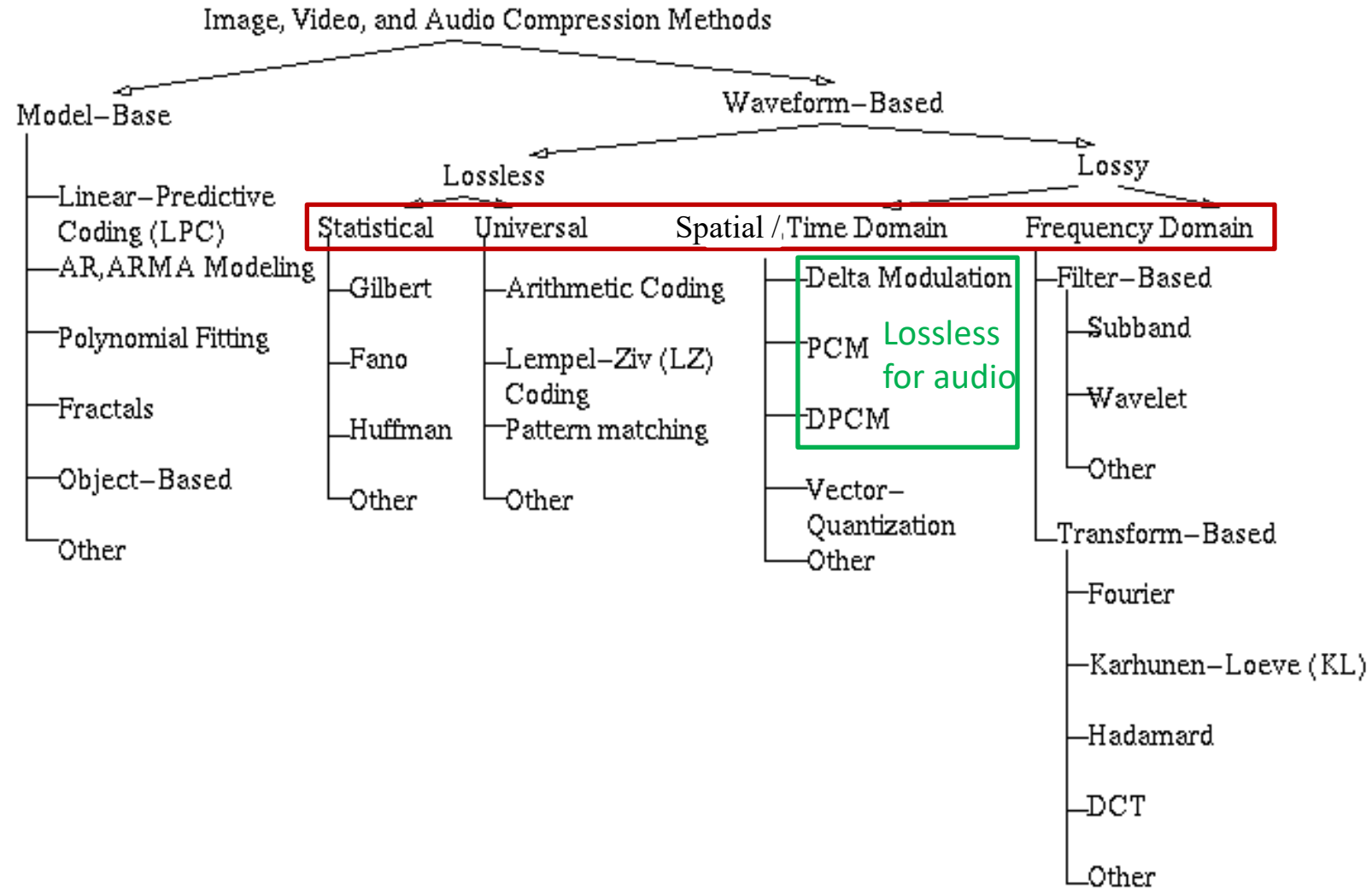
# Compression Classification

- **Lossless** Compression
  - decompressed data is identical to original data.
- **Lossy** compression (irreversible compression)
  - uses inexact approximations and partial data discarding to represent the content





# Compression Taxonomy



# Audio Codecs

## CODECS

### VIDEO

H.264 (AVC)  
H.265 (HEVC)  
VP9  
AV1

### AUDIO

MP3  
AAC

## AAC

Advanced Audio Coding



**Widely supported**



**More efficient than MP3**



**Limit on audio channels**

# Motion Picture Experts Group (MPEG)

- General Information about MPEG
- MPEG Objectives and Standards
- Audio Codecs:
  - MPEG-1 / MPEG-2 Audio Layer 3 (MP3)
  - MPEG-2 / MPEG-4 Advanced Audio Coding (AAC)

# Motion Picture Experts Group (MPEG)

About 350 members from 200 industries, universities, and research institutions from 20 countries

ISO/IEC JTC 1/SC 29 **Coding of audio, picture, multimedia and hypermedia information** (since 1991) is a standardization subcommittee (7 working groups, 3 advisory groups)

- SC 29/AG 2: MPEG Technical Coordination
- SC 29/AG 3: MPEG Liaison and Communication
- SC 29/AG 5: MPEG Visual Quality Assessment
- SC 29/WG 2: MPEG Technical requirements
- SC 29/WG 3: MPEG Systems
- SC 29/WG 4: MPEG Video Coding
- SC 29/WG 5: MPEG Joint Video Coding Team(s) with ITU-T SG 16
- SC 29/WG 6: MPEG Audio coding
- SC 29/WG 7: MPEG 3D Graphics coding
- SC 29/WG 8: MPEG Genomic coding

# MPEG Objectives

- Deliver acceptable video quality at compressed data rates (for MPEG-1 between 1.0 and 1.5 Mbps)
- Support either **symmetric** or **asymmetric** compress / decompress applications
  - symmetric = compression/decompression takes same time
- If compression takes it into account, **random-access** playback is possible
- If compression takes it into account, fast-forward, fast-reverse or normal reverse playback modes are available
- Audio/video **synchronization** will be maintained

# MPEG Objectives

- Catastrophic behavior in the presence of data errors should be avoidable
- If required, compression/decompression delay can be controlled
- Editability should be available if required by applications
- Processing requirements should allow the development of low-cost chipsets which are capable of encoding in real-time

# MPEG Standards

- Standard specifies audio, video and system layers
- Many standards defined
- MPEG-1 targeted at low data rates  
(VHS quality at 1.5 Mbits/ sec) [1992]
- MPEG-2 targeted higher quality, hence higher data rates  
(studio quality up to 15 Mbits/sec) [1994]
- MPEG-4 targeted at very low bit rates (<64 kbs) with small images [1998]



| Acronym for a group of standards ⇄ | Title ⇄                                                                                                                                                                                              | ISO/IEC standards ⇄           | First public release date (First edition) ⇄ |
|------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|---------------------------------------------|
| MPEG-1                             | Coding of moving pictures and associated audio for digital storage media. Commonly limited to about 1.5 Mbit/s although specification is capable of much higher bit rates <b>Audio Layer 3 = MP3</b> | ISO/IEC 11172                 | 1993                                        |
| MPEG-2                             | Generic coding of moving pictures and associated audio information <b>Audio Layer 3 = MP3</b>                                                                                                        | ISO/IEC 13818                 | 1995                                        |
| MPEG-3                             |                                                                                                                                                                                                      |                               |                                             |
| MPEG-4                             | Coding of audio-visual objects <b>H.264 = MPEG-4/Part 10 = MPEG-4 AVC</b>                                                                                                                            | ISO/IEC 14496                 | 1999                                        |
| MPEG-7                             | Multimedia content description interface <b>AAC = MPEG-4 Part 3, MPEG-4 Audio or ISO/IEC 14496-3:1999</b>                                                                                            | ISO/IEC 15938                 | 2002                                        |
| MPEG-21                            | Multimedia framework (MPEG-21)                                                                                                                                                                       | ISO/IEC 21000                 | 2001                                        |
| MPEG-A                             | Multimedia application format (MPEG-A)                                                                                                                                                               | ISO/IEC 23000                 | 2007                                        |
| MPEG-B                             | MPEG systems technologies                                                                                                                                                                            | ISO/IEC 23001                 | 2006                                        |
| MPEG-C                             | MPEG video technologies                                                                                                                                                                              | ISO/IEC 23002                 | 2006                                        |
| MPEG-D                             | MPEG audio technologies                                                                                                                                                                              | ISO/IEC 23003                 | 2007                                        |
| MPEG-E                             | Multimedia Middleware                                                                                                                                                                                | ISO/IEC 23004                 | 2007                                        |
| MPEG-G                             | Genomic Information Representation                                                                                                                                                                   | ISO/IEC 23092                 | 2019                                        |
| (none)                             | Supplemental media technologies                                                                                                                                                                      | ISO/IEC 29116                 | 2008                                        |
| MPEG-V                             | Media context and control                                                                                                                                                                            | ISO/IEC 23005 <sup>[42]</sup> | 2011                                        |
| MPEG-M                             | MPEG extensible middleware (MXM)                                                                                                                                                                     | ISO/IEC 23006 <sup>[47]</sup> | 2010                                        |
| MPEG-U                             | Rich media user interfaces                                                                                                                                                                           | ISO/IEC 23007 <sup>[49]</sup> | 2010                                        |
| MPEG-H                             | High Efficiency Coding and Media Delivery in Heterogeneous Environments <b>H.265 = High Efficiency Video Coding (HEVC), MPEG-H Part 2</b>                                                            | ISO/IEC 23008 <sup>[54]</sup> | 2013                                        |
| MPEG-DASH                          | Information technology – DASH <b>Dynamic adaptive streaming over HTTP (DASH)</b>                                                                                                                     | ISO/IEC 23009                 | 2012                                        |
| MPEG-I                             | Coded Representation of Immersive Media <b>H.266 = Versatile Video Coding (VVC), ISO/IEC 23090-3, MPEG-I Part 3, Future Video Coding (FVC)</b>                                                       | ISO/IEC 23090                 | 2020                                        |

MPEG-H 3D Audio, Audio Objects

# MPEG General Information

- MPEG standard defines audio, video coding and system data streams with synchronization
- MPEG considers explicitly functionalities of other standards, e.g. it uses JPEG
- MPEG stream provides information on
  - aspect ratio (e.g., 1:1, 4:3, 16:9)
  - refresh frequencies (8 frequencies) encoded:  
23.976 Hz, 24 Hz, 25 Hz, 29.97 Hz, 30 Hz, 50 Hz, 59.94 Hz and 60 Hz

▼ Stream 0

- Codec: H264 - MPEG-4 AVC (part 10) (avc1)
- Type: Video
- Video resolution: 1280x720
- Buffer dimensions: 1280x720
- Frame rate: 23.976024
- Decoded format:
- Orientation: Top left
- Chroma location: Left

▼ Stream 1

- Codec: MPEG AAC Audio (mp4a)
- Type: Audio
- Channels: Stereo
- Sample rate: 48000 Hz
- Bits per sample: 32

# MPEG-1 General Information

- Audio compressor uses subband coder with psychoacoustic model to reduce bit rate
- Video compressor uses block transform coder with motion- compensated inter-frame coding
- Audio and video compression designed to operate at CD-ROM speeds (video—1.150 Mbits/sec, audio—0.256 Mbits/sec, system 0.094 Mbits/sec)

# MPEG-1 Audio Standard

- Precision: 16 Bits, sampling frequencies 32kHz, 44.1kHz, 48kHz
- Audio coding: single channel, 2 independent channels, 1 stereo channel
- 3 compression methods: Layer 1, Layer 2, Layer 3 (hierarchical decoding)
  - Layer 3: decoder is accepting Layer 2 and Layer 1 (32kbps- 320kbps, target 64 kbps)
  - Layer 2: decoder is accepting Layer 1 (32 kbps-384kbps, target 128 kbps)
  - Layer 1: 32 kbps-448kbps, target 192 kbps

# MPEG Audio

- *MPEG-1/2 Layer-3 is called MP3*
- Exponential growth of MPEG coded audio material on the internet since 1995
- “mp3” the most searched for term in early 1999
- Everybody is using MP3, not many know details of MPEG audio coding
- Compared to CD-quality digital audio, **MP3 compression can commonly achieve a 75 to 95% reduction in size**. For example, an MP3 encoded at a constant bitrate of 128 kbit/s would result in a file **approximately 9% of the size of the original CD audio**.

# Why MPEG-1/2 Layer-3?

- Open standard
  - Specified in 1993
  - specification available to everybody (patents expired 2017)
  - no single company “owns” the standard
  - public example source code available to implementers
  - format well defined
- Availability of encoders and decoders
- Widespread use and support; computers fast enough for encoding & decoding

## Anecdote:

A hacker named SoloH discovered the source code of the "dist10" MPEG reference implementation shortly after the release on the servers of the University of Erlangen. He developed a higher-quality version and spread it on the internet. This code started the widespread CD ripping and digital music distribution as MP3 over the internet.

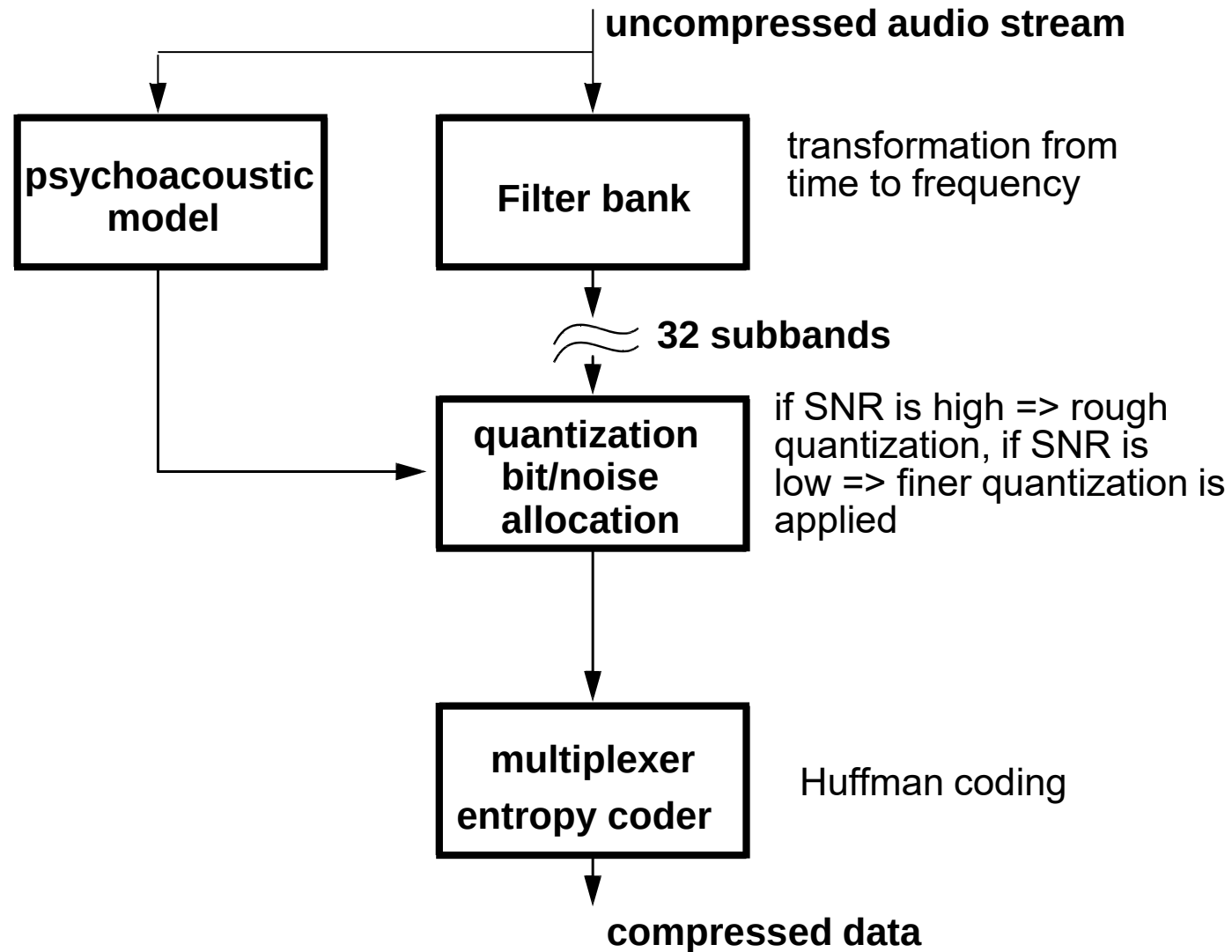


# General Approach of MPEG-1/2 Layer-3

- Encoder is not precisely specified but building blocks are described
  - Implementers of the standard were supposed to devise their own algorithms suitable for removing parts of the information from the audio input.
  - Many different MP3 encoders became available, each producing files of differing quality.
- Decoding is carefully defined in the standard. Most decoders are "bitstream compliant"
  - Only computational performance of decoders differs (depending how efficiently implemented)



# MPEG Audio Standard (Encoding)



# High Quality Audio Coding

- MPEG-1 Layer-3 has been specified 1993
- since then, research on perceptual audio coding has progressed and codes with better compression efficiency became available: MPEG-2 Advanced Audio Coding (AAC) and other proprietary compression systems.
- Basic task is to compress audio data in a way that
  - the compression is as efficient as possible (file size small)
  - the reconstructed (decoded) audio sounds exactly (or as close as possible) to the original audio data
  - requires low complexity (in software or inexpensive hardware)
  - offers flexibility for different application scenarios.

# Masking and MPEG Audio Compression

- General audio signal consists of many frequency components
  - each frequency component influences the hearing threshold curve
  - Audio signal varies with time
- Hearing threshold different at each point in time
- **Psychoacoustic model**
  - Calculates current hearing threshold curve in each case
- **Variable quantization**
  - Frequency components quantized so that quantization noise just below current hearing threshold curve
  - Where masking is used, coarse quantization, small word lengths

# A Basic Perceptual Audio Encoder

Consists of the following building blocks:

1. Filter bank—is used to decompose the input signal into subsampled spectral components (time/frequency domain)
2. Perceptual model—using either the domain input signal and/or the output of the analysis filter bank, an estimate of the actual masking threshold is computed by rules from psychoacoustics
3. Quantization and coding—the spectral components are quantized and coded; noise introduced should be kept below threshold
4. Lossless Huffman Encoding of bitstream

# A Basic Perceptual Audio Coder

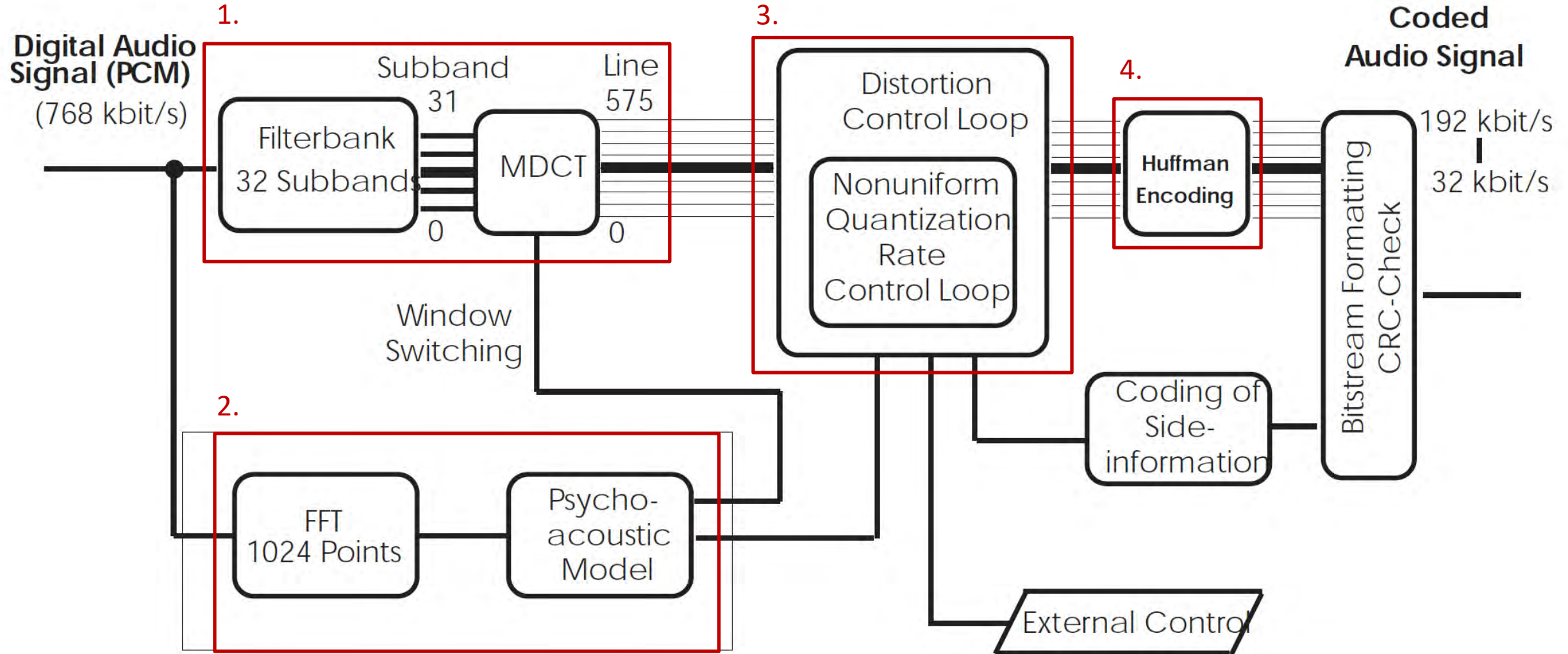


Figure 2: Block diagram of an MPEG-1 Layer-3 encoder.

# Layer-3 Encoding Algorithm

## Basic building blocks

- **Filterbank**: Actually 2 cascading filterbanks:
  - Polyphase filterbank (as in Layer-1 and 2); Cut off frequencies  $<16\text{Hz}$ ,  $>16\text{kHz}$
  - Additional Modified DCT (MDCT); the subdivision of each polyphase frequency band into **18 finer subbands** increases the potential for redundancy removal;
- **Perceptual Model**
  - Mainly determines the quality of a given encoder implementation; uses either a separate filterbank or combines the calculation of energy values (for masking calculations) and main filter bank

# Layer-3 Encoding Algorithm

- Perceptual Model (continued)
  - Output of the perceptual model consists of values for the masking threshold or allowed noise for each coder partition (roughly equivalent to the critical bands of human hearing)
- **Quantization** and coding
  - two nested iteration loops (rate loop and noise control loop)
  - quantization is done via a power-law quantizer; larger values are automatically coded with less accuracy
  - quantized values are coded by **Huffman coding**

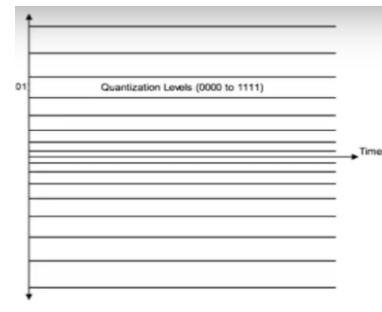


# Layer-3 Encoding Algorithm

- Quantization and coding (continued)
  - outer iteration loop (noise control loop)

If the quantization noise in a given band is found to exceed the masking threshold (allowed noise) as supplied by the perceptual model, the **scale factor** for this band is adjusted to reduce the quantization noise.  
(power-law quantizer = non-linear, can adjust quantization steps)  
= **adapt quantization steps per band**
  - inner iteration loop (rate loop – modifies overall coder rate)

If number of bits resulting from coding operation exceeds the number of **bits available** to code a **block** of data, a larger quantization step is applied.  
= **adjust global gain**



# Huffman Encoding—Example

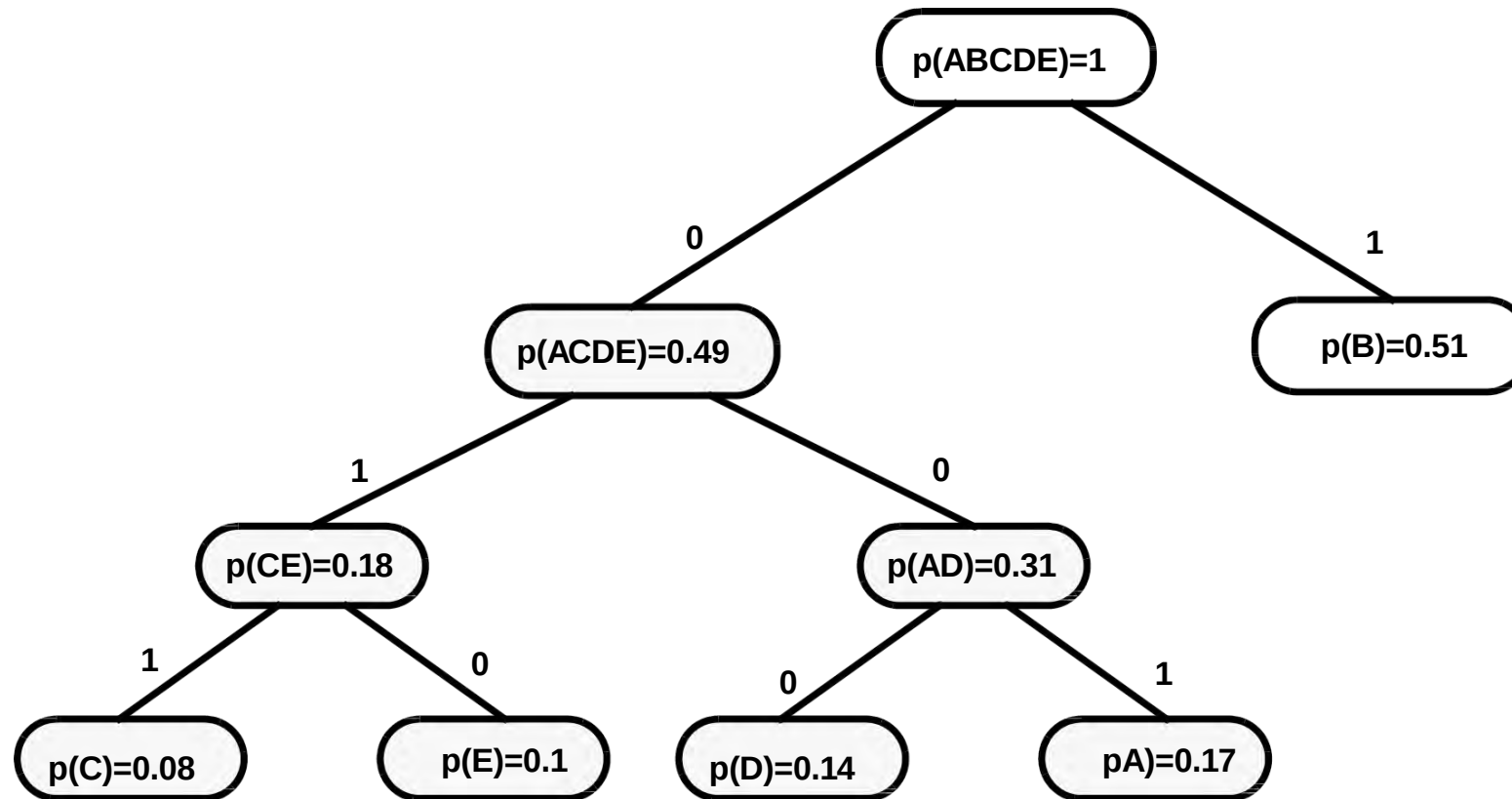
encode symbols according to tree

|   |     |
|---|-----|
| A | 001 |
| B | 1   |
| C | 011 |
| D | 000 |
| E | 010 |

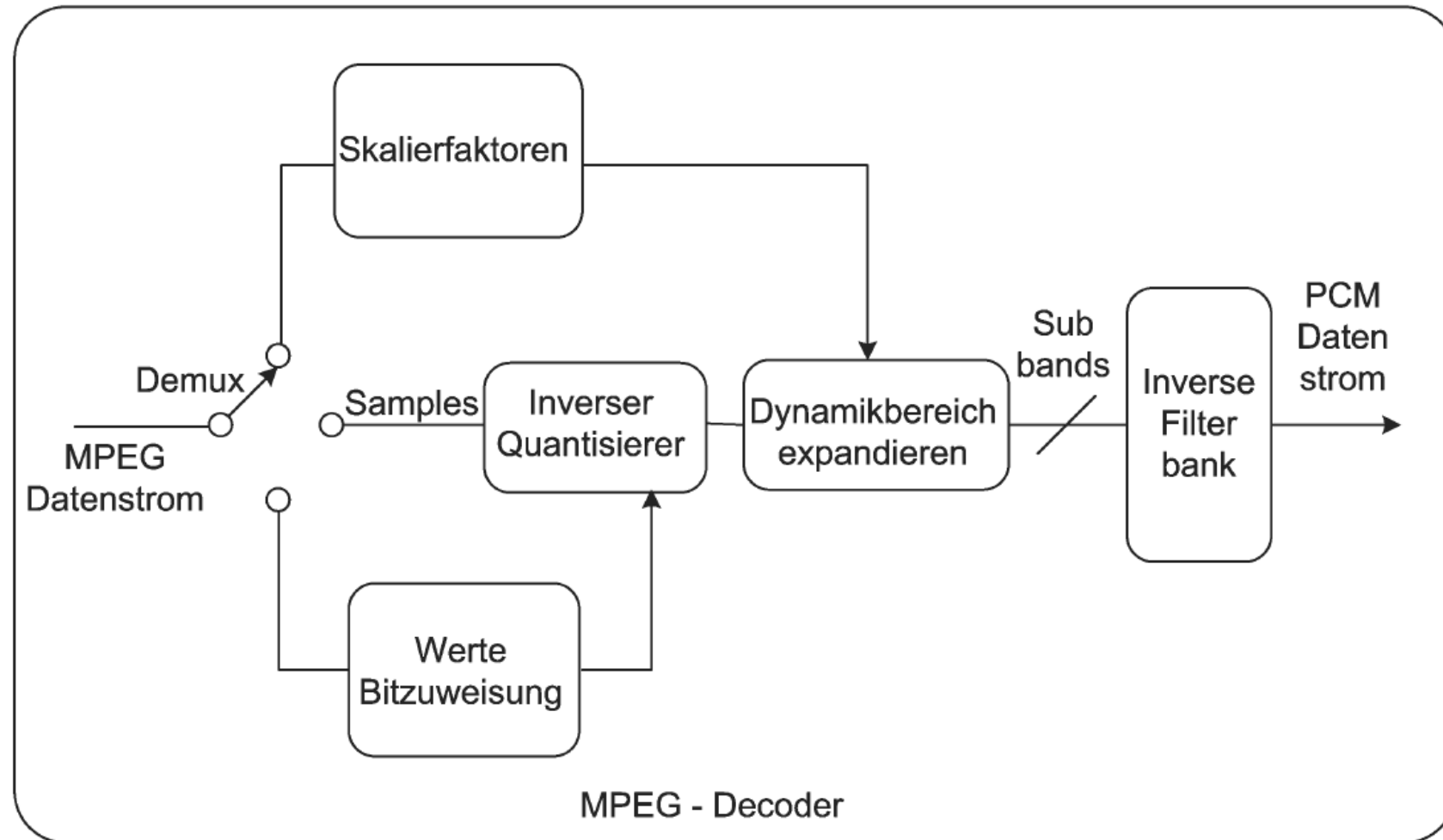
$p(C)=0.08$ ,  $p(E)=0.1$ ,  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(B)=0.51$

# Huffman Encoding—Example

last step:  $p(ACDE)=0.49$ ,  $p(B)=0.51$



# Simple MPEG decoder



# First Step: Split Audio File into “Frames”

- Split audio file into “frames”
- During encoding 576 time domain samples are taken & transformed to 576 frequency domain samples. If there is a transient 192 samples are taken.
  - This is done to limit the temporal spread of quantization noise accompanying the transient.
- The decoder expects a frame consisting of 384, 576, or 1152 samples
  - Typically in MP3 each "frame" has a length of 1152 samples = 0.026 sec (26,122449 ms)
- The following processing happens in each frame individually - with some overlap with previous/next frame to minimize temporal effects

# MP3 Algorithm – Basic Steps (Simplified)

```
algorithm MPEG-1_audio
```

```
/*Input: An audio file in the time domain
```

```
Output: The same audio file, compressed*/
```

```
{
```

```
  Divide the audio file into frames
```

```
  For each frame {
```

```
    By applying a bank of filters, separate the signal into frequency bands.
```

```
    For each frequency band {
```

```
      Perform a Fourier transform to analyze the band's frequency spectrum
```

```
      Analyze the influence of tonal and nontonal elements (i.e., transients)
```

```
      Analyze how much the frequency band is influenced by neighboring bands
```

```
      Find the masking threshold and signal-to-mask ratio (SMR) for the band,  
      and determine the bit depth in the band accordingly
```

```
      Quantize the samples from the band using the determined bit depth
```

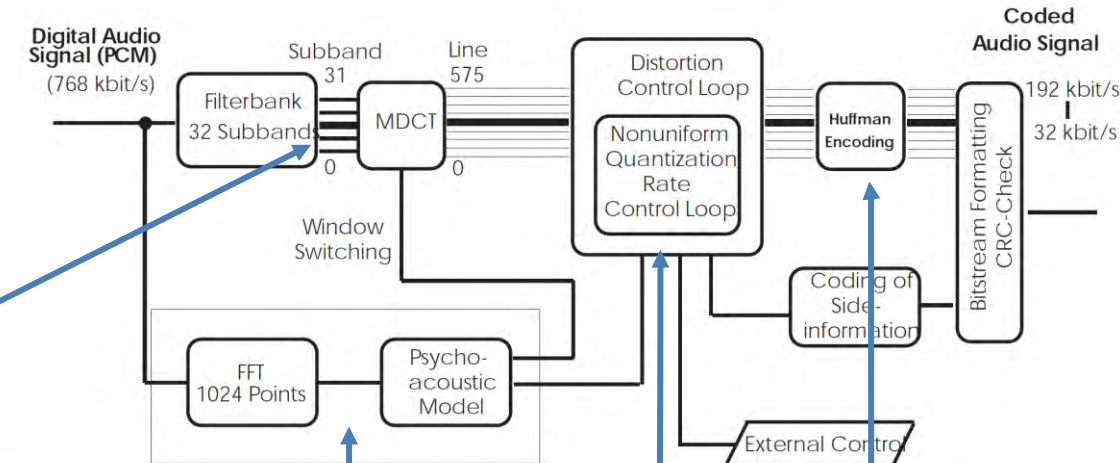
```
      Apply Huffman encoding (optional)
```

```
    }
```

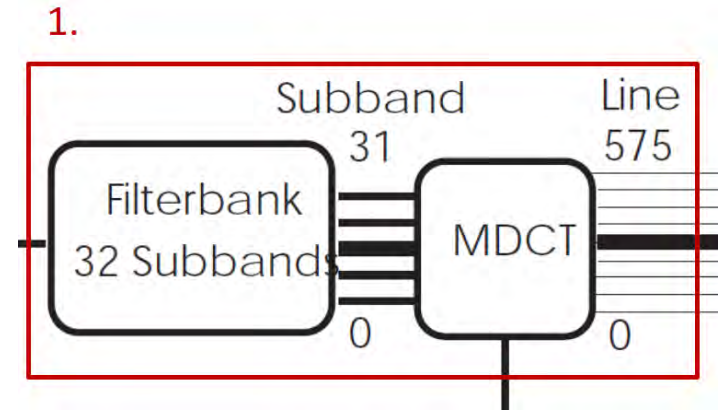
```
  Create a frame with a header and encoded samples from all bands
```

```
}
```

```
}
```



# Subband coding



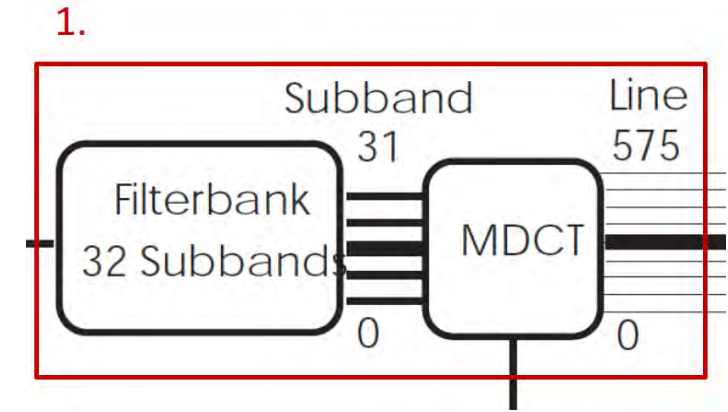
MPEG audio encodes frequency components

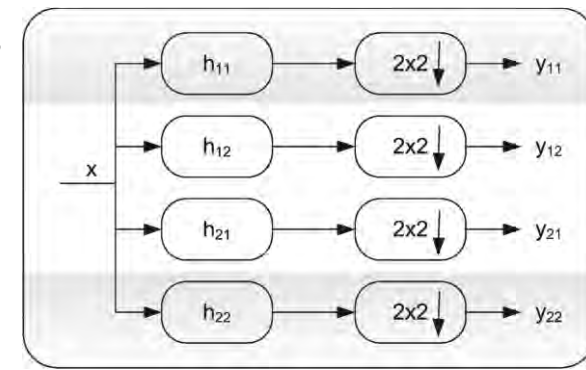
- Filter bank
  - divides audio signal into frequency bands (subbands)
- Ideal filter bank
  - Subbands identical to frequency groups of human hearing



# MPEG Polyphase Filter Bank

- Input: Digital PCM Audio Signal
- MPEG Polyphase Filter Bank
  - Audio signal is divided into 32 subbands
  - Each subband has equal size/bandwidth
- Advantage
  - simple structure
- Disadvantage
  - lower subbands span over several frequency groups
    - solved in MPEG-1/Layer 3 by MDCT, modeling human critical bands more closely
  - overlapping subbands lead to information loss





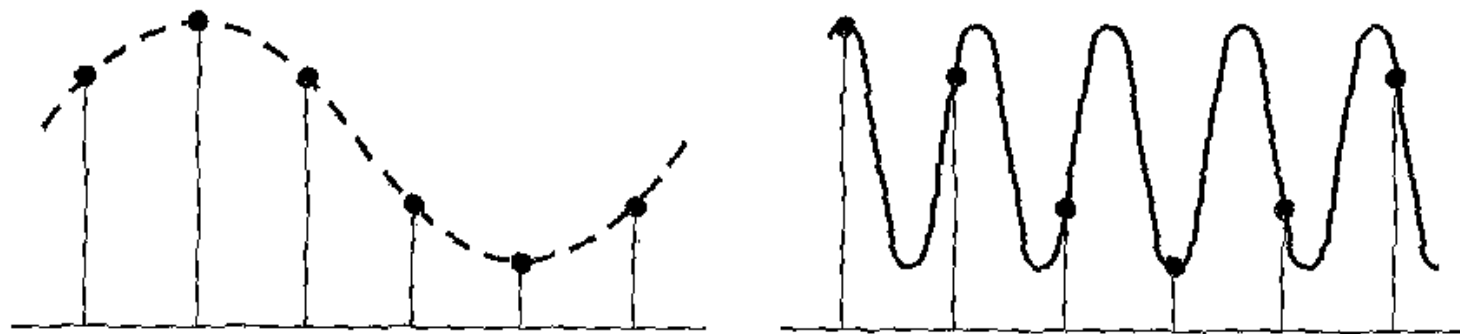
- Example filter bank with 4 subbands
- Input - 1 x PCM: frequency range 0 - 20kHz, sampling rate 48 kHz; word length 16 bit per sample
- Output - 4 x PCM
  - Subband 1: 0Hz - 5kHz; sampling rate 12kHz (!); 16 bit
  - Subband 2: 5kHz -10 kHz; sampling rate 12kHz (!); 16 bit
  - Subband 3: 10kHz - 15 kHz; sampling rate 12kHz (!); 16 bit
  - Subband 4: 15 kHz - 20 kHz; sampling rate 12kHz (!); 16 bit
- Seems to be a violation of the Nyquist-Shannon theorem BUT (see [1])
  - IF a **band does not start at zero frequency** but at some higher value and can be shifted by a linear translation, smaller sampling rate – **at least twice the width of the non-zero frequency interval** - is required.
  - For example, in order to sample the FM radio signals in the frequency range of 100–102 MHz, it is not necessary to sample at 204 MHz (twice the upper frequency), but rather it is sufficient to sample at 4 MHz (twice the width of the frequency interval).

[1]

[https://en.wikipedia.org/wiki/Nyquist%E2%80%93Shannon\\_sampling\\_theorem#Sampling\\_of\\_non-baseband\\_signals](https://en.wikipedia.org/wiki/Nyquist%E2%80%93Shannon_sampling_theorem#Sampling_of_non-baseband_signals)

# Filter Bank - Critical Sampling

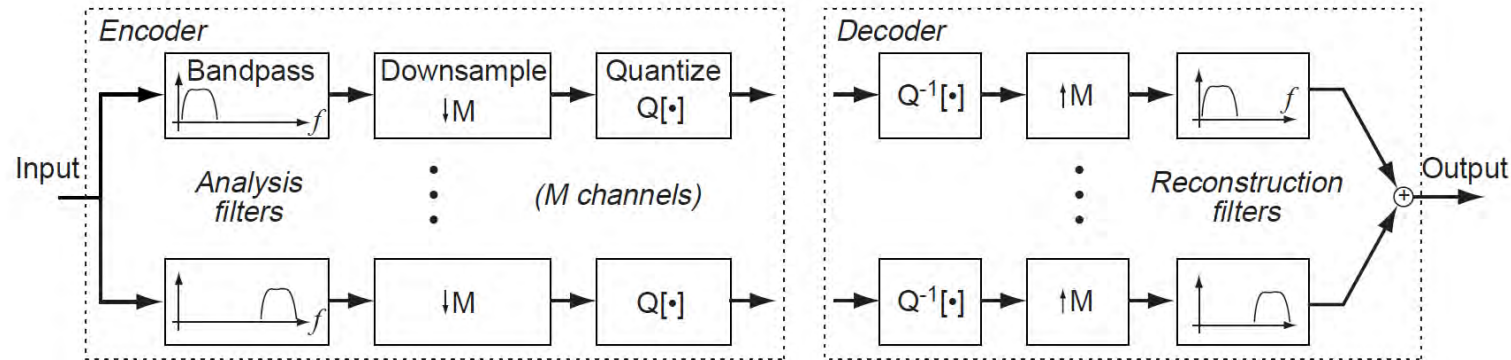
- Typically 32 subbands in MP3
- Critical sampling of digital filter banks
  - Sampling rate in subbands reciprocal to the number of subbands
    - Reduced sampling rate in subbands is called critical sampling rate
- Critical sampling of subbands ensures that **filter bank has no influence on data volume**
  - More channels are generated, but their sampling frequencies are correspondingly lower.



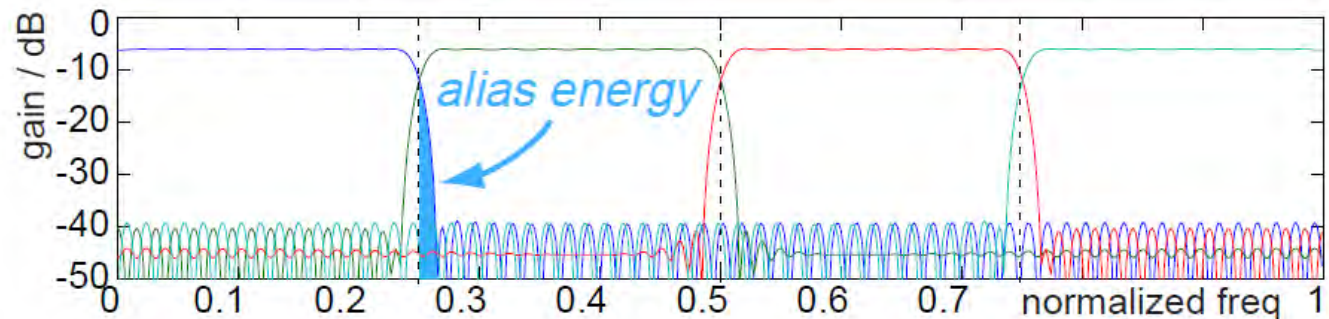
- Left: Samples / signal without specification of frequency range
- Right: Samples / signal with indication of frequency range (right)

# Subband Coding

- Idea: Quantize separately in separate bands
  - Quantization Noise stays within band, gets masked



- 'Critical sampling'  $\rightarrow 1/M$  of spectrum per band
  - some aliasing inevitable



- Trick is to cancel with alias of adjacent band
  - $\rightarrow$  'quadrature-mirror' filters / MDCT

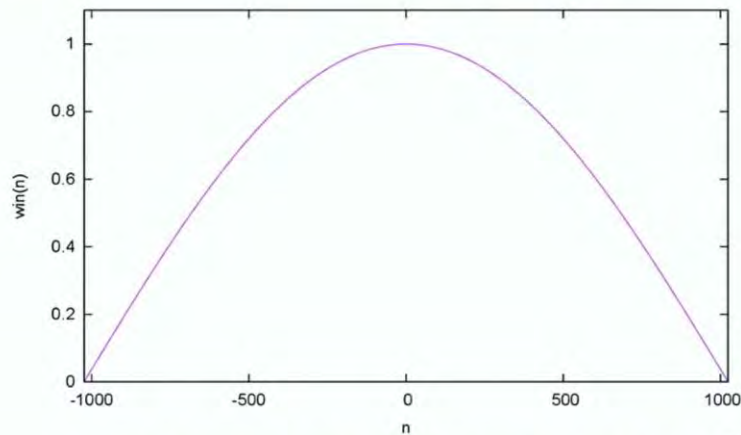
# Subbands in MPEG Standard

- MPEG-1/Layer 1 and Layer 2 - 32 Subband Filter Bank
- In MPEG-1/Layer 3 (MP3) a Hybrid filter bank is used
  - Modified Discrete Cosine Transform (MDCT) performs transformation into frequency domain
- In MPEG-2 - AAC
  - Lossless MDCT - frequency decomposition corresponds to 1024 subbands

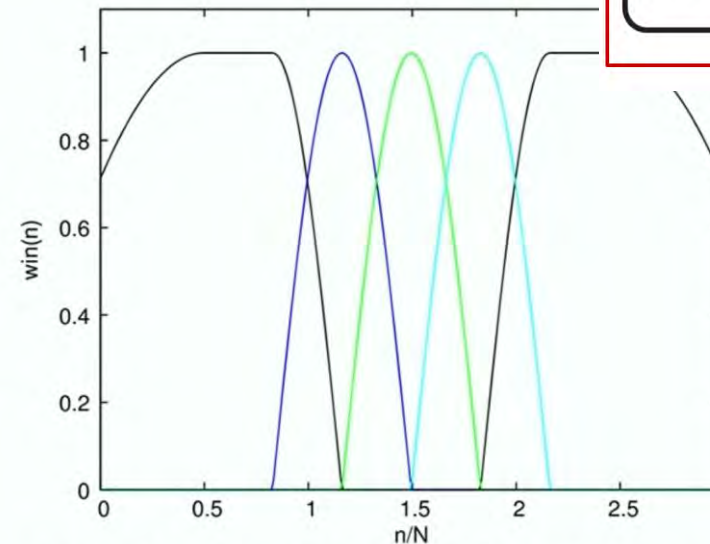
# Modified Discrete Cosine Transform (MDCT)

- Performs transformation of subbands into frequency domain
- Overlaps with neighboring frequency bands (but does not increase sample size)
- MDCT also used for avoiding temporal alias effects

## Modified Discrete Cosine Transform (MDCT)

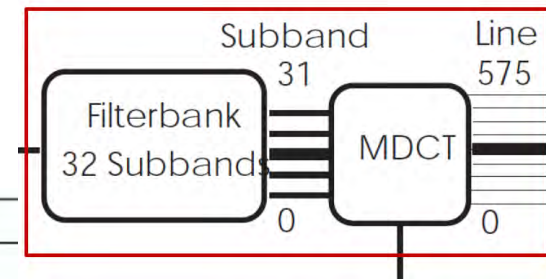


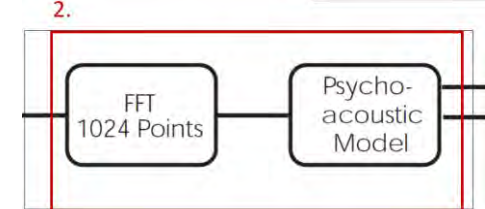
Sine window frequently used for MDCT



Adaptively switched MDCT windows

1.

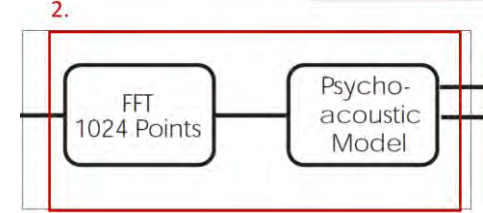




# MPEG Psychoacoustic Model: Overview

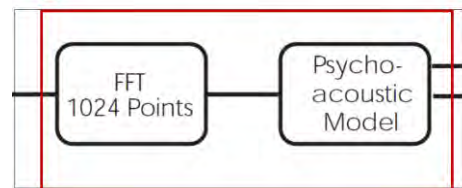
- First: Fast Fourier Transform into frequency domain
  - Psychoacoustic model operates on frequency spectrum
- Analyzes audio signal section by section for masking effects
  - Algorithms simulate human hearing
- Calculates masking threshold per subband
  - calculates maximum allowed quantization noise per subband
- Quality of the psychoacoustic model mainly responsible for overall quality of coding



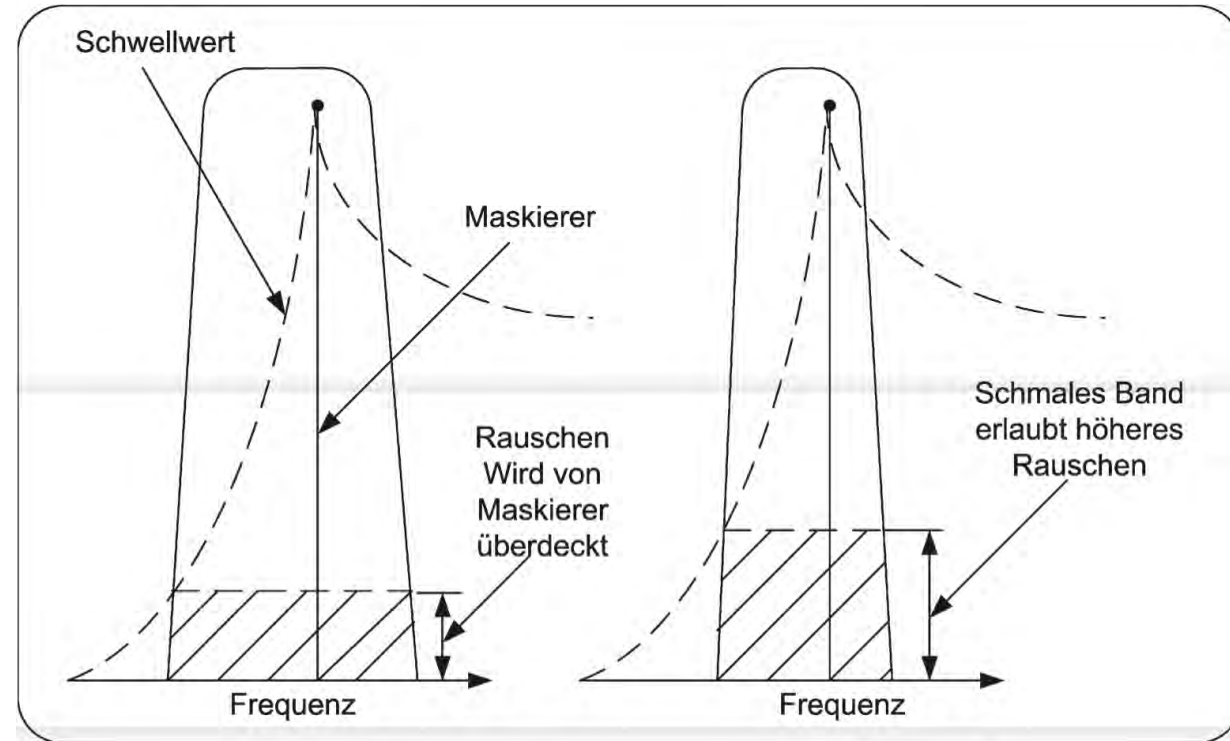


# Fast Fourier Transform

- For signal analysis FFT is particularly well suited
  - Provides fine frequency resolution of the signal
    - provides phase information of the frequency components (important for psychoacoustic model 2 (mentioned later))
  - Very accurate calculation of the masking thresholds
- ➡ high compression possible
- Technique for minimizing the computational effort
    - Bundling values in frequency groups
    - Frequency groups correspond to critical bandwidths of human hearing

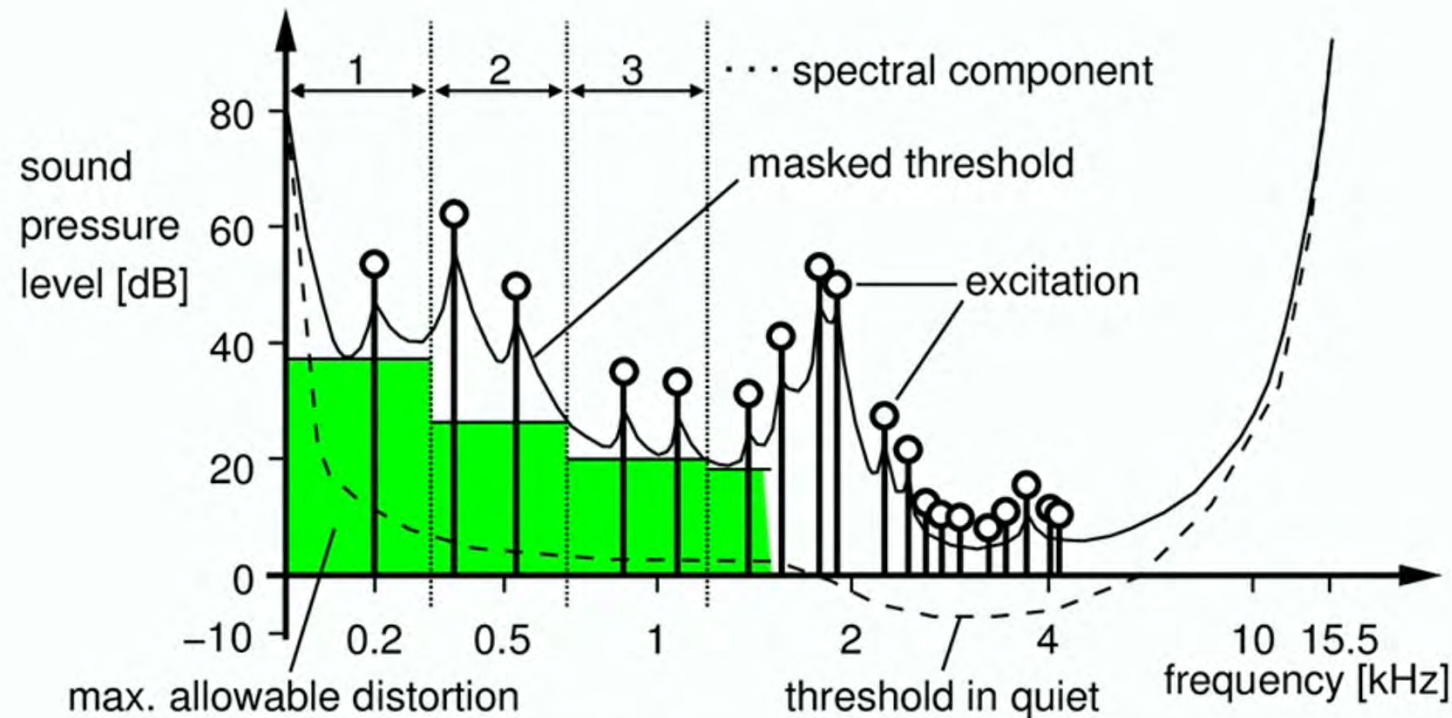


# Masking in Psychoacoustic Model

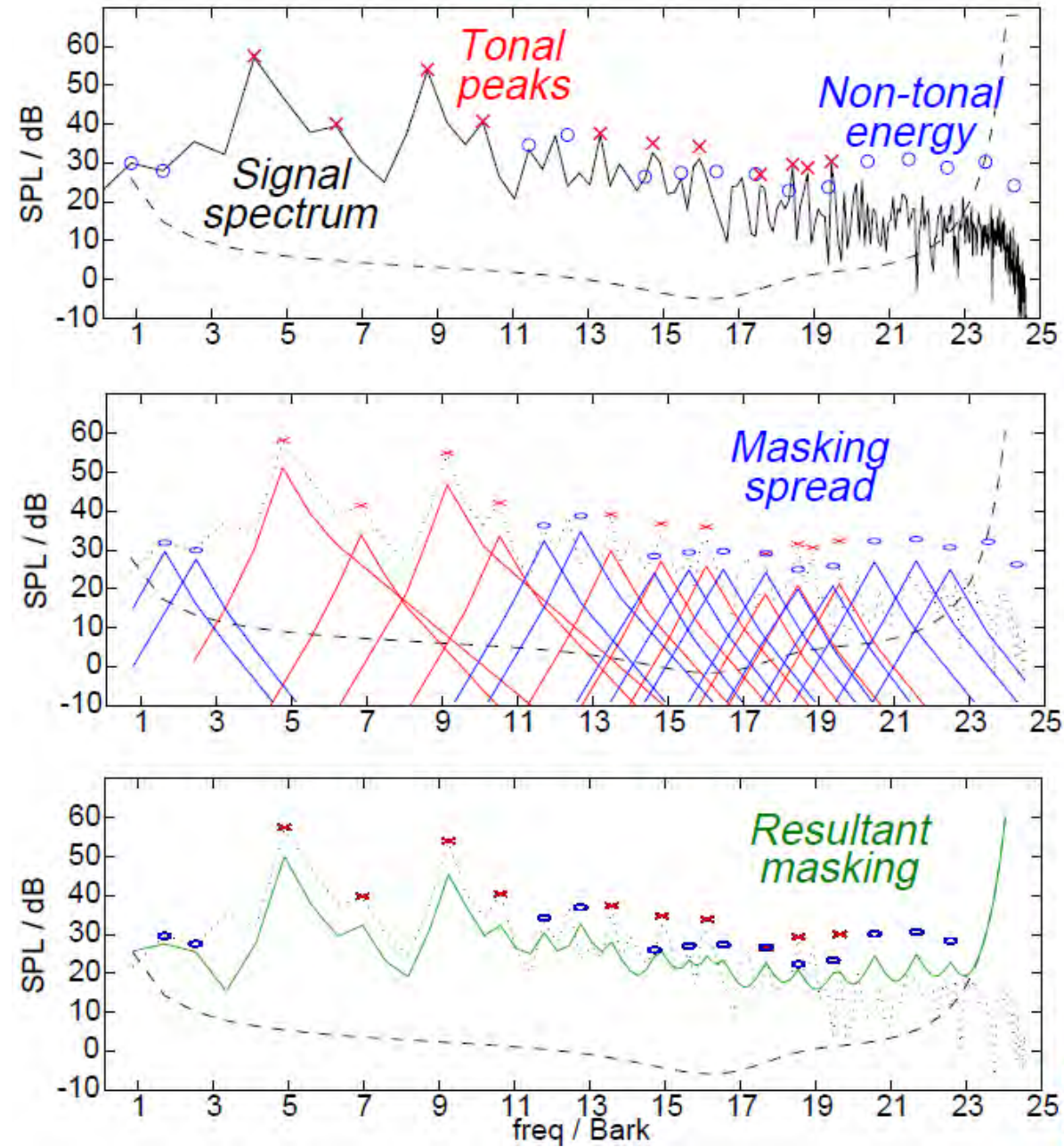


# Masking in Psychoacoustic Model

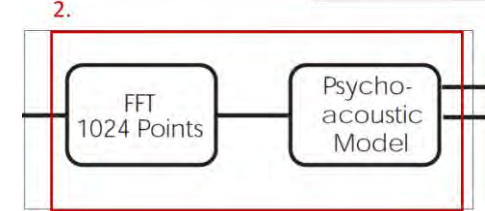
- For each subband: How high can quantization noise be, so that it is still masked?



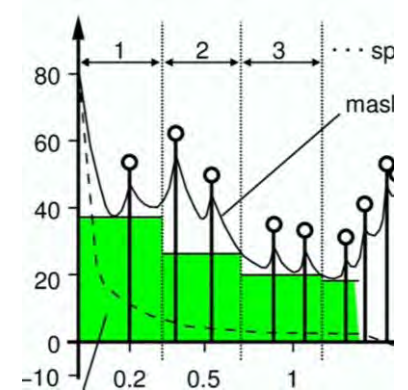
Spectral noise shaping in transform/subband coding



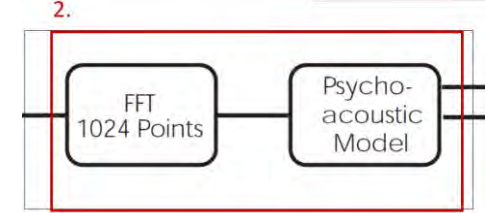
# Masking in Psychoacoustic Model



- Because of narrow subbands
  - high masking threshold for noise possible
  - frequency components require only few bits, which corresponds to high compression
- In general
  - the higher the subband number the narrower the subbands
  - the narrower the subbands the higher the compression

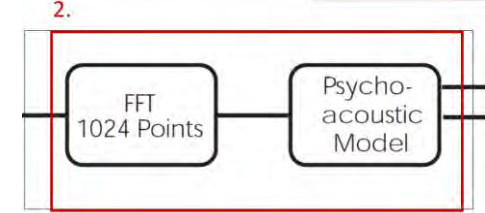






# Tonal / Atonal Components

- Tonal and noise-like acoustic events have different masking properties
- Psychoacoustic model needs algorithms to distinguish between sound and noise
- Model 1
  - strong peaks in the frequency spectrum evaluated as tonal components
  - remaining parts interpreted as noises
- Model 2
  - more complex algorithms, considering phase, etc..

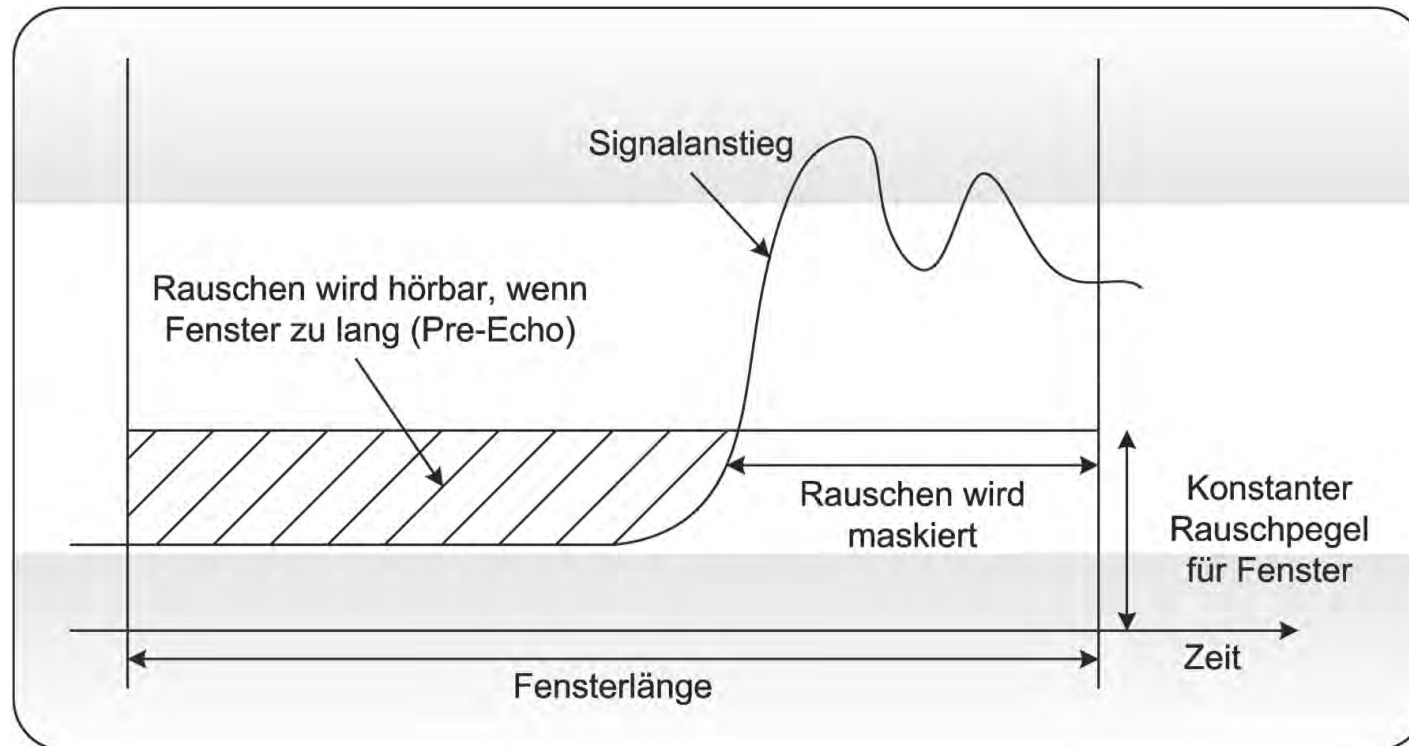


# Result Psychoacoustic Model

- Psychoacoustic model calculates hearing threshold curve
  - Noise below curve is not audible
  - Noise above curve is audible
  - for noise-free coding, audible noise level must not be exceeded at any point
- Calculation of the masking thresholds
  - for each subband the masking threshold is calculated
  - the smaller the subbands are, the better the fit of the step function to hearing threshold curve (high compression)

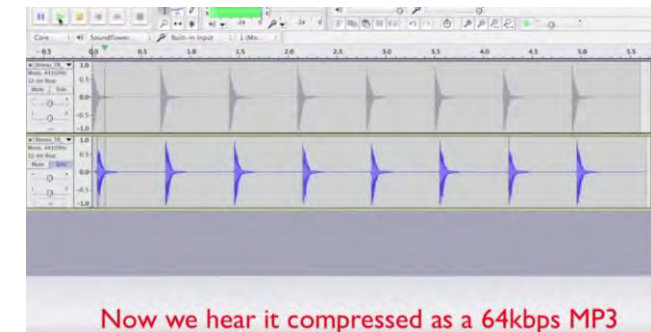


# Compression with Steep Signal Rise – Temporal Issues



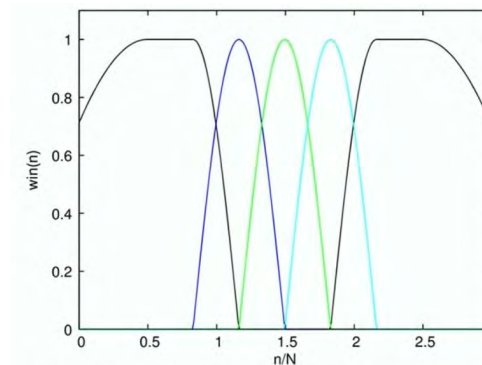
# Compression with Steep Signal Rise – Pre-Echo

- Signal sections (windows) with steep signal slopes are very difficult to compress
  - Quantization noise equal over entire window length
  - Noise before signal rise audible as **pre-echo**
    - Quantization noise is unmasked in the silence before “attack”
- MPEG Strategies
  - Layer1/Layer2 - no artifact-free compression possible
  - MP3 - uses short window for this, pre-echo effect is reduced, but moderate compression, as less subbands are achievable with short window
  - AAC - short window, Temporal Noise Shaping (TNS) module, good compression



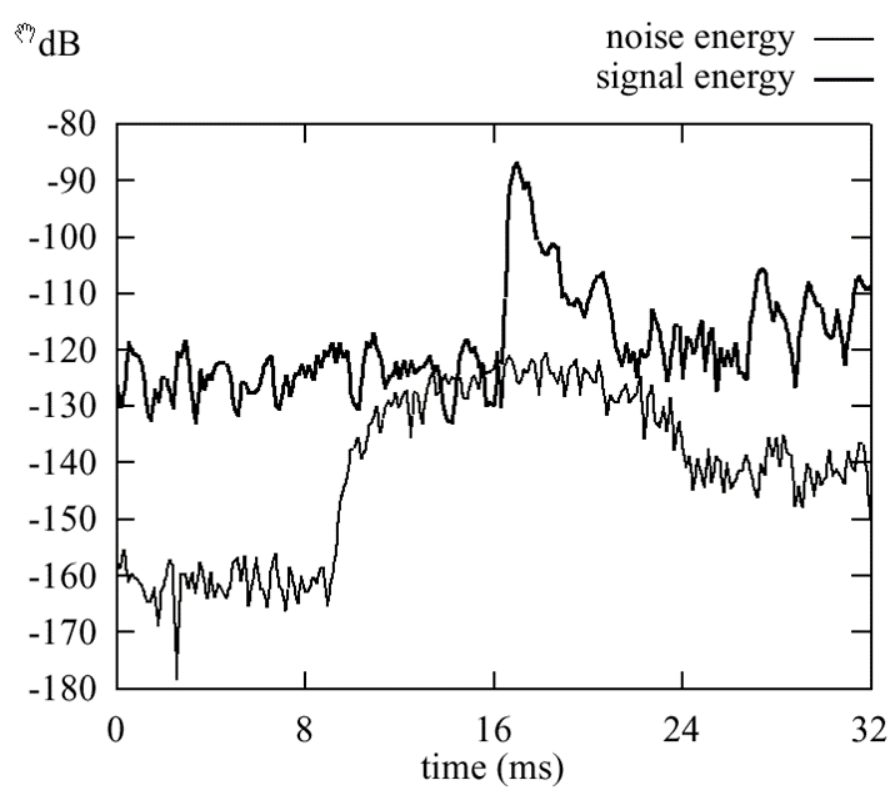
Pre-Echo

<https://www.youtube.com/watch?v=MtiRBFwKRs>

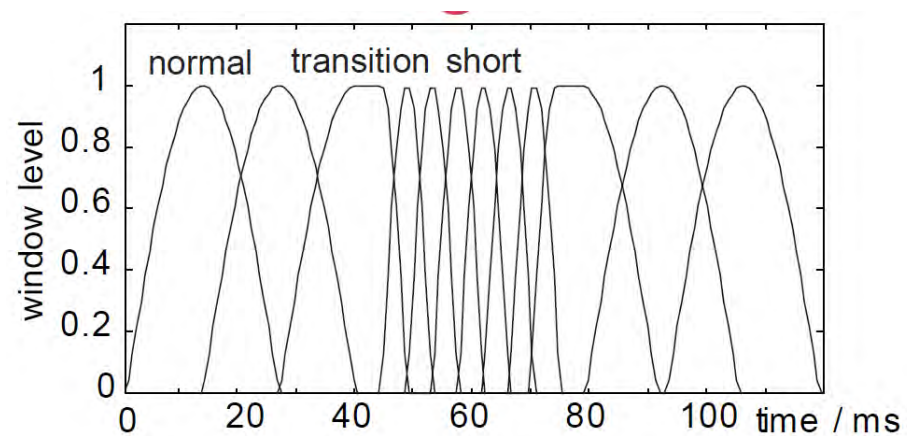


Adaptively switched MDCT windows

# Pre-Echo



- Adaptive switching of time-window



# Frequency Resolution / Time Window

High frequency resolution with long time window

+ : many subbands, high compression

- Subbands well adaptable to hearing threshold curve

- : Quantization noise not changeable for a long time

- bad with signal increases

Low frequency resolution with short time window

+ : Quantization noise changeable in short time intervals

- Noise can be well matched to time curve

- : Fewer subbands, moderate compression

# A Basic Perceptual Audio Coder

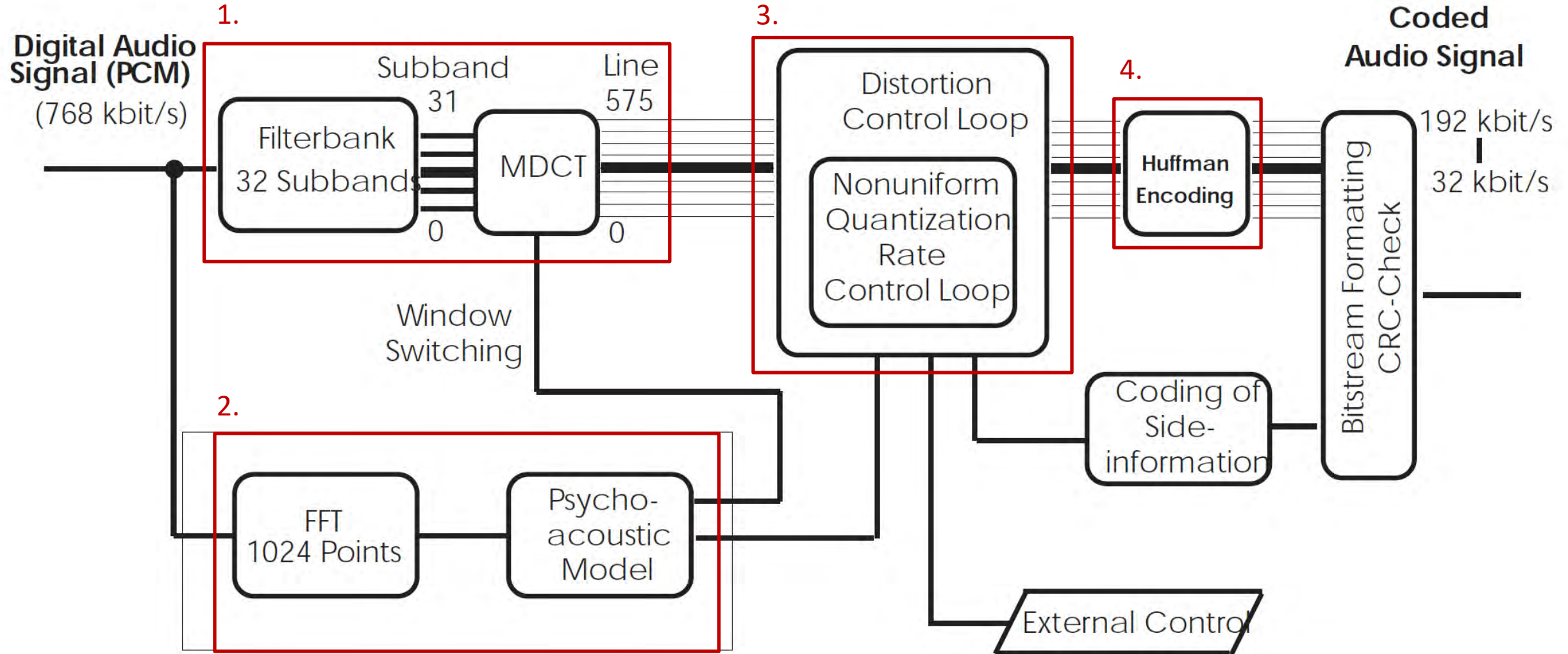


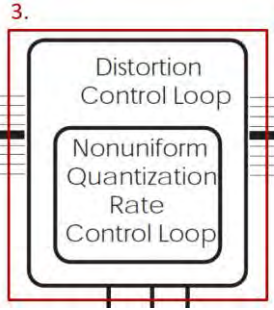
Figure 2: Block diagram of an MPEG-1 Layer-3 encoder.

- Basic idea: Subband coding plus psychoacoustic masking model to choose dynamic Quantization levels in subbands

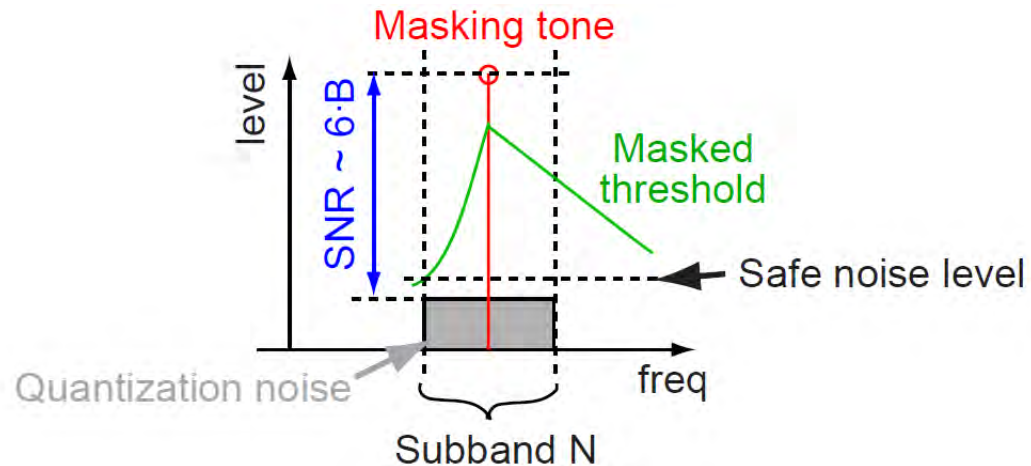




# MPEG Bit allocation



- **Result of psychoacoustic model is maximum tolerable noise per subband**

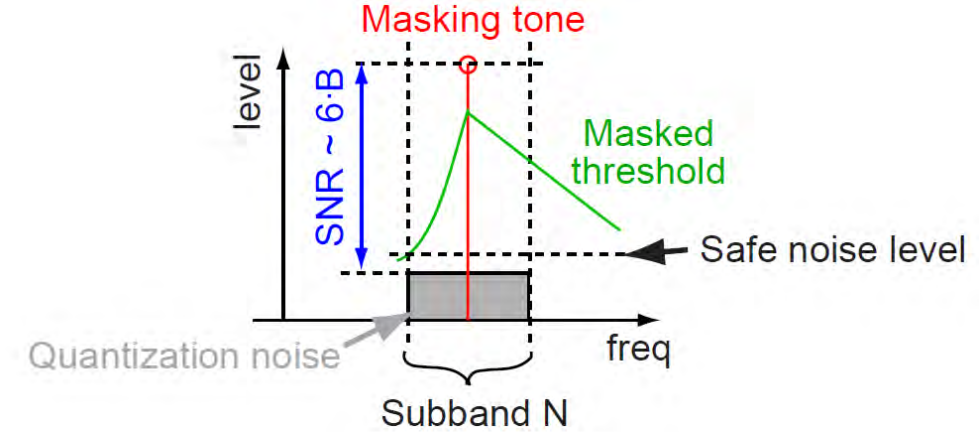


- safe noise level  $\rightarrow$  required SNR  $\rightarrow$  bits B

- **Bands with no signal above masking curve can be skipped entirely**
- **Bit allocation procedure (fixed bit rate):**
  - pick channel with worst noise-masker ratio
  - improve its quantization by one step
  - repeat while more bits available for this frame

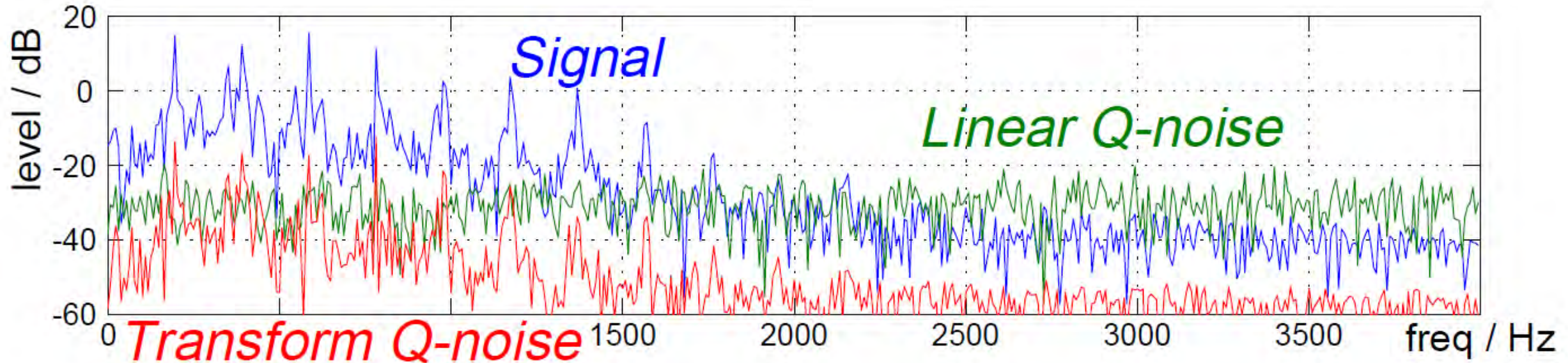


# Layer-3 Quantization Algorithm



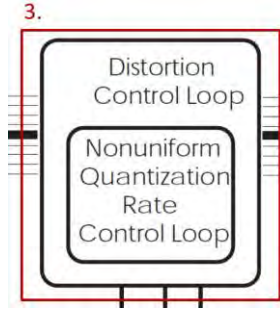
- **Quantization** and coding
  - quantization is done via a power-law quantizer; larger values are automatically coded with less accuracy
  - two nested iteration loops (rate loop and noise control loop)
- Outer iteration loop (noise control loop) – for each band
  - If the quantization noise in a given band is found to exceed the masking threshold (allowed noise) - supplied by the perceptual model - the scale factor for this band is adjusted to reduce the quantization noise.
- Inner iteration loop (rate loop – modifies overall coder rate) – for each frame
  - Check if Huffman coding is small enough
  - If number of bits resulting from coding operation exceeds the number of bits available to code a block of data, **adjust global gain** -> results in larger quantization step.

# Example

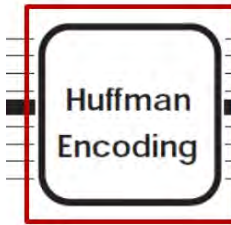


Comparison: Linear Quantization Noise over whole frequency band (green) compared to reduced quantization noise (in red) where noise is scaled in different frequency bands.

# Bit assignment



- User can select desired MPEG bit rate
  - Psychoacoustic model calculates values so that no quantization noise is audible
  - at low data rates, available bits are usually not sufficient
  - Bit allocation algorithms
    - high frequencies are removed
    - Stereo will be spatially inaccurately resolved
    - Well-adjusted introduction of quantization noise
- Algorithms are left to software developers
- MPEG Audio standard offers tables as recommendation



# Huffman Encoding

- Final quantized bands will be Huffman encoded = lossless encoding (the lossy part happened before)
- This is all still done in the frequency domain!

# Huffman Encoding

- Statistical encoding, depends on **occurrence frequency of single characters or sequences** of data bytes
- Characters stored with their probabilities
- Length (number of bits) of the coded characters differs
- Shortest code assigned to the most frequently occurring character
- To determine Code, it is useful to construct a **binary tree**
  - leaves are characters to be encoded
  - nodes carry **occurrence probabilities** of the characters belonging to the subtree

# Huffman Encoding — Algorithm

build a Huffman Tree:

$p(A)=0.17$ ,  $p(B)=0.51$ ,  $p(C)=0.08$ ,  $p(D)=0.14$ ,  $p(E)=0.1$

```
build ordered list of symbols (increasing probability)
```

```
do while list contains at least 2 elements
```

```
    construct tree using the first two elements in list
```

```
    add parent node for the union of these elements and compute probability
```

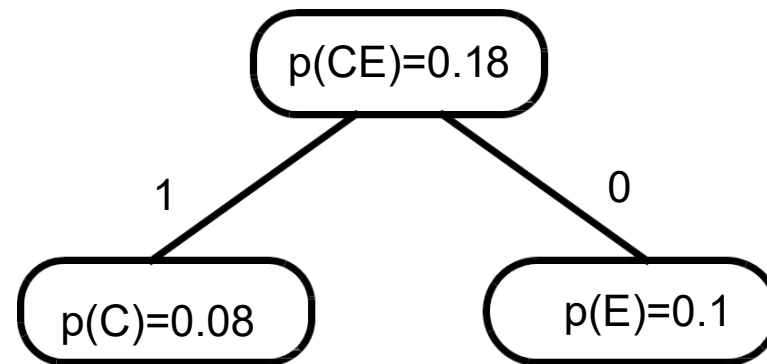
```
    mark edges by '0' and '1'
```

```
    delete the first 2 elements in list; insert parent into list
```

```
end
```

# Huffman Encoding—Example

step 1:  $p(C)=0.08$ ,  $p(E)=0.1$ ,  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(B)=0.51$





# Huffman Encoding—Example

step 2:  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(CE)=0.18$ ,  $p(B)=0.51$

build ordered list of symbols (increasing probability)

do while list contains at least 2 elements

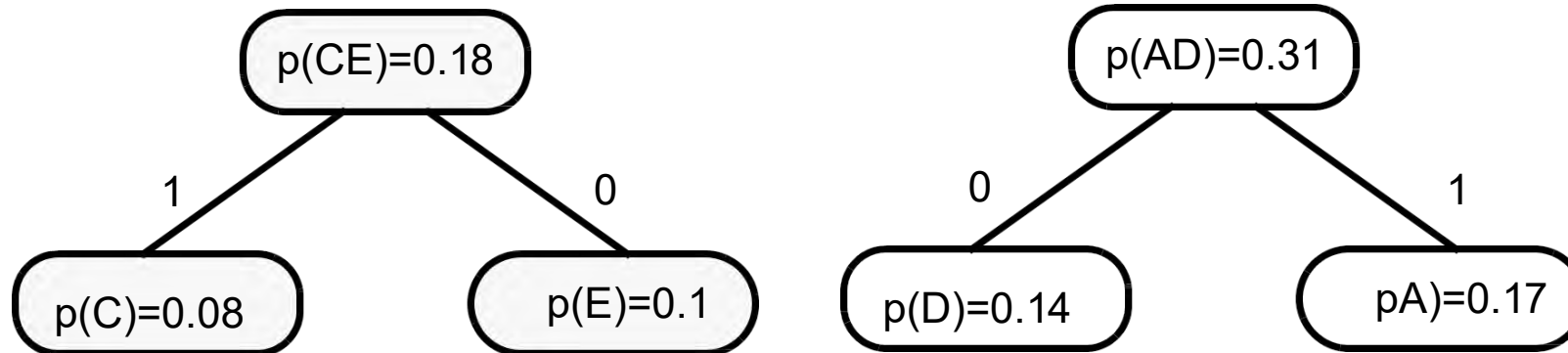
construct tree using the first two elements in list

add parent node for the union of these elements and compute probability

mark edges by '0' and '1'

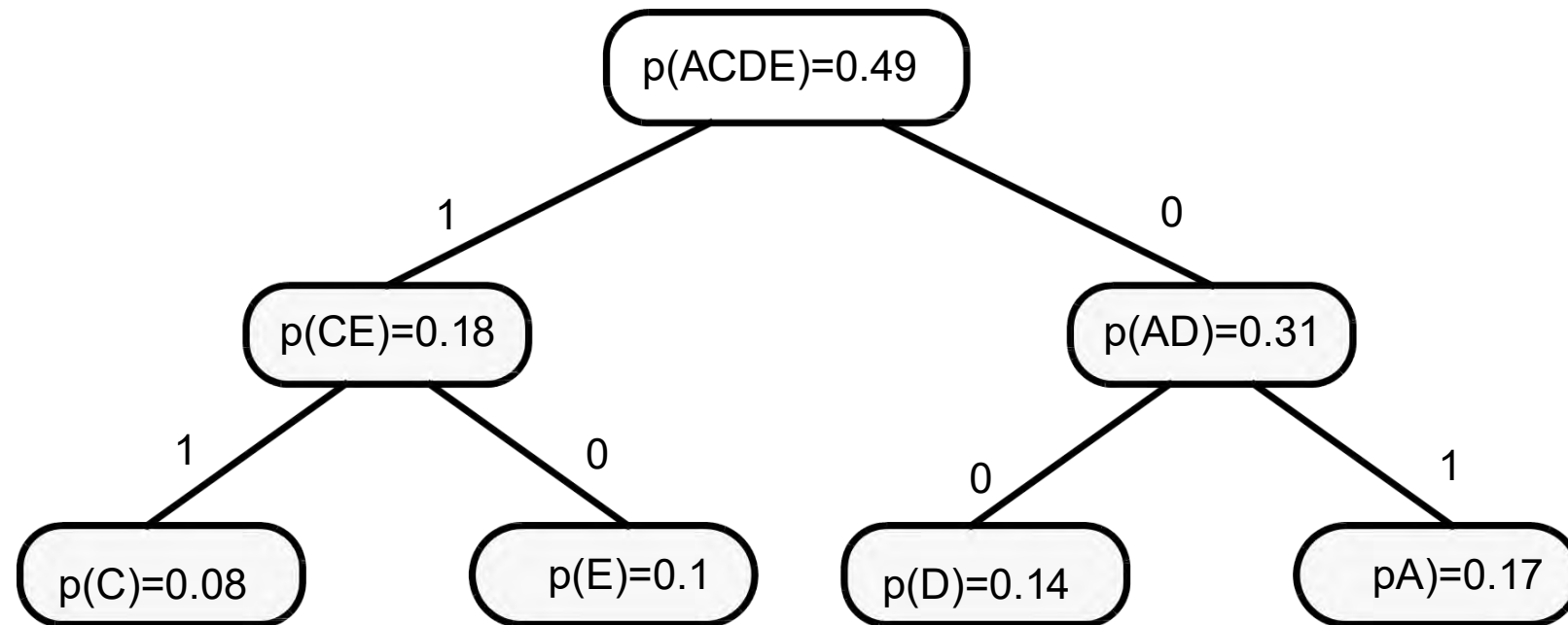
delete the first 2 elements in list;  
insert parent into list

end



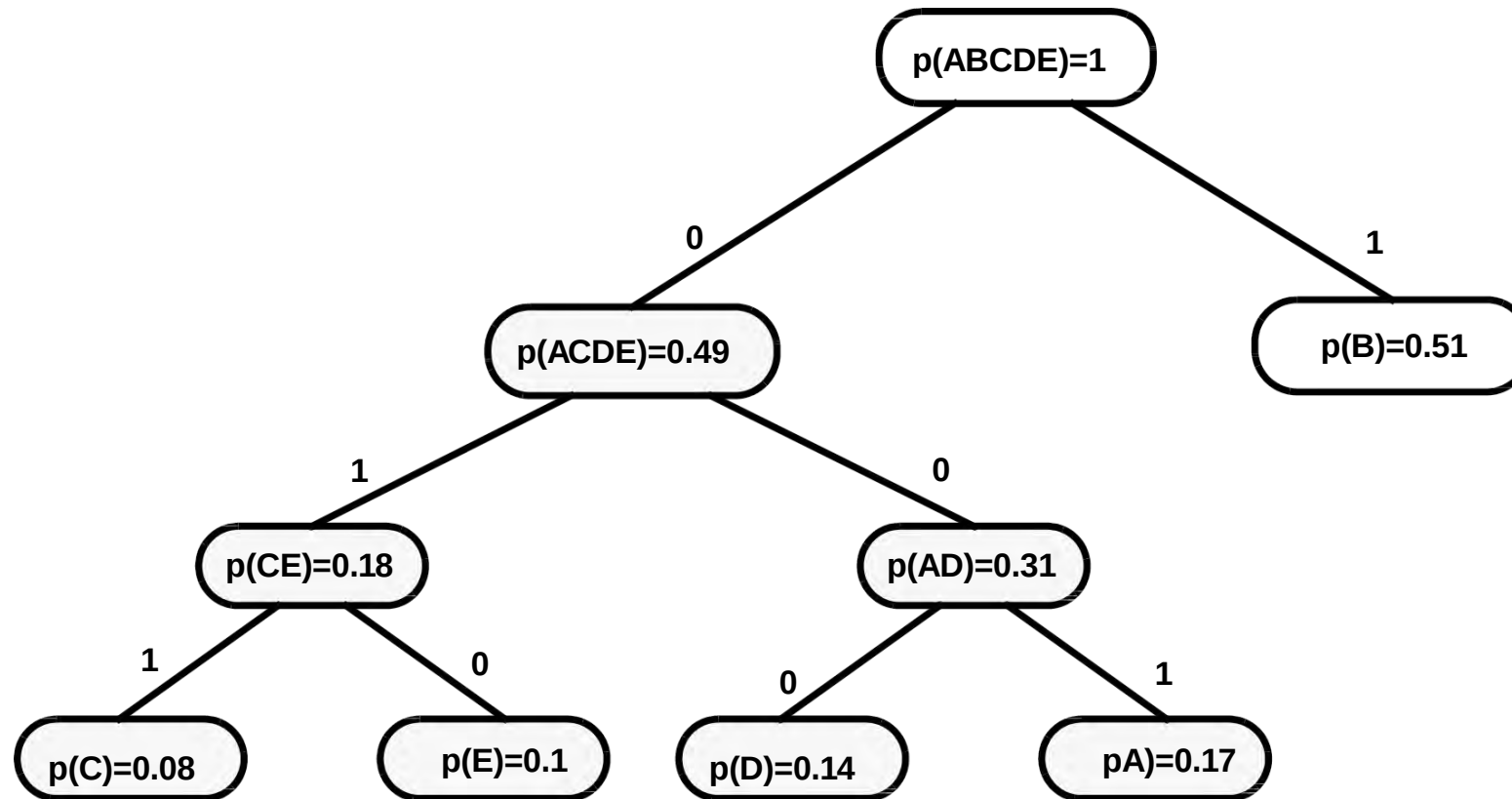
# Huffman Encoding—Example

step 3:  $p(CE)=0.18$ ,  $p(AD)=0.31$ ,  $p(B)=0.51$



# Huffman Encoding—Example

last step:  $p(ACDE)=0.49$ ,  $p(B)=0.51$



# Huffman Encoding—Example

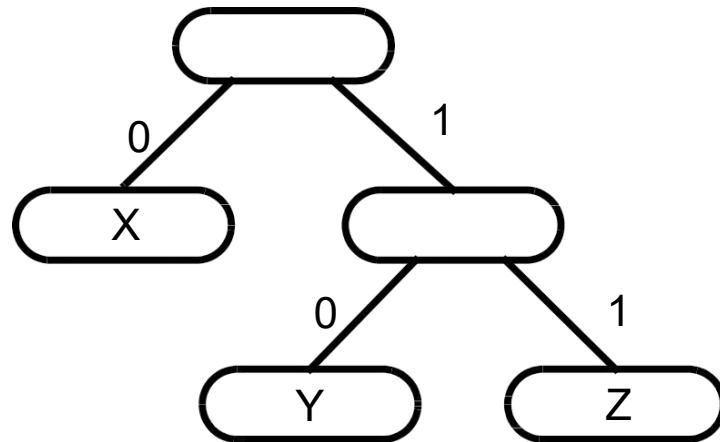
encode symbols according to tree

|   |     |
|---|-----|
| A | 001 |
| B | 1   |
| C | 011 |
| D | 000 |
| E | 010 |

$p(C)=0.08$ ,  $p(E)=0.1$ ,  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(B)=0.51$

# Huffman Decoding—Example

given: 000101011 and Huffman Tree

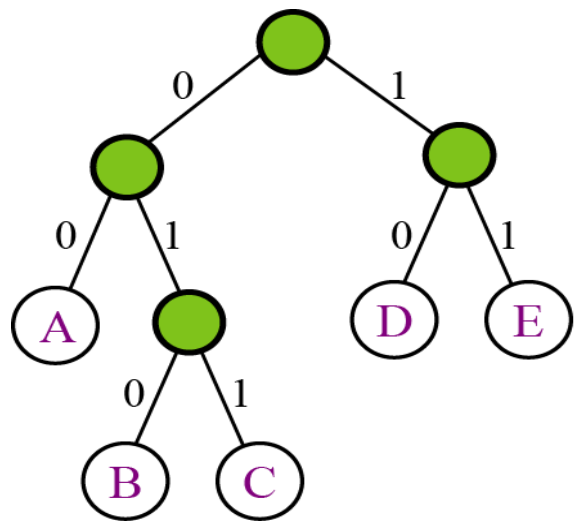


reconstruct message from bit stream using  
finite state machine

Solution: XXXYYZ

# Huffman Encoding

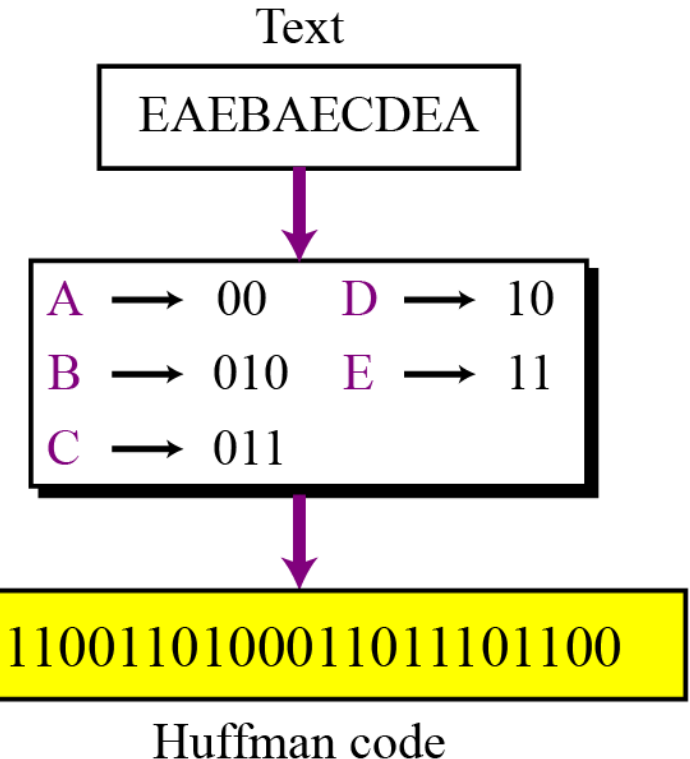
- Character code found by starting at the root and following the branches that lead to that character.
- The code itself is the bit value of each branch on the path, taken in sequence.



|        |       |
|--------|-------|
| A: 00  | D: 10 |
| B: 010 | E: 11 |
| C: 011 |       |

Code

Encoder



- Decoding: reverse process

# MP3 Bandwidth

The MP3 audio format provides lossy data compression. Audio quality improves with increasing bitrate.

- Constant Bit Rate (CBR) – constant over whole file
- Variable Bit Rate (VBR) – a bit reservoir allows distribution of bit rate over frames
  - lower bit rate for the less complex passages and a higher one for the more complex parts
  - In MP3: Average Bitrate (ABR)
- 32 kbit/s – generally acceptable only for speech
- 96 kbit/s – generally used for speech or low-quality streaming
- 128 or 160 kbit/s – mid-range bitrate quality
- 192 kbit/s – medium quality bitrate
- 256 kbit/s – a commonly used high-quality bitrate
- 320 kbit/s – highest level supported by the MP3 standard

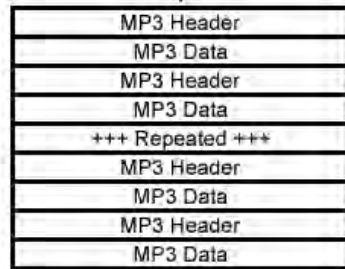




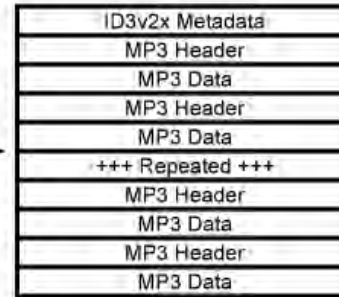
# Simple MPEG data stream

| Header | CRC | Bit Zuweisung | Skal. Faktor | Sub-band | Werte | Hilfs-daten |
|--------|-----|---------------|--------------|----------|-------|-------------|
|--------|-----|---------------|--------------|----------|-------|-------------|

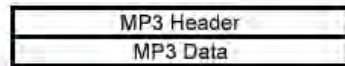
Internal Structure of an MP3 File



Note that the MP3 file structure may be 'encapsulated' within an ID3 tag.



An MP3 Frame



Example MP3 Header

FFBFA000

Colour-coding shows binary bit mapping to hex values below

Detail of an MP3 Header

| Bits    | 1             | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13       | 14           | 15 | 16 | 17               | 18 | 19 | 20 | 21 | 22 | 23         | 24 | 25 | 26 | 27 | 28 | 29                                         | 30 | 31 | 32                      |  |           |  |                   |  |  |                          |  |                   |      |                     |          |                            |  |           |  |
|---------|---------------|---|---|---|---|---|---|---|---|----|----|----|----------|--------------|----|----|------------------|----|----|----|----|----|------------|----|----|----|----|----|--------------------------------------------|----|----|-------------------------|--|-----------|--|-------------------|--|--|--------------------------|--|-------------------|------|---------------------|----------|----------------------------|--|-----------|--|
| Binary  | 1             | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1  | 1  | 1  | 1        | 0            | 1  | 1  | 0                | 1  | 0  | 0  | 0  | 0  | 0          | 0  | 0  | 1  | 0  | 0  | 0                                          | 0  | 0  | 0                       |  |           |  |                   |  |  |                          |  |                   |      |                     |          |                            |  |           |  |
| Hex     | F F F F       |   |   |   |   |   |   |   |   |    |    |    | B        |              |    | 2  |                  |    |    |    |    | 0  |            |    | 4  |    |    | 0  |                                            |    |    |                         |  |           |  |                   |  |  |                          |  |                   |      |                     |          |                            |  |           |  |
|         |               |   |   |   |   |   |   |   |   |    |    |    |          |              |    |    |                  |    |    |    |    |    |            |    |    |    |    |    | Mode Extension<br>(Used With Joint Stereo) |    |    |                         |  |           |  |                   |  |  |                          |  |                   |      |                     |          |                            |  |           |  |
| Meaning | MP3 Sync Word |   |   |   |   |   |   |   |   |    |    |    | Version  | Layer        |    |    | Error Protection |    |    |    |    |    | Bit Rate   |    |    |    |    |    | Frequency                                  |    |    | Pad. Bit                |  | Priv. Bit |  | Mode              |  |  |                          |  |                   | Copy | Original            | Emphasis |                            |  |           |  |
| Value   | Sync Word     |   |   |   |   |   |   |   |   |    |    |    | 1 = MPEG | 01 = Layer 3 |    |    | 1 = No           |    |    |    |    |    | 1010 = 160 |    |    |    |    |    | 00 = 44100 Hz                              |    |    | 0 = Frame is not padded |  | Unknown   |  | 01 = Joint Stereo |  |  | 0 = Intensity Stereo Off |  | 0 = MS Stereo Off |      | 0 = Not Copyrighted |          | 0 = Copy Of Original Media |  | 00 = None |  |

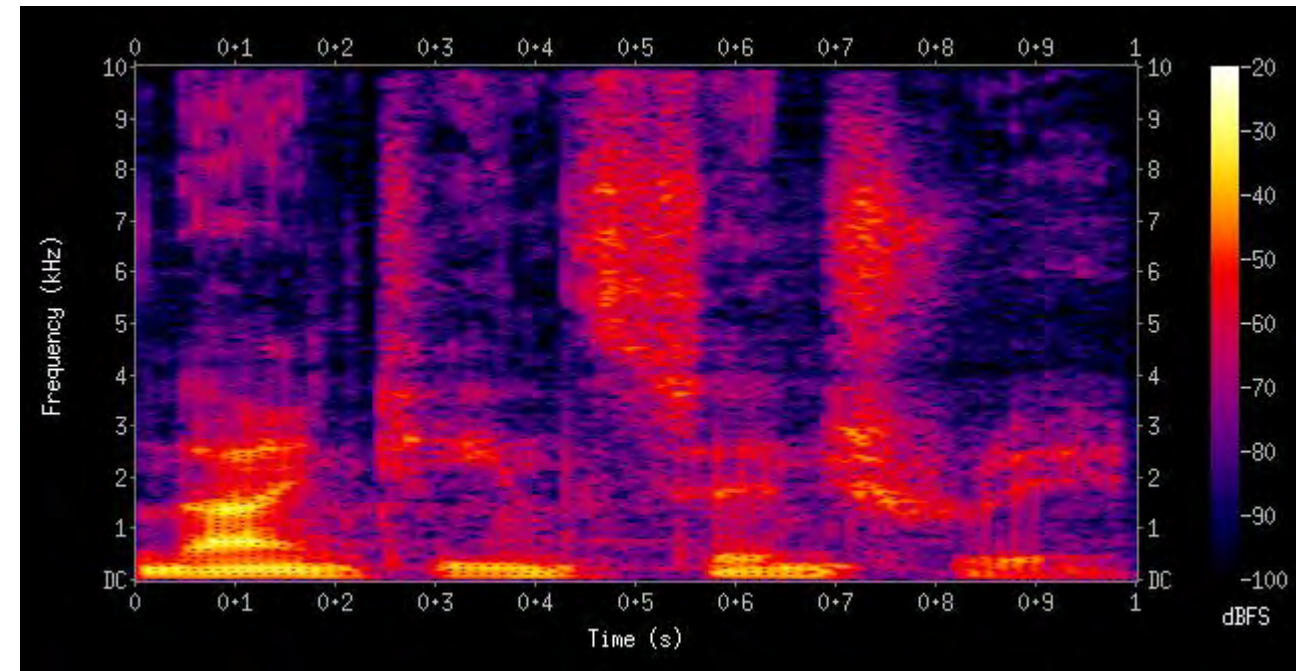
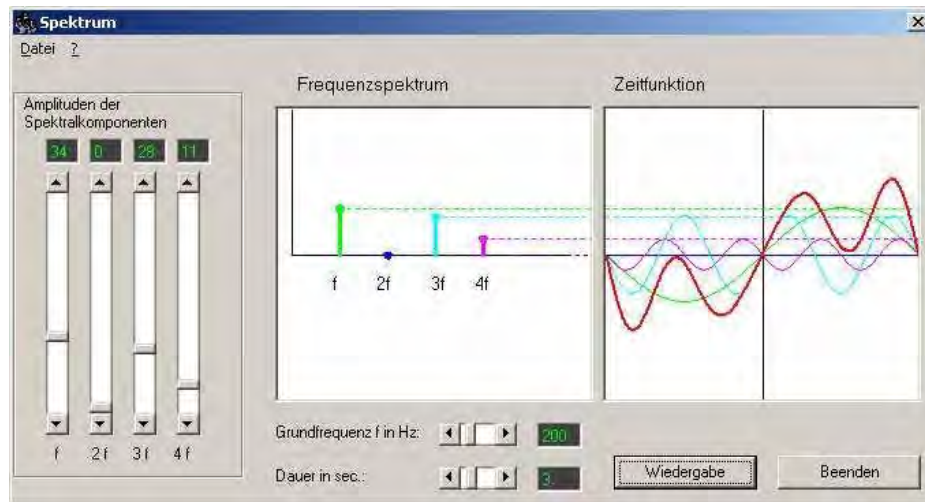
- Header - description of the data stream
- Redundancy Code (CRC) - detection of data stream errors
- Bit assignment - word length of the following subband values
- Scaling factor - 6-bit scaling factor for the following subband values
- Auxiliary data - any data can be inserted

# MP3 Further Details

- Further details the algorithm and implementation specifics (based on LAME MP3 encoder)
- <https://wiki.hydrogenaud.io/index.php?title=MP3>

# Spectrogram

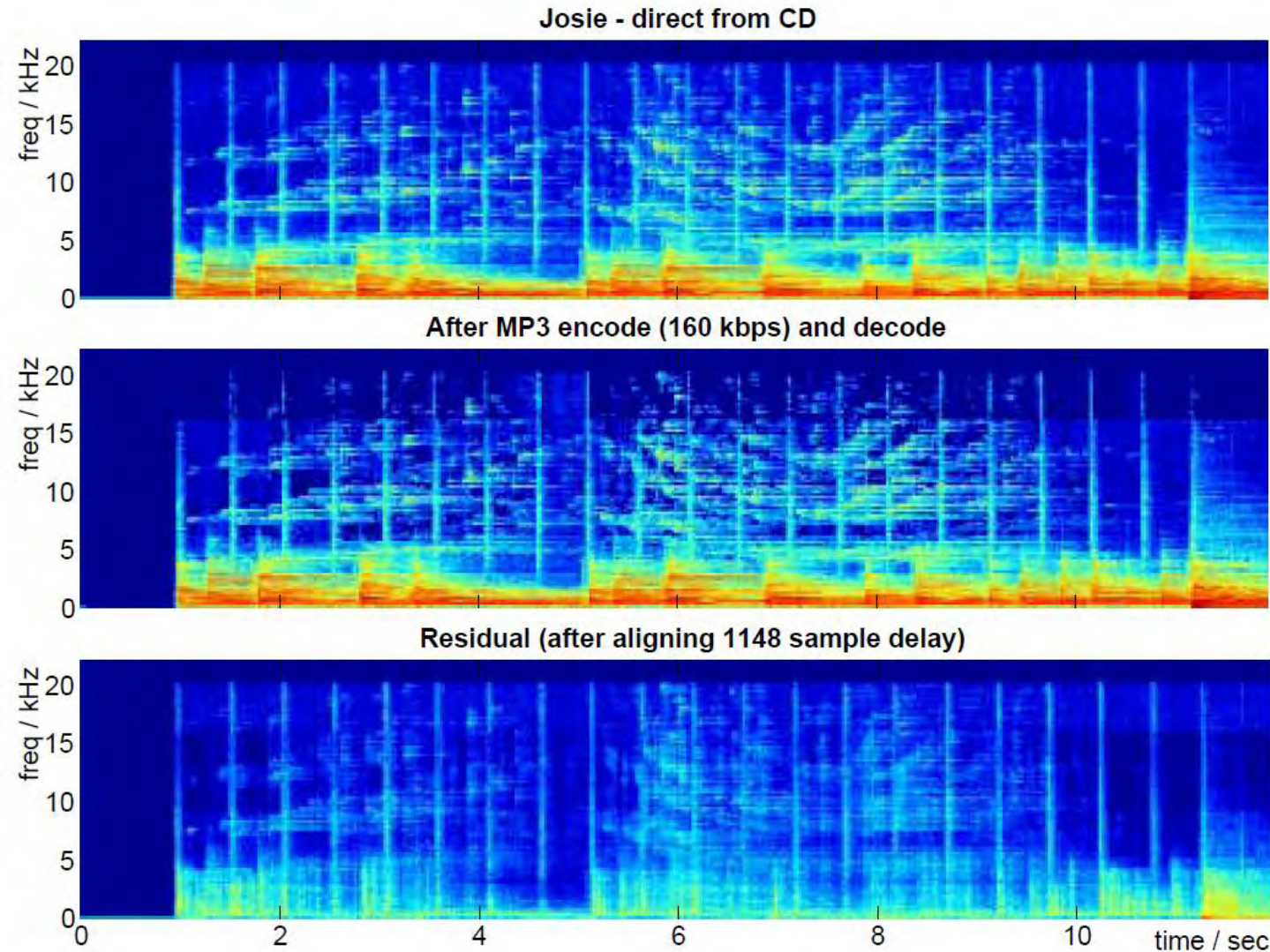
- A **spectrogram** is a visual representation of the spectrum of frequencies of a signal as it varies with time.





# The Effects of MP3

- Chop off high frequency (above 16 kHz)
- Occasional other time-frequency 'holes'
- Quantization noise under signal



# Quality Considerations

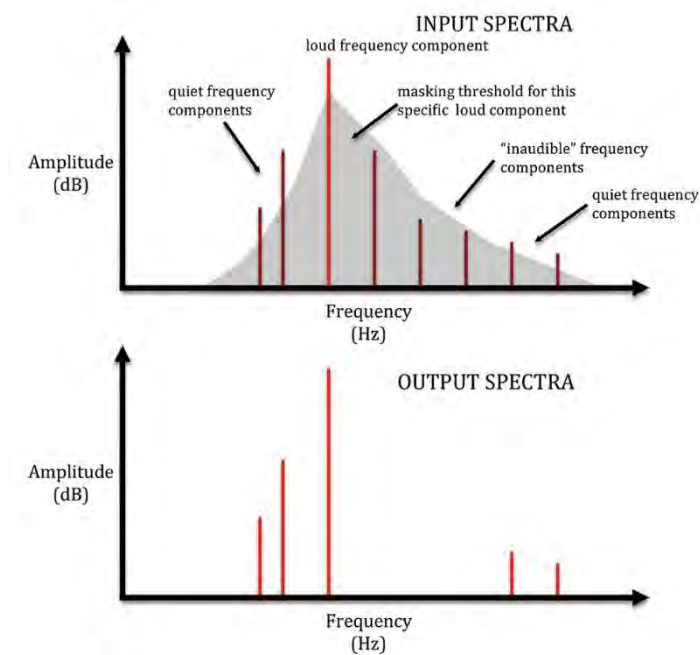
- Common types of artifacts

- loss of bandwidth (no signal >16kHz)
- pre-echoes

Name is misleading: the basic coding artifact is noise spread out over some time even before the music event causing the noise

- roughness, double-speak

at low bit-rates and lower sampling frequencies there is a mismatch between time resolution of the coder and the requirements to follow the time structure of some signals. (e.g. robot like “double-speak”)



Further examples:

<https://www.soundonsound.com/techniques/what-data-compression-does-your-music>

# How to Measure Codec Quality

- Simple objective measurement techniques
  - Are **misleading**; these methods defy the whole purpose of perceptual coding
  - Examples of such measurements: signal-to-noise-ratio, bandwidth of decoded signal

# How to Measure Codec Quality

## Listening tests

- large scale and well-controlled listening tests are still the only method available
- the aim of these tests is to stress the encoders under worst case conditions
- During the ISO-MPEG process: a number of international listening tests with trained listeners.
- tests used the "triple stimulus, hidden reference" (ABC/HR) method and the "CCIR impairment scale"



# ABC/HR Test

- listening sequence is "ABC" , with A = original, BC = pair of original / coded signal with random sequence,
- listener must first distinguish the encoded version from the original (Hidden Reference, "HR" )
- then assign a score as a subjective judgment of the quality.

# CCIR Impairment Scale

- the listener has to evaluate both B and C with a number between 1.0 and 5.0.
- meaning of these values:
  - 5.0 = transparent (this should be the original signal)
  - 4.0 = perceptible, but not annoying (first differences noticable)
  - 3.0 = slightly annoying
  - 2.0 = annoying
  - 1.0 = very annoying

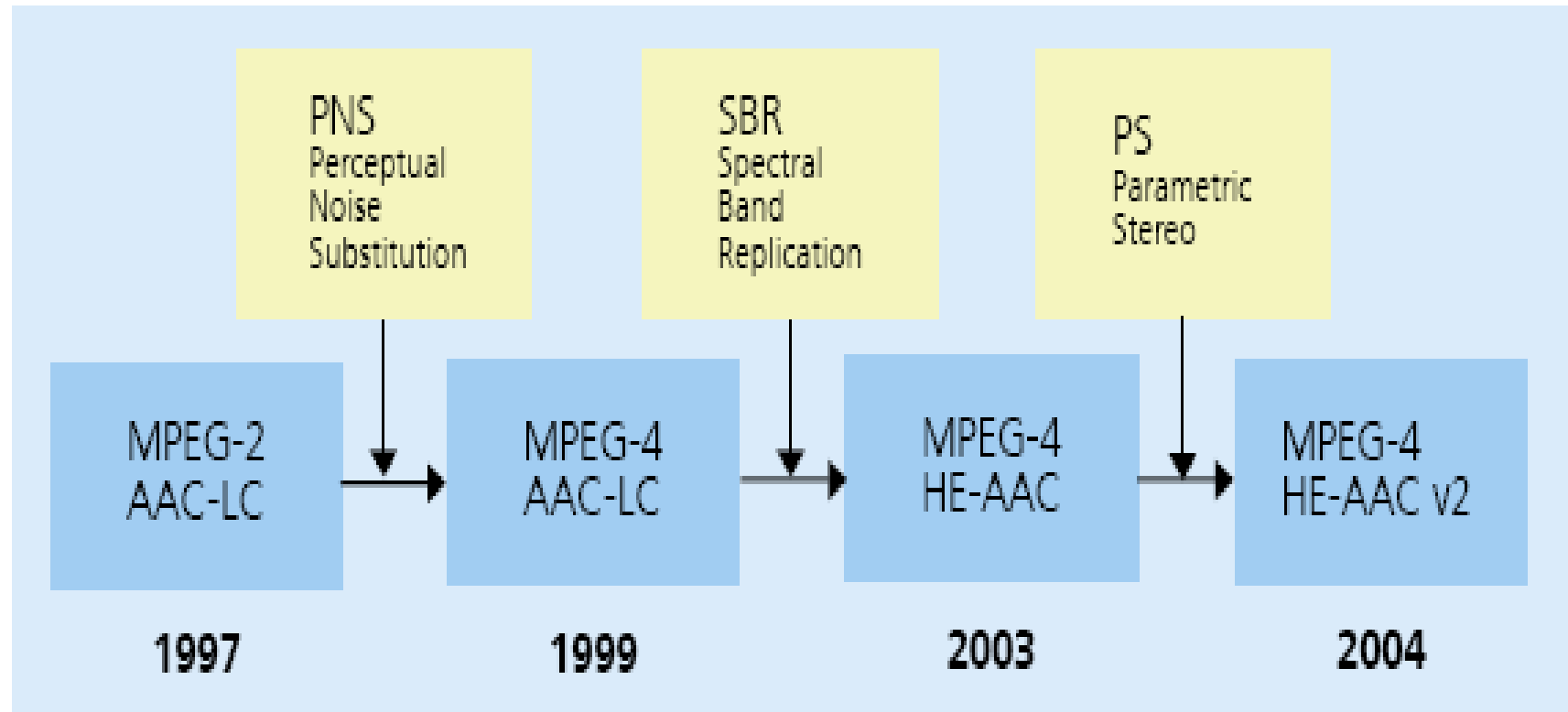
# How to Measure Codec Quality

- Perceptual Measurement Techniques (PMT)
  - state of the art not yet sufficient to make large scale and well-prepared listening tests obsolete
  - however, PMT have progressed to the point where they are a very useful supplement to listening tests and can replace them in some cases.
  - ITU-R Task Group 10/4 (Int. Telecommunications Union, Radiocommunications sector) produced a recommendation on a system called PEAQ (Perceptual Evaluation of Audio Quality)
    - a standardized algorithm for objectively measuring perceived audio quality
    - utilizes software to simulate perceptual properties of the human ear and then integrates multiple model output variables into a single metric. PEAQ characterizes the perceived audio quality as subjects would do in a listening test according to ITU-R BS.1116

# MPEG-2 Audio

- Original MPEG-2 Audio standard (1995) consists of 2 extensions to MPEG-1:
  - backwards compatible multichannel coding adds option of forward and backwards compatible coding of multichannel signals (including 5.1 channel configurations)
  - Coding at lower sampling frequencies of 16kHz, 22.05 kHz and 24 kHz
- Verification tests in 1994 showed that introducing new coding algorithms and giving up backwards compatibility promised a significant improvement in coding efficiency => AAC

# MPEG AAC FAMILY



# Advanced Audio Codec (AAC)

- successor of MP3 format
- Specified in MPEG-2 (Part 7) (1997) and MPEG-4 (Part 3) (1999)
- AAC generally achieves higher sound quality than MP3 at the same bit rate
- HE-AAC and HE-AAC v2 (in MPEG-4/Part 3)
  - extensions of the original AAC, but do not replace it
  - are optimized exclusively for low bit rates and are used, for example, for live broadcasts of digital radio and television.
  - At higher bit rates (from 96 kbit/s), they perform worse than AAC and should not be used

# AAC's Improvements over MP3 1/2

- More sample rates (from 8 to 96 kHz) than MP3 (16 to 48 kHz);
- Up to **48 channels** (MP3 supports up to two channels in MPEG-1 mode and up to 5.1 channels in MPEG-2 mode);
- Arbitrary bit rates and **variable frame length**. Standardized constant bit rate with bit reservoir;
- Higher efficiency and **simpler filter bank**. AAC uses a pure **MDCT** (modified discrete cosine transform), rather than MP3's hybrid coding (which was part MDCT and part FFT);
- Higher coding efficiency for stationary signals (AAC uses a blocksize of 1024 or 960 samples, allowing more efficient coding than MP3's 576 sample blocks);
- Higher coding accuracy for transient signals (AAC uses a blocksize of 128 or 120 samples, allowing more accurate coding than MP3's 192 sample blocks);
- Possibility to use Kaiser-Bessel derived window function to eliminate spectral leakage at the expense of widening the main lobe;



# AAC's Improvements over MP3 2/2

- Much better handling of **audio frequencies above 16 kHz**;
- More flexible joint stereo (different methods for different frequency ranges)
- Additional modules (tools) added to increase compression efficiency:  
**Temporal Noise Shaping** (TNS), backwards prediction, perceptual noise substitution (PNS), etc.
  - Modules can be combined to constitute different encoding profiles
- Overall, AAC format allows developers more flexibility to design codecs than MP3 does, and corrects many of the design choices made in the original MPEG-1 audio specification. This **increased flexibility** often leads to more concurrent encoding strategies and more efficient compression – especially at **very low bit rates** where superior stereo coding, pure MDCT, and better transform window sizes leave MP3 unable to compete.

# Advantages Over MP3

## AAC

1. Multi Channel Audio – up to 48 audio channels
2. Sample frequencies from 8KHz - 96KHz
3. Simpler filter bank (pure MDCT used)
4. Better stationary and transient response due to block sizes of 1024 and 128 samples
5. Excellent handling of high frequency signals
6. CD quality audio at 64Kbits/sec
7. Much better quality of audio at lower bit rates (down to 32Kbps)

## MP3

1. Stereo signal – maximum of only 2 channels
2. Sampling frequencies from 16KHz ~ 48KHz
3. Hybrid filter bank (requires more computational power)
4. Poorer stationary and transient response due to block sizes of 576 and 192 samples
5. Signal handling up to 15.5/15.8 KHz
6. CD quality audio at 128Kbits/sec
7. Audio quality is poorer at low bit rates and may present coding artifacts

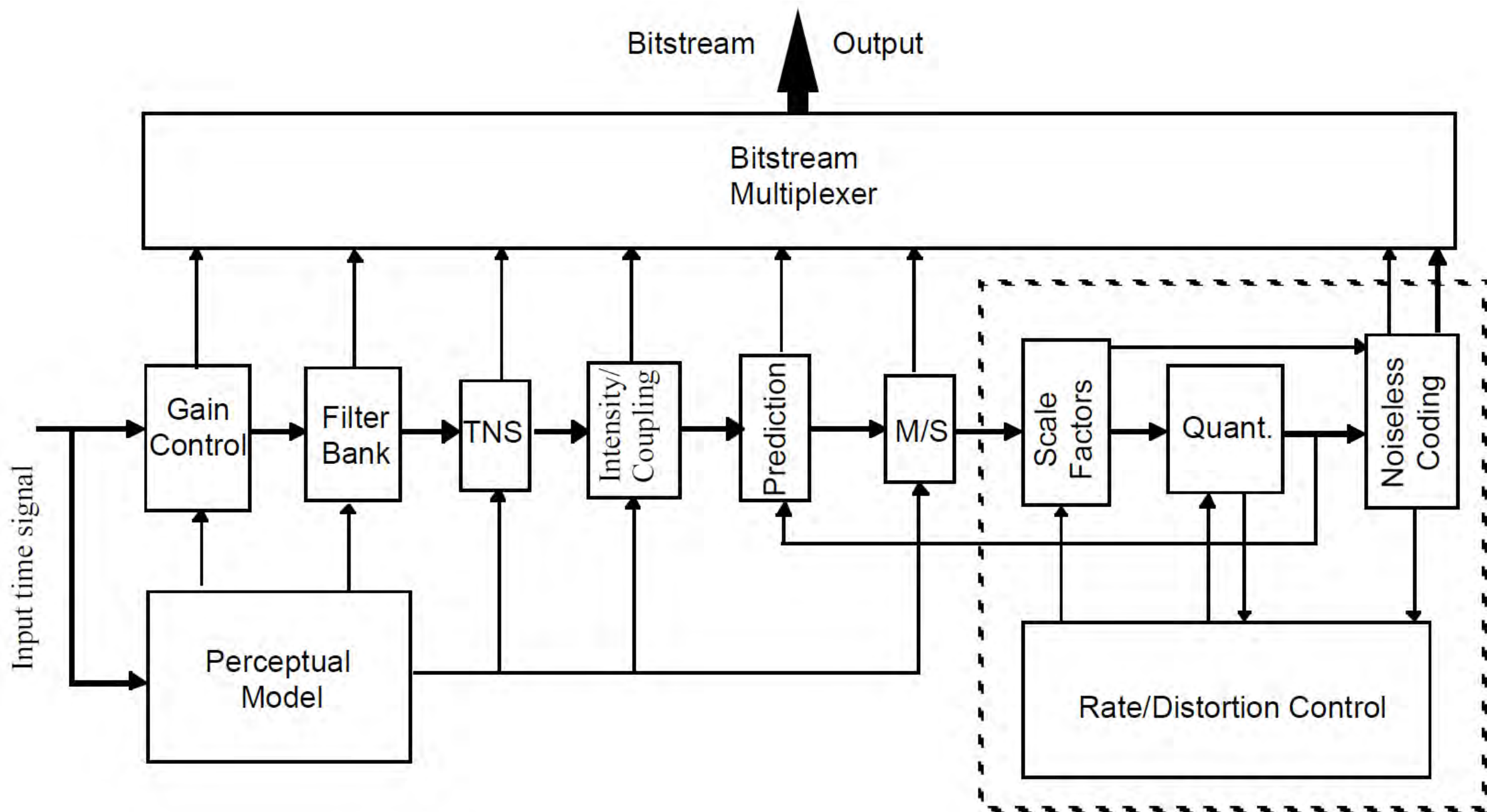


Figure 3: Block diagram of an MPEG-2 AAC encoder.

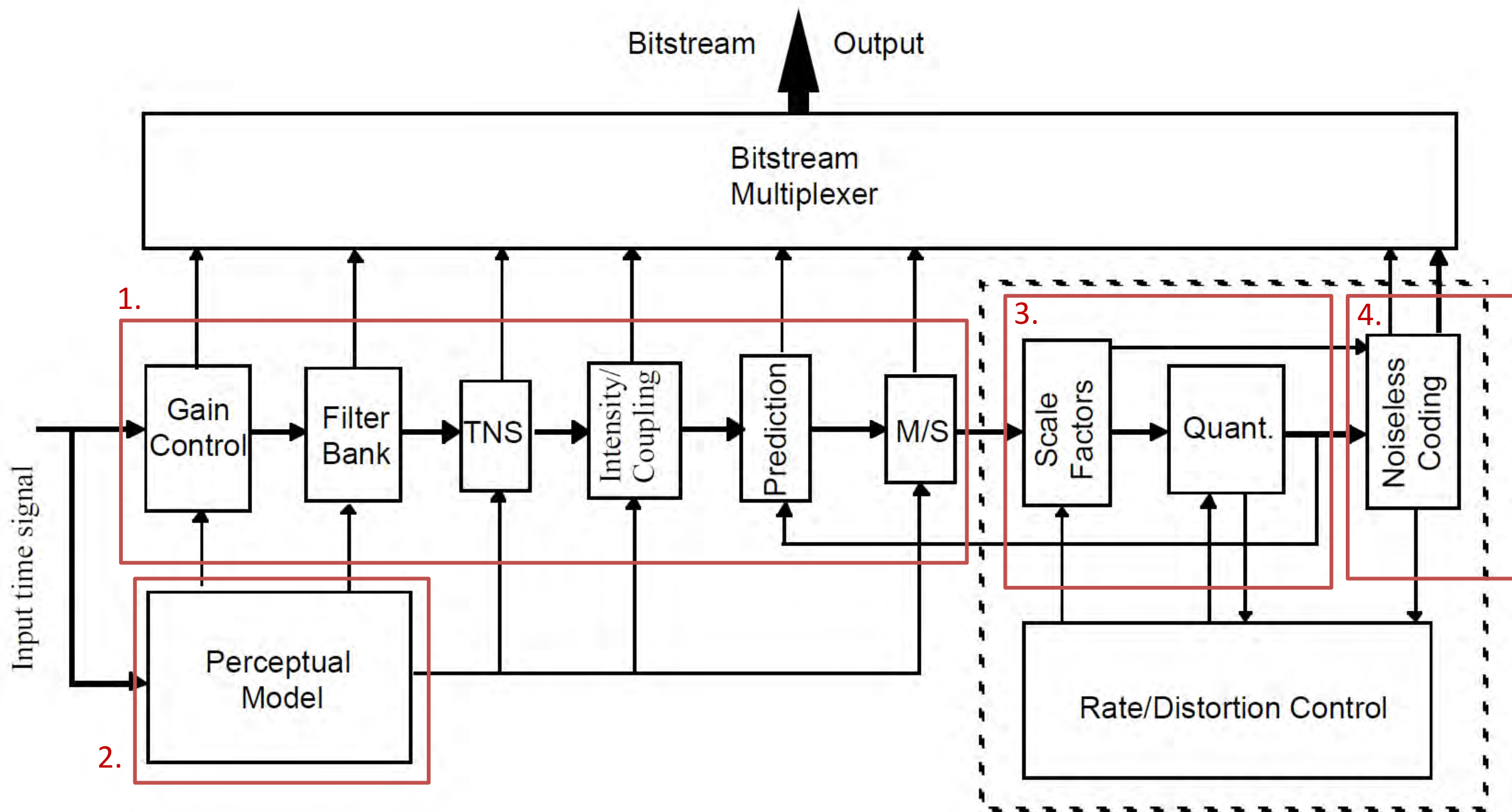
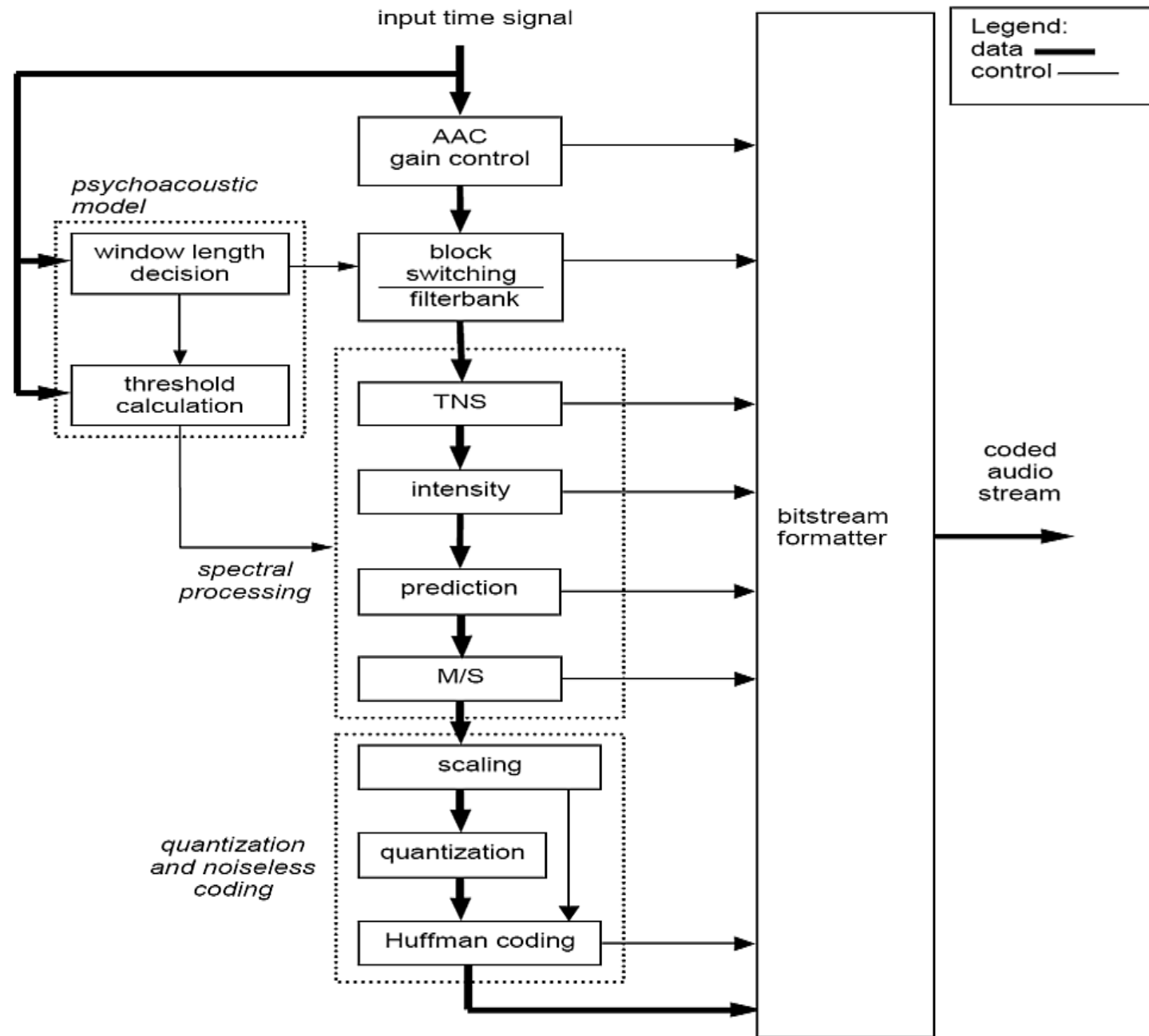
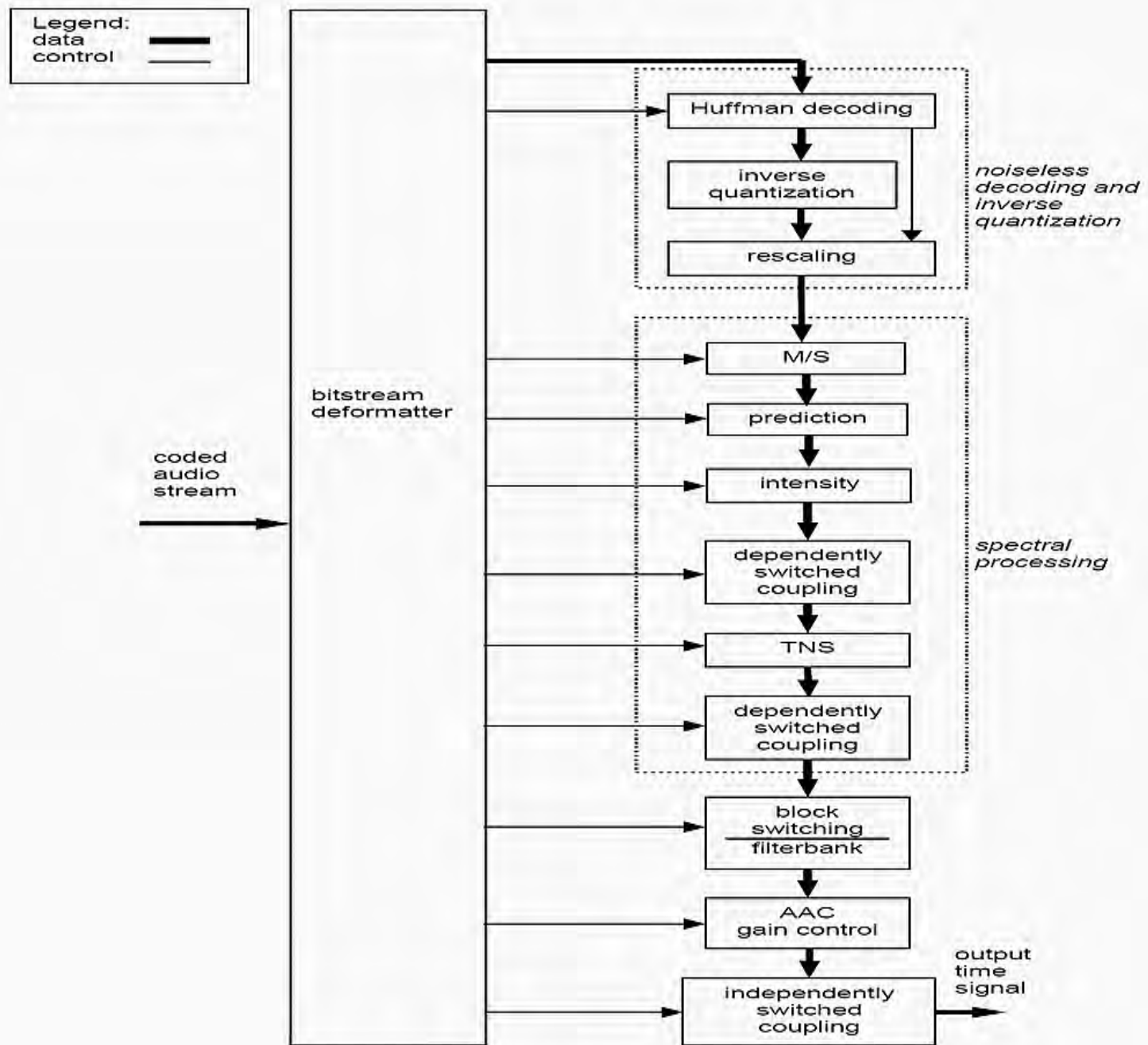


Figure 3: Block diagram of an MPEG-2 AAC encoder.



**MPEG-2 AAC Encoder Block Diagram**

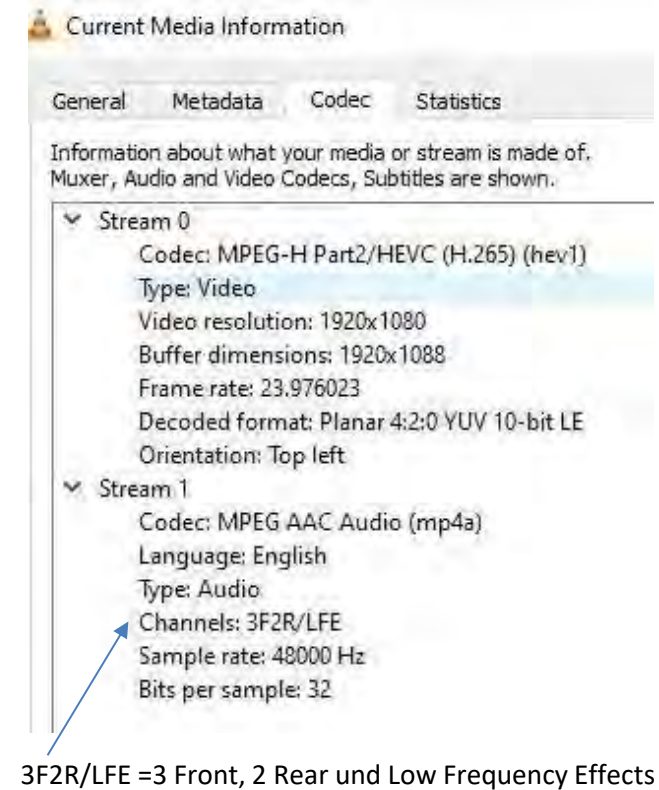


**MPEG-2 AAC Decoder Block Diagram**



# Dolby Digital 1/2

- Dolby Digital contains up to six discrete channels of sound. 5.1 mode has
  - five channels for normal-range speakers (20 Hz – 20,000 Hz) (right, center, left, right surround, left surround) and
  - one channel (20 Hz – 120 Hz allotted audio) for the subwoofer driven low-frequency effects.
  - Mono and stereo modes are also supported.
- Dolby **AC-3** (1991) multi-channel audio coding standard = *ATSC A/52* or *AC-3 Bitstream-Format (Adaptive Transform Coder 3)*
  - Basis is the modified discrete cosine transform (MDCT), a lossy audio compression algorithm. It is a modification of the discrete cosine transform (DCT) algorithm.
  - AC-3 supports audio sample-rates up to 48 kHz.
- Patents expired (since 2017), free to use





# Dolby Digital 2/2

Mögliche Kanalkonfigurationen bei Dolby Digital

| Name                | Modus                              | Vorne |                   |        | Hinten            |                    | Subwoofer<br>(LFE) | Verwendung                                                             |
|---------------------|------------------------------------|-------|-------------------|--------|-------------------|--------------------|--------------------|------------------------------------------------------------------------|
|                     |                                    | Links | Mitte<br>(Center) | Rechts | Surround<br>Links | Surround<br>Rechts |                    |                                                                        |
| Dolby Digital 1/0   | Mono                               | Nein  | Ja                | Nein   | Nein              | Nein               | Nein               | Vor allem alte restaurierte Filme.                                     |
| Dolby Digital 2/0   | Stereo                             | Ja    | Nein              | Ja     | Nein              | Nein               | Nein               | Audiokommentar. Kann auch Dolby Surround Informationen enthalten.      |
| Dolby Digital 3/0   | 3-Stereo                           | Ja    | Ja                | Ja     | Nein              | Nein               | Nein               |                                                                        |
| Dolby Digital 2/1   | Stereo mit Mono-Surround           | Ja    | Nein              | Ja     | Ja                |                    | Nein               |                                                                        |
| Dolby Digital 2/1.1 | Stereo mit Mono-Surround und LFE   | Ja    | Nein              | Ja     | Ja                |                    | Ja                 |                                                                        |
| Dolby Digital 3/1   | 3-Stereo mit Mono-Surround         | Ja    | Ja                | Ja     | Ja                |                    | Nein               |                                                                        |
| Dolby Digital 3/1.1 | 3-Stereo mit Mono-Surround und LFE | Ja    | Ja                | Ja     | Ja                |                    | Ja                 |                                                                        |
| Dolby Digital 2/2   | Quadrophonie                       | Ja    | Nein              | Ja     | Ja                | Ja                 | Nein               |                                                                        |
| Dolby Digital 2/2.1 | Quadrophonie mit LFE               | Ja    | Nein              | Ja     | Ja                | Ja                 | Ja                 |                                                                        |
| Dolby Digital 3/2   | 5-Kanal-Surround                   | Ja    | Ja                | Ja     | Ja                | Ja                 | Nein               |                                                                        |
| Dolby Digital 3/2.1 | 5-Kanal-Surround mit LFE           | Ja    | Ja                | Ja     | Ja                | Ja                 | Ja                 | Meistens als Haupttonspur bei Spielfilmen oder auch Serien eingesetzt. |



Extensions to 7.1 (2003) and 9.1 (2006) speaker configurations, resulting in Dolby Surround.

# Spatial Audio

# Binaural Audio

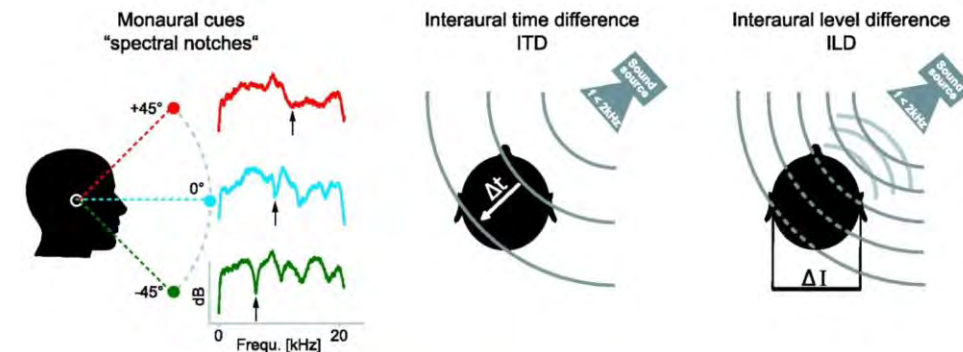
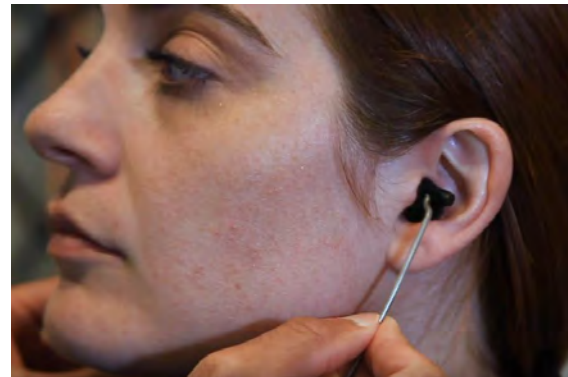
- Spatialize (=render) audio in 3D for headphones
  - more advanced processing than stereo
  - Sends individual signals to left/right ear
  - Uses psychoacoustics to spatialize sound



<https://www.youtube.com/watch?v=FMdabRXWwDE&list=PL7w4cOVVxL6FTg6i9osmlWK-w2wU4xvI2&index=5>

## Head Related Transfer Function (HRTF)

- characterizes how an ear receives a sound from a point in space
- Different for every individual

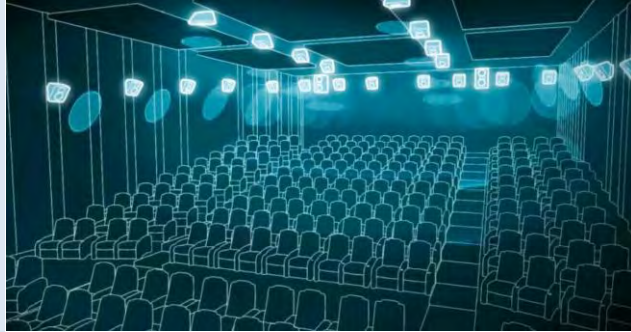


<https://www.youtube.com/watch?v=cB-IW9KKu4>

<http://physrev.physiology.org/content/90/3/983.long>

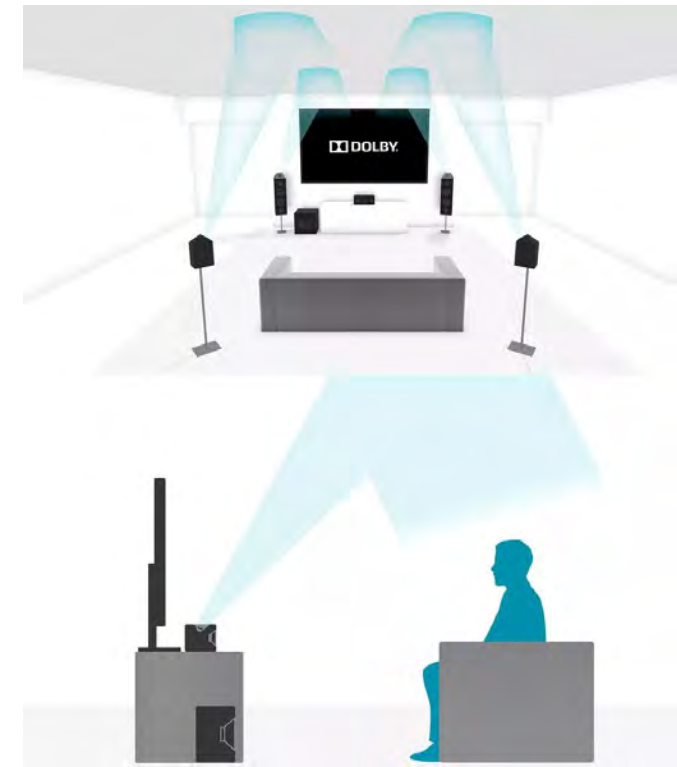
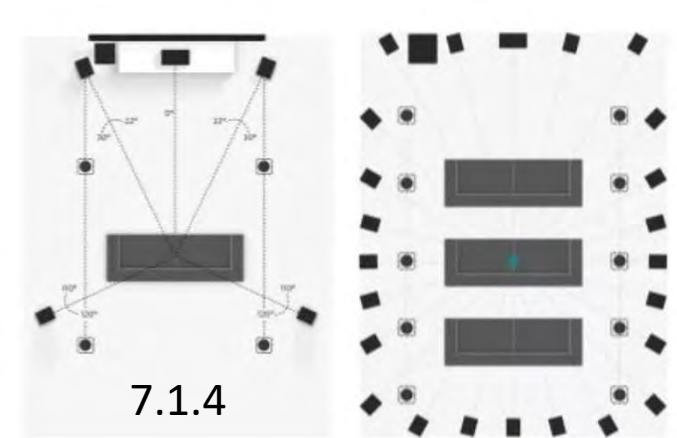
Example: Virtual Barber Shop  
<https://www.youtube.com/watch?v=IUDTlvagjJA>





# Dolby Atmos

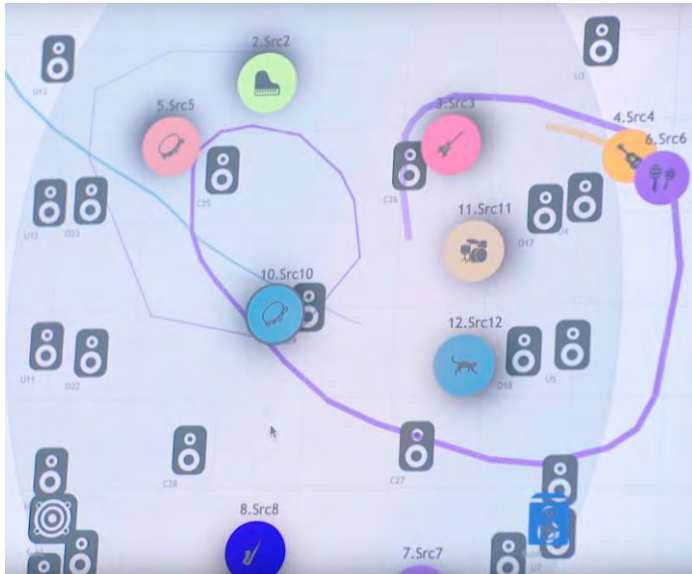
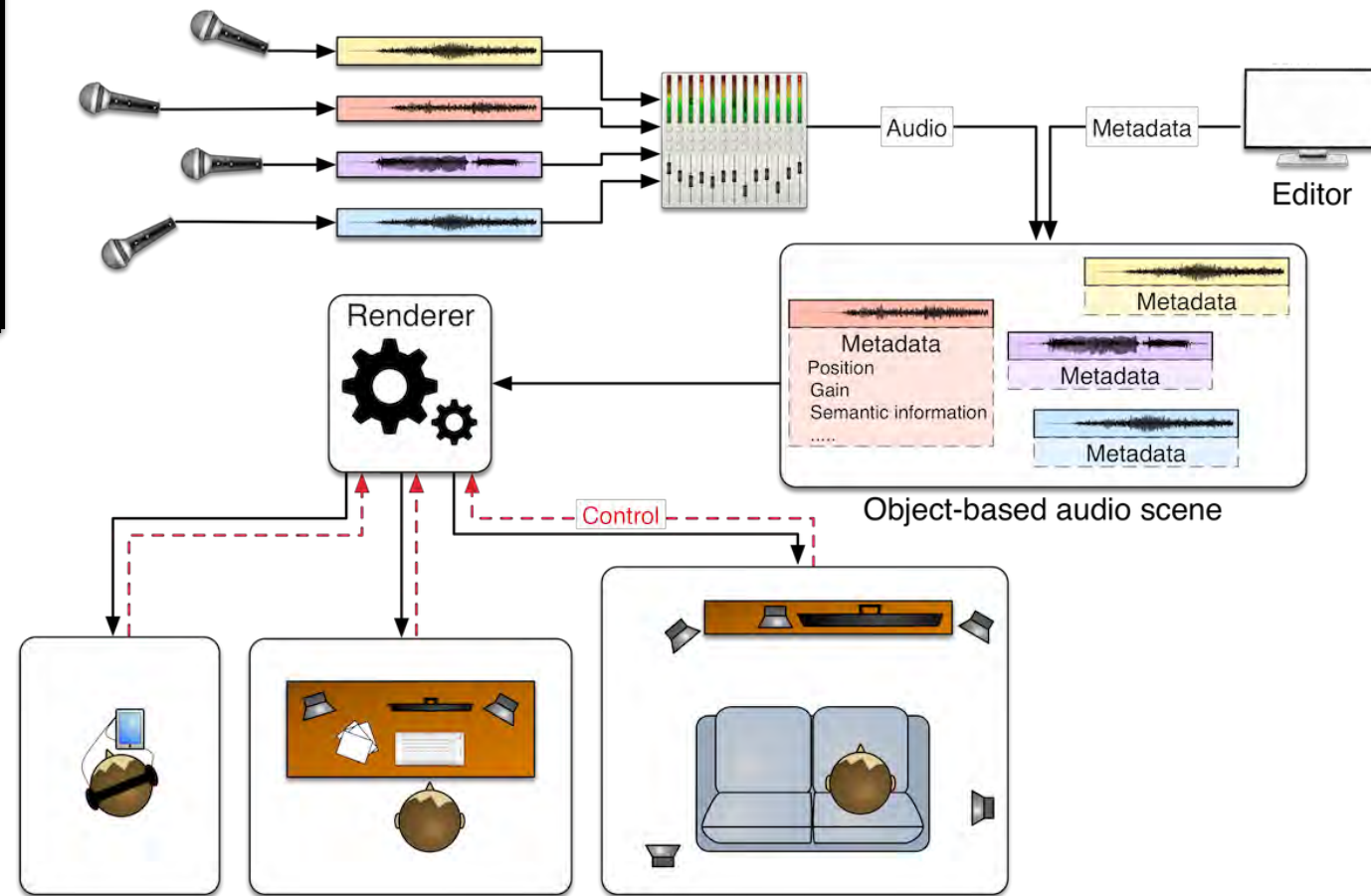
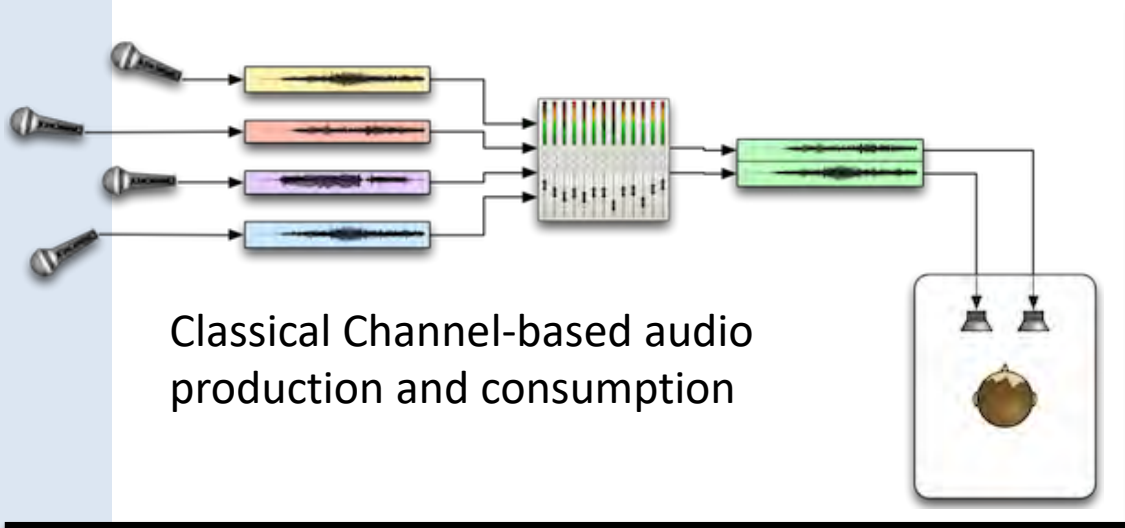
- Up to 128 **audio objects** (or channels)
- For home theaters typically 5.1 or 7.1 base channels with overhead or “Dolby Atmos speakers” (projecting to ceiling) e.g. 7.1.4 (7.1 + 4 overhead)
- Audio objects allow specification of location of sound – rendered in real time based on speaker positions
- Dolby-TrueHD-Codec – **lossless** – based on
  - Meridian Lossless Packing, also known as Packed PCM (PPCM) or Dolby Lossless for compressing PCM audio data
    - 1.5-3 : 1 compression rates (uses redundancy in multiple channels); incl. error correction up to 2ms
    - 44.1-192 kHz, up to 24 bits, 64 channels



# Audio Objects



<https://www.youtube.com/watch?v=Y79Q1sfvNmc>

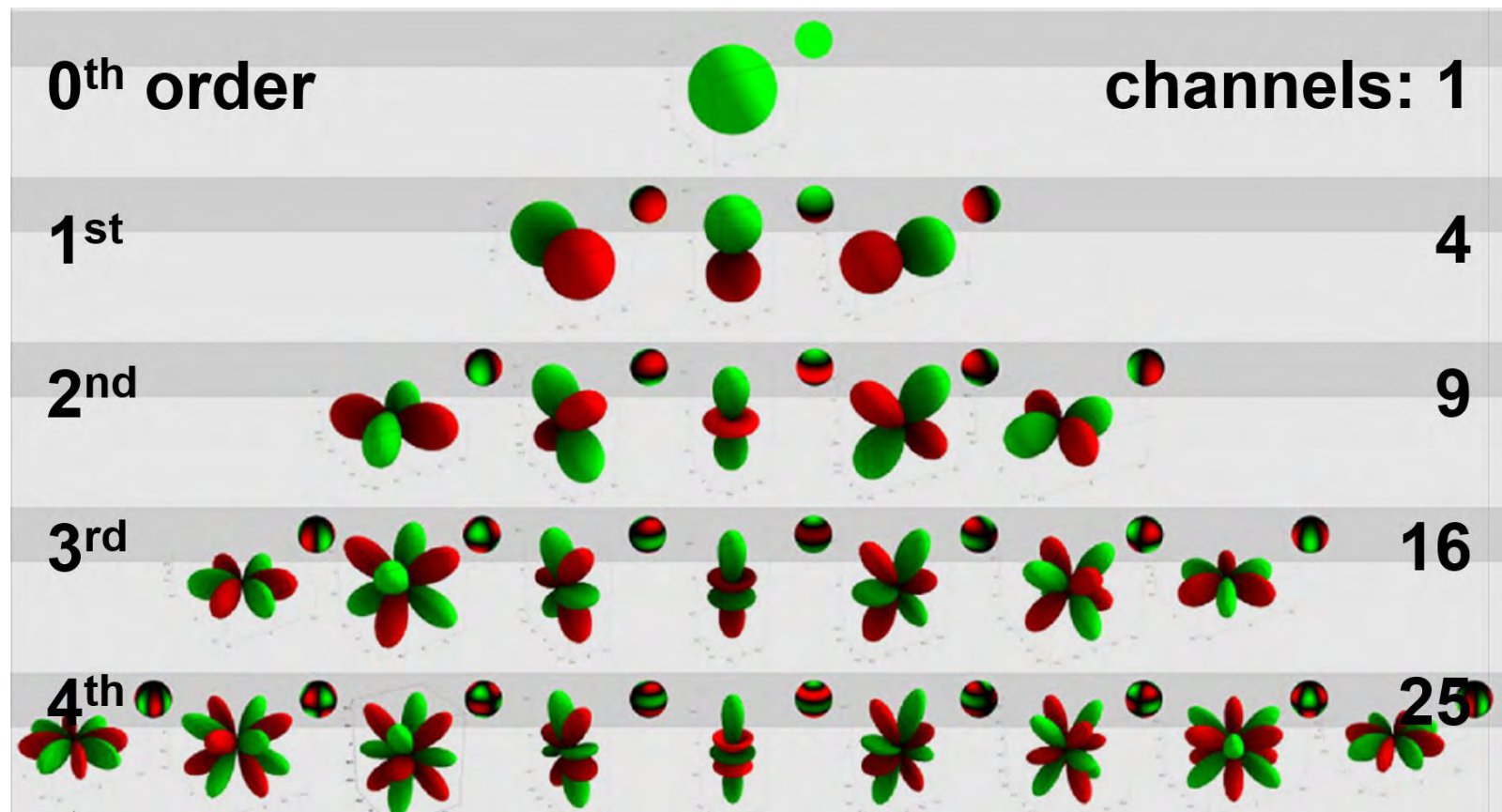


<https://www.youtube.com/watch?v=cqC1rWSxT4c>

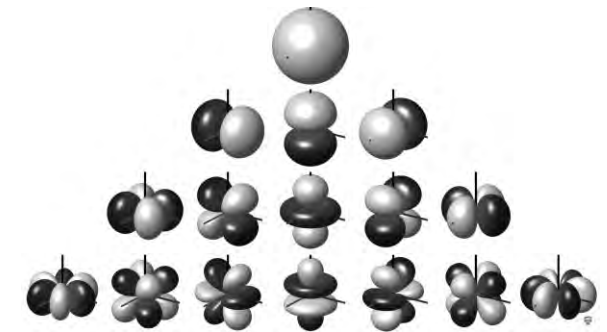


# Spatial Audio: Higher-Order Ambisonics

- Full-sphere surround sound format



3<sup>rd</sup> order Ambisonics microphone  
19 microphone omnidirectional  
digital MEMS capsules on a sphere



<https://www.youtube.com/watch?v=9m4VxFSYSEA>



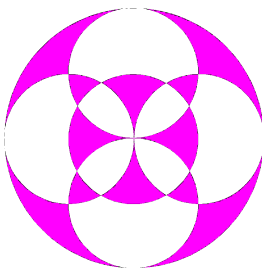
# Mono, Stereo, 3<sup>rd</sup> order Ambisonics compared



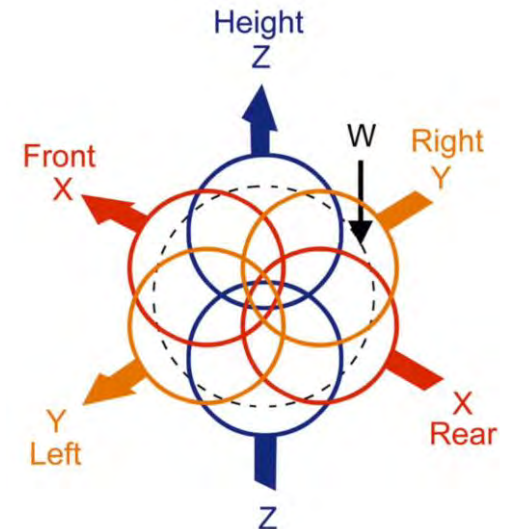
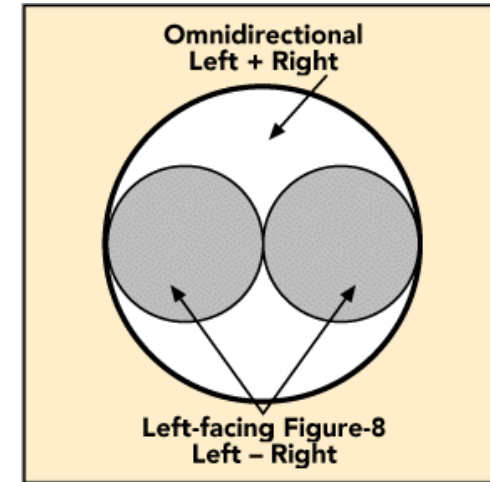
[https://www.youtube.com/watch?v=an9N4CKCO\\_o](https://www.youtube.com/watch?v=an9N4CKCO_o)



# Principles of Ambisonics



- Taking mid-side (M-S) coincident pair scheme into three dimensions
- **A-Format** Recording: All individual microphone channels
- **B-Format**: Processed individual channels e.g.
- 1<sup>st</sup> order Ambisonics **B-Format**:
  - $W = \text{Omni (F+B+L+R+U+D)}$
  - $X = \text{Front} - \text{Back}$
  - $Y = \text{Left} - \text{Right}$
  - $Z = \text{Up} - \text{Down}$
- These signals are symbolized in the Ambisonics logo
- 2<sup>nd</sup> order Ambisonics uses 9 channels
- 3<sup>rd</sup> order Ambisonics jumps up to 16 channels
- ... 6<sup>th</sup> order Ambisonics with 49 channels



# What Is Ambisonics?

- “Ambisonics” means “Surround Sound”
- Surround system developed by a team of British researchers in the 1970s, notably the late Michael Gerzon. All patents expired.
- Designed to bring the concert hall into the living room, by accurately capturing and replaying a soundfield on practical speaker arrays
- Four channels (B-Format) (in case of 1<sup>st</sup> order Ambisonics) capture entire soundfield – including height (“Periphony”)
- Transmission systems compatible with stereo (UJJ) and modern “discrete” surround (G-Format)
- Ambisonic production can use special microphone (Soundfield mic) or multitrack sources

# The Kirchhoff-Helmholtz Integral

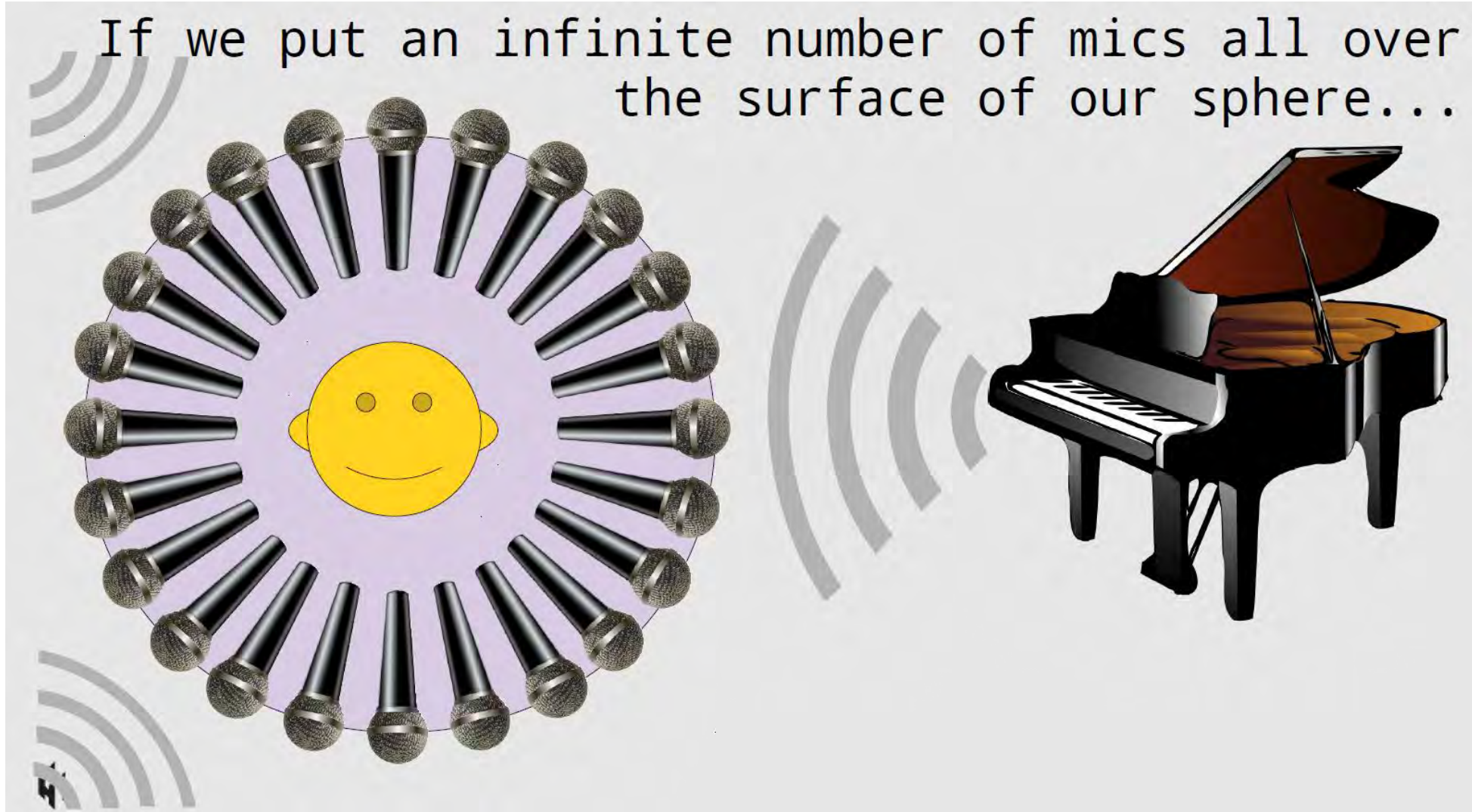
= If you can record the boundary,

"If you know the sound pressure and velocity in any point on the surface of a volume free of sources, you have complete knowledge of the sound field inside."

you can reproduce the inside.

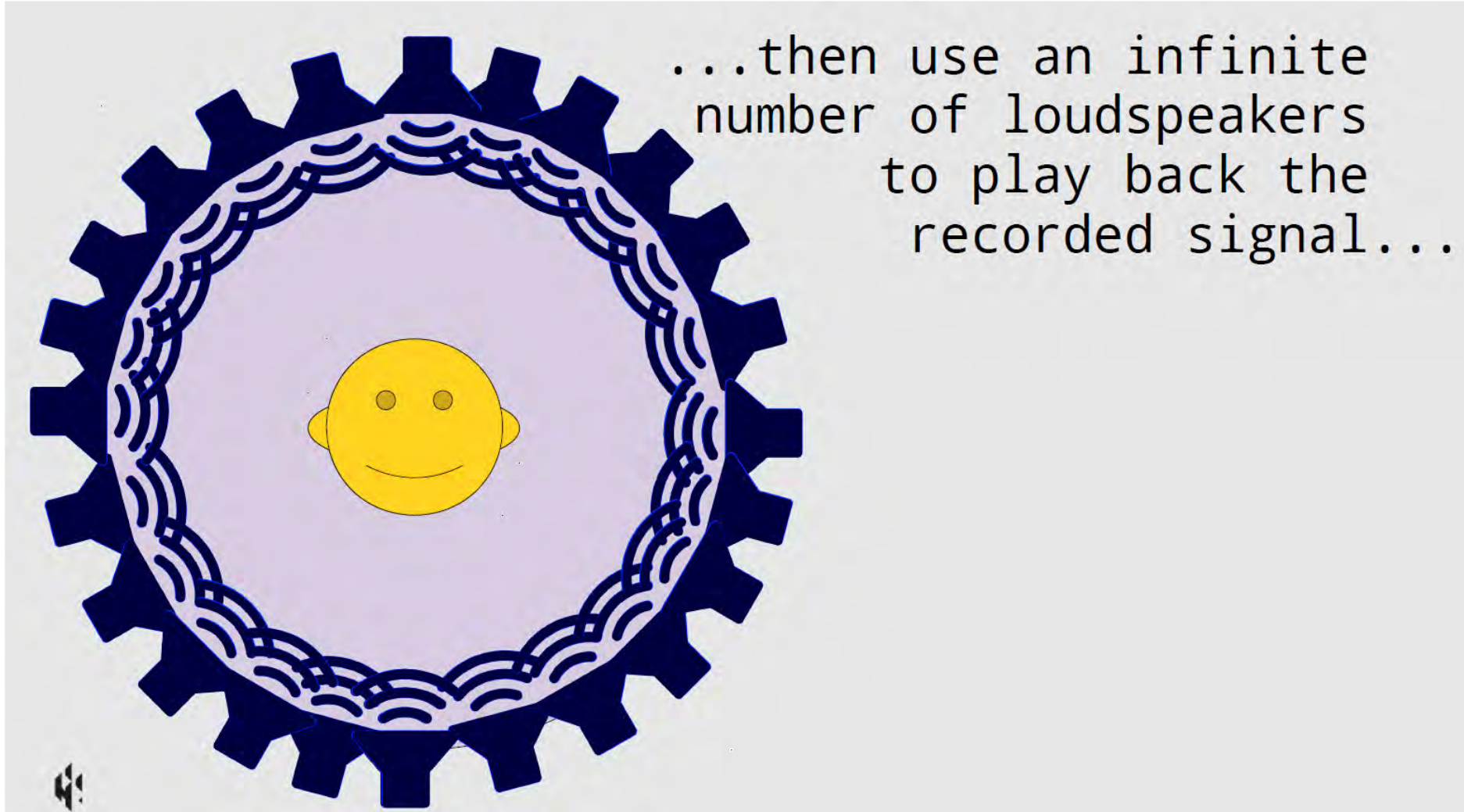
# The Kirchhoff-Helmholtz Integral

If we put an infinite number of mics all over the surface of our sphere...



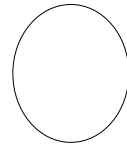
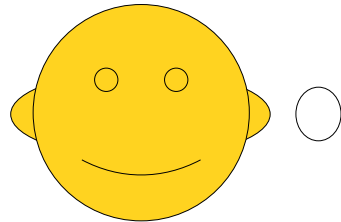


# The Kirchhoff-Helmholtz Integral



# The Kirchhoff-Helmholtz Integral

...the result will be  
*identical* to the  
original performance.



# Sampling the Sphere

The systematic approach:

Find a set of microphones that cover the sphere completely.

Avoid redundant information.

- Omnidirectional microphone = 0<sup>th</sup> order component
- **W** is the pressure component.  
Remains when Ambisonics is played back in mono.

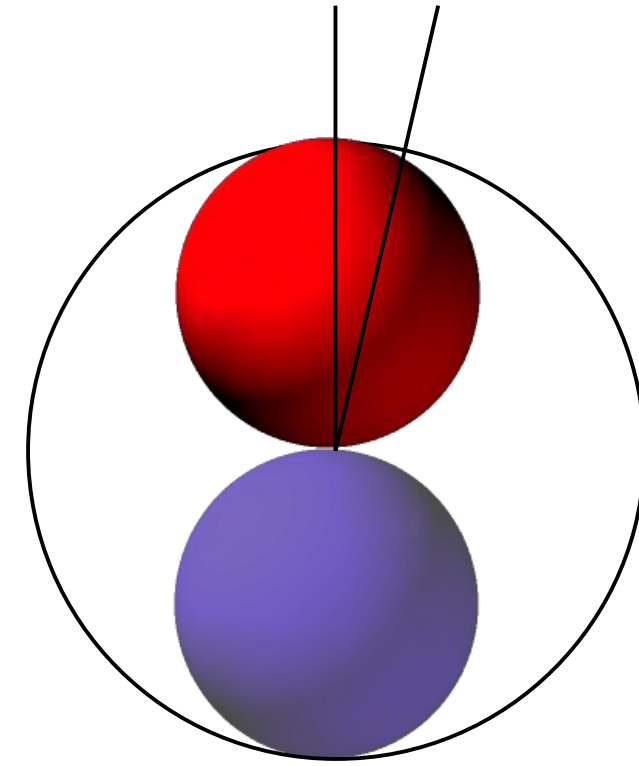
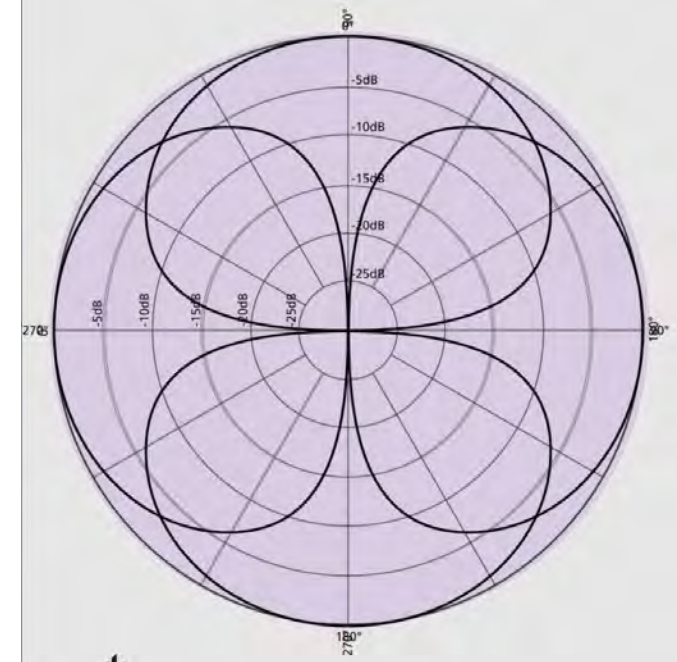


by Jörn Nettingsmeier, <http://stackingdwarves.net>



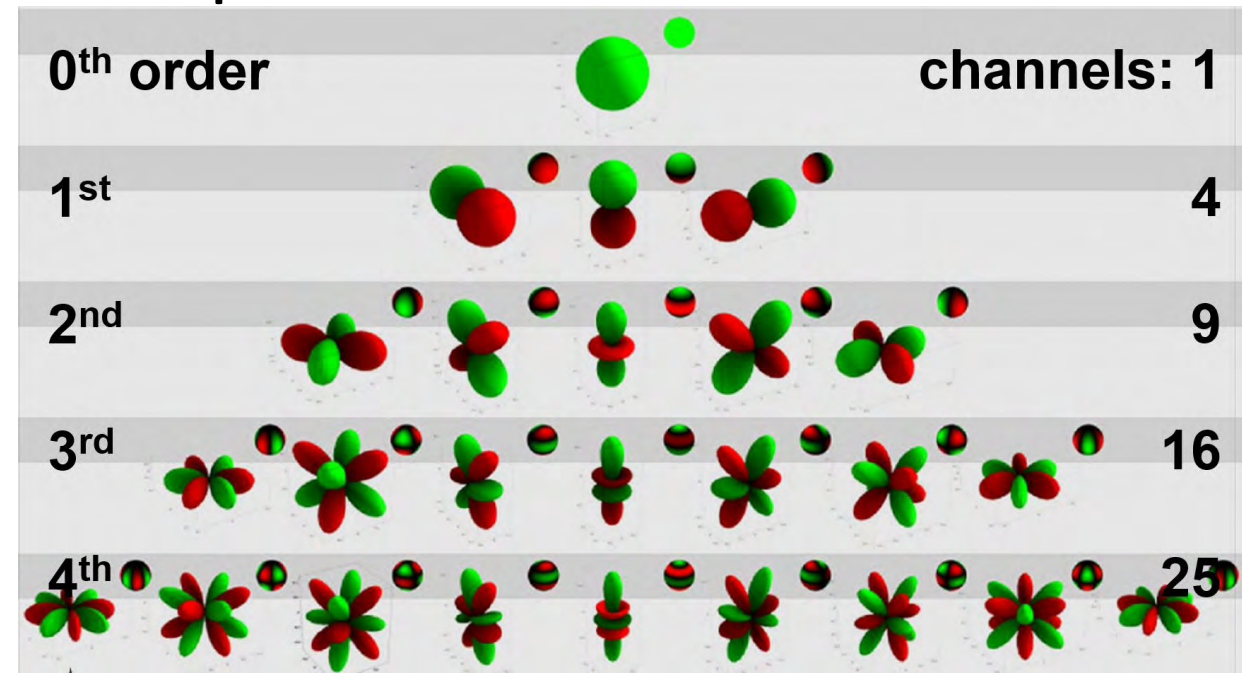
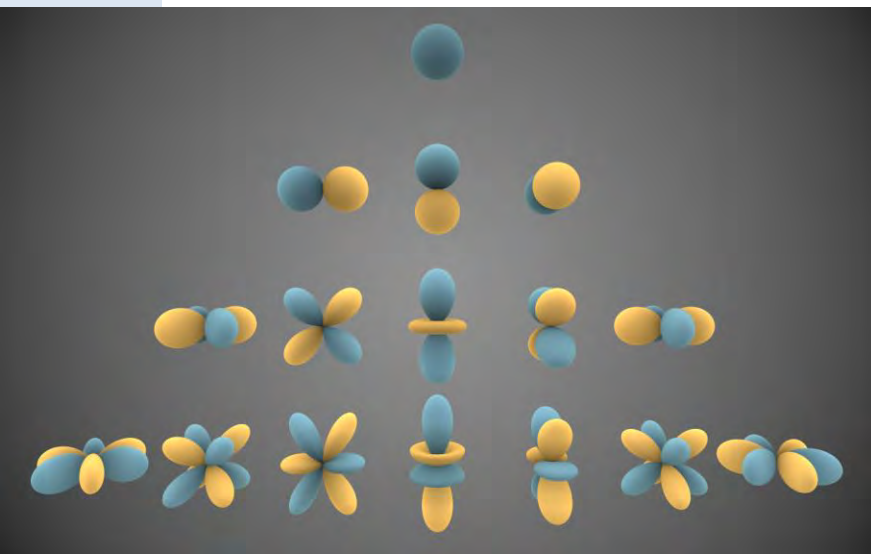
# 1<sup>st</sup> order Ambisonics

- Three figures-of-eight will also cover the sphere uniformly.
- Together with W, they provide directional information.
- In the B-format, they are denoted as X,Y,Z
- What's wrong with 1<sup>st</sup> order Ambisonics?
  - Low angular resolution:  
off-axis sources have a shallow roll-off.
  - In practice, this means a small sweet spot  
(= listening area)
- So we need narrower polar patterns which also sample the sphere uniformly and linearly independent



# Spherical Harmonics: Higher-order Ambisonics

- **Spherical harmonics** are a set of functions used to represent functions on the surface of the **sphere**.
- They are a higher-dimensional analogy of Fourier series, which form a complete basis for the set of periodic functions of a single variable.



# Ambisonics Defined

“**Ambisonics** goes further than stereo in that what it does... is to sample the acoustic field in such a way that the combination of the signals from all the loudspeakers in the array produces... a reconstruction of the original acoustic wave field (both traveling and standing wave components).

“If a listener puts their head in the sound field, because the wavefronts are similar to the original, the perception of directionality and space should correspond to the original too. It is a ‘wavefront reconstruction’ scheme in the small. The ear signals (crosstalk and all) will be correct if the reconstructed wavefronts are correct. This is just like natural hearing.

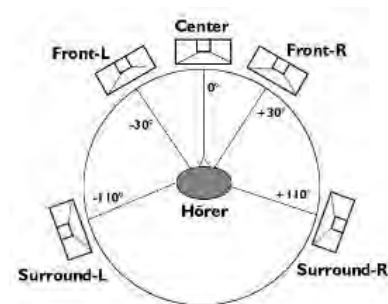
“Increasing the number of loudspeakers in Ambisonics (each fed its correctly-decoded signal) increases the accuracy of the reconstruction and the region over which it holds up. The interaural arrival-time differences also correspond to natural hearing in Ambisonics.”

—*Stanley Lipshitz*

# 5.1 and Ambisonics - Compared

## 5.1 surround

- Localization achieved solely with level
- Poor inter-speaker imaging except at front
- Localization varies with listener position
- Small sweet spot
- Horizontal surround uses six channels
- Speakers in special positions (e.g. ITU)
- One-to-one channel relationship from studio to speakers



ITU-Circle

## Ambisonics

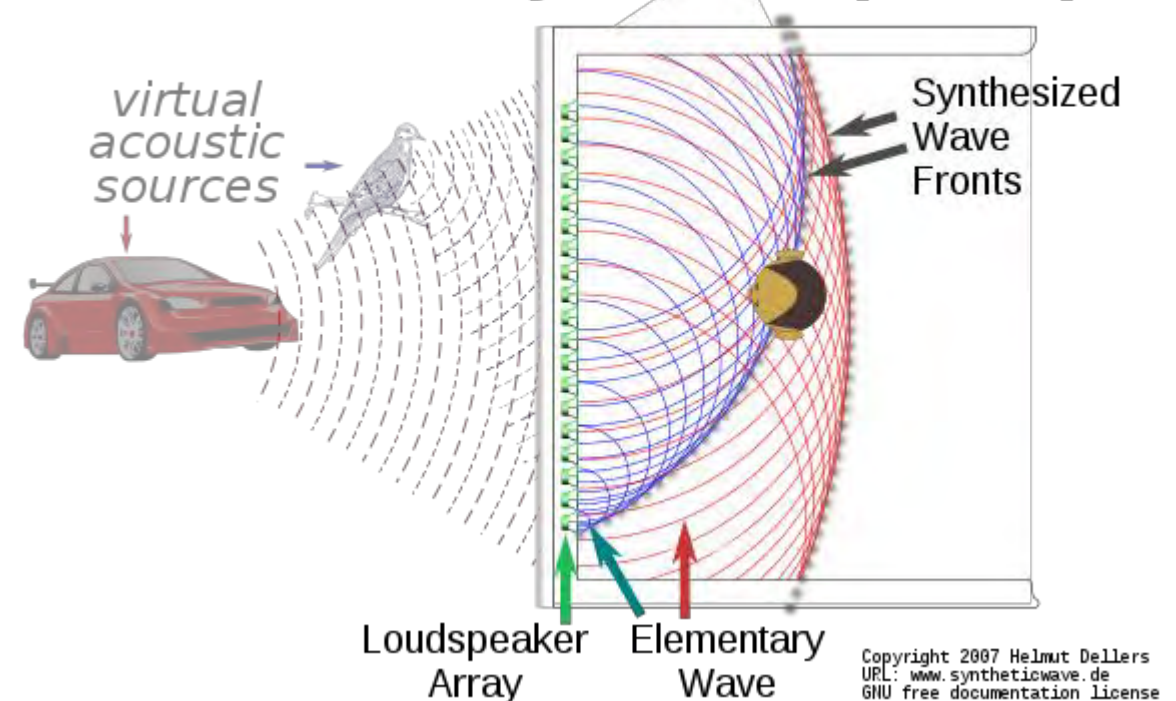
- Localization includes other mechanisms
- Good inter-speaker imaging all round
- Less variation of image with position
- Larger sweet spot
- Even with height, only four channels needed (1<sup>st</sup> order)
- Decode to 5.1 or other configurations
- Recover B-Format and decode for completely flexible speaker arrays



# Spatial Audio: Wave Field Synthesis (WFS)

- A spatial audio rendering technique, characterized by creation of virtual acoustic environments
- Produces artificial wavefronts synthesized by a large number of individually driven loudspeakers.
- The localization of virtual sources in WFS does not depend on or change with the listener's position

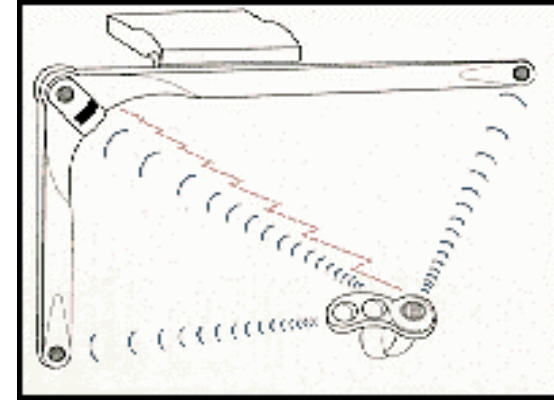
## Wave Field Synthesis principle



# MPEG-H 3D Audio (2013 - )

- Supports coding audio as **audio channels**, **audio objects**, or **higher order ambisonics** (HOA)
- Can support up to **64 loudspeaker channels** and **128 codec core channels**
- Audio objects may be used alone or in combination with channels or HOA components
- Use of audio objects allows **interactivity** or personalization of a program by adjusting gain or position of the objects during rendering in the decoder
- Audio is encoded using an improved modified discrete cosine transform (MDCT) algorithm
- Transmit immersive sound as well as mono, stereo, or surround sound.
- MPEG-H 3D Audio decoder renders bitstream to a number of standard speaker configurations as well as to misplaced speakers.
- **Binaural rendering** of sound for headphone listening is also supported
- Used since May 2017 in terrestrial South Korean UHD TV broadcast

# Audio Processing Examples

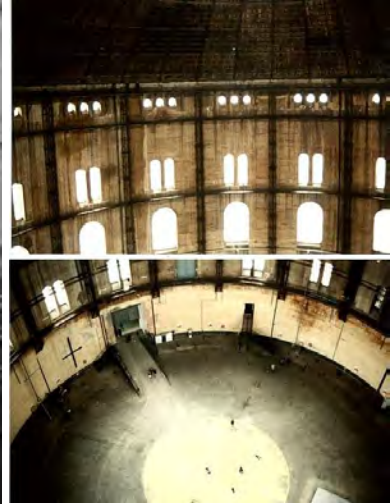


- reshape impulse response to simulate a different room
- change perceived location from which sound comes
- locate speaker in 3D space using microphone arrays
- acoustic tracking
- cover missing samples
- echo cancellation
- Spatial audio (multi-speaker setups)
  - Audio objects
  - High number of channels





# In Retrospect: Importance of Space “Delay Maschine” Gasometer





# Multimedia

## Vorlesung

Hannes Kaufmann

Research Unit Virtual and Augmented Reality  
Institute of Visual Computing & Human-Centered Technology

Based on material by Prof. Christian Breiteneder, a few slides by Prof. Robert Sablatnig (EVC)  
Material on HDR by Scott Janus, Principal Engineer at Intel; Thuong Nguyen [tnguyen@humaxdigital.com](mailto:tnguyen@humaxdigital.com); Prof. Marc Pollefeys and Dr. Gabriel Brostow at ETHZ  
A few slides on JPEG used by courtesy of Dr. Dirk Thißen from RWTH Aachen

# Media Types

- History, representation, formats and operations of media types
- Text
- Audio
- Image
- Video
- No established digital standards for other media types:  
smell, taste, haptics

# Media Type: Image

- Human Visual System & Color Models
- High Dynamic Range
- Image Compression



# Human Visual System

Helligkeitssehen:

~120 Mill. "Stäbchen"/Rods

Farbsehen:

~7 Mill. "Zapfen"/Cones



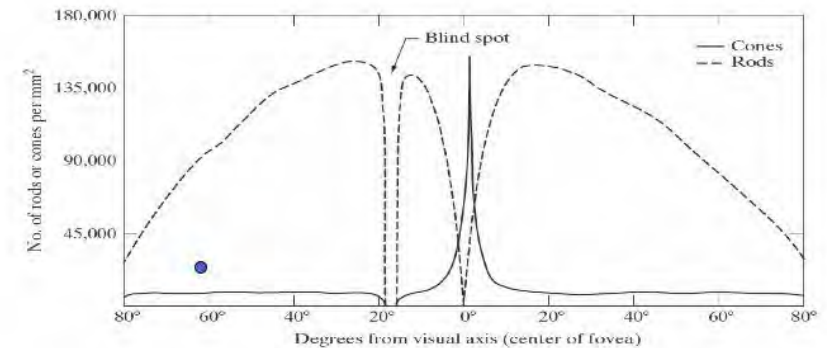
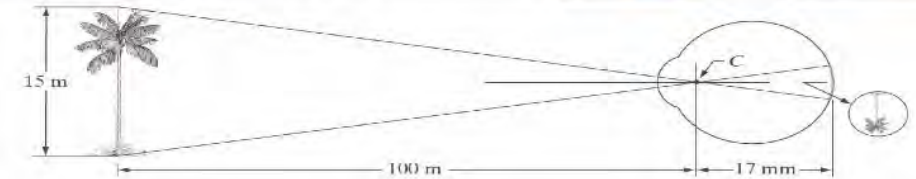
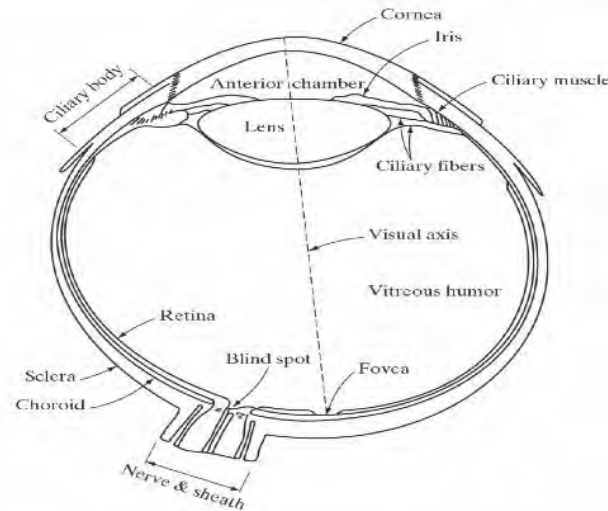
*Digital Image Processing, 3rd ed.*

Gonzalez & Woods

[www.ImageProcessingPlace.com](http://www.ImageProcessingPlace.com)

Chapter 2

Digital Image Fundamentals



© 1992–2008 R. C. Gonzalez & R. E. Woods

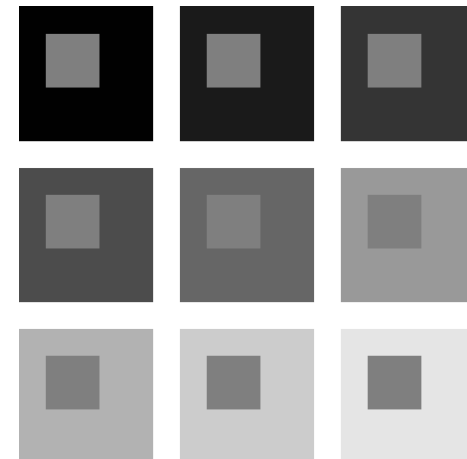


# Light

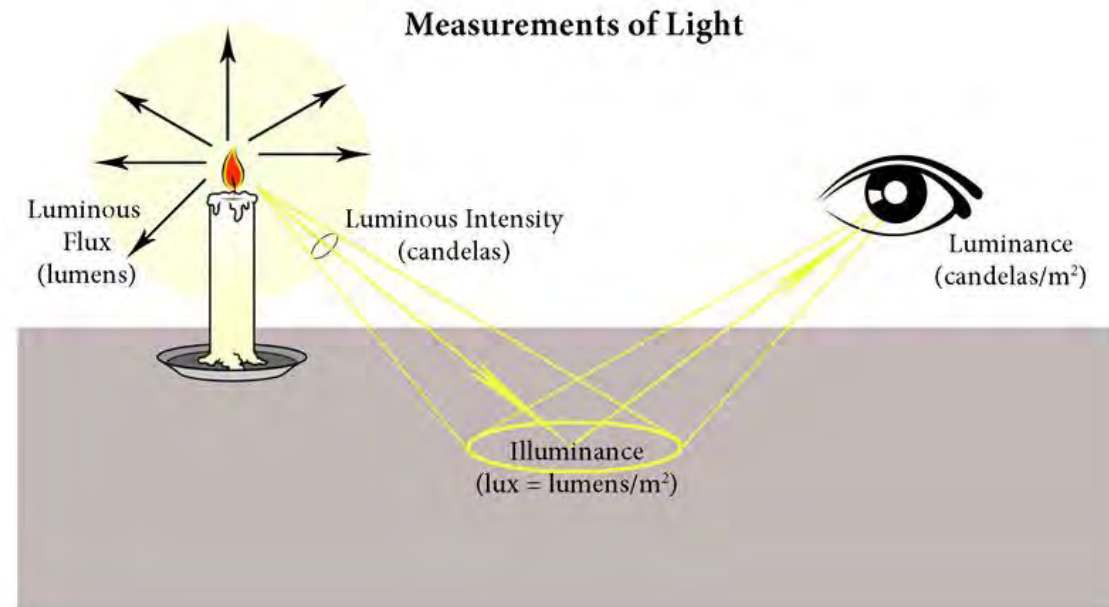
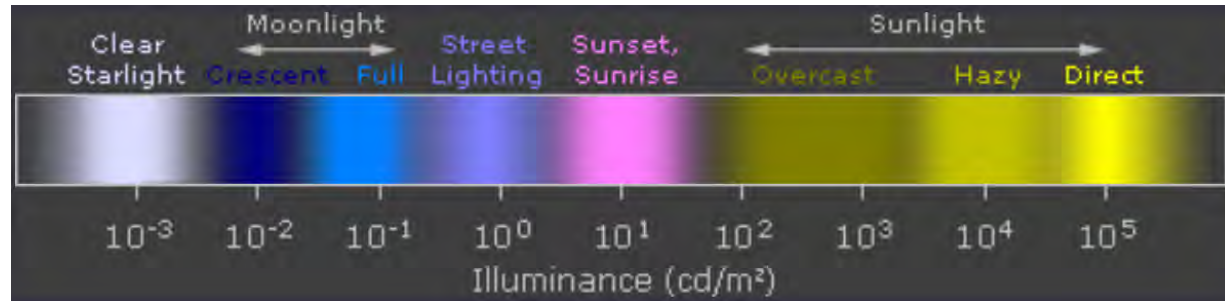
- Chromatic: of color
- Achromatic: without color (i.e. B&W)
- Color of an object depends upon:
  - object surface: reflectivity, physical properties, composition
  - light source(s) illuminating it
  - color of surrounding environment
  - visual system (eyes, brain, photoelectric cell, ...) of perceiving entity

# Achromatic Light

- black, white, usually greys (rare to see pure black or pure white)
- quantity or energy of light is the only measurement used for achromatic light
  - Intensity, luminance, brightness:
    - **Luminance** describes the amount of light that is emitted from, passed through, or reflected off an object. It is measured using candela per square meter ( $\text{cd/m}^2$ ), which is the concentration of luminous intensity per unit area.
    - **Brightness** is subjective and perceived by the human eye. It cannot be measured by any formula
- eye is sensitive to ratios of intensity changes, not the changes in intensity values themselves
- small changes of low intensity seem to have the same effect as larger changes of high intensity
- **Dynamic range**: ratio between min. and max. intensities



# Illuminance

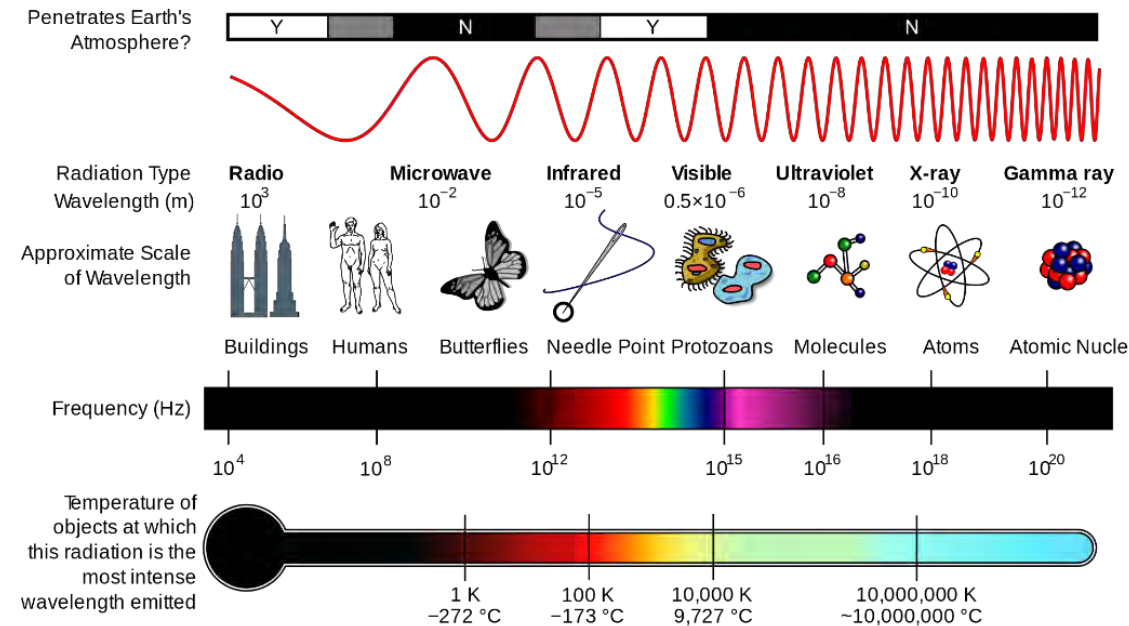
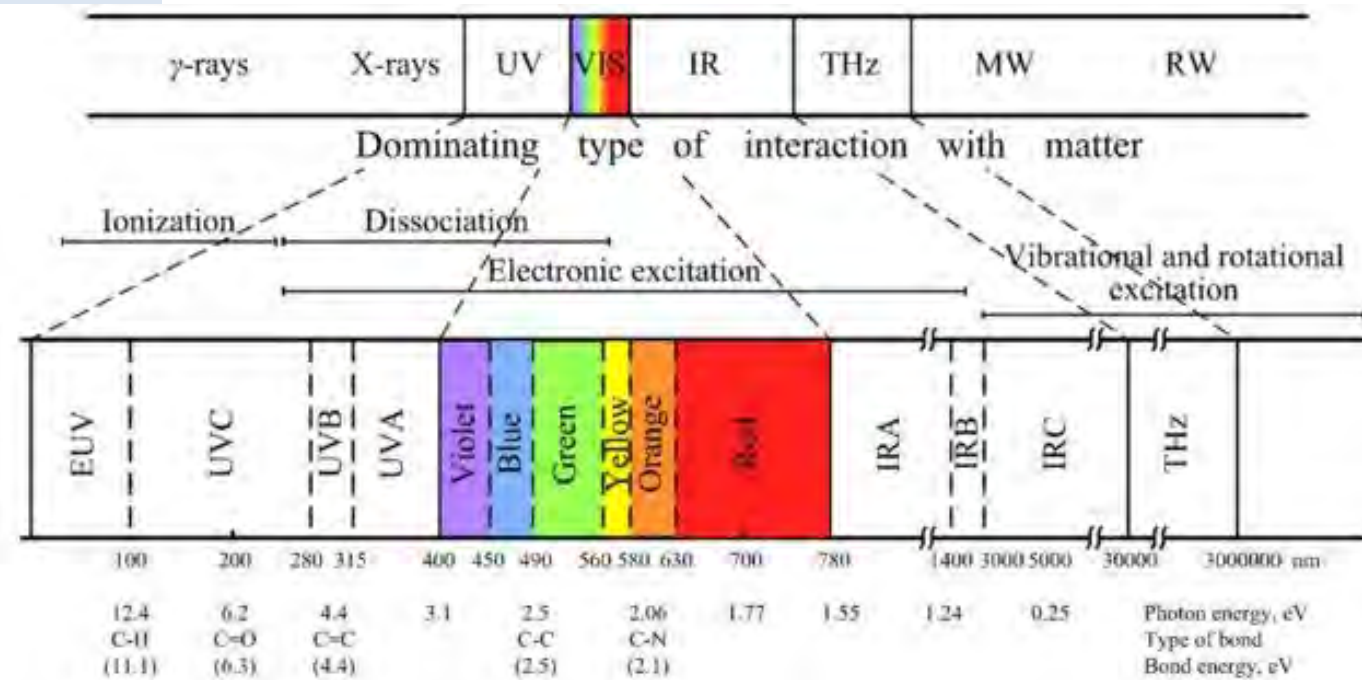


Illuminance = only incident light

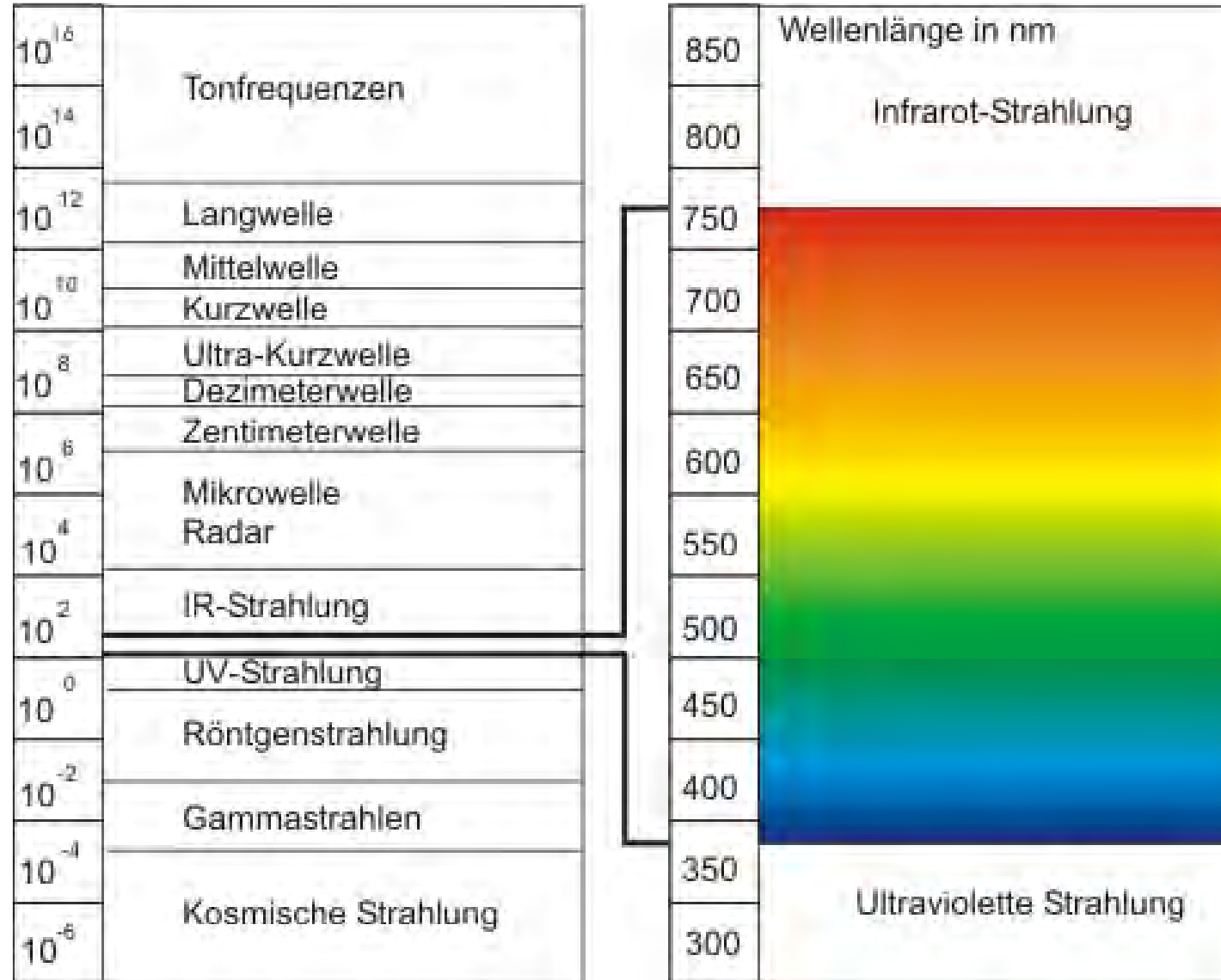
- Illuminance is light we can't see directly, so-called ambient light;
- Light passing through air which is actually invisible until it reflects off an object (which we then see as luminance).
- It is measured in lux - **luminous flux** per unit area ( $\text{lx}$  or  $\text{lm/m}^2$ ).

# Color, Light - Wavelengths

- Color is how we perceive beams of electromagnetic energy that fall on our retina
- We see light between 400 and 700 nanometers in wavelength ( $10^{-9}$  meters)



# Light Spectrum – Wavelengths



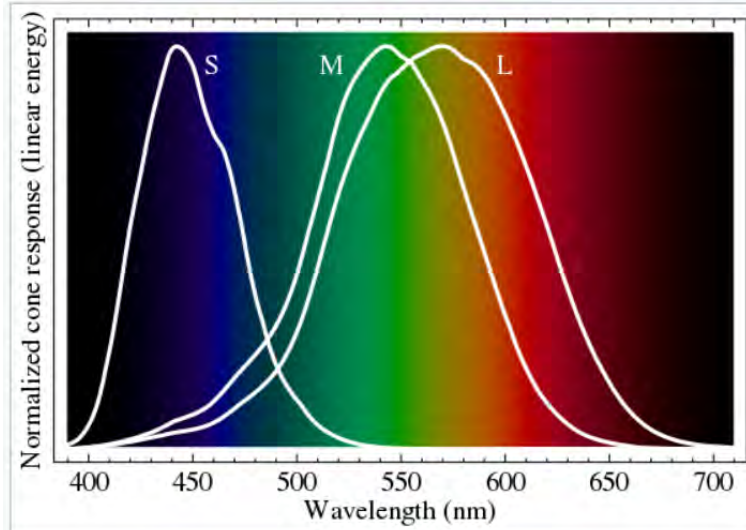
# Color

- Various models of color used:
  - Printing & graphics industry: color sample books and codes
  - Artists: tint (adding white to pure pigment), shade (adding black), tone (adding both)
  - Displays: use hardware capabilities (RGB)
  - Physicists: use optical models (wavelengths, energy measures)
- Human perception of color depends on how brain reacts to the whole visual environment, as well as brain & eye biological factors



# Tristimulus Model

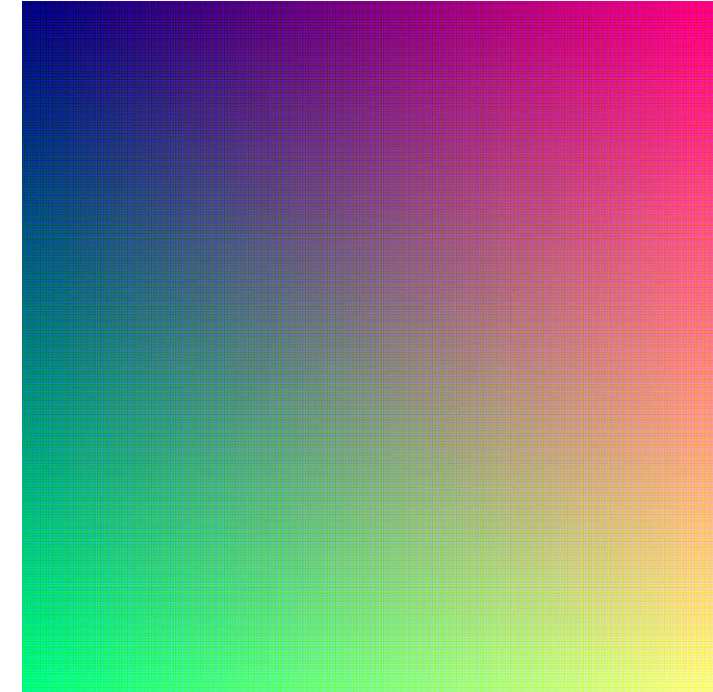
- The retina contains cones that sense either red, green, or blue light
- 6-7 million cones per eye, concentrated in central area called "fovea"
- Rods: sensors that surround fovea, and perceive weak "night" images (>100 million per eye)



Normalized response spectra of human cones, to monochromatic spectral stimuli, with wavelength given in nanometers.

# Color vision in Nonhumans

The human eye can distinguish about 10 million different colors (up to debate)



16.7 million colors

Color vision table

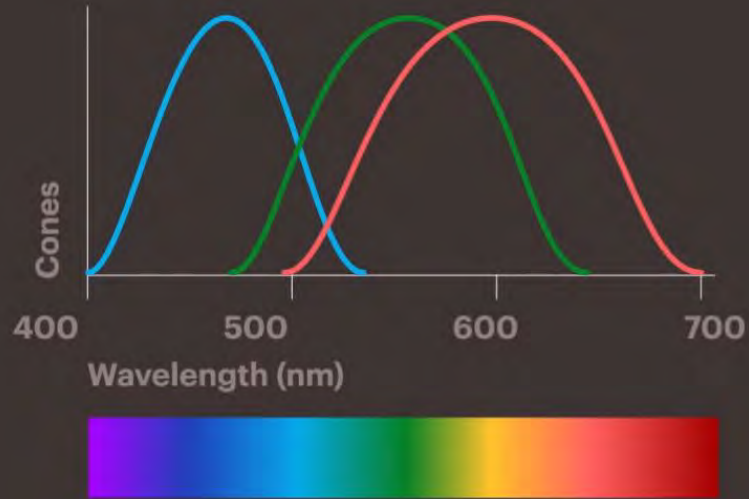
| State                | Types of cone cells | Approx. number of colors perceived | Carriers                                                                                            |
|----------------------|---------------------|------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Monochromacy</b>  | 1                   | 200                                | Marine mammals, owl monkey, Australian sea lion, achromat primates                                  |
| <b>Dichromacy</b>    | 2                   | 40,000                             | Most terrestrial non-primate mammals, color blind primates                                          |
| <b>Trichromacy</b>   | 3                   | 10 million <sup>[54]</sup>         | Most primates, especially great apes (such as humans), marsupials, some insects (such as honeybees) |
| <b>Tetrachromacy</b> | 4                   | 100 million                        | Most reptiles, amphibians, birds and insects, rarely humans                                         |
| <b>Pentachromacy</b> | 5                   | 10 billion                         | Some insects (specific species of butterflies), some birds (pigeons for instance)                   |

**The Amazing Ways Animals See the World**

<https://www.youtube.com/watch?v=LlfKk37bkyk>

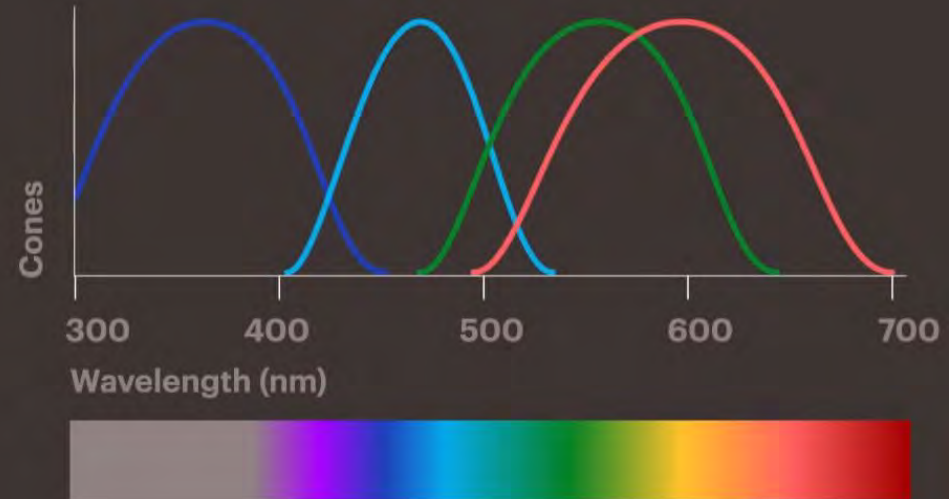


## Humans



3 types of cones

## Birds



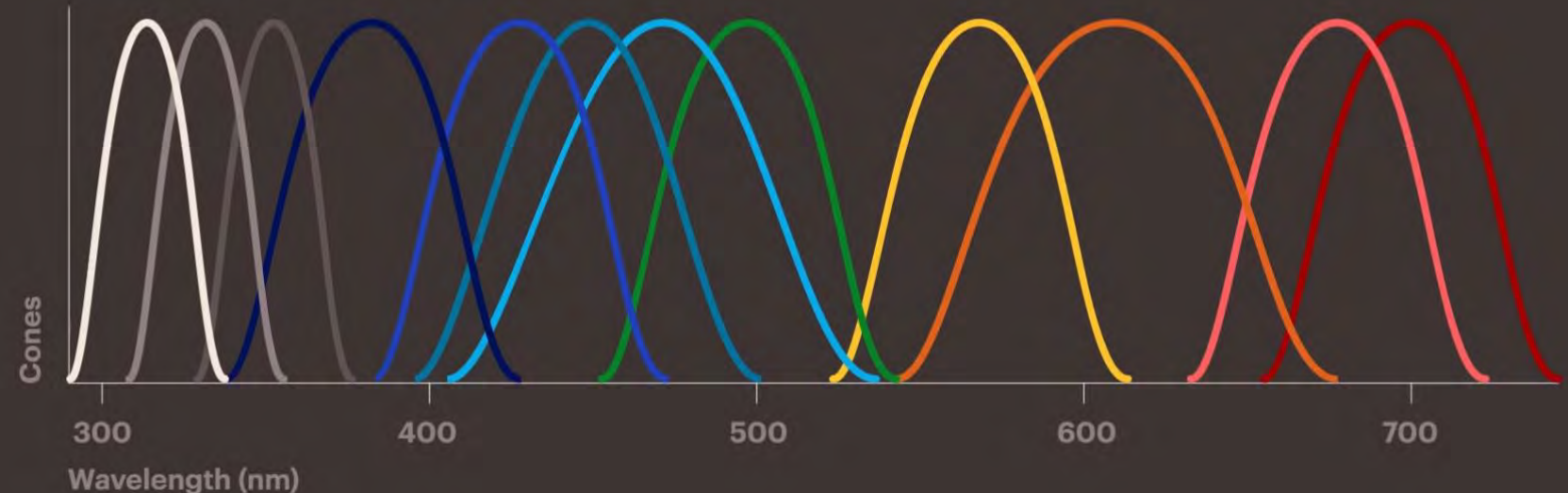
4 types of cones

## Mantis Shrimp:

- 12-16 types of cones
- independent eye movement
- Sees ultraviolet, infrared and 6 types of polarization

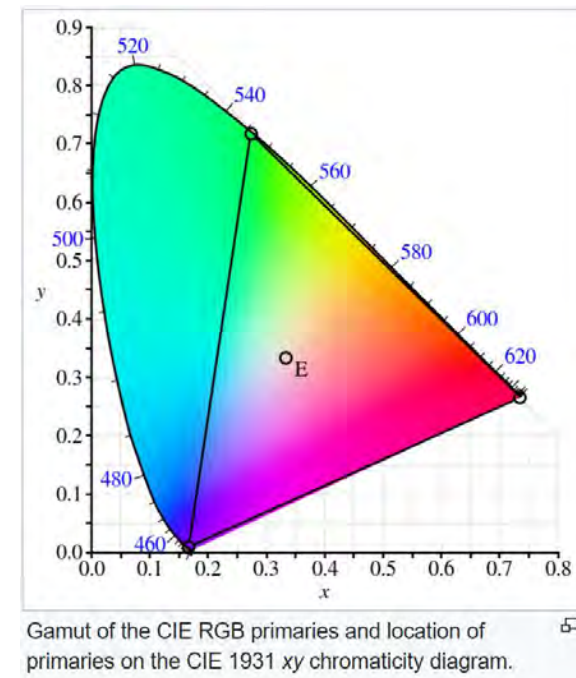


## Mantis Shrimp



# CIE XYZ Color Space (1931)

- CIE color space (CIE = Commission Internationale de l'Eclairage, International Commission on Illumination) used to calibrate other color models;  
1931 CIE XYZ
- Device-invariant representation of color
- Serves as a standard reference against which many other color spaces are defined
- Many combinations of dominant wavelength, luminance, saturation appear the same to us



# CIE Chromaticity Diagram

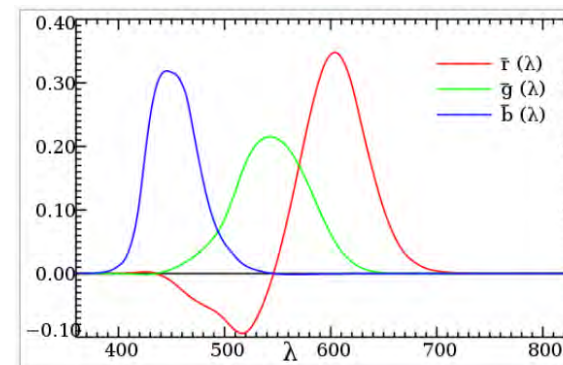
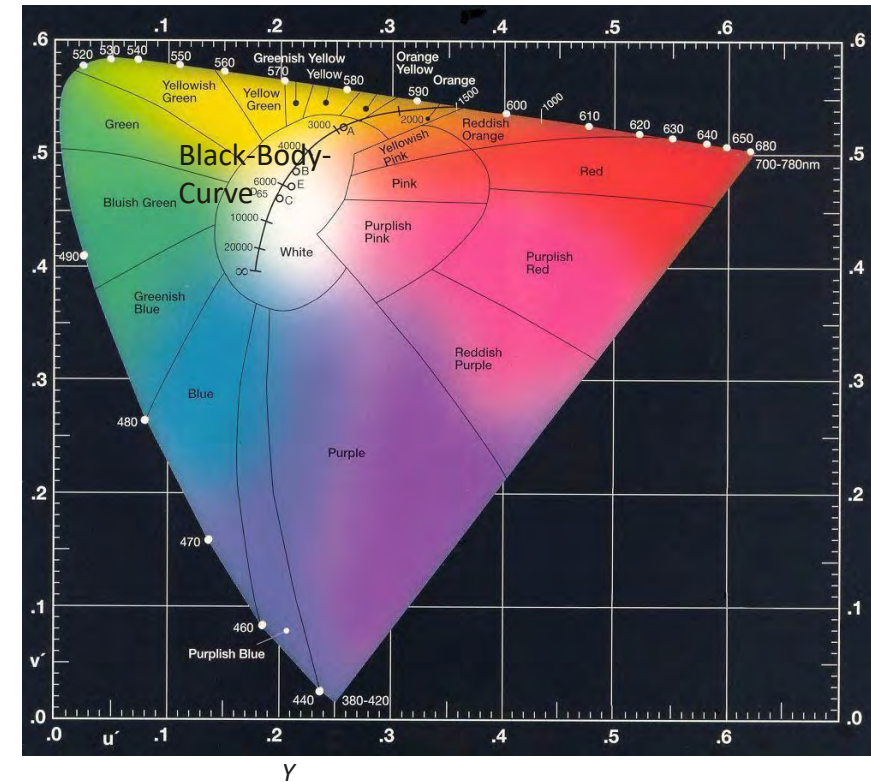
- A full plot of all human visible colors is a three-dimensional figure (cone).
- The concept of color can be divided into two parts: **luminance** and **chromaticity**
- CIE Diagram shows a **planar cut** through that cone at a specific luminance
- Color described in  $x, y$  (with  $x+y+z=1$ )
- Luminance =  $Y$
- Full color:  $(x, y, Y)$

Helligkeitssehen:

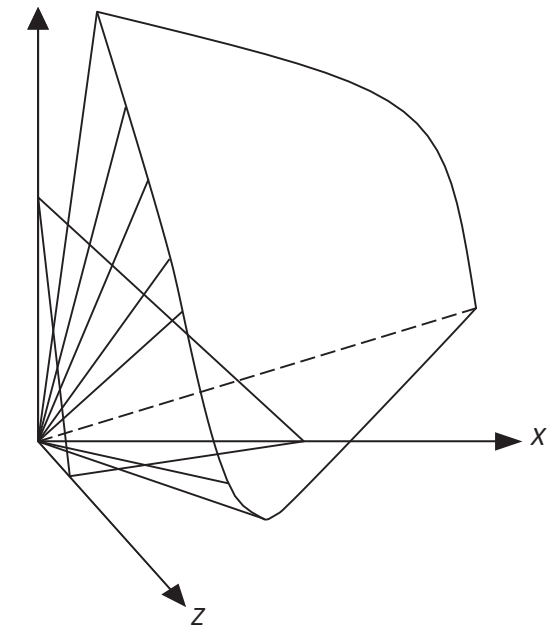
~120 Mill. "Stäbchen"/Rods

Farbsehen:

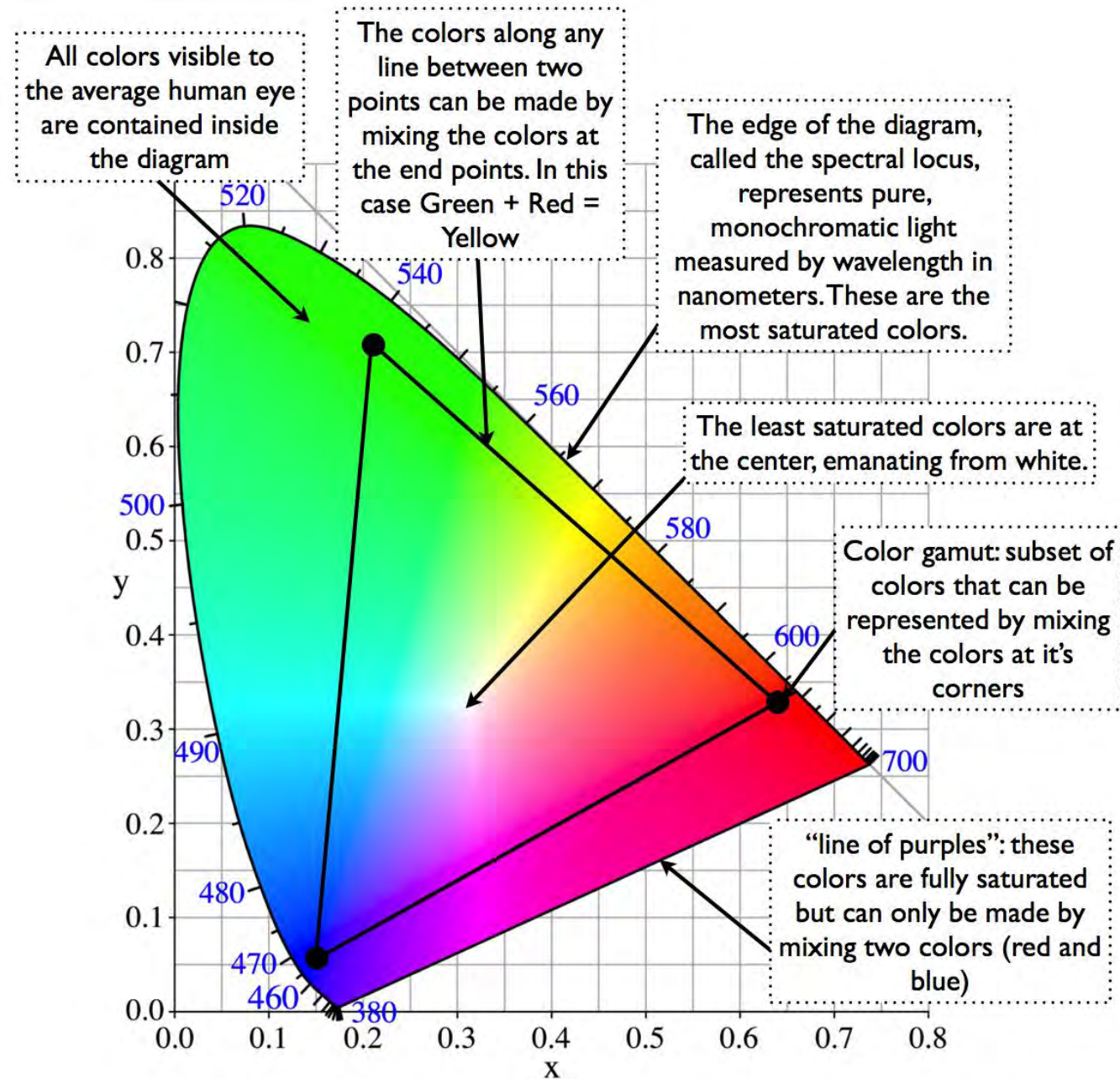
~7 Mill. "Zapfen"/Cones



Die Gewichtungen der drei Primärfarben für die reinen Spektralfarben entlang des Außenrandes des „Hufeisens“ von 360 bis 830 nm







Anatomy of a CIE Chromaticity Diagram



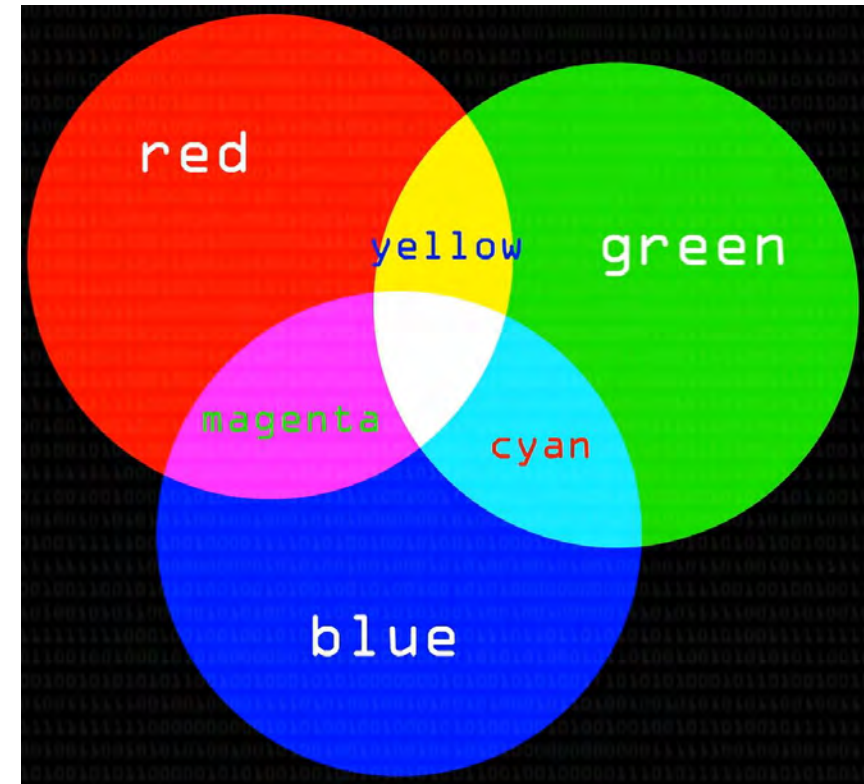
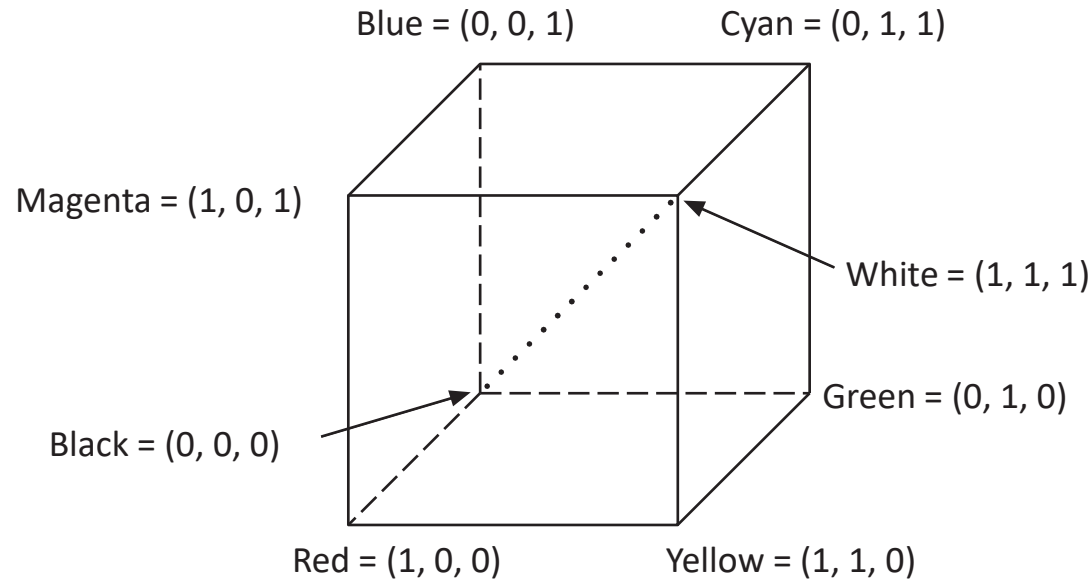
# Quick Overview of Color Spaces



# Color Models (Spaces) Overview

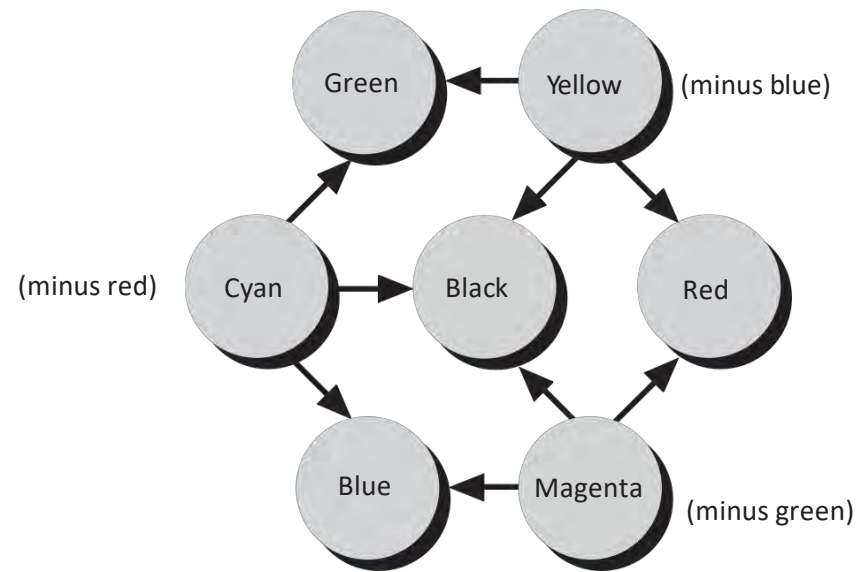
- CIE color space (Commission Internationale de l'Eclairage) used to calibrate other color models 1931 CIE XYZ, tristimulus theory
- RGB — for video display drivers
- CMYK — cyan, magenta, yellow, black, (subtractive primaries)
- HSB — hue (dominant color), saturation (amount of gray), brightness (intensity)
- YUV — used in television industry, Y (luminance), UV (color difference signals) = analog
- YCbCr = digital; Y (luminance),  $C_B$  and  $C_R$  are the blue-difference and red-difference chroma components

# The RGB Color Model

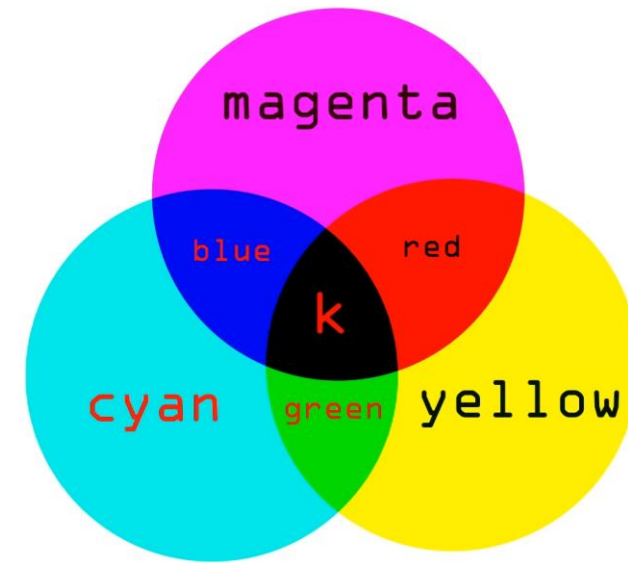


- Hardware-oriented model that turns on red, green, and blue pixels at varying intensities
- Additive: add red, green, blue together to get overall color
- Not all colors are exactly duplicable, and gamut (color range) covered by a display depends on performance of its display technology

# The CMY Color Model



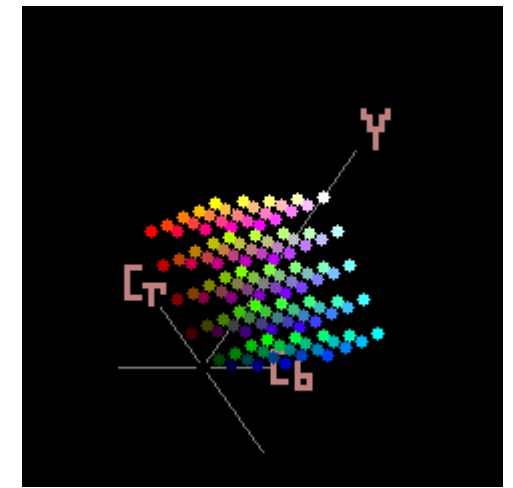
$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} C \\ M \\ Y \end{bmatrix}$$



- Another related model, CMYK, uses black as 4th color:

$$K = \min(C, M, Y), C = C - K, M = M - K, Y = Y - K$$

# YCbCr / YUV



- Splits **luminance (Y)** from color (**chromacity**)  
**Cb** (Blue-Yellow Chrominance) und **Cr** (Red-Green Chrominance)
- Historically YUV (analog version) was used to transmit color (UV) to PAL/SECAM standard color TV sets, while black/white TVs only received a Y luminance signal and would still work. The same signal was transmitted for analog b/w and color TVs.
- Humans are very sensitive to changes in luminance, less to chromacity.
- Reducing color resolution allows great compression of images & videos (is used in JPEG)

# Conversion YCbCr <-> RGB

- Necessary in preparation for JPEG compression
- YCbCr (256 levels) can be computed directly from 8-bit RGB as follows:

$$\begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$

- During decoding YCbCr color space needs to be converted back to RGB:

$$R = Y + 1.402 (Cr-128)$$

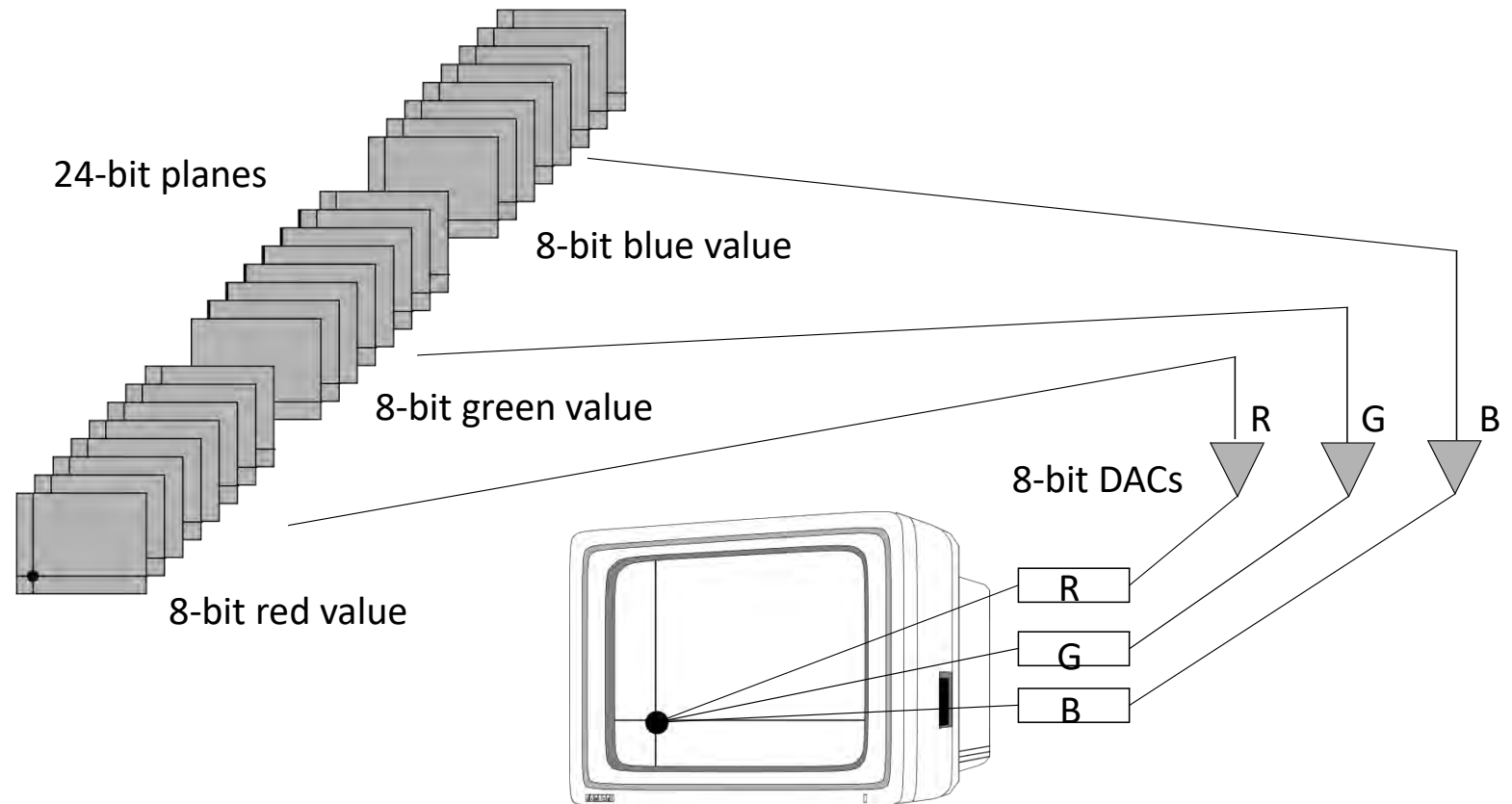
$$G = Y - 0.34414 (Cb-128) - 0.71414 (Cr-128)$$

$$B = Y + 1.772 (Cb-128)$$



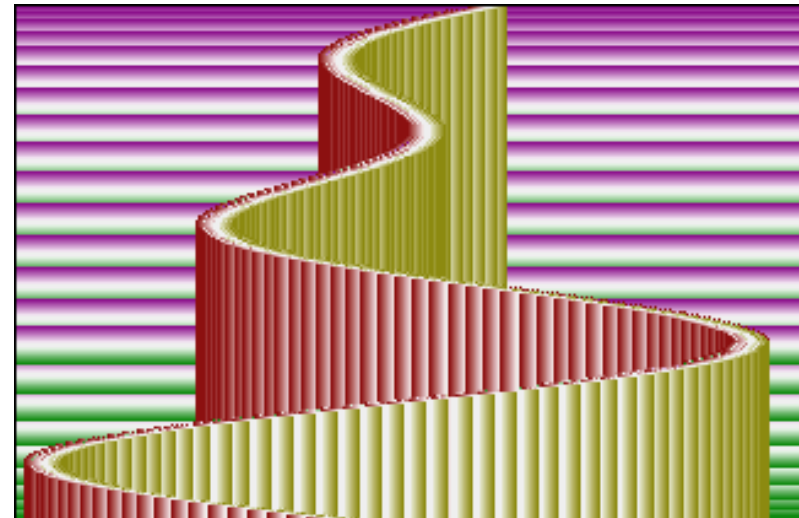
# Color Display – Color Depth

image descriptor = {  
 image width = 640  
 image height = 480  
 image depth = 24 bit  
 color model = RGB  
 encoding = YUV 8:2:2, JPEG}



# Image Representations

- Number of channels — dim. of color model plus **alpha channels**
  - e.g. 32 bit color depth = 24 bit color (8/8/8) + 8 bit alpha channel for transparency
- Channel depth — number of bits-per-pixel
- Alpha — controls transparency; masks and blends
- Pixel aspect ratio (usually square pixels nowadays)
- Interlacing (important for video)
- Compression — lossless vs. lossy
- Indexing — colors are represented by index in color maps or color lookup tables [CLUT]; map predefined or part of image





# HIGH DYNAMIC RANGE DEMYSTIFIED

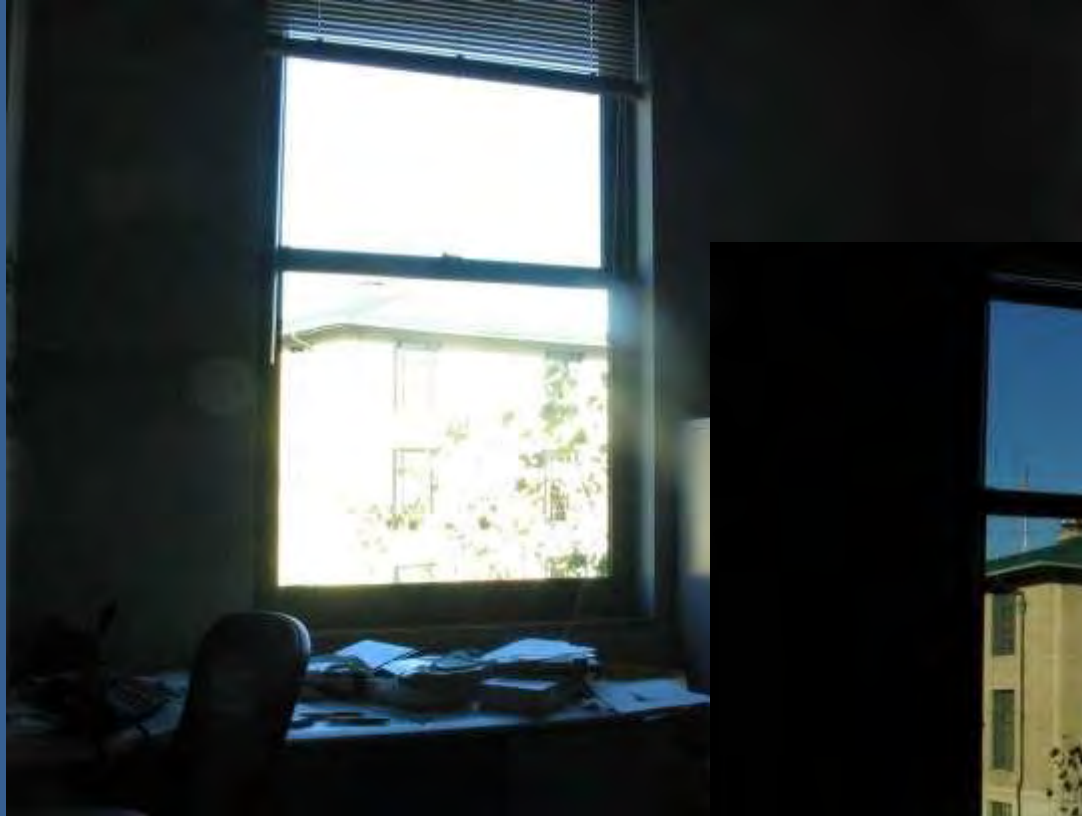
Scott Janus, Principal Engineer

Slides adapted from Scott Janus, Principal Engineer at Intel; Thuong Nguyen [tnguyen@humaxdigital.com](mailto:tnguyen@humaxdigital.com); Prof. Marc Pollefeys and Dr. Gabriel Brostow at ETHZ



# Problem of Dynamic Range

## The Grandma Problem

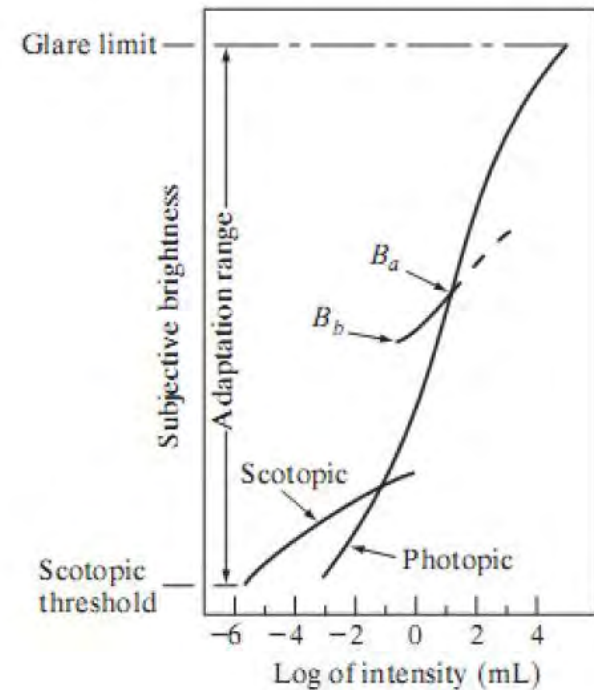


**Dynamic Range: Ratio of largest to smallest signal in a system**



# What is High Dynamic Range?

- Display does not have enough brightness to show the whole range.
- High Dynamic Range:  
Content with a **wider range of brightness and color**
  - Also an increased number of brightness and color levels
- Experiencing HDR content requires new displays!



# Color is complicated



“The Dress”

[https://en.wikipedia.org/wiki/The\\_dress](https://en.wikipedia.org/wiki/The_dress)

White and gold

Or

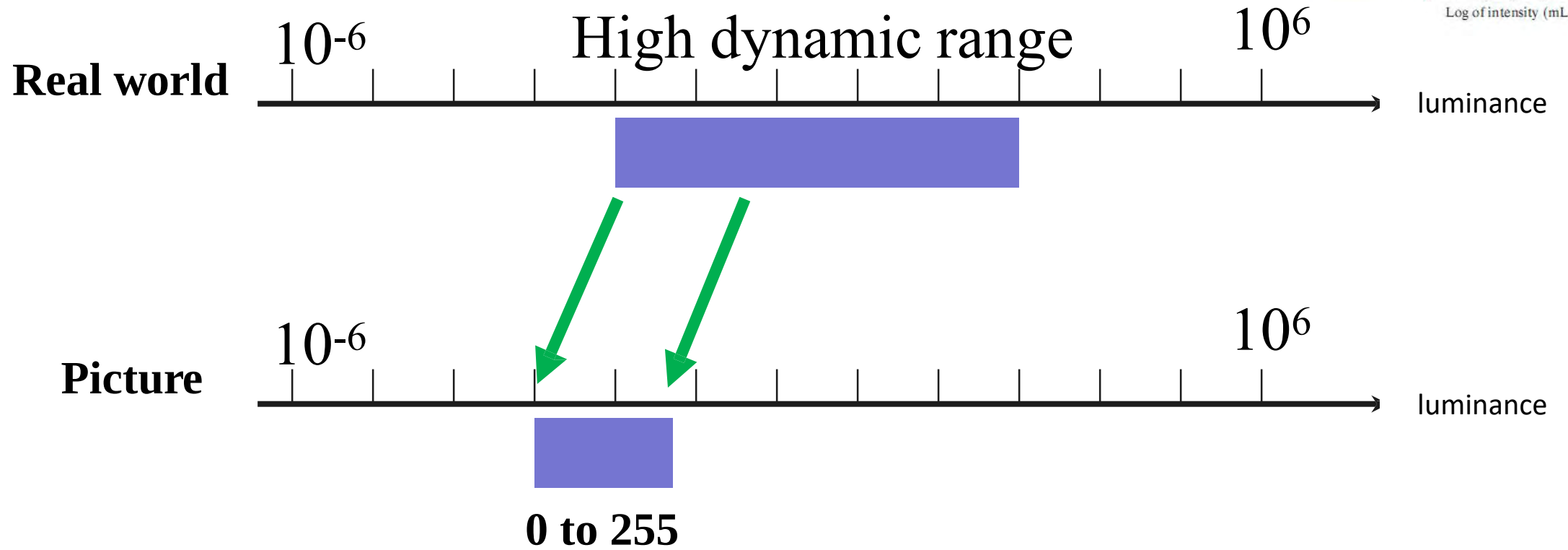
Black and blue?



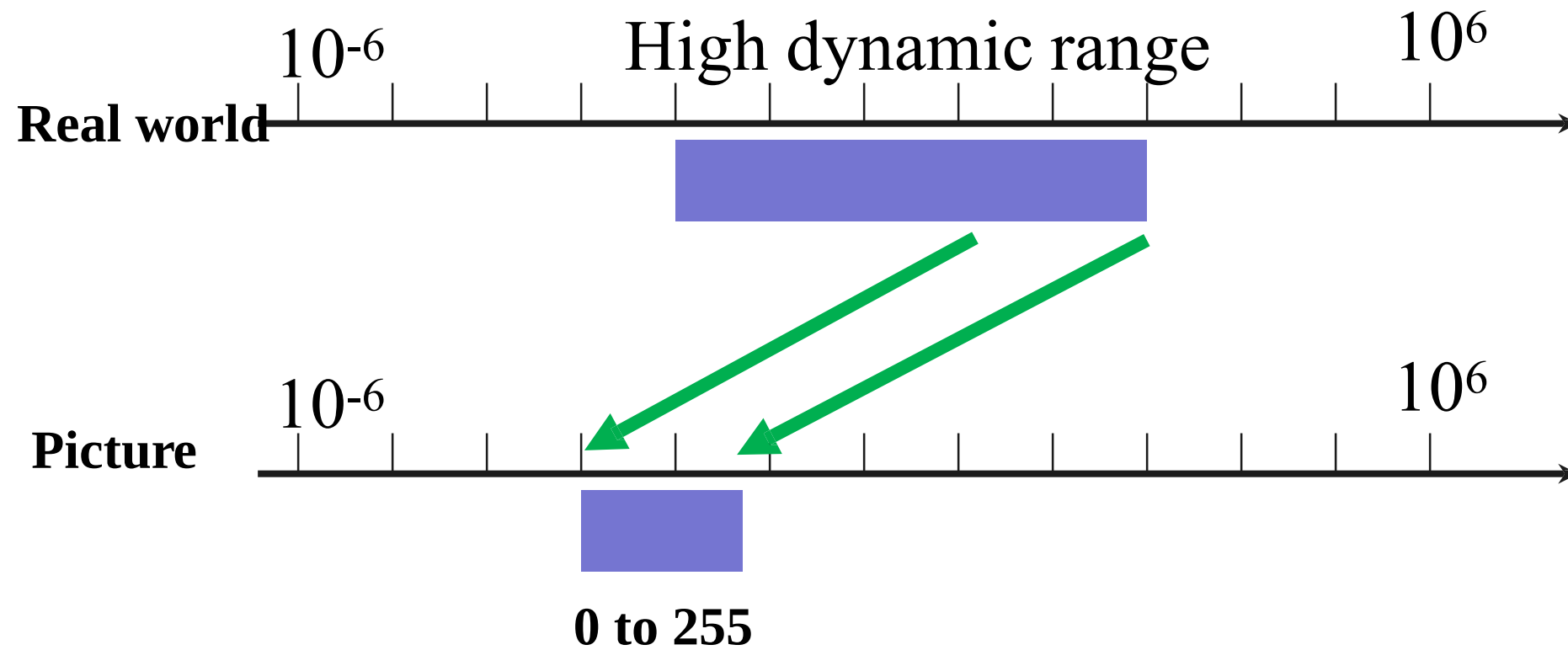


The real dress is  
Blue with black stripes

# Long Exposure

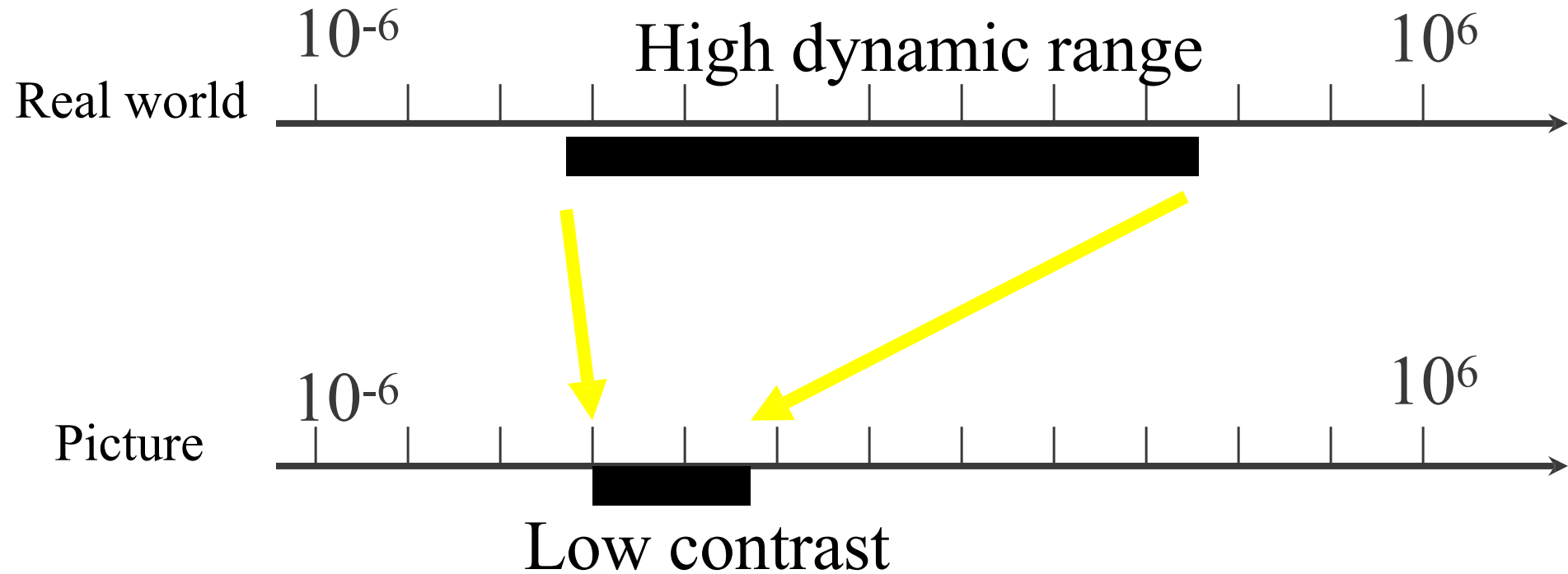


# Short Exposure



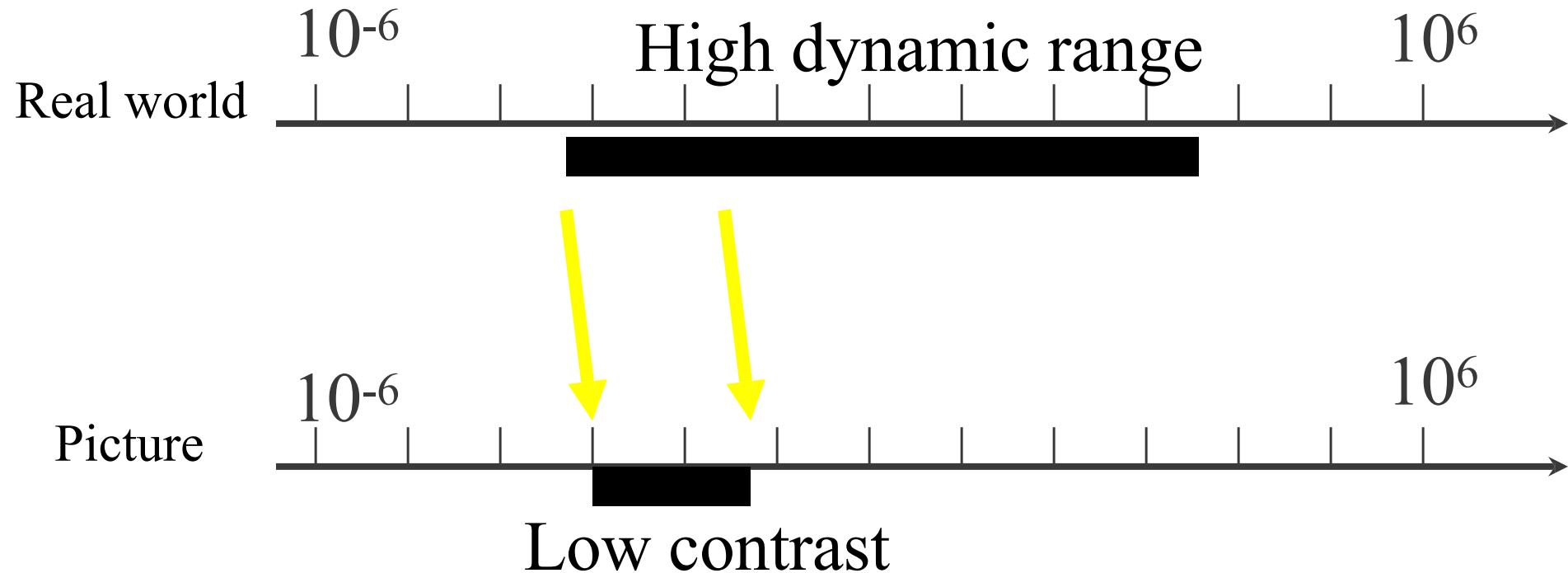
# Multiple exposure photography

- **Sequentially measure all segments of the range**



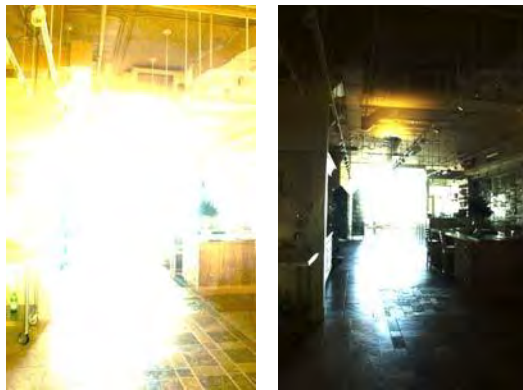
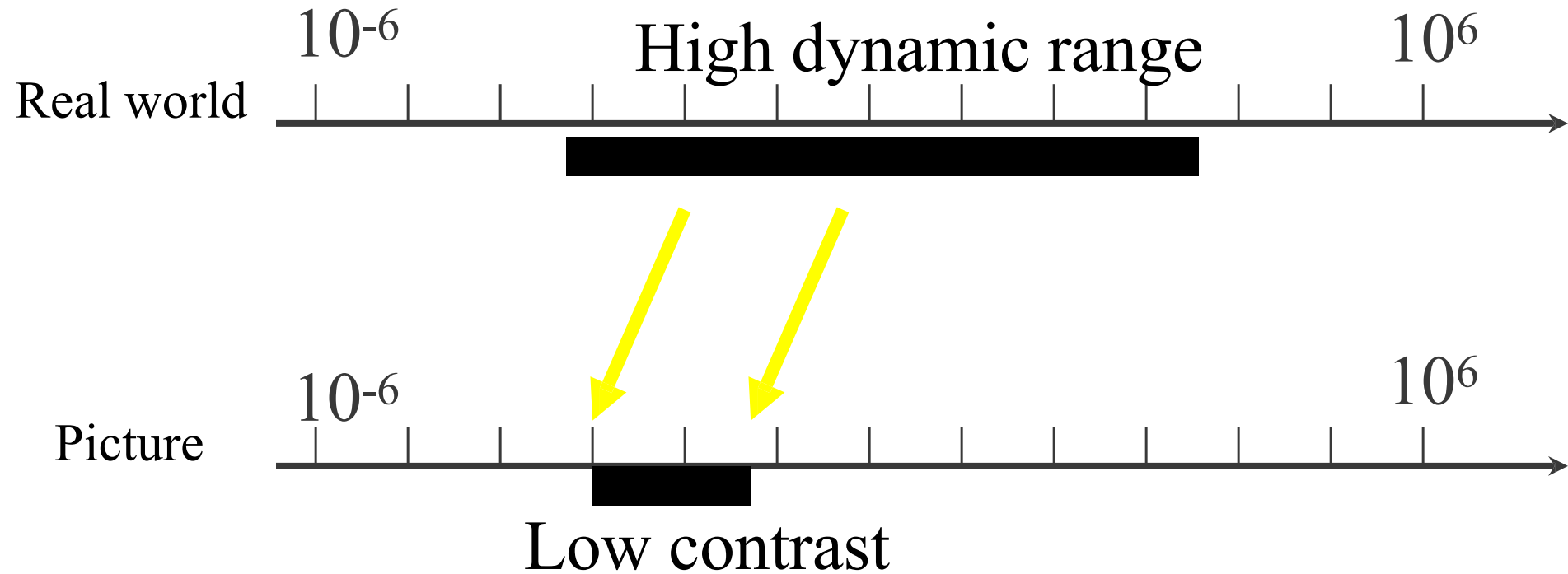
# Multiple exposure photography

- **Sequentially measure all segments of the range**



# Multiple exposure photography

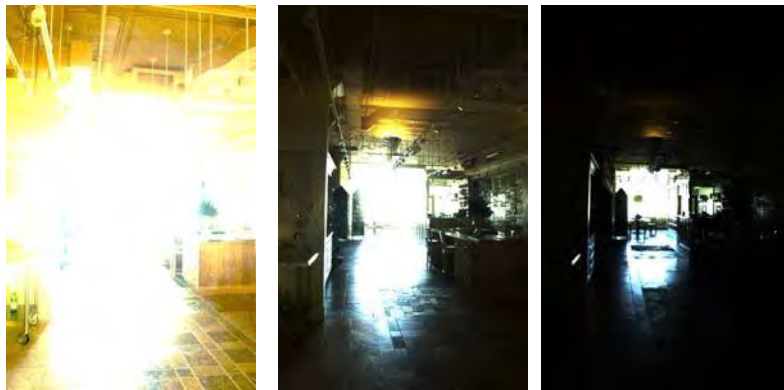
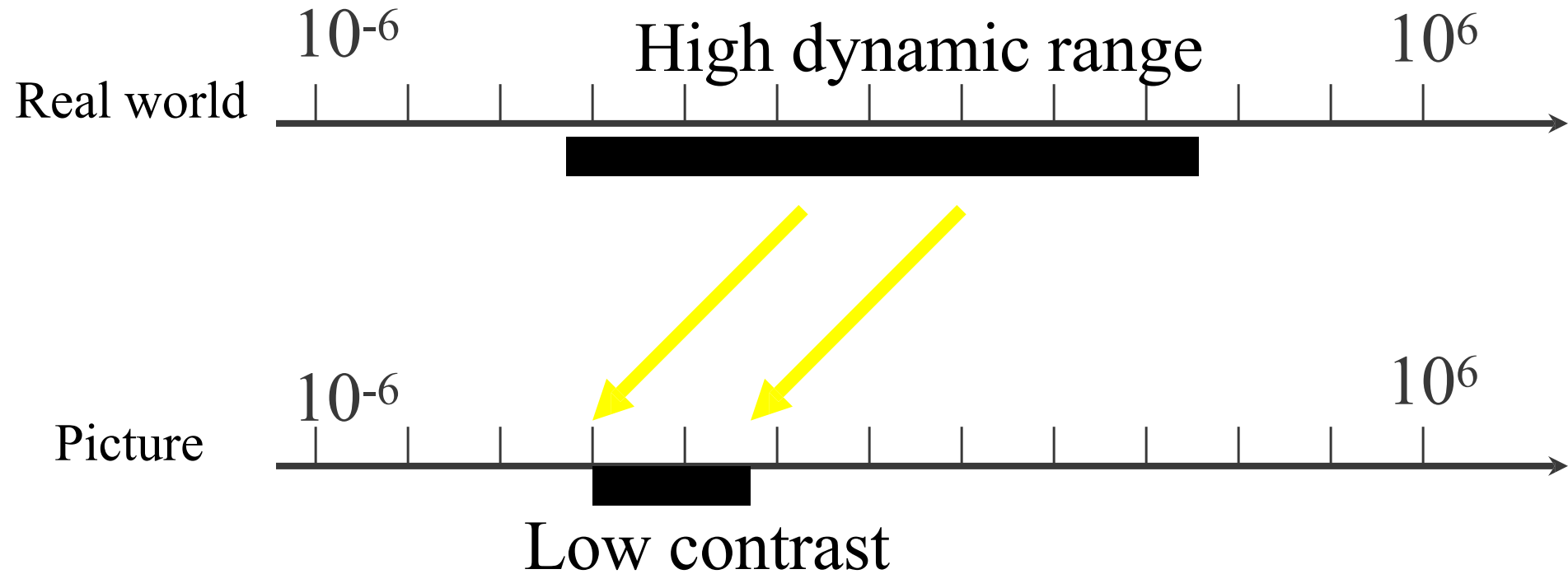
- **Sequentially measure all segments of the range**





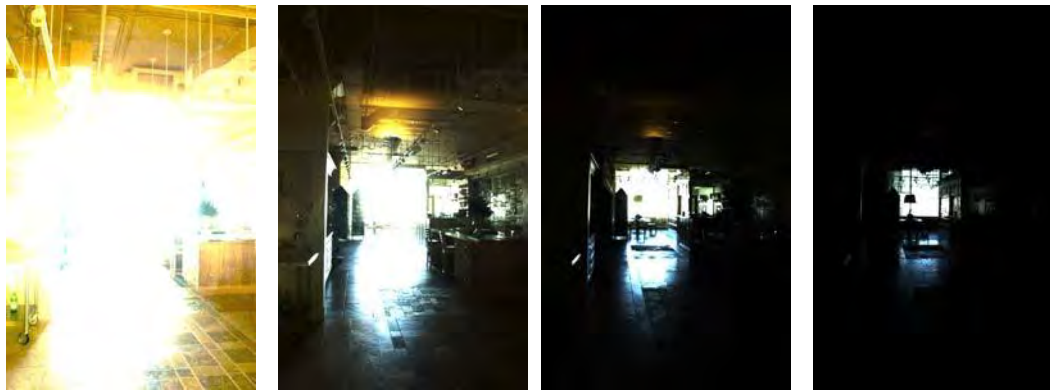
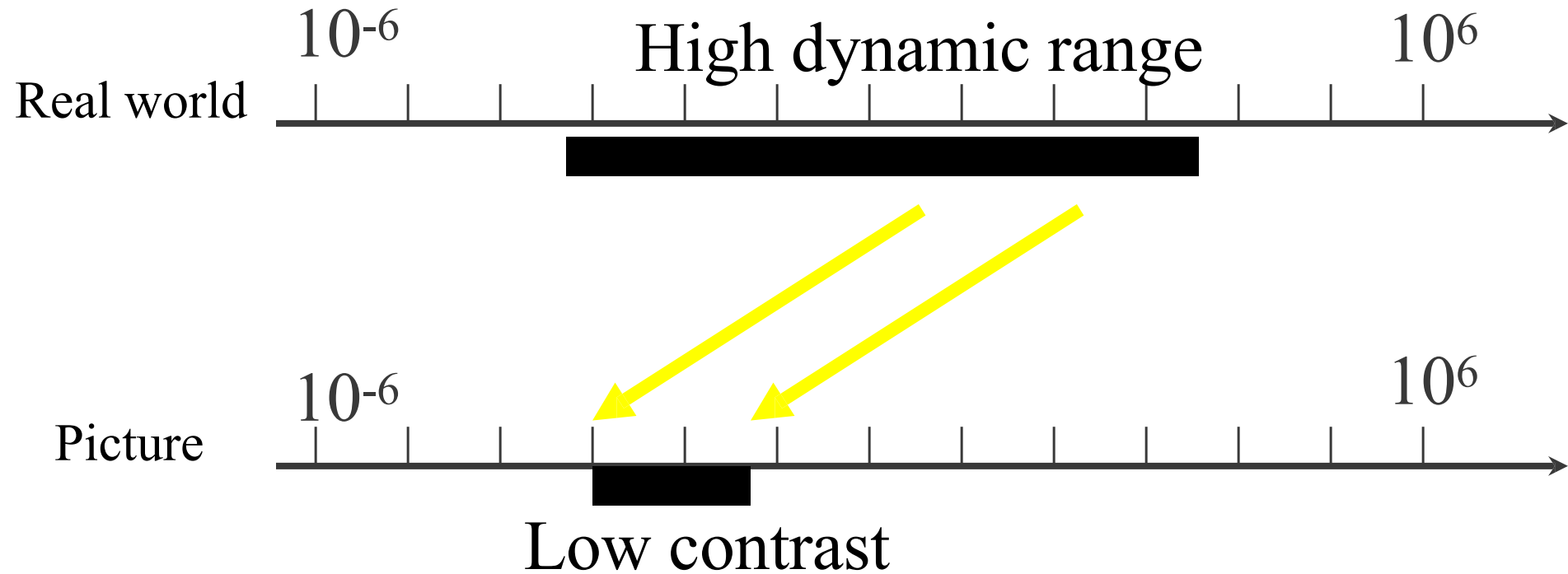
# Multiple exposure photography

- **Sequentially measure all segments of the range**



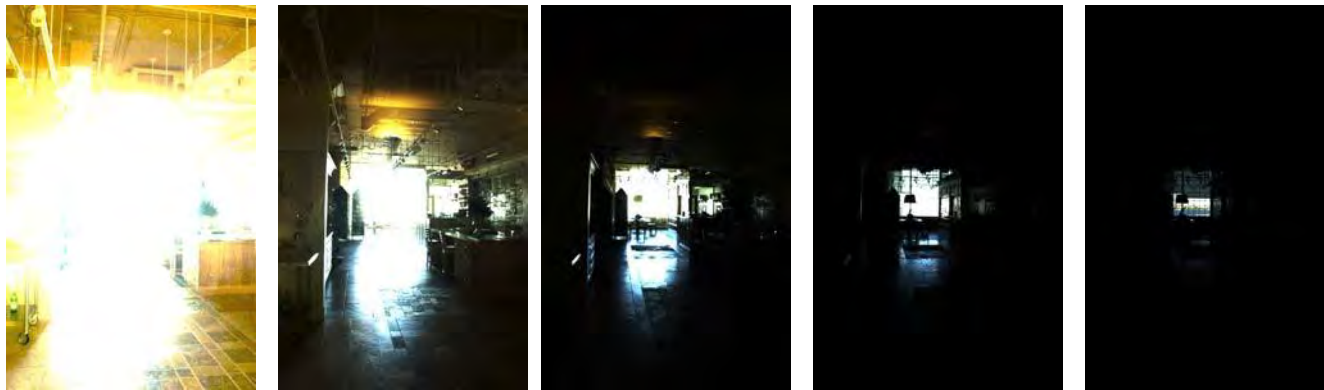
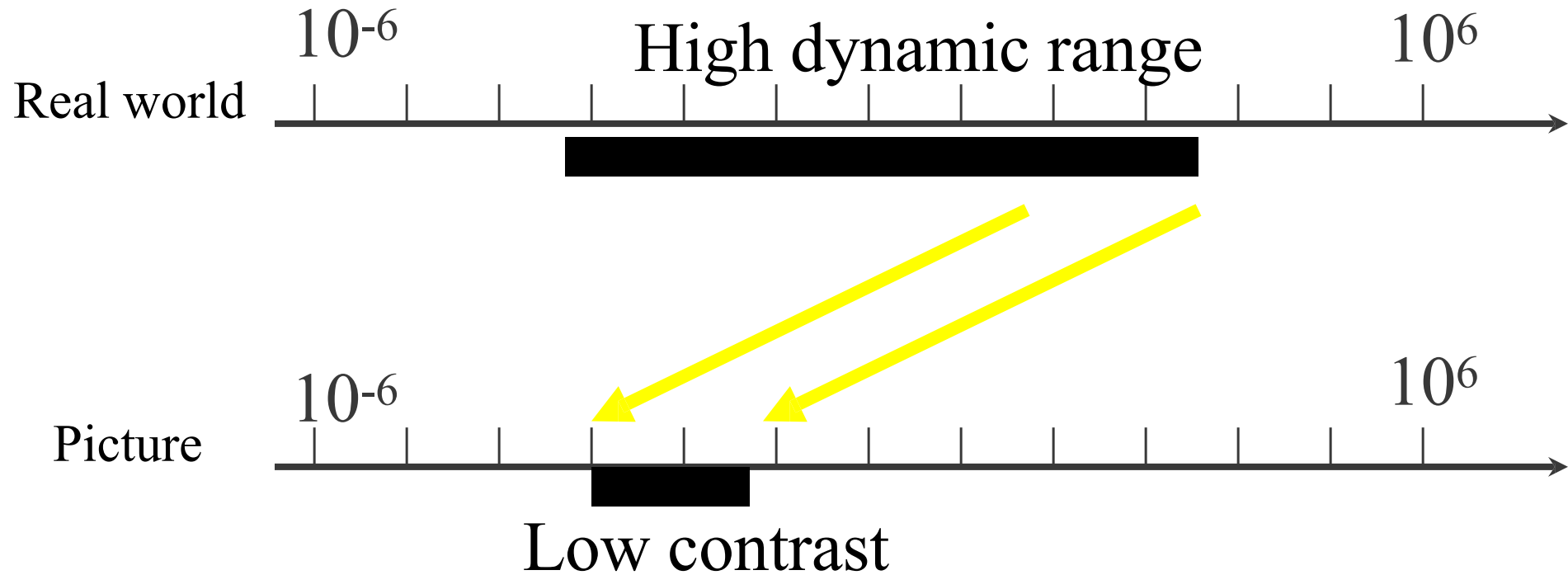
# Multiple exposure photography

- **Sequentially measure all segments of the range**



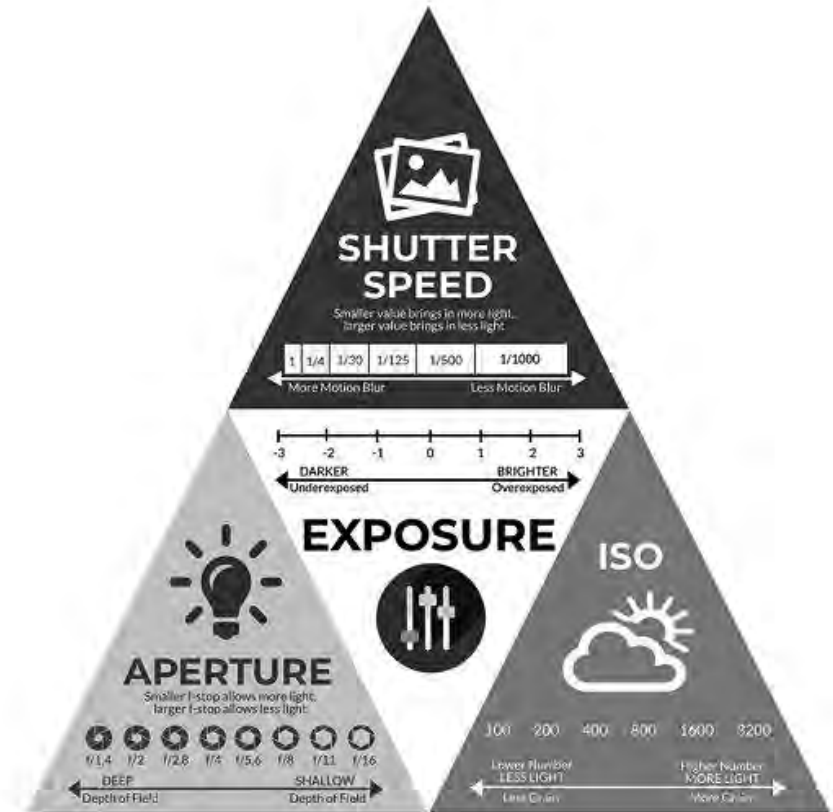
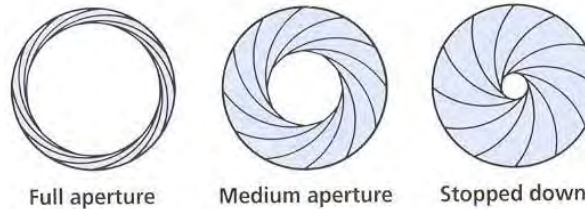
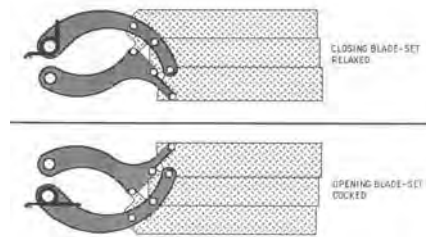
# Multiple exposure photography

- Sequentially measure all segments of the range



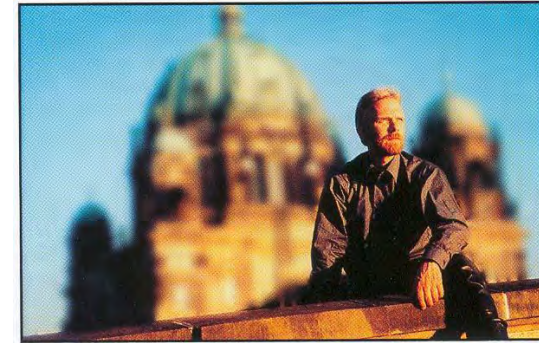
# How do we vary exposure?

- Options:
  - Shutter speed
  - Aperture
  - Light Sensitivity (ISO)
  - [Neutral density filter]

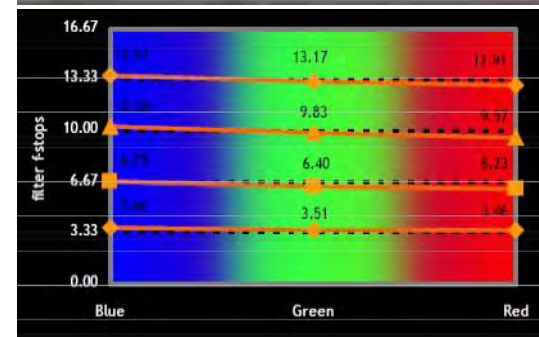


# Tradeoffs

- **Shutter speed**
  - Range: ~30 sec to 1/4000sec (6 orders of magnitude)
  - Pros: reliable, linear
  - Cons: sometimes noise for long exposure
- **Aperture**
  - Range: ~f/1.4 to f/22 (2.5 orders of magnitude)
  - Cons: changes depth of field
- **Light Sensitivity (ISO)**
  - Range: ~100 to 1600 (1.5 orders of magnitude)
  - Cons: noise
- **Neutral density filter**
  - Range: up to 4 densities (4 orders of magnitude) & can be stacked
  - Cons: not perfectly neutral (color shift), not very precise, need to touch camera (shake)
  - Pros: works with strobe/flash, good complement when desperate

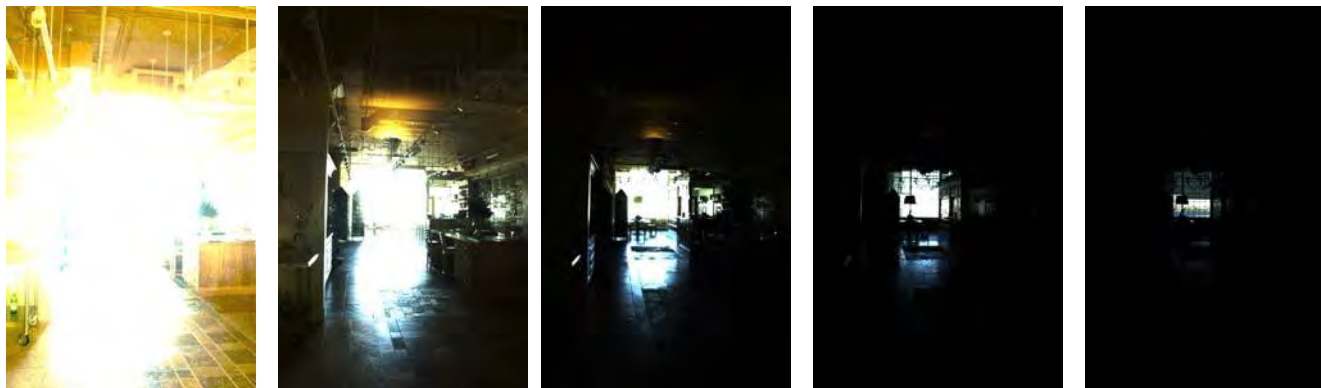


Nikon D2X  
ISO 3200



# HDR image using multiple exposure

- Given  $N$  photos at different exposure
- Recover a HDR color for each pixel

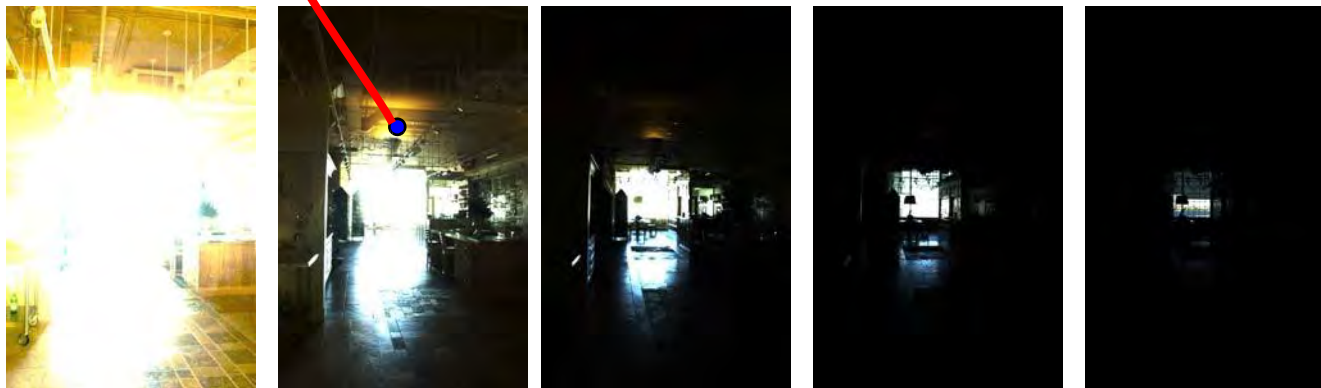
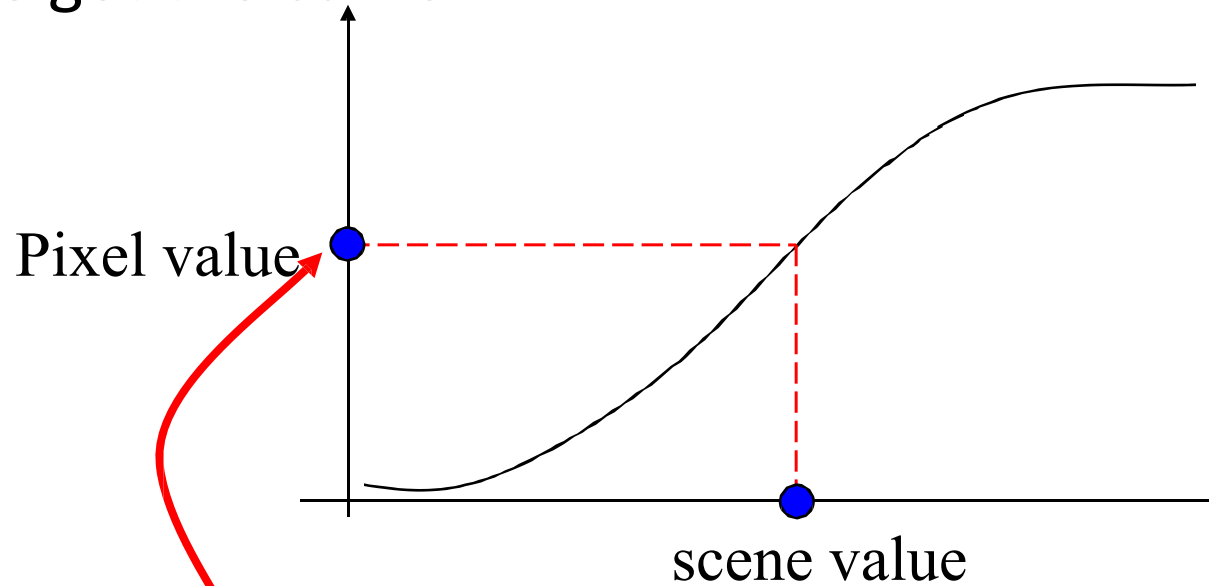




# If we know the response curve

- Just look up the inverse of the response curve
- But how do we get the curve?

*A response curve is the relation between the input and output of a sensor.*



# Calibrating the response curve

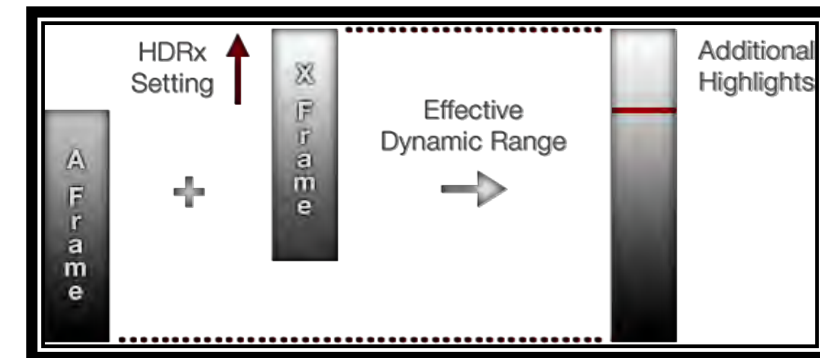
- **Two basic solutions**

- Vary scene luminance and see pixel values
  - Assumes we control and know scene luminance
- ***Vary exposure and see pixel value for one scene luminance***



- **Best of both:**

- **Vary exposure**
- Exploit the large number of pixels

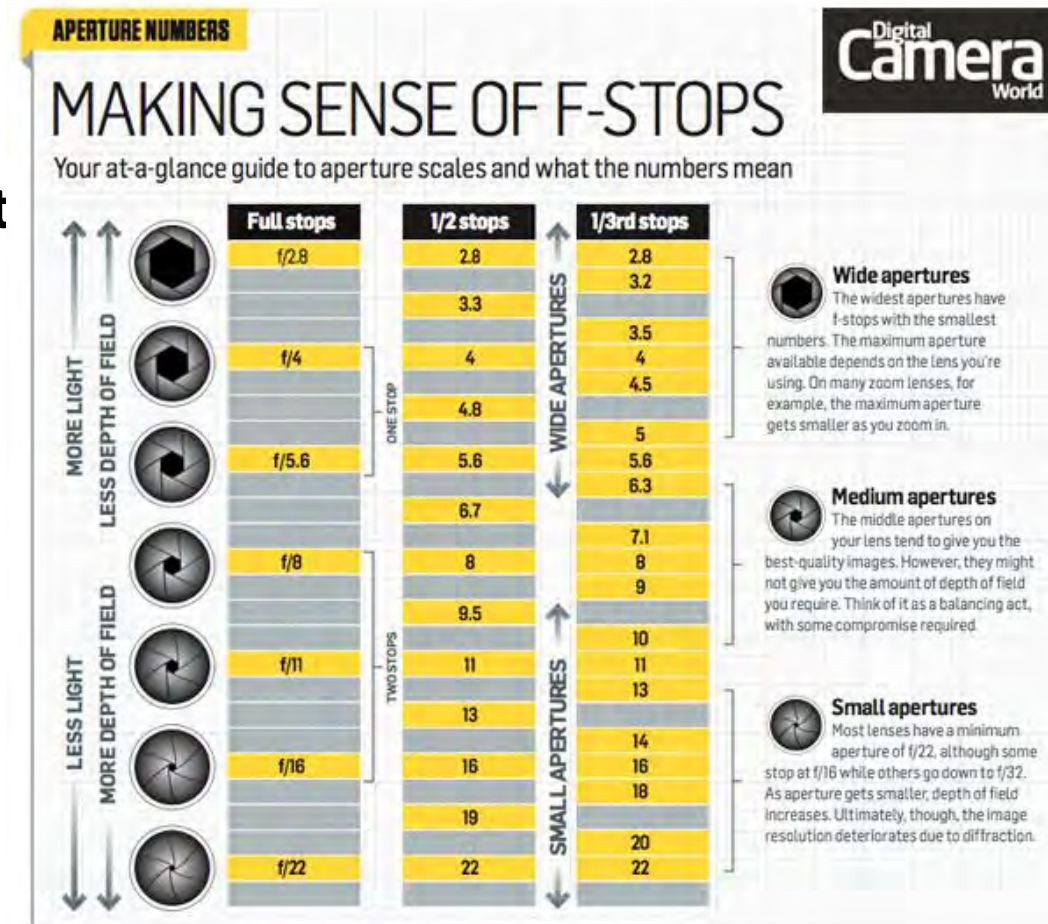


Explanation of the HDRx mode (RED film cameras) – 2 exposures captured:  
<https://www.red.com/red-101/hdrx-high-dynamic-range-video>

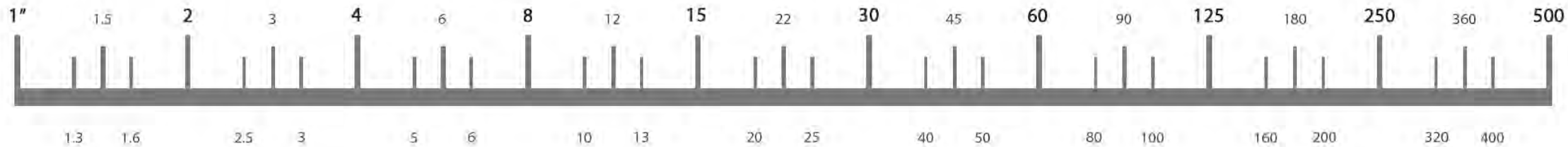
# F-stop number (Aperture Stop)

- Each stop is **twice as bright as the previous one**
- Here's the aperture scale. Each step down lets in **half** as much light:
- These are the main aperture “stops,” but most cameras and lenses today let you set some values in between, such as f/1.8 or f/3.5.
- When a camera is said to have **12 stops of dynamic range**, it means that the **brightest part of a scene can be  $2^{12}$  times brighter than the darkest part.**

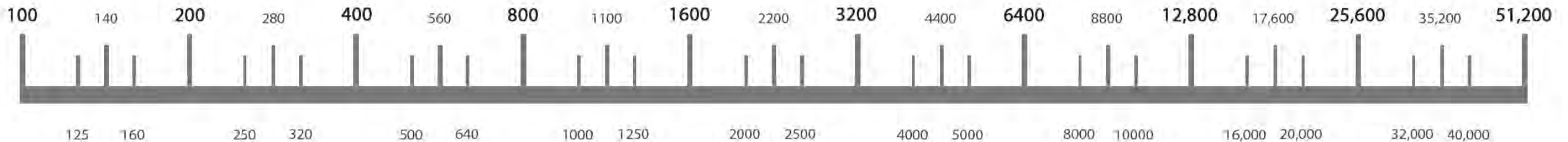
| f/1.4                          | f/2.0               | f/2.8               | f/4.0                          | f/5.6                                         | f/8.0                           | f/11.0               | f/16.0               | f/22.0                                                        |
|--------------------------------|---------------------|---------------------|--------------------------------|-----------------------------------------------|---------------------------------|----------------------|----------------------|---------------------------------------------------------------|
| Very large aperture            | Large aperture      | Large aperture      | Moderate aperture              | Moderate aperture                             | Moderate aperture               | Small aperture       | Small aperture       | Very small aperture                                           |
| Lets in a huge amount of light | Half as much light  | Half as much light  | Half as much light             | Half as much light (a very “medium” aperture) | Half as much light              | Half as much light   | Half as much light   | Half as much light (by which point your photos are very dark) |
| Very thin depth of field       | Thin depth of field | Thin depth of field | Moderately thin depth of field | Moderate depth of field                       | Moderately large depth of field | Large depth of field | Large depth of field | Very large depth of field                                     |



# Shutter Speed & ISO Stops



A shutter speed range of 10 stops with values from one second all the way to 1/500th of a second



Calculating ISO sensitivity stops is very easy – one just needs to divide or multiply the value by 2 to add a stop or go one stop down.

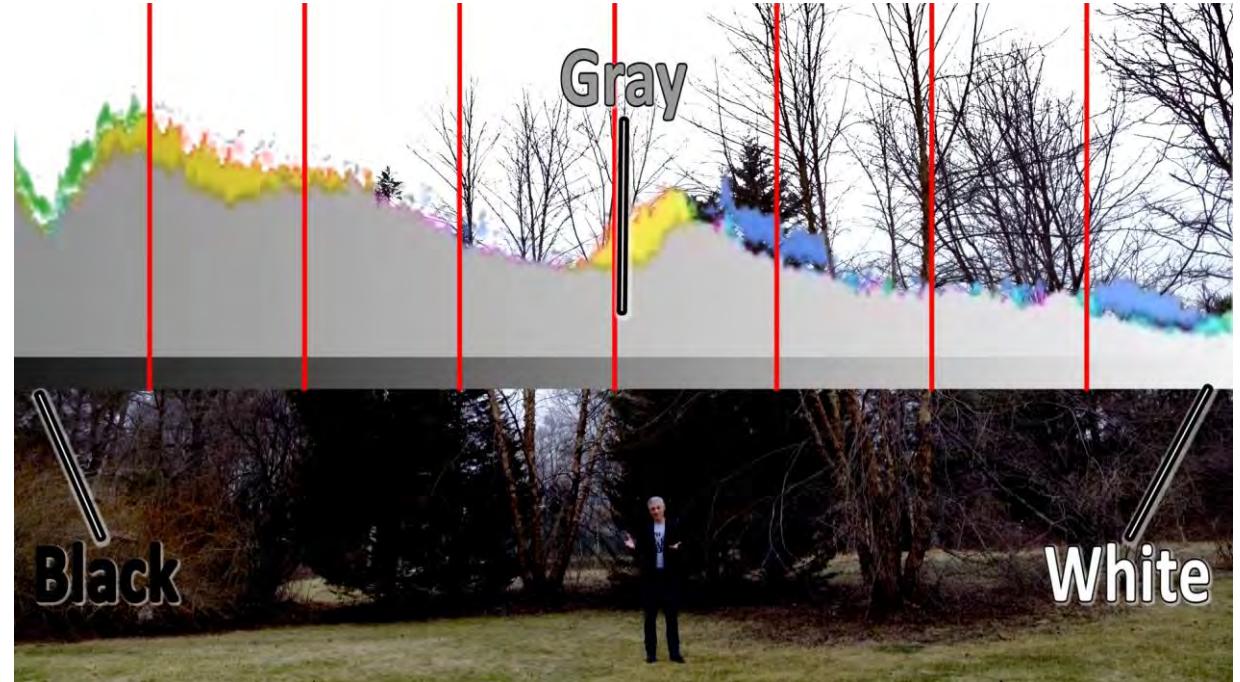
PLEASE NOTE: A digital sensor does not and cannot change its sensitivity to light. There is no setting whatsoever on a digital camera, including ISO, that can modify a sensor's sensitivity to light. ISO in digital is a level of post-capture analog amplification that occurs.



# Dynamic Range Explanations & Examples



<https://www.youtube.com/watch?v=SrXwiPH-fCg>



<https://www.youtube.com/watch?v=MfvwqcmM6FQ>

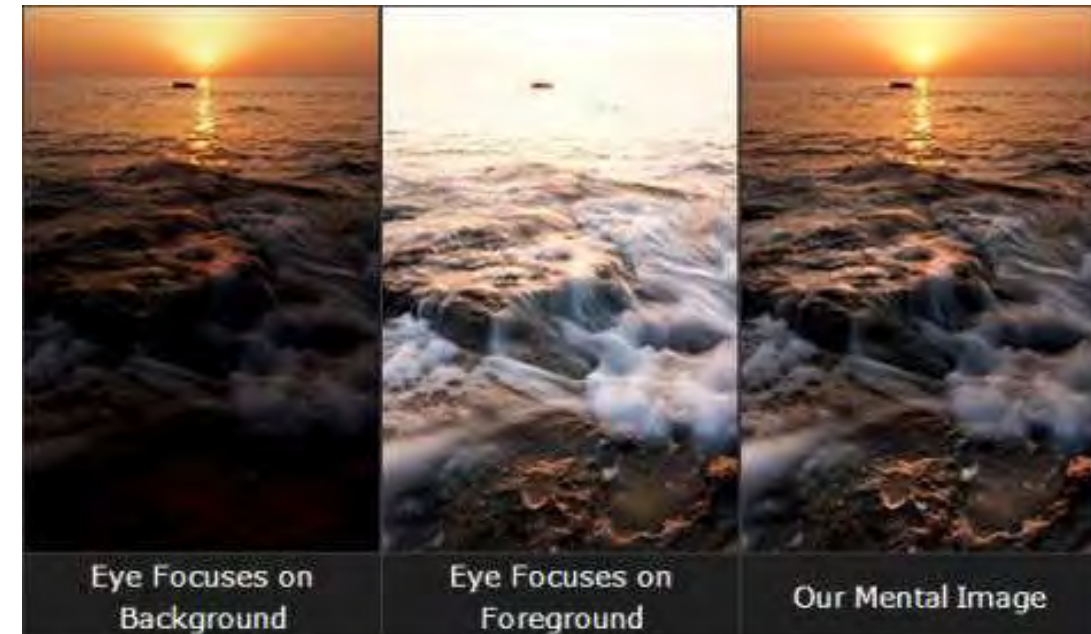
Very good detailed explanation of a film maker about Dynamic Range:

<https://www.youtube.com/watch?v=2sshGdMgJxQ>

# The Human Eye



- Human eye uses the pupil size to see darker or brighter images
  - Turn smaller to limit incoming light to see brighter scenes
  - But cannot see darker & brighter region at same time
  - Can detect a luminance range of  $10^{14}$ , or one hundred trillion (100,000,000,000,000) (**about 46.5 f-stops**), from  $10^{-6}$  cd/m<sup>2</sup>, or one millionth (0.000001 cd/m<sup>2</sup>) to  $10^8$  cd/m<sup>2</sup> or one hundred million (100,000,000 cd/m<sup>2</sup>)





# Stops - Contrast

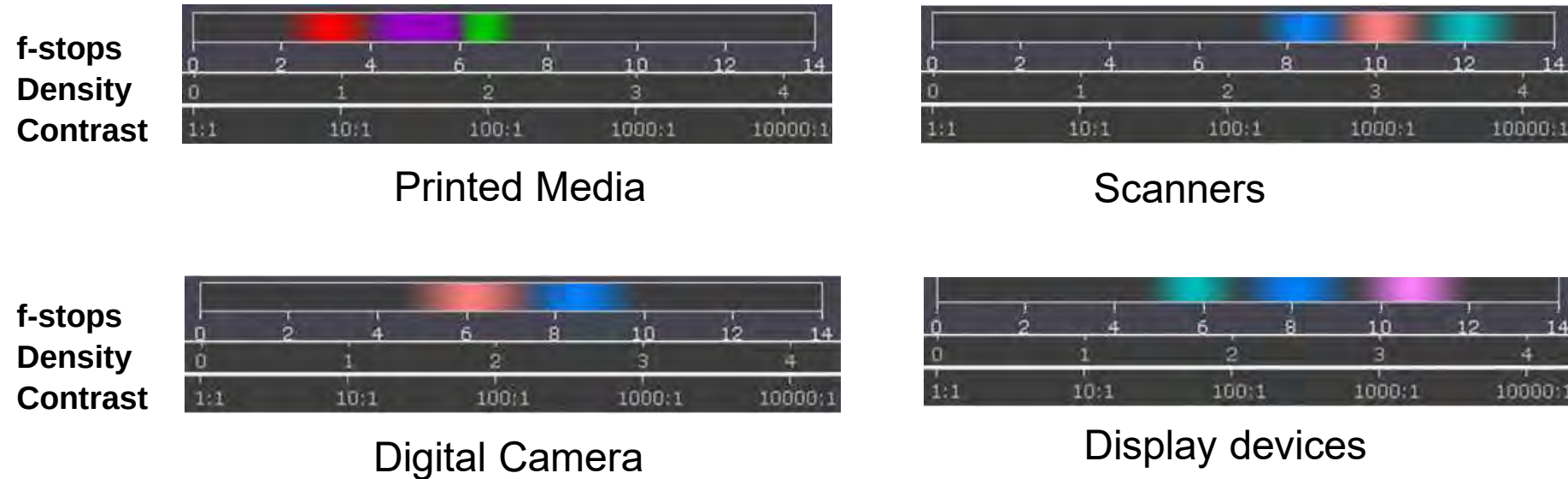
Dynamic ranges of common devices

| Device                                | Stops                                       | Contrast ratio         |
|---------------------------------------|---------------------------------------------|------------------------|
| Glossy photograph paper               | 7 (7 - 7 2/3 ) <sup>[58]</sup>              | 128:1                  |
| LCD                                   | 9.5 (8 – 10.8) <sup>[citation needed]</sup> | 700:1 (250:1 – 1750:1) |
| Negative film (Kodak VISION3)         | 13 <sup>[59]</sup>                          | 8000:1                 |
| Human eye                             | 10–14 <sup>[54]</sup>                       | 1000:1 – 16000:1       |
| High-end DSLR camera (Nikon D850)     | 14.8 <sup>[60]</sup>                        | 28500:1                |
| Digital cinema camera (Red Weapon 8k) | 16.5+ <sup>[61]</sup>                       | 92000:1                |

Without adaptation

# Contrast Comparison

- Dynamic range is commonly measured on a logarithmic scale



# Bit Depth & Dynamic Range

- (bit per pixel) Quantifies how many unique colors are available
  - How many unique shades are available in grayscale image

| Bit Precision<br>of Analog/Digital<br>Converter | Contrast Ratio | Dynamic Range |         |
|-------------------------------------------------|----------------|---------------|---------|
|                                                 |                | f-stops       | Density |
| 8                                               | 256:1          | 8             | 2.4     |
| 10                                              | 1024:1         | 10            | 3.0     |
| 12                                              | 4096:1         | 12            | 3.6     |
| 14                                              | 16384:1        | 14            | 4.2     |
| 16                                              | 65536:1        | 16            | 4.8     |

- Most digital cameras use a 10 – 14 bit A/D so their theoretical maximum dynamic range is 10-14 f-stops.
- Higher precision A/D converter does not necessarily mean greater dynamic range. Total dynamic range is usually limited by noise levels.

# Measuring Luminance



## Nit ( $\text{Candela}/\text{m}^2$ )

- 1 nit ( $\text{cd}/\text{m}^2$ ) approximately equal to the amount of light from a candle dispersed over a square meter



The real world is high dynamic range

| Object                   | Approximate Luminance (nits) |
|--------------------------|------------------------------|
| Sun                      | 1,600,000,000                |
| Arc lamp                 | 150,000,000                  |
| Maximum visual tolerance | 50,000                       |
| Cloud (sunny day)        | 35,000                       |
| 2020 UHD-TV              | 800-1000                     |
| Typical computer screen  | 100-300                      |
| White paper under lamp   | 50                           |
| Night sky                | 0.001                        |
| Threshold of vision      | 0.000003                     |





500,000  
nits



200,000  
nits



40  
nits



2,000 nits

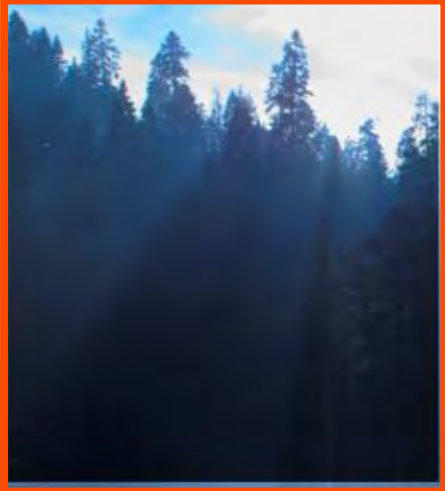
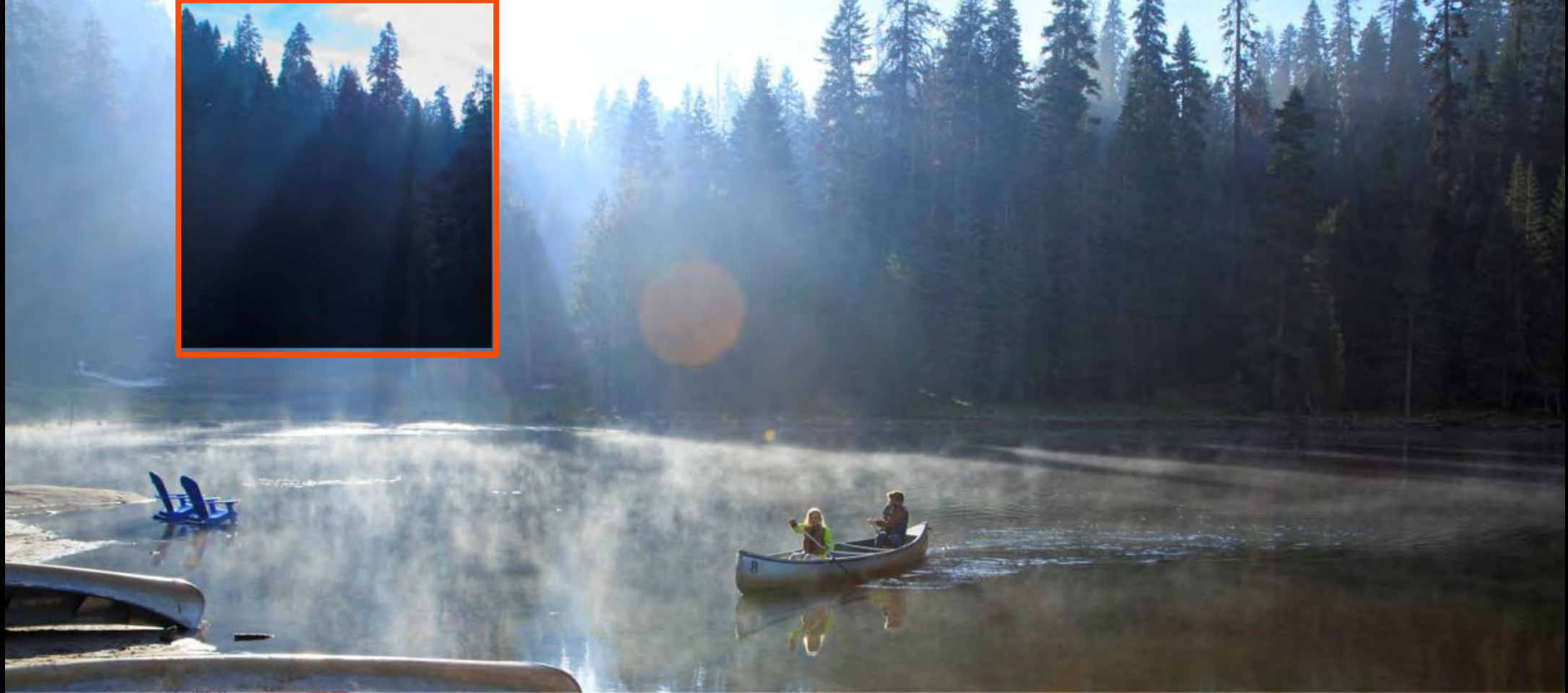














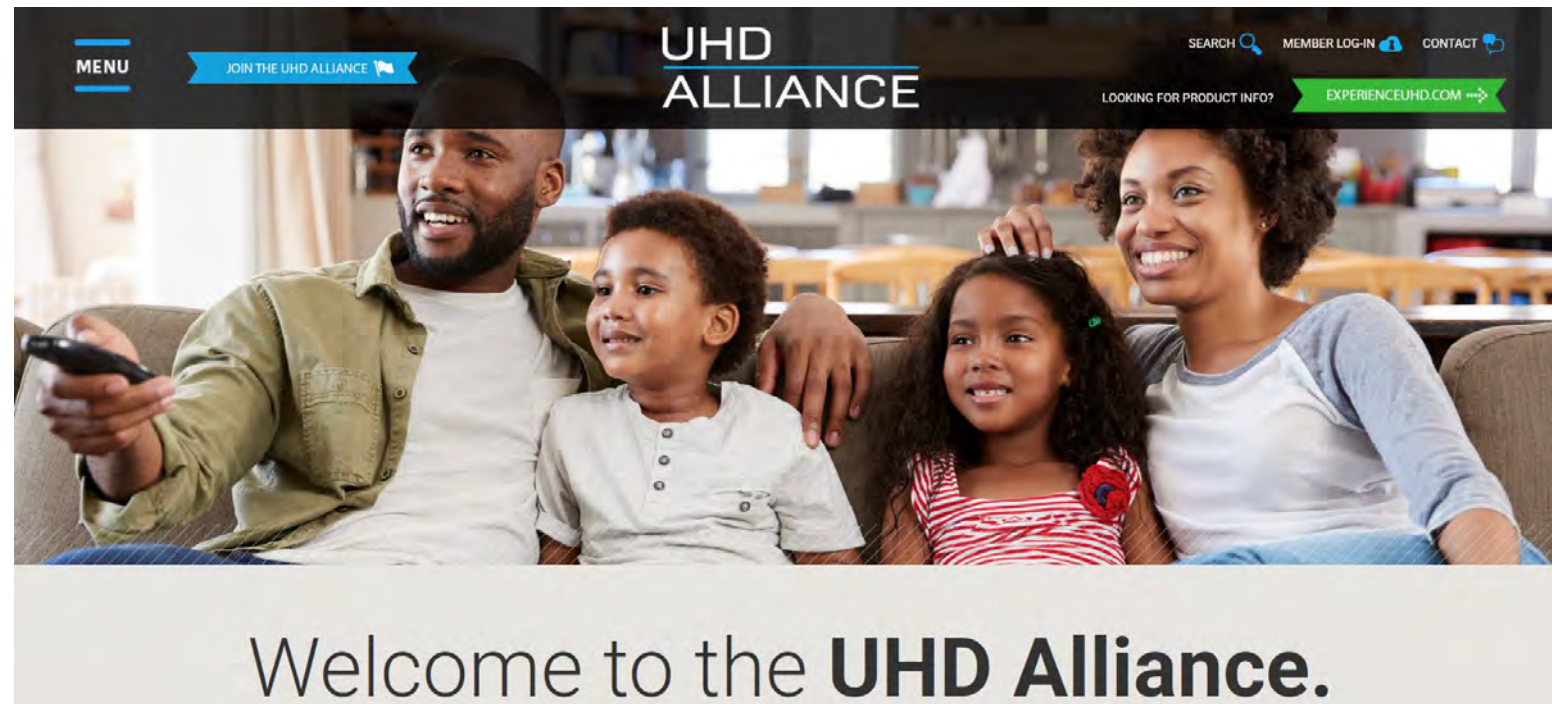
# HDR image examples





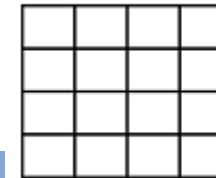
# Moving to HDR

- **UHD Alliance: with mission to create (Jan. 2015 at CES)**
  - *New standard to support video technologies including **4K**, Higher resolution, **high dynamic range**, **wider color gamut**, and **3D audio***

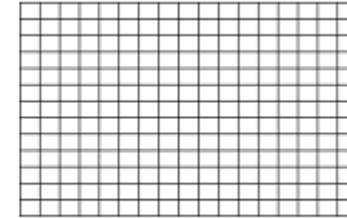


# Improving Image Quality

More pixels: SD – HD –UHD (4k, 8K)



Low resolution



High resolution

**SDTV**  
**HDTV**  
**UHDTV**



UHD vs. HD. vs SD resolution comparison

LEDTVReviews.com

**Increase spatial resolution**



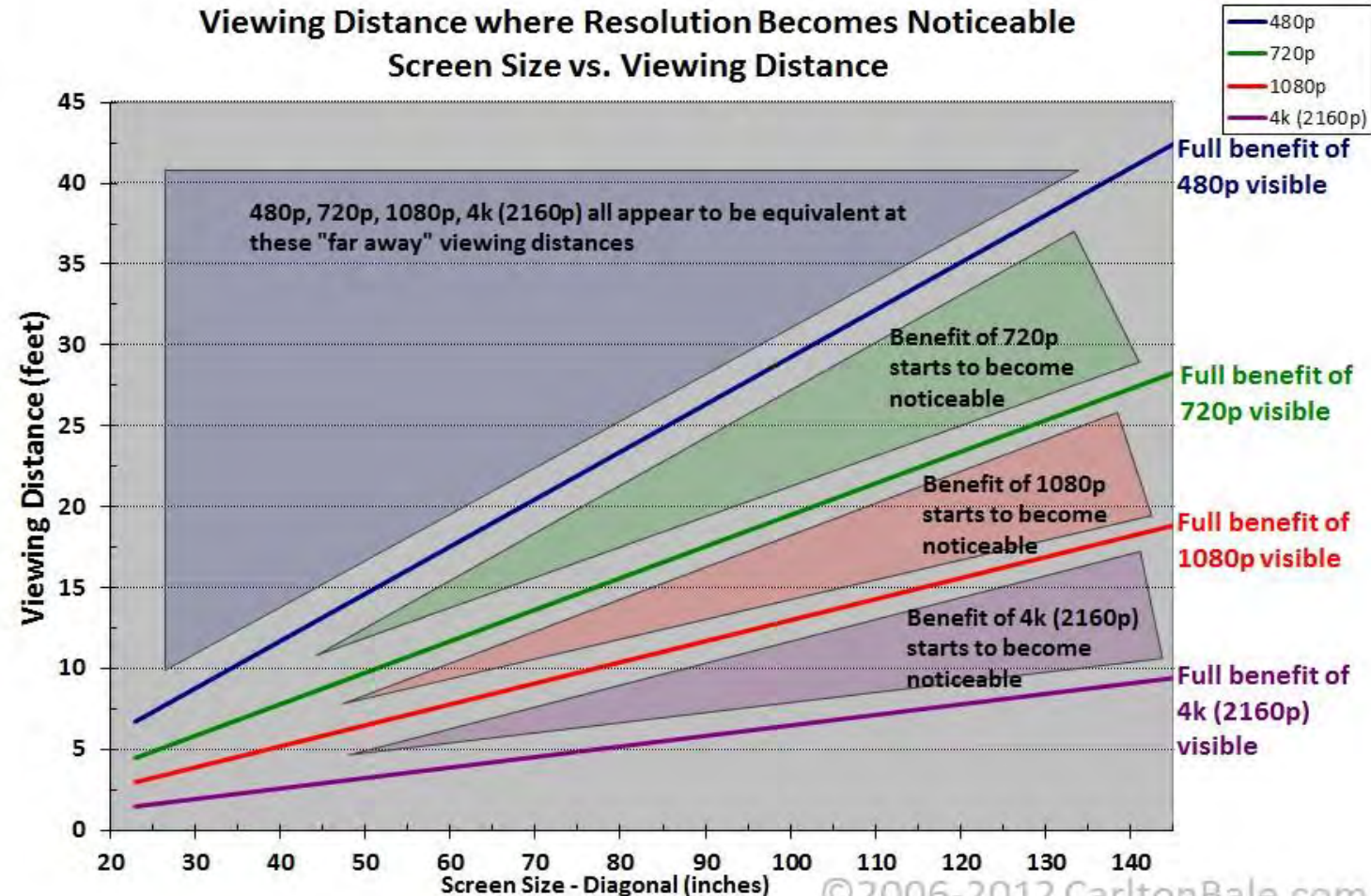
# Moving to HDR

Dramatic better  
visual quality

- (Netflix) HDR is more important for video quality than 4K
  - “We kind of ran out of more pixels to add,” Neil Hunt

Resolution is only apparent  
when you are close

**Users can notice HDR from  
any distance and screen size,  
at once**

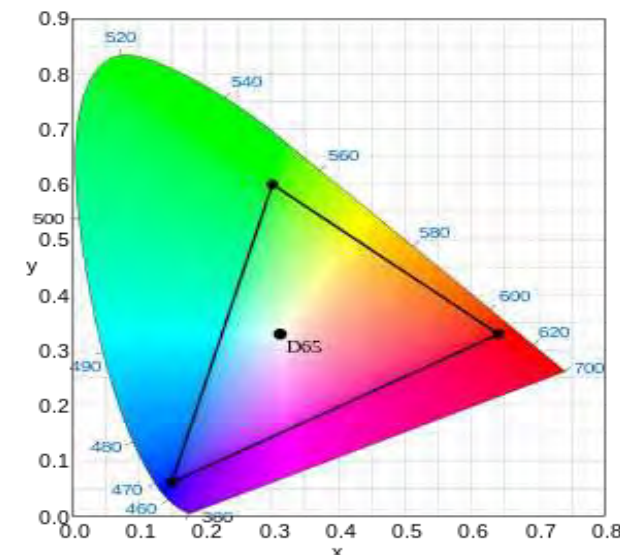


# High Dynamic Range

- Need around 10.000 nits to satisfy 90% viewer at ideal range



Current Color Space (Color Gamut)  
is not enough

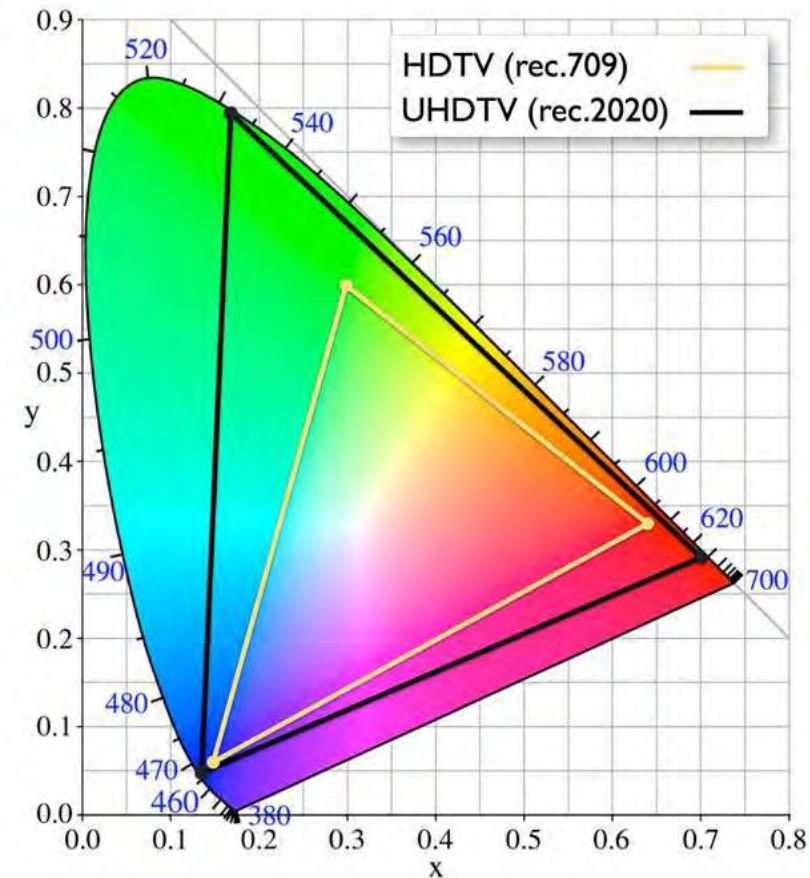


# Wider Color Gamut

- Current color gamut is limited
  - Current HDTV (BT.709) – 1990
  - Created for CRT, and HDTV
  - Around 100 nits.
  - 8 bpp (bits per pixel)

Today we have better display technology: LCD, OLED ...

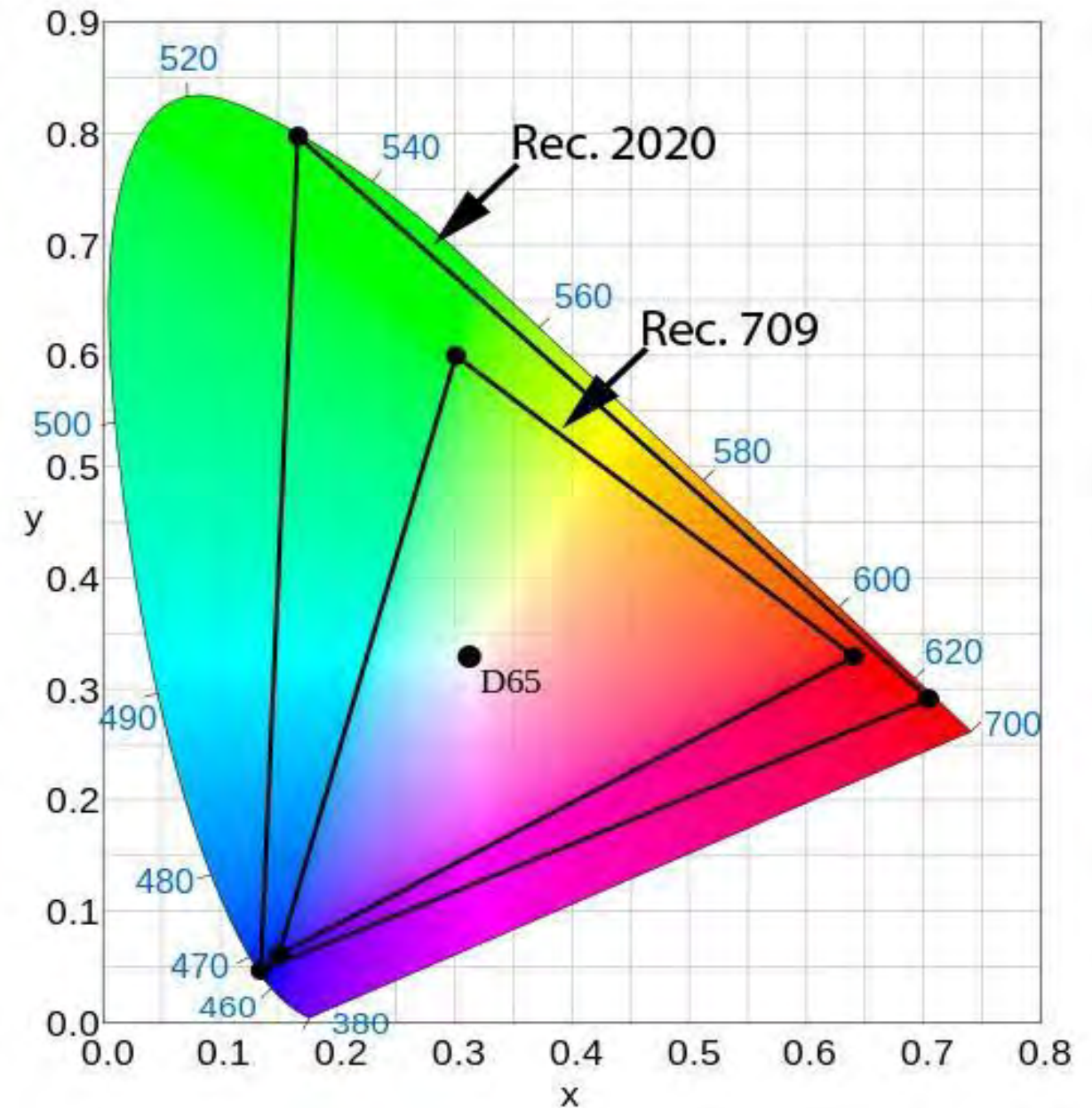
More Colors: REC. 2020 for UHDTV 10, 12 bpp.

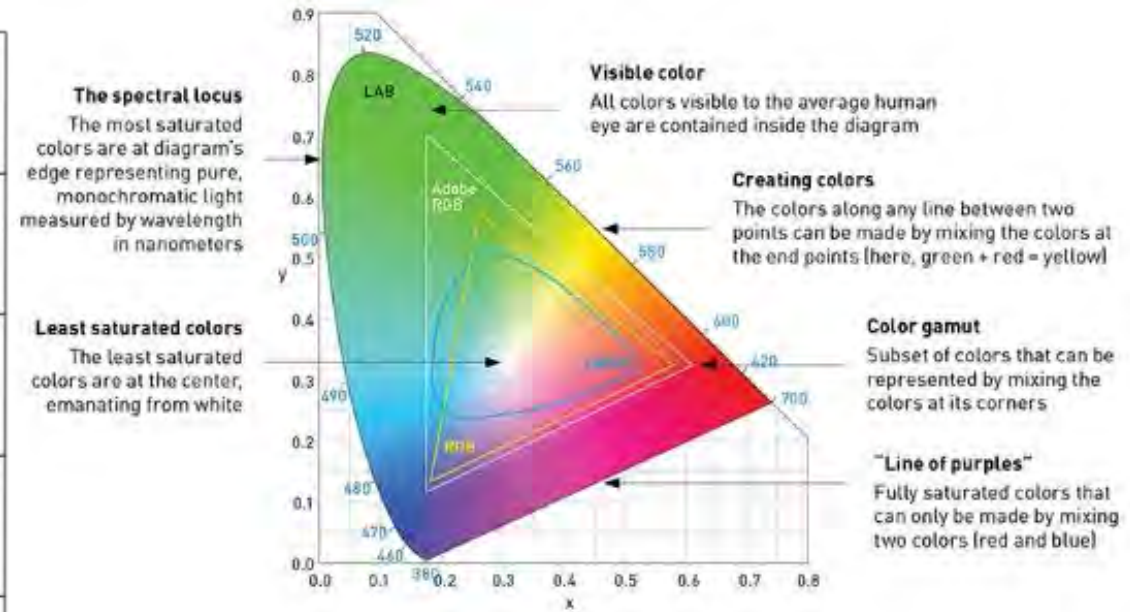
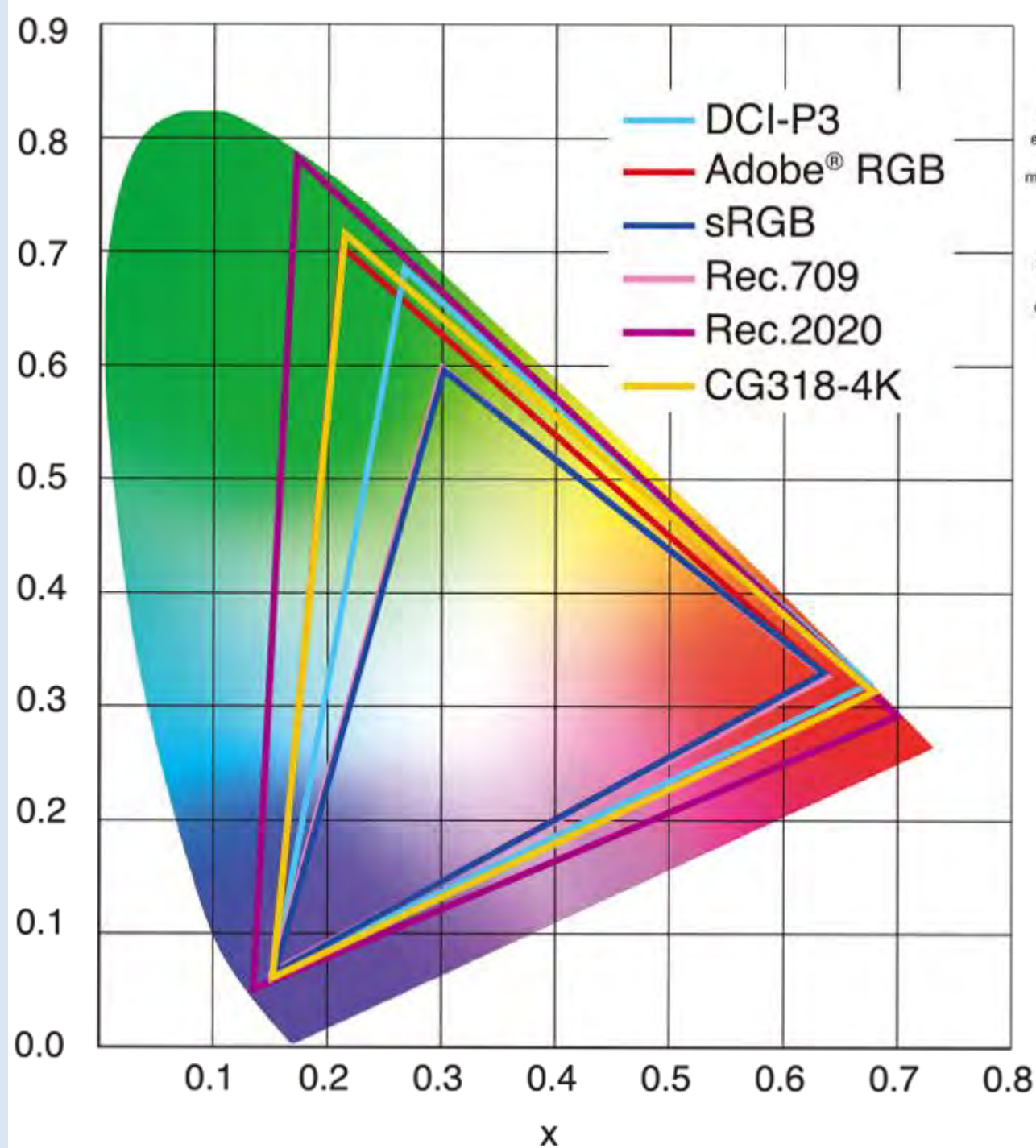


UHDTV vs HDTV Color Gamuts



- Most PCs and HDTVs use 709 gamut
- Majority of next generation content uses Bt.2020 gamut
- 2015 UHD TVs are ~85% Bt.2020





Color profiles of video and monitors



**Bt 2020 content  
displayed unmodified  
on 709 display**



**Bt 2020 content  
gamut-mapped for  
709 display**



**We must perform gamut mapping**





# Standard Dynamic Range (SDR)

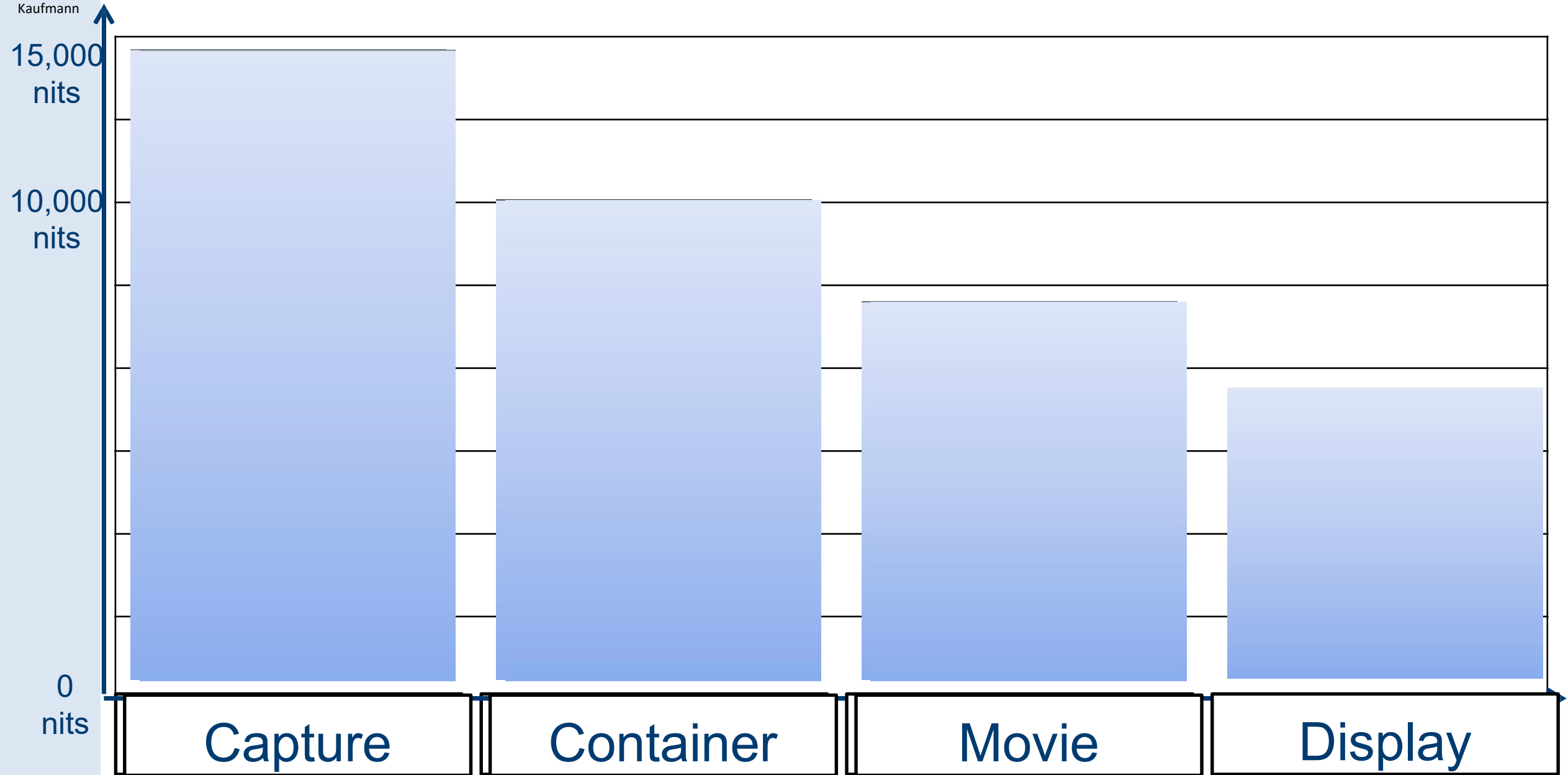
- For the past 80 years, video has been graded to appear properly on a **100 nit** display
- However, no adjustments are made to comprehend displays of differing luminance
  - 30 nit laptops in low power mode
  - 600 nit HDTVs in Vivid mode
- Similarly, no adjustments to the content are made when you surf the web and play games, switching from phones to tablets to PCs.

This is **wrong**, but has been **good enough**

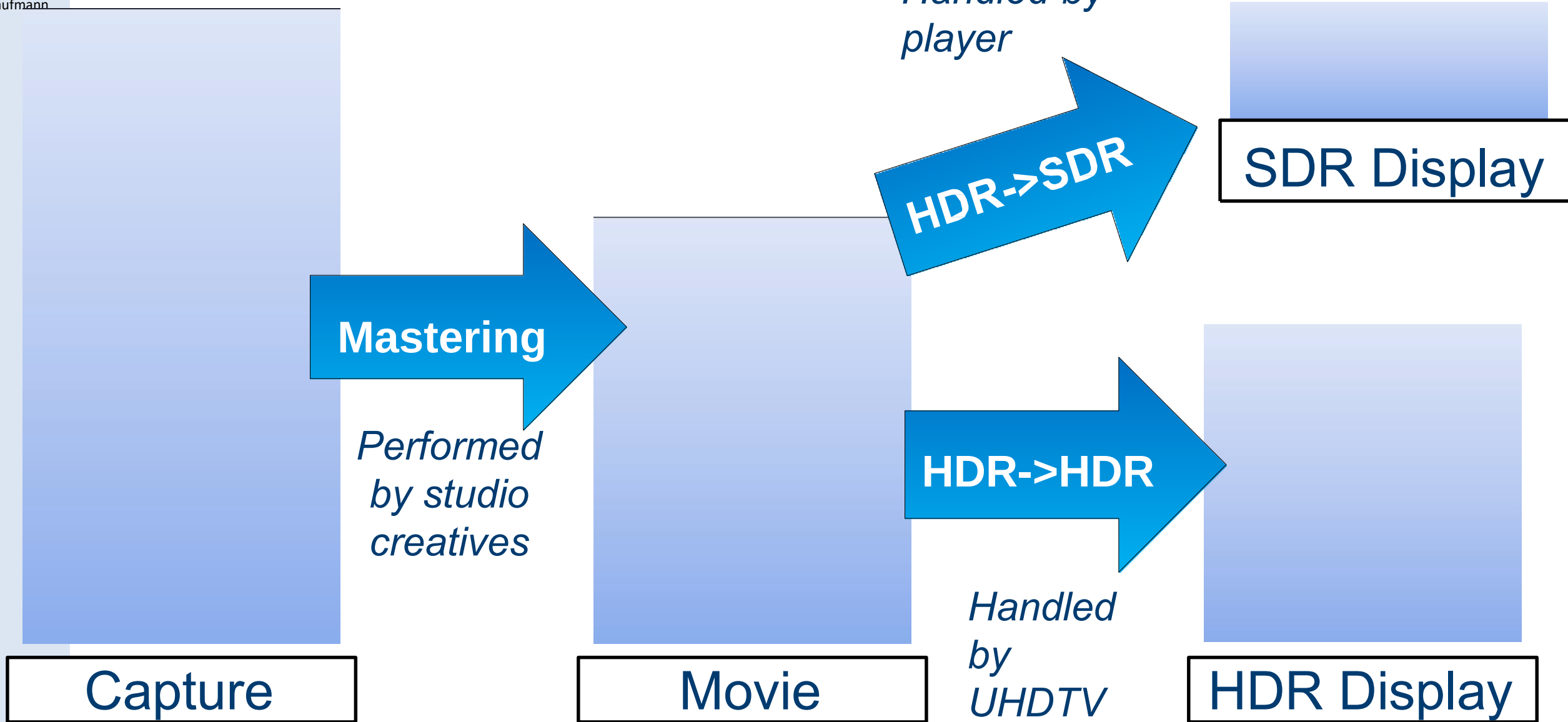
# Things to ponder

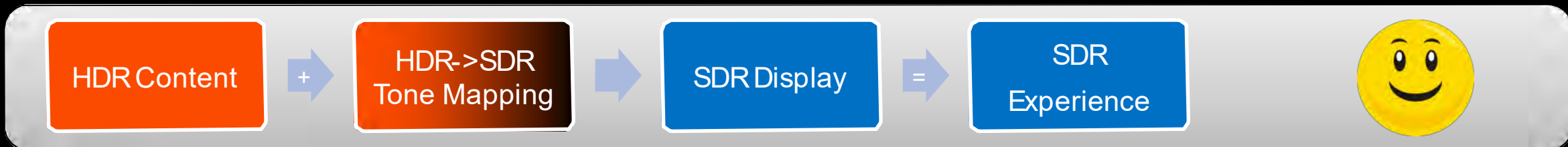
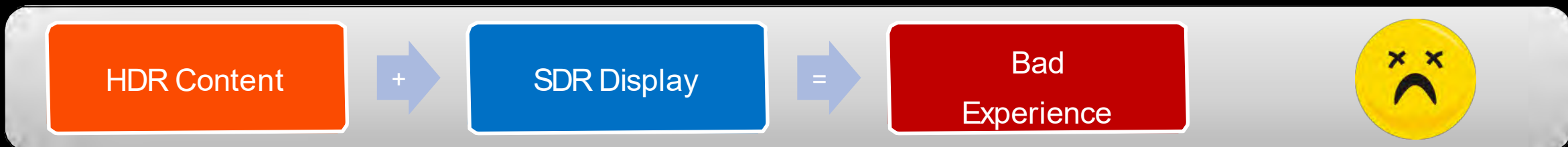
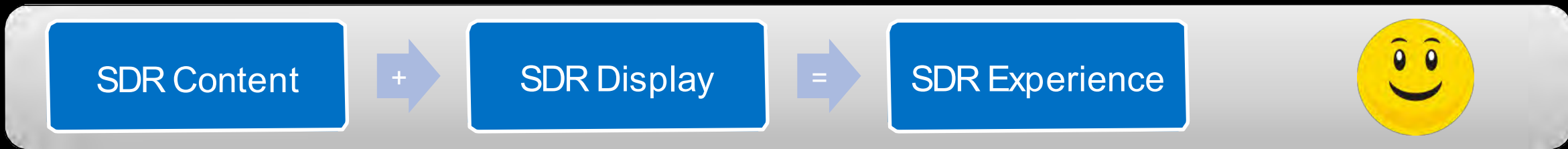
- With color, many real-world hues can be exactly reproduced on screens
- Almost every real-world situation has objects at  $>100$  nits
- So the real world is HDR, and every SDR picture you take involves HDR- $\rightarrow$ SDR conversion
- A 100-nit object on the screen almost never corresponds to a 100-nit object in the real world

# Multiple Ranges of HDR



# HDR Conversions





# Real World

**3,000  
nits**

**300,000  
nits**





# Mastered for SDR

**100  
nits**



**Coded  
value=  
0xFF**

**100  
nits**



**Coded  
value=  
0xFF**

# Mastered for HDR

200  
nits



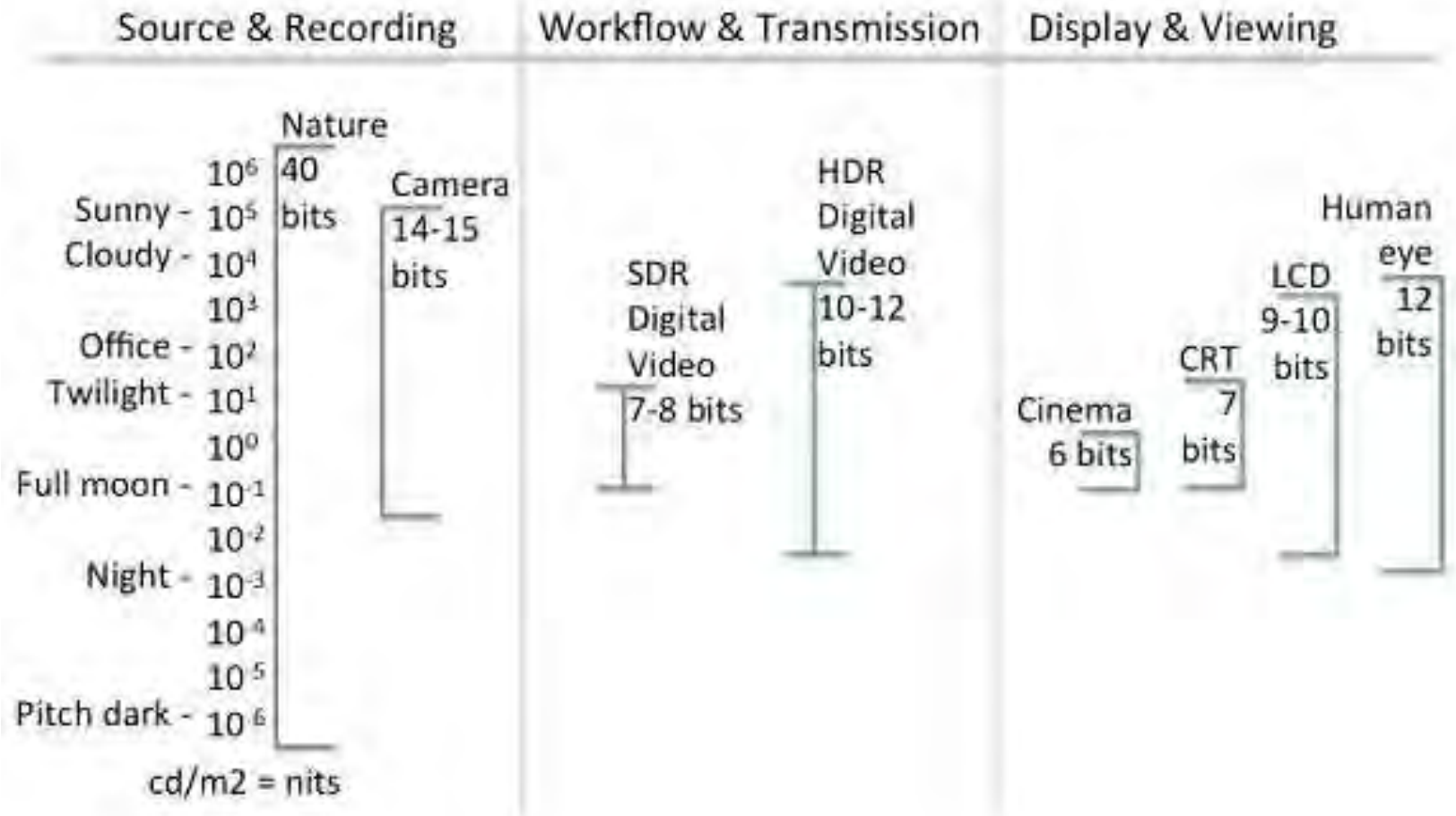
Coded  
value=  
0x23B

5,000  
nits



Coded  
value=  
0x36C

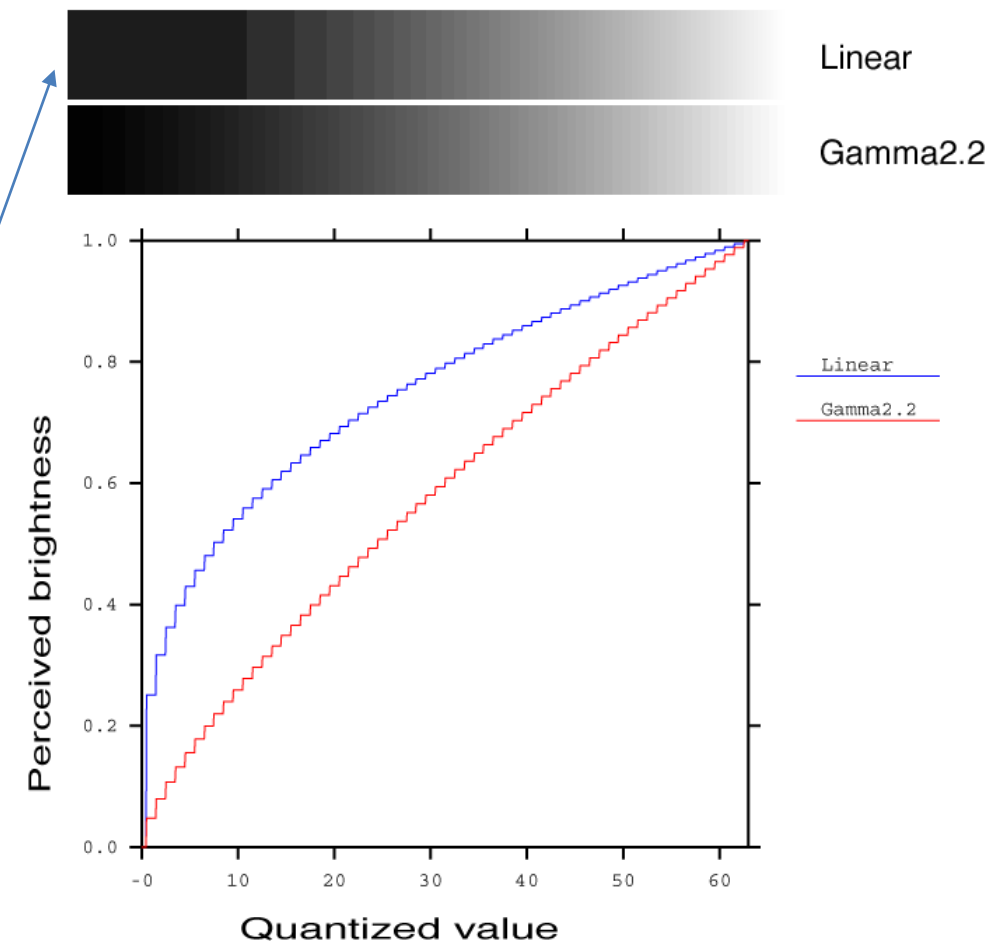
# Color and Brightness



Luminance dynamic range for various technologies

# Quantization: Linear vs non-linear light

- In the real world, luminance is determined by the number of photons
- However, brightness (the human perception of luminance) is non-linearly proportional to number of photons
  - The eye has a non-linear response to intensity. At most adaptation levels, we perceive brightness roughly as the cube root of intensity.
- If we apply a linear quantization of color values, we see more steps in darker regions than we do in the brighter regions
- Using a power law encoding with a  $g$  value of 2.2, we see a much more even distribution of quantization steps, though the behavior near black is still not ideal

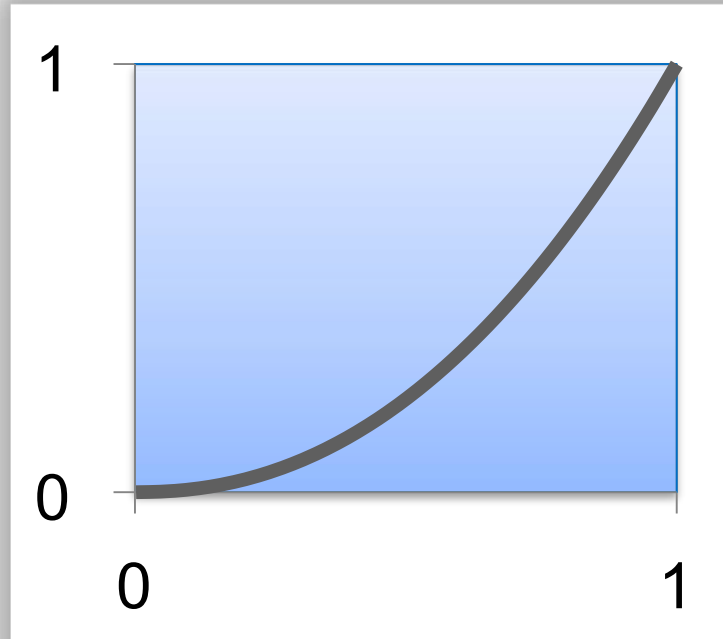
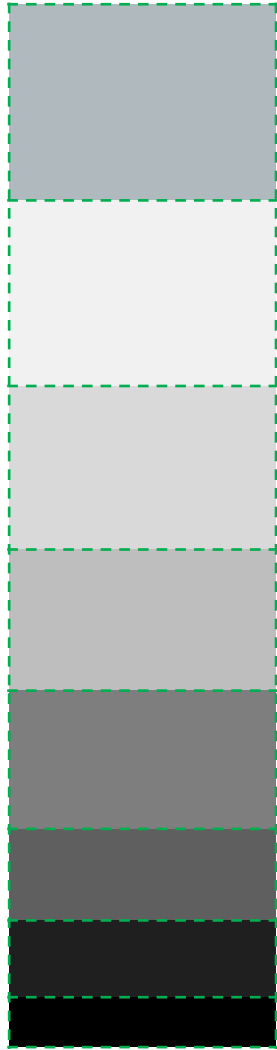


To efficiently code video signals, we apply a non-linear transform function so that we can code perceptually uniform brightness intervals

Luminance

Linear Light

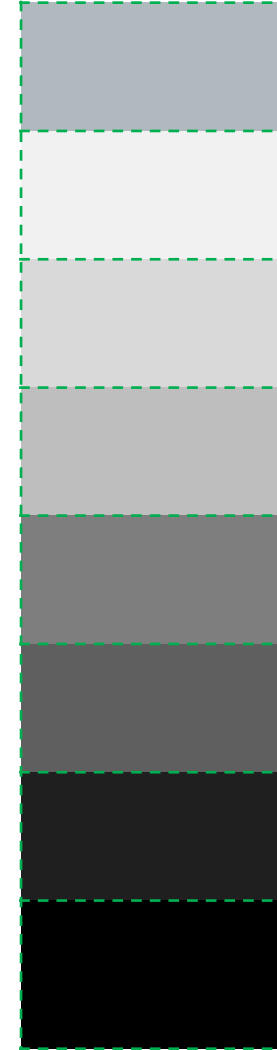
Perceptually  
non-uniform



Luma

Non-Linear Light

Perceptually  
Uniform





# Problems

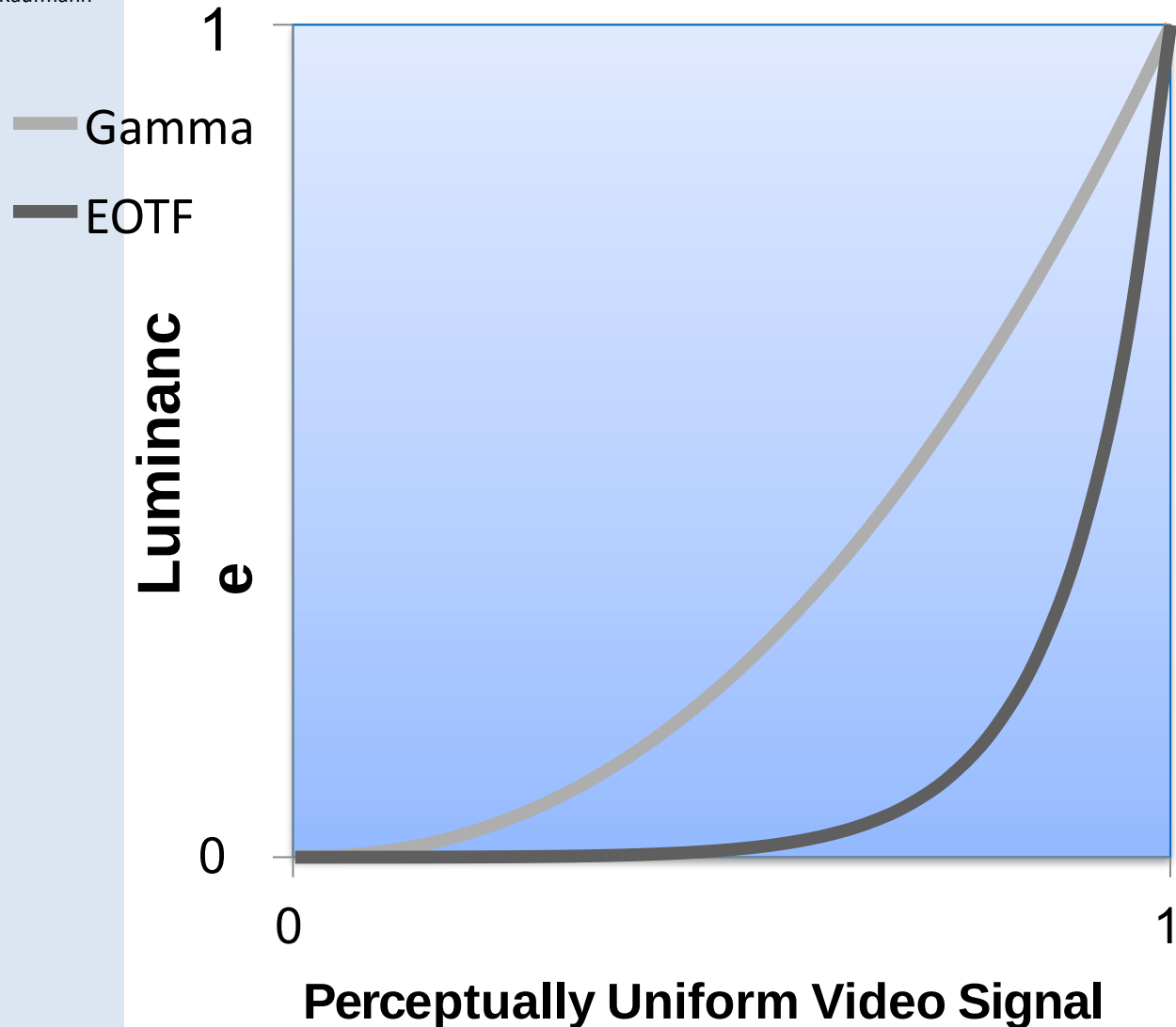
- Although gamma is a reasonable approximation of human perception in the SDR (0-100 nit) range...
- Gamma is not a good match for human perception in the 0-10,000 nit HDR
- If you use gamma for a 10,000 nit signal, you need to use two extra bits/sample to eliminate banding

Color banding





# SMPTE 2084: HDR Electro-Optical Transfer Function (EOTF)



Electro-Optical Transfer Function (EOTF) is a correction function that **converts electrical voltages into brightness values**.

- If gamma curve is used = gamma correction, if more complex dependencies between voltage and brightness, an EOTF is used
- EOTF is used to adjust the brightness of displays or televisions to the human perception of brightness. Since this does not run linearly with voltage values, the display representation must be adapted.
- **Dependencies between brightness and voltage are different for various display technologies** and resolutions, such as SDTV, HDTV, UHDTV, HDR10 and Dolby Vision.
- Therefore several correction curves in addition to gamma correction: Perceptual Quantization (PQ) and Hybrid Log Gamma (HLG), which is described in the SMPTE's ST.2084 standard for video transmission of High Dynamic Range (HDR).

# Hybrid Log Gamma (HLG) & Perceptual Quantization

- HLG correction is comparable to gamma correction in classic television.
- Hybrid Log Gamma (HLG) and Perceptual Quantization (PQ)
  - standardized by the International Telecommunication Union (ITU) in BT.2100
  - adapting the dynamic range of the display on the UHD displays to the dynamic range of the recording devices.
  - Why: So that display corresponds to the viewer's perception of brightness.
  - Correspond to a logarithmic function
  - The brightness range corresponds to a luminance between 100 nits and 5,000 nits.
  - HLG correction curve requires a resolution of at least 10 bits and is used with High Dynamic Range HDR10 and Dolby Vision in transmission and mastering.
  - When transmitting video in SDR technology and HDR technology simultaneously in one data stream, the image content is encoded with the HLG curve so that it can be played back on both SDR and HDR displays.

# What is HDR (in this context)

- Content with a wider range of brightness and color
- Requires new monitors to experience HDR content
  - Legacy content (SDR): 8-bit color, 100 nit peak brightness, 709 gamut
    - Dynamic range of about 6 stops
  - HDR content: 10-bit color, 10,000 nit peak brightness, 2020 gamut
    - Dynamic range of about 17,6 stops

HDR is an ambiguous term

- There are several industry specs defining various flavors of HDR

# Summary

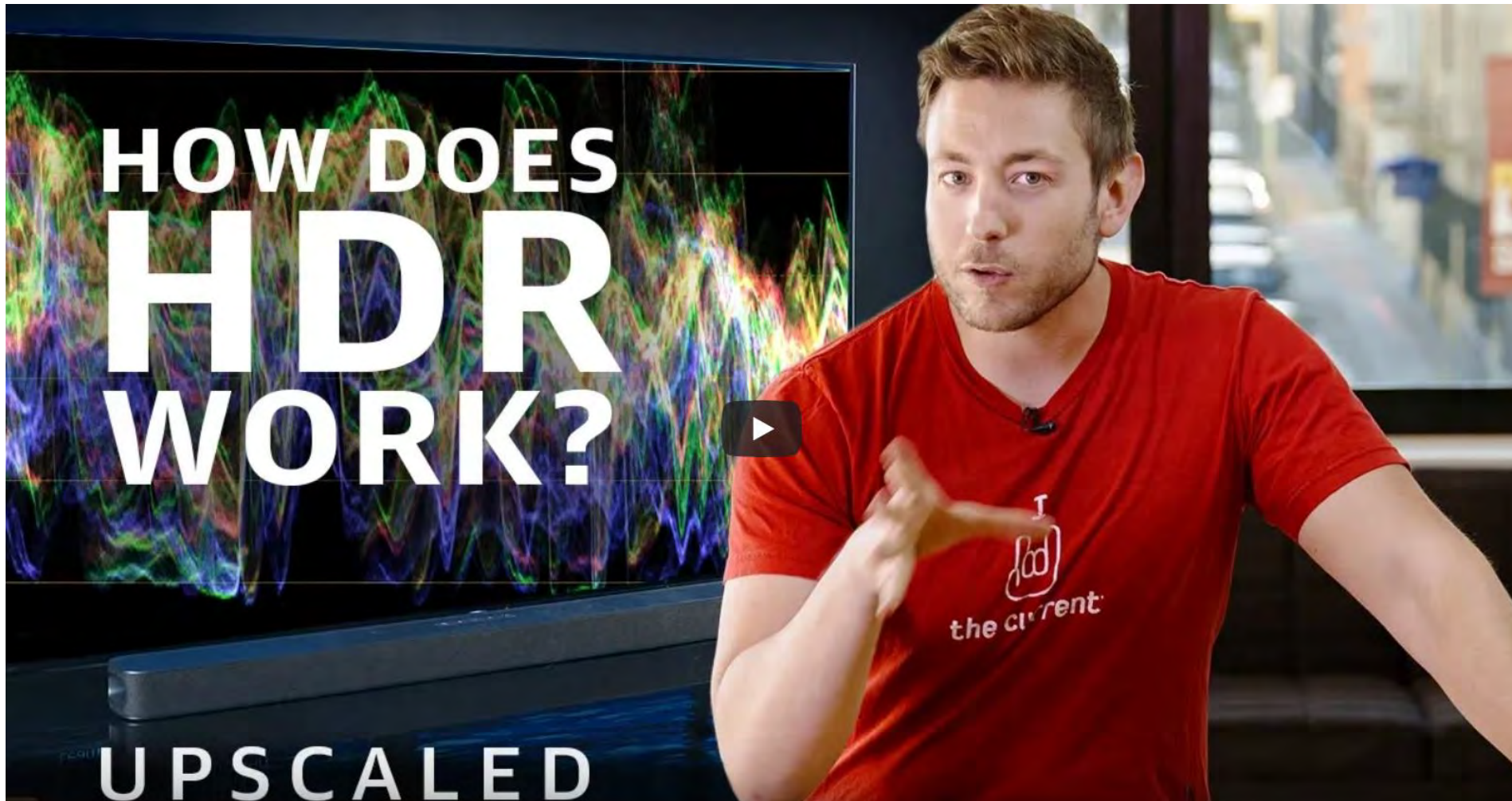
- HDR + Wide color gamut (WCG) gives us powerful new tools to create visual experiences that are clearly visible to the average viewer
- Making this all work requires lots of changes to the production pipeline and the products used to display it

# HDR Encoding

- **Most formats are lossless**
- **Adobe DNG (digital negative)**
  - Specific for RAW files, avoid proprietary formats
- **RGBE**
  - 24 bits/pixels as usual, plus 8 bit of common exponent
  - Introduced by Greg Ward for Radiance (light simulation)
  - Enormous dynamic range
- **OpenEXR**
  - By Industrial Light + Magic, also standard in graphics hardware
  - 16bit per channel (48 bits per pixel) 10 mantissa, sign, 5 exponent
  - Fine quantization (because 10 bit mantissa), only 9.6 orders of magnitude
- **JPEG 2000**
  - Has a 16 bit mode, lossy

# HDR Technologies Summary

Luminance  
Dynamic Range  
Color Space  
Bit Depth





# HDR formats

- Summary of all HDR encoding formats (Greg Ward):  
[http://www.anywhere.com/gward/hdrenc/hdr\\_encodings.html](http://www.anywhere.com/gward/hdrenc/hdr_encodings.html)
- Greg's notes: <http://www.anywhere.com/gward/pickup/CIC13course.pdf>
- <http://www.openexr.com/>

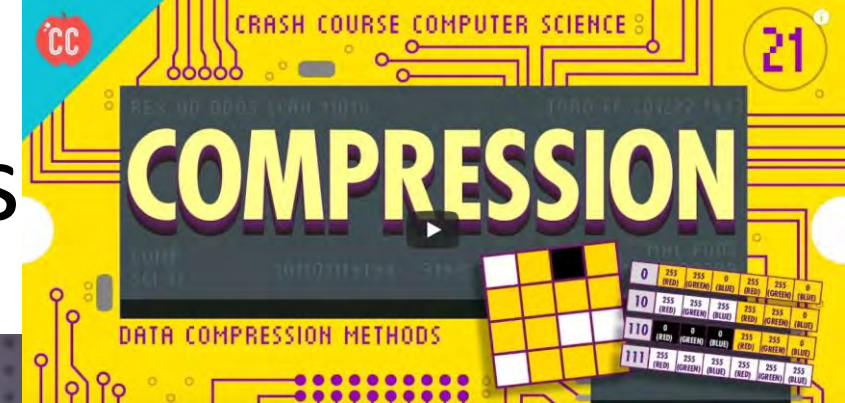
# Some Standard Image Formats

- EPS — Encapsulated PostScript
- **GIF — Graphics Interchange Format**
- PICT — may contain geometric data (lines, polygons, etc.)
- **JPEG — Joint Photographic Experts Group, lossy**
- PNG – includes transparency (alpha channel)
- and many many more....

# Image Compression

# Recap: Compression Algorithms

<https://www.youtube.com/watch?v=OtDxDvCpPL4>



# Compression Techniques - Classification

|                |                     |      |
|----------------|---------------------|------|
| Entropy coding | run-length coding   |      |
|                | Huffman coding      |      |
|                | arithmetic coding   |      |
| Source coding  | prediction          | DPCM |
|                |                     | DM   |
|                | transformation      | FFT  |
|                |                     | DCT  |
|                | vector quantization |      |
| hybrid coding  | JPEG                |      |
|                | MPEG                |      |
|                | px64                |      |
|                | DVI (RTV, PLV)      |      |

# Entropy Coding

- **lossless** encoding
- used **regardless of media's** specific **characteristics**
- data taken as simple digital sequence
- One of the main types of entropy coding
  - creates and **assigns a** unique prefix-free **code to each unique symbol** that occurs in the input.
  - then **compress data by replacing** each fixed-length **input symbol with** the corresponding variable-length prefix-free output **codeword**.
- Examples: **Run-length encoding, Huffman encoding, Arithmetic encoding**



# Source Coding & Hybrid Coding

- Usually **lossy** encoding (can also be lossless)
- **takes into account the data semantics**
- degree of compression **depends on data content**
- In source coding the particular **code** to be used **is chosen to match the source** (i.e. the relative probabilities of the symbols in the source alphabet).
- Example: Discrete Cosine Transformation (DCT) as transformation technique of the spatial domain into the two-dimensional frequency domain

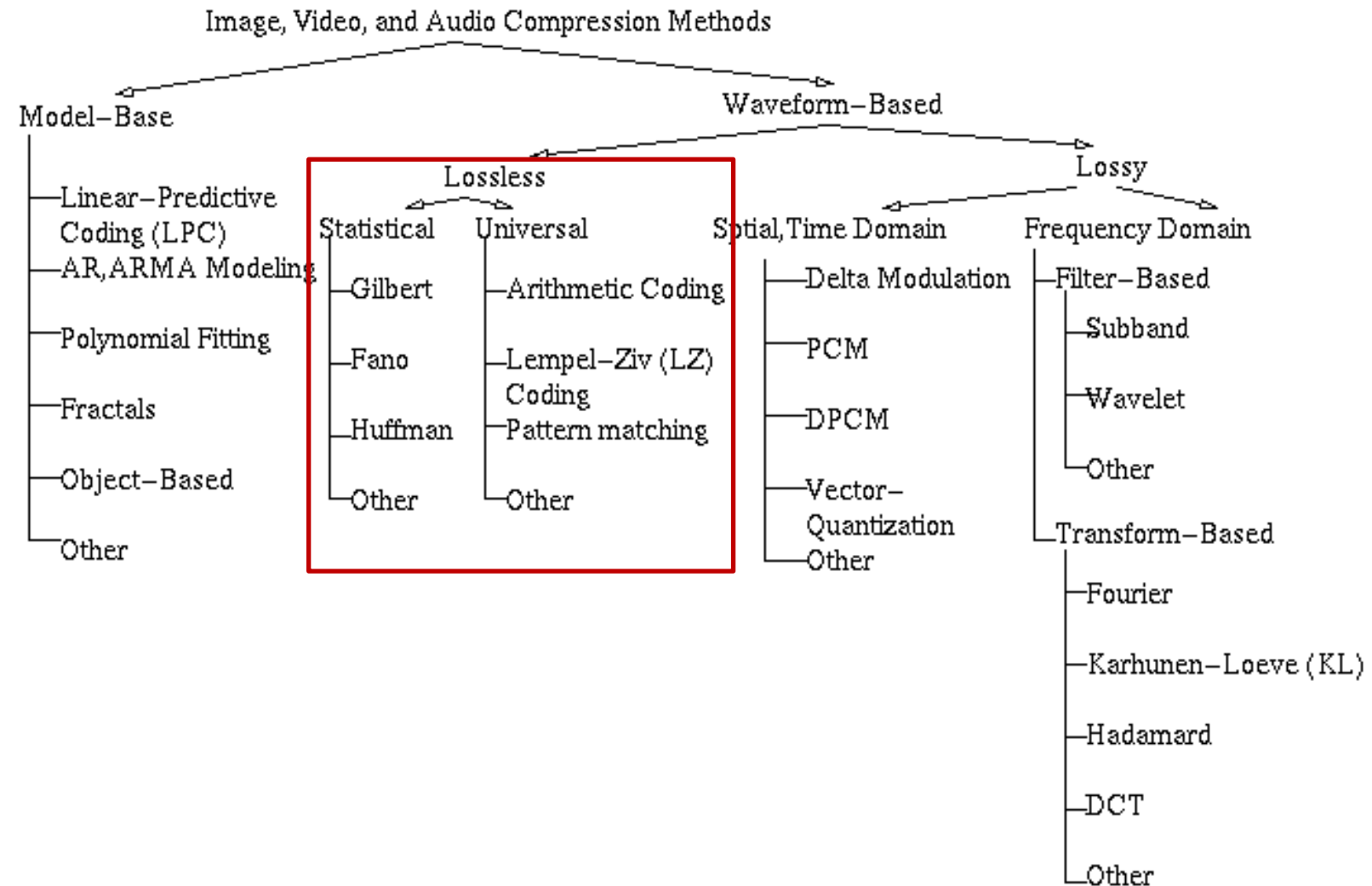
## Hybrid encoding

- Used by most multimedia systems
- Combination of entropy and source encoding
- Examples: JPEG, MPEG, H.261

# Decompression Requirements

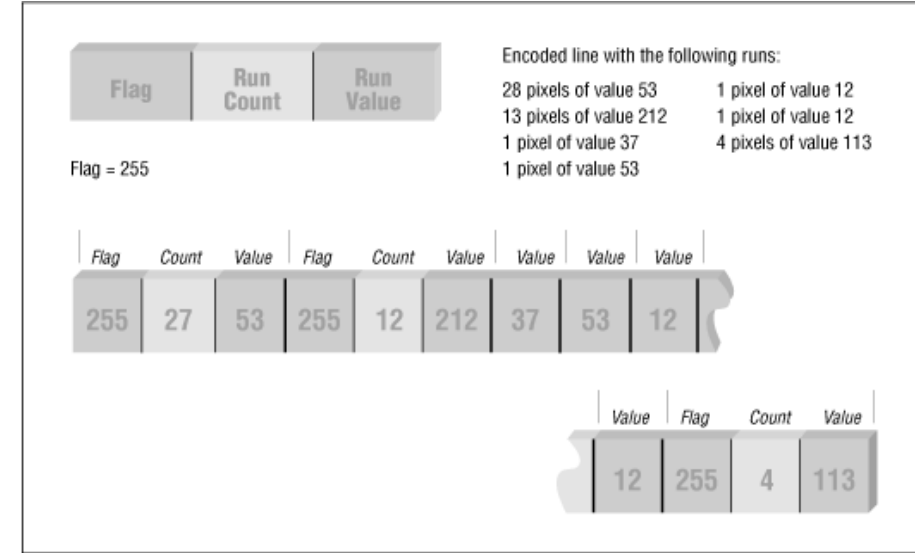
- Dialog mode applications (e.g. editing) might need **symmetric** compression  
= compression and decompression take about the same time
- Retrieval mode applications (e.g. video playback) use **asymmetric** compressions
  - compression is performed once and sample time available
  - decompression is performed frequently and needs to be done fast (in case of time-dependent media)

# Lossless Compression Algorithms



# Run-Length Encoding

- **Entropy coding** algorithm—  
content independent coding
- Replaces the sequence of the same consecutive bytes with the number of occurrences
- the number of occurrences is indicated by a *special flag (!)*
- Procedure:
  - if the same byte occurs at least 4 times count number of occurrences
  - write compressed data in the format
    - the counted byte '!' number of its occurrences



# Run-length Encoding

- Example:
  - uncompressed sequence (20 Bytes):  
ABCCCCCCCCCDEFFFFGGG
  - compressed sequence (13 Bytes): ABC!9DEF!4GGG
- Variations
  - Zero suppression technique (remove zeros)
  - text compression technique (also for images, video, audio)
  - diatomic encoding (represents a pair of characters as one special character)
- Is used for older image file formats e.g. Windows Bitmap, Targa or PCX.  
In Windows \*.rle is used

# Statistical Encoding

- Frequency dependent encoding—belongs to **entropy encoding**
- given is a sequence of symbols:  $S_1, S_2, S_3, \dots$  and the probability of occurrence of each symbol  $p(S_i)=p_i$
- Ex.:  $p(A)=0.16, p(B)=0.51, p(C)=0.09, p(D)=0.13, p(E)=0.11$   
first choice: encode A,B,C,D,E as 000,001,010,011,100

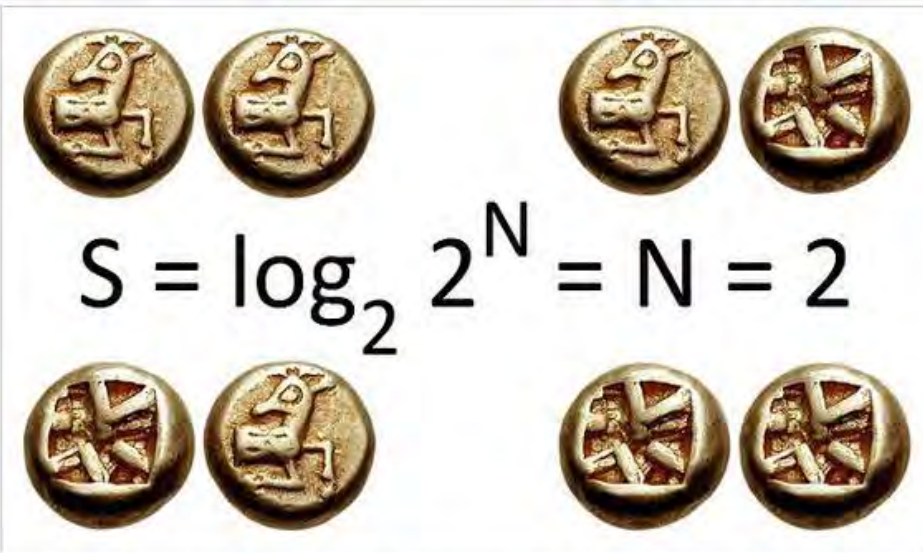
Q: What is the minimum number of bits per symbol?

A: The theoretical minimum average number of bits per code word is known as **entropy** ( $H$ ), according to Shannon



# Entropy

- Introduced by Claude Shannon in his 1948 paper "A Mathematical Theory of Communication" - sometimes called **Shannon entropy**
- Example: A biased coin with probability  $p$  of landing on heads and probability  $1-p$  of landing on tails. The maximum surprise is for  $p = 1/2$ , when there is no reason to expect one outcome over another, and in this case a coin flip has an entropy of **one bit**. The minimum surprise is when  $p = 0$  or  $p = 1$ , when the event is known and the entropy is zero bits.
- Shannon's experiments with human predictors show an information rate between 0.6 and 1.3 bits per character in English.
- Shannon's entropy **measures the information contained in a message** as opposed to the portion of the message that is determined (or predictable). Examples of the latter include redundancy in language structure or statistical properties relating to the occurrence frequencies of letter or word pairs, triplets etc.



Two bits of entropy: In the case of two fair coin tosses, the information entropy in bits is the base-2 logarithm of the number of possible outcomes; with two coins there are four possible outcomes, and two bits of entropy. Generally, information entropy is the average amount of information conveyed by an event, when considering all possible outcomes.

$$H(X) = - \sum_{i=1}^n P(x_i) \log P(x_i)$$

# Huffman Encoding

- Statistical encoding, depends on **occurrence frequency of single characters or sequences** of data bytes
- Characters stored with their probabilities
- Length (number of bits) of the coded characters differs
- Shortest code assigned to the most frequently occurring character
- To determine Code, it is useful to construct a **binary tree**
  - leaves are characters to be encoded
  - nodes carry **occurrence probabilities** of the characters belonging to the subtree

# Huffman Encoding — Algorithm

build a Huffman Tree:

$p(A)=0.17$ ,  $p(B)=0.51$ ,  $p(C)=0.08$ ,  $p(D)=0.14$ ,  $p(E)=0.1$

```
build ordered list of symbols (increasing probability)
```

```
do while list contains at least 2 elements
```

```
    construct tree using the first two elements in list
```

```
    add parent node for the union of these elements and compute probability
```

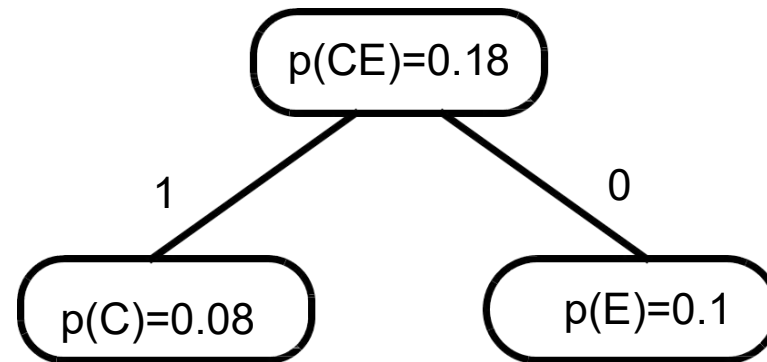
```
    mark edges by '0' and '1'
```

```
    delete the first 2 elements in list; insert parent into list
```

```
end
```

# Huffman Encoding—Example

step 1:  $p(C)=0.08$ ,  $p(E)=0.1$ ,  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(B)=0.51$



# Huffman Encoding—Example

step 2:  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(CE)=0.18$ ,  $p(B)=0.51$

build ordered list of symbols (increasing probability)

do while list contains at least 2 elements

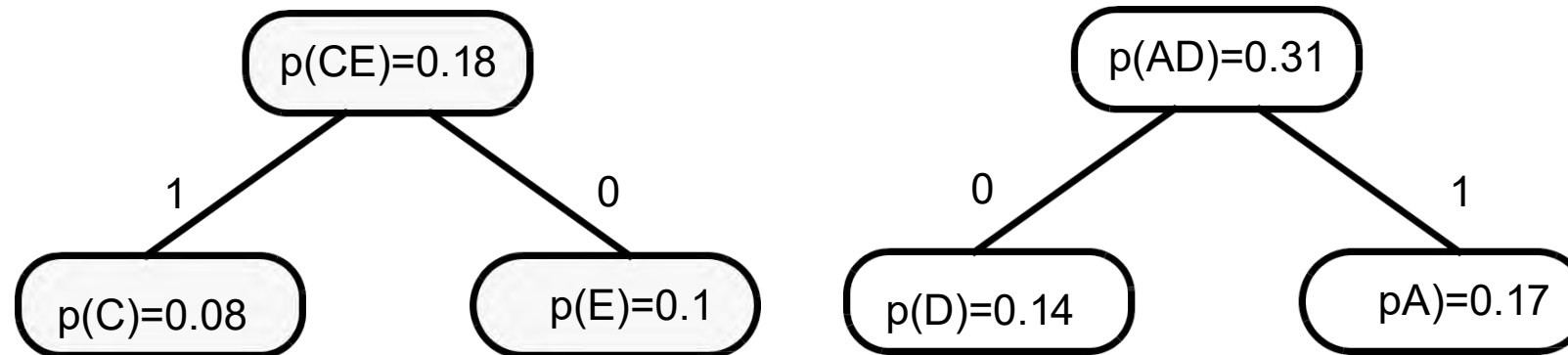
construct tree using the first two elements in list

add parent node for the union of these elements and compute probability

mark edges by '0' and '1'

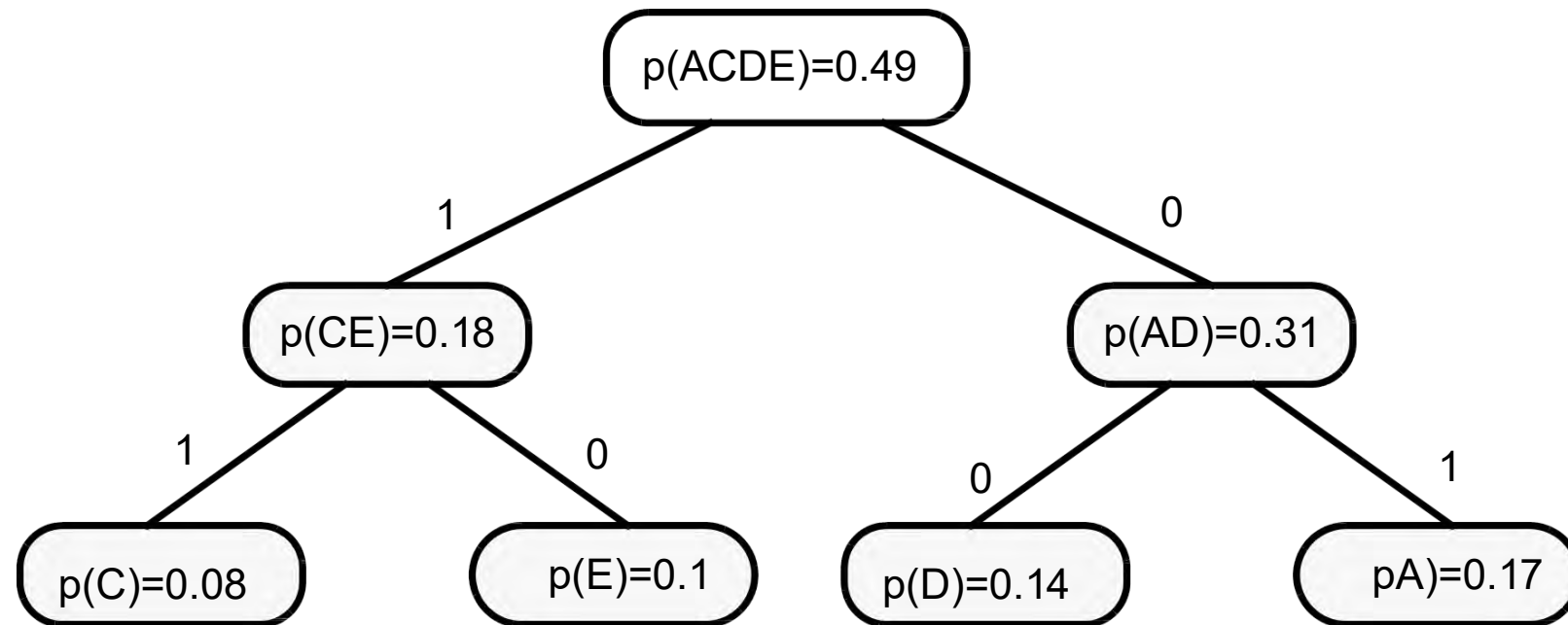
delete the first 2 elements in list;  
insert parent into list

end



# Huffman Encoding—Example

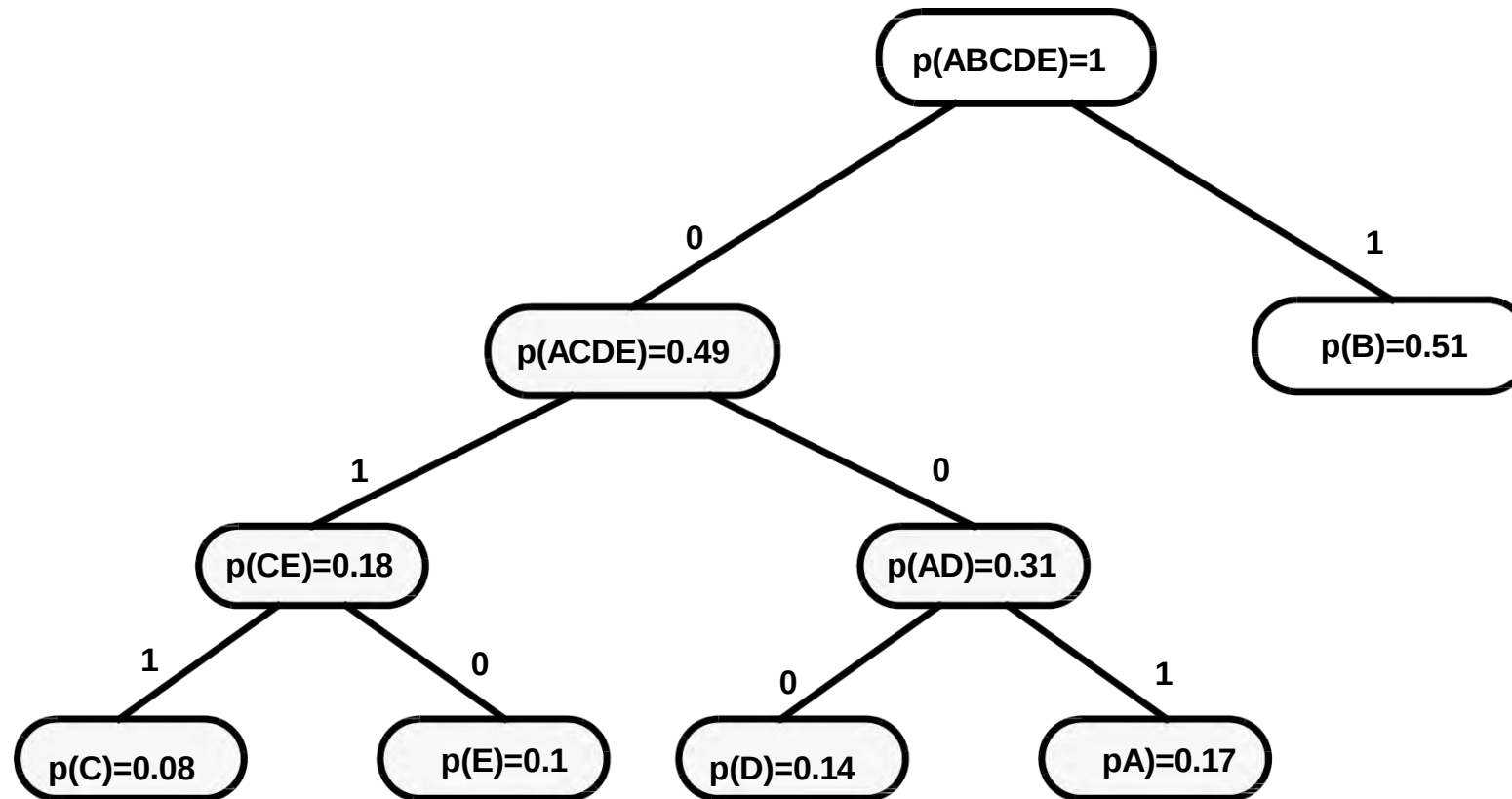
step 3:  $p(CE)=0.18$ ,  $p(AD)=0.31$ ,  $p(B)=0.51$





# Huffman Encoding—Example

last step:  $p(ACDE)=0.49$ ,  $p(B)=0.51$



# Huffman Encoding—Example

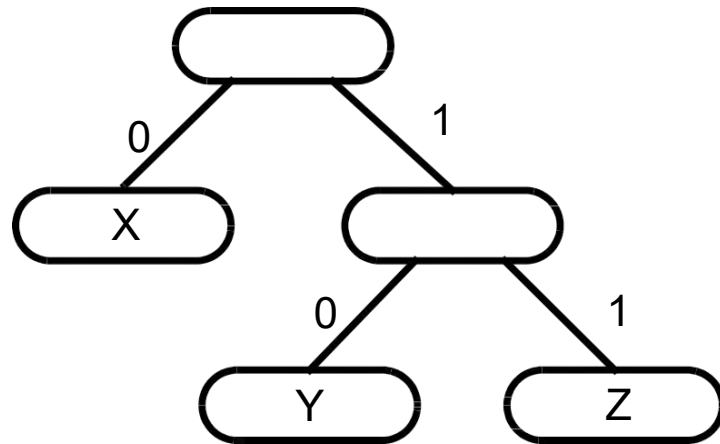
encode symbols according to tree

|   |     |
|---|-----|
| A | 001 |
| B | 1   |
| C | 011 |
| D | 000 |
| E | 010 |

$p(C)=0.08$ ,  $p(E)=0.1$ ,  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(B)=0.51$

# Huffman Decoding—Example

given: 000101011 and Huffman Tree

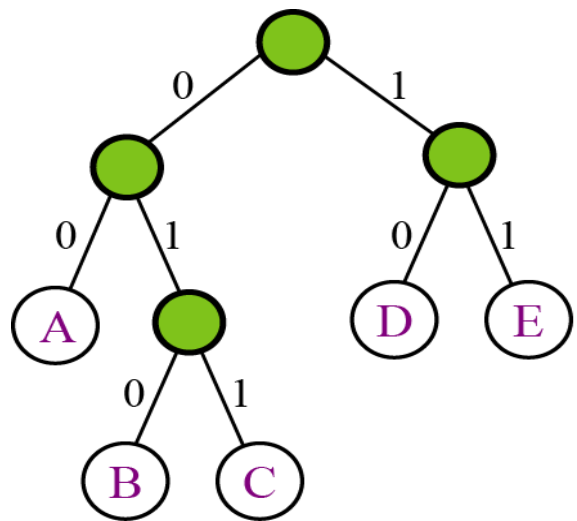


reconstruct message from bit stream using  
finite state machine

Solution: XXXYYZ

# Huffman Encoding

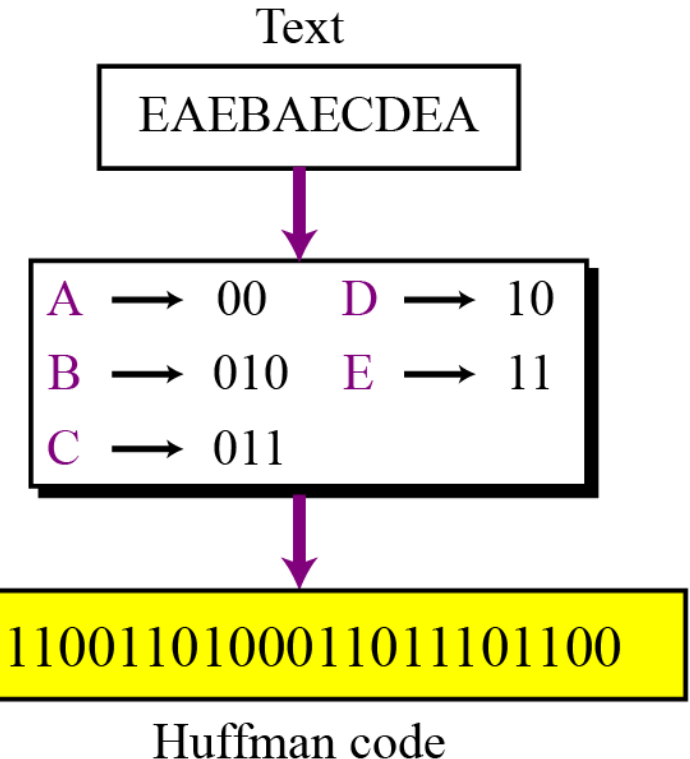
- Character code found by starting at the root and following the branches that lead to that character.
- The code itself is the bit value of each branch on the path, taken in sequence.



|        |       |
|--------|-------|
| A: 00  | D: 10 |
| B: 010 | E: 11 |
| C: 011 |       |

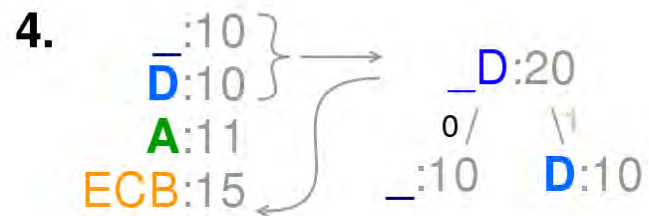
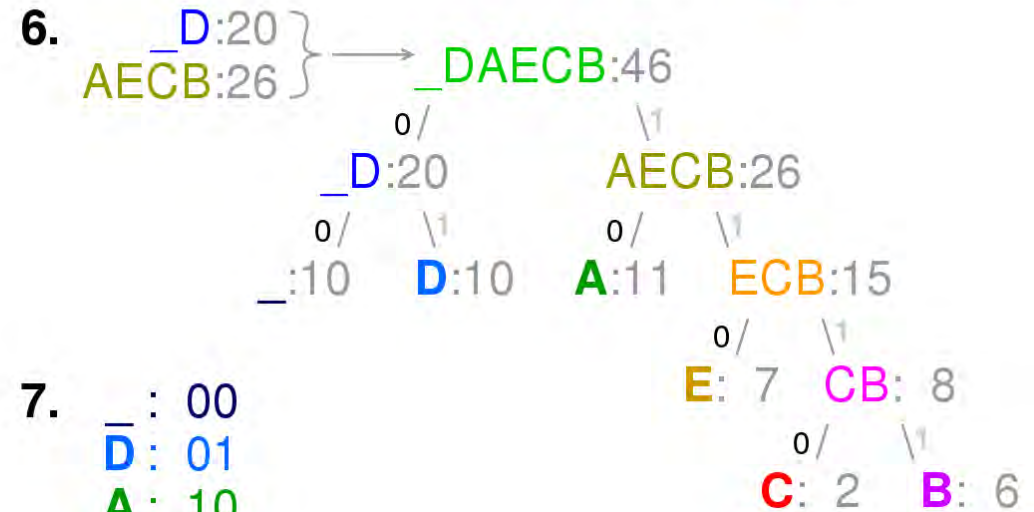
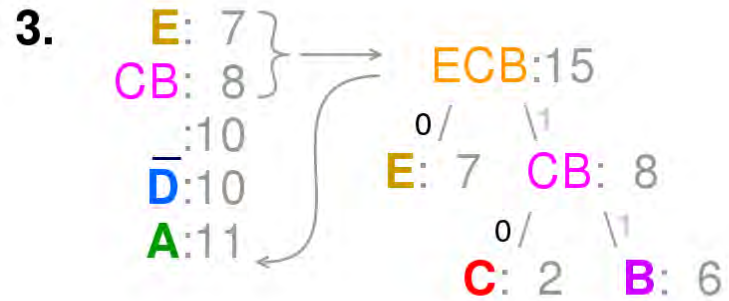
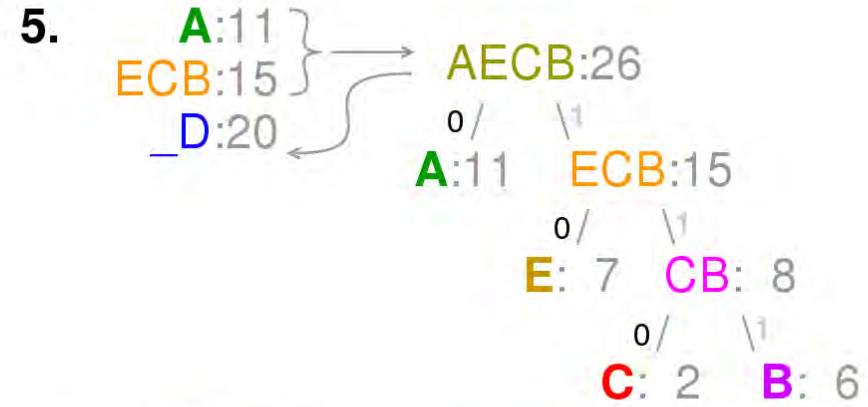
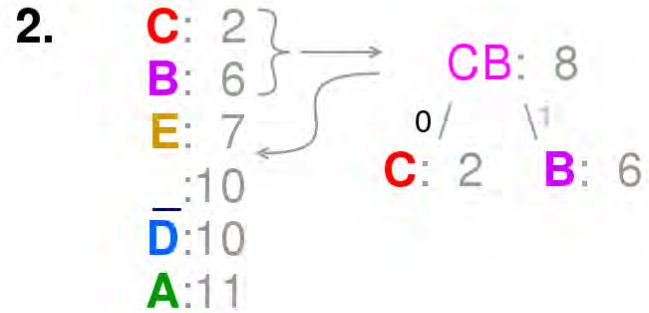
Code

Encoder



- Decoding: reverse process

1. "A\_DEAD\_DAD\_CEDD\_A\_BAD\_BABE\_A\_BEADED\_ABACA\_BED"



8. "100001110100100011001001110110011100100100011111001001111101111110  
0010001111110100111001001011111011101000111111001"

# Application of Huffman Coding for Video

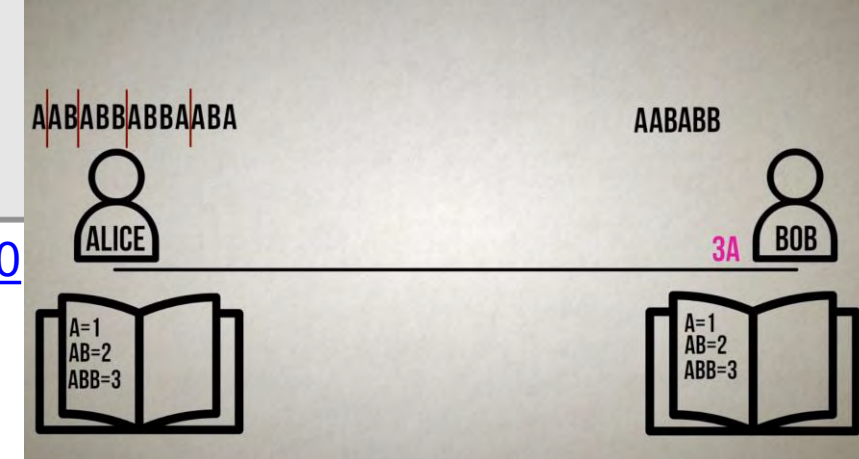
- If the image information can be transformed into a bit stream, Huffman table can be used to compress data without loss
- Simple way to generate a bit stream is to code pixels individually and read them line by line
- For video: Huffman table can be used for a single sequence of images
  - for a set of scenes
  - for an entire film clip
- need Huffman table for encoding and decoding
- Is **used for MP3** encoding in the final step!



# Lempel Ziv (LZ) Dictionary-based Encoding

<https://www.youtube.com/watch?v=RV5aUr8sZD0>

- Dictionary is a table of strings
  - Sender and receiver have a copy of dictionary
  - Previously-encountered strings are substituted by their index in dictionary
- Compression - two concurrent events:
  - Building an indexed dictionary
  - Compressing a string of symbols.
- Algorithm extracts smallest substring not in the dictionary from remaining uncompressed string.
- Stores a copy of this substring in dictionary as a new entry and assigns it an index value.
- Compression occurs when substring (except for the last character) is replaced with the index found in the dictionary.
- Process inserts the index and the last character of the substring into the compressed string.



Used in GIF & ZIP

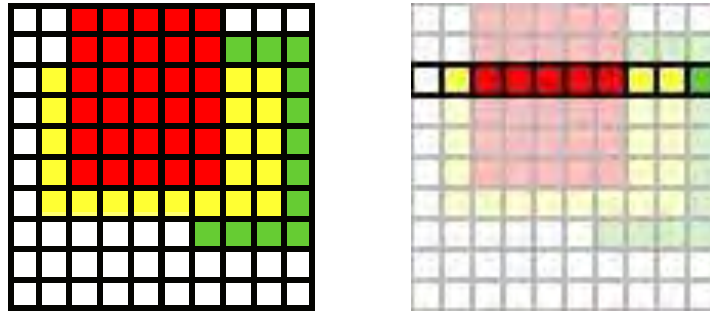
Abraham Lempel



# Lossless Image Formats



- GIF-Format (Graphics Interchange Format)
  - LZW-Compression (Lempel, Ziv, Welch): works line-wise



- 1 st line, 2nd line
- 3rd line is compressed as:
  - 1 white, 1 yellow, 5 red, 2 yellow, 1 green
- Row 4 to 6: „as row 3“
- Indexed (1-8 bit Color Lookup Table)



# CompuServ GIF (gif)



- First standardized in 1987 by CompuServ (called GIF87a)
- Updated in 1989 to include transparency, interlacing, and animation (called GIF89a)
- Use the LZW (Lempel-Ziv Welch) algorithm for compression (not free, licenses necessary)
- A maximum of **256 colors** freely selectable out of 24 bit ( $2^{24} = 16.777.216$ )
- Does not work for photographs
- Suitable for small images such as icons
- Simple animations



# Portable Network Graphics (png)



- Developed because of licensed LZW with gif
- Replacing GIF and TIFF (not JPEG!)
- 3 Types
  - True Color (3 x 16 bit/pixel)
  - Grayscale (1 x 8 bit/pixel)
  - Palette (256 colors)
- Compression similar to PKZIP (Phil Katz)
- The standard zip format provides approximately **62 percent** compression. (RAR results in a **71 percent** compression rate; 7-zip has **76 percent**)



# Differential Encoding

- belongs to **source coding**
- consider sequence of symbols  $S_1, S_2, S_3, \dots$  where values are not zeros, but do not vary much  $\rightarrow$  calculate difference from previous value
- Ex.: still images—calculation of differences between nearby pixels or pixel groups

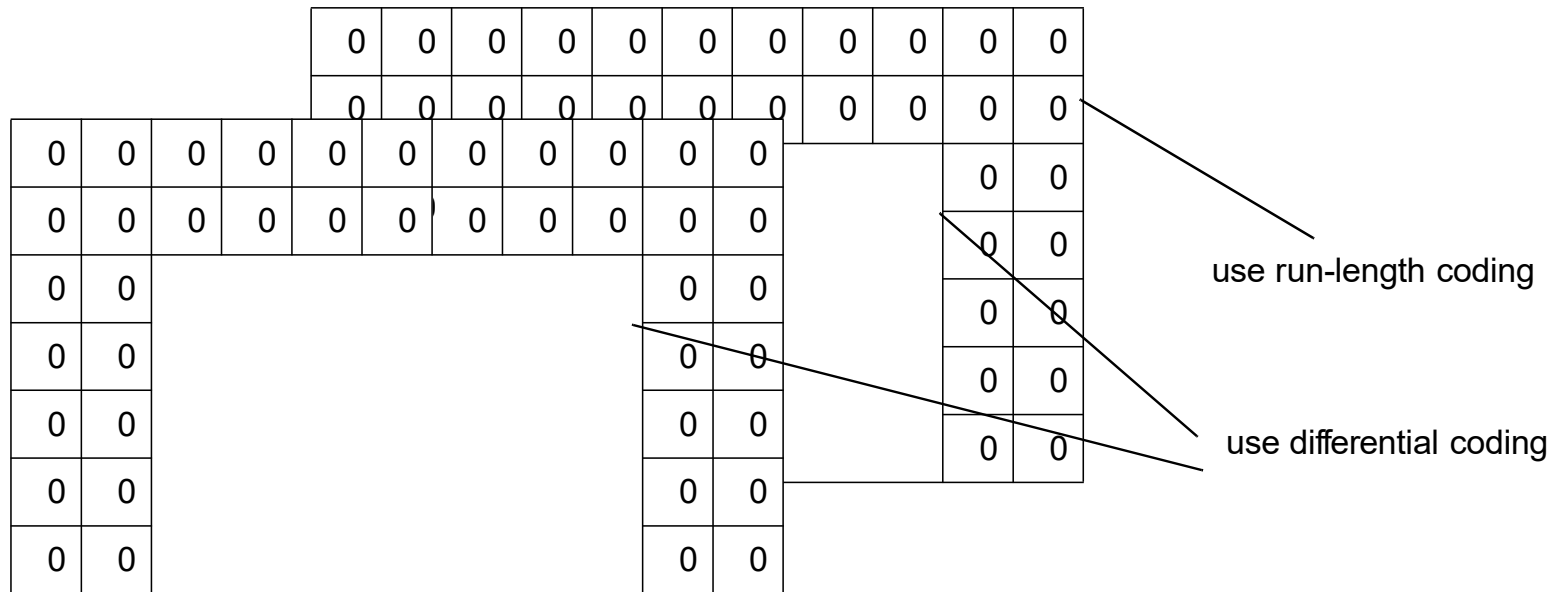
|   |     |    |    |     |
|---|-----|----|----|-----|
| 0 | 0   | 0  | 0  | 0   |
| 0 | 255 | 94 | 87 | 100 |
| 0 | 0   | 0  | 0  | 255 |
| 0 | 0   | 0  | 0  | 0   |

- edges result in large values
- similar chrominance or luminance values result in small values

- zeros can be suppressed by run-length coding

# Differential Coding in Video (Examples)

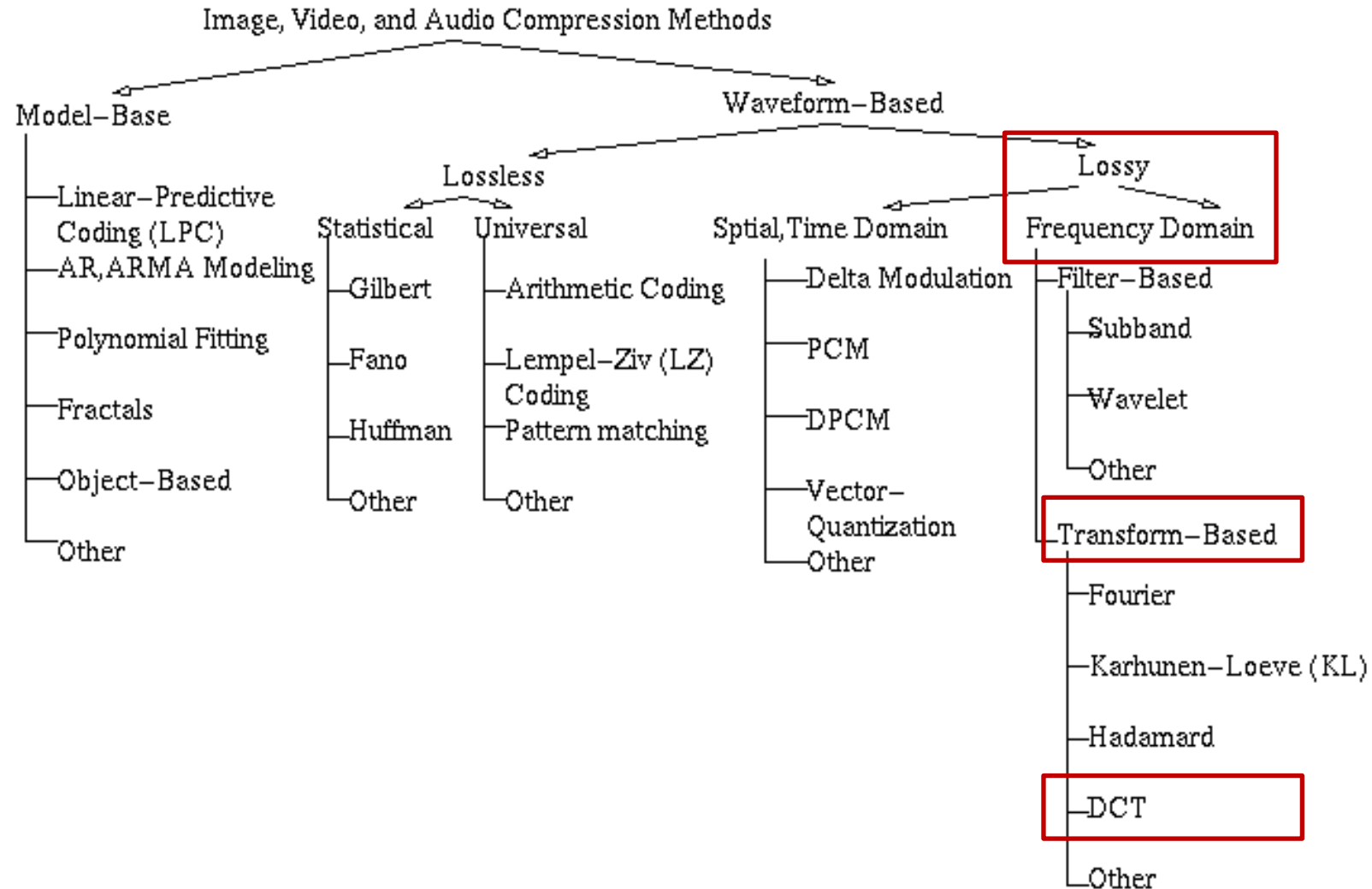
- static background (videoconferencing, newscast, etc.)



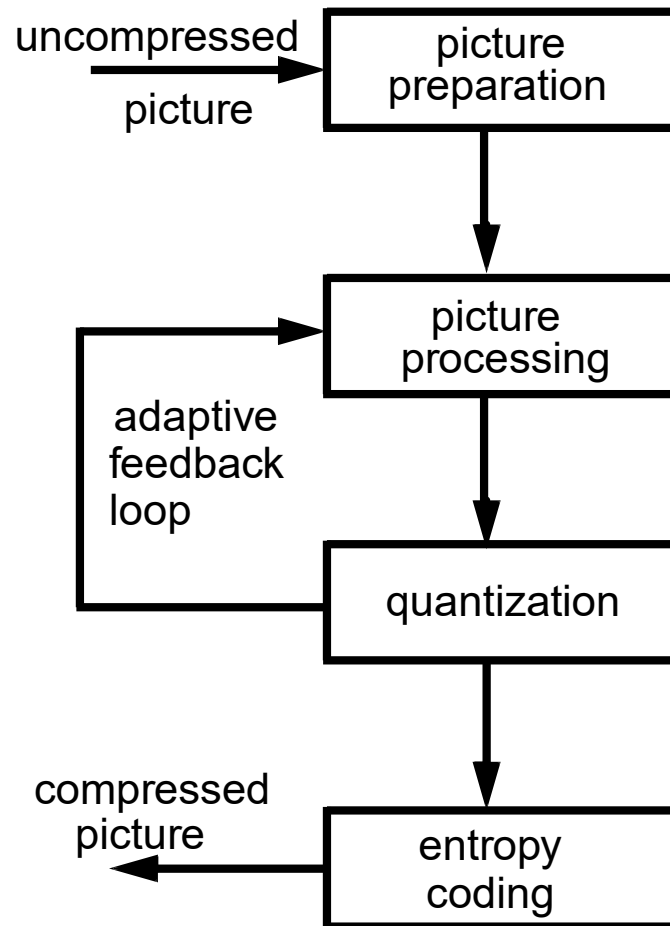
- motion compensation—8x8 blocks are compared; areas similar, only shifted, e.g., to the right (*motion vector*)



# Lossy Image Compression



# Image Compression Steps



Possible operations:

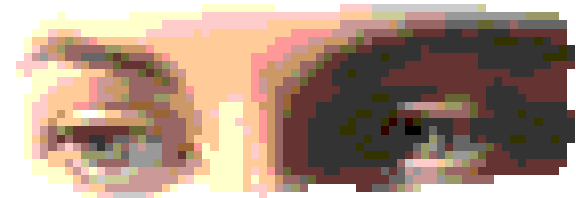
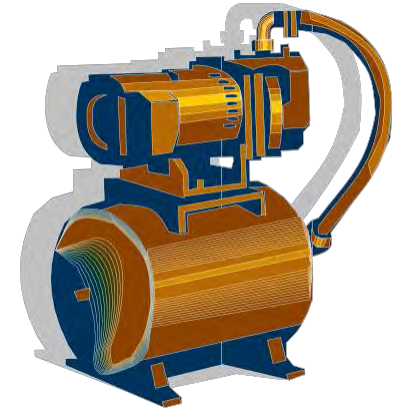
- analog-to-digital conversion
- generation of appropriate digital representation
- image division into 8 x 8 blocks
- fix number of bits/pixel

Compression Algorithm

- transformation from time to frequency domain e.g., using DCT
- motion vector application for motion video
- Typically a lossy step
- mapping of real numbers to integers (reduction in precision)  
example:  $\mu$ -law audio—from 12 bits to 8 bit quantization
- compress a sequential digital stream without loss

# Lossy Compression

- Run-length coding works well for simple images (graphics) and computer data
- However, a large, detailed picture usually is not reduced enough
- In order to reduce information further lossy compression is needed
  - Information is permanently removed
  - The trick is to remove details that are not perceived by a human observer
  - Many of these psycho-visual coding systems take advantage of a number of aspects of the human visual system



# Human Visual System



- The eye is more sensitive to brightness changes than to color changes
- The eye is not able to perceive brightness above or below certain threshold values
- The eye does not perceive little brightness or color changes. The strength of this phenomenon is dependent on the color



# Compression of Still Images



- Characteristics of human vision have been transferred to **lossy image compression techniques**
  - remove some image details to achieve a higher compression rate by suppressing higher frequencies
- Such a development is the JPEG format
  - Stands for "Joint Photographic Experts Group"
  - JPEG is the worldwide standard
  - JPEG compresses brightness and color information separately
  - Color information is more compressed (lower sensitivity)
- Color space for JPEG compression
  - RGB values are a combination of brightness and color
  - If the RGB values are separated into a luminance and a color component, the color component is more compressed



# Joint Photographic Experts Group (JPEG)

JPEG is a joint standard of ISO and ITU-T

- In June 1987, an adaptive transformation coding technique based on DCT was adopted for JPEG
- In 1992, JPEG became a ISO international standard
- Lossy compression combined with lossless techniques
- Trade-Off between file size and quality
- JPEG typically achieves 10:1 compression with little perceptible loss in image quality

Requirements:

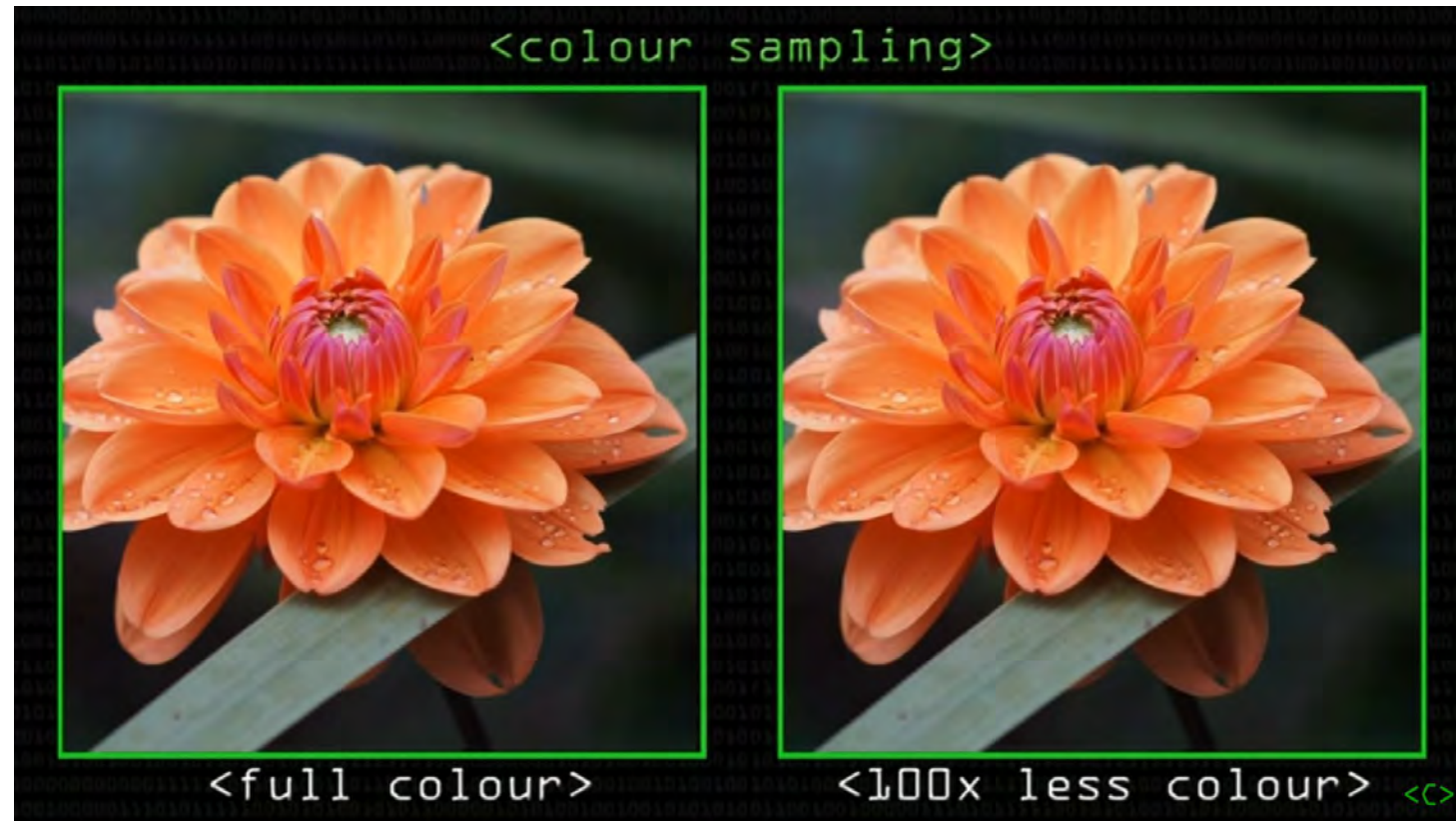
- to be state-of-the-art for degree of compression versus image quality,
- to be parameterizable (user can select parameters),
- to be applicable to any kind of source image, without regard to dimensions, content, image aspect ratio, pixel aspect ratio, etc.,
- to have computational requirements that are reasonable for hardware and software implementations,
- to run on as many standard platforms as possible



# Joint Photographic Experts Group (JPEG)

- to support four different modes of operation:
  - **sequential encoding—encoded in same order as scanned**
  - progressive encoding—multiple pass encoding
  - lossless encoding
  - hierarchical encoding—encoded at multiple resolutions

# How JPEG Works – Part 1



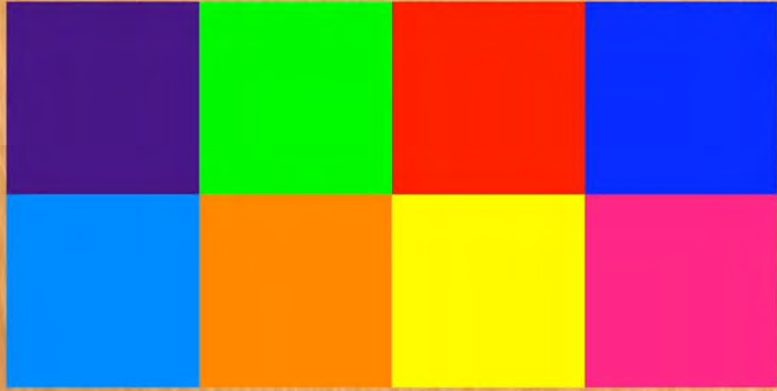
# 1<sup>st</sup> Step: Color Conversion YCbCr <-> RGB

- Necessary in preparation for JPEG compression
  - Color space conversion and Downsampling:
    - Separation of the color components from the luminance information
    - Insensitivity of human eye to rapid color changes allows coarser sampling of the color components
- > No loss of subjective quality
- > Significant data reduction

# Chroma Subsampling

**4:4:4**

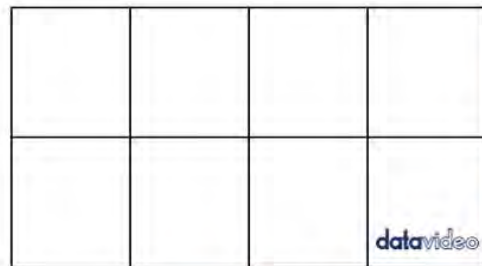
**Chroma**



Some visual examples  
<https://www.red.com/red-101/video-chroma-subsampling>

<https://www.youtube.com/watch?v=32PPzwPjDZ8>

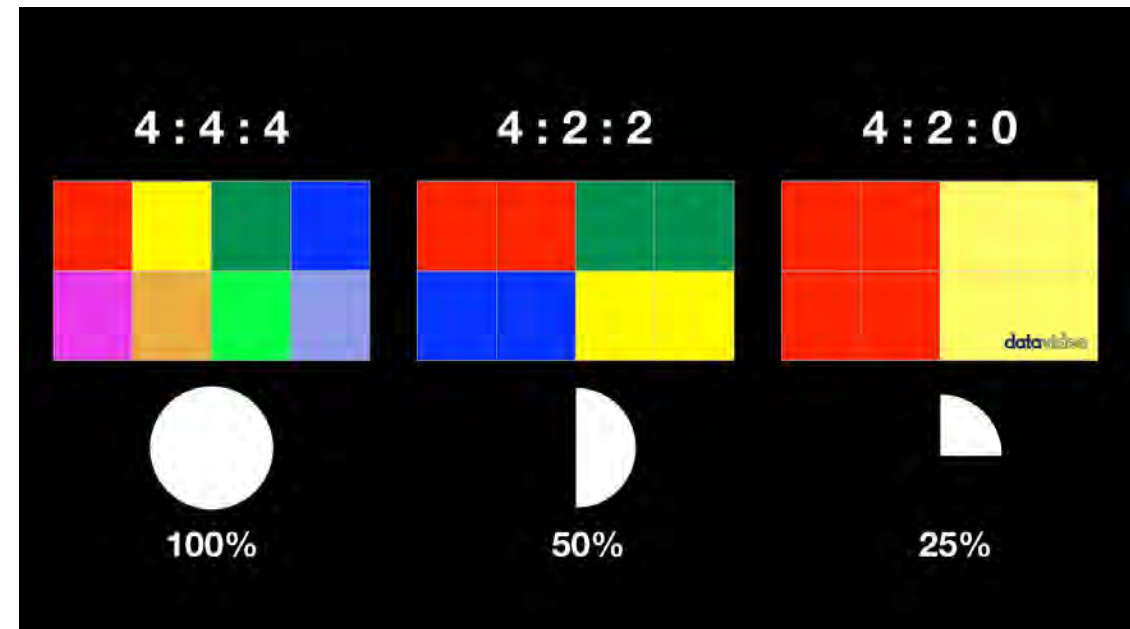
| Y                                            | Cr                                | Cb                                 |
|----------------------------------------------|-----------------------------------|------------------------------------|
| The Numbers of Across<br>Pixels <b>4</b> x 2 | First Row<br>The Numbers of Color | Second Row<br>The Numbers of Color |



Cr

Cb

**Y** **4** x 2



# 1<sup>st</sup> Step: Color Conversion YCbCr <-> RGB

- YCbCr (256 levels) can be computed directly from 8-bit RGB as follows:

$$\begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$

- During decoding YCbCr color space needs to be converted back to RGB:

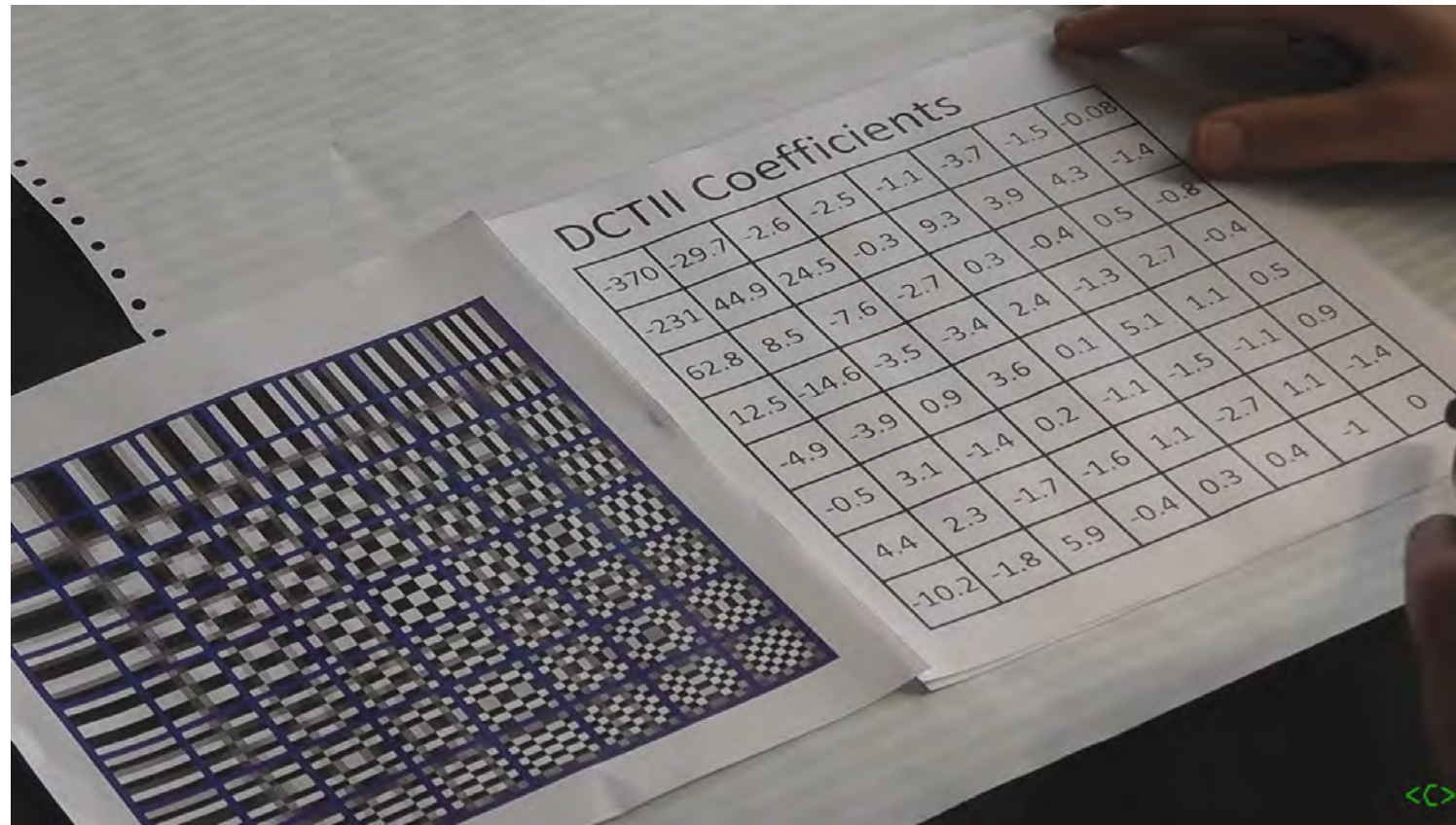
$$R = Y + 1.402 (Cr-128)$$

$$G = Y - 0.34414 (Cb-128) - 0.71414 (Cr-128)$$

$$B = Y + 1.772 (Cb-128)$$



# How JPEG Works – Part 2

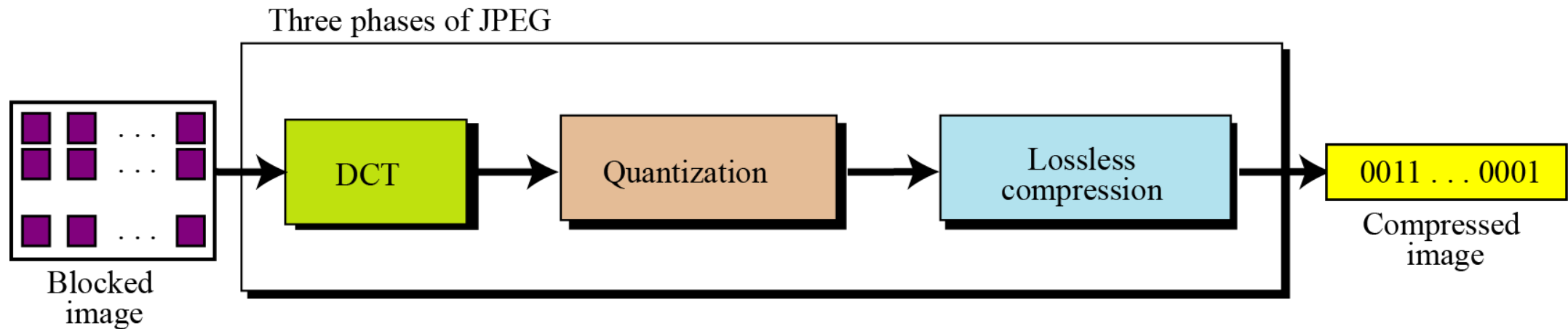




# JPEG Compression - Overview

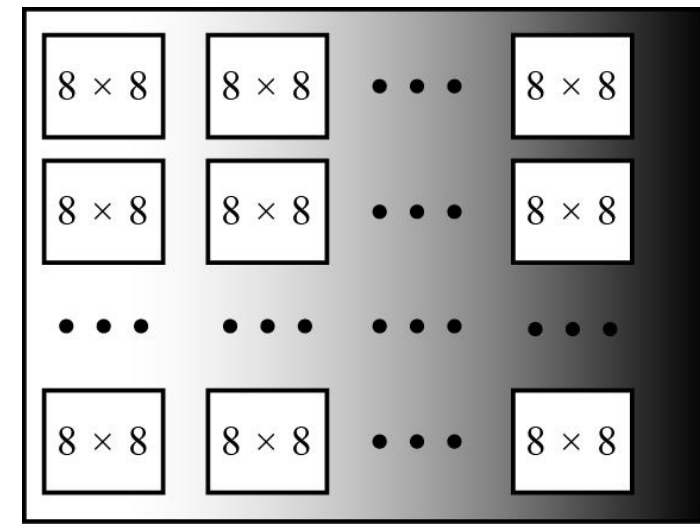


- Idea: change image into a linear (vector) set of numbers that reveals redundancies.
- Redundancies can be removed using one of the lossless compression methods



# JPEG Image Preparation

- Images – each channel – divided in data units (blocks), DCT operates on blocks to decrease the number of calculations
- Lossy mode operates on 8x8 pixel blocks (4:4:4 – no subsampling), 16×8 (4:2:2), or most commonly 16×16 (4:2:0).
- Lossless mode operates on data units of 1 pixel
- In most cases data units are processed component by component and passed to image processing
- processing order of data units
  - left-to-right, top-to-bottom
  - interleaved data ordering
- Interleaved data units of different color channels are combined to
  - **Minimum Coded Units (MCU)**



# Discrete Cosine Transform

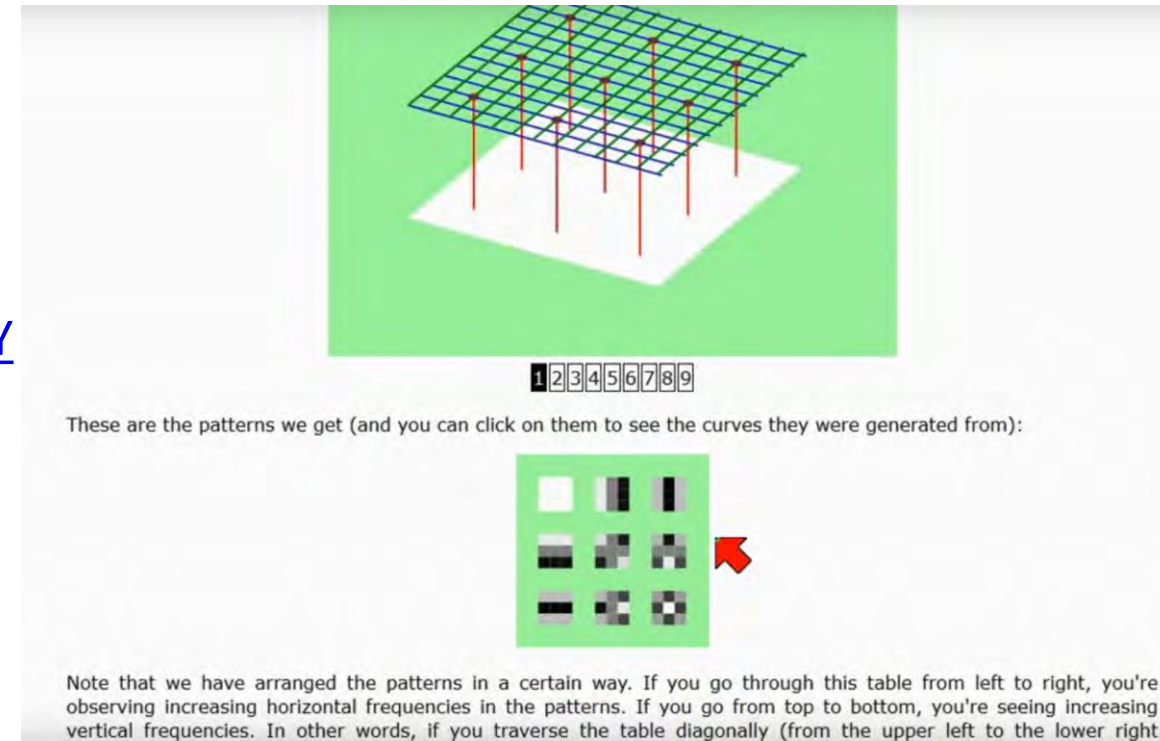
- First step: pixel values shifted into the range  $[-128, 127]$
- Forward Discrete Cosine Transformation (Forward DCT) maps values from time to frequency domain

- Interactive Website

<http://weitz.de/dct/>

Explanation:

<https://www.youtube.com/watch?v=7fhHQgu2OcY>





- DCT-based codecs use a two-dimensional version of the transform.
- The 2-D DCT of an 8 x 8 block:

$$F(u, v) = \sum_{x=0}^7 \sum_{y=0}^7 \alpha(u) \alpha(v) f(x, y) \cos\left[\frac{\pi}{8} \left(x + \frac{1}{2}\right) u\right] \cos\left[\frac{\pi}{8} \left(y + \frac{1}{2}\right) v\right]$$

$u$  is the horizontal spatial frequency, for the integers  $0 \leq u < 8$

$v$  is the vertical spatial frequency, for the integers  $0 \leq v < 8$

$$\alpha(u) = \begin{cases} \sqrt{\frac{1}{8}}, & \text{if } u = 0 \\ \sqrt{\frac{2}{8}}, & \text{otherwise} \end{cases}$$

is a normalizing scale factor to make the transformation orthonormal

$f(x, y)$  is the pixel value at coordinates  $(x, y)$  “space domain”

$F(u, v)$  is the DCT coefficient at coordinates  $(u, v)$  “frequency domain” (indicates how fast the information moves inside the rectangle)

◆ Note: The DCT decomposes a signal into a series of harmonic cosine functions.

# DCT Basis Functions

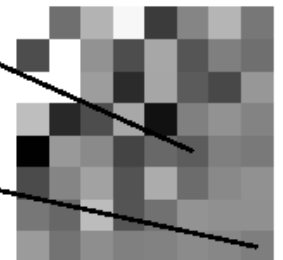
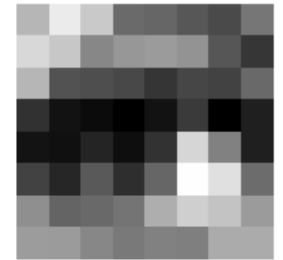
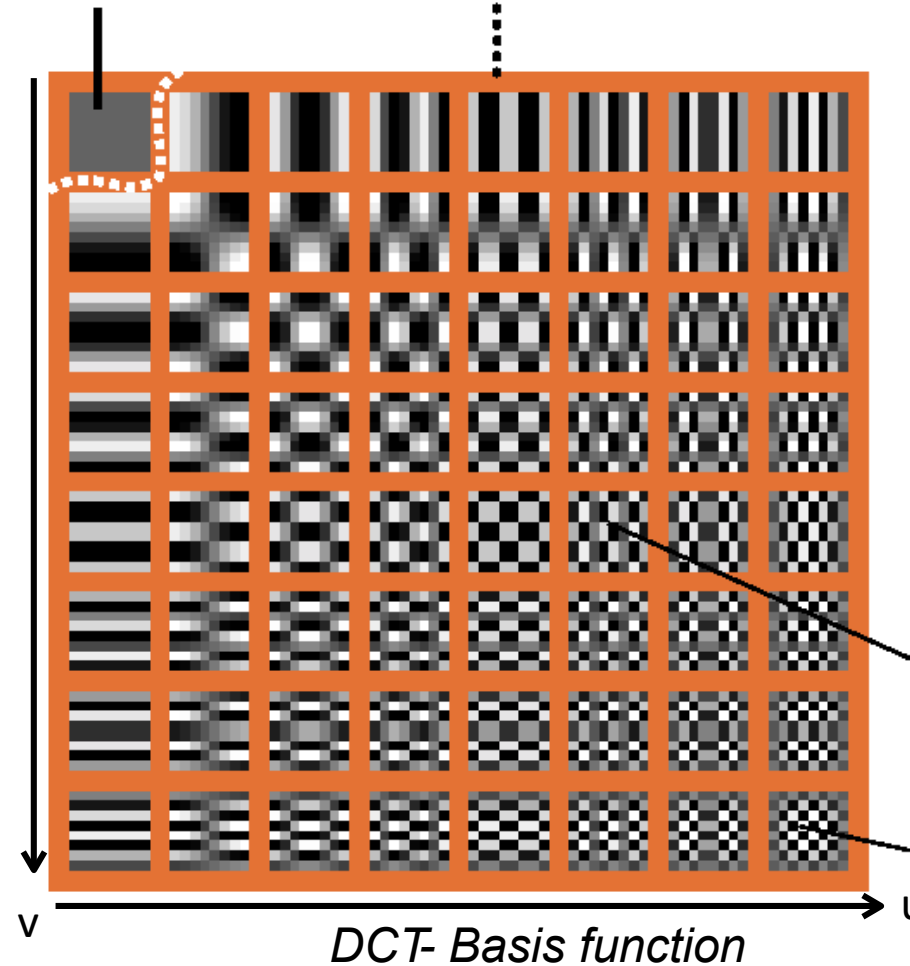


- DCT values indicate the weighting of frequency images
- The brighter the pixel, the greater the value of DCT

*Constant component*

*Alternating component*

*8 x 8 Imageblock*



# JPEG Image Processing (DCT)

- $S(u,v)$  coefficients
- $S(0,0)$ —lowest frequency in both directions is called DC coefficient
  - determines the fundamental luminance/color of the block
  - frequency = 0 in both directions
- $S(0,1), \dots, S(7,7)$  are called AC coefficients
  - frequency in one or both directions non-zero
- computing of DCT—use factoring



# Discrete Cosine Transform

Humans have only a restricted capability to perceive high frequencies.  
Images reduced from their high frequent elements may have a high degree of quality while being compressed quite considerably

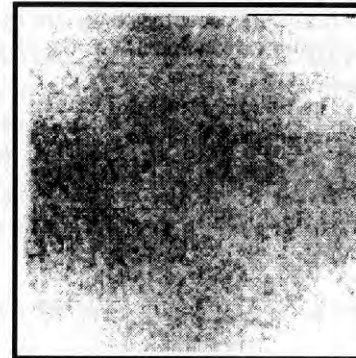
- Low frequency components of an image correspond to brief outlines of image objects
- High frequency components of an image correspond to fine structures

To decrease the high frequencies the data must be sorted by frequencies  
->DCT

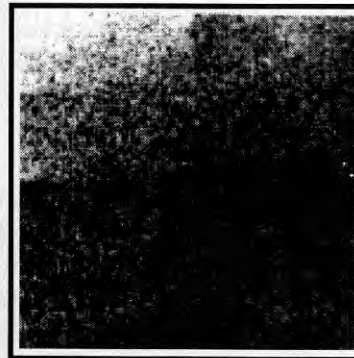
# Why DCT?



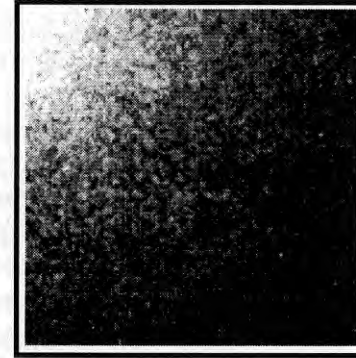
(a) Test Image



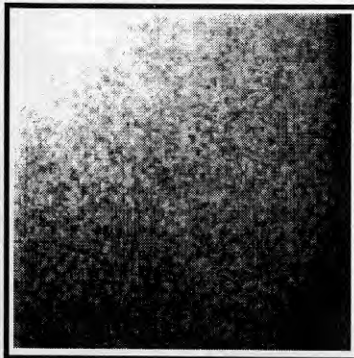
(b) Discrete Fourier transform



(c) Discrete Cosine transform



(d) Discrete Sine transform



(e) Discrete Hadamard transform



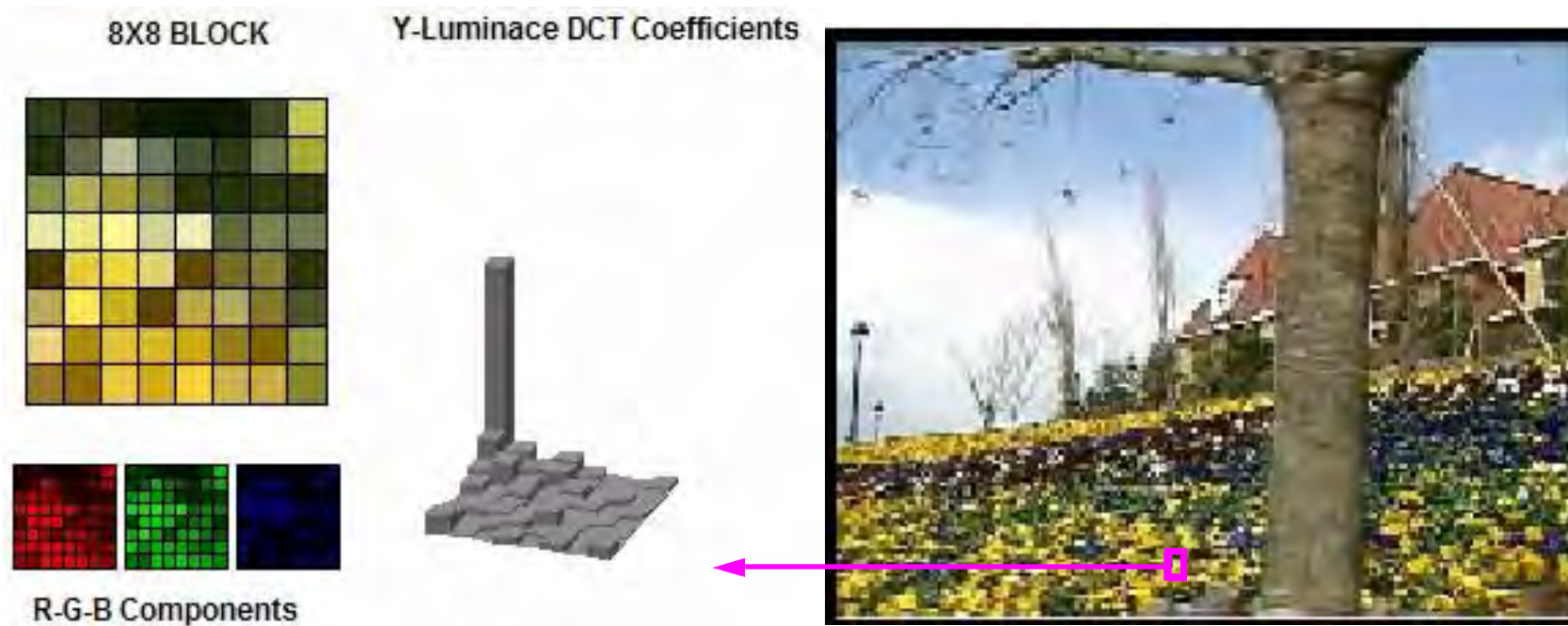
(f) Karhunen-Loeve transform

Karhunen-Loeve not used in practice for image compression because it needs to transmit the transform kernel for every block, which requires huge amount of side information.

# DCT-Coefficients



# DCT-Coefficients

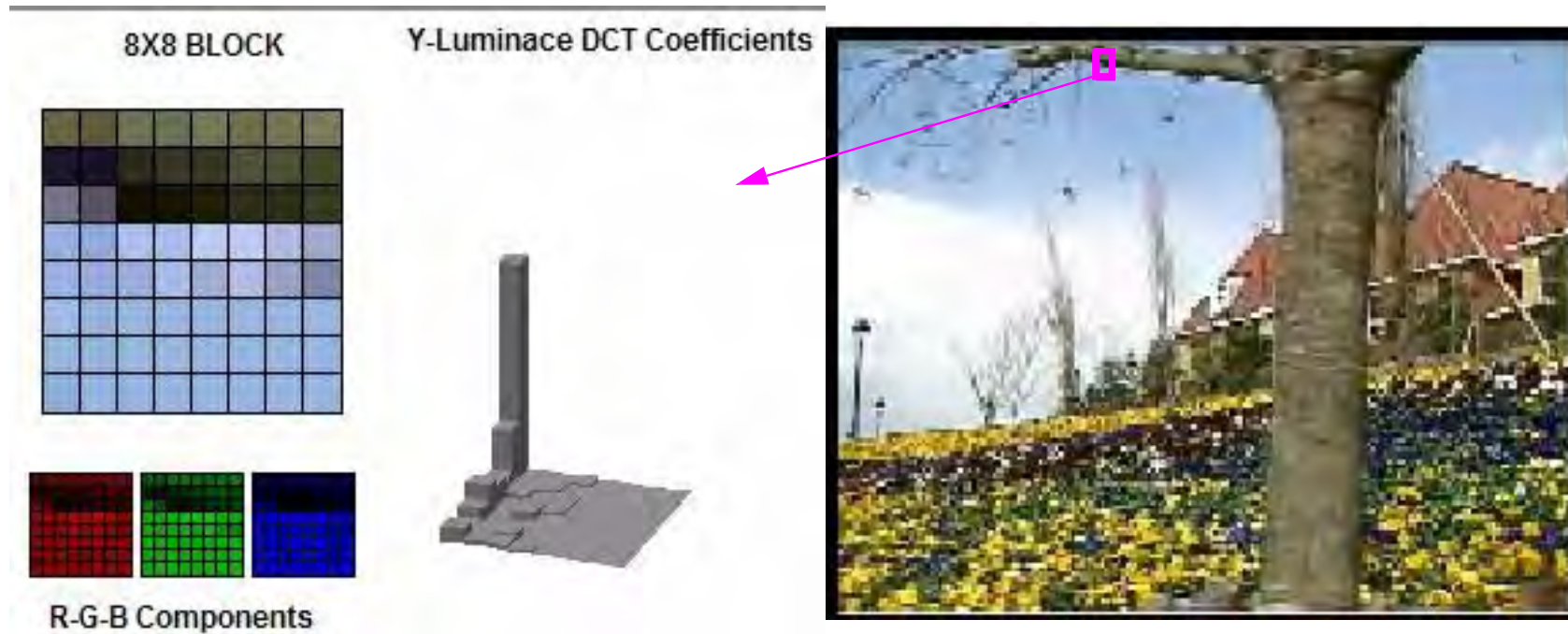




# DCT-Coefficients



# DCT-Coefficients

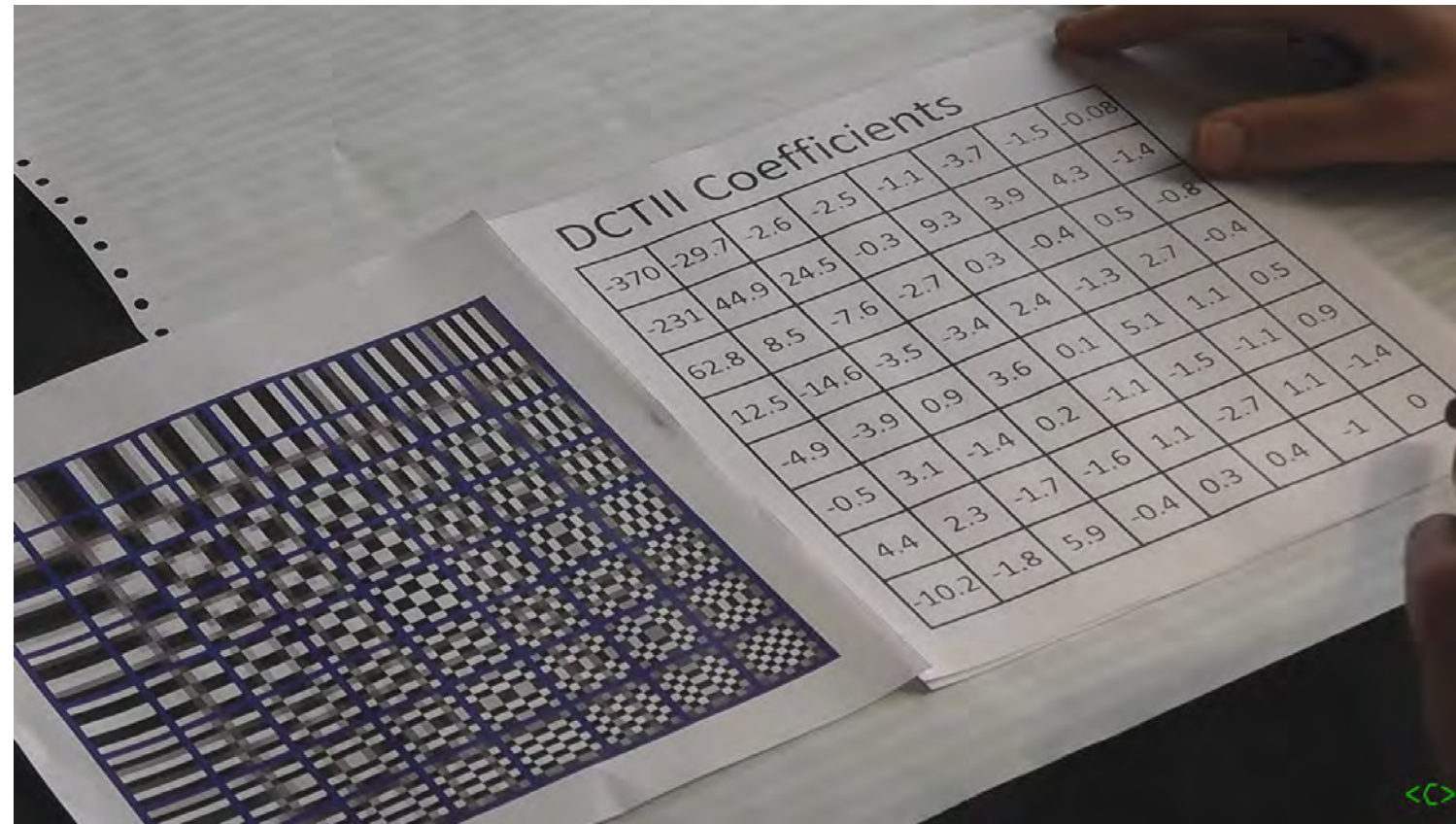




# DCT-Coefficients



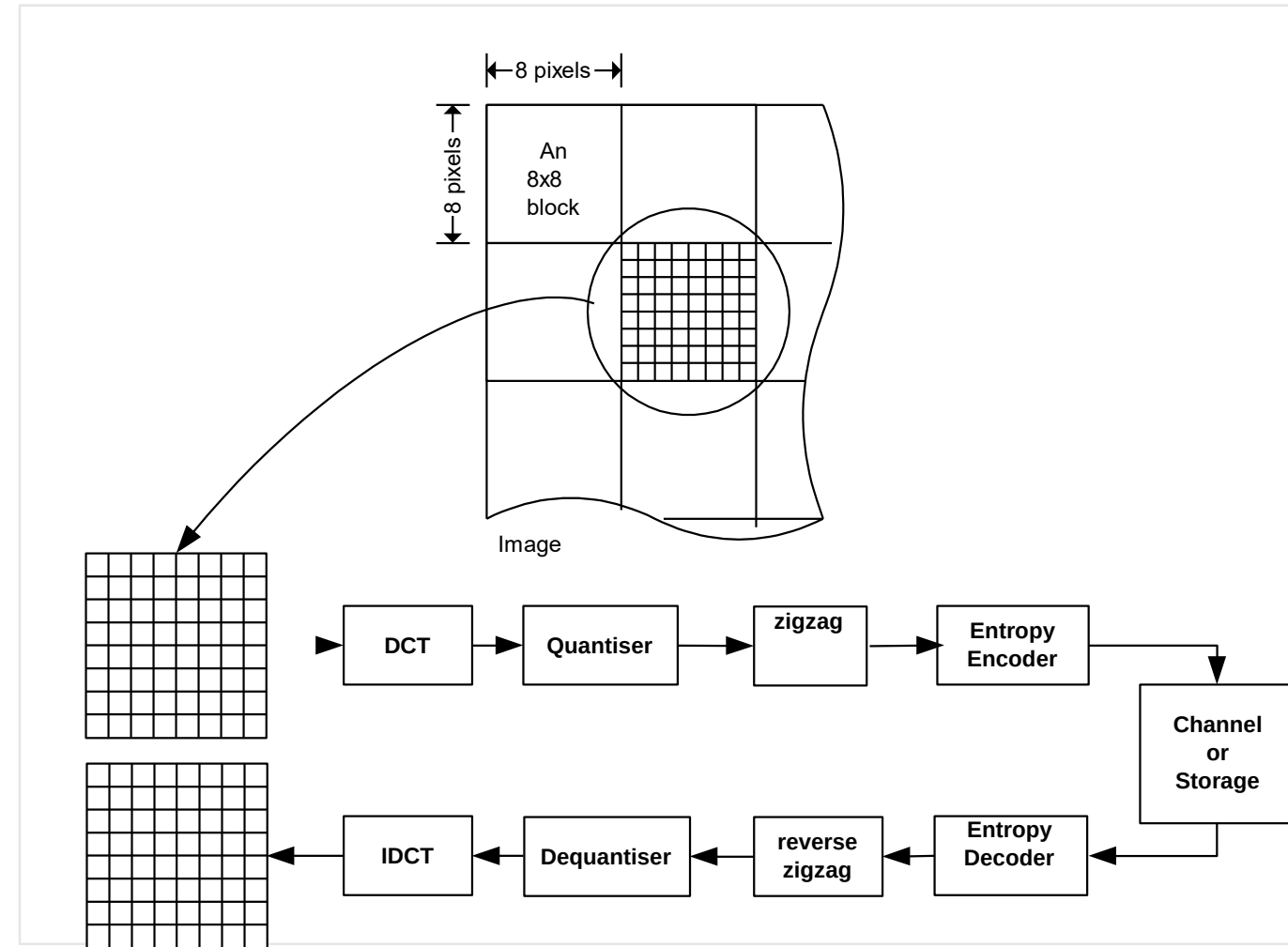
# How JPEG Works – Quantization - Part 3



# JPEG Compression



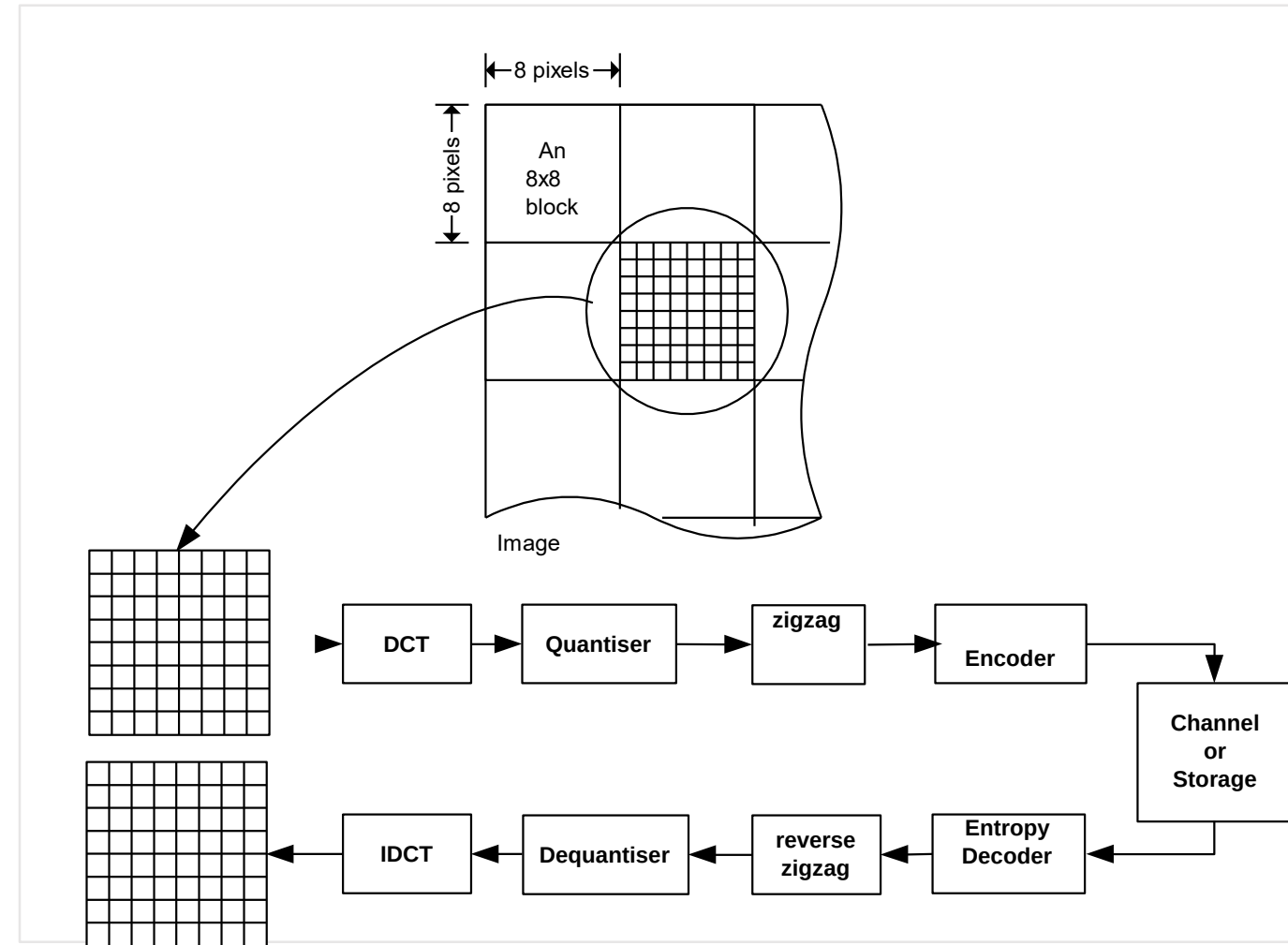
- To perform the JPEG coding, an image (in color or grey scales) is first subdivided into blocks of 8x8 pixels.
- The Discrete Cosine Transform (DCT) is then performed on each
- This generates 64 coefficients which are then quantized to reduce their magnitude.



# JPEG Compression



- The coefficients are then reordered into a one-dimensional array in a zigzag manner before further entropy encoding.
- The **compression is achieved in two stages**; the first is **during quantization** and the second **during the entropy coding** process.
- JPEG decoding is the reverse process of coding.



# JPEG Quantization

- Goal: to “throw out” bits (truncation)
- Quantization in JPEG = Process of removing high frequency data
- Uniform quantization: divide coefficient values  $S(u,v)$  by  $N$  and round result

Q: In  $S(u,v)$  how many bits should be truncated?

A: use quantization tables

- quantization tables consist of 64 elements, each value uses 8-bit:  $Q_{uv}$
- new compressed values by using tables  $Sq_{uv} = S_{uv} / Q_{uv}$
- standard defines two default tables (luminance, chroma)

# JPEG Quantization

## Luminance quantization table

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 16  | 11  | 12  | 14  | 12  | 10  | 16  | 14  |
| 13  | 14  | 18  | 17  | 16  | 19  | 24  | 40  |
| 26  | 24  | 22  | 22  | 24  | 49  | 35  | 37  |
| 29  | 40  | 58  | 51  | 61  | 60  | 57  | 51  |
| 56  | 55  | 64  | 72  | 92  | 78  | 64  | 68  |
| 87  | 69  | 55  | 56  | 80  | 109 | 81  | 87  |
| 95  | 98  | 103 | 104 | 103 | 62  | 77  | 113 |
| 121 | 112 | 100 | 120 | 92  | 101 | 103 | 99  |

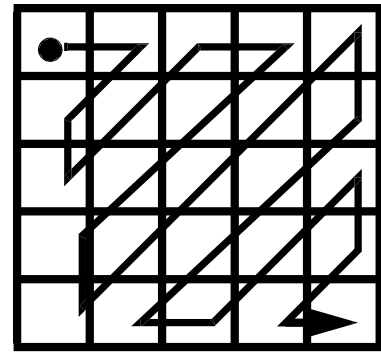
Human eye most sensitive to low frequencies (upper left corner)



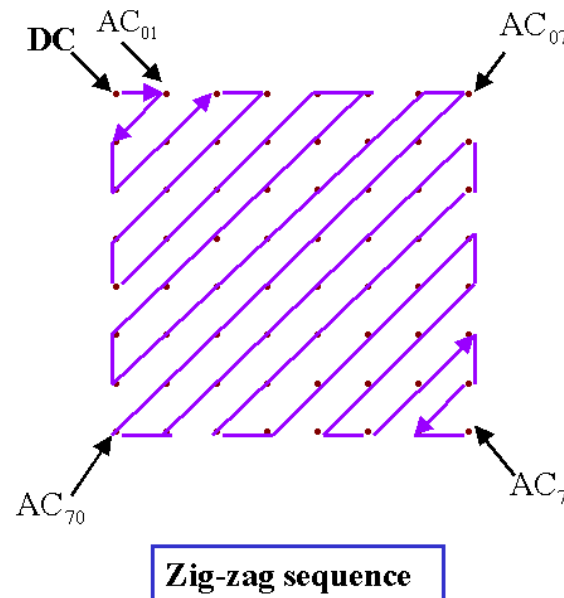
# Quantization process

- Divide DCT-coefficient value by an integer number from the quantization table
- Round the result to the nearest integer
- Quantization of all DCT-coefficients results in a *lossy* transformation – some image details given by higher frequencies are cut off.
- JPEG application provides a table with 64 entries, each used for quantization of one DCT-coefficient
  - each coefficient can be adjusted separately
- A high compression factor is achievable on the expense of image quality
  - large quantization numbers: high data reduction but information loss increases
- No default values for quantization tables are specified in JPEG

# JPEG Entropy Encoding



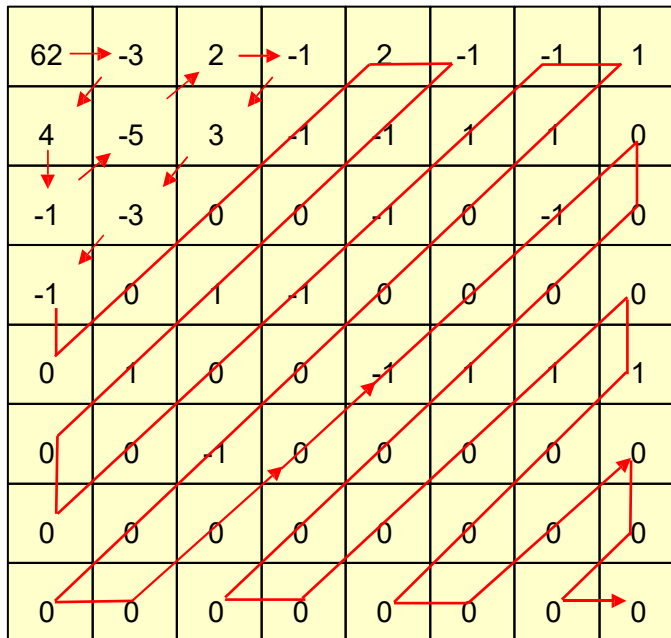
First step: map 8x8 pixel blocks into 64 element vector using zig-zag-scan



- DC coefficients processing:
  - DC coefficients determine basic color of data unit
  - DC coefficient is large, but often close to previous value  
==> encode difference

# Example

## Zig-zag ordering



*DC-coefficient*: code coefficients for one block as difference to the previous one

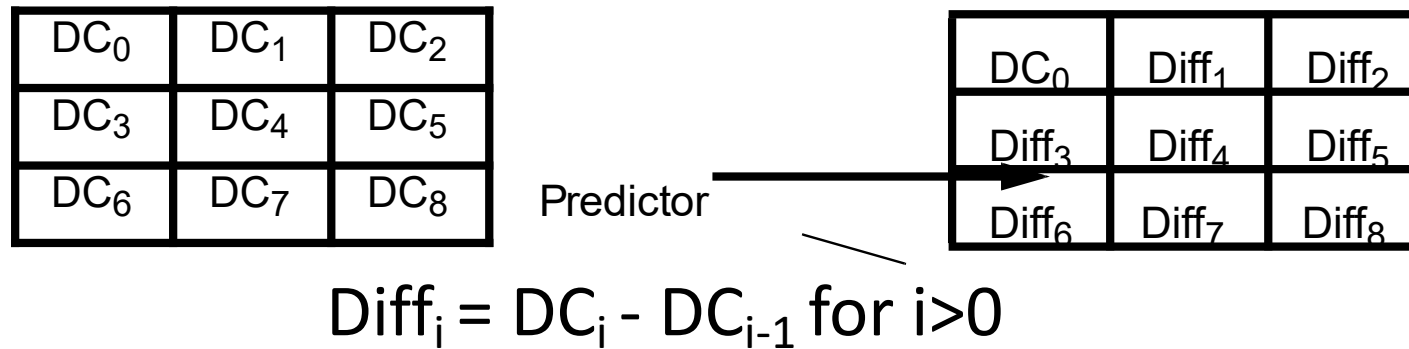
*AC-coefficients*: consider each block separately, order data using zig-zag sequence to achieve long sequences of zero- values:

[illegible]

*Entropy encoding:*

- Run-length encoding of zero values of quantized AC-coefficients
- Huffman encoding on DC- and AC-coefficients

# JPEG Entropy Encoding



- AC coefficients processing:
  - processing order of the AC coefficients using zig-zag scan (coefficients with lower frequencies are encoded first) ==>
  - sequence of similar data bytes ==> efficient entropy coding
- JPEG standard specifies Huffman or Arithmetic coding, but sequential encoding mode uses only Huffman coding.

# JPEG — Entropy Encoding

Algorithm:

1. apply run-length coding of AC coefficient of zero values
2. apply Huffman coding on DC and AC coefficients

(only DC coefficients explained in detail)

# JPEG — Entropy Encoding

- DC coefficients—Huffman coding
  - categorize DC values into DC code tables
  - difference magnitude categories for DC coefficients (12 categories)

| Diff Values      | SSSS (number of bits<br>needed to encode Diff) |
|------------------|------------------------------------------------|
| 0                | 0 bit                                          |
| -1, 1            | 1 bit                                          |
| -3, ..., 3       | 2 bits                                         |
| -7, ..., 7       | 3 bits                                         |
| .                |                                                |
| .                |                                                |
| .                |                                                |
| -2047, ..., 2047 | 11 bits                                        |



# JPEG — Entropy Encoding

- DC coefficients — Huffman encoding
  - handle SSSS as Huffman symbol, get  $p(0)$ ,  $p(1)$ , ...  $p(11)$  and create Huffman tree ==> Huffman code for SSSS
  - for each category, an additional bits field is appended to the code word to uniquely identify which difference in that category actually occurred.
  - send: (Huffman codeword, actual value)

Example: If SSSS=2 has the Huffman code 001 and Diff=-3 ==>

send 00100

Huffman code for 2

-3 in two's complement is 00  
because 3=>11

Details about JPEG Entropy Coding in

<https://cnx.org/contents/IJvrSfU5@3.3:eMc9DKWD@5/JPEG-Entropy-Coding>

# JPEG Image Processing (DCT) - Decoding

- Inverse Discrete Cosine Transformation (I-DCT) maps DCT coefficients to sampled values

$$s_{xy} = \frac{1}{4} \sum_{u=0}^7 \sum_{v=0}^7 c_u c_v s_{vu} \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}$$

with  $c_u, c_v = 1/\sqrt{2}$  for  $u,v=0$ ; otherwise  $c_u, c_v = 1$

- DCT and I-DCT cannot be calculated in full precision  
==> lossy compression
- JPEG does not define precision parameters, therefore various implementations exist
- many AC coefficients with small values (around zero)

# Comments

- Applications do not have to include both an encoder and decoder if the compression process agrees on a common table
- The encoded data stream has a fixed *Interchange Format*
  - encoded image data
  - chosen parameters
  - tables of the coding process
- In regular mode, the interchange format includes all of the information necessary for decoding without any previous knowledge of the coding process

# DCT Examples



- DCT sorts values from the lowest to the highest frequency
- In image blocks it is likely that the energy is concentrated in low frequencies
  - Regions with smooth colors or little details (= low spatial frequencies) have high values
  - Regions with different colors and detail (= high spatial frequencies), most values are almost zero.



8x8 Block (Luminance)

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 134 | 142 | 145 | 131 | 114 | 122 | 131 | 130 |
| 129 | 143 | 134 | 130 | 135 | 144 | 134 | 118 |
| 123 | 117 | 118 | 111 | 97  | 109 | 130 | 143 |
| 129 | 116 | 112 | 116 | 120 | 126 | 130 | 118 |
| 118 | 127 | 141 | 138 | 138 | 148 | 141 | 125 |
| 125 | 129 | 119 | 127 | 143 | 149 | 145 | 136 |
| 131 | 126 | 128 | 142 | 141 | 135 | 126 | 116 |
| 131 | 140 | 146 | 154 | 133 | 118 | 124 | 124 |

Pixelvalues

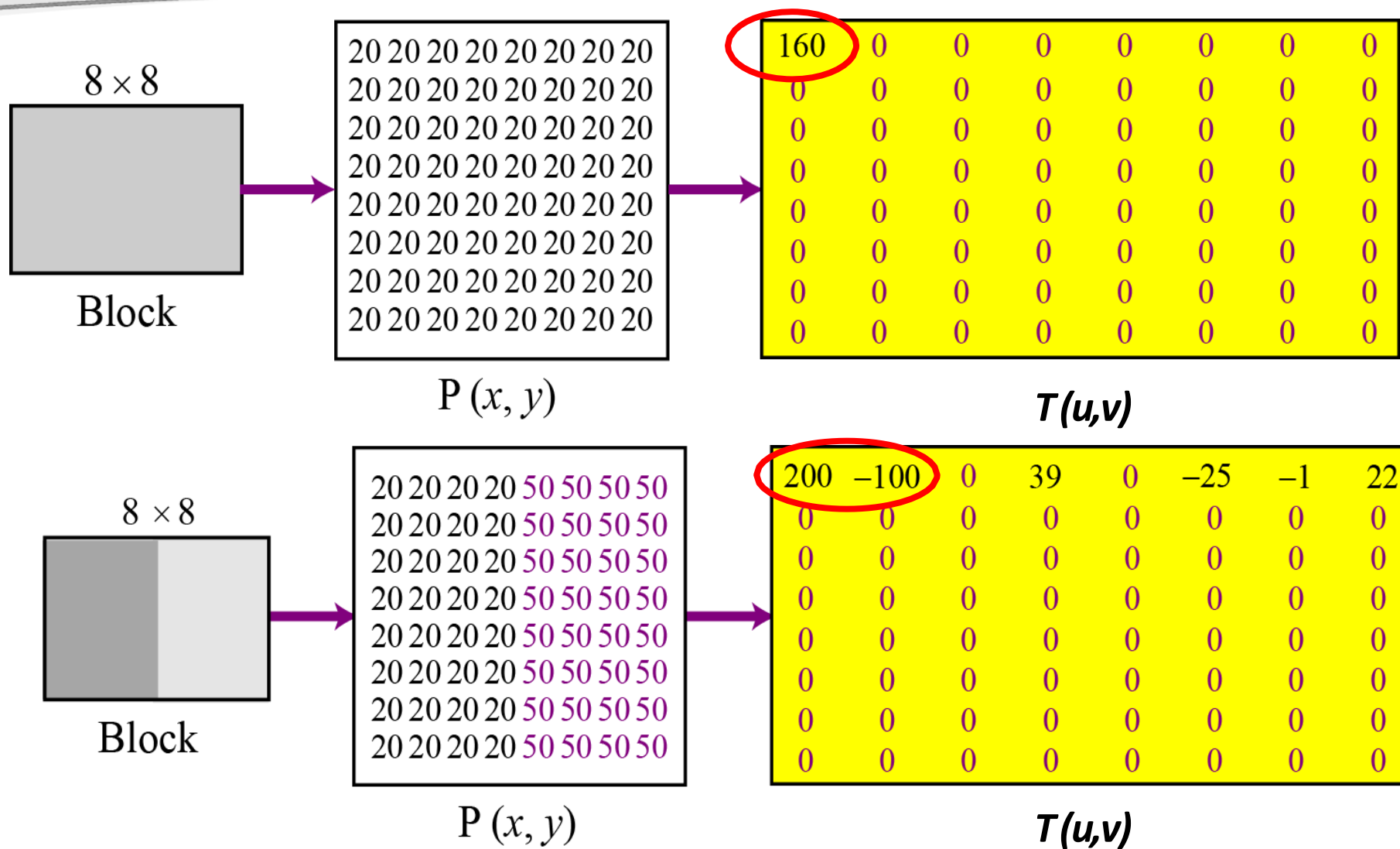
Low frequencies

|      |     |     |     |     |    |    |    |
|------|-----|-----|-----|-----|----|----|----|
| 1037 | -1  | -6  | 1   | -12 | 8  | -4 | -4 |
| -16  | 1   | 28  | -6  | -14 | 0  | 4  | 0  |
| 19   | 32  | -7  | -19 | 2   | -1 | -4 | -3 |
| 29   | -9  | -14 | 13  | -10 | -6 | 1  | 0  |
| 4    | 14  | -6  | -13 | -2  | 7  | 1  | 2  |
| -26  | 2   | 16  | 2   | 11  | 6  | 1  | 1  |
| -10  | -11 | 27  | -18 | 4   | 1  | 0  | 0  |
| -2   | 1   | 1   | -19 | -1  | 6  | 6  | 0  |

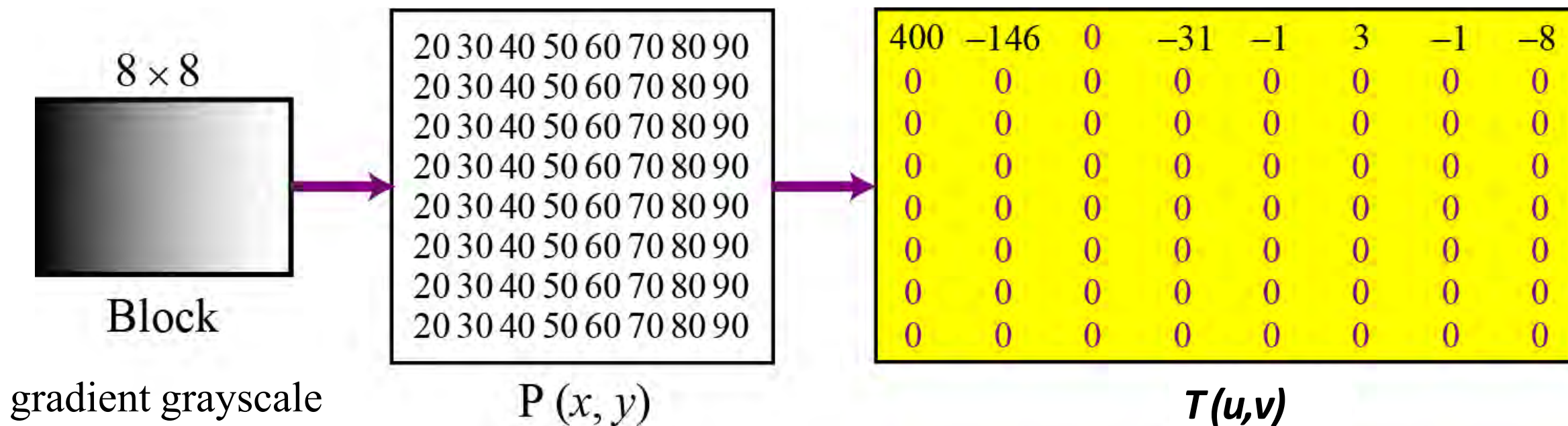
High frequencies

DCT-Values

# DCT Examples



# DCT Examples





# DCT Basis Functions

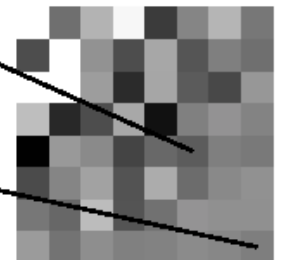
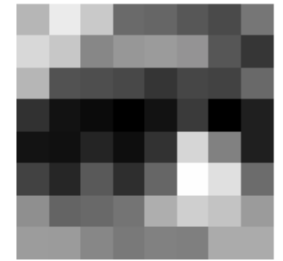
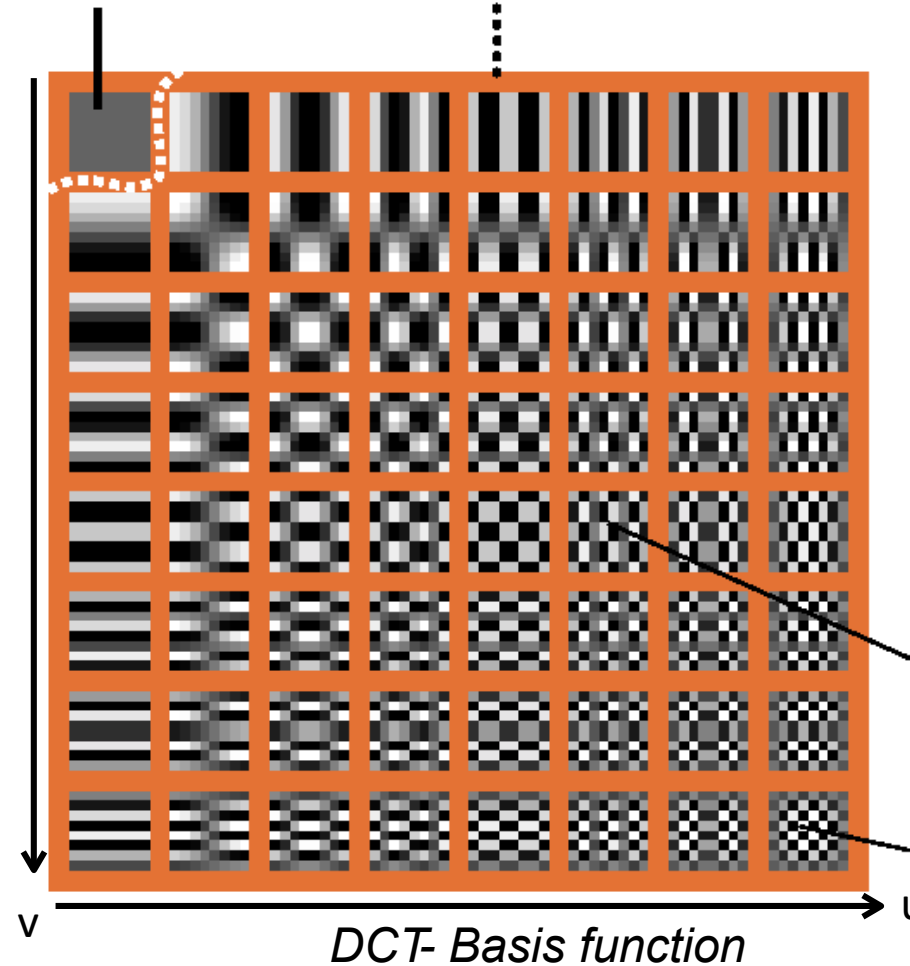


- DCT values indicate the weighting of frequency images
- The brighter the pixel, the greater the value of DCT

*Constant component*

*Alternating component*

*8 x 8 Imageblock*



# Quantization



- Compression effect: DCT values are approximated (quantized) to make them smaller and to repeat themselves
  - Each DCT value is divided by a quantization factor and then rounded
  - The larger the quantization factor, the smaller the values to be stored

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 134 | 142 | 145 | 131 | 114 | 122 | 131 | 130 |
| 129 | 143 | 134 | 130 | 135 | 144 | 134 | 118 |
| 123 | 117 | 118 | 111 | 97  | 109 | 130 | 143 |
| 129 | 116 | 112 | 116 | 120 | 126 | 130 | 118 |
| 118 | 127 | 141 | 138 | 138 | 148 | 141 | 125 |
| 125 | 129 | 119 | 127 | 143 | 149 | 145 | 136 |
| 131 | 126 | 128 | 142 | 141 | 135 | 126 | 116 |
| 131 | 140 | 146 | 154 | 133 | 118 | 124 | 124 |

*Pixelvalues*

|      |     |     |     |     |    |    |    |
|------|-----|-----|-----|-----|----|----|----|
| 1037 | -1  | -6  | 1   | -12 | 8  | -4 | -4 |
| -16  | 1   | 28  | -6  | -14 | 0  | 4  | 0  |
| 19   | 32  | -7  | -19 | 2   | -1 | -4 | -3 |
| 29   | -9  | -14 | 13  | -10 | -6 | 1  | 0  |
| 4    | 14  | -6  | -13 | -2  | 7  | 1  | 2  |
| -26  | 2   | 16  | 2   | 11  | 6  | 1  | 1  |
| -10  | -11 | 27  | -18 | 4   | 1  | 0  | 0  |
| -2   | 1   | 1   | -19 | -1  | 6  | 6  | 0  |

*DCT- Values*

|     |    |    |    |    |   |   |   |
|-----|----|----|----|----|---|---|---|
| 130 | 0  | 0  | 0  | -1 | 0 | 0 | 0 |
| -2  | 0  | 3  | 0  | -1 | 0 | 0 | 0 |
| 2   | 3  | 0  | -1 | 0  | 0 | 0 | 0 |
| 3   | -1 | -1 | 1  | 0  | 0 | 0 | 0 |
| 0   | 1  | 0  | -1 | 0  | 0 | 0 | 0 |
| -2  | 0  | 2  | 0  | 0  | 0 | 0 | 0 |
| -1  | -1 | 2  | -1 | 0  | 0 | 0 | 0 |
| 0   | 0  | 0  | -1 | 0  | 0 | 0 | 0 |

*Quantized DCT- Values*

# Quantization Matrix



- Quantization factor for each DCT value is defined using a quantization matrix
  - Disturbances in low frequency parts of the image are perceived strongly
  - Disturbances in high frequency parts of the image are less noticeable
  - Quantization matrices in JPEG standard ensures that the DCT values for low frequencies are stored more accurately than for high frequencies
  - Quantization table can be freely selected in the compression

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 8  | 16 | 19 | 22 | 26 | 27 | 29 | 34 |
| 16 | 16 | 22 | 24 | 27 | 29 | 34 | 37 |
| 19 | 22 | 26 | 27 | 29 | 34 | 34 | 38 |
| 22 | 22 | 26 | 27 | 29 | 34 | 37 | 40 |
| 22 | 26 | 27 | 29 | 32 | 35 | 40 | 48 |
| 26 | 27 | 29 | 32 | 35 | 40 | 48 | 58 |
| 26 | 27 | 29 | 34 | 38 | 46 | 56 | 69 |
| 27 | 29 | 35 | 38 | 46 | 56 | 69 | 83 |

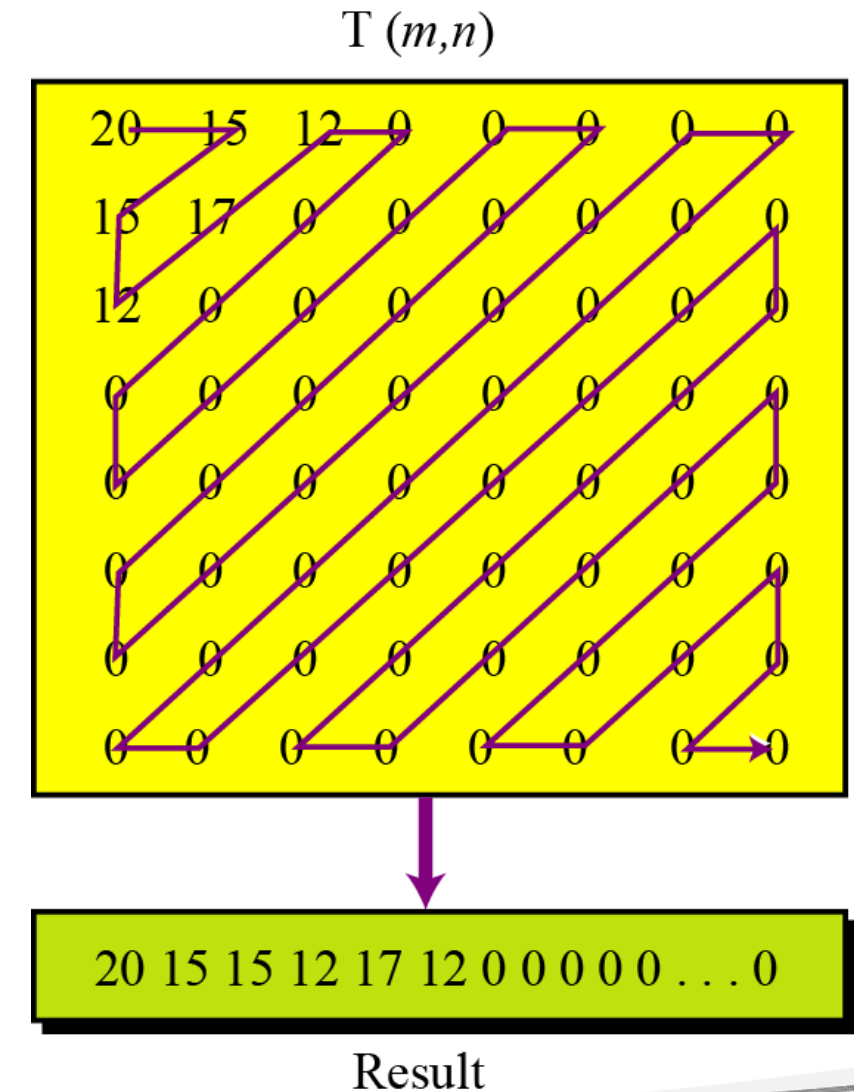
*Quantization Matrix*

|    |    |    |    |     |     |     |     |
|----|----|----|----|-----|-----|-----|-----|
| 16 | 11 | 10 | 16 | 24  | 40  | 51  | 61  |
| 12 | 12 | 14 | 19 | 26  | 58  | 60  | 55  |
| 14 | 13 | 16 | 24 | 40  | 57  | 69  | 56  |
| 14 | 17 | 22 | 29 | 51  | 87  | 80  | 62  |
| 18 | 22 | 37 | 56 | 68  | 109 | 103 | 77  |
| 24 | 35 | 55 | 64 | 81  | 104 | 113 | 92  |
| 49 | 64 | 78 | 87 | 103 | 121 | 120 | 101 |
| 72 | 92 | 95 | 98 | 112 | 100 | 103 | 99  |

*Quantization Matrix*

# Compression

- After quantization values are read from the table, and redundant 0s are removed.
- To cluster 0s together process reads the table diagonally in a zigzag fashion because if image does not have fine changes bottom right corner of table is all 0s.
- JPEG uses run-length encoding at the compression phase to compress the bit pattern resulting from the zigzag linearization.



# Quantization



## Coding

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 134 | 142 | 145 | 131 | 114 | 122 | 131 | 130 |
| 129 | 143 | 134 | 130 | 135 | 144 | 134 | 118 |
| 123 | 117 | 118 | 111 | 97  | 109 | 130 | 143 |
| 129 | 116 | 112 | 116 | 120 | 126 | 130 | 118 |
| 118 | 127 | 141 | 138 | 138 | 148 | 141 | 125 |
| 125 | 129 | 119 | 127 | 143 | 149 | 145 | 136 |
| 131 | 126 | 128 | 142 | 141 | 135 | 126 | 116 |
| 131 | 140 | 146 | 154 | 133 | 118 | 124 | 124 |

*Pixelvalues*

|      |     |     |     |     |    |    |    |
|------|-----|-----|-----|-----|----|----|----|
| 1037 | -1  | -6  | 1   | -12 | 8  | -4 | -4 |
| -16  | 1   | 28  | -6  | -14 | 0  | 4  | 0  |
| 19   | 32  | -7  | -19 | 2   | -1 | -4 | -3 |
| 29   | -9  | -14 | 13  | -10 | -6 | 1  | 0  |
| 4    | 14  | -6  | -13 | -2  | 7  | 1  | 2  |
| -26  | 2   | 16  | 2   | 11  | 6  | 1  | 1  |
| -10  | -11 | 27  | -18 | 4   | 1  | 0  | 0  |
| -2   | 1   | 1   | -19 | -1  | 6  | 6  | 0  |

*DCT- Values*

|     |    |    |    |    |   |   |   |
|-----|----|----|----|----|---|---|---|
| 130 | 0  | 0  | 0  | -1 | 0 | 0 | 0 |
| -2  | 0  | 3  | 0  | -1 | 0 | 0 | 0 |
| 2   | 3  | 0  | -1 | 0  | 0 | 0 | 0 |
| 3   | -1 | -1 | 1  | 0  | 0 | 0 | 0 |
| 0   | 1  | 0  | -1 | 0  | 0 | 0 | 0 |
| -2  | 0  | 2  | 0  | 0  | 0 | 0 | 0 |
| -1  | -1 | 2  | -1 | 0  | 0 | 0 | 0 |
| 0   | 0  | 0  | -1 | 0  | 0 | 0 | 0 |

*Quantized DCT- Values*

## Decoding

|      |     |    |     |     |   |   |   |
|------|-----|----|-----|-----|---|---|---|
| 1040 | 0   | 0  | 0   | -9  | 0 | 0 | 0 |
| -12  | 0   | 24 | 0   | -10 | 0 | 0 | 0 |
| 14   | 24  | 0  | -10 | 0   | 0 | 0 | 0 |
| 24   | -8  | -9 | 10  | 0   | 0 | 0 | 0 |
| 0    | 9   | 0  | -10 | 0   | 0 | 0 | 0 |
| -19  | 0   | 10 | 0   | 0   | 0 | 0 | 0 |
| -9   | -10 | 21 | -12 | 0   | 0 | 0 | 0 |
| 0    | 0   | 0  | -14 | 0   | 0 | 0 | 0 |

*Reconstructed DCT- Values*

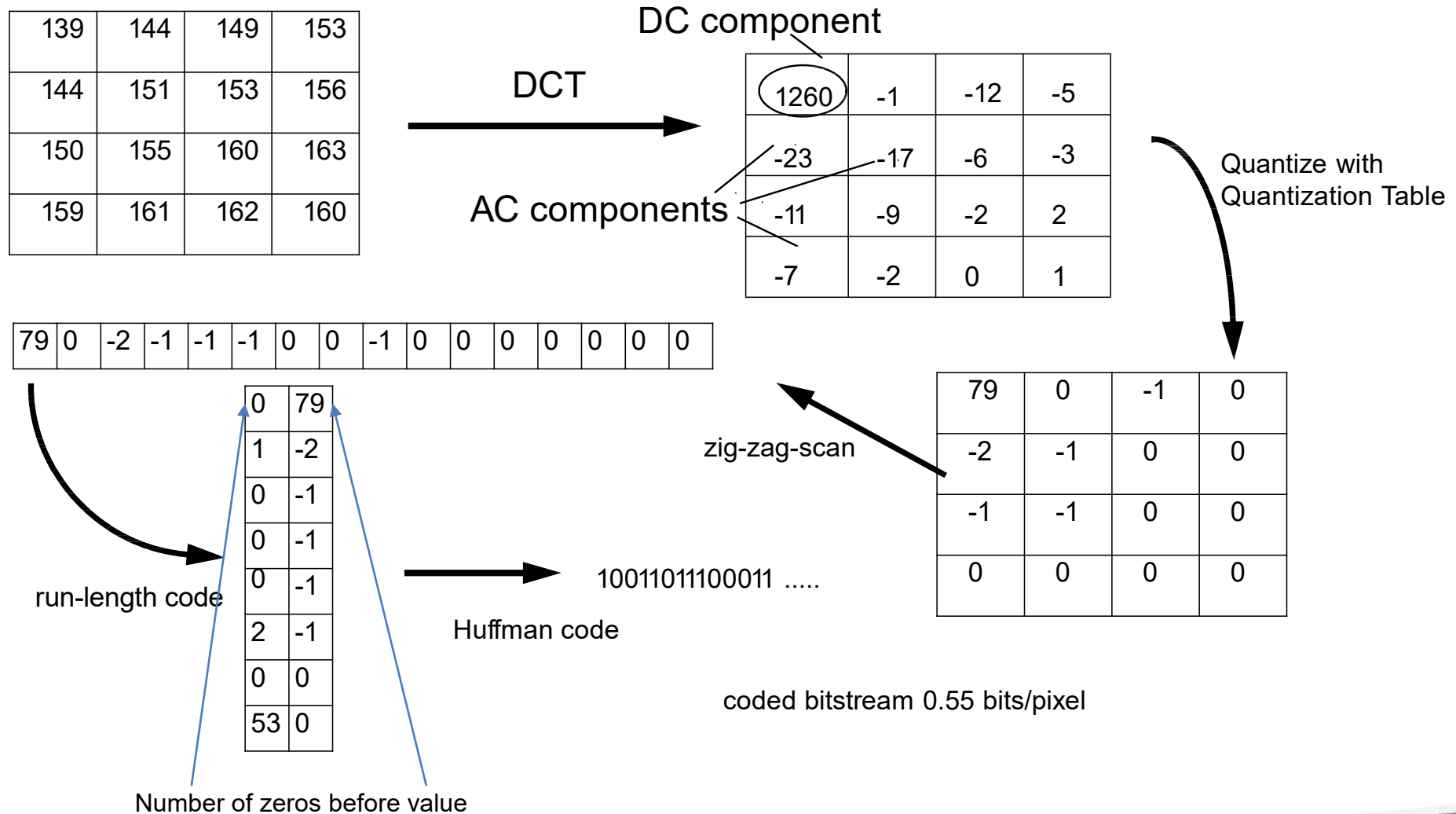
|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 136 | 141 | 138 | 125 | 119 | 125 | 132 | 134 |
| 136 | 137 | 133 | 130 | 134 | 139 | 133 | 121 |
| 121 | 125 | 122 | 112 | 107 | 117 | 130 | 138 |
| 125 | 123 | 119 | 117 | 121 | 128 | 127 | 122 |
| 123 | 129 | 135 | 139 | 140 | 139 | 136 | 131 |
| 129 | 125 | 124 | 130 | 139 | 144 | 141 | 136 |
| 129 | 130 | 134 | 139 | 140 | 135 | 127 | 120 |
| 132 | 138 | 144 | 141 | 131 | 122 | 122 | 127 |

*Reconstructed Pixels (IDCT)*

|    |    |    |    |     |    |    |    |
|----|----|----|----|-----|----|----|----|
| -2 | 1  | 7  | 6  | -5  | -3 | -1 | -4 |
| -7 | 6  | 1  | 0  | 1   | 5  | 1  | -3 |
| 2  | -8 | -4 | -1 | -10 | -8 | 0  | 5  |
| 4  | -7 | -7 | -1 | -1  | -2 | 3  | -4 |
| -5 | -2 | 6  | -1 | -2  | 9  | 5  | -6 |
| -4 | 4  | -5 | -3 | 4   | 5  | 4  | 0  |
| 2  | -4 | -6 | 3  | 1   | 0  | -1 | -4 |
| 1  | 2  | 2  | 13 | 2   | -4 | 1  | -3 |

*Errors (Difference)*

# JPEG Block Encoding (Example)

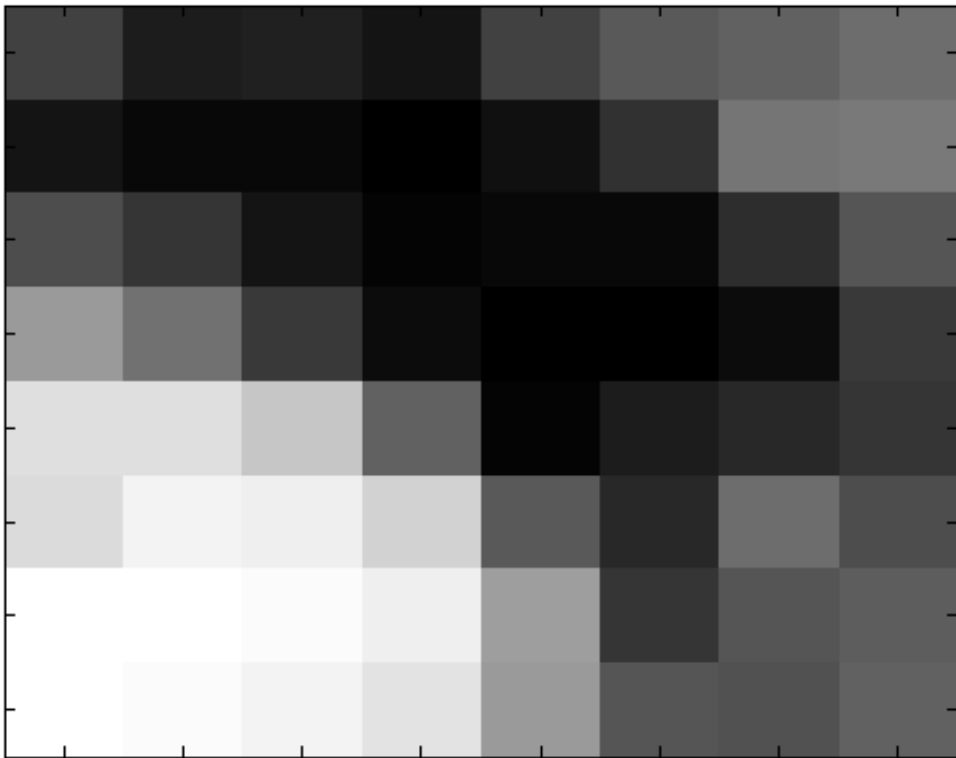




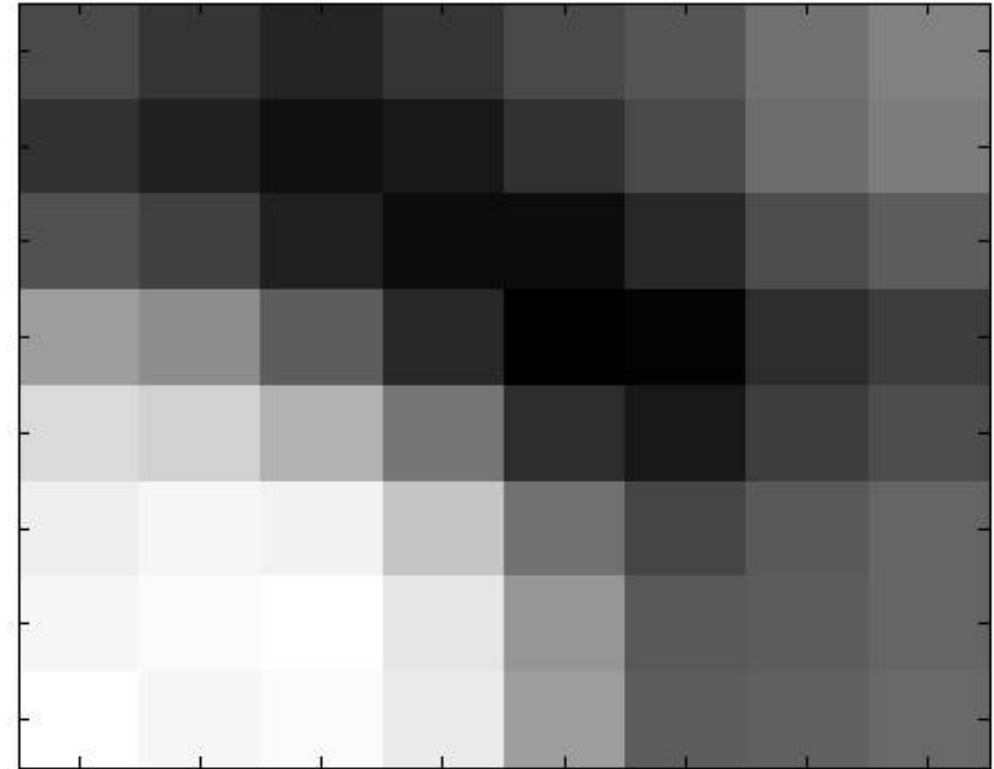
# Can You Tell the Difference?



Original



Compressed



# Image Compression



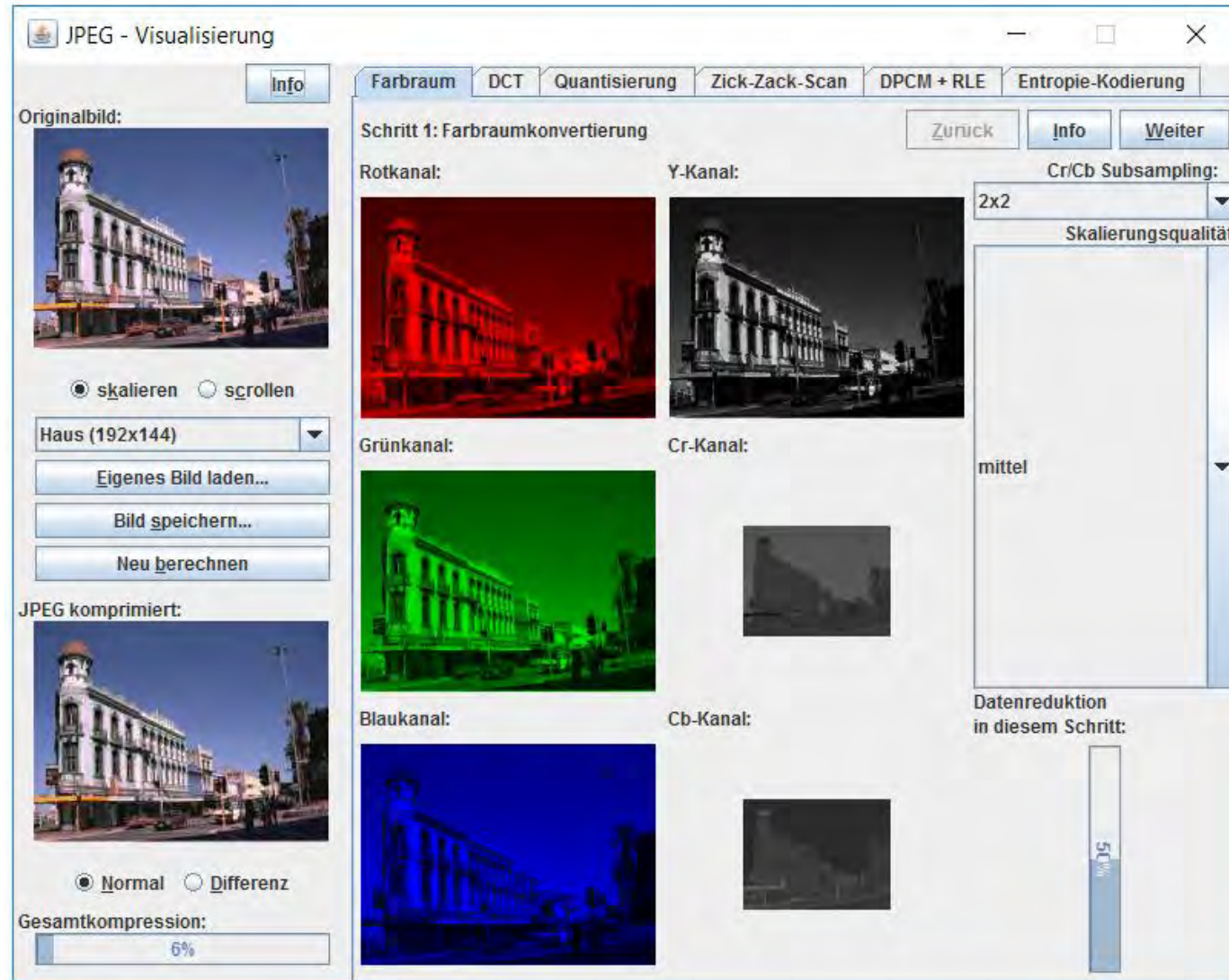
Original



Compressed



# VisJPEG-Applet



# Problems with JPEG

- Behavior in sharp transitions (e.g. fonts)
- Emergence of the 8x8 blocks at high compression rates.
- The greater the compression (quantization), the more artefacts occur (block formation)
- JPEG is made for natural images and not for artificial images (computer graphics)!



<https://www.youtube.com/watch?v=yBX8GFqt6GA&list=PLzH6n4zXuckoAod3z31QEST1ZaizBuNHh&index=4>



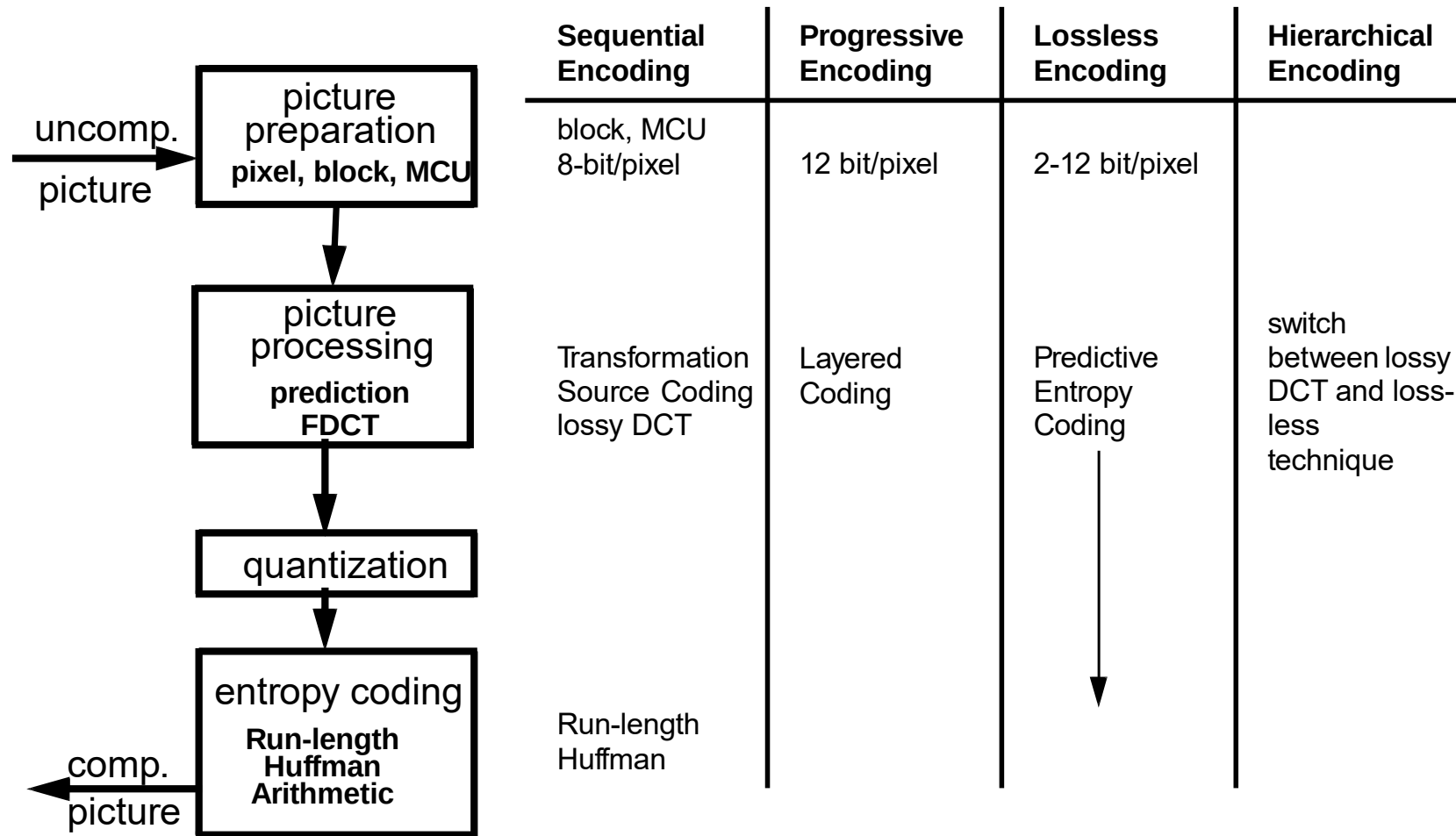
# Variations & Additions to JPEG

## Additional Material (not for Exam!)

Slides by Prof. Dr. Otto Spaniol, Dirk Thißen  
RWTH Aachen



# JPEG Processing Steps

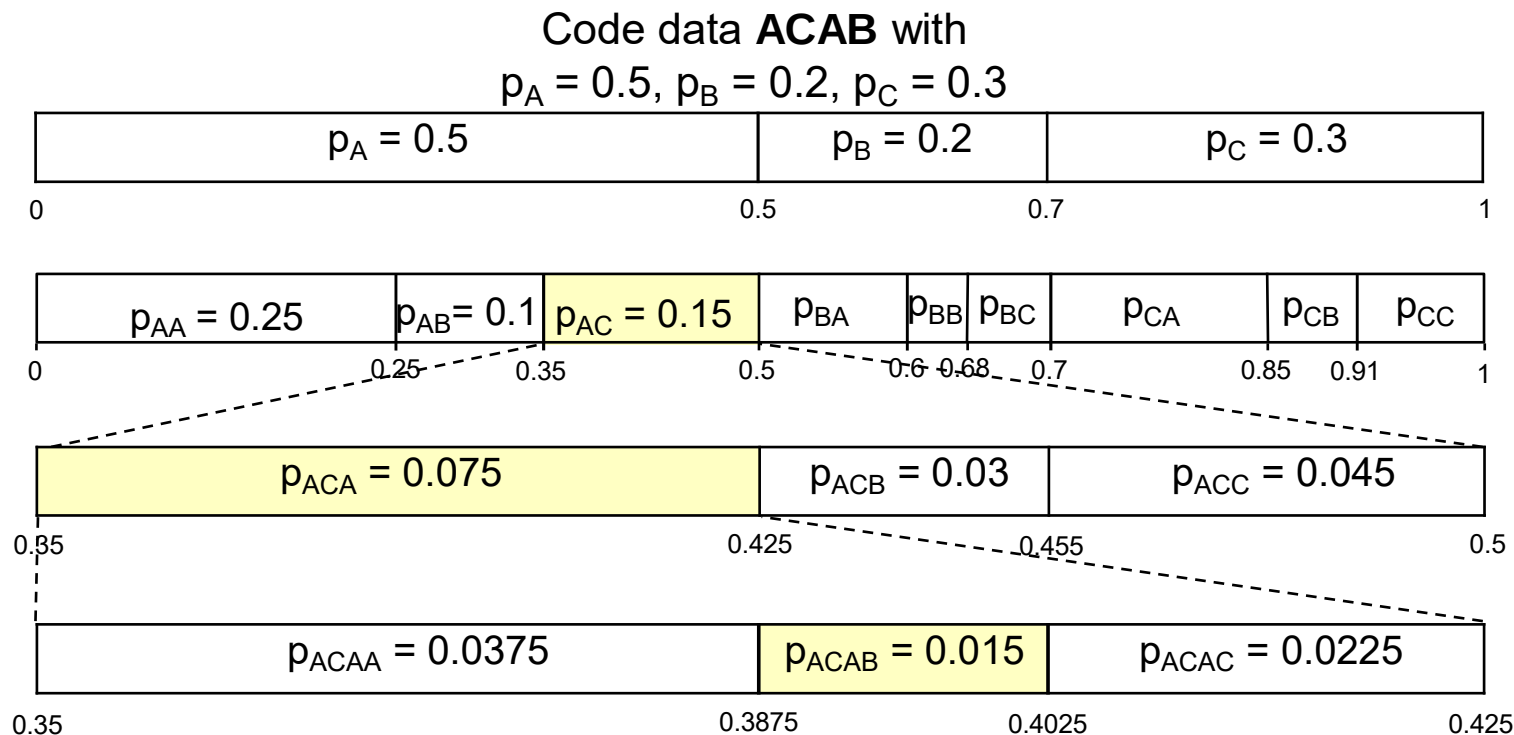


## Alternative to Huffman: Arithmetic Coding

### Characteristics:

- Achieves optimality (coding rate) as the Huffman coding
- Difference to Huffman: the entire data stream has an assigned probability, which consists of the probabilities of the contained characters. Coding a character takes place with consideration of all previous characters.
- The data are coded as an interval of real numbers between 0 and 1. Each value within the interval can be used as code word.
- The minimum length of the code is determined by the assigned probability.
- Disadvantage: the data stream can be decoded only as a whole.

## Arithmetic Coding: Example



ACAB can be coded by each binary number from the interval  $[0.3875, 0.4025)$ , rounded up to  $-\log_2(p_{ACAB}) = 6.06$  i.e. 7 bit, e.g. **0.0110010**

## Variants of Image Compression

JPEG is not a single format, but it can be chosen from a number of modes:

*Lossy sequential DCT-based mode* (baseline process)

- Presented before, but not the only method

*Expanded lossy DCT-based mode*

- Enhancement to the baseline process by adding progressive encoding

*Lossless mode*

- Low compression ratio → „perfect“ reconstruction of original image
- No DCT, but differential encoding

*Hierarchical mode*

- Accommodates images of different resolutions
- Selects its algorithms from the three other modes

## Variants: Expanded Lossy DCT-based Mode

With *sequential encoding* as in the baseline process the whole image is coded and decoded in a single run. An alternative to sequential encoding is *progressive encoding*, done in the entropy encoding step.

Two alternatives for progressive encoding are possible:

- *Spectral selection*

At first, coefficients of low frequencies are passed to entropy encoding, coefficients of higher frequencies are processed in successive runs

- *Successive approximation*

All coefficients are transferred in one run, but most-significant bits are encoded prior to less-significant bits.

12 possible coding alternatives in the expanded mode:

- Using sequential encoding, spectral selection, or successive approximation (3 variants)
- Using Huffman or Arithmetic encoding (2 variants)
- Using 8 or 12 bits for representing the samples (2 variants)

*Most popular mode*: sequential display mode with 8 bits/sample and Huffman encoding

## Expanded Lossy DCT-based Mode (Example)

**Sequential encoding:** image is coded and decoded in a single run



Step 1



Step 2



Step 3

**Progressive encoding:** image is coded and decoded in refining steps



Step 1



Step 2



Step 3



## Variants: Lossless Mode

Lossless mode uses *differential encoding*

(Differential encoding is also known as *prediction* or *relative encoding*)

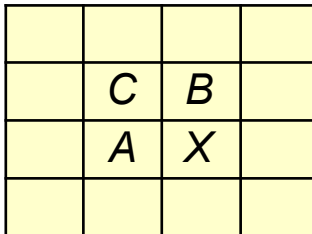
- Sequence of characters whose values are different from zero, but which do not differ much.
  - Calculate only the difference wrt. the previous value (used also for DC-coefficients)

Differential encoding for still images:

- Avoid using DCT/quantization
- Instead: calculation of differences between nearby pixels or pixel groups
  - Edges are represented by large values
  - Areas with similar luminance and chrominance are represented by small values
  - Homogenous area is represented by a large number of zeros
    - further compression with run-length encoding is possible as for DCT

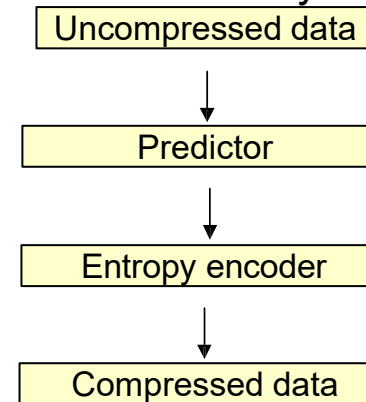
## Variants: Lossless Mode

- Uses data units of single pixels for image preparation
- Any precision between 2 and 16 bits/pixel can be used
- Image processing and quantization use a predictive technique instead of transformation encoding
- 8 predictors are specified for each pixel  $X$  by means of a combination of the already known adjacent samples  $A$ ,  $B$ , and  $C$



The actual predictor should give the best approximation of  $x$  by the already known values  $A, B, C$

| predictor | predicted values $X$ |
|-----------|----------------------|
| 0         | no prediction        |
| 1         | $A$                  |
| 2         | $B$                  |
| 3         | $C$                  |
| 4         | $A+B-C$              |
| 5         | $A+(B-C)/2$          |
| 6         | $B+(A-C)/2$          |
| 7         | $(A+B)/2$            |



The number of the chosen predictor and the difference of the prediction to the actual value are passed to entropy encoding (Huffman or Arithmetic Encoding)

### Example:

(4,0):  $X$  is exactly given by  $A+B-C$   
 (7,1):  $X$  is  $(A+B)/2+1$

## Variants: Hierarchical Mode

The *Hierarchical mode* uses either the lossy DCT-based algorithms or the lossless compression technique

The idea: encoding of an image at different resolutions

*Algorithm:*

- Image is initially sampled at a low resolution
- Subsequently, the resolution is raised and the compressed image is subtracted from the previous result
- The process is repeated until the full resolution of the image is obtained in a compressed form

*Disadvantage:*

- Requires substantially more storage capacity

*Advantage:*

- Compressed image is immediately available at different resolutions  
→ *scaling becomes cheap*

# Multimedia

## Vorlesung

Hannes Kaufmann

Research Unit Virtual and Augmented Reality  
Institute of Visual Computing & Human-Centered Technology

Based on material by Prof. Christian Breiteneder, a few slides by Prof. Robert Sablatnig (EVC)  
Material on HDR by Scott Janus, Principal Engineer at Intel; Thuong Nguyen [tnguyen@humaxdigital.com](mailto:tnguyen@humaxdigital.com); Prof. Marc Pollefeys and Dr. Gabriel Brostow at ETHZ  
A few slides on JPEG used by courtesy of Dr. Dirk Thißen from RWTH Aachen

# Media Types

- History, representation, formats and operations of media types
- Text
- Audio
- **Image**
- Video
- No established digital standards for other media types:  
smell, taste, haptics

# Media Type: Image

- Human Visual System & Color Models
- High Dynamic Range
- Image Compression



# Human Visual System

Helligkeitssehen:

~120 Mill. "Stäbchen"/Rods

Farbsehen:

~7 Mill. "Zapfen"/Cones



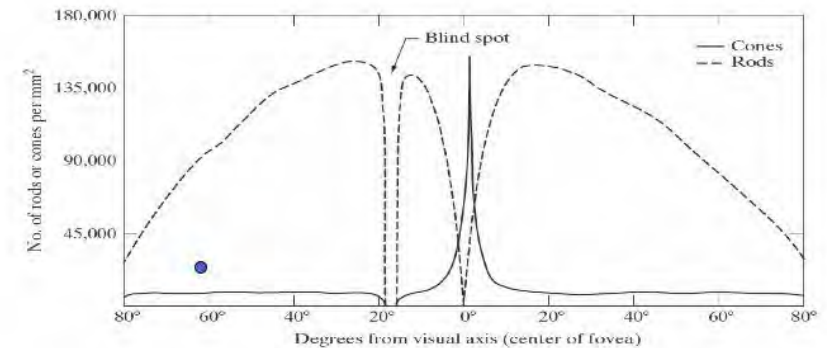
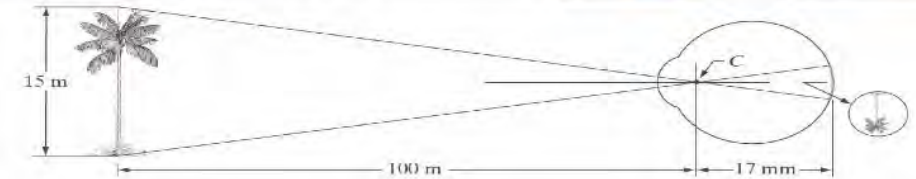
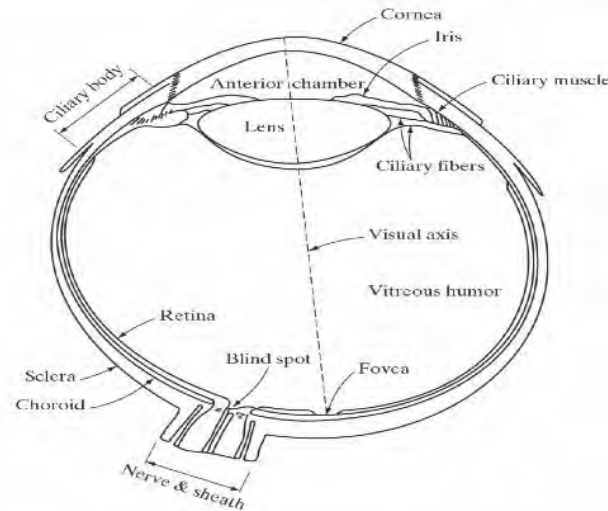
*Digital Image Processing, 3rd ed.*

Gonzalez & Woods

[www.ImageProcessingPlace.com](http://www.ImageProcessingPlace.com)

Chapter 2

Digital Image Fundamentals



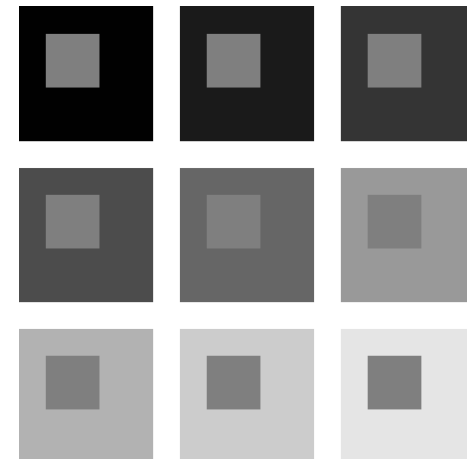
© 1992–2008 R. C. Gonzalez & R. E. Woods

# Light

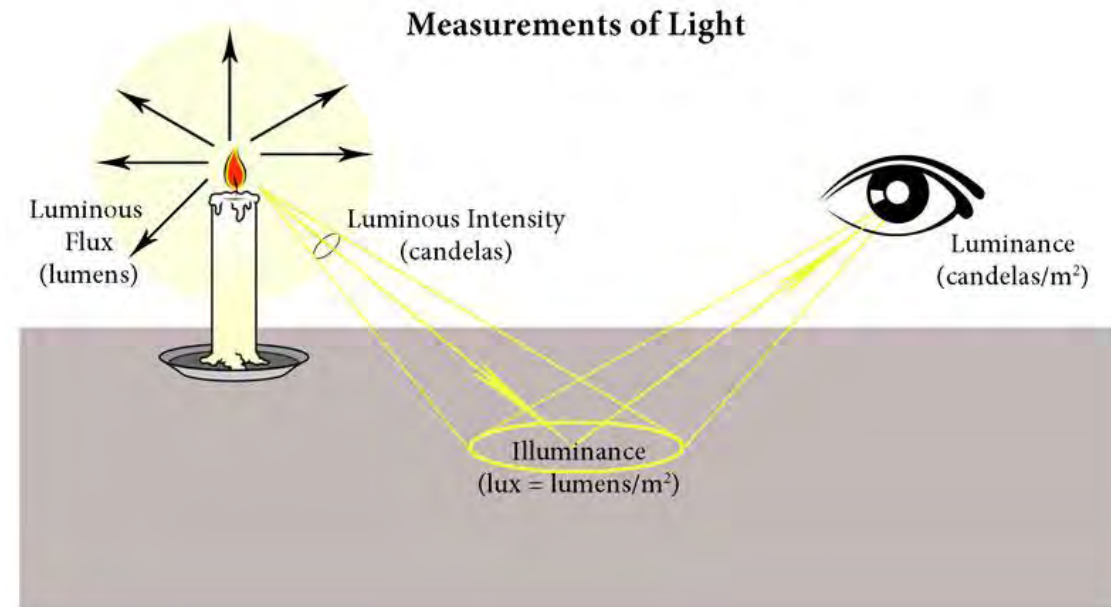
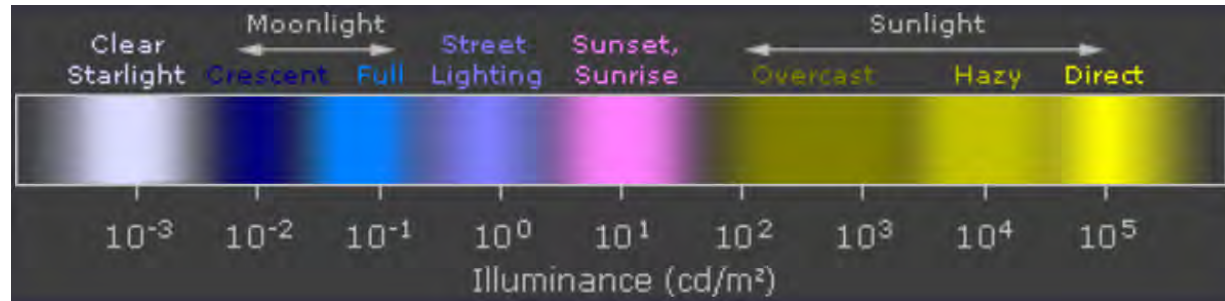
- Chromatic: of color
- Achromatic: without color (i.e. B&W)
- Color of an object depends upon:
  - object surface: reflectivity, physical properties, composition
  - light source(s) illuminating it
  - color of surrounding environment
  - visual system (eyes, brain, photoelectric cell, ...) of perceiving entity

# Achromatic Light

- black, white, usually greys (rare to see pure black or pure white)
- quantity or energy of light is the only measurement used for achromatic light
  - Intensity, luminance, brightness:
    - **Luminance** describes the amount of light that is emitted from, passed through, or reflected off an object. It is measured using candela per square meter ( $\text{cd/m}^2$ ), which is the concentration of luminous intensity per unit area.
    - **Brightness** is subjective and perceived by the human eye. It cannot be measured by any formula
- eye is sensitive to ratios of intensity changes, not the changes in intensity values themselves
- small changes of low intensity seem to have the same effect as larger changes of high intensity
- **Dynamic range**: ratio between min. and max. intensities



# Illuminance

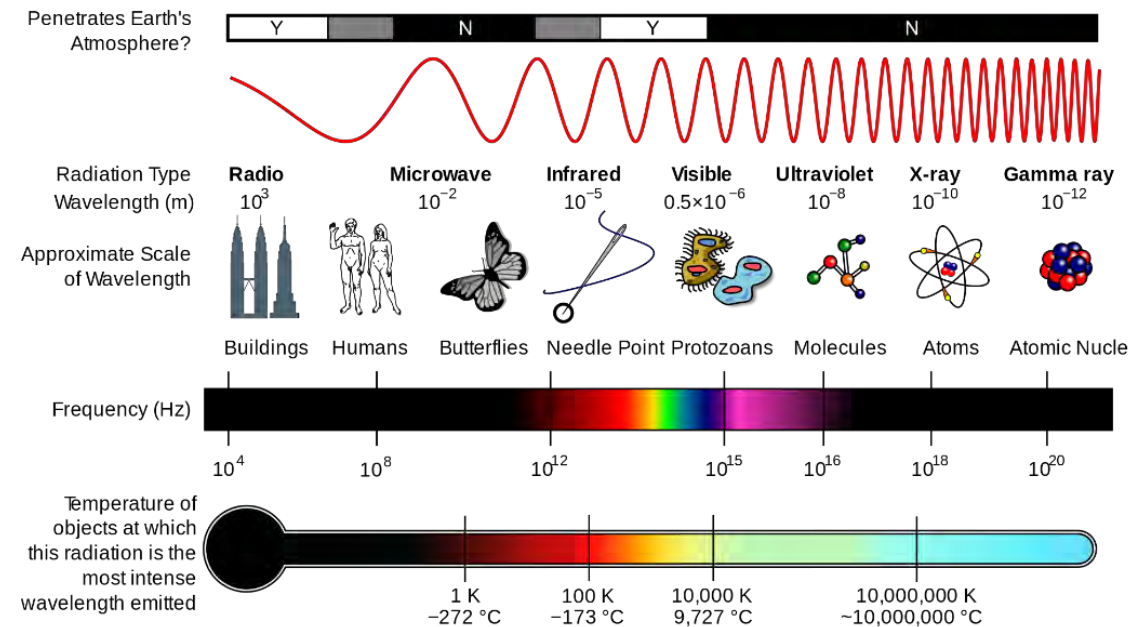
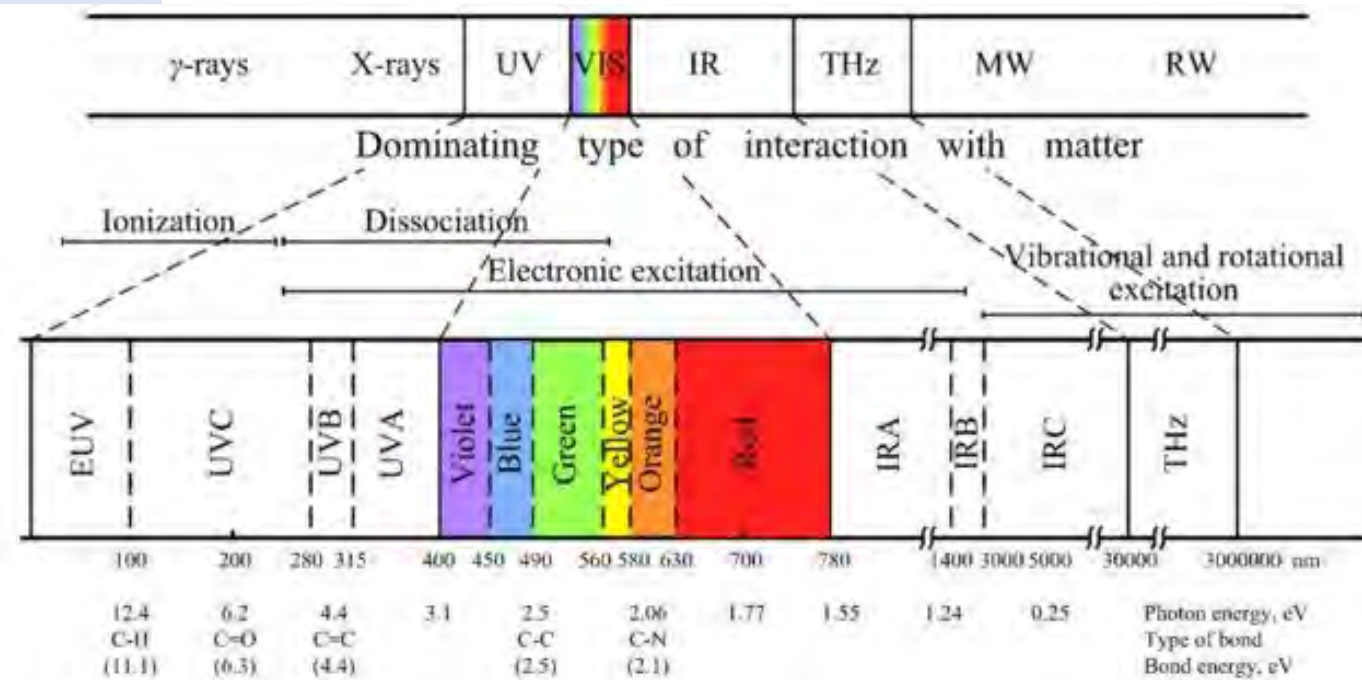


Illuminance = only incident light

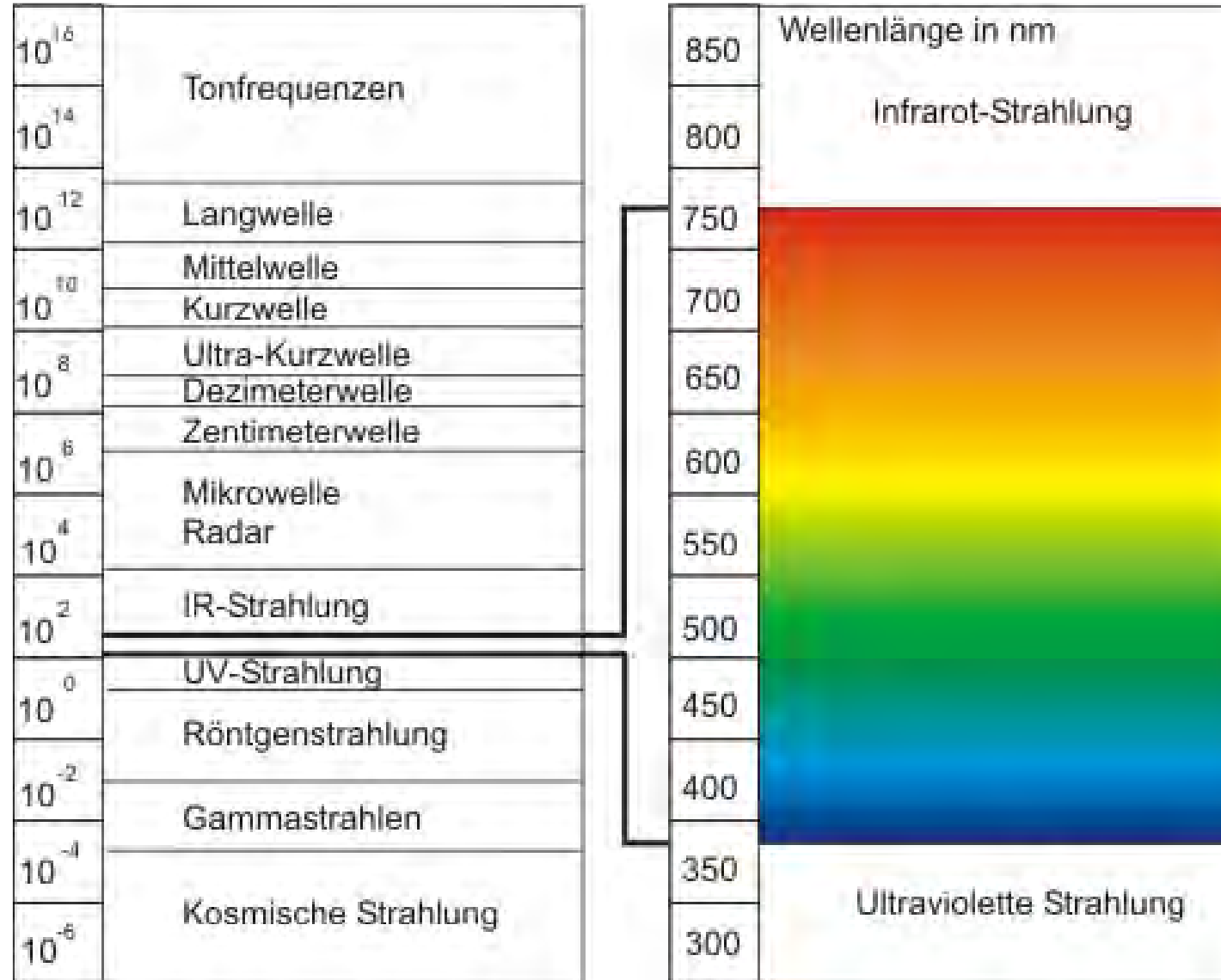
- Illuminance is light we can't see directly, so-called ambient light;
- Light passing through air which is actually invisible until it reflects off an object (which we then see as luminance).
- It is measured in lux - **luminous flux** per unit area ( $\text{lx}$  or  $\text{lm/m}^2$ ).

# Color, Light - Wavelengths

- Color is how we perceive beams of electromagnetic energy that fall on our retina
- We see light between 400 and 700 nanometers in wavelength ( $10^{-9}$  meters)



# Light Spectrum – Wavelengths



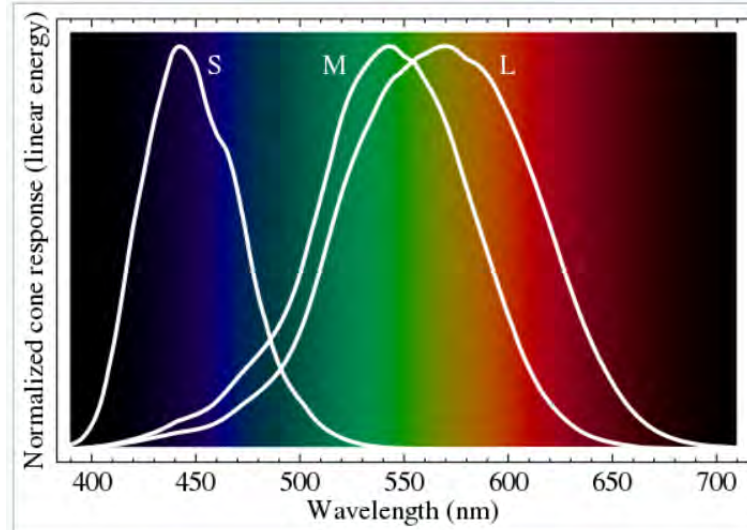


# Color

- Various models of color used:
  - Printing & graphics industry: color sample books and codes
  - Artists: tint (adding white to pure pigment), shade (adding black), tone (adding both)
  - Displays: use hardware capabilities (RGB)
  - Physicists: use optical models (wavelengths, energy measures)
- Human perception of color depends on how brain reacts to the whole visual environment, as well as brain & eye biological factors

# Tristimulus Model

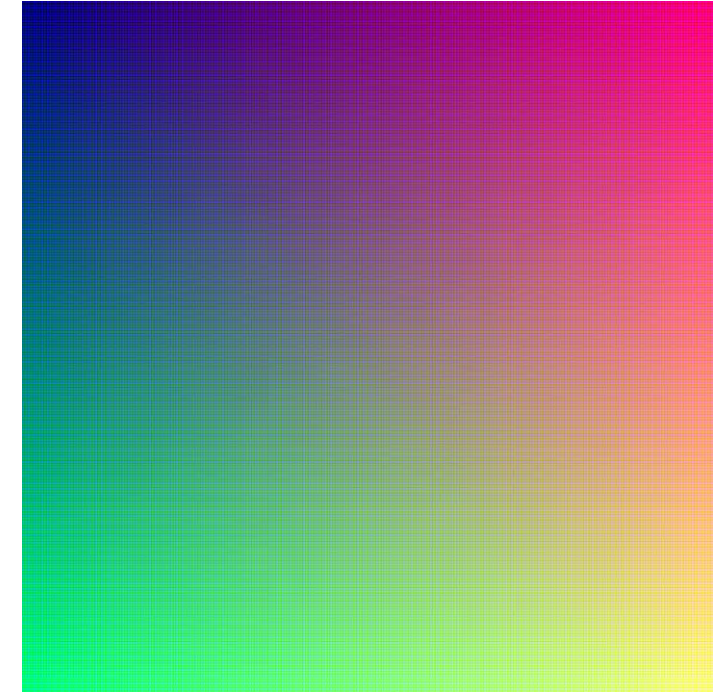
- The retina contains cones that sense either red, green, or blue light
- 6-7 million cones per eye, concentrated in central area called "fovea"
- Rods: sensors that surround fovea, and perceive weak "night" images (>100 million per eye)



Normalized response spectra of human cones, to monochromatic spectral stimuli, with wavelength given in nanometers.

# Color vision in Nonhumans

The human eye can distinguish about 10 million different colors (up to debate)



16.7 million colors

Color vision table

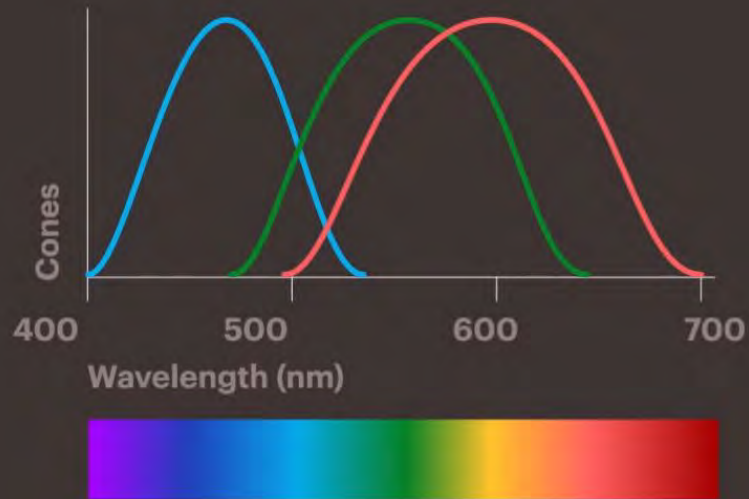
| State                | Types of cone cells | Approx. number of colors perceived | Carriers                                                                                            |
|----------------------|---------------------|------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Monochromacy</b>  | 1                   | 200                                | Marine mammals, owl monkey, Australian sea lion, achromat primates                                  |
| <b>Dichromacy</b>    | 2                   | 40,000                             | Most terrestrial non-primate mammals, color blind primates                                          |
| <b>Trichromacy</b>   | 3                   | 10 million <sup>[54]</sup>         | Most primates, especially great apes (such as humans), marsupials, some insects (such as honeybees) |
| <b>Tetrachromacy</b> | 4                   | 100 million                        | Most reptiles, amphibians, birds and insects, rarely humans                                         |
| <b>Pentachromacy</b> | 5                   | 10 billion                         | Some insects (specific species of butterflies), some birds (pigeons for instance)                   |

**The Amazing Ways Animals See the World**

<https://www.youtube.com/watch?v=LlfKk37bkyk>

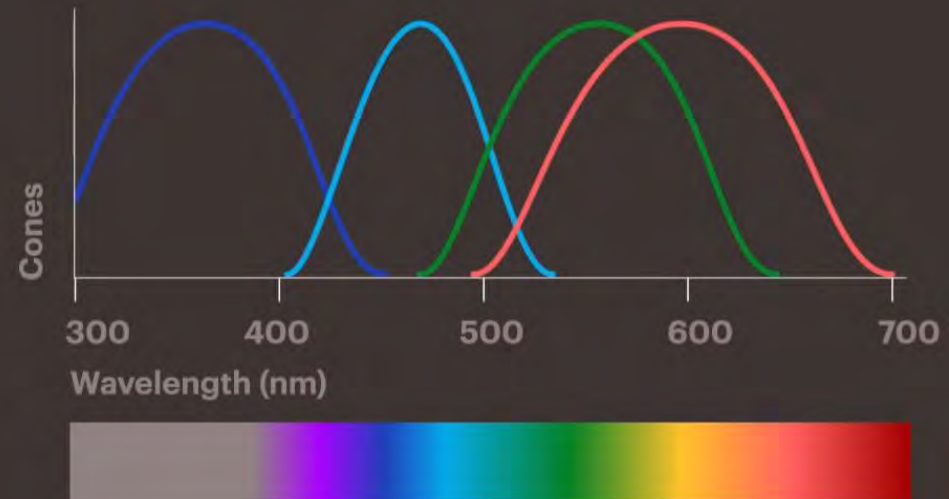


## Humans



3 types of cones

## Birds



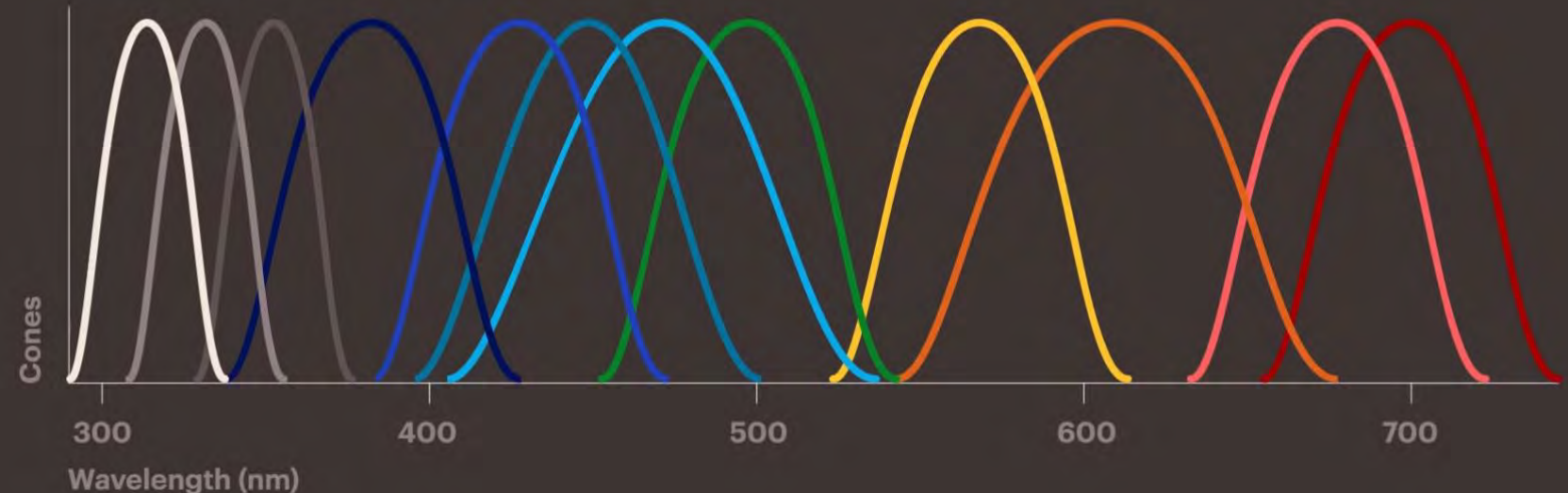
4 types of cones

## Mantis Shrimp:

- 12-16 types of cones
- independent eye movement
- Sees ultraviolet, infrared and 6 types of polarization

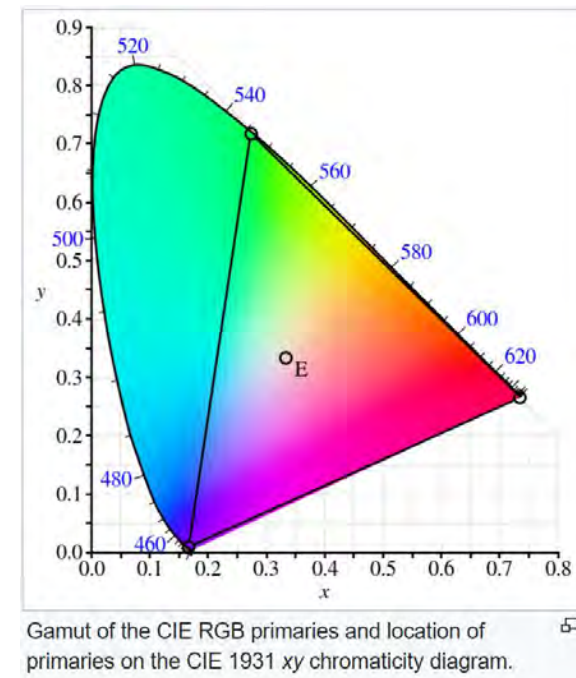


## Mantis Shrimp



# CIE XYZ Color Space (1931)

- CIE color space (CIE = Commission Internationale de l'Eclairage, International Commission on Illumination) used to calibrate other color models;  
1931 CIE XYZ
- Device-invariant representation of color
- Serves as a standard reference against which many other color spaces are defined
- Many combinations of dominant wavelength, luminance, saturation appear the same to us



# CIE Chromaticity Diagram

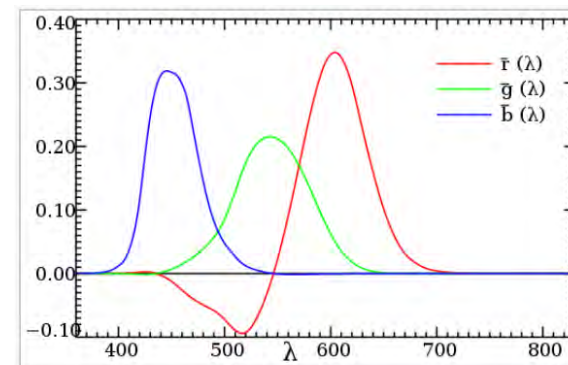
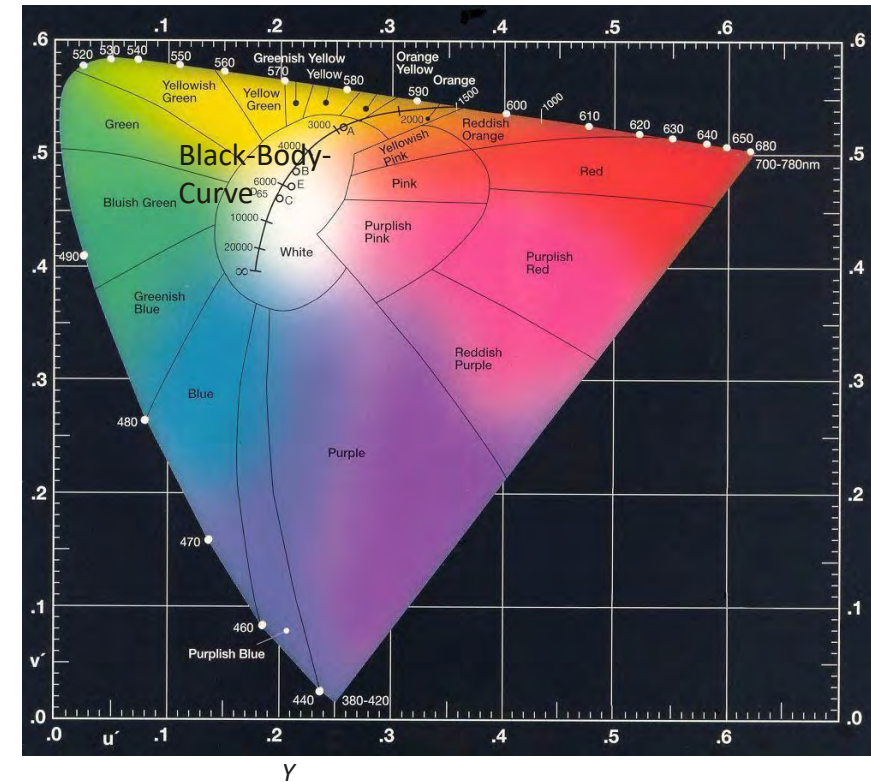
- A full plot of all human visible colors is a three-dimensional figure (cone).
- The concept of color can be divided into two parts: **luminance** and **chromaticity**
- CIE Diagram shows a **planar cut** through that cone at a specific luminance
- Color described in  $x, y$  (with  $x+y+z=1$ )
- Luminance =  $Y$
- Full color:  $(x, y, Y)$

Helligkeitssehen:

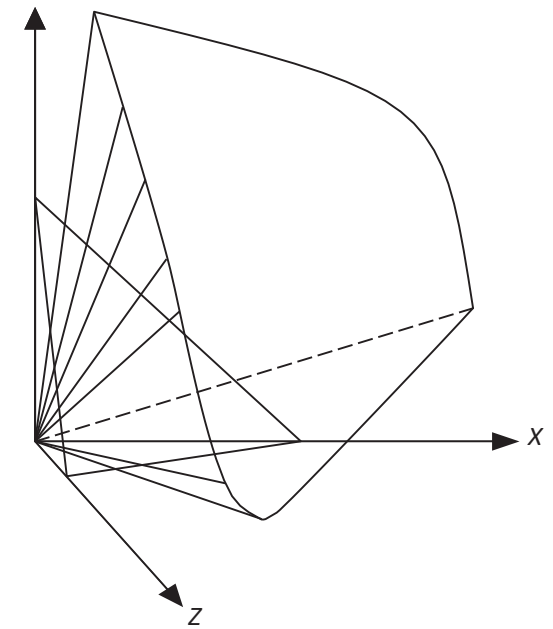
~120 Mill. "Stäbchen"/Rods

Farbsehen:

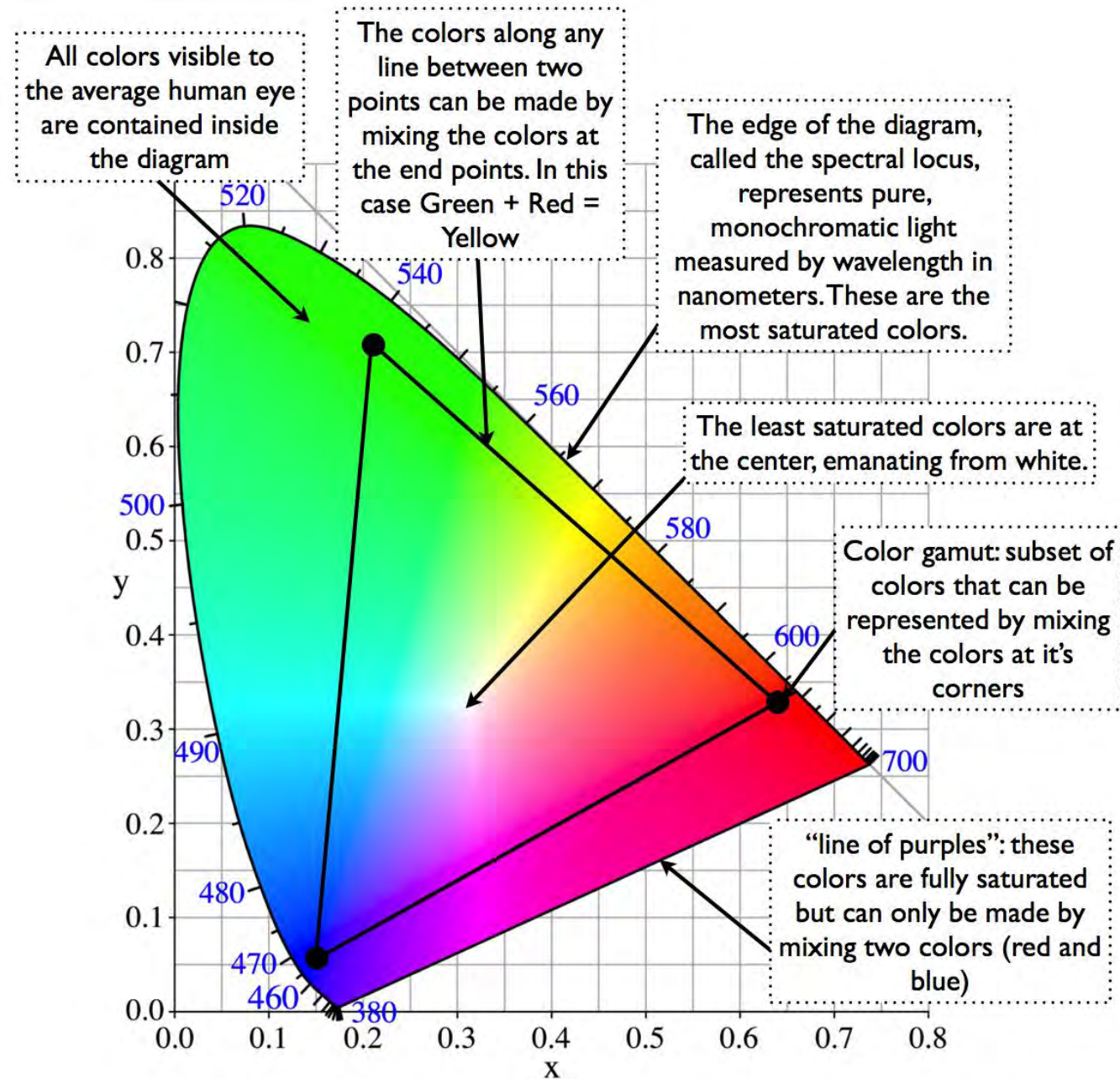
~7 Mill. "Zapfen"/Cones



Die Gewichtungen der drei Primärfarben für die reinen Spektralfarben entlang des Außenrandes des „Hufeisens“ von 360 bis 830 nm







Anatomy of a CIE Chromaticity Diagram

# Quick Overview of Color Spaces

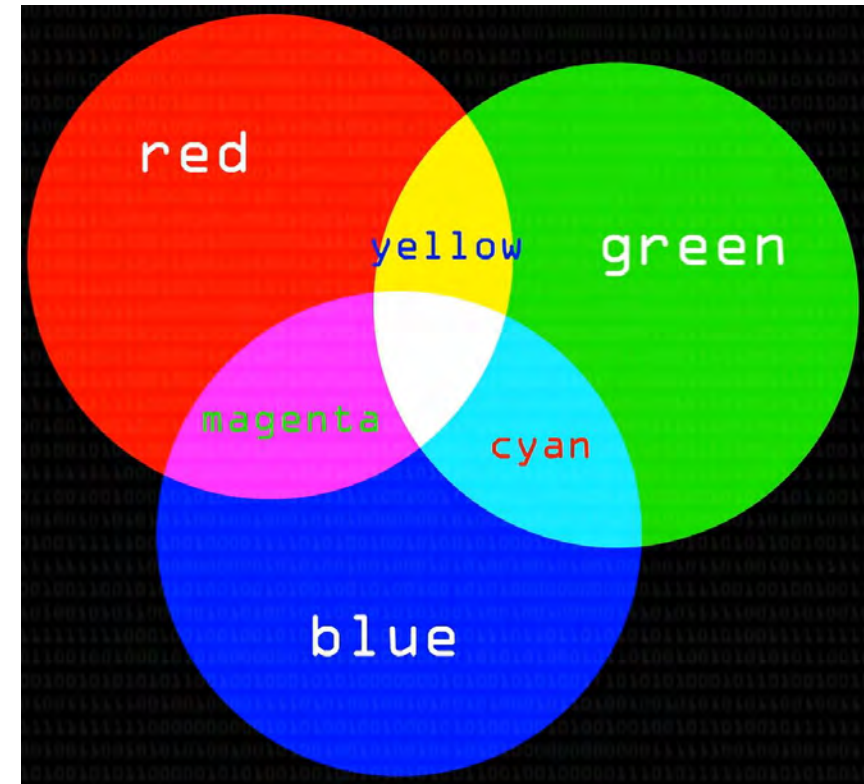
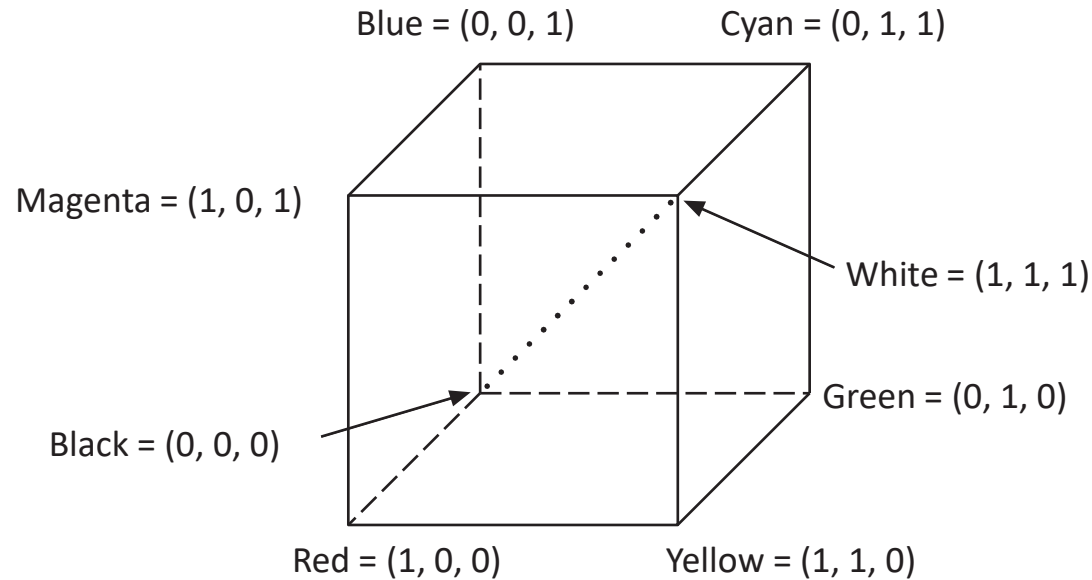


# Color Models (Spaces) Overview

- CIE color space (Commission Internationale de l'Eclairage) used to calibrate other color models 1931 CIE XYZ, tristimulus theory
- RGB — for video display drivers
- CMYK — cyan, magenta, yellow, black, (subtractive primaries)
- HSB — hue (dominant color), saturation (amount of gray), brightness (intensity)
- YUV — used in television industry, Y (luminance), UV (color difference signals) = analog
- YCbCr = digital; Y (luminance),  $C_B$  and  $C_R$  are the blue-difference and red-difference chroma components

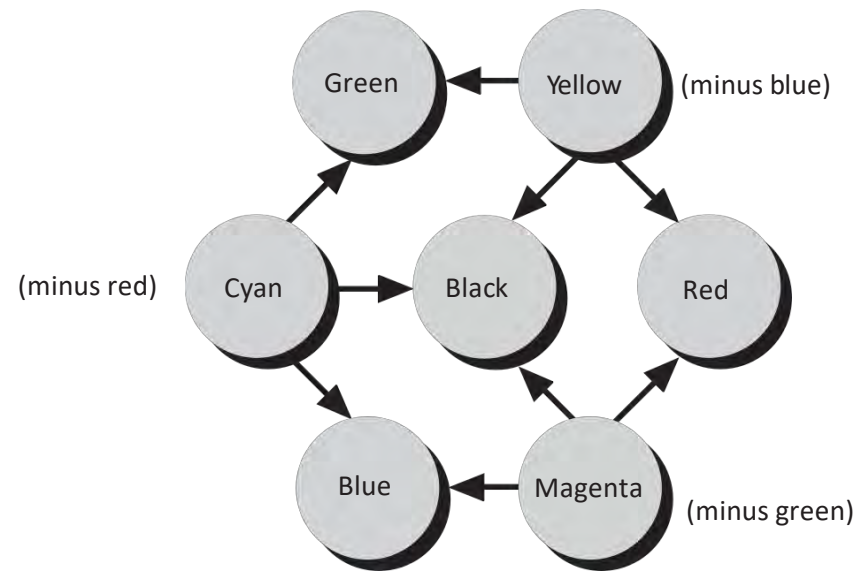


# The RGB Color Model

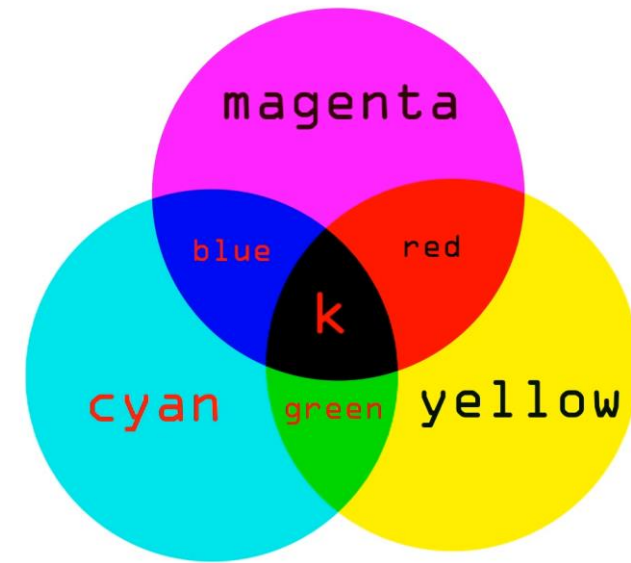


- Hardware-oriented model that turns on red, green, and blue pixels at varying intensities
- Additive: add red, green, blue together to get overall color
- Not all colors are exactly duplicable, and gamut (color range) covered by a display depends on performance of its display technology

# The CMY Color Model



$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} C \\ M \\ Y \end{bmatrix}$$

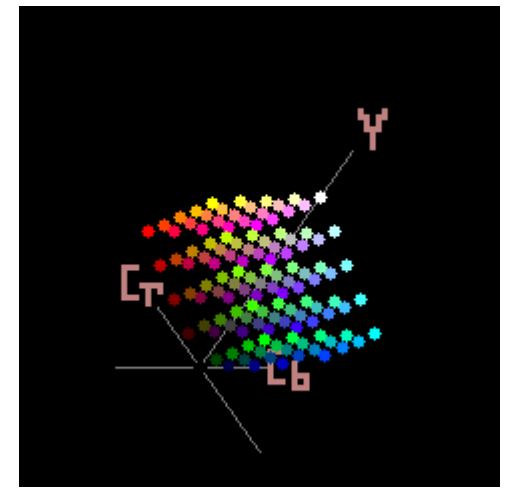


- Another related model, CMYK, uses black as 4th color:

$$K = \min(C, M, Y), C = C - K, M = M - K, Y = Y - K$$

# YCbCr / YUV

- Splits **luminance** (**Y**) from color (**chromacity**) **Cb** (Blue-Yellow Chrominance) und **Cr** (Red-Green Chrominance)
- Historically YUV (analog version) was used to transmit color (UV) to PAL/SECAM standard color TV sets, while black/white TVs only received a Y luminance signal and would still work. The same signal was transmitted for analog b/w and color TVs.
- Humans are very sensitive to changes in luminance, less to chromacity.
- Reducing color resolution allows great compression of images & videos (is used in JPEG)





# Conversion YCbCr <-> RGB

- Necessary in preparation for JPEG compression
- YCbCr (256 levels) can be computed directly from 8-bit RGB as follows:

$$\begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$

- During decoding YCbCr color space needs to be converted back to RGB:

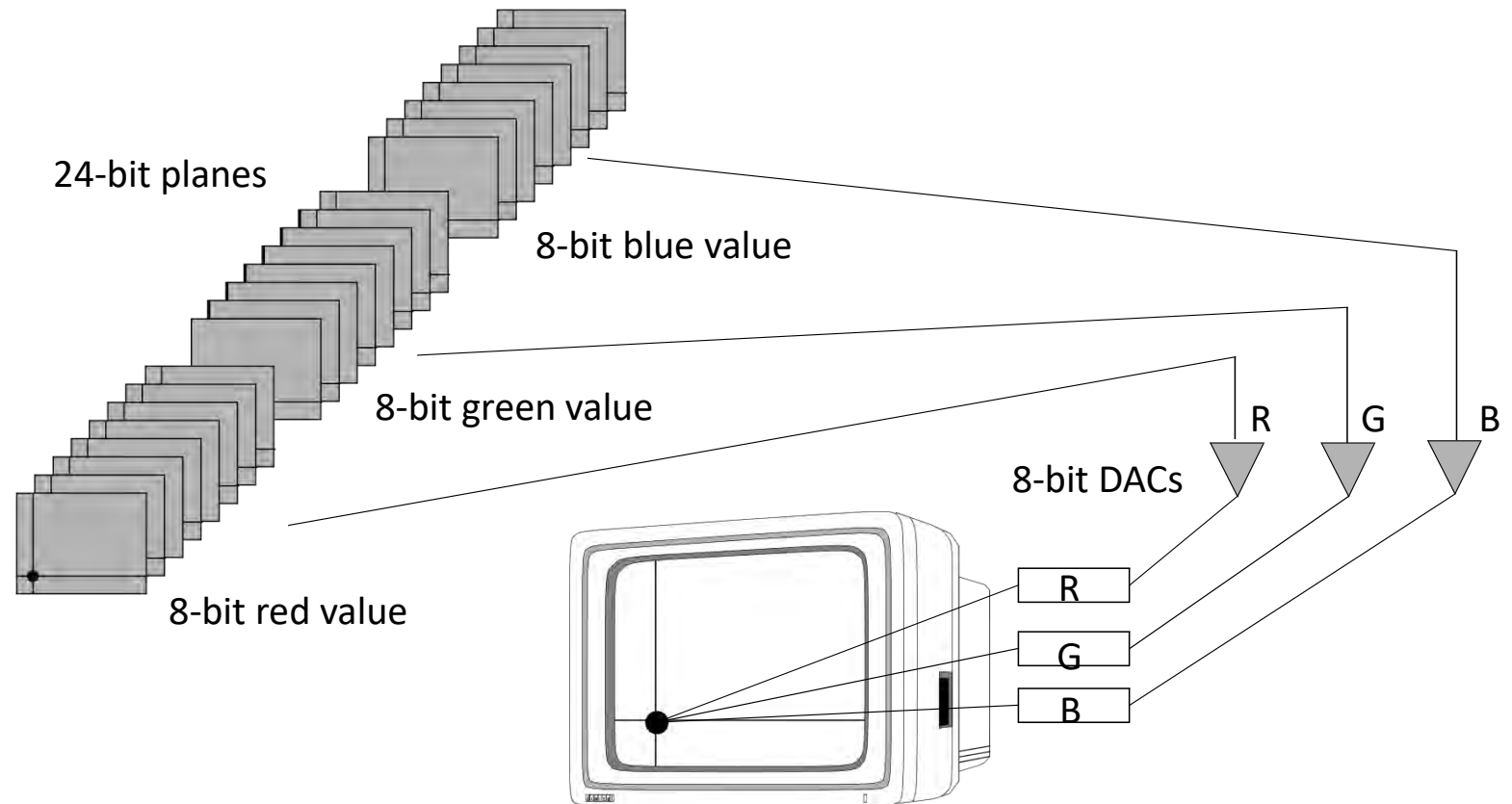
$$R = Y + 1.402 (Cr-128)$$

$$G = Y - 0.34414 (Cb-128) - 0.71414 (Cr-128)$$

$$B = Y + 1.772 (Cb-128)$$

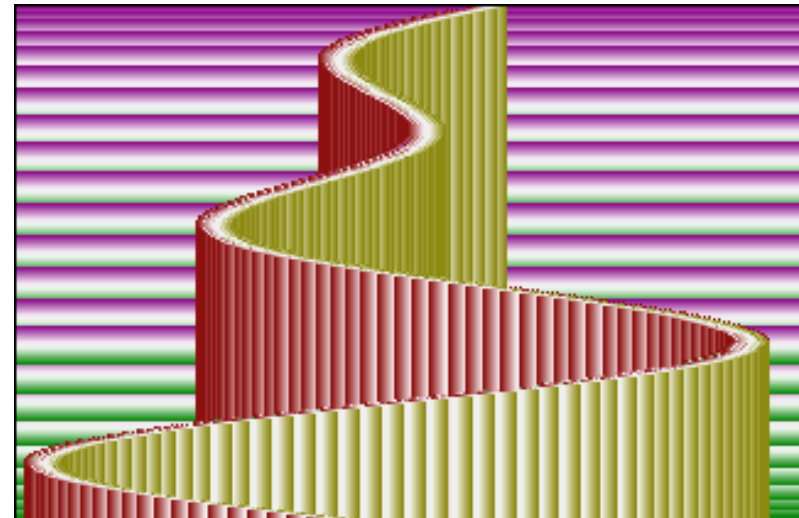
# Color Display – Color Depth

image descriptor = {  
 image width = 640  
 image height = 480  
 image depth = 24 bit  
 color model = RGB  
 encoding = YUV 8:2:2, JPEG}



# Image Representations

- Number of channels — dim. of color model plus **alpha channels**
  - e.g. 32 bit color depth = 24 bit color (8/8/8) + 8 bit alpha channel for transparency
- Channel depth — number of bits-per-pixel
- Alpha — controls transparency; masks and blends
- Pixel aspect ratio (usually square pixels nowadays)
- Interlacing (important for video)
- Compression — lossless vs. lossy
- Indexing — colors are represented by index in color maps or color lookup tables [CLUT]; map predefined or part of image







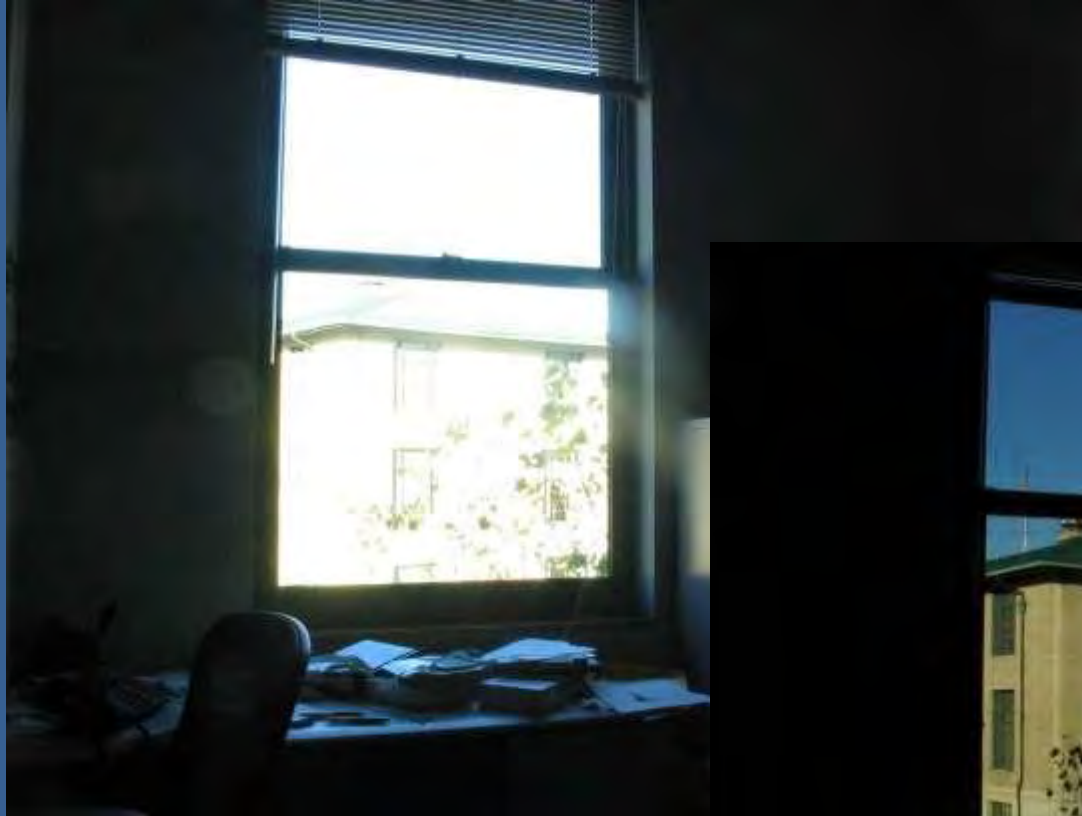
# HIGH DYNAMIC RANGE DEMYSTIFIED

Scott Janus, Principal Engineer

Slides adapted from Scott Janus, Principal Engineer at Intel; Thuong Nguyen [tnguyen@humaxdigital.com](mailto:tnguyen@humaxdigital.com); Prof. Marc Pollefeys and Dr. Gabriel Brostow at ETHZ

# Problem of Dynamic Range

## The Grandma Problem



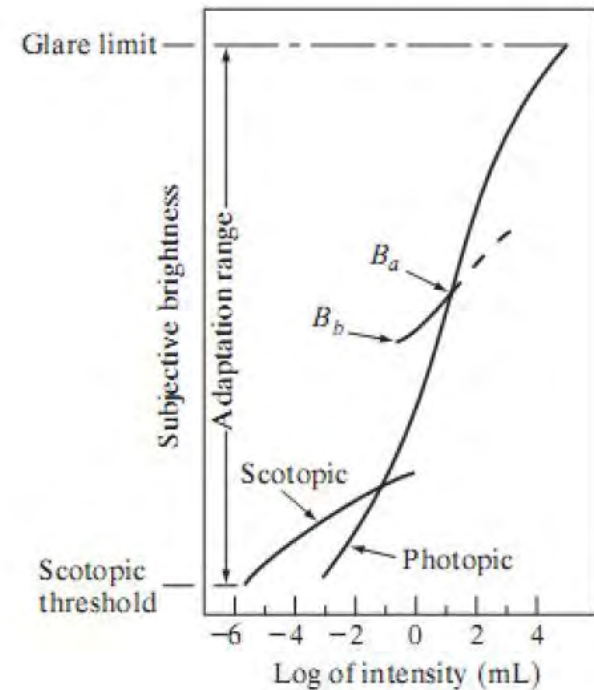
**Dynamic Range: Ratio of largest to smallest signal in a system**





# What is High Dynamic Range?

- Display does not have enough brightness to show the whole range.
- High Dynamic Range:  
Content with a **wider range of brightness and color**
  - Also an increased number of brightness and color levels
- Experiencing HDR content requires new displays!





# Color is complicated



“The Dress”

[https://en.wikipedia.org/wiki/The\\_dress](https://en.wikipedia.org/wiki/The_dress)

White and gold

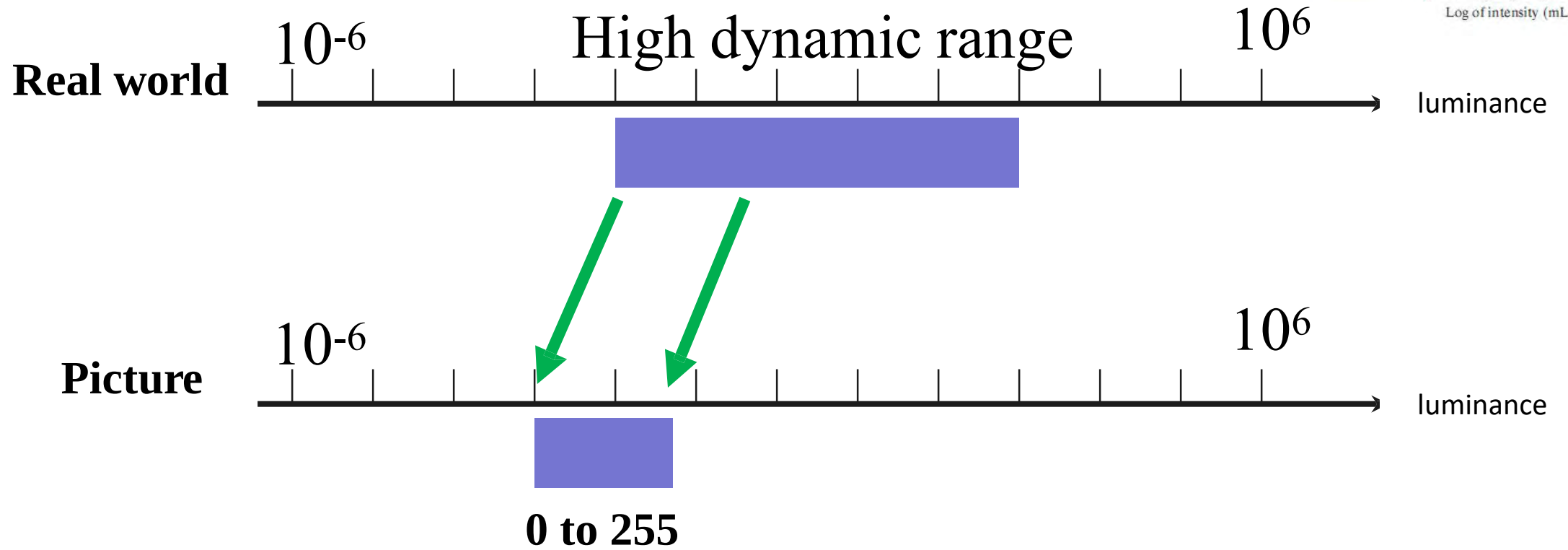
Or

Black and blue?

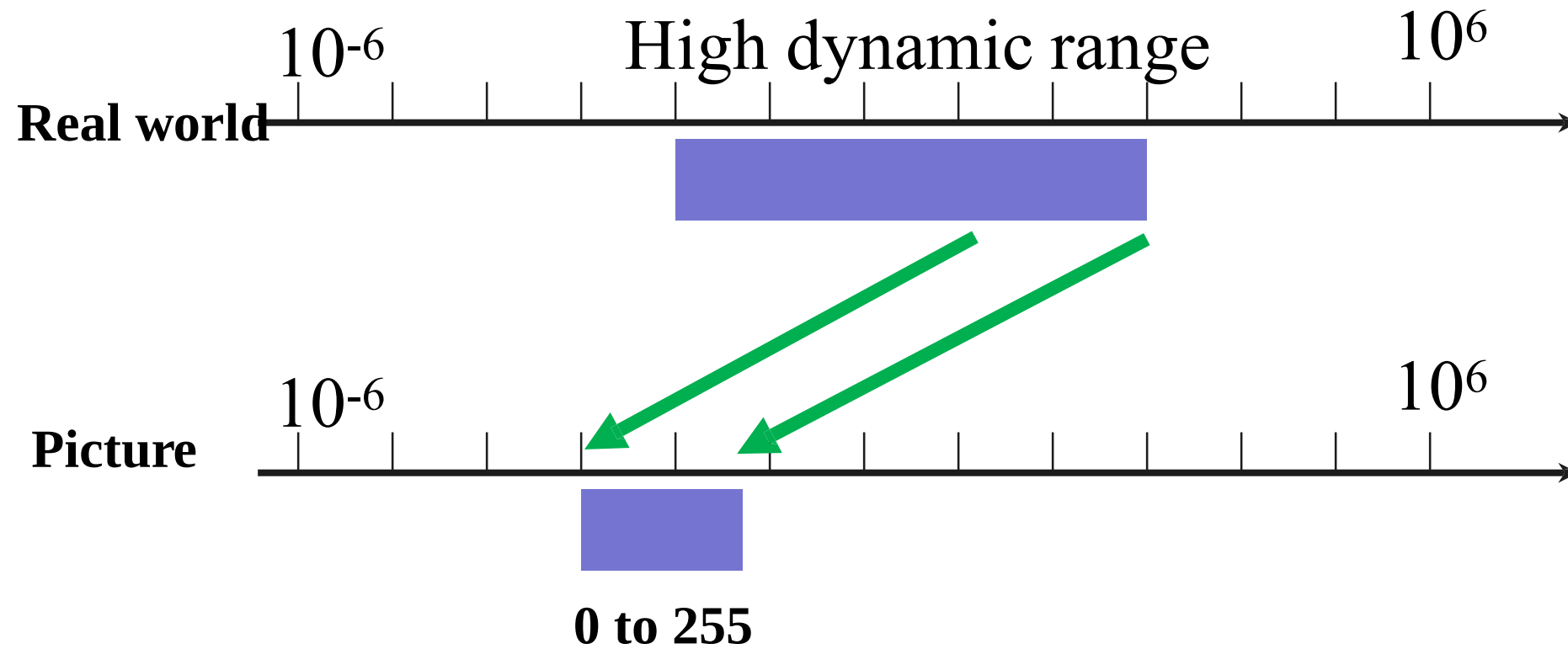


The real dress is  
Blue with black stripes

# Long Exposure

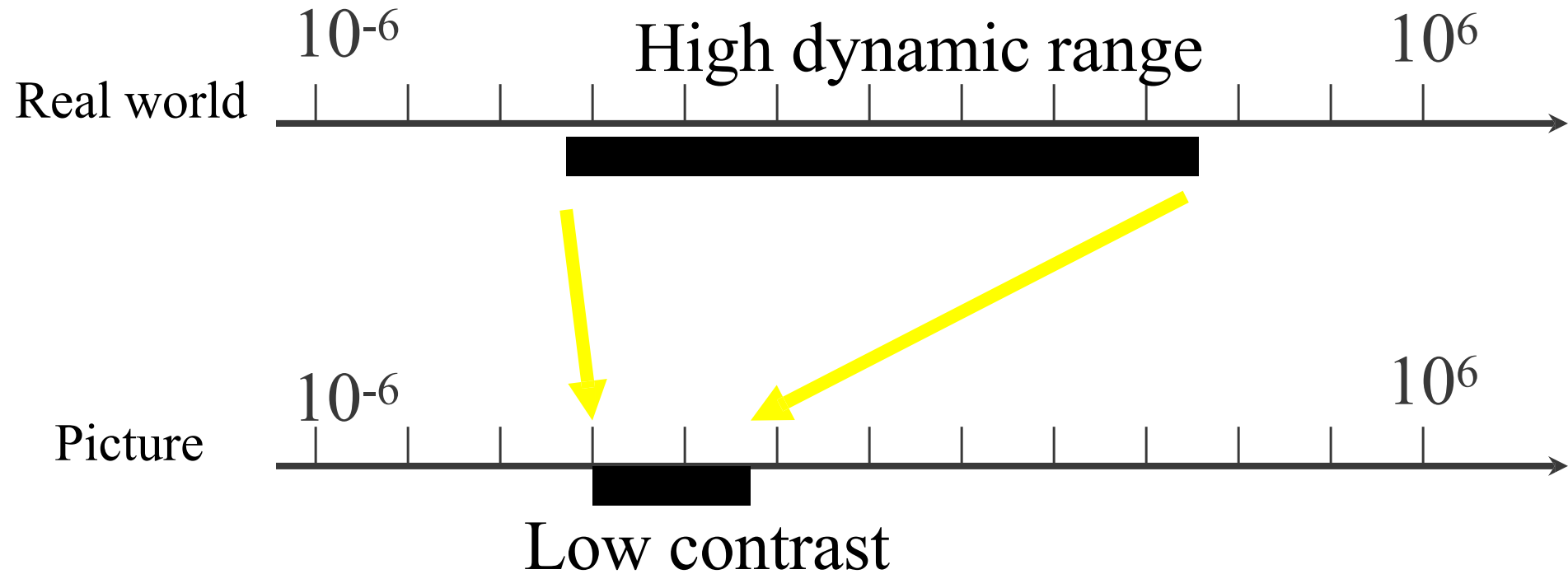


# Short Exposure



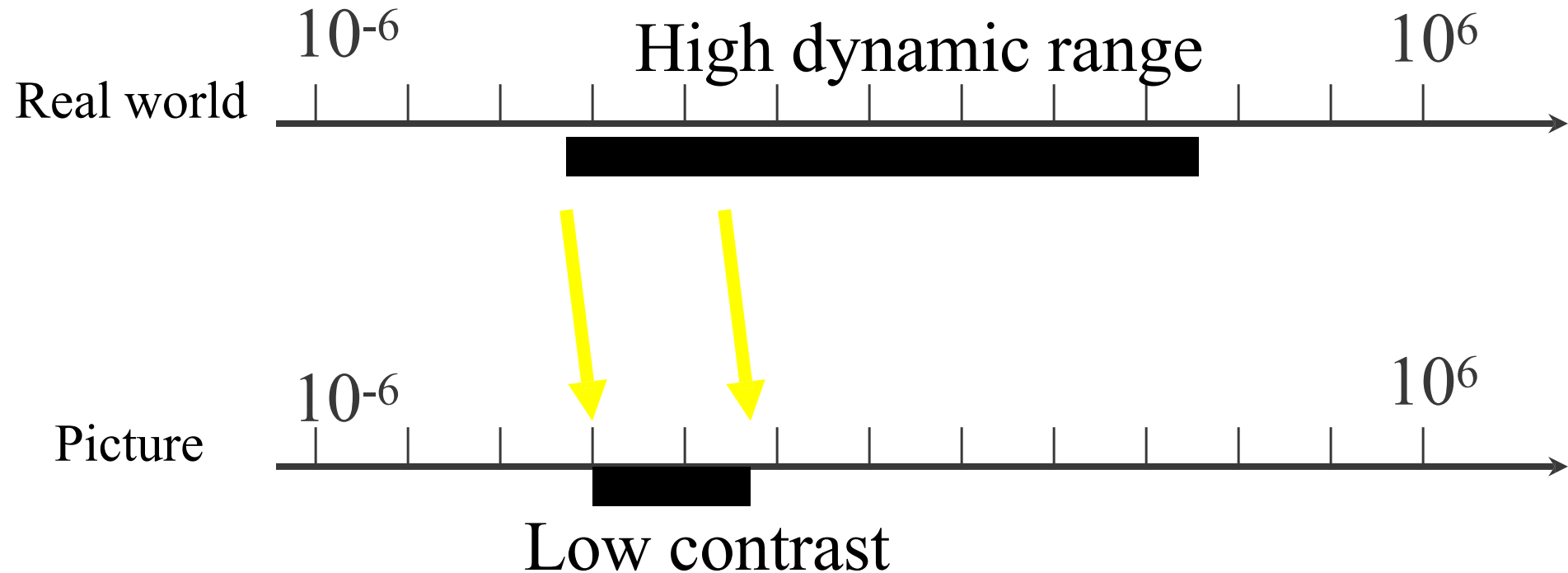
# Multiple exposure photography

- **Sequentially measure all segments of the range**



# Multiple exposure photography

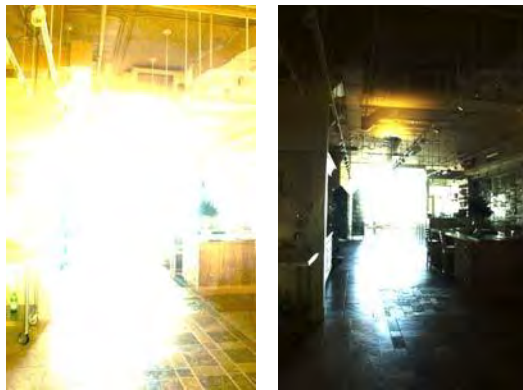
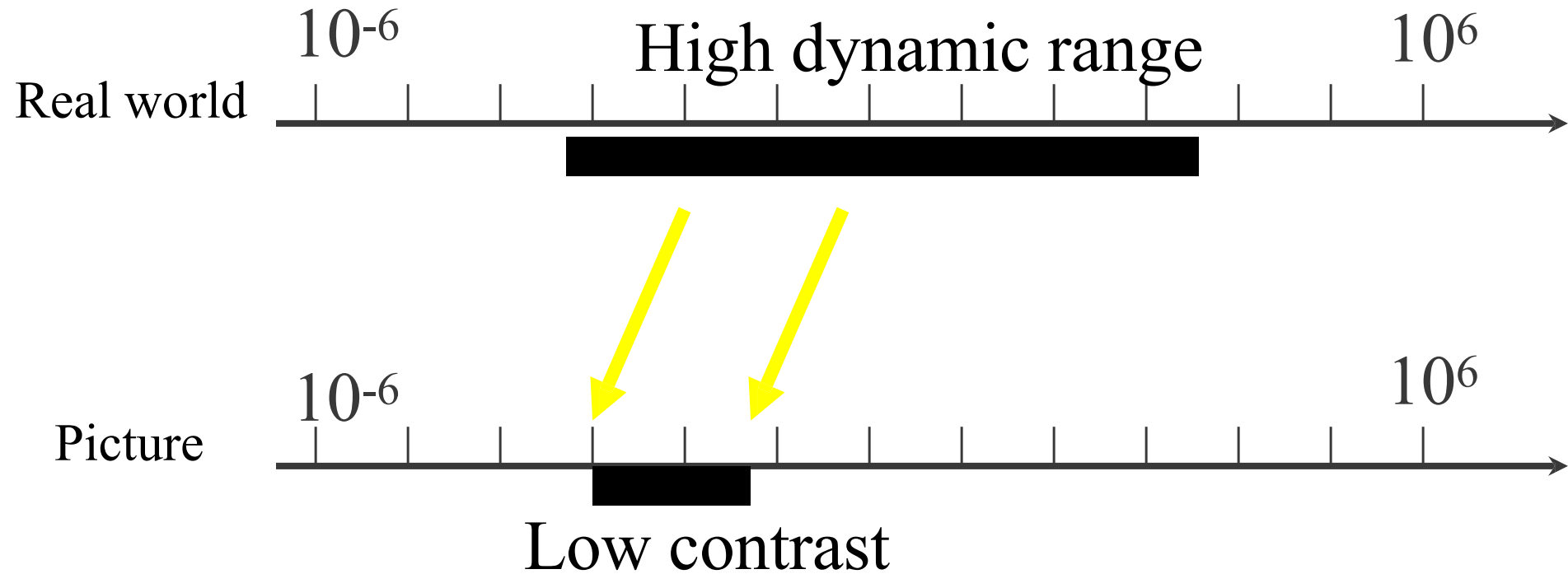
- **Sequentially measure all segments of the range**





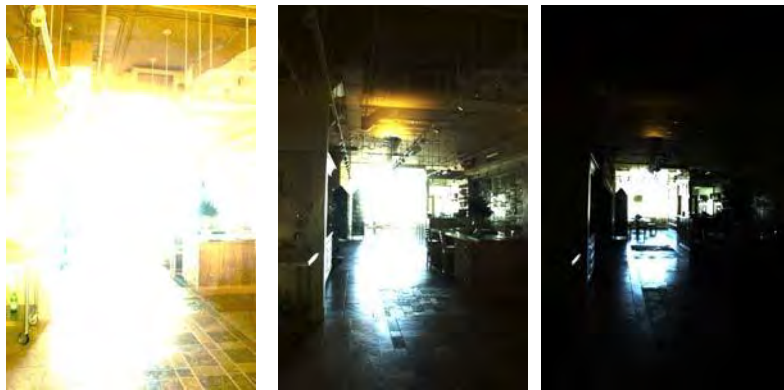
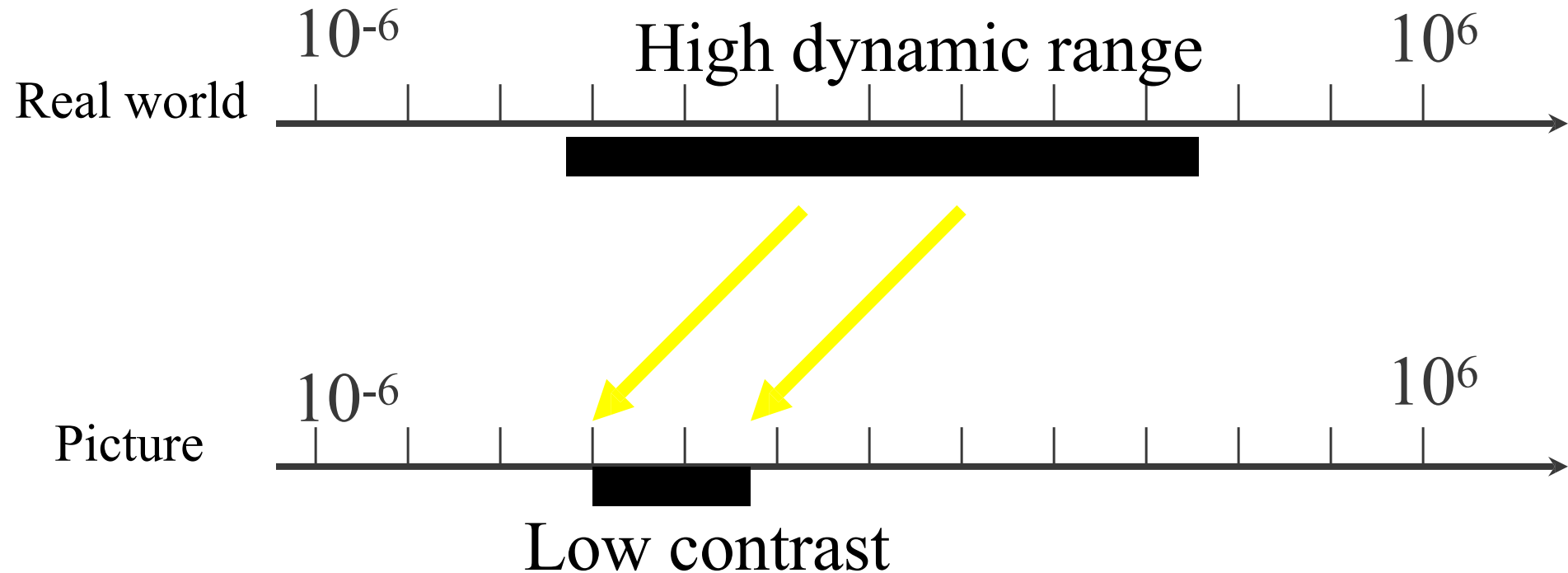
# Multiple exposure photography

- **Sequentially measure all segments of the range**



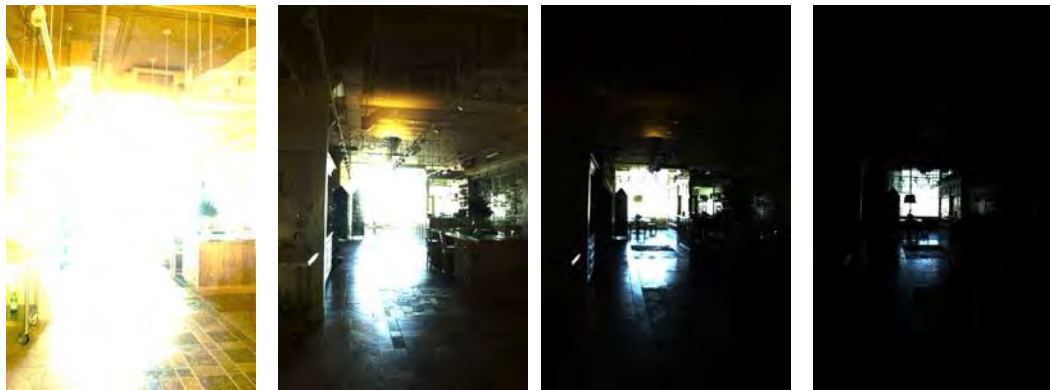
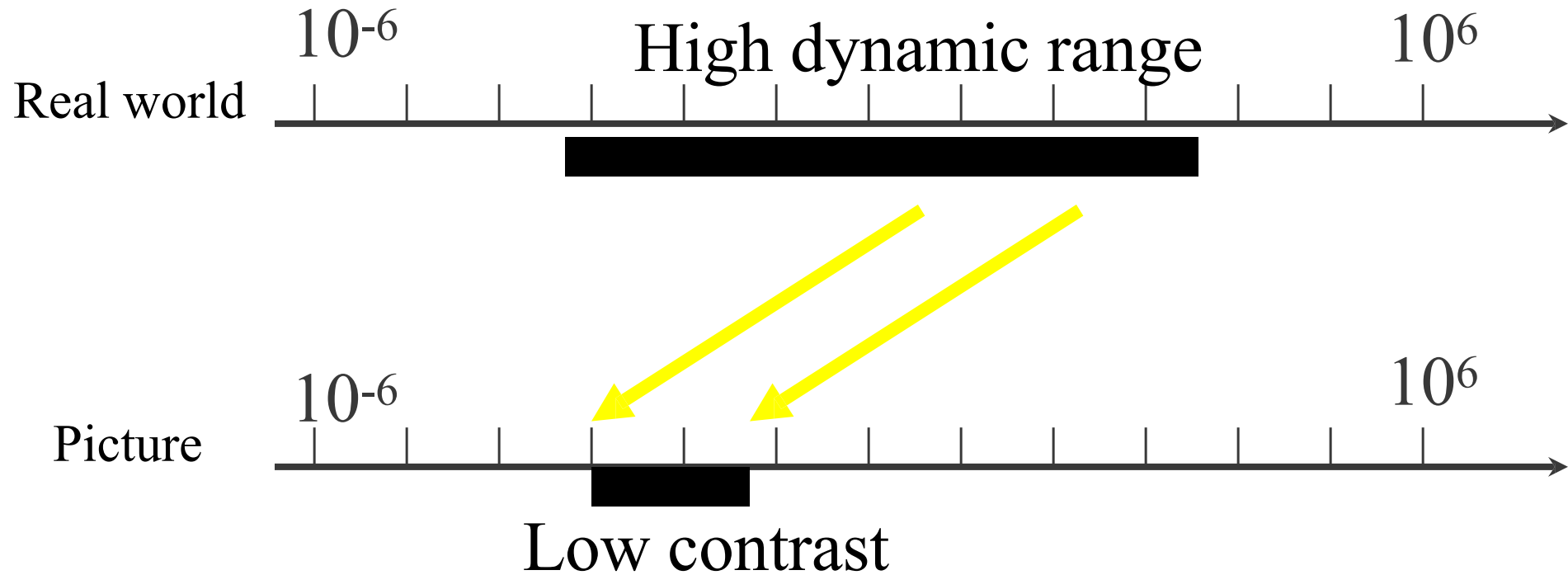
# Multiple exposure photography

- **Sequentially measure all segments of the range**



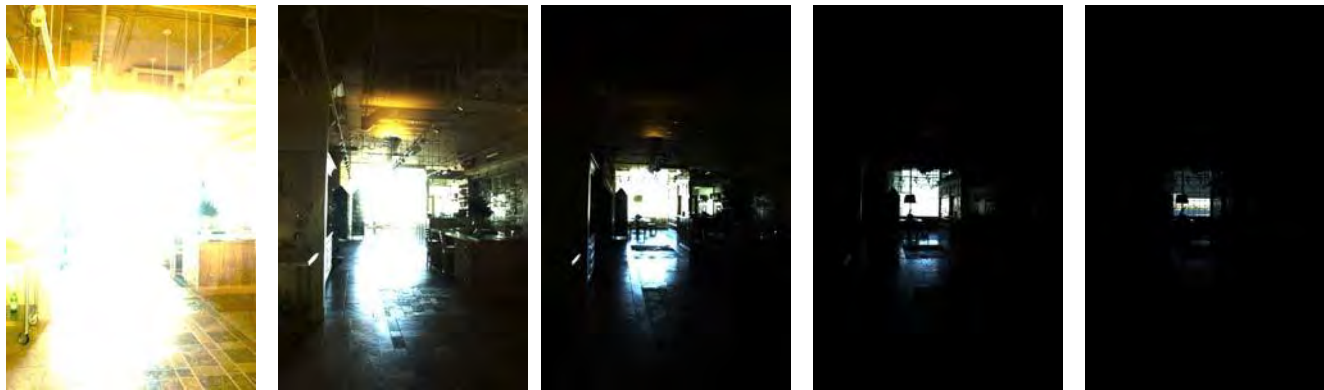
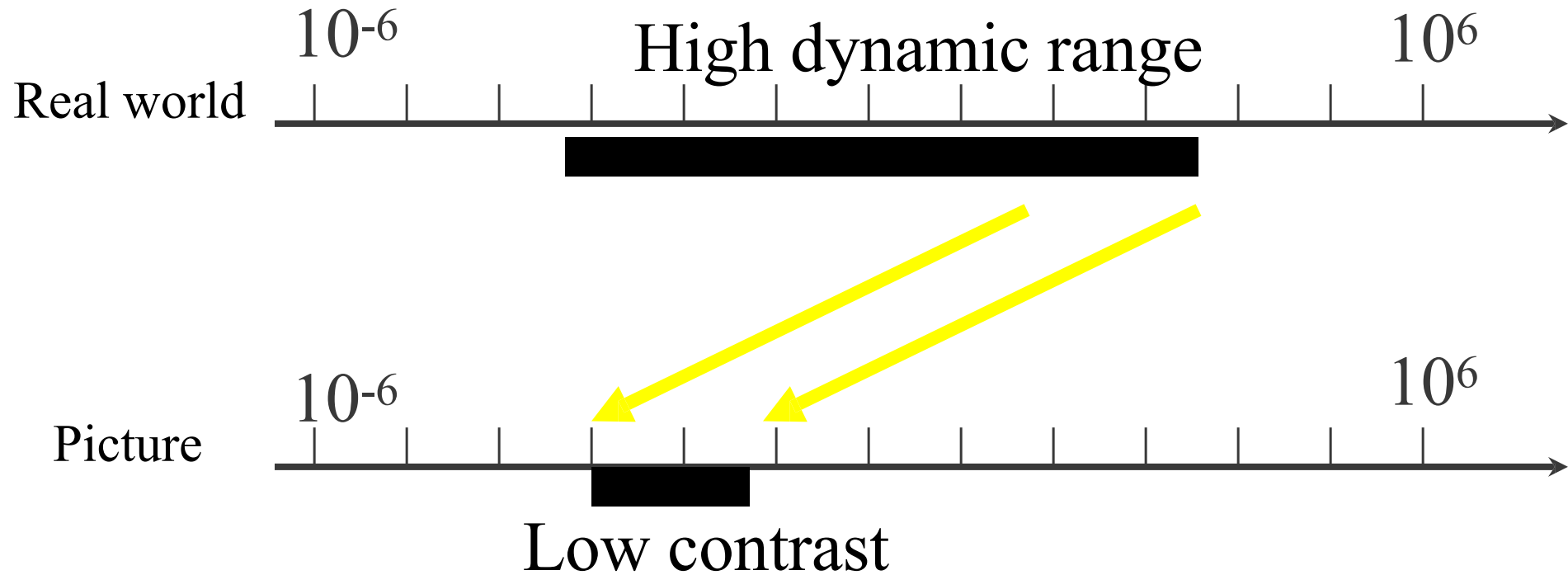
# Multiple exposure photography

- **Sequentially measure all segments of the range**



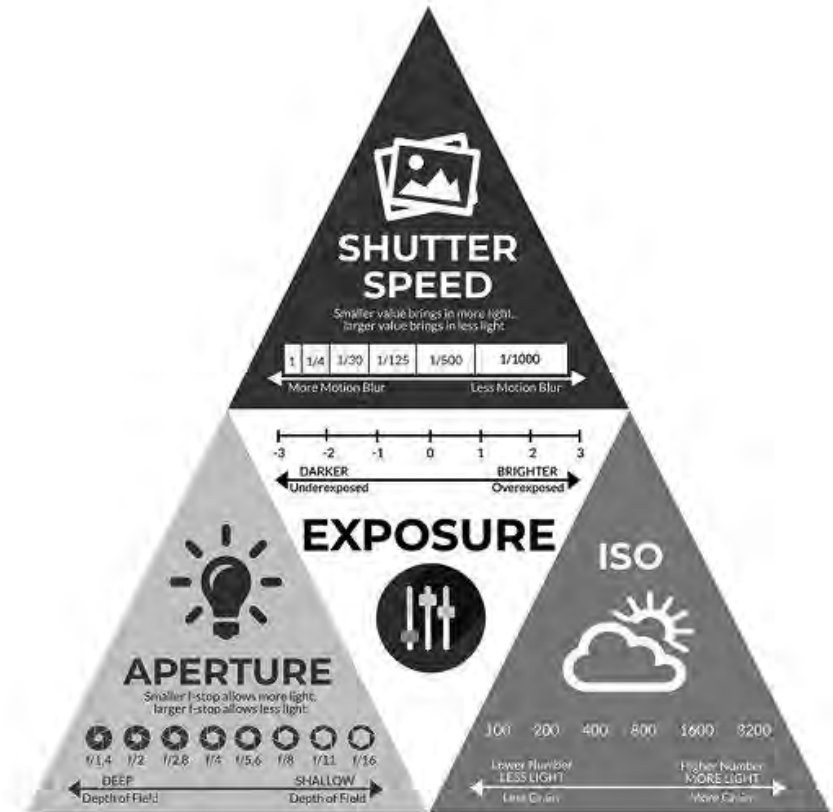
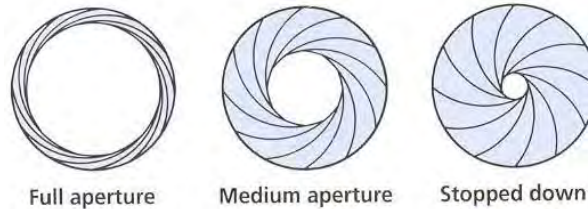
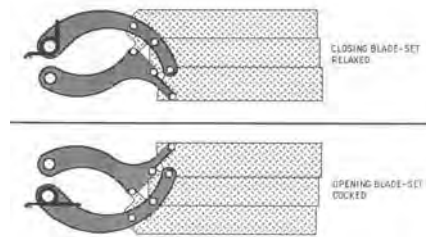
# Multiple exposure photography

- **Sequentially measure all segments of the range**



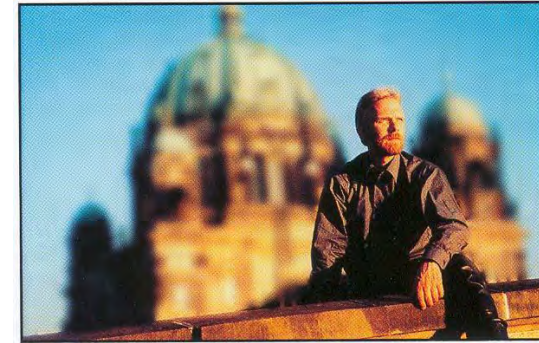
# How do we vary exposure?

- Options:
  - Shutter speed
  - Aperture
  - Light Sensitivity (ISO)
  - [Neutral density filter]

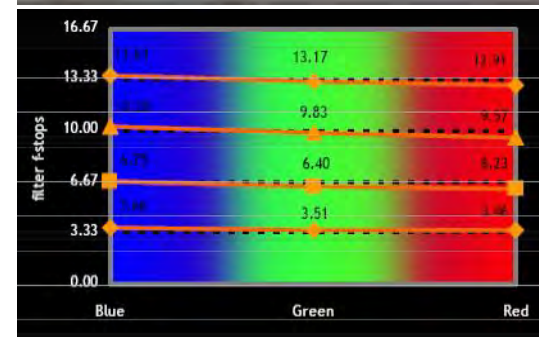


# Tradeoffs

- **Shutter speed**
  - Range: ~30 sec to 1/4000sec (6 orders of magnitude)
  - Pros: reliable, linear
  - Cons: sometimes noise for long exposure
- **Aperture**
  - Range: ~f/1.4 to f/22 (2.5 orders of magnitude)
  - Cons: changes depth of field
- **Light Sensitivity (ISO)**
  - Range: ~100 to 1600 (1.5 orders of magnitude)
  - Cons: noise
- **Neutral density filter**
  - Range: up to 4 densities (4 orders of magnitude) & can be stacked
  - Cons: not perfectly neutral (color shift), not very precise, need to touch camera (shake)
  - Pros: works with strobe/flash, good complement when desperate



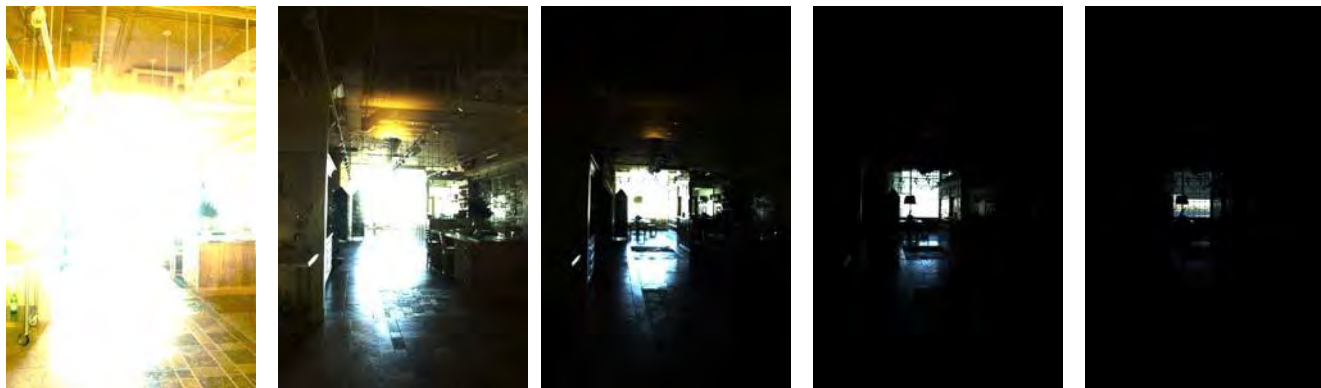
Nikon D2X  
ISO 3200





# HDR image using multiple exposure

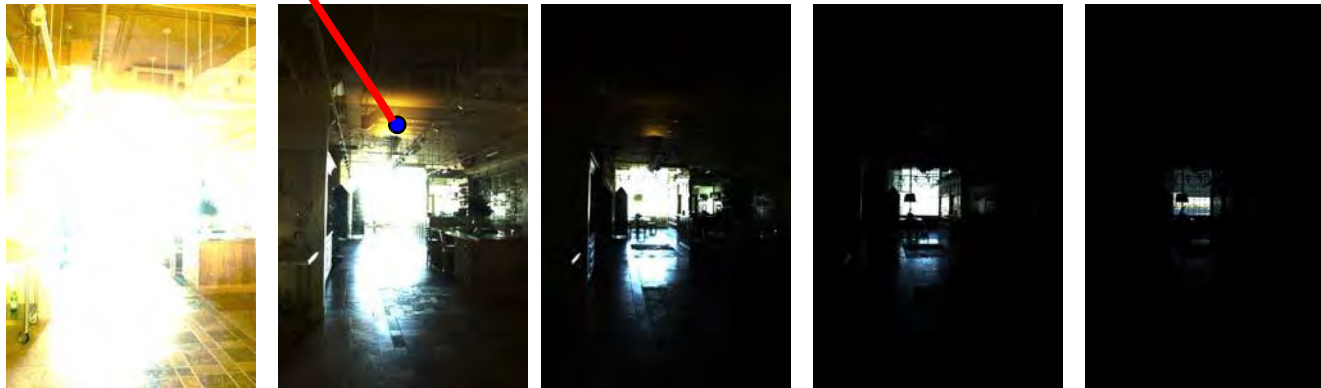
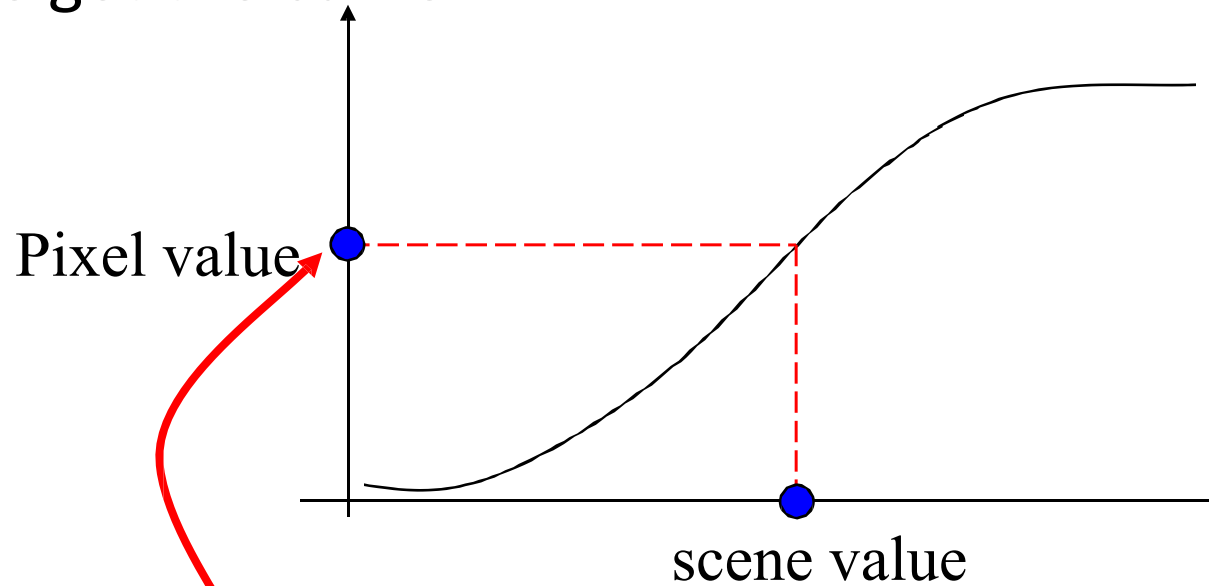
- Given  $N$  photos at different exposure
- Recover a HDR color for each pixel



# If we know the response curve

- Just look up the inverse of the response curve
- But how do we get the curve?

*A response curve is the relation between the input and output of a sensor.*



# Calibrating the response curve

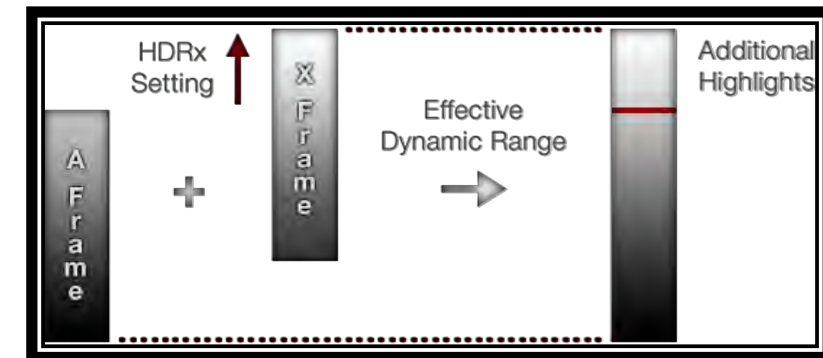
- **Two basic solutions**

- Vary scene luminance and see pixel values
  - Assumes we control and know scene luminance
- ***Vary exposure and see pixel value for one scene luminance***



- **Best of both:**

- **Vary exposure**
- Exploit the large number of pixels

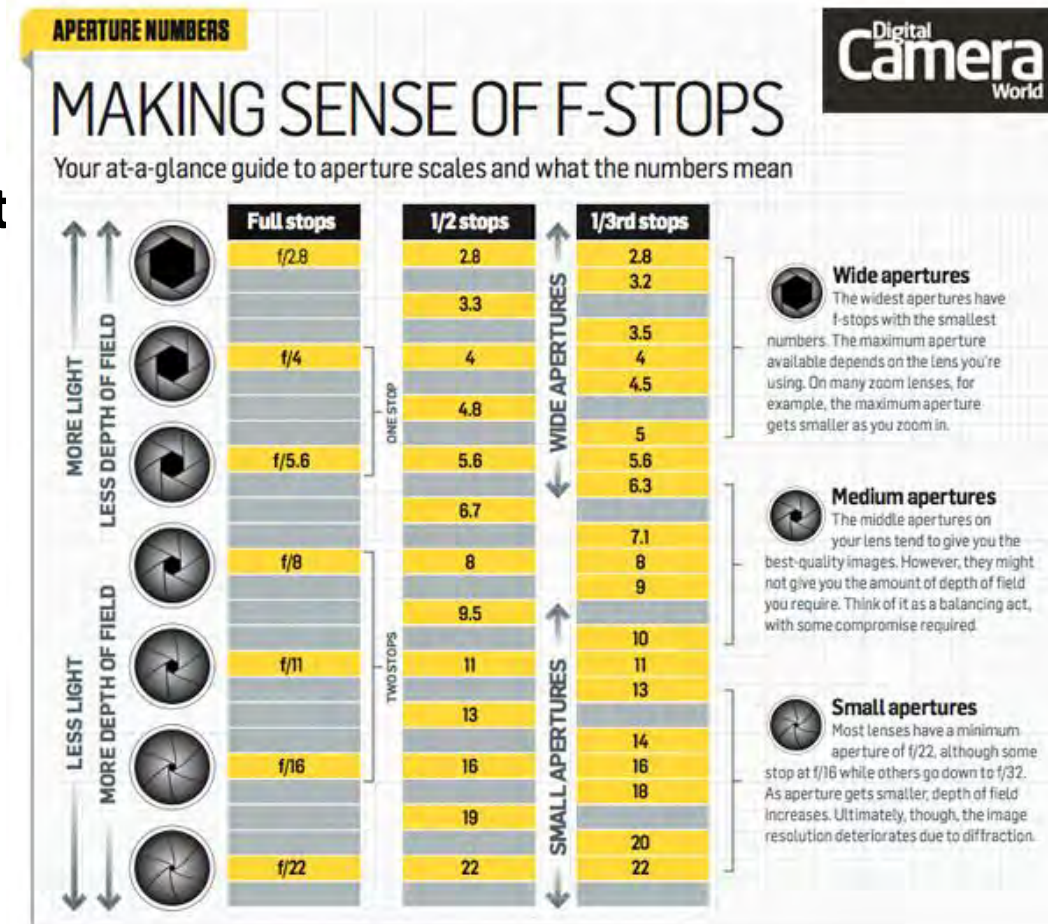


Explanation of the HDRx mode (RED film cameras) – 2 exposures captured:  
<https://www.red.com/red-101/hdrx-high-dynamic-range-video>

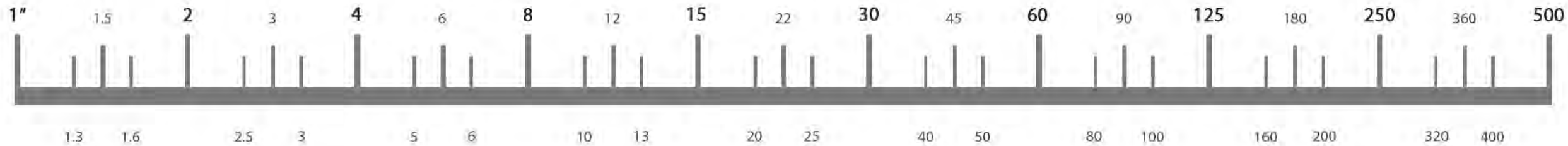
# F-stop number (Aperture Stop)

- Each stop is **twice as bright as the previous one**
- Here's the aperture scale. Each step down lets in **half** as much light:
- These are the main aperture “stops,” but most cameras and lenses today let you set some values in between, such as f/1.8 or f/3.5.
- When a camera is said to have **12 stops of dynamic range**, it means that the **brightest part of a scene can be  $2^{12}$  times brighter than the darkest part.**

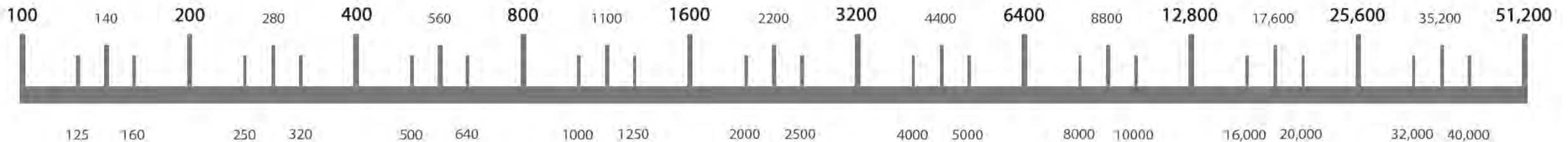
| f/1.4                          | f/2.0               | f/2.8               | f/4.0                          | f/5.6                                         | f/8.0                           | f/11.0               | f/16.0               | f/22.0                                                        |
|--------------------------------|---------------------|---------------------|--------------------------------|-----------------------------------------------|---------------------------------|----------------------|----------------------|---------------------------------------------------------------|
| Very large aperture            | Large aperture      | Large aperture      | Moderate aperture              | Moderate aperture                             | Moderate aperture               | Small aperture       | Small aperture       | Very small aperture                                           |
| Lets in a huge amount of light | Half as much light  | Half as much light  | Half as much light             | Half as much light (a very “medium” aperture) | Half as much light              | Half as much light   | Half as much light   | Half as much light (by which point your photos are very dark) |
| Very thin depth of field       | Thin depth of field | Thin depth of field | Moderately thin depth of field | Moderate depth of field                       | Moderately large depth of field | Large depth of field | Large depth of field | Very large depth of field                                     |



# Shutter Speed & ISO Stops



A shutter speed range of 10 stops with values from one second all the way to 1/500th of a second



Calculating ISO sensitivity stops is very easy – one just needs to divide or multiply the value by 2 to add a stop or go one stop down.

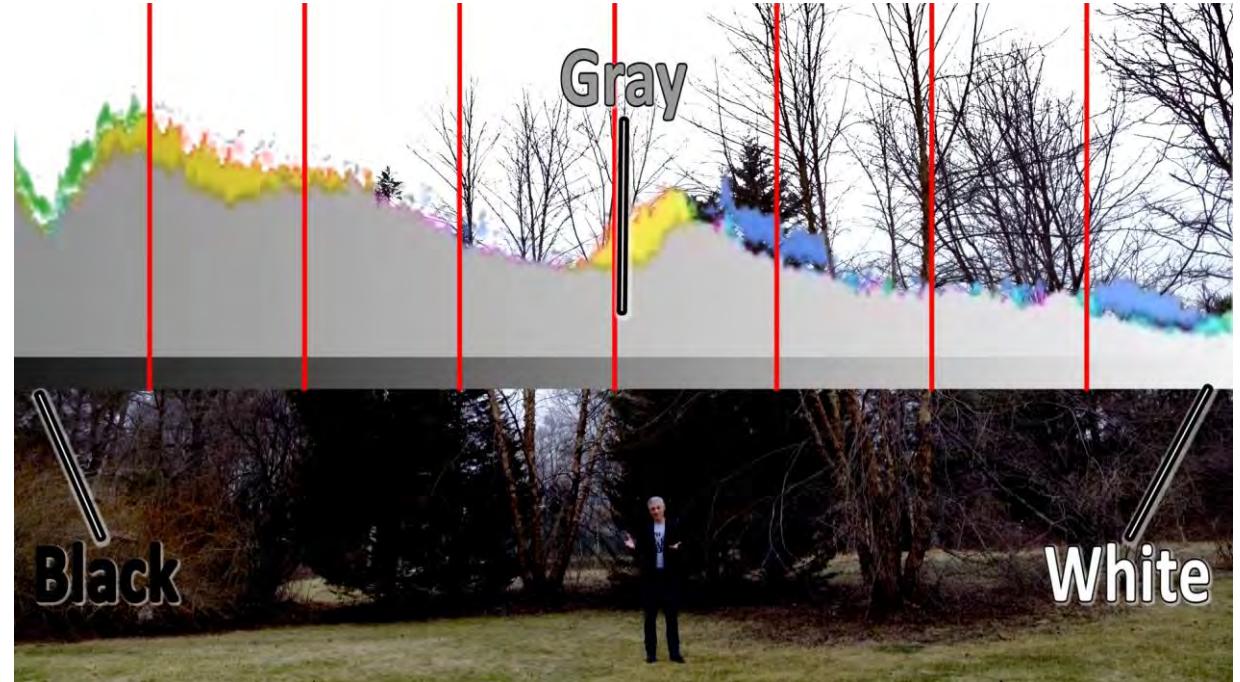
PLEASE NOTE: A digital sensor does not and cannot change its sensitivity to light. There is no setting whatsoever on a digital camera, including ISO, that can modify a sensor's sensitivity to light. ISO in digital is a level of post-capture analog amplification that occurs.



# Dynamic Range Explanations & Examples



<https://www.youtube.com/watch?v=SrXwiPH-fCg>



<https://www.youtube.com/watch?v=MfvwqcmM6FQ>

Very good detailed explanation of a film maker about Dynamic Range:

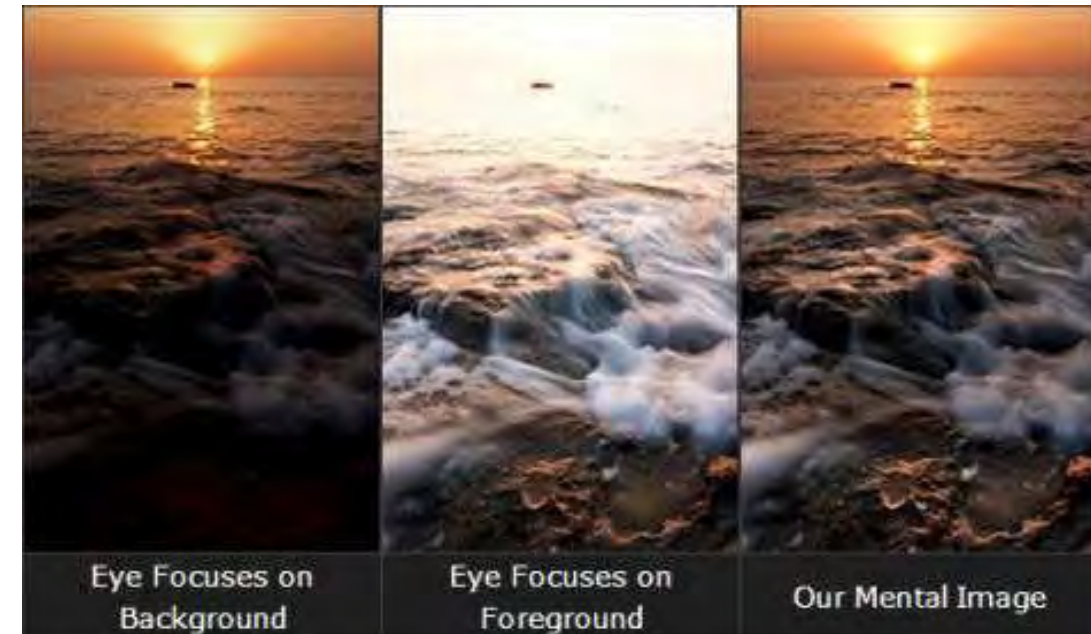
<https://www.youtube.com/watch?v=2sshGdMgJxQ>



# The Human Eye



- Human eye uses the pupil size to see darker or brighter images
  - Turn smaller to limit incoming light to see brighter scenes
  - But cannot see darker & brighter region at same time
  - Can detect a luminance range of  $10^{14}$ , or one hundred trillion (100,000,000,000,000) (**about 46.5 f-stops**), from  $10^{-6}$  cd/m<sup>2</sup>, or one millionth (0.000001 cd/m<sup>2</sup>) to  $10^8$  cd/m<sup>2</sup> or one hundred million (100,000,000 cd/m<sup>2</sup>)



# Stops - Contrast

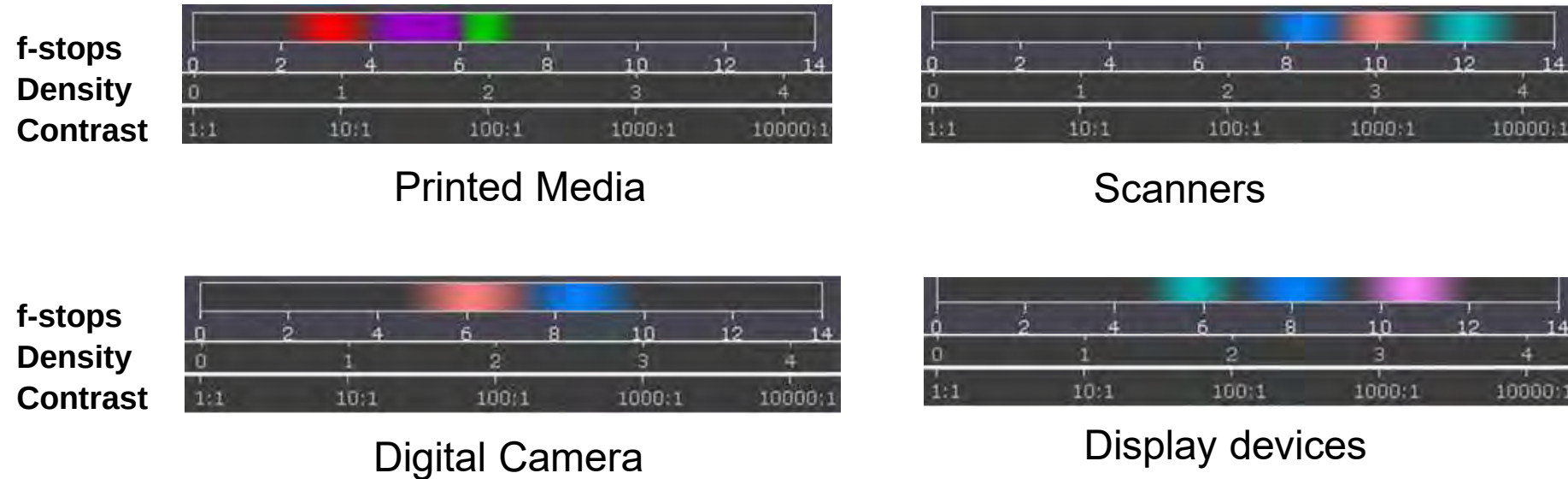
Dynamic ranges of common devices

| Device                                | Stops                                       | Contrast ratio         |
|---------------------------------------|---------------------------------------------|------------------------|
| Glossy photograph paper               | 7 (7 - 7 2/3 ) <sup>[58]</sup>              | 128:1                  |
| LCD                                   | 9.5 (8 – 10.8) <sup>[citation needed]</sup> | 700:1 (250:1 – 1750:1) |
| Negative film (Kodak VISION3)         | 13 <sup>[59]</sup>                          | 8000:1                 |
| Human eye                             | 10–14 <sup>[54]</sup>                       | 1000:1 – 16000:1       |
| High-end DSLR camera (Nikon D850)     | 14.8 <sup>[60]</sup>                        | 28500:1                |
| Digital cinema camera (Red Weapon 8k) | 16.5+ <sup>[61]</sup>                       | 92000:1                |

Without adaptation

# Contrast Comparison

- Dynamic range is commonly measured on a logarithmic scale



Optische Dichte  $d$  (Density): dekadischer Logarithmus der Lichtleistung

# Bit Depth & Dynamic Range

- (bit per pixel) Quantifies how many unique colors are available
  - How many unique shades are available in grayscale image

| Bit Precision<br>of Analog/Digital<br>Converter | Contrast Ratio | Dynamic Range |         |
|-------------------------------------------------|----------------|---------------|---------|
|                                                 |                | f-stops       | Density |
| 8                                               | 256:1          | 8             | 2.4     |
| 10                                              | 1024:1         | 10            | 3.0     |
| 12                                              | 4096:1         | 12            | 3.6     |
| 14                                              | 16384:1        | 14            | 4.2     |
| 16                                              | 65536:1        | 16            | 4.8     |

- Most digital cameras use a 10 – 14 bit A/D so their theoretical maximum dynamic range is 10-14 f-stops.
- Higher precision A/D converter does not necessarily mean greater dynamic range. Total dynamic range is usually limited by noise levels.

# Measuring Luminance



## Nit ( $\text{Candela}/\text{m}^2$ )

- 1 nit ( $\text{cd}/\text{m}^2$ ) approximately equal to the amount of light from a candle dispersed over a square meter



The real world is high dynamic range



| Object                   | Approximate Luminance (nits) |
|--------------------------|------------------------------|
| Sun                      | 1,600,000,000                |
| Arc lamp                 | 150,000,000                  |
| Maximum visual tolerance | 50,000                       |
| Cloud (sunny day)        | 35,000                       |
| 2020 UHD-TV              | 800-1000                     |
| Typical computer screen  | 100-300                      |
| White paper under lamp   | 50                           |
| Night sky                | 0.001                        |
| Threshold of vision      | 0.000003                     |





500,000  
nits



200,000  
nits



40  
nits



2,000 nits

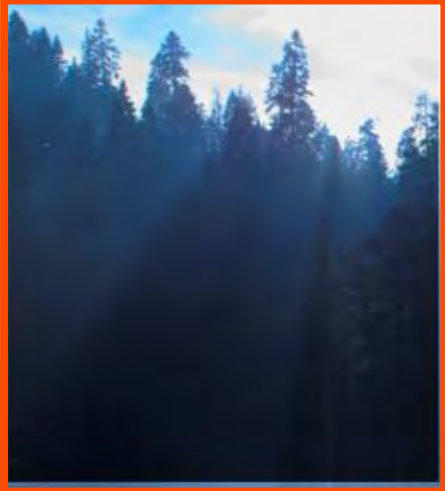
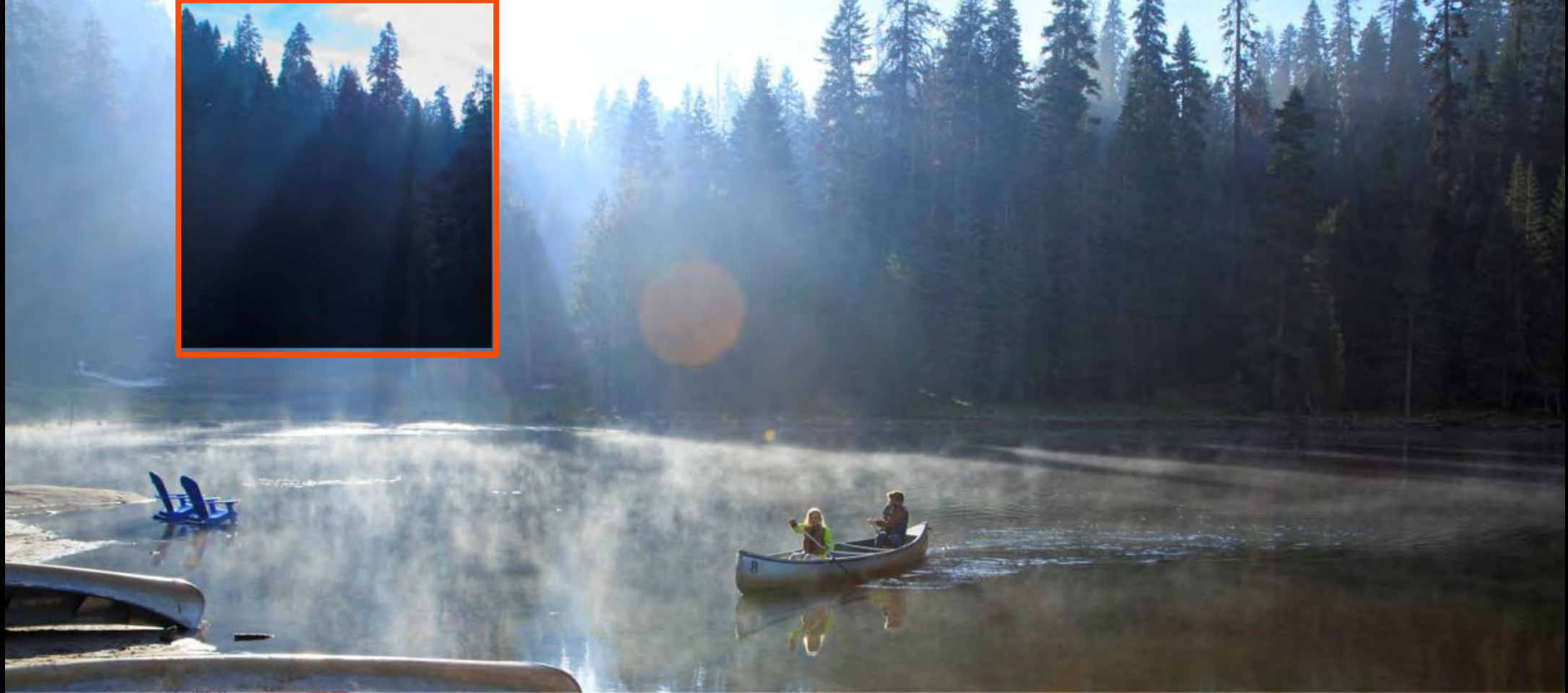














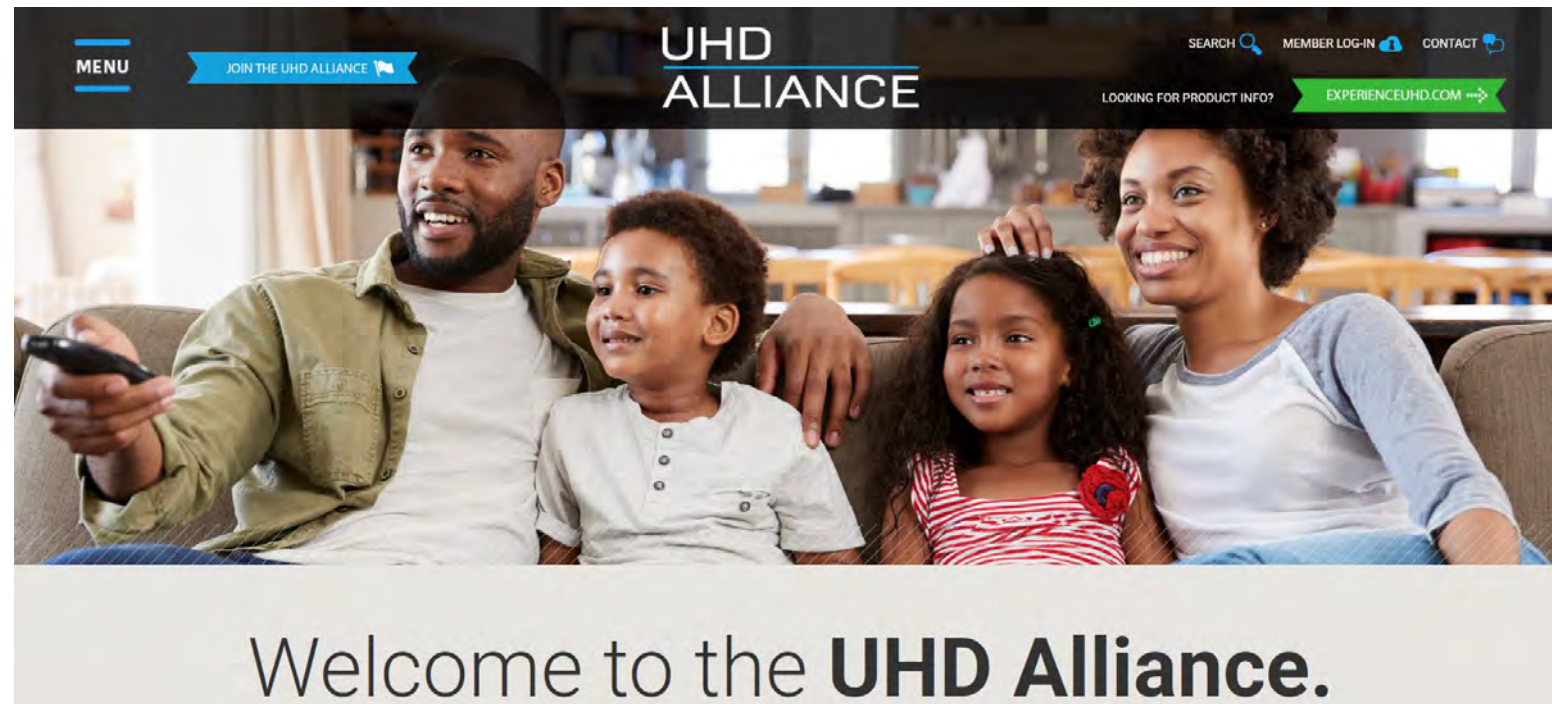
# HDR image examples





# Moving to HDR

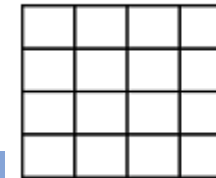
- **UHD Alliance: with mission to create (Jan. 2015 at CES)**
  - *New standard to support video technologies including **4K**, Higher resolution, **high dynamic range**, **wider color gamut**, and **3D audio***



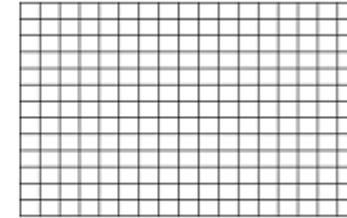


# Improving Image Quality

More pixels: SD – HD –UHD (4k, 8K)



Low resolution



High resolution

**SDTV**  
**HDTV**  
**UHDTV**



UHD vs. HD. vs SD resolution comparison

LEDTVReviews.com

**Increase spatial resolution**

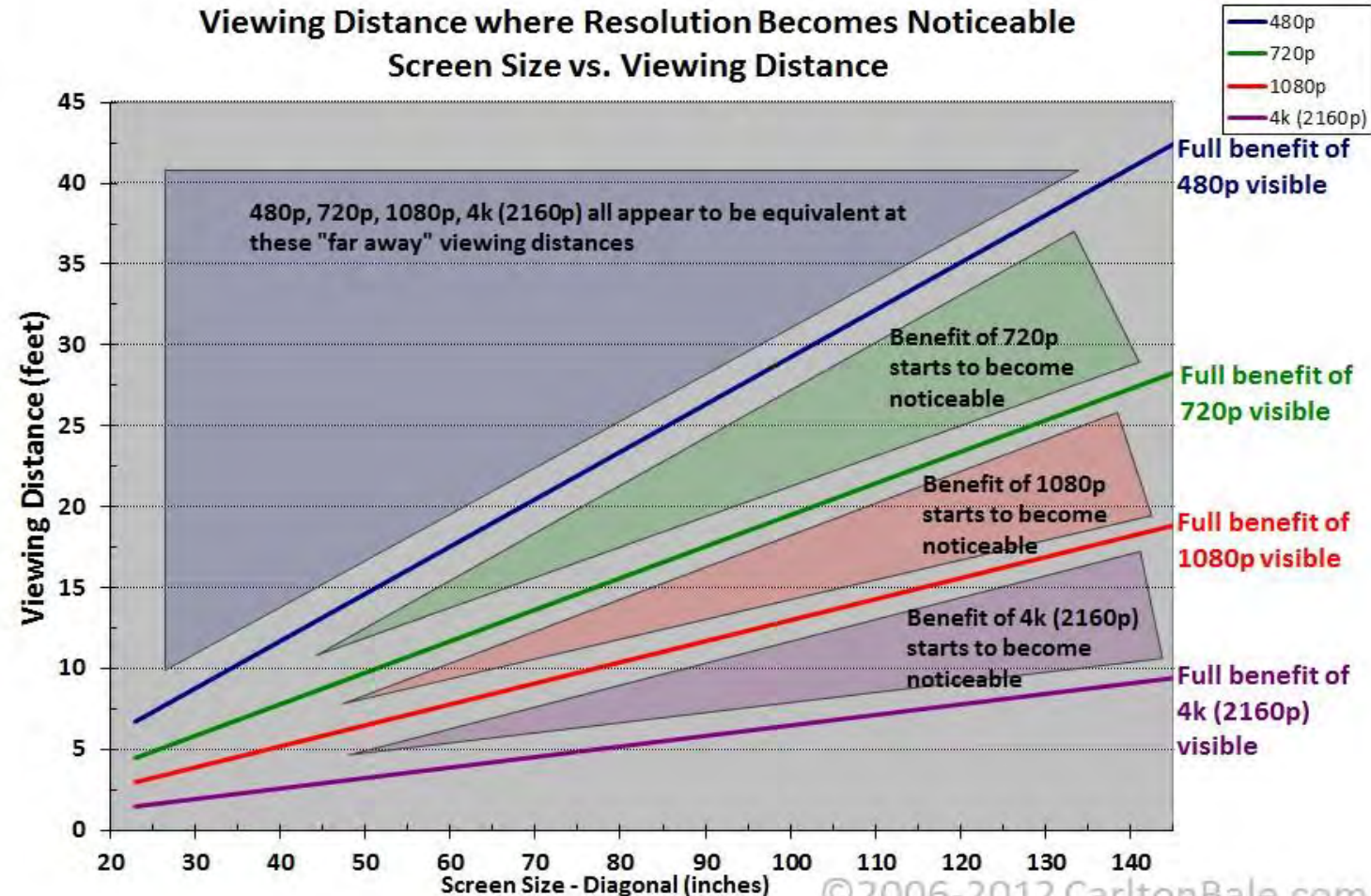
# Moving to HDR

Dramatic better  
visual quality

- (Netflix) HDR is more important for video quality than 4K
  - “We kind of ran out of more pixels to add,” Neil Hunt

Resolution is only apparent  
when you are close

**Users can notice HDR from  
any distance and screen size,  
at once**

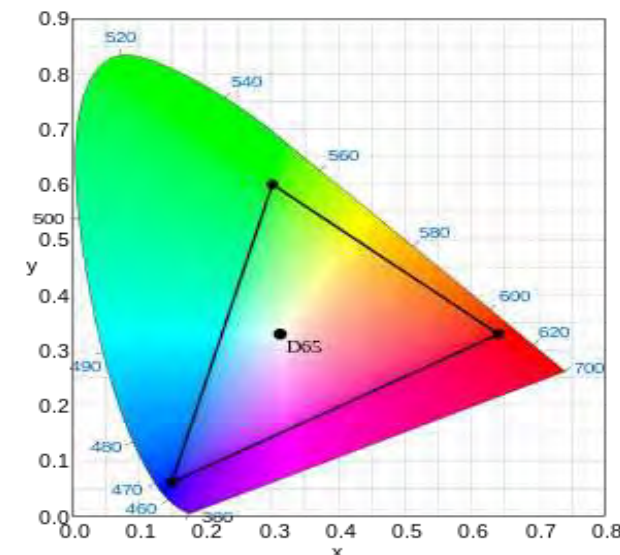


# High Dynamic Range

- Need around 10.000 nits to satisfy 90% viewer at ideal range



Current Color Space (Color Gamut)  
is not enough



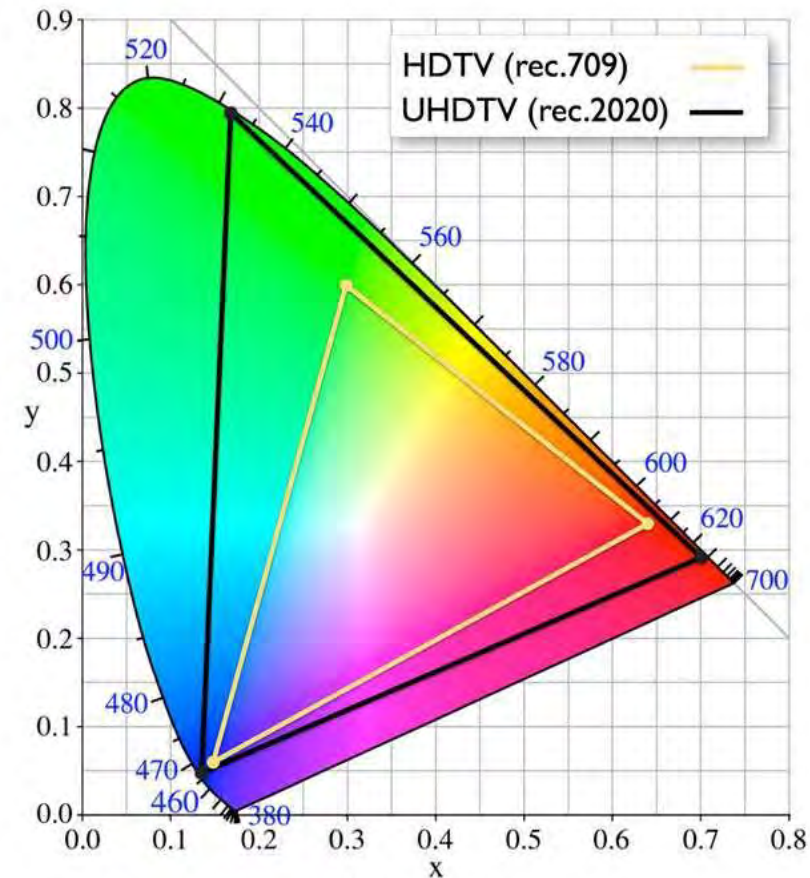


# Wider Color Gamut

- Current color gamut is limited
  - Current HDTV (BT.709) – 1990
  - Created for CRT, and HDTV
  - Around 100 nits.
  - 8 bpp (bits per pixel)

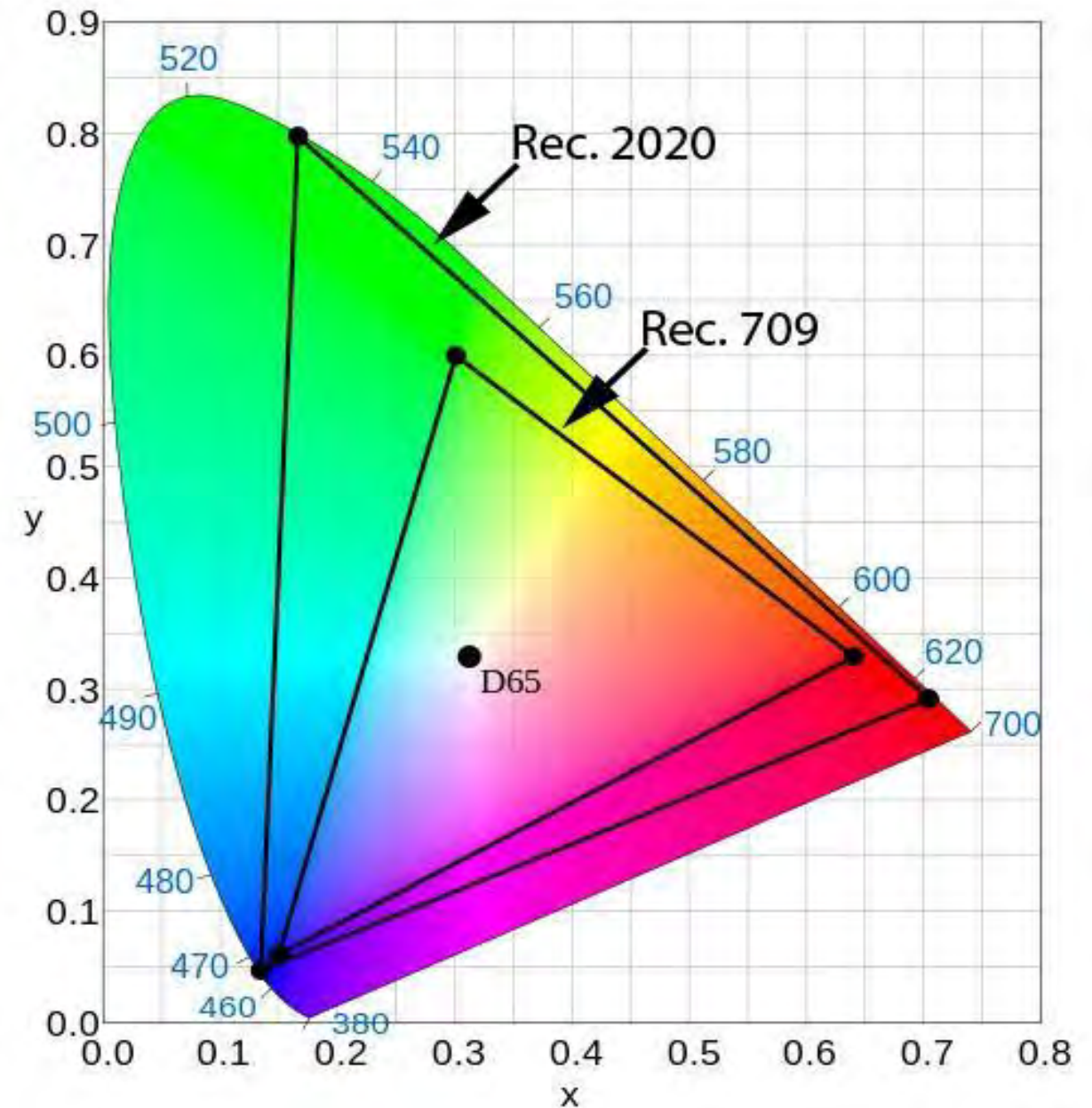
Today we have better display technology: LCD, OLED ...

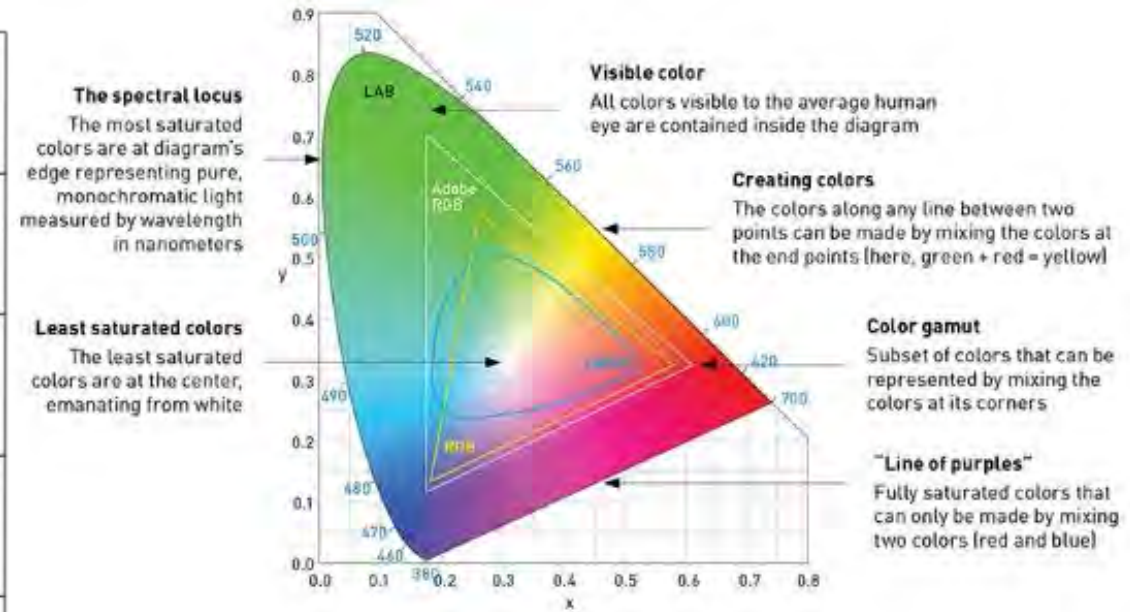
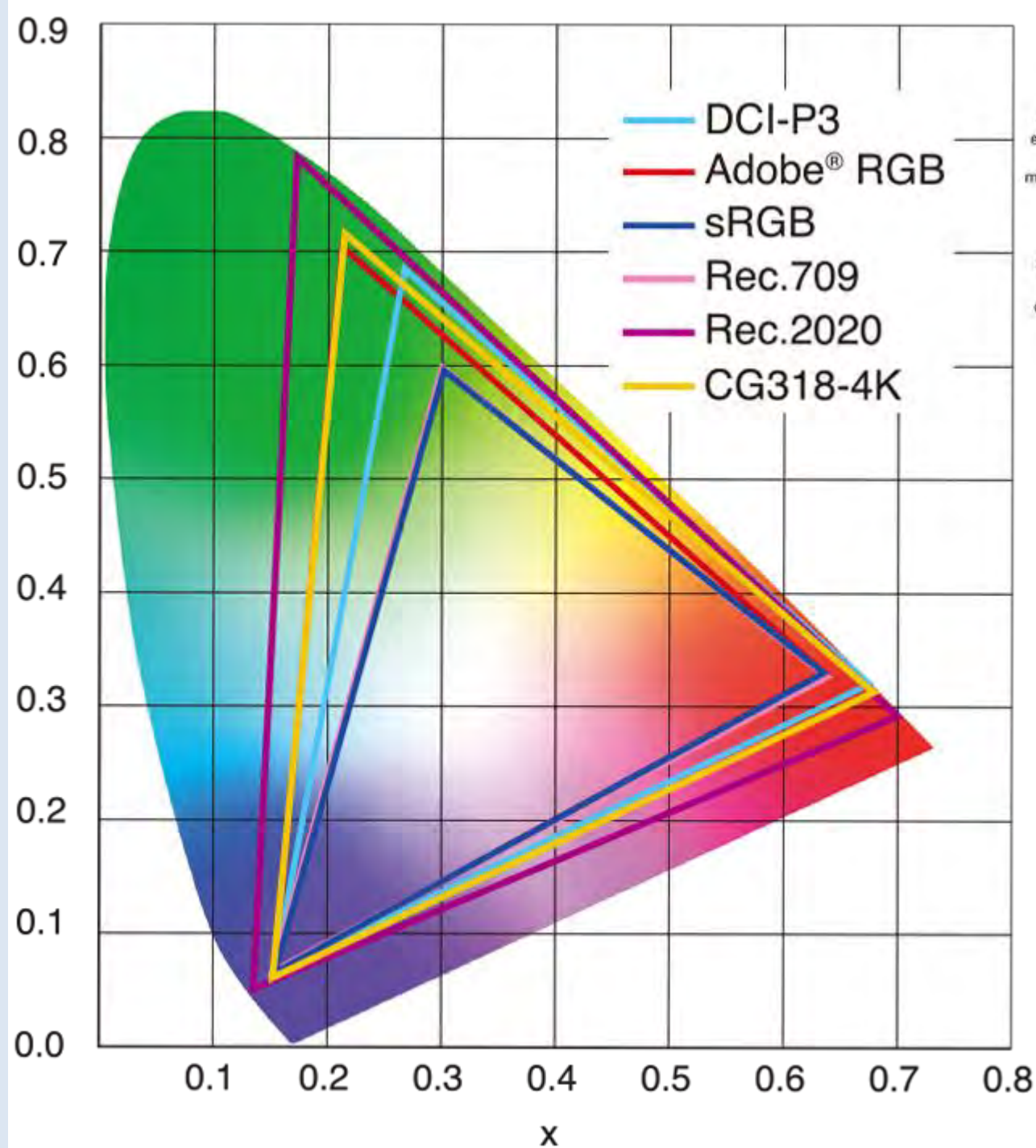
More Colors: REC. 2020 for UHDTV 10, 12 bpp.



UHDTV vs HDTV Color Gamuts

- Most PCs and HDTVs use 709 gamut
- Majority of next generation content uses Bt.2020 gamut
- 2015 UHD TVs are ~85% Bt.2020





Color profiles of video and monitors



**Bt 2020 content  
displayed unmodified  
on 709 display**



**Bt 2020 content  
gamut-mapped for  
709 display**



**We must perform gamut mapping**



# Standard Dynamic Range (SDR)

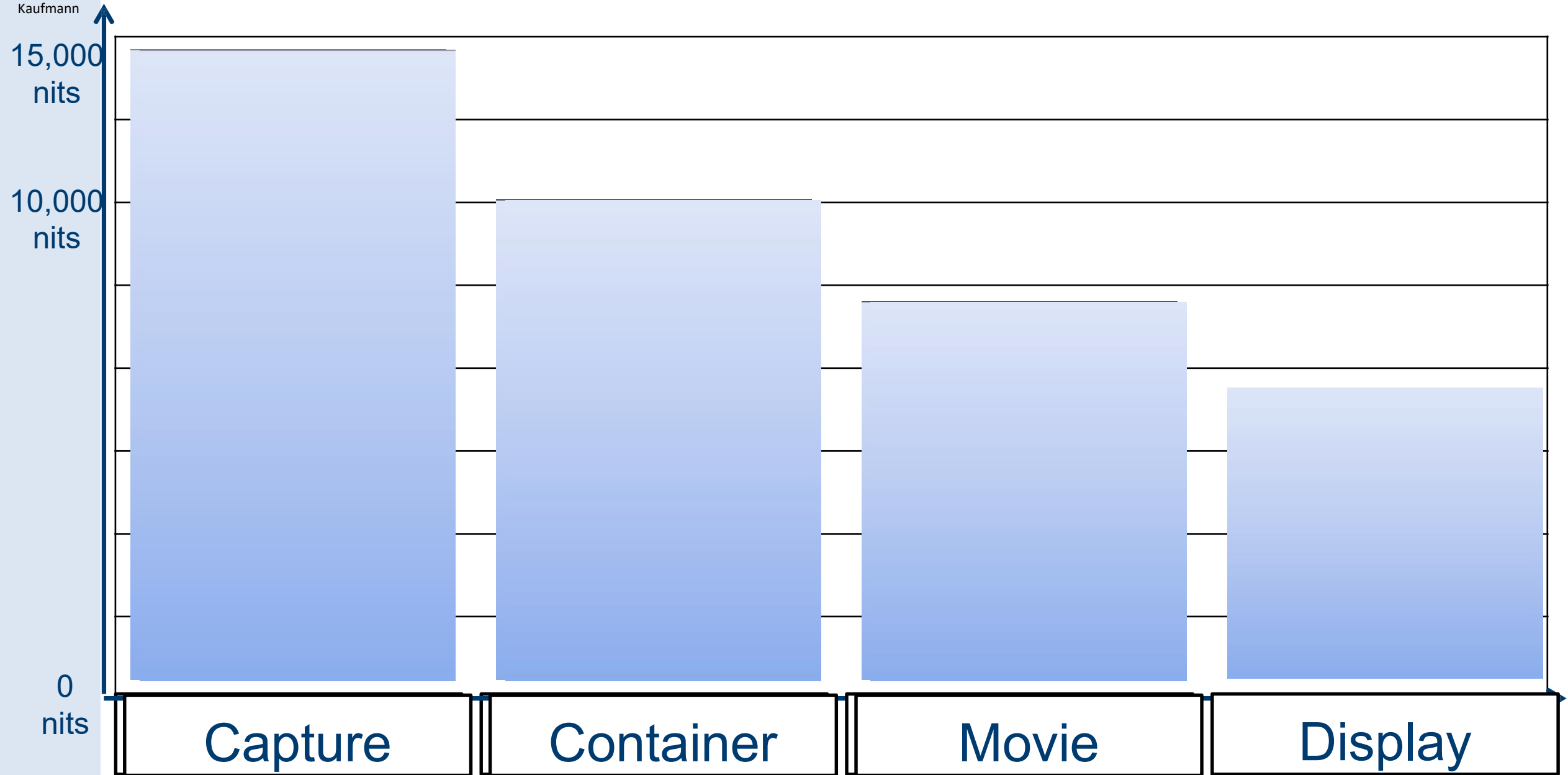
- For the past 80 years, video has been graded to appear properly on a **100 nit** display
- However, no adjustments are made to comprehend displays of differing luminance
  - 30 nit laptops in low power mode
  - 600 nit HDTVs in Vivid mode
- Similarly, no adjustments to the content are made when you surf the web and play games, switching from phones to tablets to PCs.

This is **wrong**, but has been **good enough**

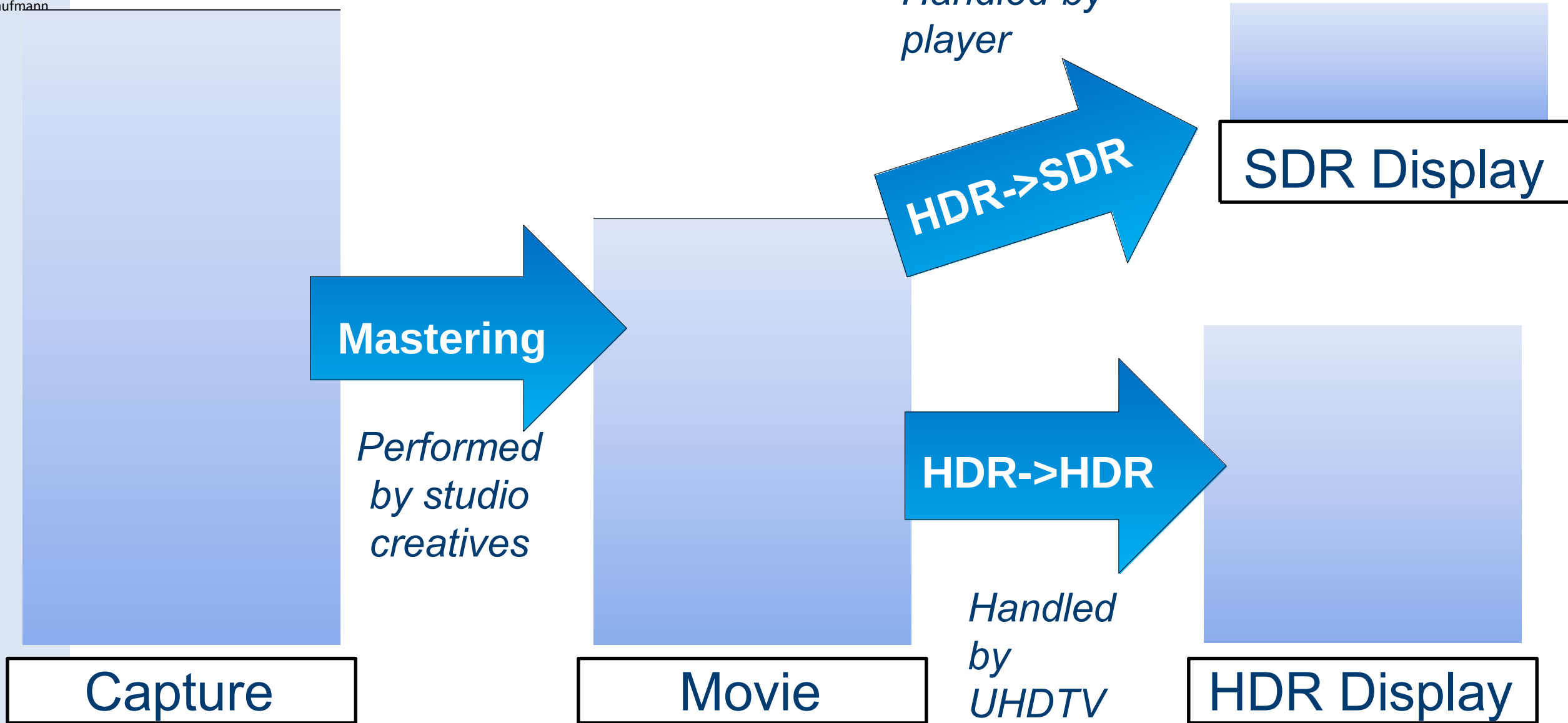
# Things to ponder

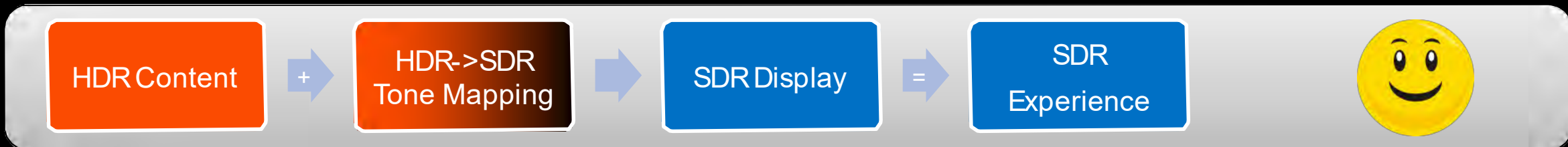
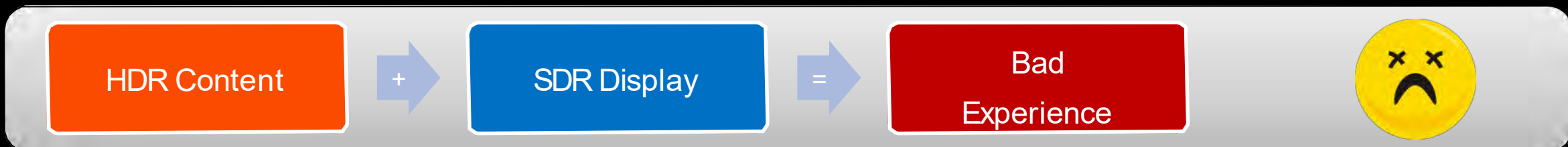
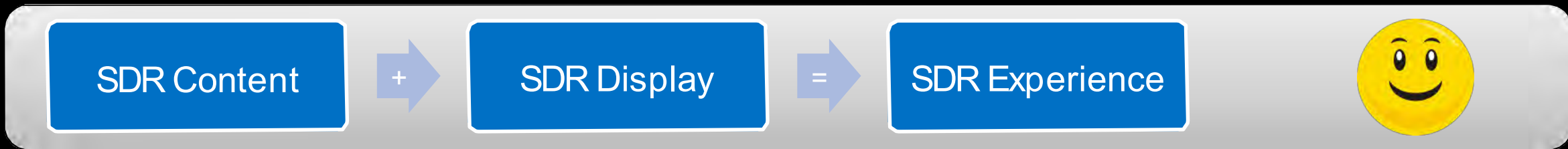
- With color, many real-world hues can be exactly reproduced on screens
- Almost every real-world situation has objects at  $>100$  nits
- So the real world is HDR, and every SDR picture you take involves HDR- $\rightarrow$ SDR conversion
- A 100-nit object on the screen almost never corresponds to a 100-nit object in the real world

# Multiple Ranges of HDR



# HDR Conversions







# Real World

**3,000  
nits**

**300,000  
nits**



# Mastered for SDR

**100  
nits**



**Coded  
value=  
0xFF**

**100  
nits**



**Coded  
value=  
0xFF**

# Mastered for HDR

200  
nits



Coded  
value=  
0x23B

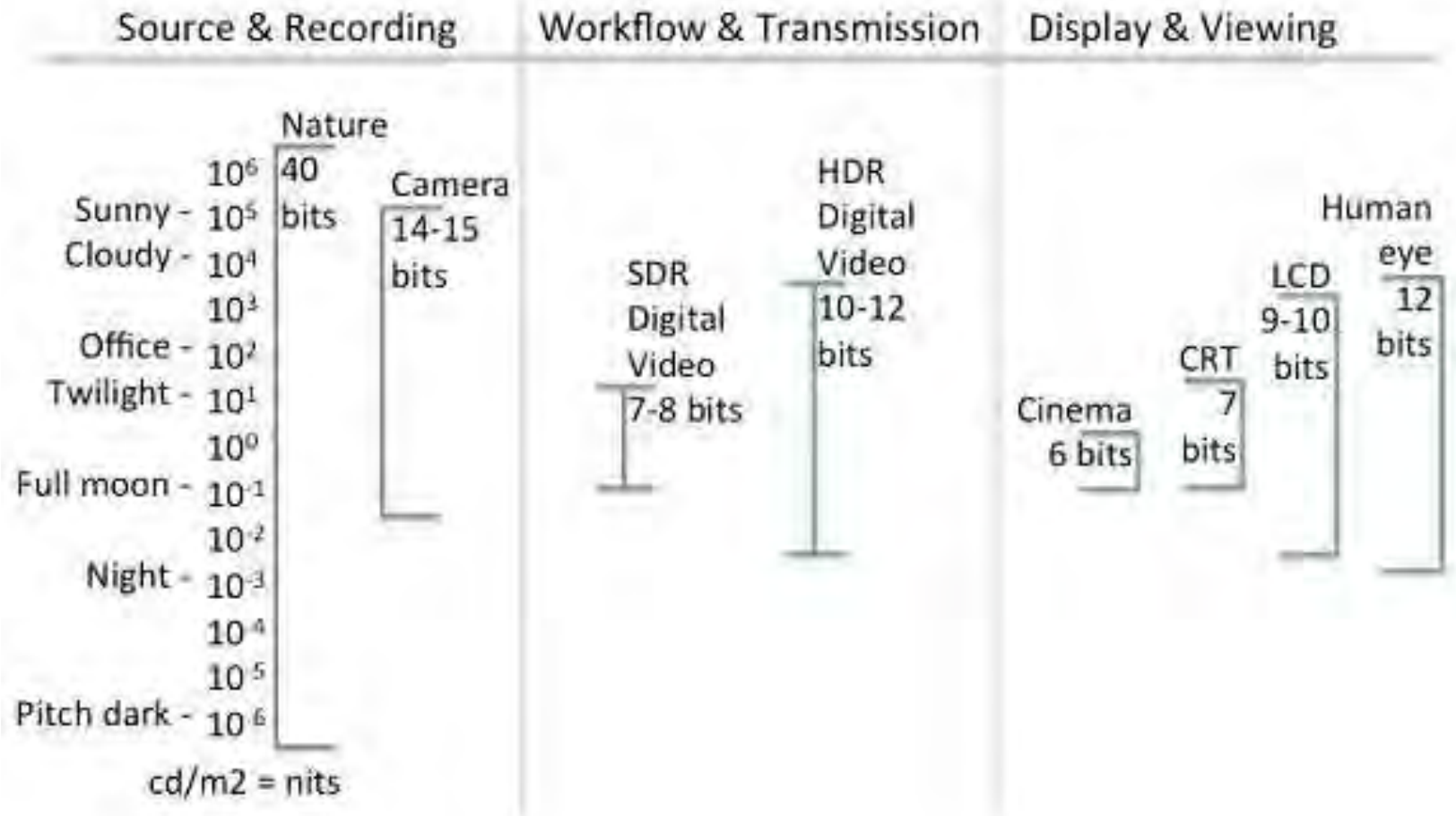
5,000  
nits



Coded  
value=  
0x36C



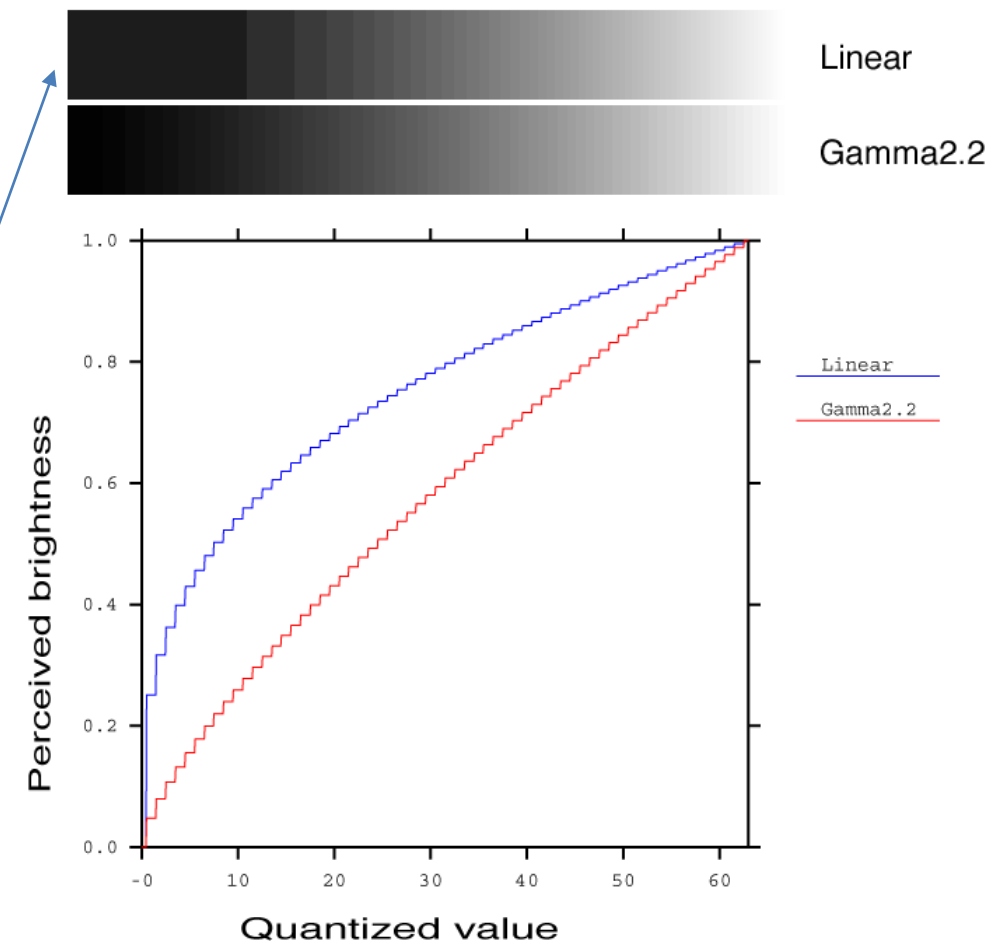
# Color and Brightness



Luminance dynamic range for various technologies

# Quantization: Linear vs non-linear light

- In the real world, luminance is determined by the number of photons
- However, brightness (the human perception of luminance) is non-linearly proportional to number of photons
  - The eye has a non-linear response to intensity. At most adaptation levels, we perceive brightness roughly as the cube root of intensity.
- If we apply a linear quantization of color values, we see more steps in darker regions than we do in the brighter regions
- Using a power law encoding with a  $g$  value of 2.2, we see a much more even distribution of quantization steps, though the behavior near black is still not ideal

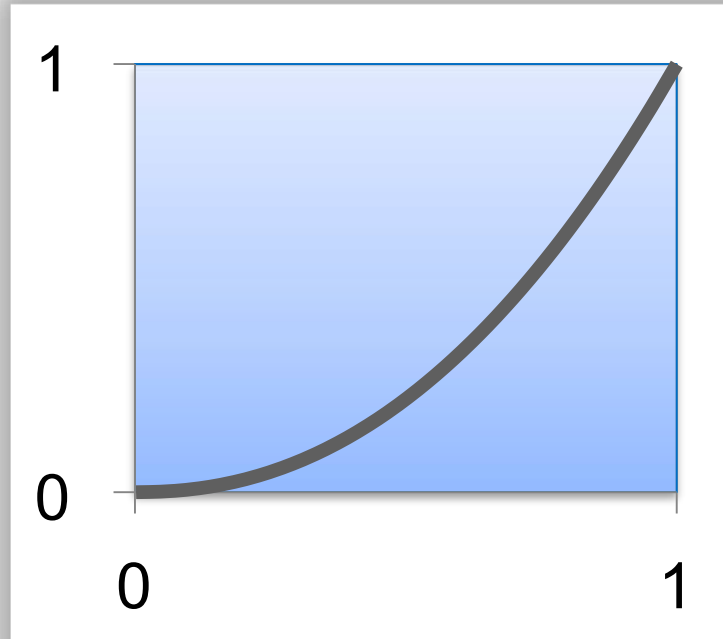
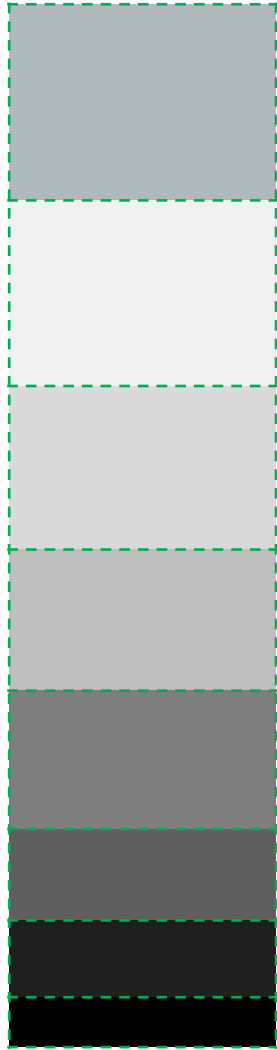


To efficiently code video signals, we apply a non-linear transform function so that we can code perceptually uniform brightness intervals

Luminance

Linear Light

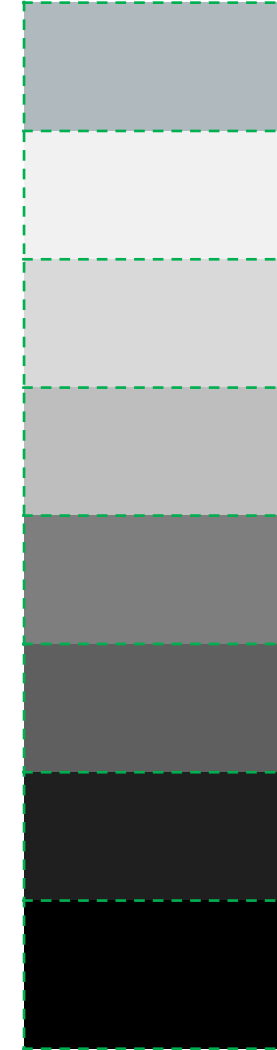
Perceptually  
non-uniform



Luma

Non-Linear Light

Perceptually  
Uniform





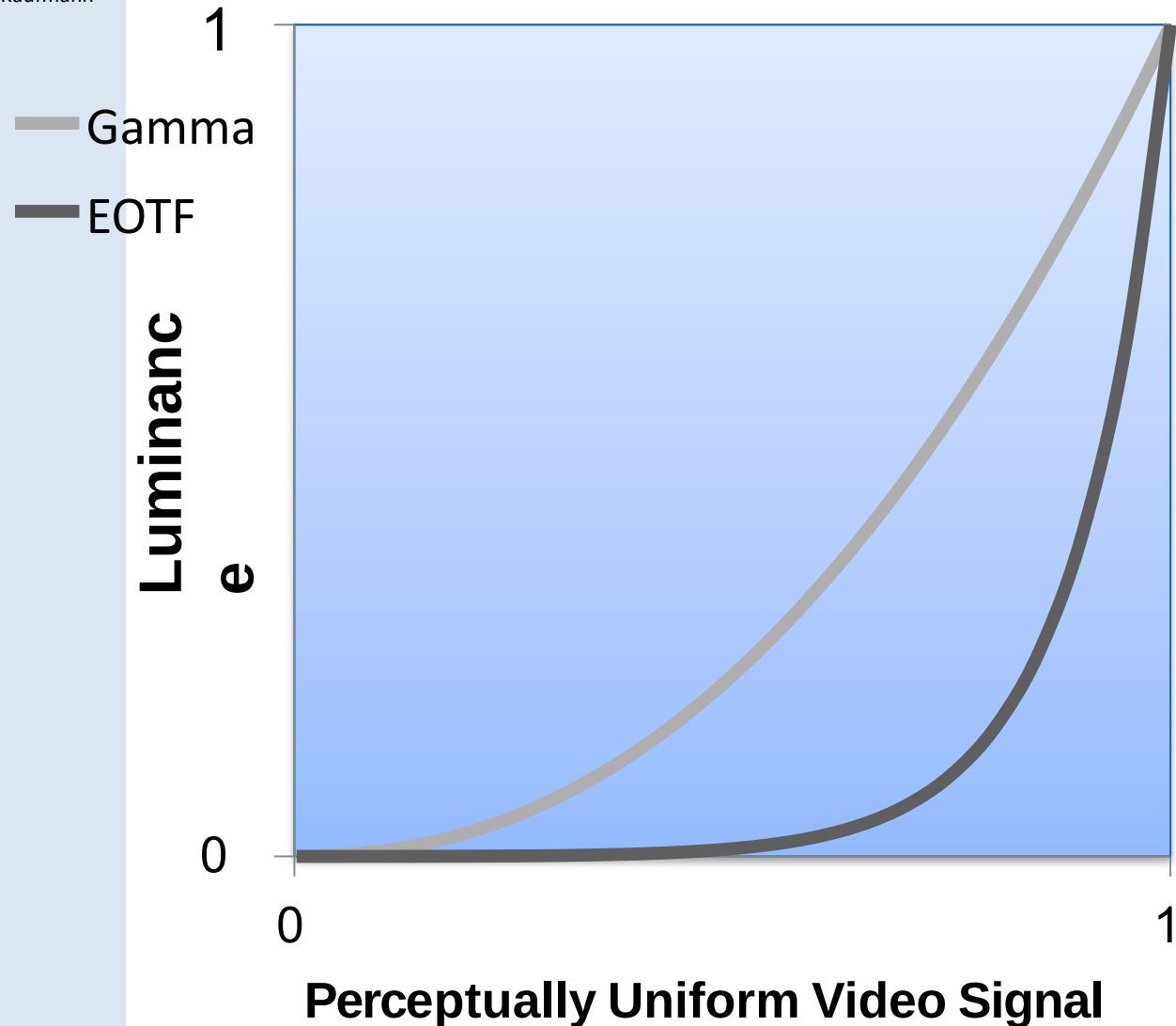
# Problems

- Although gamma is a reasonable approximation of human perception in the SDR (0-100 nit) range...
- Gamma is not a good match for human perception in the 0-10,000 nit HDR
- If you use gamma for a 10,000 nit signal, you need to use two extra bits/sample to eliminate banding

Color banding



# SMPTE 2084: HDR Electro-Optical Transfer Function (EOTF)



Electro-Optical Transfer Function (EOTF) is a correction function that **converts electrical voltages into brightness values**.

- If gamma curve is used = gamma correction, if more complex dependencies between voltage and brightness, an EOTF is used
- EOTF is used to adjust the brightness of displays or televisions to the human perception of brightness. Since this does not run linearly with voltage values, the display representation must be adapted.
- **Dependencies between brightness and voltage are different for various display technologies** and resolutions, such as SDTV, HDTV, UHDTV, HDR10 and Dolby Vision.
- Therefore several correction curves in addition to gamma correction: Perceptual Quantization (PQ) and Hybrid Log Gamma (HLG), which is described in the SMPTE's ST.2084 standard for video transmission of High Dynamic Range (HDR).

# Hybrid Log Gamma (HLG) & Perceptual Quantization

- HLG correction is comparable to gamma correction in classic television.
- Hybrid Log Gamma (HLG) and Perceptual Quantization (PQ)
  - standardized by the International Telecommunication Union (ITU) in BT.2100
  - adapting the dynamic range of the display on the UHD displays to the dynamic range of the recording devices.
  - Why: So that display corresponds to the viewer's perception of brightness.
  - Correspond to a logarithmic function
  - The brightness range corresponds to a luminance between 100 nits and 5,000 nits.
  - HLG correction curve requires a resolution of at least 10 bits and is used with High Dynamic Range HDR10 and Dolby Vision in transmission and mastering.
  - When transmitting video in SDR technology and HDR technology simultaneously in one data stream, the image content is encoded with the HLG curve so that it can be played back on both SDR and HDR displays.

# What is HDR (in this context)

- Content with a wider range of brightness and color
- Requires new monitors to experience HDR content
  - Legacy content (SDR): 8-bit color, 100 nit peak brightness, 709 gamut
    - Dynamic range of about 6 stops
  - HDR content: 10-bit color, 10,000 nit peak brightness, 2020 gamut
    - Dynamic range of about 17,6 stops

HDR is an ambiguous term

- There are several industry specs defining various flavors of HDR

# Summary

- HDR + Wide color gamut (WCG) gives us powerful new tools to create visual experiences that are clearly visible to the average viewer
- Making this all work requires lots of changes to the production pipeline and the products used to display it

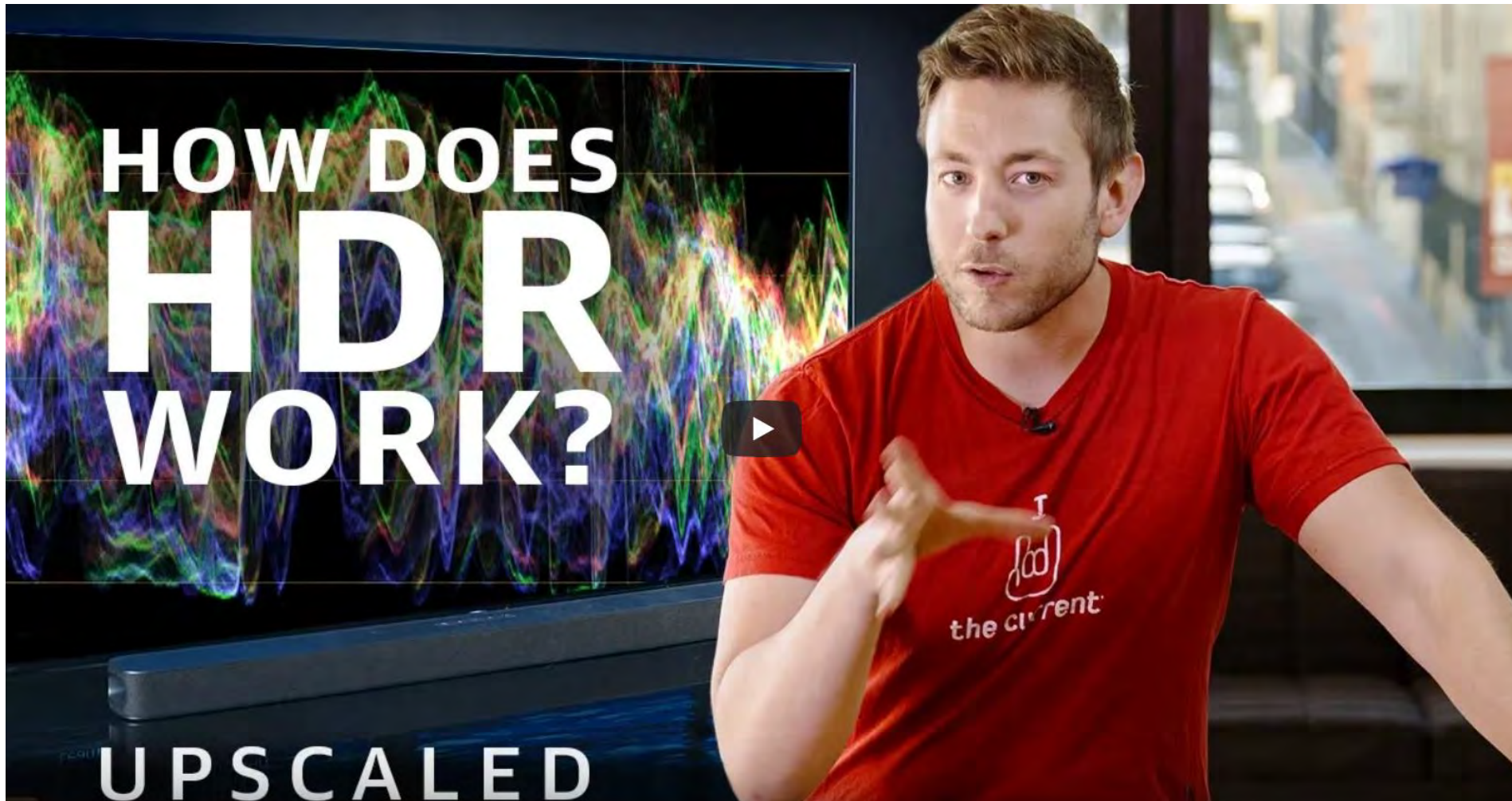
# HDR Encoding

- **Most formats are lossless**
- **Adobe DNG (digital negative)**
  - Specific for RAW files, avoid proprietary formats
- **RGBE**
  - 24 bits/pixels as usual, plus 8 bit of common exponent
  - Introduced by Greg Ward for Radiance (light simulation)
  - Enormous dynamic range
- **OpenEXR**
  - By Industrial Light + Magic, also standard in graphics hardware
  - 16bit per channel (48 bits per pixel) 10 mantissa, sign, 5 exponent
  - Fine quantization (because 10 bit mantissa), only 9.6 orders of magnitude
- **JPEG 2000**
  - Has a 16 bit mode, lossy



# HDR Technologies Summary

Luminance  
Dynamic Range  
Color Space  
Bit Depth



# HDR formats

- Summary of all HDR encoding formats (Greg Ward):  
[http://www.anywhere.com/gward/hdrenc/hdr\\_encodings.html](http://www.anywhere.com/gward/hdrenc/hdr_encodings.html)
- Greg's notes: <http://www.anywhere.com/gward/pickup/CIC13course.pdf>
- <http://www.openexr.com/>

# Some Standard Image Formats

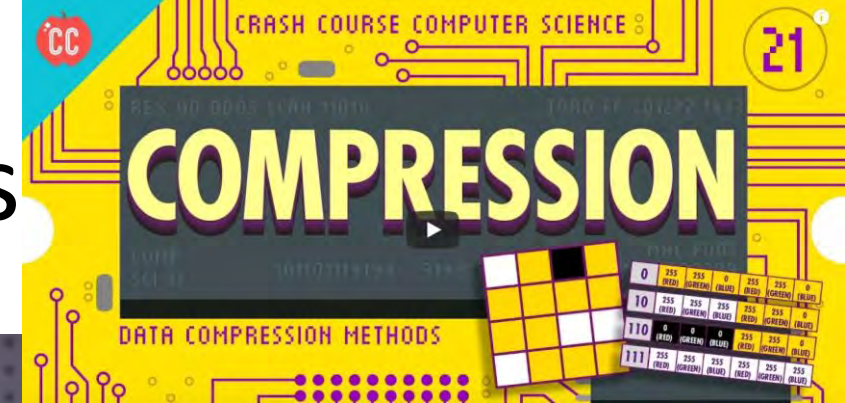
- EPS — Encapsulated PostScript
- **GIF — Graphics Interchange Format**
- PICT — may contain geometric data (lines, polygons, etc.)
- **JPEG — Joint Photographic Experts Group, lossy**
- PNG – includes transparency (alpha channel)
- and many many more....

# Image Compression



# Recap: Compression Algorithms

<https://www.youtube.com/watch?v=OtDxDvCpPL4>



# Compression Techniques - Classification

|                |                     |      |
|----------------|---------------------|------|
| Entropy coding | run-length coding   |      |
|                | Huffman coding      |      |
|                | arithmetic coding   |      |
| Source coding  | prediction          | DPCM |
|                |                     | DM   |
|                | transformation      | FFT  |
|                |                     | DCT  |
|                | vector quantization |      |
| hybrid coding  | JPEG                |      |
|                | MPEG                |      |
|                | px64                |      |
|                | DVI (RTV, PLV)      |      |



# Entropy Coding

- **lossless** encoding
- used **regardless of media's** specific **characteristics**
- data taken as simple digital sequence
- One of the main types of entropy coding
  - creates and **assigns a** unique prefix-free **code to each unique symbol** that occurs in the input.
  - then **compress data by replacing** each fixed-length **input symbol with** the corresponding variable-length prefix-free output **codeword**.
- Examples: **Run-length encoding, Huffman encoding, Arithmetic encoding**

# Source Coding & Hybrid Coding

- Usually **lossy** encoding (can also be lossless)
- **takes into account the data semantics**
- degree of compression **depends on data content**
- In source coding the particular **code** to be used **is chosen to match the source** (i.e. the relative probabilities of the symbols in the source alphabet).
- Example: Discrete Cosine Transformation (DCT) as transformation technique of the spatial domain into the two-dimensional frequency domain

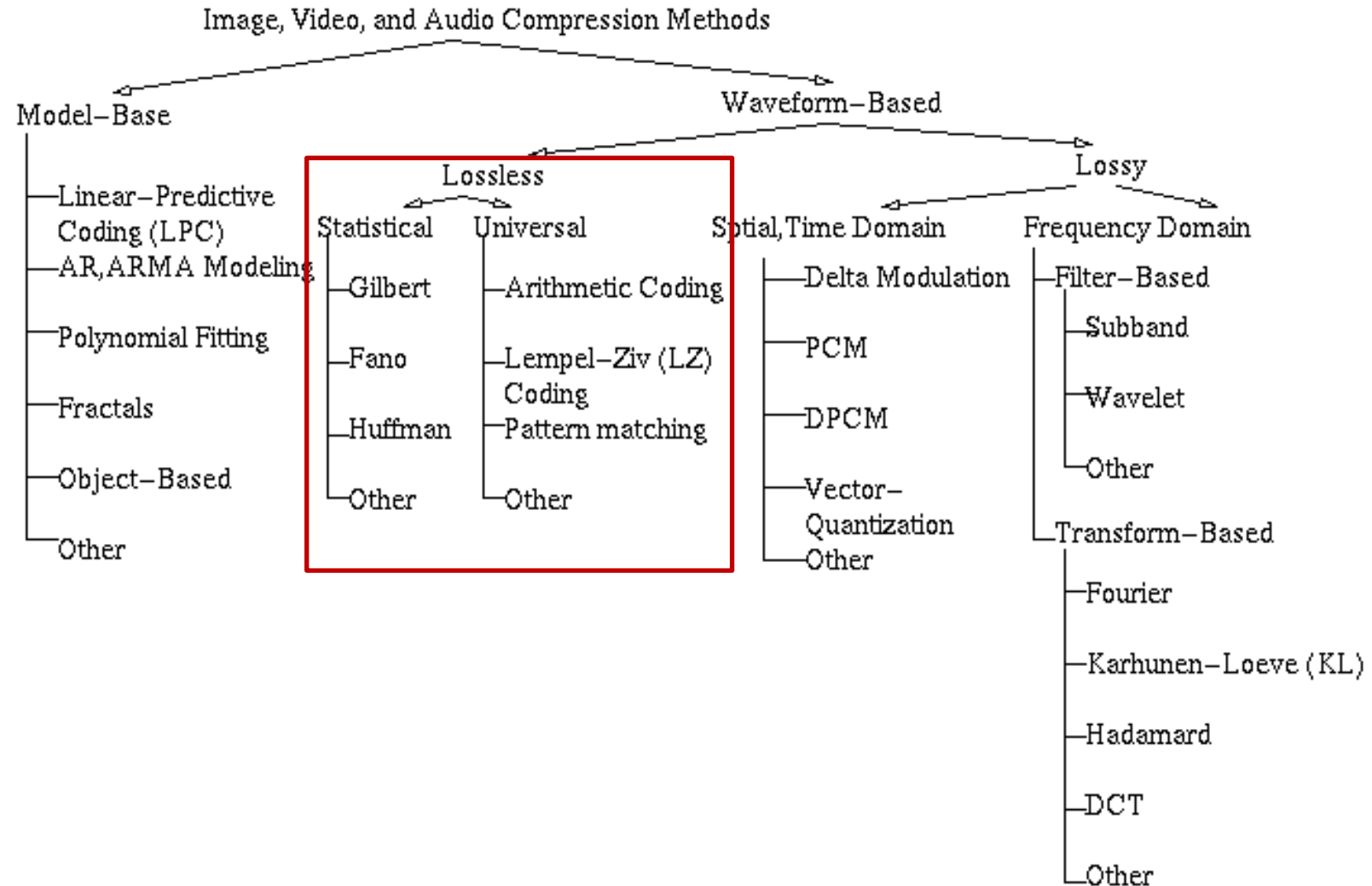
## Hybrid encoding

- Used by most multimedia systems
- Combination of entropy and source encoding
- Examples: JPEG, MPEG, H.261

# Decompression Requirements

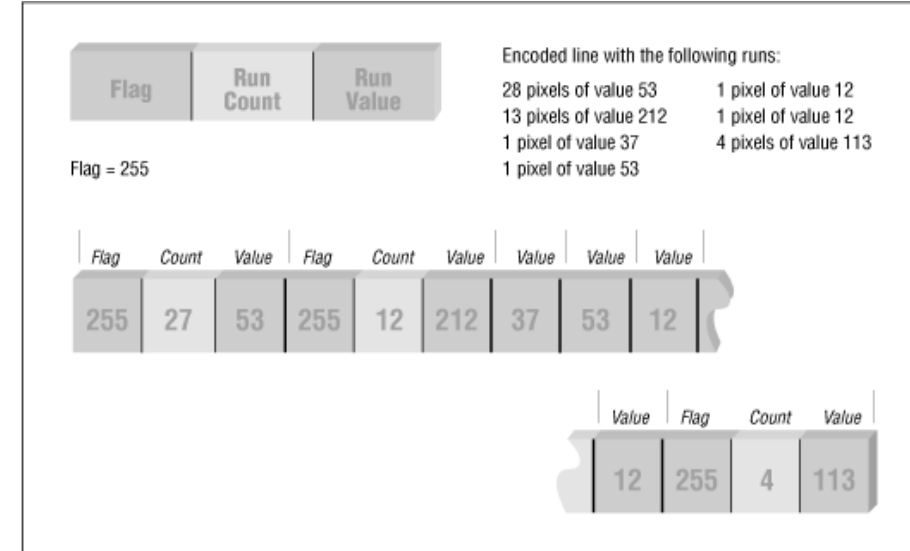
- Dialog mode applications (e.g. editing) might need **symmetric** compression  
= compression and decompression take about the same time
- Retrieval mode applications (e.g. video playback) use **asymmetric** compressions
  - compression is performed once and sample time available
  - decompression is performed frequently and needs to be done fast (in case of time-dependent media)

# Lossless Compression Algorithms



# Run-Length Encoding

- **Entropy coding** algorithm—  
content independent coding
- Replaces the sequence of the same consecutive bytes with the number of occurrences
- the number of occurrences is indicated by a *special flag (!)*
- Procedure:
  - if the same byte occurs at least 4 times count number of occurrences
  - write compressed data in the format
    - the counted byte '!' number of its occurrences



# Run-length Encoding

- Example:
  - uncompressed sequence (20 Bytes):  
ABCCCCCCCCCDEFFFFGGG
  - compressed sequence (13 Bytes): ABC!9DEF!4GGG
- Variations
  - Zero suppression technique (remove zeros)
  - text compression technique (also for images, video, audio)
  - diatomic encoding (represents a pair of characters as one special character)
- Is used for older image file formats e.g. Windows Bitmap, Targa or PCX.  
In Windows \*.rle is used



# Statistical Encoding

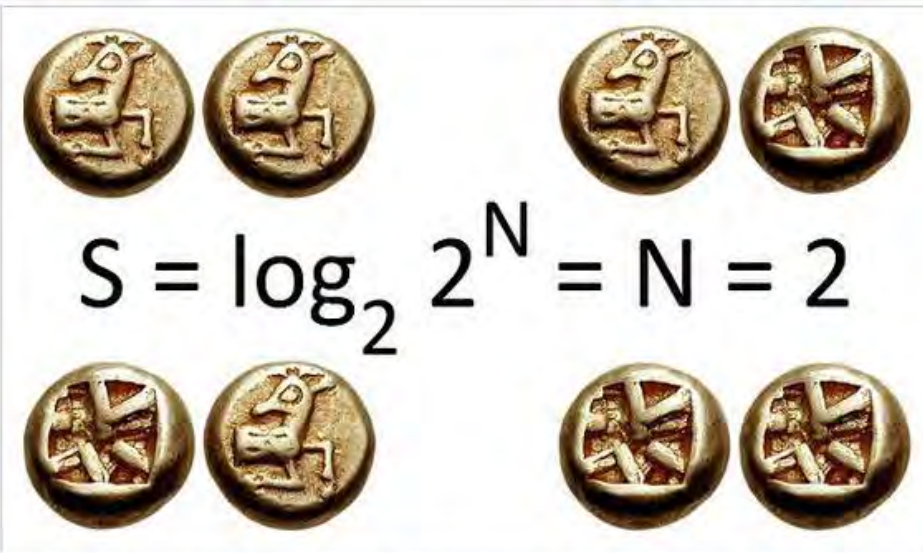
- Frequency dependent encoding—belongs to **entropy encoding**
- given is a sequence of symbols:  $S_1, S_2, S_3, \dots$  and the probability of occurrence of each symbol  $p(S_i)=p_i$
- Ex.:  $p(A)=0.16, p(B)=0.51, p(C)=0.09, p(D)=0.13, p(E)=0.11$   
first choice: encode A,B,C,D,E as 000,001,010,011,100

Q: What is the minimum number of bits per symbol?

A: The theoretical minimum average number of bits per code word is known as **entropy** ( $H$ ), according to Shannon

# Entropy

- Introduced by Claude Shannon in his 1948 paper "A Mathematical Theory of Communication" - sometimes called **Shannon entropy**
- Example: A biased coin with probability  $p$  of landing on heads and probability  $1-p$  of landing on tails. The maximum surprise is for  $p = 1/2$ , when there is no reason to expect one outcome over another, and in this case a coin flip has an entropy of **one bit**. The minimum surprise is when  $p = 0$  or  $p = 1$ , when the event is known and the entropy is zero bits.
- Shannon's experiments with human predictors show an information rate between 0.6 and 1.3 bits per character in English.
- Shannon's entropy **measures the information contained in a message** as opposed to the portion of the message that is determined (or predictable). Examples of the latter include redundancy in language structure or statistical properties relating to the occurrence frequencies of letter or word pairs, triplets etc.



Two bits of entropy: In the case of two fair coin tosses, the information entropy in bits is the base-2 logarithm of the number of possible outcomes; with two coins there are four possible outcomes, and two bits of entropy. Generally, information entropy is the average amount of information conveyed by an event, when considering all possible outcomes.

$$H(X) = - \sum_{i=1}^n P(x_i) \log P(x_i)$$

# Huffman Encoding

- Statistical encoding, depends on **occurrence frequency of single characters or sequences** of data bytes
- Characters stored with their probabilities
- Length (number of bits) of the coded characters differs
- Shortest code assigned to the most frequently occurring character
- To determine Code, it is useful to construct a **binary tree**
  - leaves are characters to be encoded
  - nodes carry **occurrence probabilities** of the characters belonging to the subtree

# Huffman Encoding — Algorithm

build a Huffman Tree:

$p(A)=0.17$ ,  $p(B)=0.51$ ,  $p(C)=0.08$ ,  $p(D)=0.14$ ,  $p(E)=0.1$

```
build ordered list of symbols (increasing probability)
```

```
do while list contains at least 2 elements
```

```
    construct tree using the first two elements in list
```

```
    add parent node for the union of these elements and compute probability
```

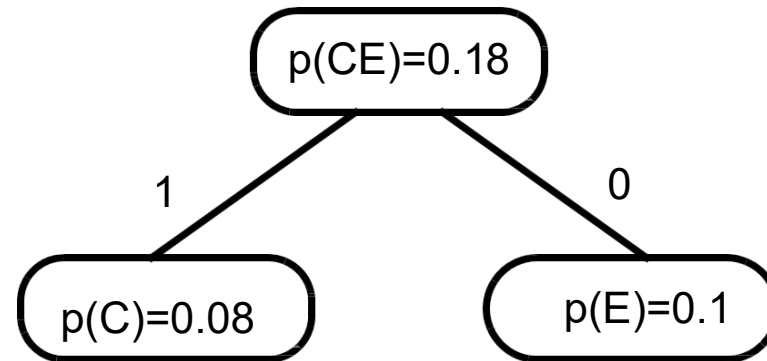
```
    mark edges by '0' and '1'
```

```
    delete the first 2 elements in list; insert parent into list
```

```
end
```

# Huffman Encoding—Example

step 1:  $p(C)=0.08$ ,  $p(E)=0.1$ ,  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(B)=0.51$



# Huffman Encoding—Example

step 2:  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(CE)=0.18$ ,  $p(B)=0.51$

build ordered list of symbols (increasing probability)

do while list contains at least 2 elements

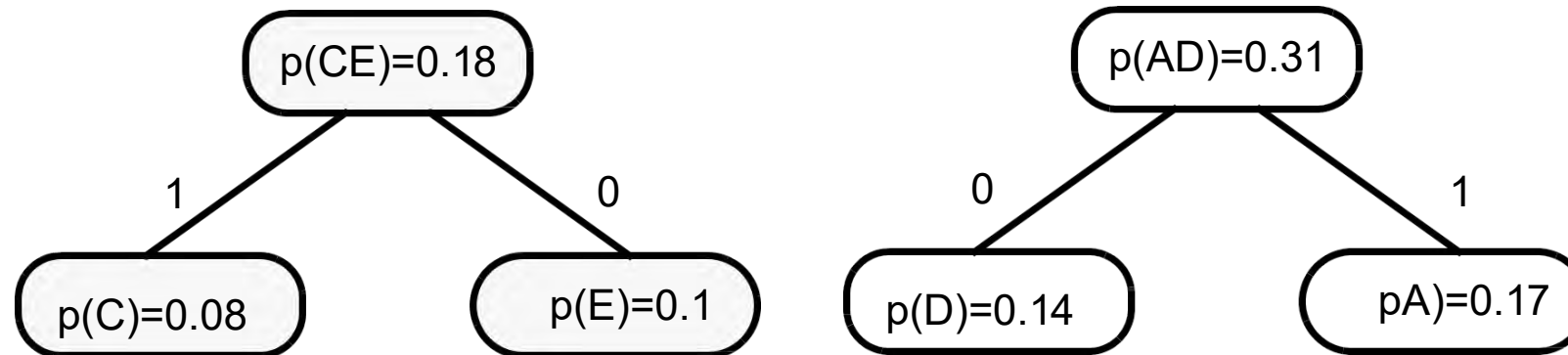
construct tree using the first two elements in list

add parent node for the union of these elements and compute probability

mark edges by '0' and '1'

delete the first 2 elements in list;  
insert parent into list

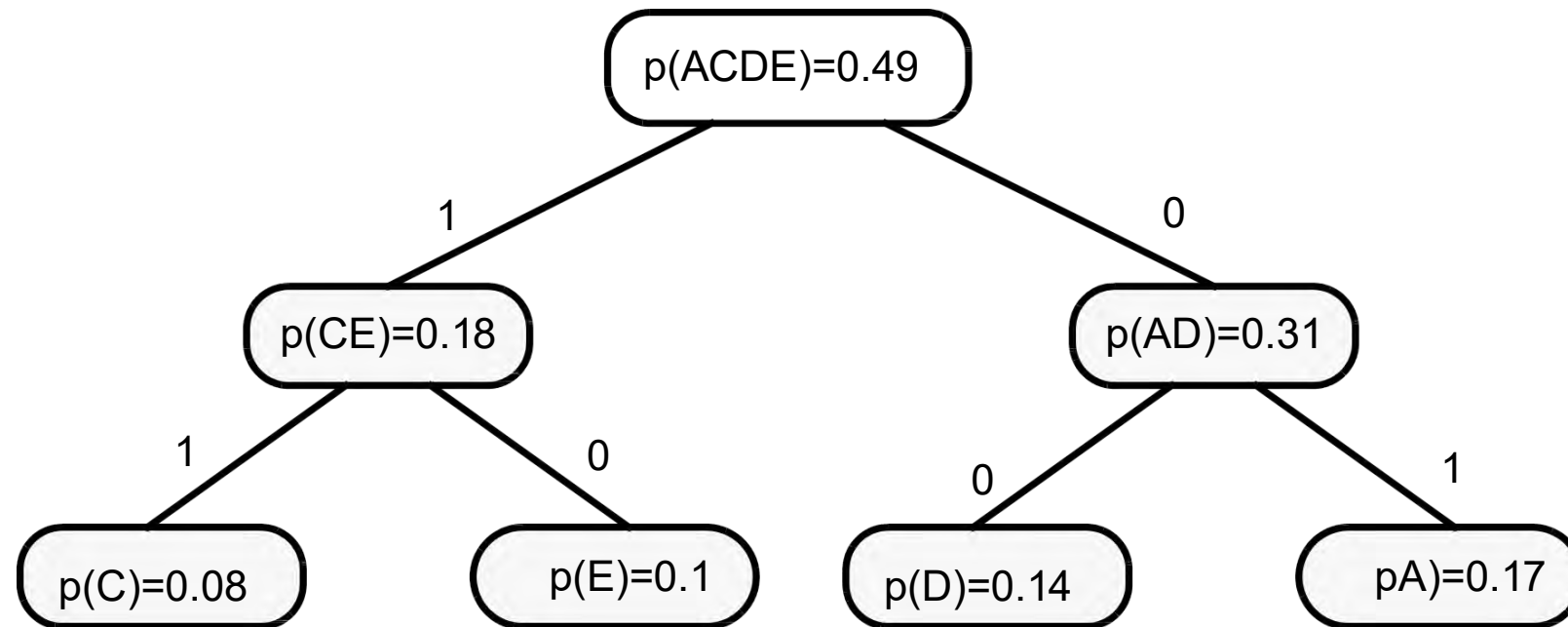
end





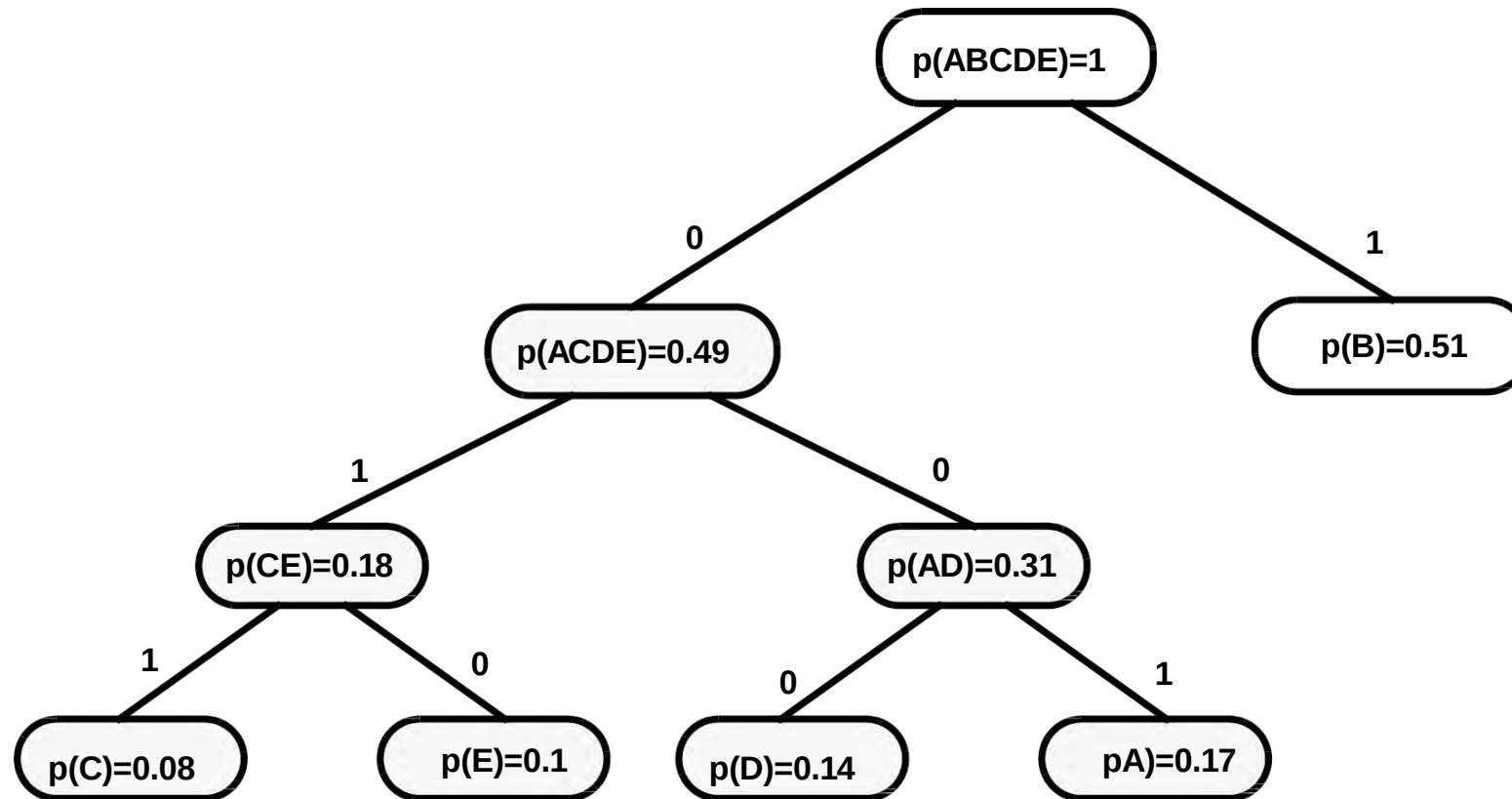
# Huffman Encoding—Example

step 3:  $p(CE)=0.18$ ,  $p(AD)=0.31$ ,  $p(B)=0.51$



# Huffman Encoding—Example

last step:  $p(ACDE)=0.49$ ,  $p(B)=0.51$



# Huffman Encoding—Example

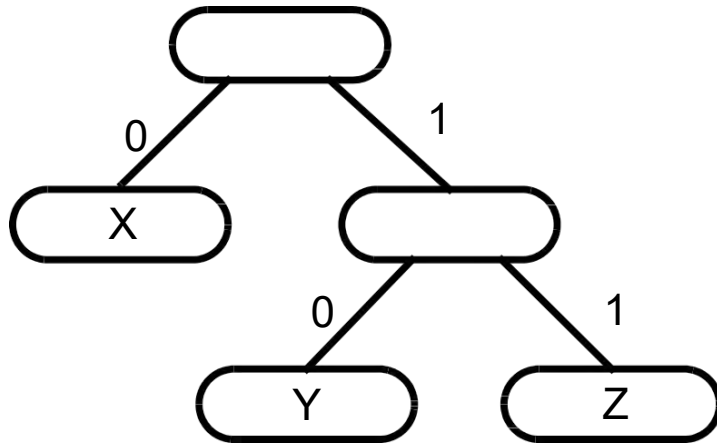
encode symbols according to tree

|   |     |
|---|-----|
| A | 001 |
| B | 1   |
| C | 011 |
| D | 000 |
| E | 010 |

$p(C)=0.08$ ,  $p(E)=0.1$ ,  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(B)=0.51$

# Huffman Decoding—Example

given: 000101011 and Huffman Tree

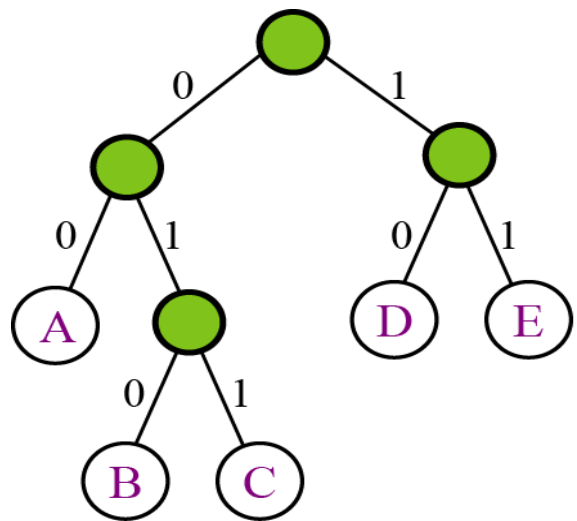


reconstruct message from bit stream using  
finite state machine

Solution: XXXYYZ

# Huffman Encoding

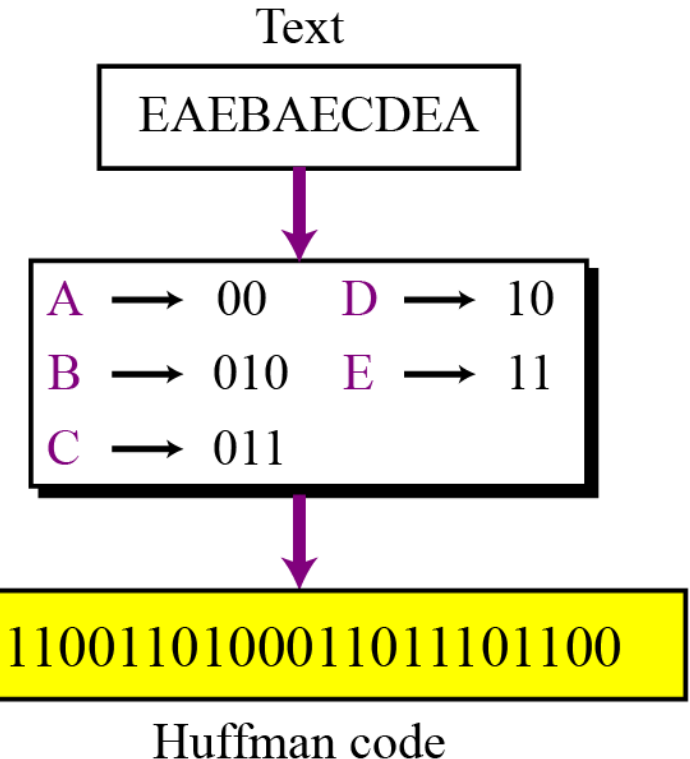
- Character code found by starting at the root and following the branches that lead to that character.
- The code itself is the bit value of each branch on the path, taken in sequence.



|        |       |
|--------|-------|
| A: 00  | D: 10 |
| B: 010 | E: 11 |
| C: 011 |       |

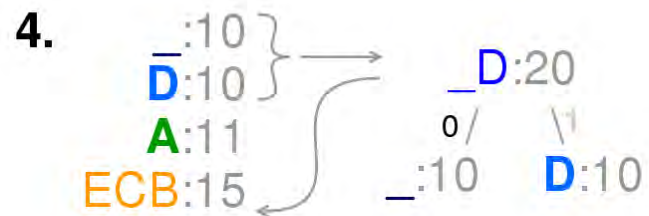
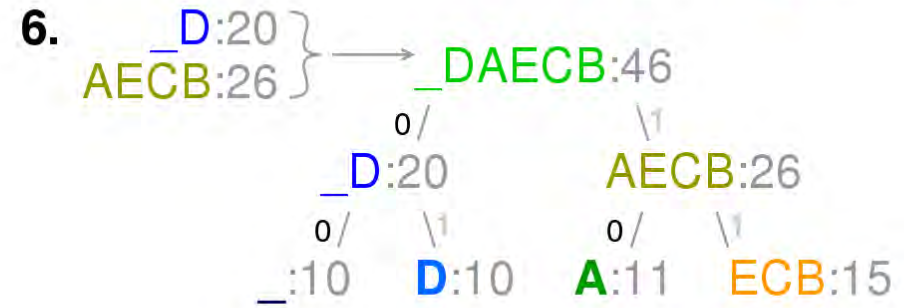
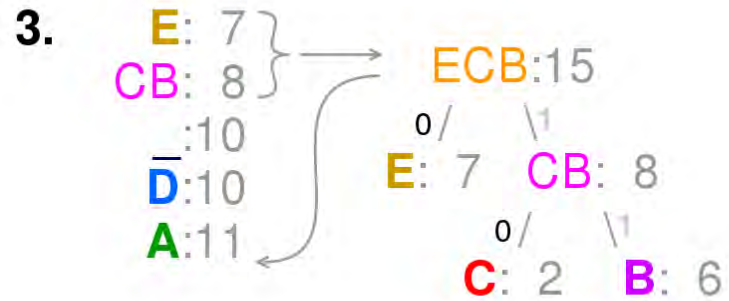
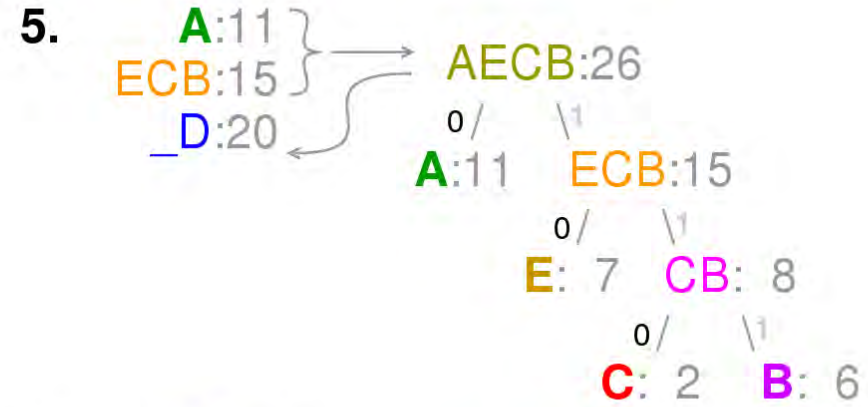
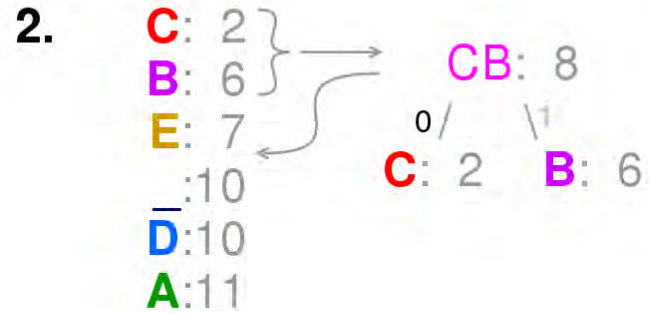
Code

Encoder



- Decoding: reverse process

1. "A\_DEAD\_DAD\_CEDD\_A\_BAD\_BABE\_A\_BEADED\_ABACA\_BED"



8. "100001110100100011001001110110011100100100011111001001111101111110  
0010001111110100111001001011111011101000111111001"



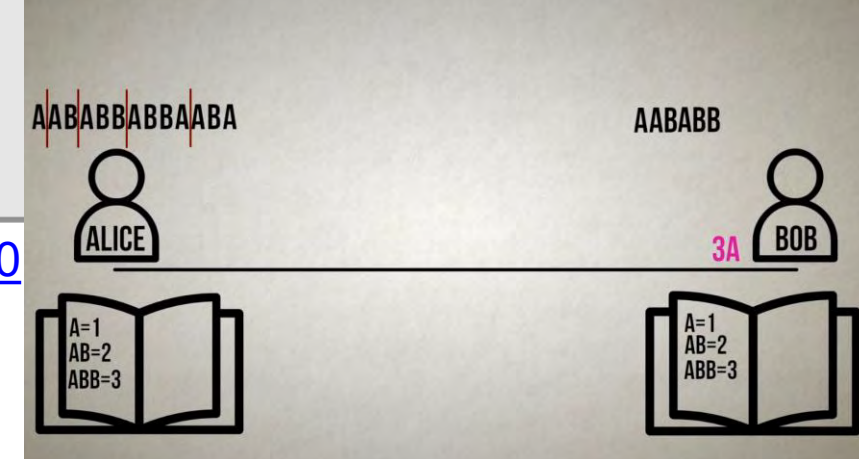
# Application of Huffman Coding for Video

- If the image information can be transformed into a bit stream, Huffman table can be used to compress data without loss
- Simple way to generate a bit stream is to code pixels individually and read them line by line
- For video: Huffman table can be used for a single sequence of images
  - for a set of scenes
  - for an entire film clip
- need Huffman table for encoding and decoding
- Is **used for MP3** encoding in the final step!

# Lempel Ziv (LZ) Dictionary-based Encoding

<https://www.youtube.com/watch?v=RV5aUr8sZD0>

- Dictionary is a table of strings
  - Sender and receiver have a copy of dictionary
  - Previously-encountered strings are substituted by their index in dictionary
- Compression - two concurrent events:
  - Building an indexed dictionary
  - Compressing a string of symbols.
- Algorithm extracts smallest substring not in the dictionary from remaining uncompressed string.
- Stores a copy of this substring in dictionary as a new entry and assigns it an index value.
- Compression occurs when substring (except for the last character) is replaced with the index found in the dictionary.
- Process inserts the index and the last character of the substring into the compressed string.



Used in GIF & ZIP

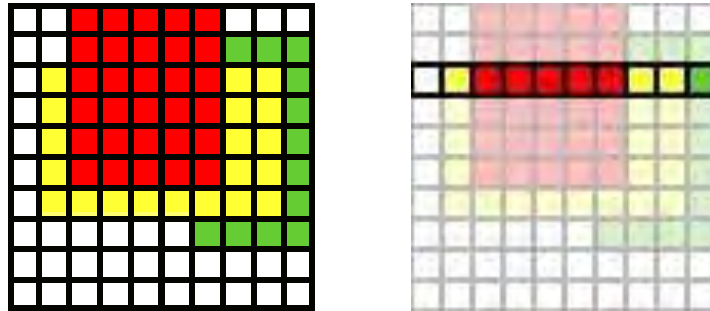
Abraham Lempel



# Lossless Image Formats



- GIF-Format (Graphics Interchange Format)
  - LZW-Compression (Lempel, Ziv, Welch): works line-wise



- 1 st line, 2nd line
- 3rd line is compressed as:
  - 1 white, 1 yellow, 5 red, 2 yellow, 1 green
- Row 4 to 6: „as row 3“
- Indexed (1-8 bit Color Lookup Table)



# CompuServ GIF (gif)



- First standardized in 1987 by CompuServ (called GIF87a)
- Updated in 1989 to include transparency, interlacing, and animation (called GIF89a)
- Use the LZW (Lempel-Ziv Welch) algorithm for compression (not free, licenses necessary)
- A maximum of **256 colors** freely selectable out of 24 bit ( $2^{24} = 16.777.216$ )
- Does not work for photographs
- Suitable for small images such as icons
- Simple animations



# Portable Network Graphics (png)



- Developed because of licensed LZW with gif
- Replacing GIF and TIFF (not JPEG!)
- 3 Types
  - True Color (3 x 16 bit/pixel)
  - Grayscale (1 x 8 bit/pixel)
  - Palette (256 colors)
- Compression similar to PKZIP (Phil Katz)
- The standard zip format provides approximately **62 percent** compression. (RAR results in a **71 percent** compression rate; 7-zip has **76 percent**)



# Differential Encoding

- belongs to **source coding**
- consider sequence of symbols  $S_1, S_2, S_3, \dots$  where values are not zeros, but do not vary much  $\rightarrow$  calculate difference from previous value
- Ex.: still images—calculation of differences between nearby pixels or pixel groups

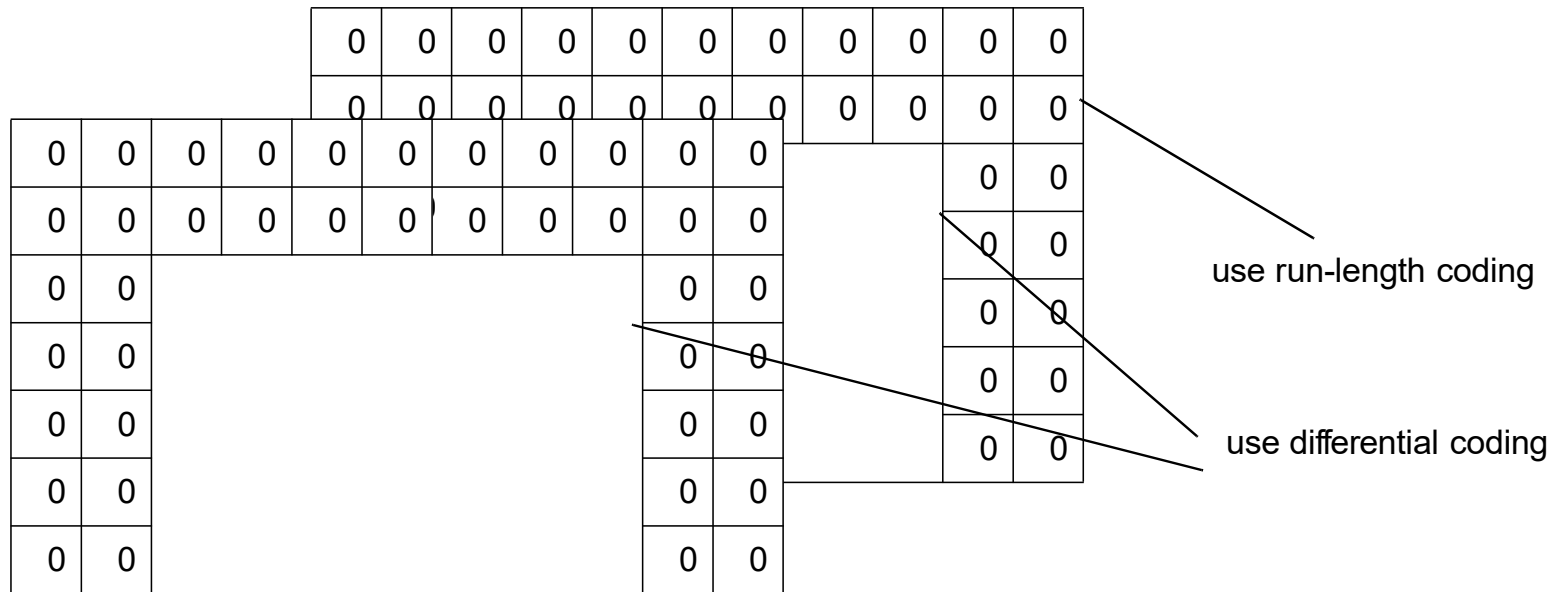
|   |     |    |    |     |
|---|-----|----|----|-----|
| 0 | 0   | 0  | 0  | 0   |
| 0 | 255 | 94 | 87 | 100 |
| 0 | 0   | 0  | 0  | 255 |
| 0 | 0   | 0  | 0  | 0   |

- edges result in large values
  - similar chrominance or luminance values result in small values
- zeros can be suppressed by run-length coding



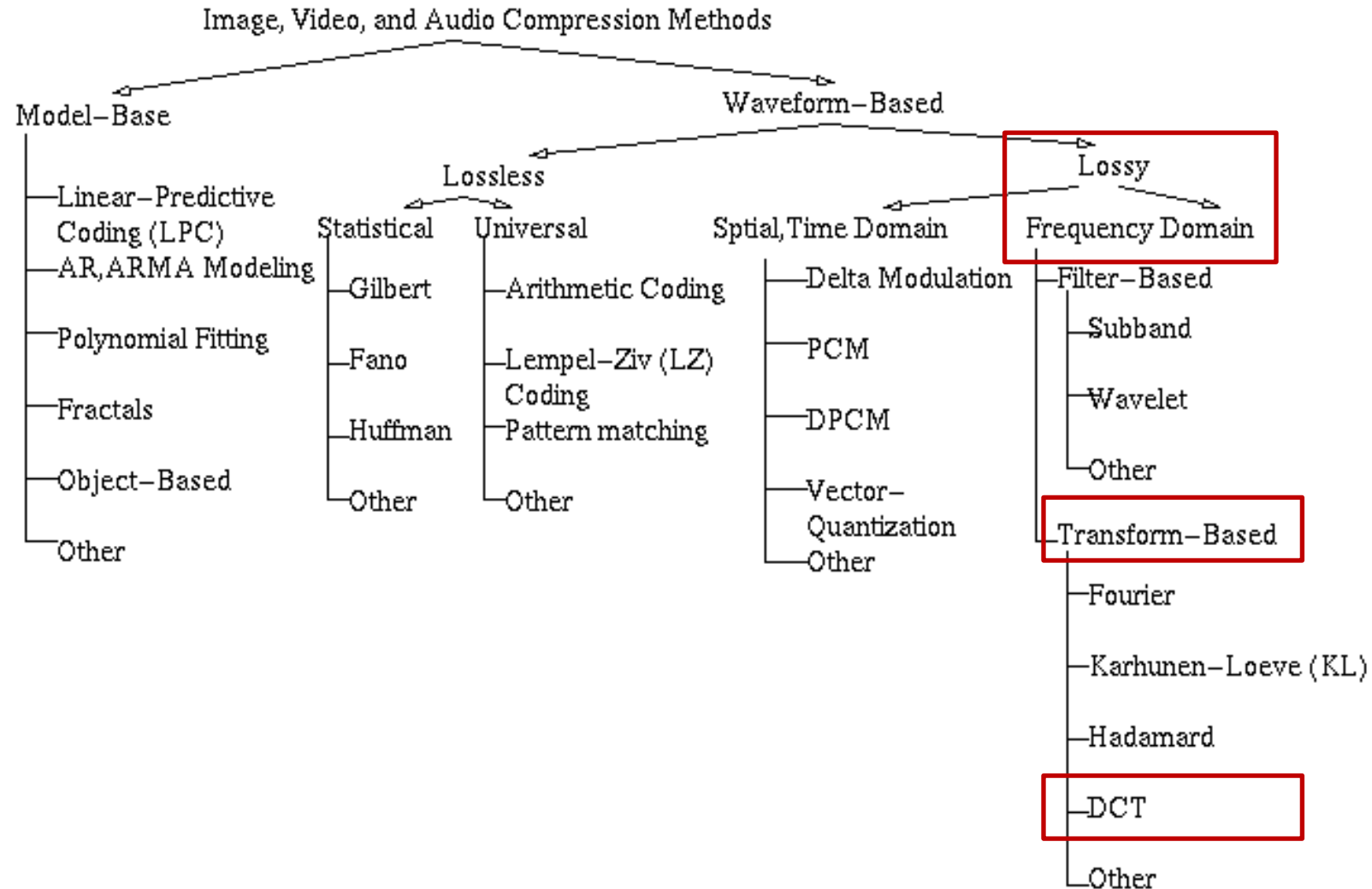
# Differential Coding in Video (Examples)

- static background (videoconferencing, newscast, etc.)

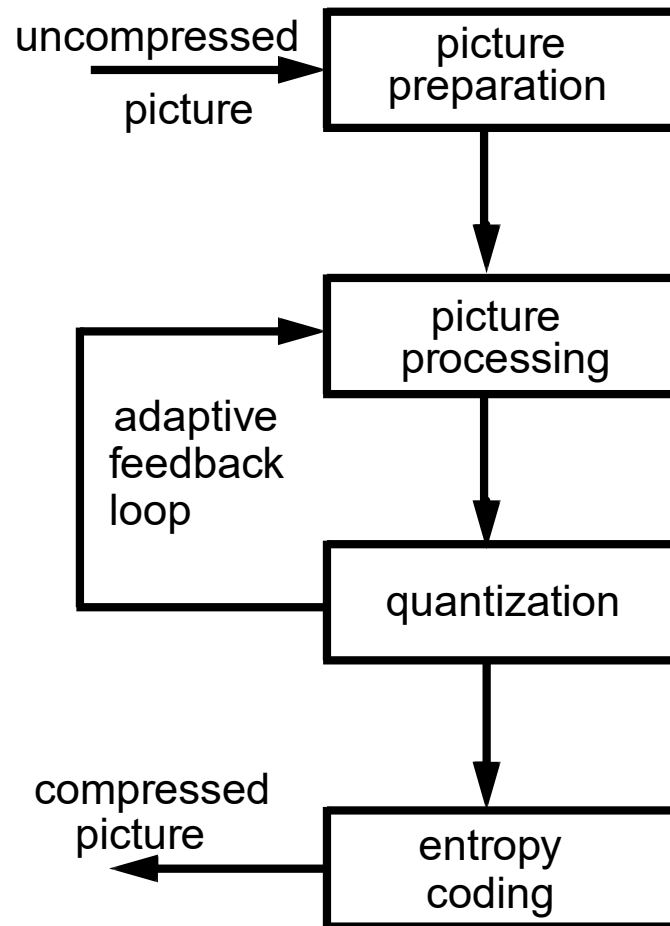


- motion compensation—8x8 blocks are compared; areas similar, only shifted, e.g., to the right (*motion vector*)

# Lossy Image Compression



# Image Compression Steps



Possible operations:

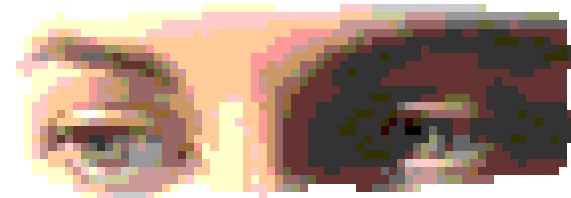
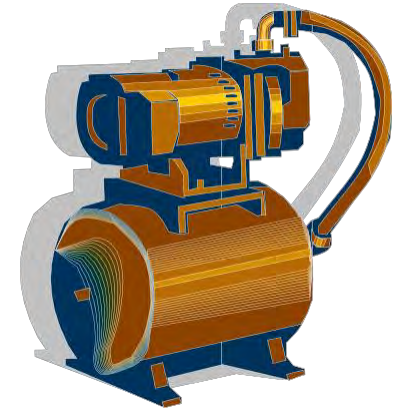
- analog-to-digital conversion
- generation of appropriate digital representation
- image division into 8 x 8 blocks
- fix number of bits/pixel

Compression Algorithm

- transformation from time to frequency domain e.g., using DCT
- motion vector application for motion video
- Typically a lossy step
- mapping of real numbers to integers (reduction in precision)  
example:  $\mu$ -law audio—from 12 bits to 8 bit quantization
- compress a sequential digital stream without loss

# Lossy Compression

- Run-length coding works well for simple images (graphics) and computer data
- However, a large, detailed picture usually is not reduced enough
- In order to reduce information further lossy compression is needed
  - Information is permanently removed
  - The trick is to remove details that are not perceived by a human observer
  - Many of these psycho-visual coding systems take advantage of a number of aspects of the human visual system



# Human Visual System



- The eye is more sensitive to brightness changes than to color changes
- The eye is not able to perceive brightness above or below certain threshold values
- The eye does not perceive little brightness or color changes. The strength of this phenomenon is dependent on the color



# Compression of Still Images



- Characteristics of human vision have been transferred to **lossy image compression techniques**
  - remove some image details to achieve a higher compression rate by suppressing higher frequencies
- Such a development is the JPEG format
  - Stands for "Joint Photographic Experts Group"
  - JPEG is the worldwide standard
  - JPEG compresses brightness and color information separately
  - Color information is more compressed (lower sensitivity)
- Color space for JPEG compression
  - RGB values are a combination of brightness and color
  - If the RGB values are separated into a luminance and a color component, the color component is more compressed





# Joint Photographic Experts Group (JPEG)

JPEG is a joint standard of ISO and ITU-T

- In June 1987, an adaptive transformation coding technique based on DCT was adopted for JPEG
- In 1992, JPEG became a ISO international standard
- Lossy compression combined with lossless techniques
- Trade-Off between file size and quality
- JPEG typically achieves 10:1 compression with little perceptible loss in image quality

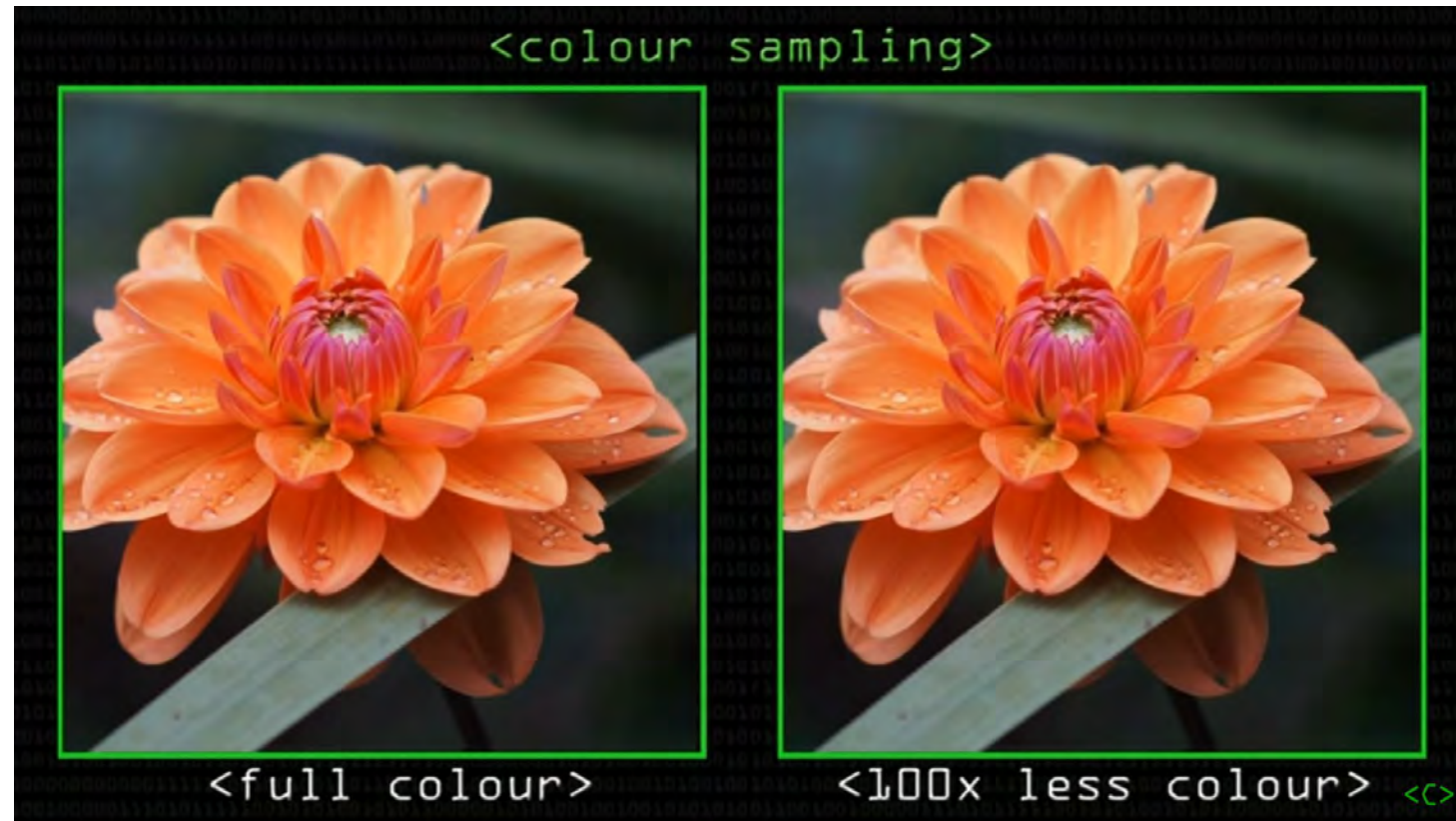
Requirements:

- to be state-of-the-art for degree of compression versus image quality,
- to be parameterizable (user can select parameters),
- to be applicable to any kind of source image, without regard to dimensions, content, image aspect ratio, pixel aspect ratio, etc.,
- to have computational requirements that are reasonable for hardware and software implementations,
- to run on as many standard platforms as possible

# Joint Photographic Experts Group (JPEG)

- to support four different modes of operation:
  - **sequential encoding—encoded in same order as scanned**
  - progressive encoding—multiple pass encoding
  - lossless encoding
  - hierarchical encoding—encoded at multiple resolutions

# How JPEG Works – Part 1



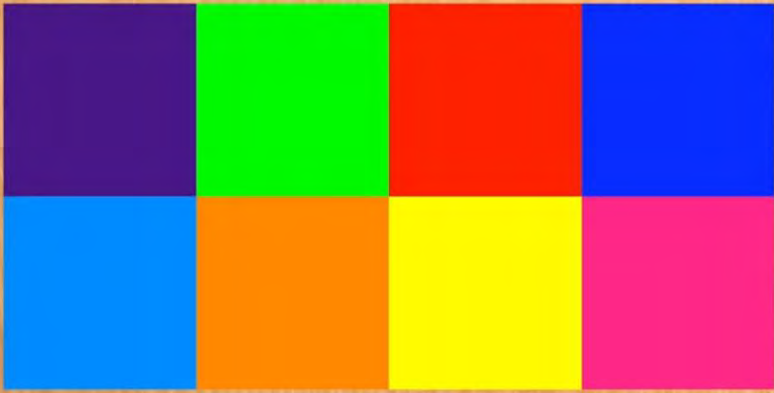
# 1<sup>st</sup> Step: Color Conversion YCbCr <-> RGB

- Necessary in preparation for JPEG compression
  - Color space conversion and Downsampling:
    - Separation of the color components from the luminance information
    - Insensitivity of human eye to rapid color changes allows coarser sampling of the color components
- > No loss of subjective quality
- > Significant data reduction

# Chroma Subsampling

**4:4:4**

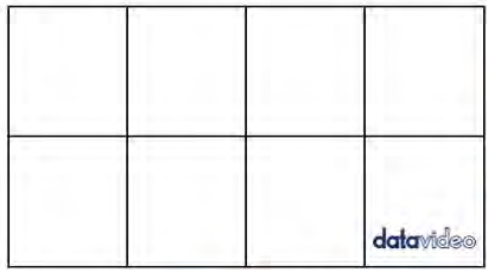
**Chroma**



Some visual examples  
<https://www.red.com/red-101/video-chroma-subsampling>

<https://www.youtube.com/watch?v=32PPzwPjDZ8>

| Y                                            | Cr                                | Cb                                 |
|----------------------------------------------|-----------------------------------|------------------------------------|
| The Numbers of Across<br>Pixels <b>4</b> x 2 | First Row<br>The Numbers of Color | Second Row<br>The Numbers of Color |



Cr

Cb

**Y** **4** x 2

**4 : 4 : 4**

100%

**4 : 2 : 2**

50%

**4 : 2 : 0**

25%

# 1<sup>st</sup> Step: Color Conversion YCbCr <-> RGB

- YCbCr (256 levels) can be computed directly from 8-bit RGB as follows:

$$\begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$

- During decoding YCbCr color space needs to be converted back to RGB:

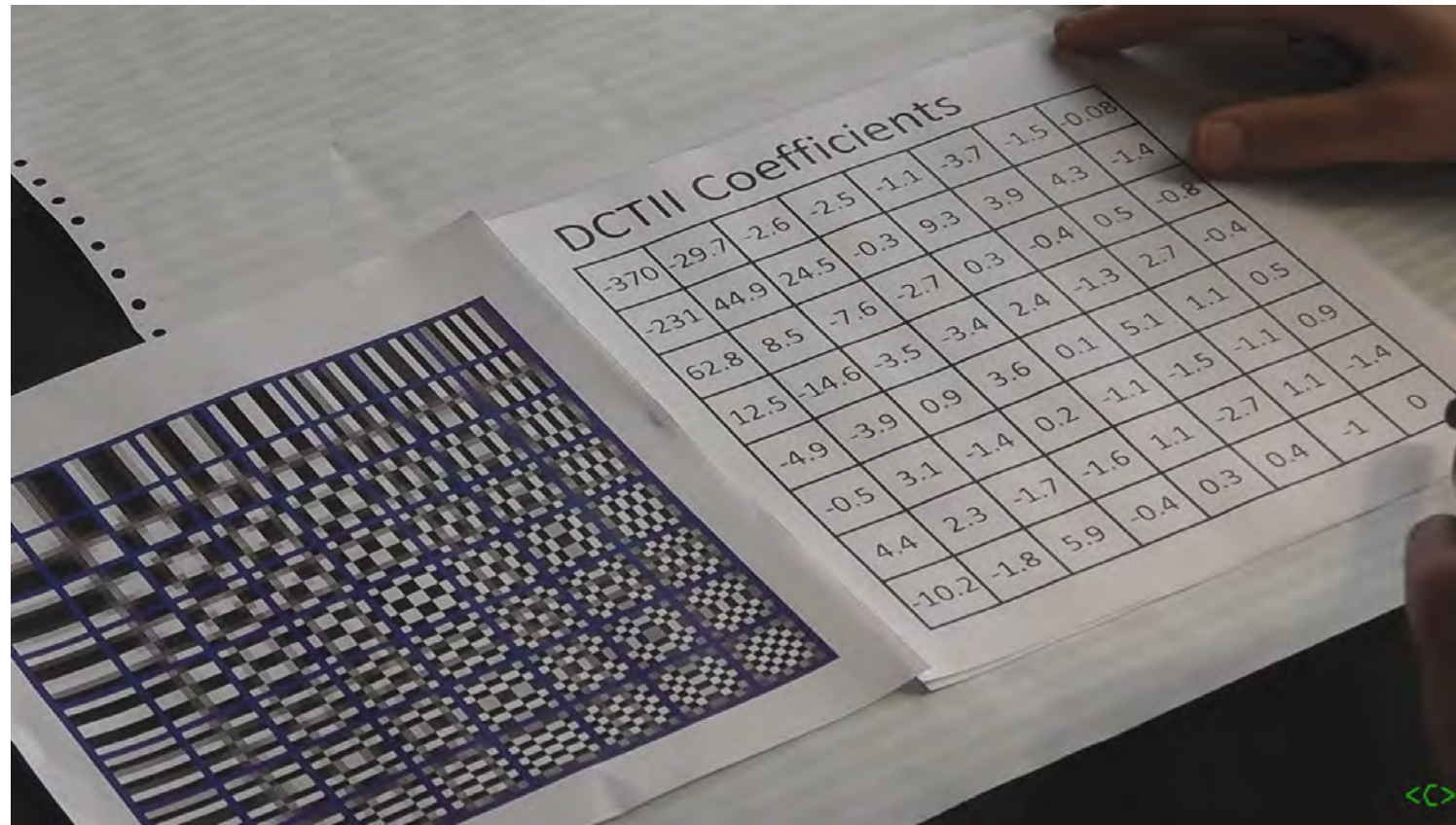
$$R = Y + 1.402 (Cr-128)$$

$$G = Y - 0.34414 (Cb-128) - 0.71414 (Cr-128)$$

$$B = Y + 1.772 (Cb-128)$$



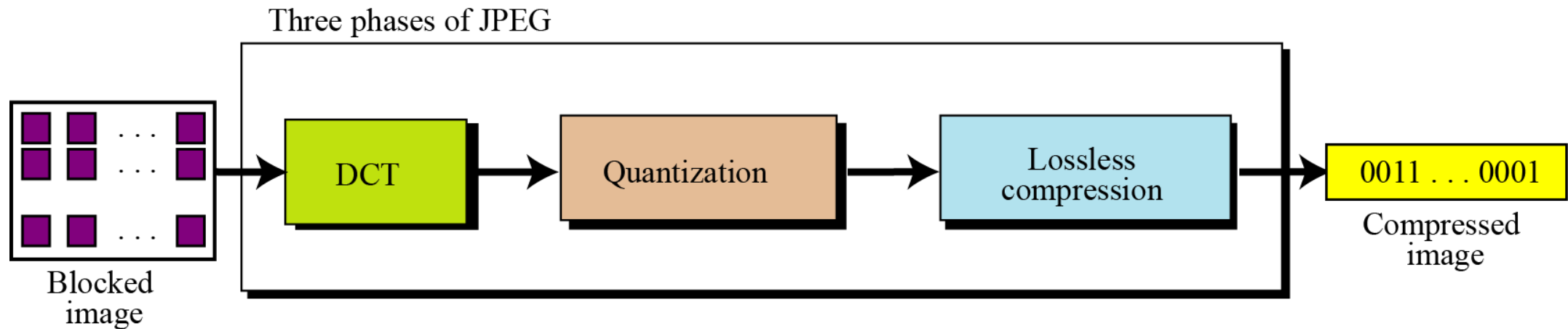
# How JPEG Works – Part 2



# JPEG Compression - Overview

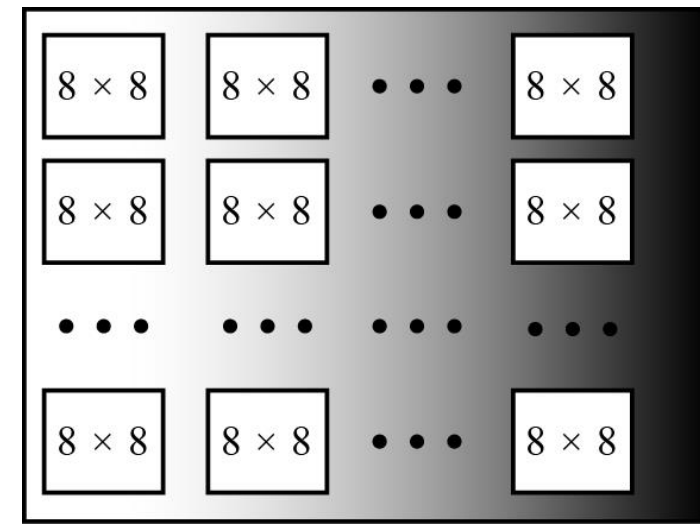


- Idea: change image into a linear (vector) set of numbers that reveals redundancies.
- Redundancies can be removed using one of the lossless compression methods



# JPEG Image Preparation

- Images – each channel – divided in data units (blocks), DCT operates on blocks to decrease the number of calculations
- Lossy mode operates on 8x8 pixel blocks (4:4:4 – no subsampling), 16×8 (4:2:2), or most commonly 16×16 (4:2:0).
- Lossless mode operates on data units of 1 pixel
- In most cases data units are processed component by component and passed to image processing
- processing order of data units
  - left-to-right, top-to-bottom
  - interleaved data ordering
- Interleaved data units of different color channels are combined to
  - **Minimum Coded Units (MCU)**



# Discrete Cosine Transform

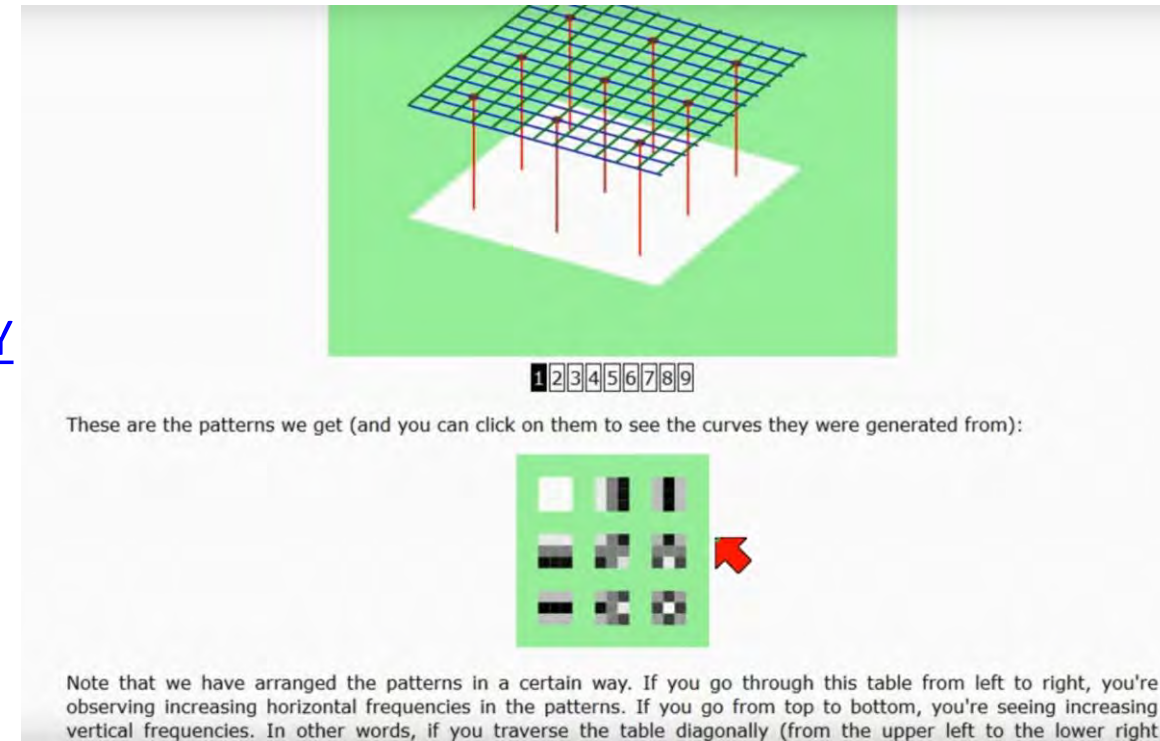
- First step: pixel values shifted into the range  $[-128, 127]$
- Forward Discrete Cosine Transformation (Forward DCT) maps values from time to frequency domain

- Interactive Website

<http://weitz.de/dct/>

Explanation:

<https://www.youtube.com/watch?v=7fhHQgu2OcY>





- DCT-based codecs use a two-dimensional version of the transform.
- The 2-D DCT of an 8 x 8 block:

$$F(u, v) = \sum_{x=0}^7 \sum_{y=0}^7 \alpha(u) \alpha(v) f(x, y) \cos\left[\frac{\pi}{8} \left(x + \frac{1}{2}\right) u\right] \cos\left[\frac{\pi}{8} \left(y + \frac{1}{2}\right) v\right]$$

$u$  is the horizontal spatial frequency, for the integers  $0 \leq u < 8$

$v$  is the vertical spatial frequency, for the integers  $0 \leq v < 8$

$$\alpha(u) = \begin{cases} \sqrt{\frac{1}{8}}, & \text{if } u = 0 \\ \sqrt{\frac{2}{8}}, & \text{otherwise} \end{cases} \quad \text{is a normalizing scale factor to make the transformation orthonormal}$$

$f(x, y)$  is the pixel value at coordinates  $(x, y)$  “space domain”

$F(u, v)$  is the DCT coefficient at coordinates  $(u, v)$  “frequency domain” (indicates how fast the information moves inside the rectangle)

◆ Note: The DCT decomposes a signal into a series of harmonic cosine functions.

# DCT Basis Functions

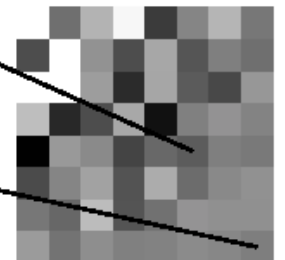
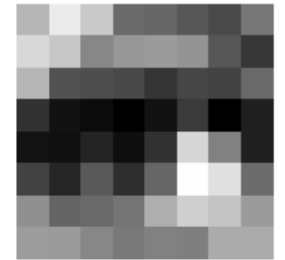
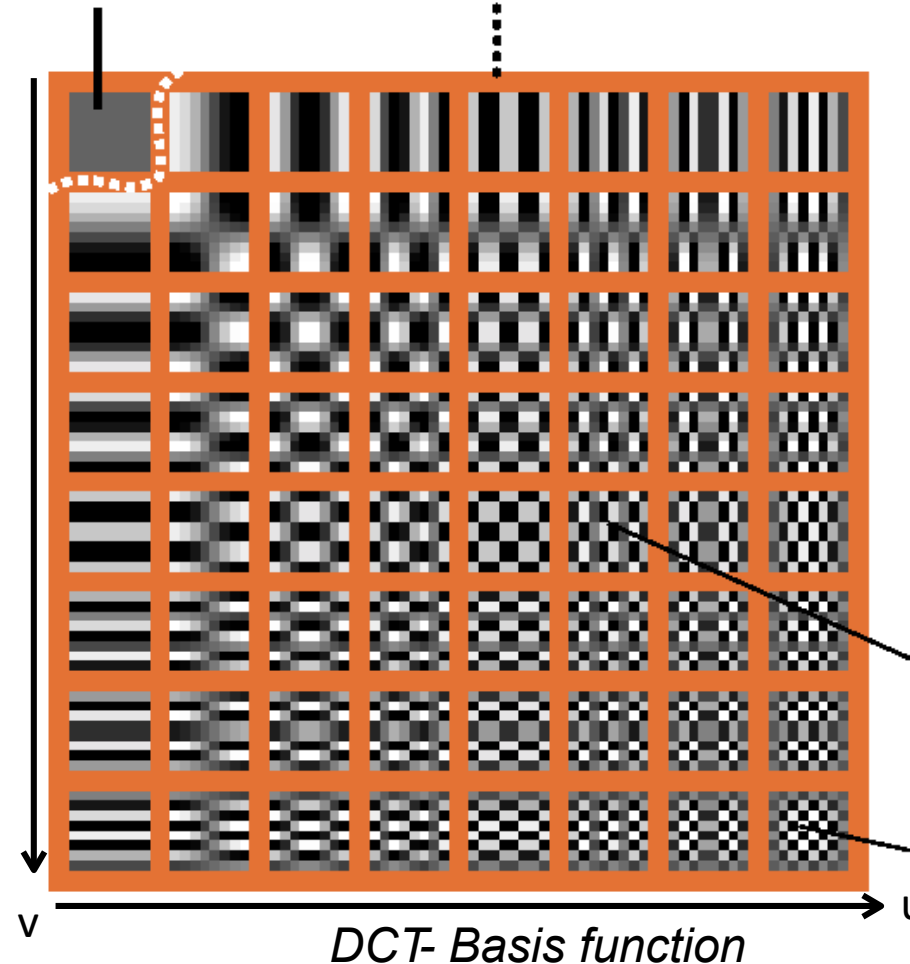


- DCT values indicate the weighting of frequency images
- The brighter the pixel, the greater the value of DCT

*Constant component*

*Alternating component*

*8 x 8 Imageblock*





# JPEG Image Processing (DCT)

- $S(u,v)$  coefficients
- $S(0,0)$ —lowest frequency in both directions is called DC coefficient
  - determines the fundamental luminance/color of the block
  - frequency = 0 in both directions
- $S(0,1), \dots, S(7,7)$  are called AC coefficients
  - frequency in one or both directions non-zero
- computing of DCT—use factoring

# Discrete Cosine Transform

Humans have only a restricted capability to perceive high frequencies. Images reduced from their high frequent elements may have a high degree of quality while being compressed quite considerably

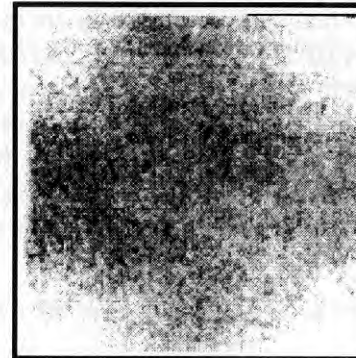
- Low frequency components of an image correspond to brief outlines of image objects
- High frequency components of an image correspond to fine structures

To decrease the high frequencies the data must be sorted by frequencies  
->DCT

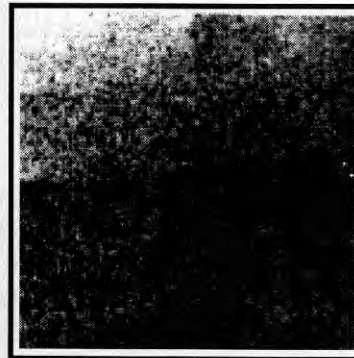
# Why DCT?



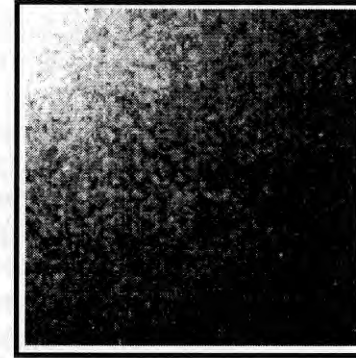
(a) Test Image



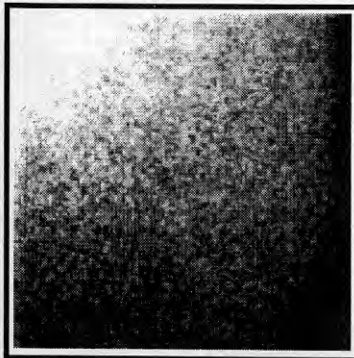
(b) Discrete Fourier transform



(c) Discrete Cosine transform



(d) Discrete Sine transform



(e) Discrete Hadamard transform



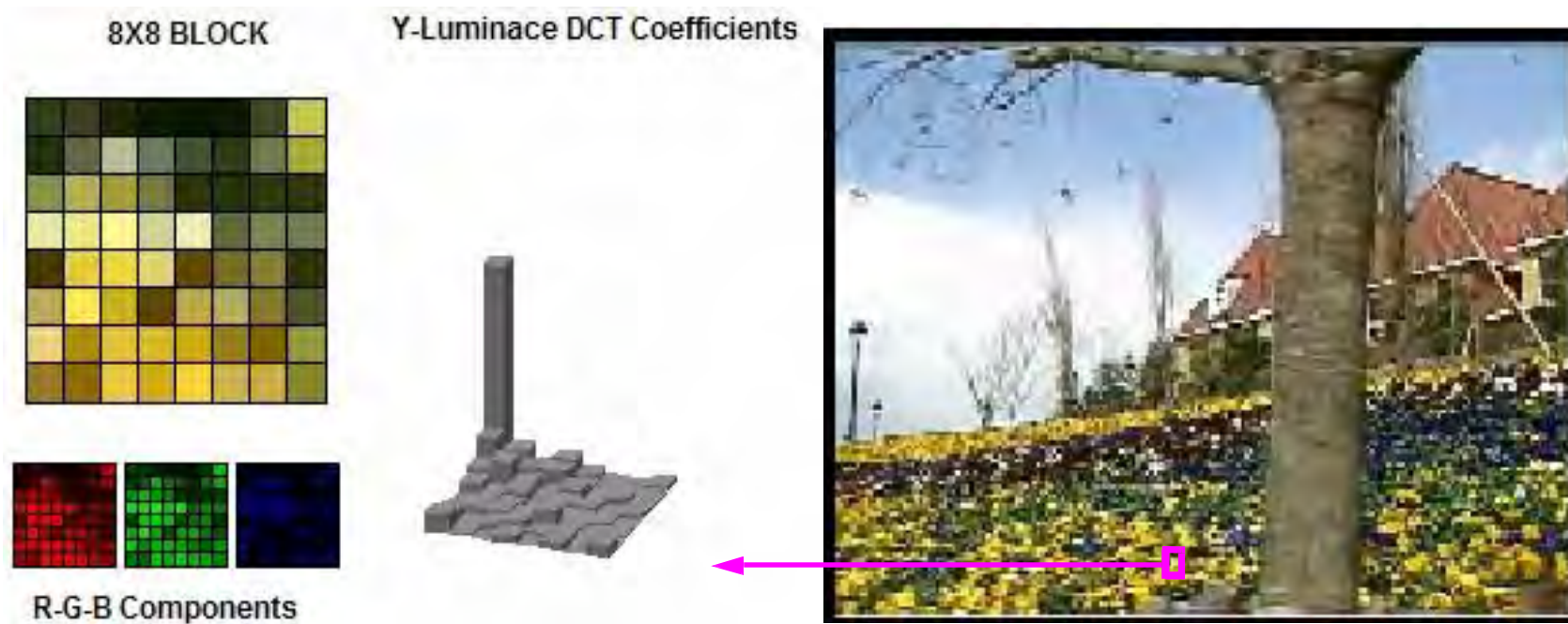
(f) Karhunen-Loeve transform

Karhunen-Loeve not used in practice for image compression because it needs to transmit the transform kernel for every block, which requires huge amount of side information.

# DCT-Coefficients

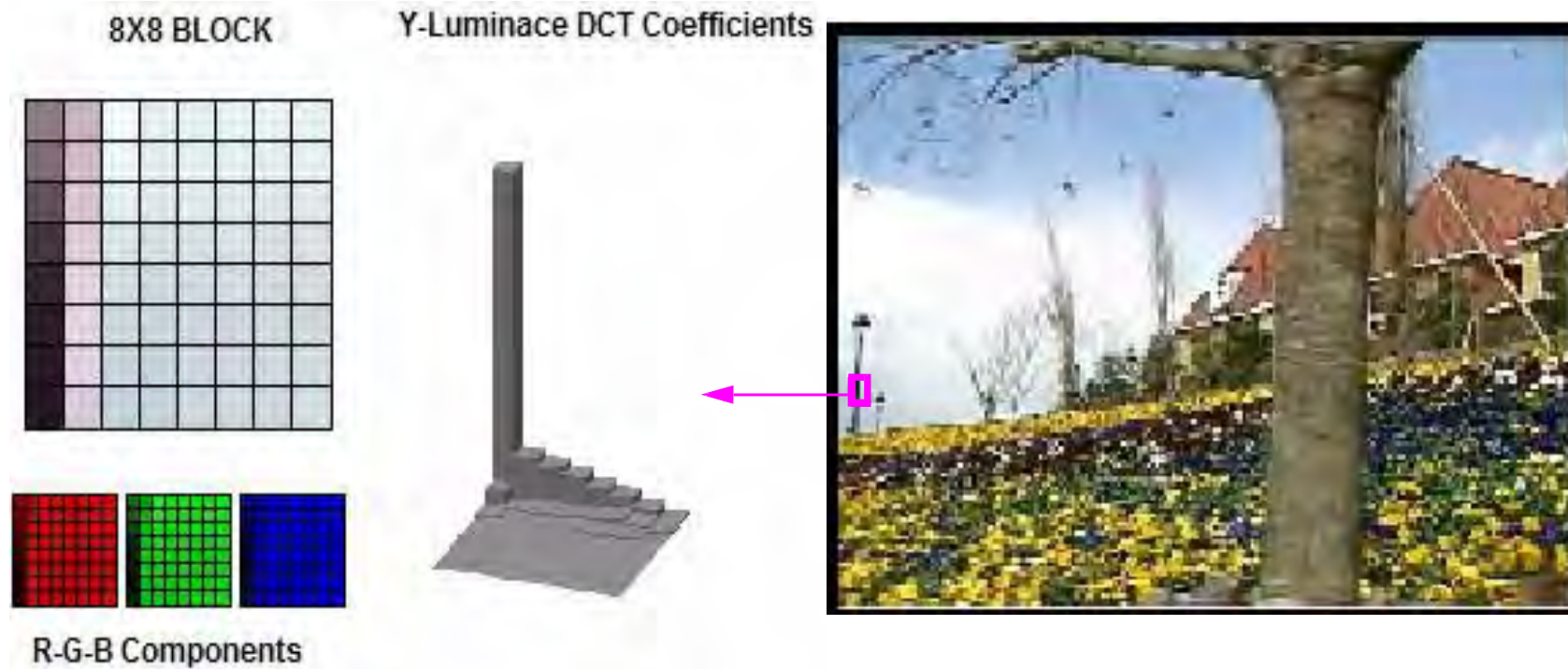


# DCT-Coefficients



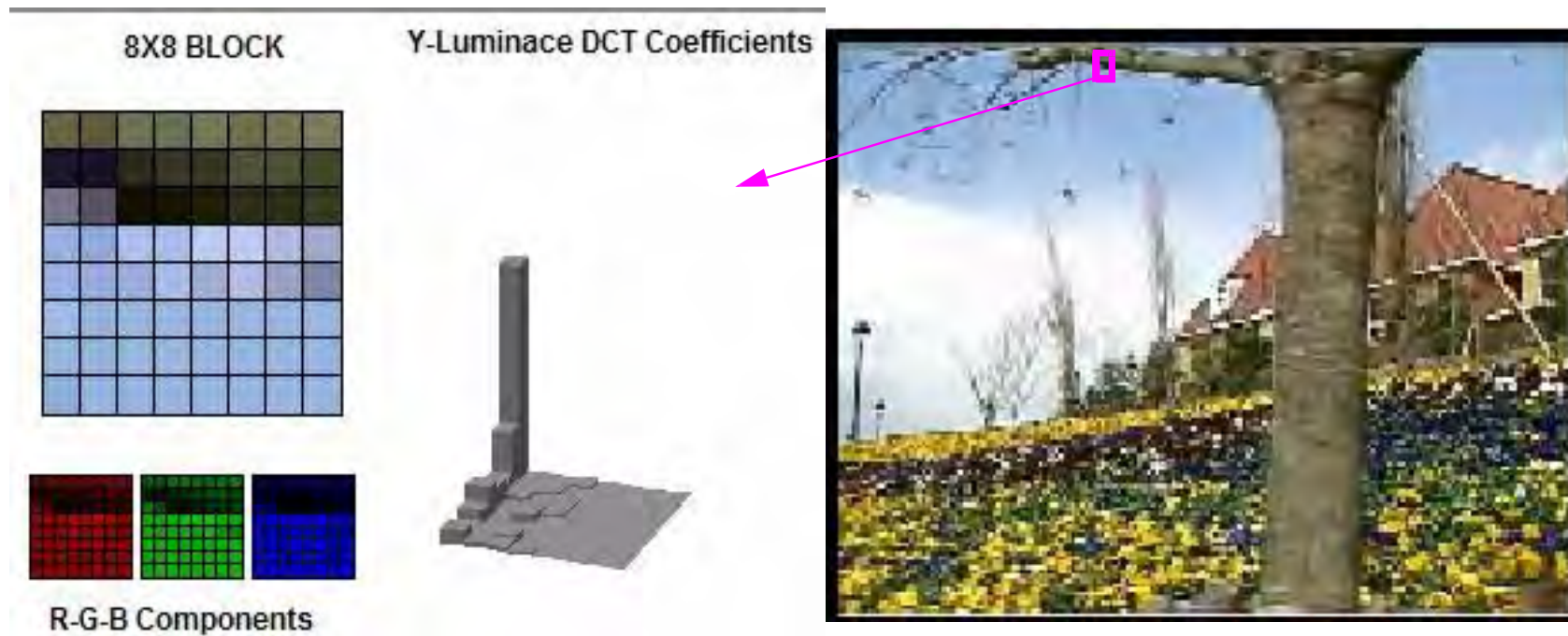


# DCT-Coefficients





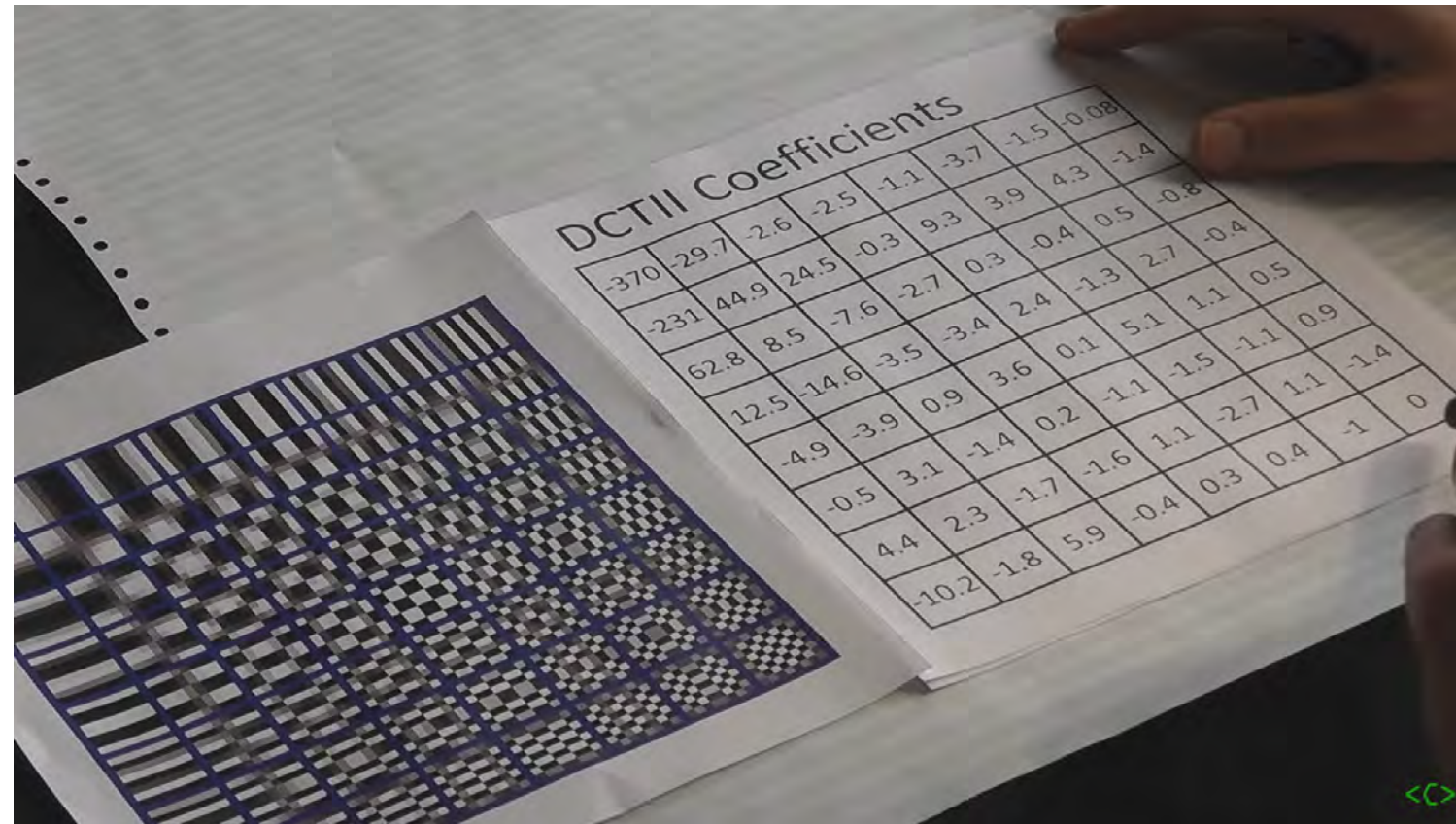
# DCT-Coefficients



# DCT-Coefficients

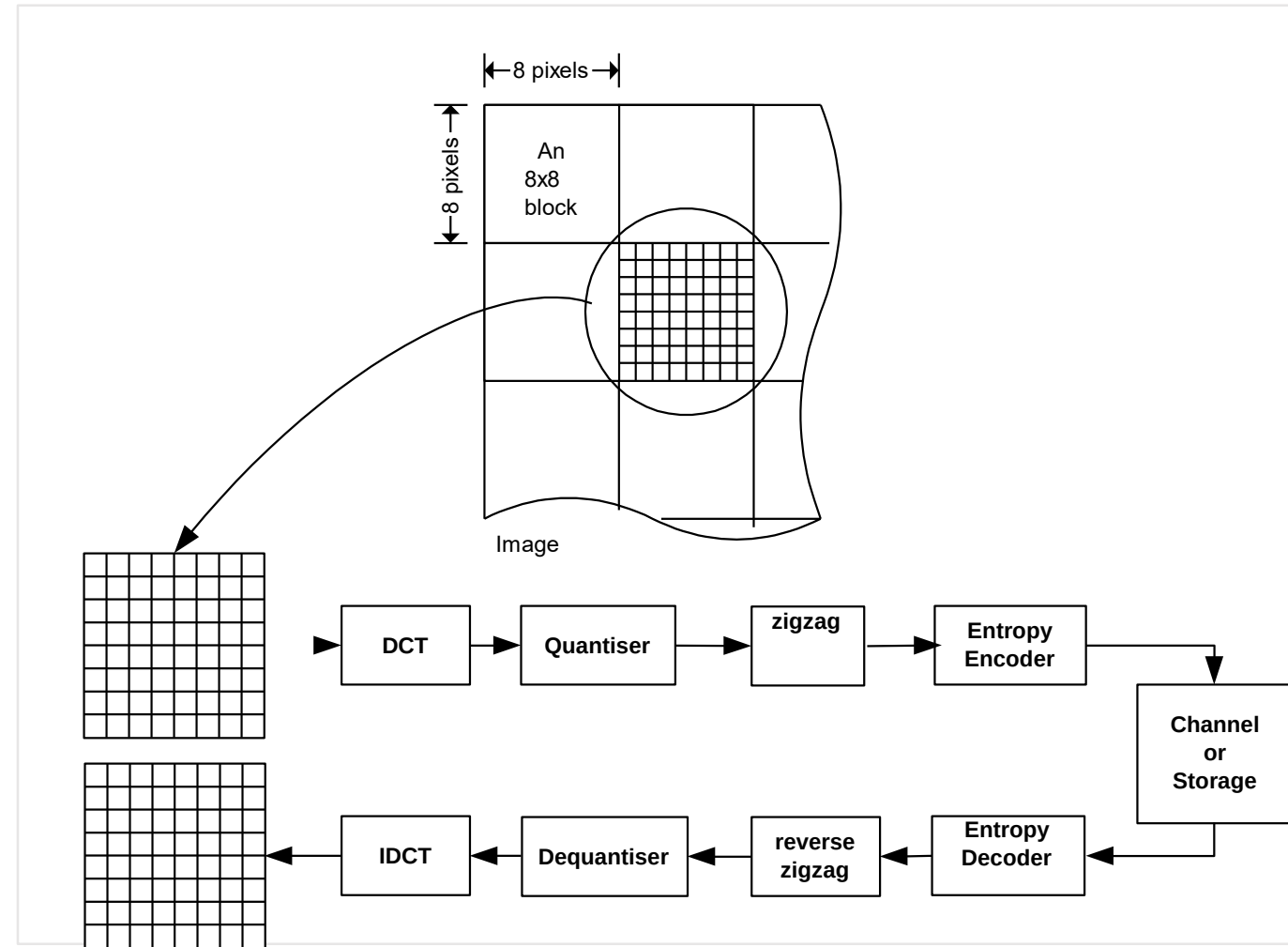


# How JPEG Works – Quantization - Part 3



# JPEG Compression

- To perform the JPEG coding, an image (in color or grey scales) is first subdivided into blocks of 8x8 pixels.
- The Discrete Cosine Transform (DCT) is then performed on each
- This generates 64 coefficients which are then quantized to reduce their magnitude.

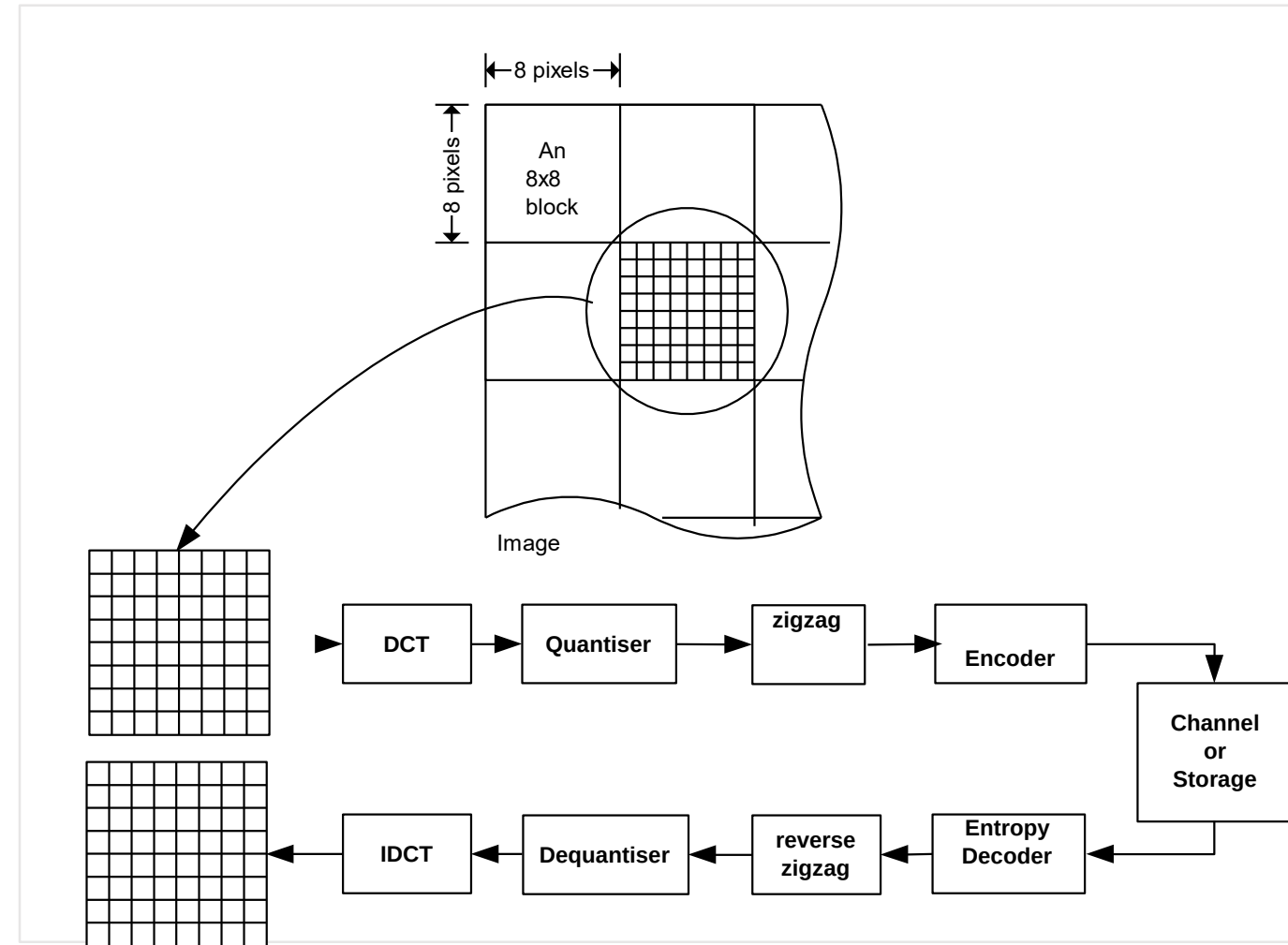




# JPEG Compression



- The coefficients are then reordered into a one-dimensional array in a zigzag manner before further entropy encoding.
- The **compression is achieved in two stages**; the first is **during quantization** and the second **during the entropy coding** process.
- JPEG decoding is the reverse process of coding.



# JPEG Quantization

- Goal: to “throw out” bits (truncation)
- Quantization in JPEG = Process of removing high frequency data
- Uniform quantization: divide coefficient values  $S(u,v)$  by  $N$  and round result

Q: In  $S(u,v)$  how many bits should be truncated?

A: use quantization tables

- quantization tables consist of 64 elements, each value uses 8-bit:  $Q_{uv}$
- new compressed values by using tables  $Sq_{uv} = S_{uv} / Q_{uv}$
- standard defines two default tables (luminance, chroma)



# JPEG Quantization

Luminance quantization table

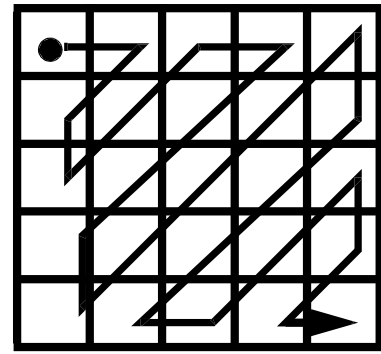
|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 16  | 11  | 12  | 14  | 12  | 10  | 16  | 14  |
| 13  | 14  | 18  | 17  | 16  | 19  | 24  | 40  |
| 26  | 24  | 22  | 22  | 24  | 49  | 35  | 37  |
| 29  | 40  | 58  | 51  | 61  | 60  | 57  | 51  |
| 56  | 55  | 64  | 72  | 92  | 78  | 64  | 68  |
| 87  | 69  | 55  | 56  | 80  | 109 | 81  | 87  |
| 95  | 98  | 103 | 104 | 103 | 62  | 77  | 113 |
| 121 | 112 | 100 | 120 | 92  | 101 | 103 | 99  |

Human eye most sensitive to low frequencies (upper left corner)

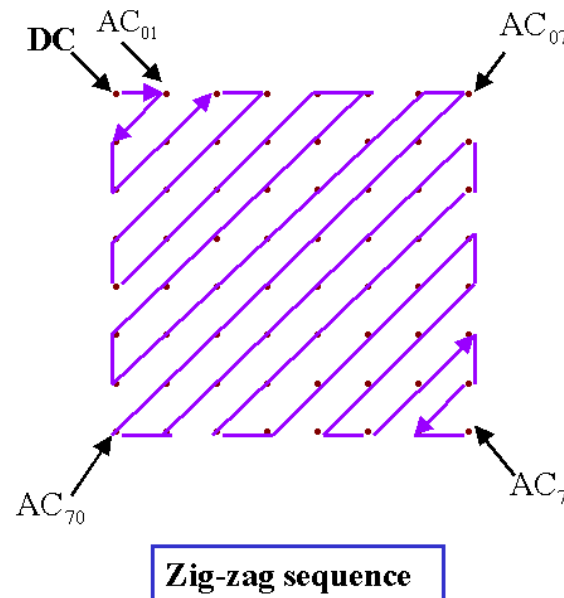
# Quantization process

- Divide DCT-coefficient value by an integer number from the quantization table
- Round the result to the nearest integer
- Quantization of all DCT-coefficients results in a *lossy* transformation – some image details given by higher frequencies are cut off.
- JPEG application provides a table with 64 entries, each used for quantization of one DCT-coefficient
  - each coefficient can be adjusted separately
- A high compression factor is achievable on the expense of image quality
  - large quantization numbers: high data reduction but information loss increases
- No default values for quantization tables are specified in JPEG

# JPEG Entropy Encoding



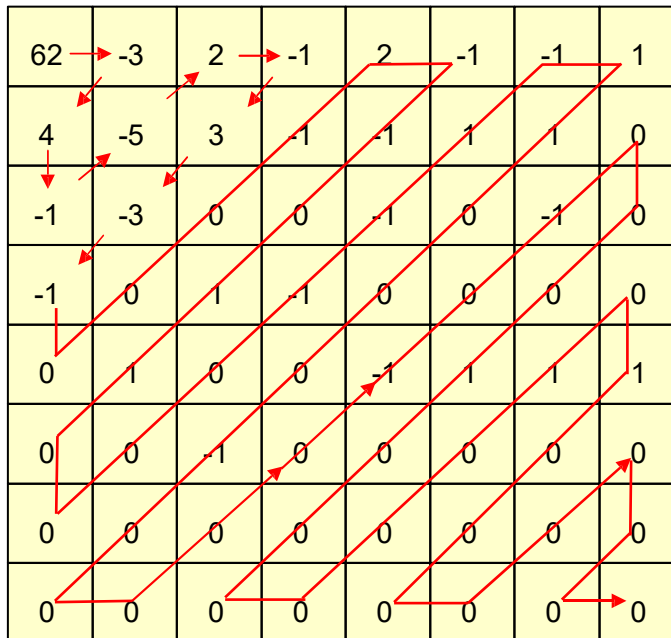
First step: map 8x8 pixel blocks into 64 element vector using zig-zag-scan



- DC coefficients processing:
  - DC coefficients determine basic color of data unit
  - DC coefficient is large, but often close to previous value  
==> encode difference

# Example

## Zig-zag ordering



*DC-coefficient*: code coefficients for one block as difference to the previous one

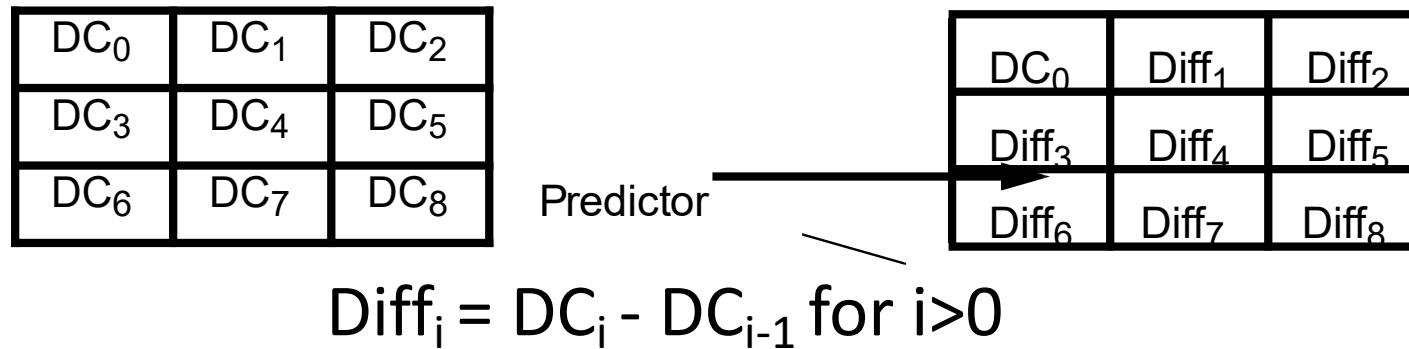
*AC-coefficients*: consider each block separately, order data using zig-zag sequence to achieve long sequences of zero- values:

$$\begin{pmatrix} -3 & 4 & -1 & -5 & 2 & -1 & 3 & -3 & -1 & 0 & 0 & 0 & -1 & 2 & -1 & -1 \\ 0 & 1 & 1 & 0 & 0 & 0 & 0 & -1 & -1 & 1 & -1 & 1 & 1 & 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 & 0 & -1 & 0 & -1 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

*Entropy encoding:*

- Run-length encoding of zero values of quantized AC-coefficients
- Huffman encoding on DC- and AC-coefficients

# JPEG Entropy Encoding



- AC coefficients processing:
  - processing order of the AC coefficients using zig-zag scan (coefficients with lower frequencies are encoded first) ==>
  - sequence of similar data bytes ==> efficient entropy coding
- JPEG standard specifies Huffman or Arithmetic coding, but sequential encoding mode uses only Huffman coding.

# JPEG — Entropy Encoding

Algorithm:

1. apply run-length coding of AC coefficient of zero values
2. apply Huffman coding on DC and AC coefficients

(only DC coefficients explained in detail)



# JPEG — Entropy Encoding

- DC coefficients—Huffman coding
  - categorize DC values into DC code tables
  - difference magnitude categories for DC coefficients (12 categories)

| Diff Values      | SSSS (number of bits<br>needed to encode Diff) |
|------------------|------------------------------------------------|
| 0                | 0 bit                                          |
| -1, 1            | 1 bit                                          |
| -3, ..., 3       | 2 bits                                         |
| -7, ..., 7       | 3 bits                                         |
| .                |                                                |
| .                |                                                |
| .                |                                                |
| -2047, ..., 2047 | 11 bits                                        |

# JPEG — Entropy Encoding

- DC coefficients — Huffman encoding
  - handle SSSS as Huffman symbol, get  $p(0)$ ,  $p(1)$ , ...  $p(11)$  and create Huffman tree ==> Huffman code for SSSS
  - for each category, an additional bits field is appended to the code word to uniquely identify which difference in that category actually occurred.
  - send: (Huffman codeword, actual value)

Example: If SSSS=2 has the Huffman code 001 and Diff=-3 ==>

send 00100

Huffman code for 2

-3 in two's complement is 00  
because  $3 \Rightarrow 11$

## Details about JPEG Entropy Coding in

<https://cnx.org/contents/IJvrSfU5@3.3:eMc9DKWD@5/JPEG-Entropy-Coding>

# JPEG Image Processing (DCT) - Decoding

- Inverse Discrete Cosine Transformation (I-DCT) maps DCT coefficients to sampled values

$$s_{xy} = \frac{1}{4} \sum_{u=0}^7 \sum_{v=0}^7 c_u c_v s_{vu} \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}$$

with  $c_u, c_v = 1/\sqrt{2}$  for  $u,v=0$ ; otherwise  $c_u, c_v = 1$

- DCT and I-DCT cannot be calculated in full precision  
==> lossy compression
- JPEG does not define precision parameters, therefore various implementations exist
- many AC coefficients with small values (around zero)

# Comments

- Applications do not have to include both an encoder and decoder if the compression process agrees on a common table
- The encoded data stream has a fixed *Interchange Format*
  - encoded image data
  - chosen parameters
  - tables of the coding process
- In regular mode, the interchange format includes all of the information necessary for decoding without any previous knowledge of the coding process

# DCT Examples



- DCT sorts values from the lowest to the highest frequency
- In image blocks it is likely that the energy is concentrated in low frequencies
  - Regions with smooth colors or little details (= low spatial frequencies) have high values
  - Regions with different colors and detail (= high spatial frequencies), most values are almost zero.



8x8 Block (Luminance)

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 134 | 142 | 145 | 131 | 114 | 122 | 131 | 130 |
| 129 | 143 | 134 | 130 | 135 | 144 | 134 | 118 |
| 123 | 117 | 118 | 111 | 97  | 109 | 130 | 143 |
| 129 | 116 | 112 | 116 | 120 | 126 | 130 | 118 |
| 118 | 127 | 141 | 138 | 138 | 148 | 141 | 125 |
| 125 | 129 | 119 | 127 | 143 | 149 | 145 | 136 |
| 131 | 126 | 128 | 142 | 141 | 135 | 126 | 116 |
| 131 | 140 | 146 | 154 | 133 | 118 | 124 | 124 |

Pixelvalues

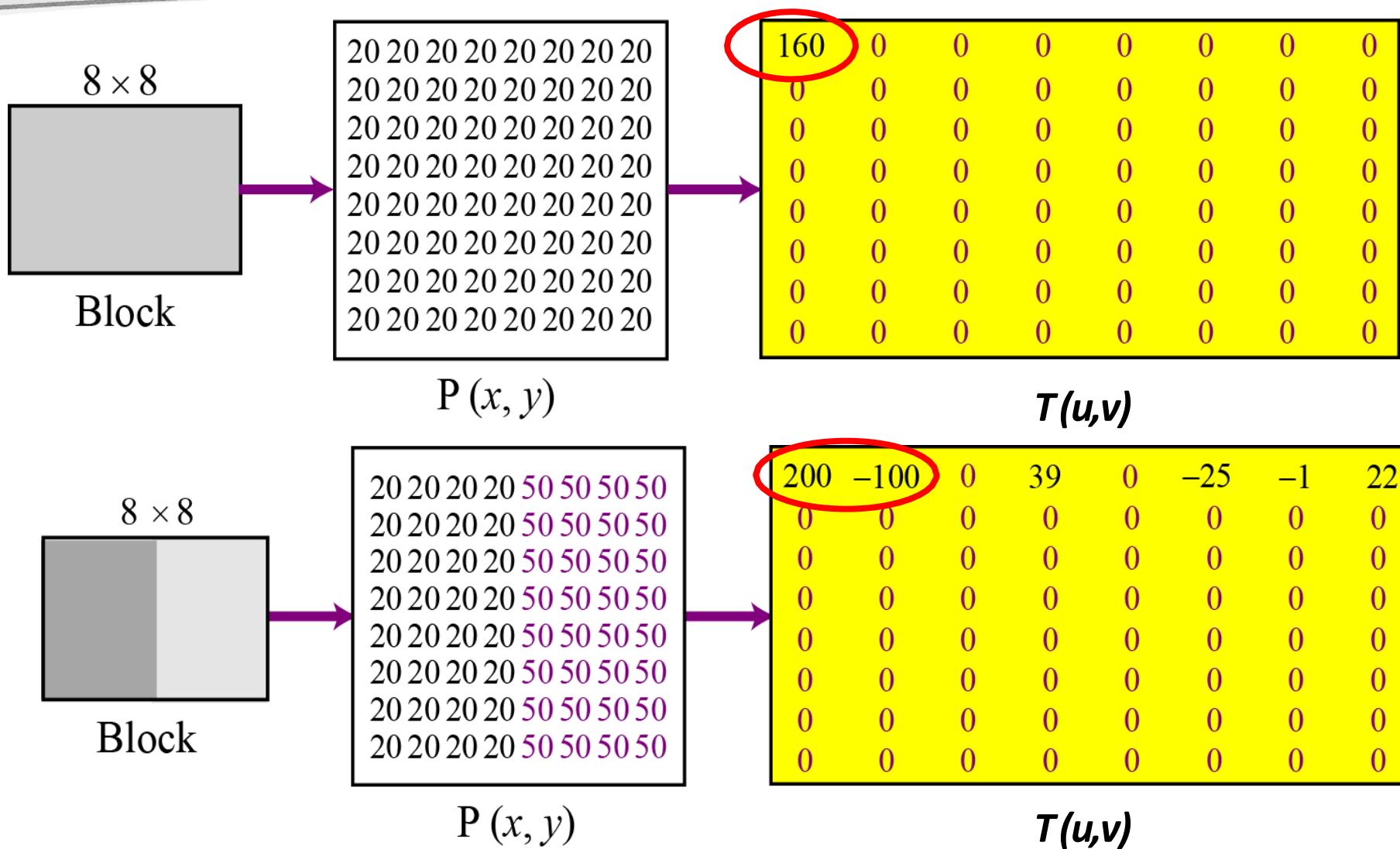
Low frequencies

|      |     |     |     |     |    |    |    |
|------|-----|-----|-----|-----|----|----|----|
| 1037 | -1  | -6  | 1   | -12 | 8  | -4 | -4 |
| -16  | 1   | 28  | -6  | -14 | 0  | 4  | 0  |
| 19   | 32  | -7  | -19 | 2   | -1 | -4 | -3 |
| 29   | -9  | -14 | 13  | -10 | -6 | 1  | 0  |
| 4    | 14  | -6  | -13 | -2  | 7  | 1  | 2  |
| -26  | 2   | 16  | 2   | 11  | 6  | 1  | 1  |
| -10  | -11 | 27  | -18 | 4   | 1  | 0  | 0  |
| -2   | 1   | 1   | -19 | -1  | 6  | 6  | 0  |

High frequencies

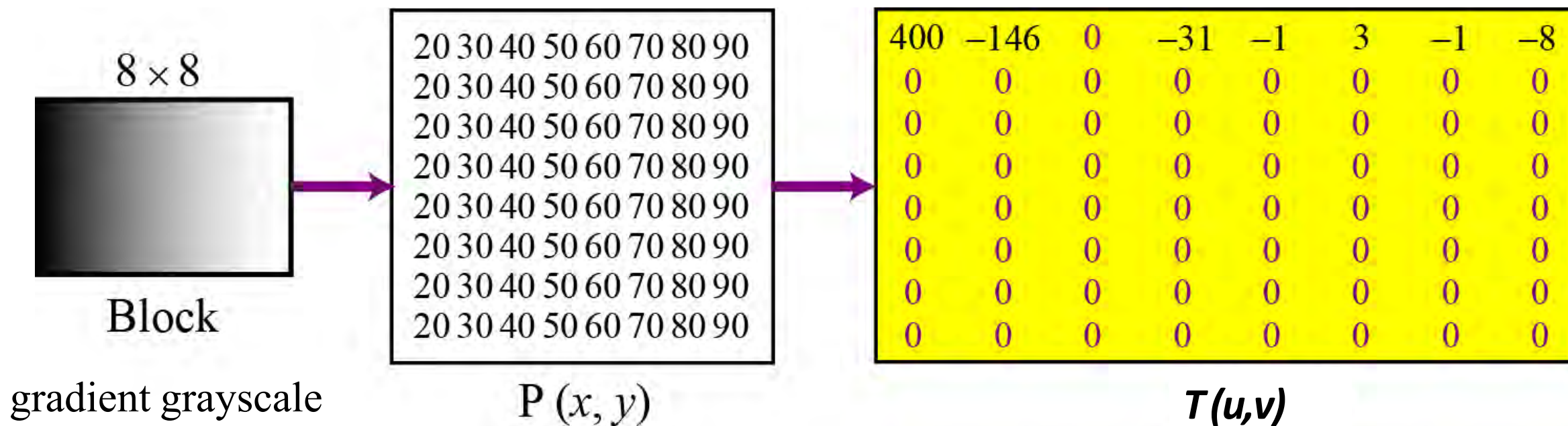
DCT- Values

# DCT Examples





# DCT Examples



# DCT Basis Functions

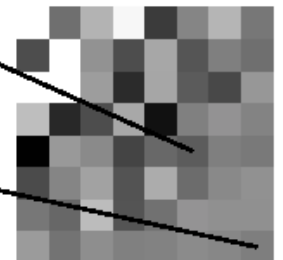
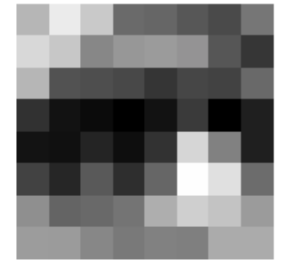
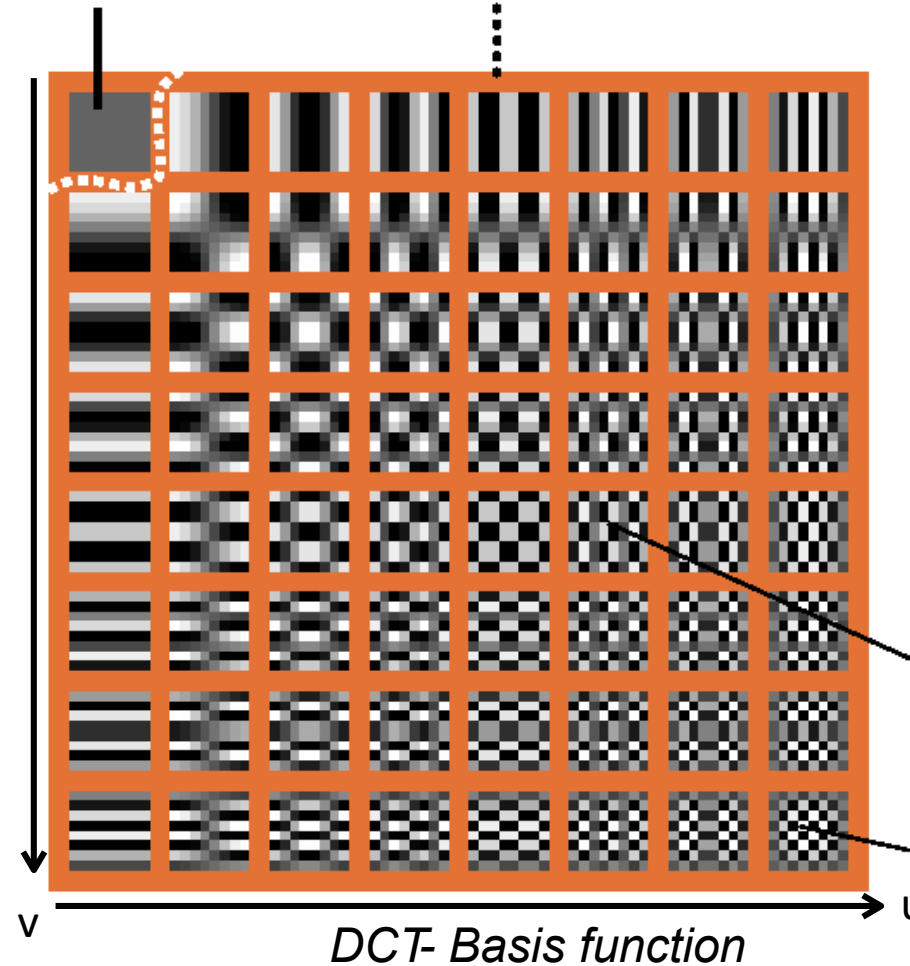


- DCT values indicate the weighting of frequency images
- The brighter the pixel, the greater the value of DCT

*Constant component*

*Alternating component*

*8 x 8 Imageblock*



# Quantization



- Compression effect: DCT values are approximated (quantized) to make them smaller and to repeat themselves
  - Each DCT value is divided by a quantization factor and then rounded
  - The larger the quantization factor, the smaller the values to be stored

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 134 | 142 | 145 | 131 | 114 | 122 | 131 | 130 |
| 129 | 143 | 134 | 130 | 135 | 144 | 134 | 118 |
| 123 | 117 | 118 | 111 | 97  | 109 | 130 | 143 |
| 129 | 116 | 112 | 116 | 120 | 126 | 130 | 118 |
| 118 | 127 | 141 | 138 | 138 | 148 | 141 | 125 |
| 125 | 129 | 119 | 127 | 143 | 149 | 145 | 136 |
| 131 | 126 | 128 | 142 | 141 | 135 | 126 | 116 |
| 131 | 140 | 146 | 154 | 133 | 118 | 124 | 124 |

*Pixelvalues*

|      |     |     |     |     |    |    |    |
|------|-----|-----|-----|-----|----|----|----|
| 1037 | -1  | -6  | 1   | -12 | 8  | -4 | -4 |
| -16  | 1   | 28  | -6  | -14 | 0  | 4  | 0  |
| 19   | 32  | -7  | -19 | 2   | -1 | -4 | -3 |
| 29   | -9  | -14 | 13  | -10 | -6 | 1  | 0  |
| 4    | 14  | -6  | -13 | -2  | 7  | 1  | 2  |
| -26  | 2   | 16  | 2   | 11  | 6  | 1  | 1  |
| -10  | -11 | 27  | -18 | 4   | 1  | 0  | 0  |
| -2   | 1   | 1   | -19 | -1  | 6  | 6  | 0  |

*DCT- Values*

|     |    |    |    |    |   |   |   |
|-----|----|----|----|----|---|---|---|
| 130 | 0  | 0  | 0  | -1 | 0 | 0 | 0 |
| -2  | 0  | 3  | 0  | -1 | 0 | 0 | 0 |
| 2   | 3  | 0  | -1 | 0  | 0 | 0 | 0 |
| 3   | -1 | -1 | 1  | 0  | 0 | 0 | 0 |
| 0   | 1  | 0  | -1 | 0  | 0 | 0 | 0 |
| -2  | 0  | 2  | 0  | 0  | 0 | 0 | 0 |
| -1  | -1 | 2  | -1 | 0  | 0 | 0 | 0 |
| 0   | 0  | 0  | -1 | 0  | 0 | 0 | 0 |

*Quantized DCT- Values*

# Quantization Matrix



- Quantization factor for each DCT value is defined using a quantization matrix
  - Disturbances in low frequency parts of the image are perceived strongly
  - Disturbances in high frequency parts of the image are less noticeable
  - Quantization matrices in JPEG standard ensures that the DCT values for low frequencies are stored more accurately than for high frequencies
  - Quantization table can be freely selected in the compression

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 8  | 16 | 19 | 22 | 26 | 27 | 29 | 34 |
| 16 | 16 | 22 | 24 | 27 | 29 | 34 | 37 |
| 19 | 22 | 26 | 27 | 29 | 34 | 34 | 38 |
| 22 | 22 | 26 | 27 | 29 | 34 | 37 | 40 |
| 22 | 26 | 27 | 29 | 32 | 35 | 40 | 48 |
| 26 | 27 | 29 | 32 | 35 | 40 | 48 | 58 |
| 26 | 27 | 29 | 34 | 38 | 46 | 56 | 69 |
| 27 | 29 | 35 | 38 | 46 | 56 | 69 | 83 |

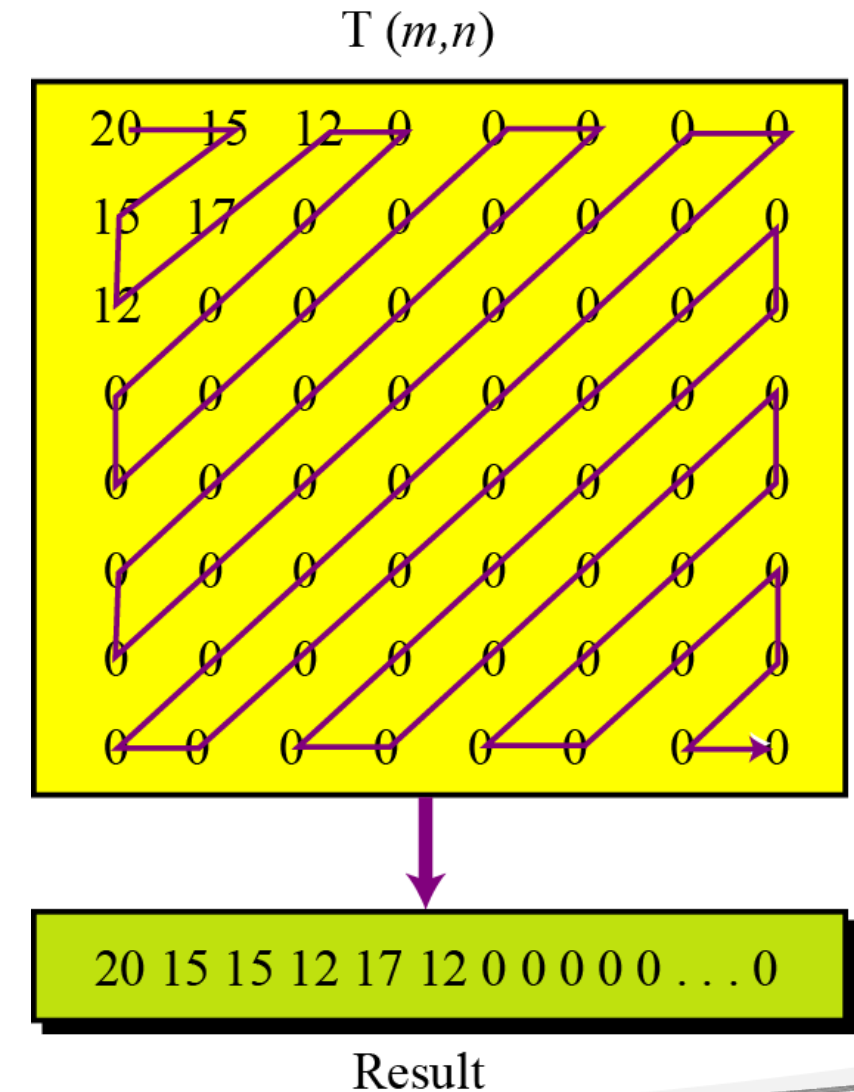
*Quantization Matrix*

|    |    |    |    |     |     |     |     |
|----|----|----|----|-----|-----|-----|-----|
| 16 | 11 | 10 | 16 | 24  | 40  | 51  | 61  |
| 12 | 12 | 14 | 19 | 26  | 58  | 60  | 55  |
| 14 | 13 | 16 | 24 | 40  | 57  | 69  | 56  |
| 14 | 17 | 22 | 29 | 51  | 87  | 80  | 62  |
| 18 | 22 | 37 | 56 | 68  | 109 | 103 | 77  |
| 24 | 35 | 55 | 64 | 81  | 104 | 113 | 92  |
| 49 | 64 | 78 | 87 | 103 | 121 | 120 | 101 |
| 72 | 92 | 95 | 98 | 112 | 100 | 103 | 99  |

*Quantization Matrix*

# Compression

- After quantization values are read from the table, and redundant 0s are removed.
- To cluster 0s together process reads the table diagonally in a zigzag fashion because if image does not have fine changes bottom right corner of table is all 0s.
- JPEG uses run-length encoding at the compression phase to compress the bit pattern resulting from the zigzag linearization.



# Quantization



## Coding

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 134 | 142 | 145 | 131 | 114 | 122 | 131 | 130 |
| 129 | 143 | 134 | 130 | 135 | 144 | 134 | 118 |
| 123 | 117 | 118 | 111 | 97  | 109 | 130 | 143 |
| 129 | 116 | 112 | 116 | 120 | 126 | 130 | 118 |
| 118 | 127 | 141 | 138 | 138 | 148 | 141 | 125 |
| 125 | 129 | 119 | 127 | 143 | 149 | 145 | 136 |
| 131 | 126 | 128 | 142 | 141 | 135 | 126 | 116 |
| 131 | 140 | 146 | 154 | 133 | 118 | 124 | 124 |

*Pixelvalues*

|      |     |     |     |     |    |    |    |
|------|-----|-----|-----|-----|----|----|----|
| 1037 | -1  | -6  | 1   | -12 | 8  | -4 | -4 |
| -16  | 1   | 28  | -6  | -14 | 0  | 4  | 0  |
| 19   | 32  | -7  | -19 | 2   | -1 | -4 | -3 |
| 29   | -9  | -14 | 13  | -10 | -6 | 1  | 0  |
| 4    | 14  | -6  | -13 | -2  | 7  | 1  | 2  |
| -26  | 2   | 16  | 2   | 11  | 6  | 1  | 1  |
| -10  | -11 | 27  | -18 | 4   | 1  | 0  | 0  |
| -2   | 1   | 1   | -19 | -1  | 6  | 6  | 0  |

*DCT- Values*

|     |    |    |    |    |   |   |   |
|-----|----|----|----|----|---|---|---|
| 130 | 0  | 0  | 0  | -1 | 0 | 0 | 0 |
| -2  | 0  | 3  | 0  | -1 | 0 | 0 | 0 |
| 2   | 3  | 0  | -1 | 0  | 0 | 0 | 0 |
| 3   | -1 | -1 | 1  | 0  | 0 | 0 | 0 |
| 0   | 1  | 0  | -1 | 0  | 0 | 0 | 0 |
| -2  | 0  | 2  | 0  | 0  | 0 | 0 | 0 |
| -1  | -1 | 2  | -1 | 0  | 0 | 0 | 0 |
| 0   | 0  | 0  | -1 | 0  | 0 | 0 | 0 |

*Quantized DCT- Values*

## Decoding

|      |     |    |     |     |   |   |   |
|------|-----|----|-----|-----|---|---|---|
| 1040 | 0   | 0  | 0   | -9  | 0 | 0 | 0 |
| -12  | 0   | 24 | 0   | -10 | 0 | 0 | 0 |
| 14   | 24  | 0  | -10 | 0   | 0 | 0 | 0 |
| 24   | -8  | -9 | 10  | 0   | 0 | 0 | 0 |
| 0    | 9   | 0  | -10 | 0   | 0 | 0 | 0 |
| -19  | 0   | 10 | 0   | 0   | 0 | 0 | 0 |
| -9   | -10 | 21 | -12 | 0   | 0 | 0 | 0 |
| 0    | 0   | 0  | -14 | 0   | 0 | 0 | 0 |

*Reconstructed DCT- Values*

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 136 | 141 | 138 | 125 | 119 | 125 | 132 | 134 |
| 136 | 137 | 133 | 130 | 134 | 139 | 133 | 121 |
| 121 | 125 | 122 | 112 | 107 | 117 | 130 | 138 |
| 125 | 123 | 119 | 117 | 121 | 128 | 127 | 122 |
| 123 | 129 | 135 | 139 | 140 | 139 | 136 | 131 |
| 129 | 125 | 124 | 130 | 139 | 144 | 141 | 136 |
| 129 | 130 | 134 | 139 | 140 | 135 | 127 | 120 |
| 132 | 138 | 144 | 141 | 131 | 122 | 122 | 127 |

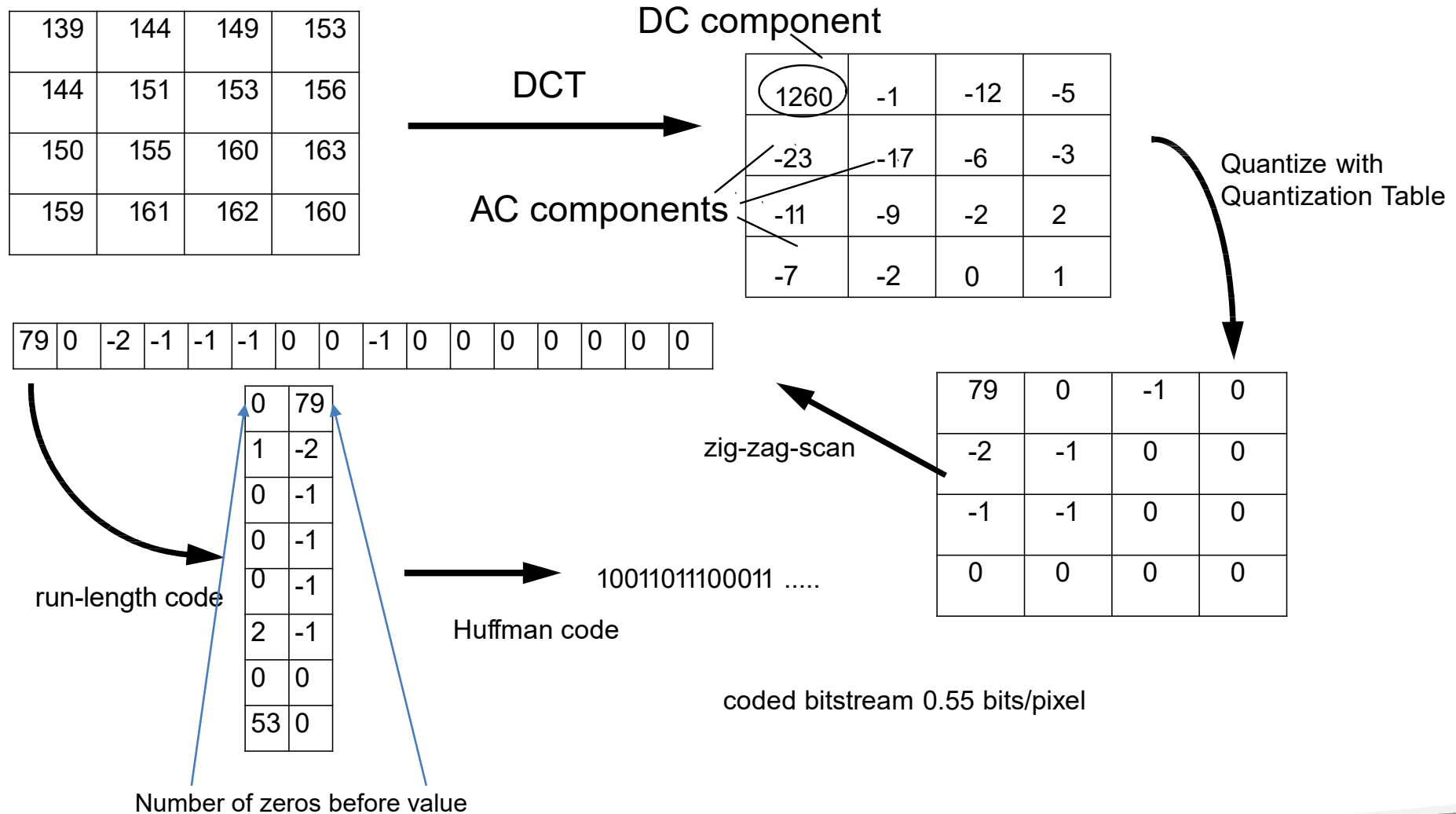
*Reconstructed Pixels (IDCT)*

|    |    |    |    |     |    |    |    |
|----|----|----|----|-----|----|----|----|
| -2 | 1  | 7  | 6  | -5  | -3 | -1 | -4 |
| -7 | 6  | 1  | 0  | 1   | 5  | 1  | -3 |
| 2  | -8 | -4 | -1 | -10 | -8 | 0  | 5  |
| 4  | -7 | -7 | -1 | -1  | -2 | 3  | -4 |
| -5 | -2 | 6  | -1 | -2  | 9  | 5  | -6 |
| -4 | 4  | -5 | -3 | 4   | 5  | 4  | 0  |
| 2  | -4 | -6 | 3  | 1   | 0  | -1 | -4 |
| 1  | 2  | 2  | 13 | 2   | -4 | 1  | -3 |

*Errors (Difference)*



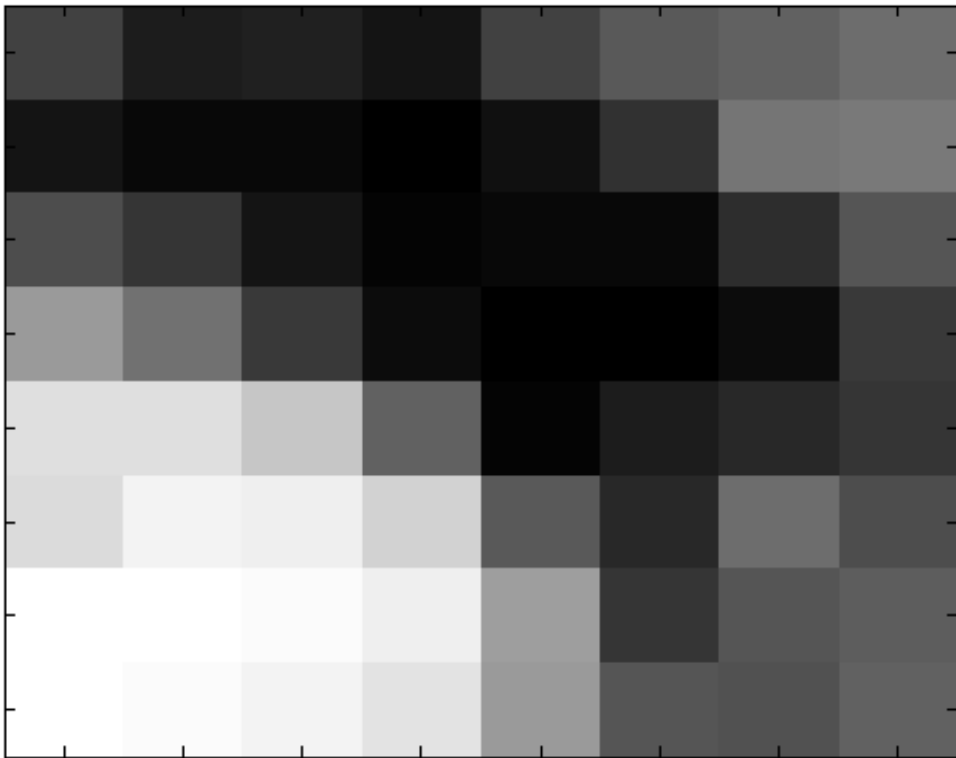
# JPEG Block Encoding (Example)



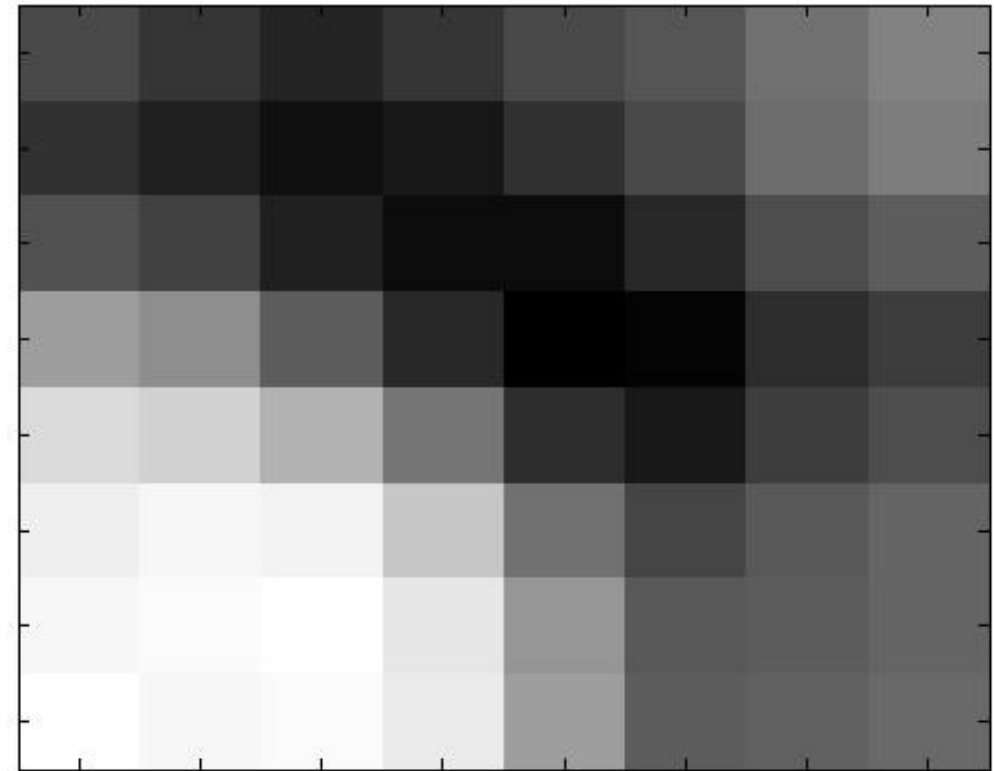
# Can You Tell the Difference?



Original



Compressed



# Image Compression



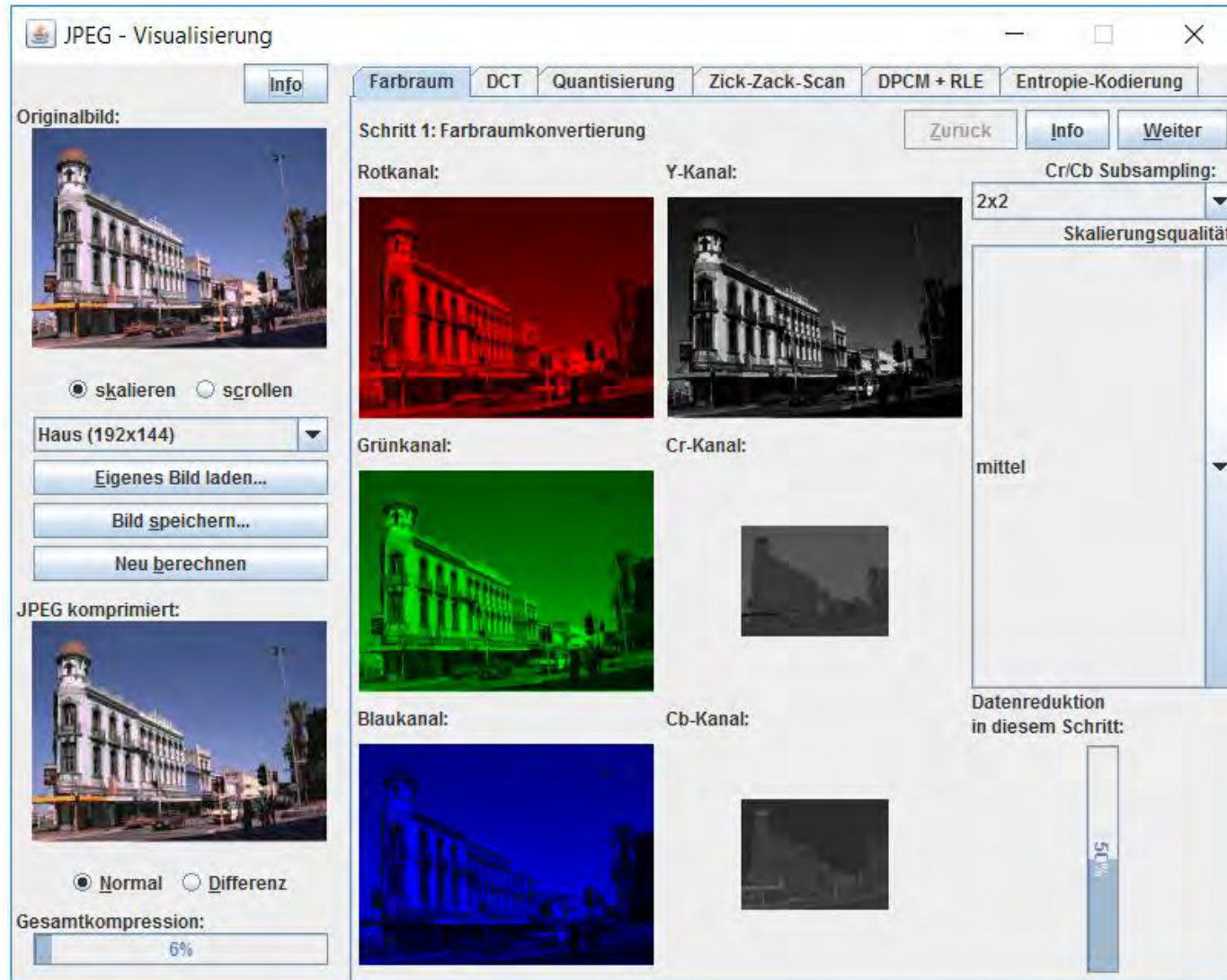
Original



Compressed



# VisJPEG-Applet





# Problems with JPEG

- Behavior in sharp transitions (e.g. fonts)
- Emergence of the 8x8 blocks at high compression rates.
- The greater the compression (quantization), the more artefacts occur (block formation)
- JPEG is made for natural images and not for artificial images (computer graphics)!





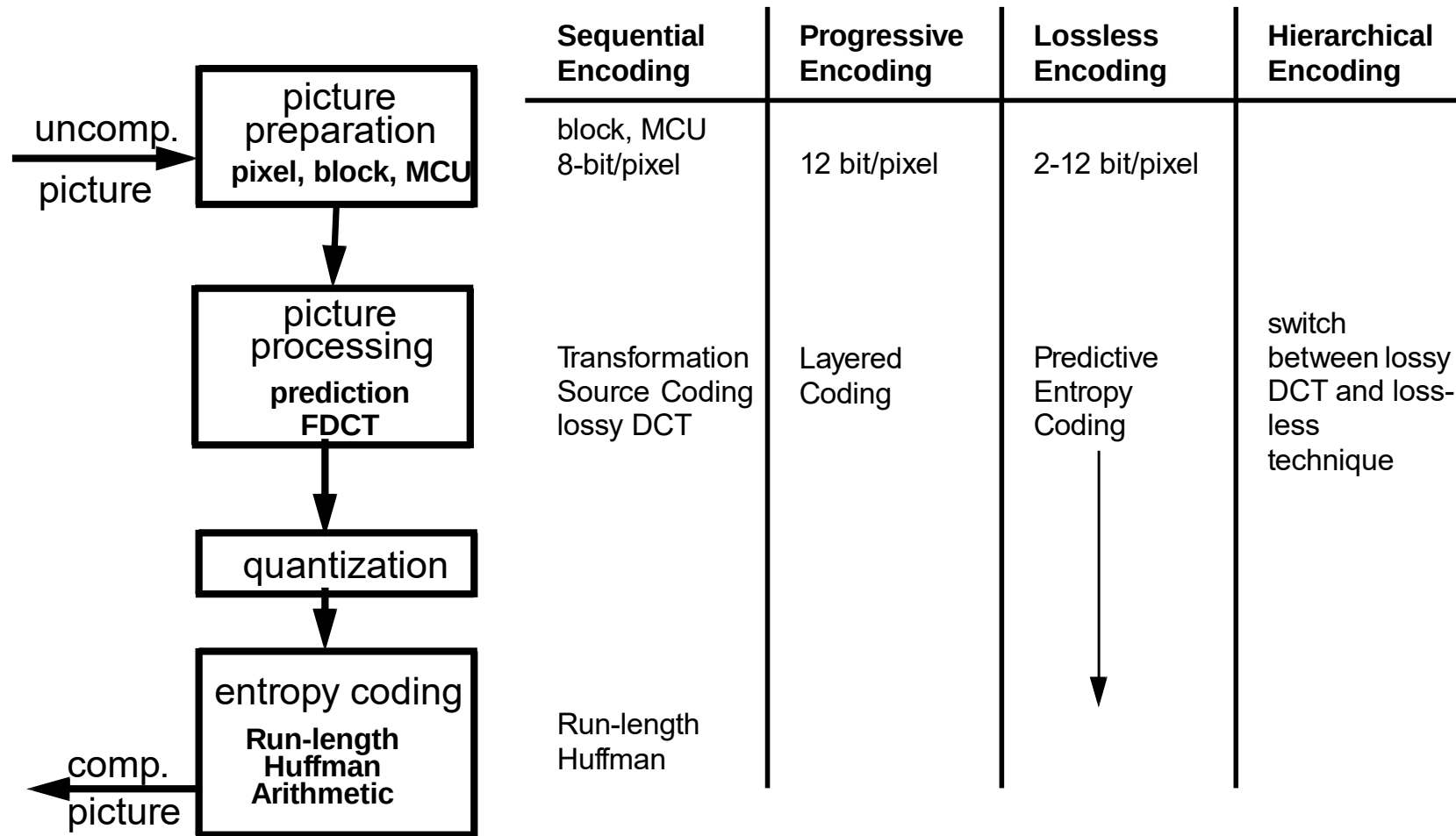


# Variations & Additions to JPEG

## Additional Material (not for Exam!)

Slides by Prof. Dr. Otto Spaniol, Dirk Thißen  
RWTH Aachen

# JPEG Processing Steps

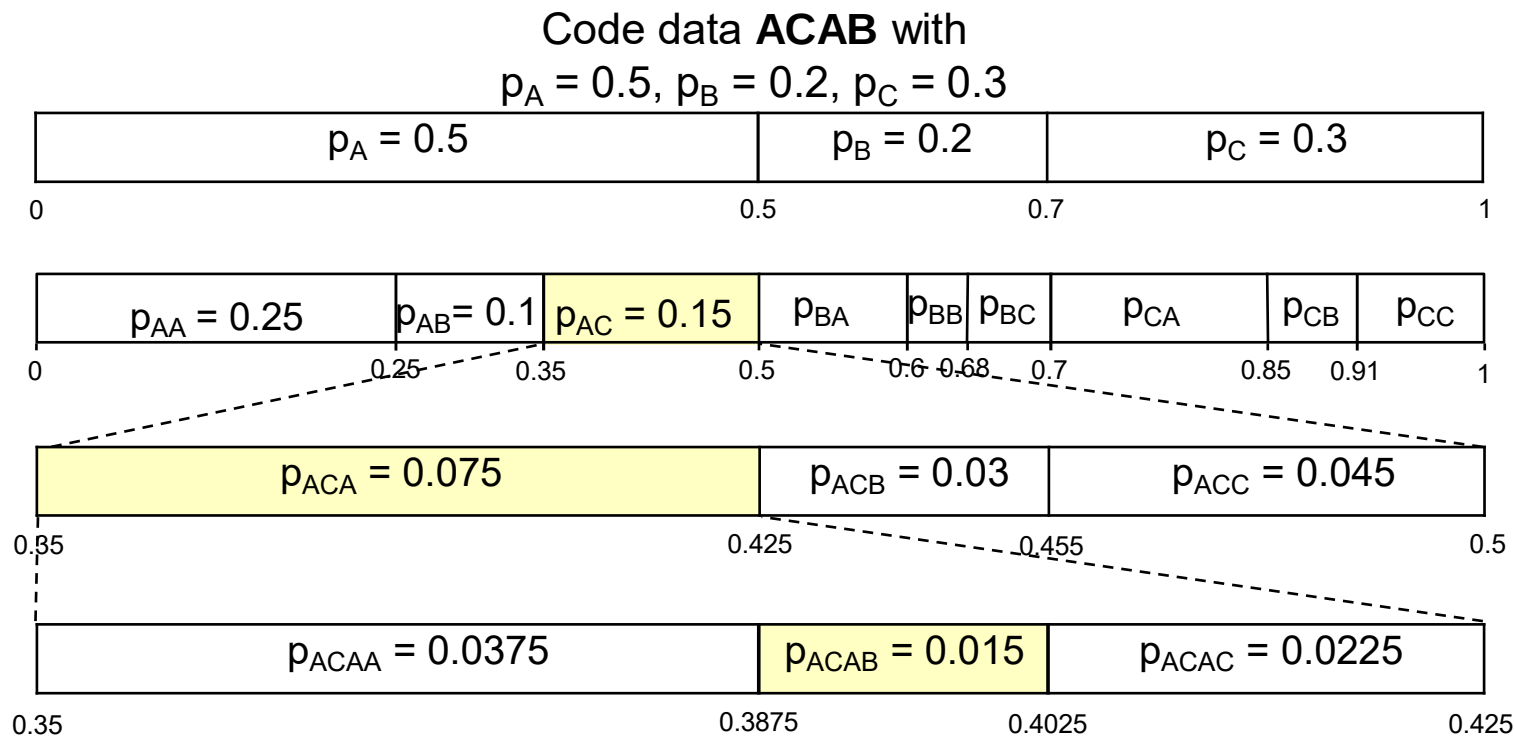


## Alternative to Huffman: Arithmetic Coding

### Characteristics:

- Achieves optimality (coding rate) as the Huffman coding
- Difference to Huffman: the entire data stream has an assigned probability, which consists of the probabilities of the contained characters. Coding a character takes place with consideration of all previous characters.
- The data are coded as an interval of real numbers between 0 and 1. Each value within the interval can be used as code word.
- The minimum length of the code is determined by the assigned probability.
- Disadvantage: the data stream can be decoded only as a whole.

## Arithmetic Coding: Example



ACAB can be coded by each binary number from the interval  $[0.3875, 0.4025)$ , rounded up to  $-\log_2(p_{ACAB}) = 6.06$  i.e. 7 bit, e.g. **0.0110010**

## Variants of Image Compression

JPEG is not a single format, but it can be chosen from a number of modes:

*Lossy sequential DCT-based mode* (baseline process)

- Presented before, but not the only method

*Expanded lossy DCT-based mode*

- Enhancement to the baseline process by adding progressive encoding

*Lossless mode*

- Low compression ratio → „perfect“ reconstruction of original image
- No DCT, but differential encoding

*Hierarchical mode*

- Accommodates images of different resolutions
- Selects its algorithms from the three other modes

## Variants: Expanded Lossy DCT-based Mode

With *sequential encoding* as in the baseline process the whole image is coded and decoded in a single run. An alternative to sequential encoding is *progressive encoding*, done in the entropy encoding step.

Two alternatives for progressive encoding are possible:

- *Spectral selection*

At first, coefficients of low frequencies are passed to entropy encoding, coefficients of higher frequencies are processed in successive runs

- *Successive approximation*

All coefficients are transferred in one run, but most-significant bits are encoded prior to less-significant bits.

12 possible coding alternatives in the expanded mode:

- Using sequential encoding, spectral selection, or successive approximation (3 variants)
- Using Huffman or Arithmetic encoding (2 variants)
- Using 8 or 12 bits for representing the samples (2 variants)

*Most popular mode*: sequential display mode with 8 bits/sample and Huffman encoding



## Expanded Lossy DCT-based Mode (Example)

**Sequential encoding:** image is coded and decoded in a single run



Step 1

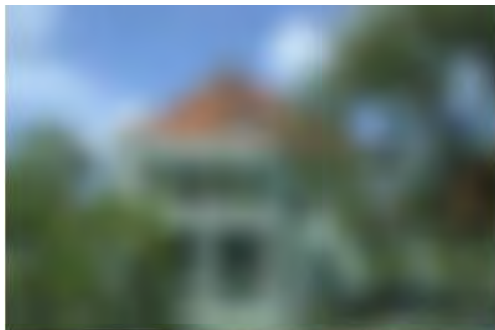


Step 2



Step 3

**Progressive encoding:** image is coded and decoded in refining steps



Step 1



Step 2



Step 3

## Variants: Lossless Mode

Lossless mode uses *differential encoding*

(Differential encoding is also known as *prediction* or *relative encoding*)

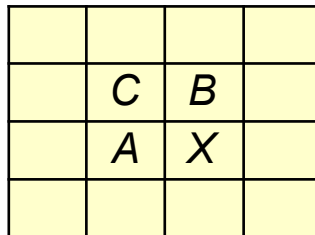
- Sequence of characters whose values are different from zero, but which do not differ much.
  - Calculate only the difference wrt. the previous value (used also for DC-coefficients)

Differential encoding for still images:

- Avoid using DCT/quantization
- Instead: calculation of differences between nearby pixels or pixel groups
  - Edges are represented by large values
  - Areas with similar luminance and chrominance are represented by small values
  - Homogenous area is represented by a large number of zeros
    - further compression with run-length encoding is possible as for DCT

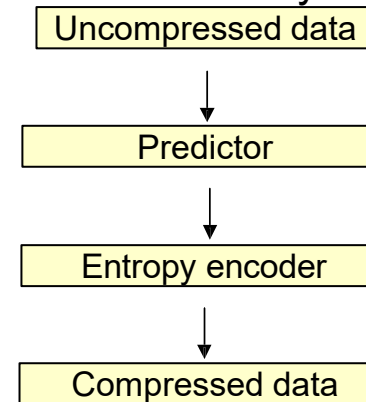
## Variants: Lossless Mode

- Uses data units of single pixels for image preparation
- Any precision between 2 and 16 bits/pixel can be used
- Image processing and quantization use a predictive technique instead of transformation encoding
- 8 predictors are specified for each pixel  $X$  by means of a combination of the already known adjacent samples  $A$ ,  $B$ , and  $C$



The actual predictor should give the best approximation of  $x$  by the already known values  $A, B, C$

| predictor | predicted values $X$ |
|-----------|----------------------|
| 0         | no prediction        |
| 1         | $A$                  |
| 2         | $B$                  |
| 3         | $C$                  |
| 4         | $A+B-C$              |
| 5         | $A+(B-C)/2$          |
| 6         | $B+(A-C)/2$          |
| 7         | $(A+B)/2$            |



The number of the chosen predictor and the difference of the prediction to the actual value are passed to entropy encoding (Huffman or Arithmetic Encoding)

### Example:

(4,0):  $X$  is exactly given by  $A+B-C$   
 (7,1):  $X$  is  $(A+B)/2+1$

## Variants: Hierarchical Mode

The *Hierarchical mode* uses either the lossy DCT-based algorithms or the lossless compression technique

The idea: encoding of an image at different resolutions

*Algorithm:*

- Image is initially sampled at a low resolution
- Subsequently, the resolution is raised and the compressed image is subtracted from the previous result
- The process is repeated until the full resolution of the image is obtained in a compressed form

*Disadvantage:*

- Requires substantially more storage capacity

*Advantage:*

- Compressed image is immediately available at different resolutions  
→ *scaling becomes cheap*

# Multimedia

## Vorlesung

Hannes Kaufmann

Research Unit Virtual and Augmented Reality  
Institute of Visual Computing & Human-Centered Technology

Based on material by Prof. Christian Breiteneder, a few slides by Prof. Robert Sablatnig (EVC)  
Material on HDR by Scott Janus, Principal Engineer at Intel; Thuong Nguyen [tnguyen@humaxdigital.com](mailto:tnguyen@humaxdigital.com); Prof. Marc Pollefeys and Dr. Gabriel Brostow at ETHZ  
A few slides on JPEG used by courtesy of Dr. Dirk Thißen from RWTH Aachen

# Media Types

- History, representation, formats and operations of media types
- Text
- Audio
- **Image**
- Video
- No established digital standards for other media types:  
smell, taste, haptics



# Media Type: Image

- Human Visual System & Color Models
- High Dynamic Range
- Image Compression

# Human Visual System

Helligkeitssehen:

~120 Mill. "Stäbchen"/Rods

Farbsehen:

~7 Mill. "Zapfen"/Cones



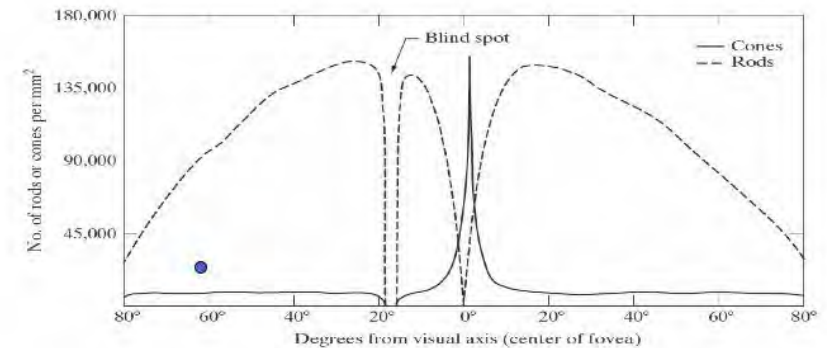
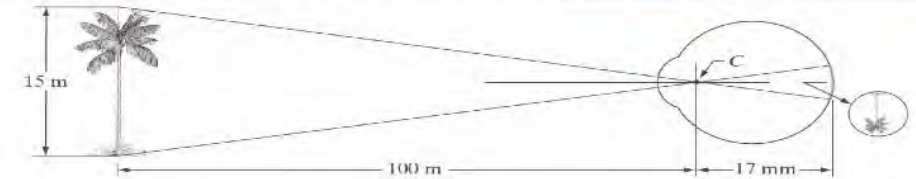
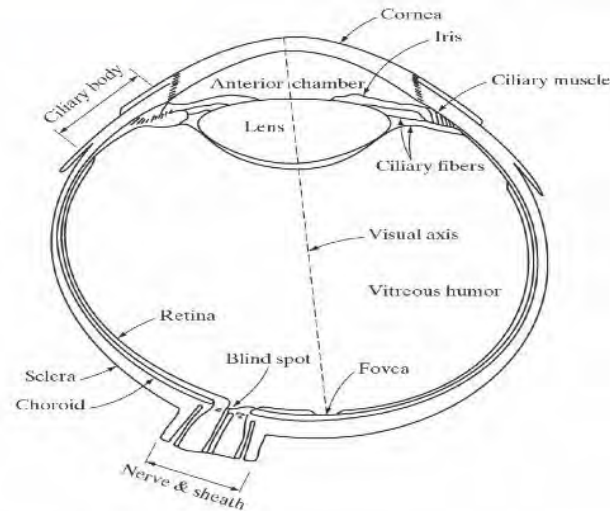
*Digital Image Processing, 3rd ed.*

Gonzalez & Woods

[www.ImageProcessingPlace.com](http://www.ImageProcessingPlace.com)

Chapter 2

Digital Image Fundamentals



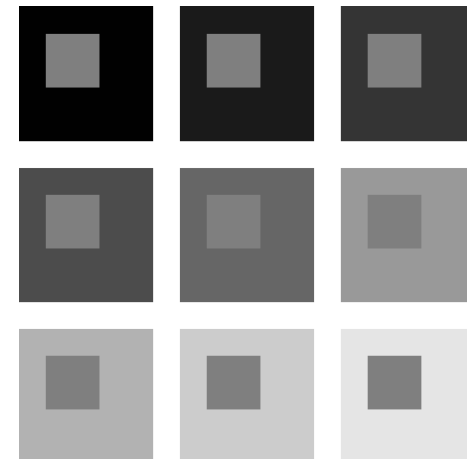
© 1992–2008 R. C. Gonzalez & R. E. Woods

# Light

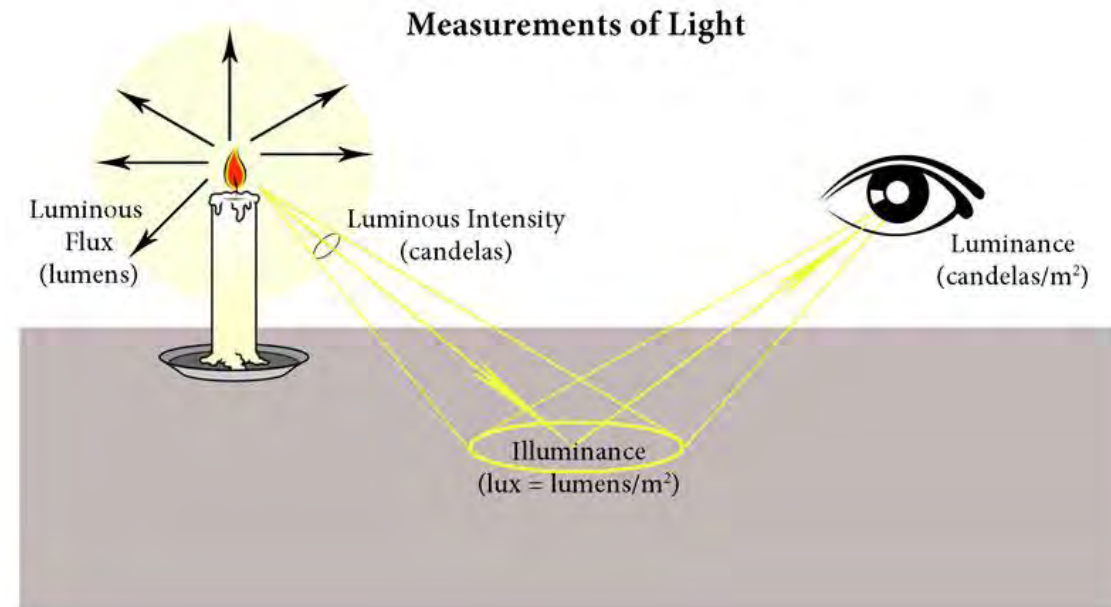
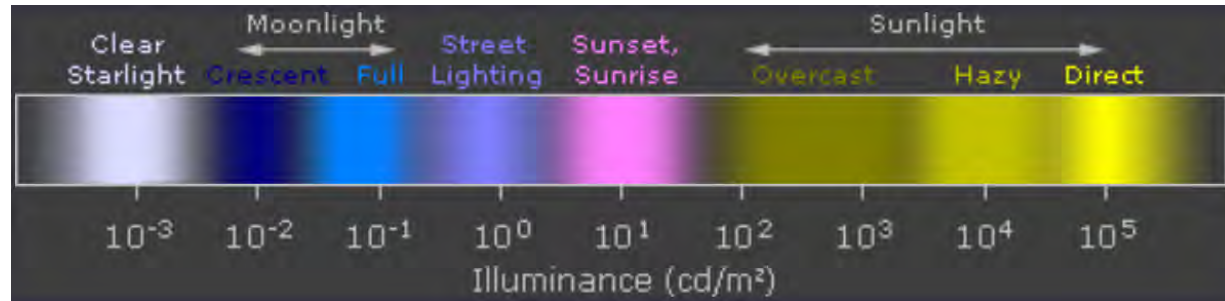
- Chromatic: of color
- Achromatic: without color (i.e. B&W)
- Color of an object depends upon:
  - object surface: reflectivity, physical properties, composition
  - light source(s) illuminating it
  - color of surrounding environment
  - visual system (eyes, brain, photoelectric cell, ...) of perceiving entity

# Achromatic Light

- black, white, usually greys (rare to see pure black or pure white)
- quantity or energy of light is the only measurement used for achromatic light
  - Intensity, luminance, brightness:
    - **Luminance** describes the amount of light that is emitted from, passed through, or reflected off an object. It is measured using candela per square meter ( $\text{cd/m}^2$ ), which is the concentration of luminous intensity per unit area.
    - **Brightness** is subjective and perceived by the human eye. It cannot be measured by any formula
- eye is sensitive to ratios of intensity changes, not the changes in intensity values themselves
- small changes of low intensity seem to have the same effect as larger changes of high intensity
- **Dynamic range**: ratio between min. and max. intensities



# Illuminance

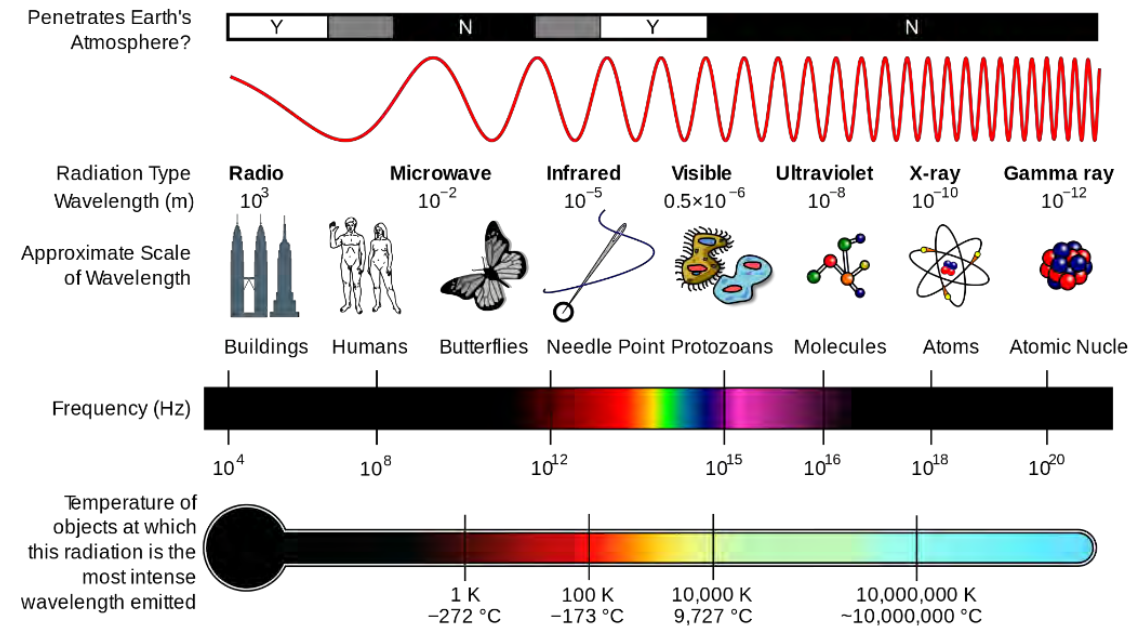
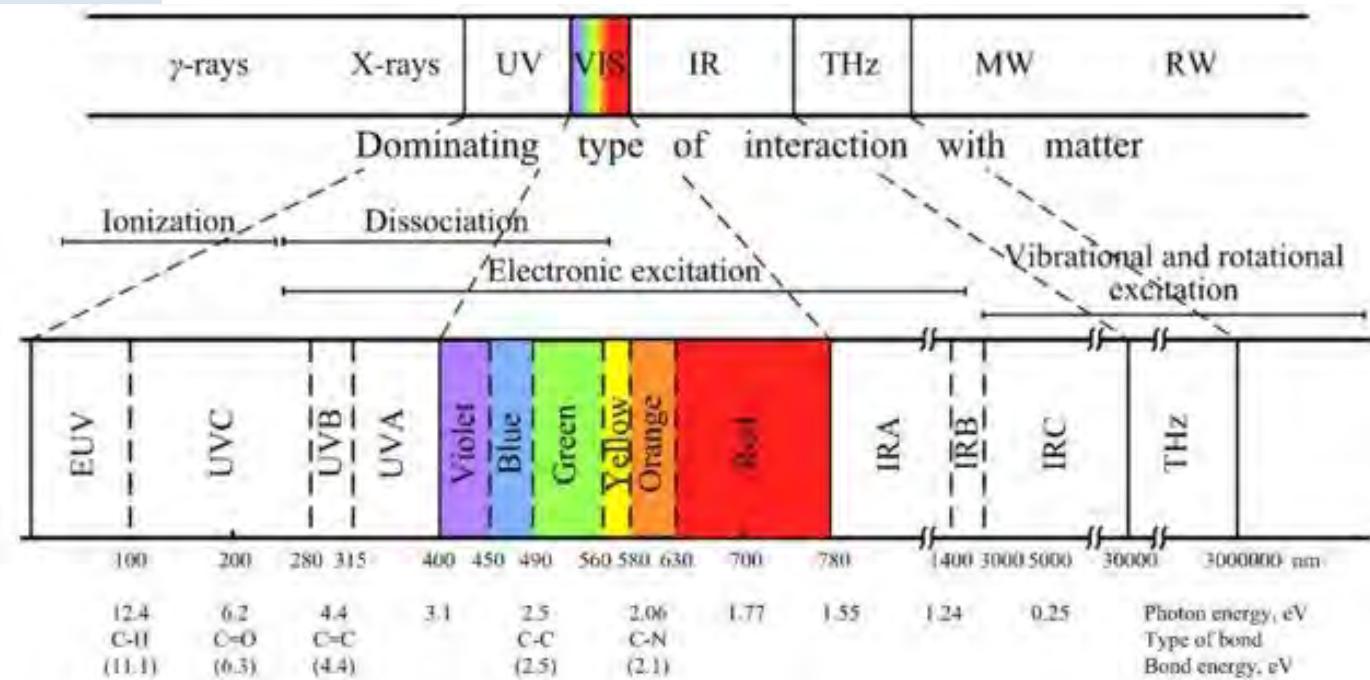


Illuminance = only incident light

- Illuminance is light we can't see directly, so-called ambient light;
- Light passing through air which is actually invisible until it reflects off an object (which we then see as luminance).
- It is measured in lux - **luminous flux** per unit area ( $\text{lx}$  or  $\text{lm/m}^2$ ).

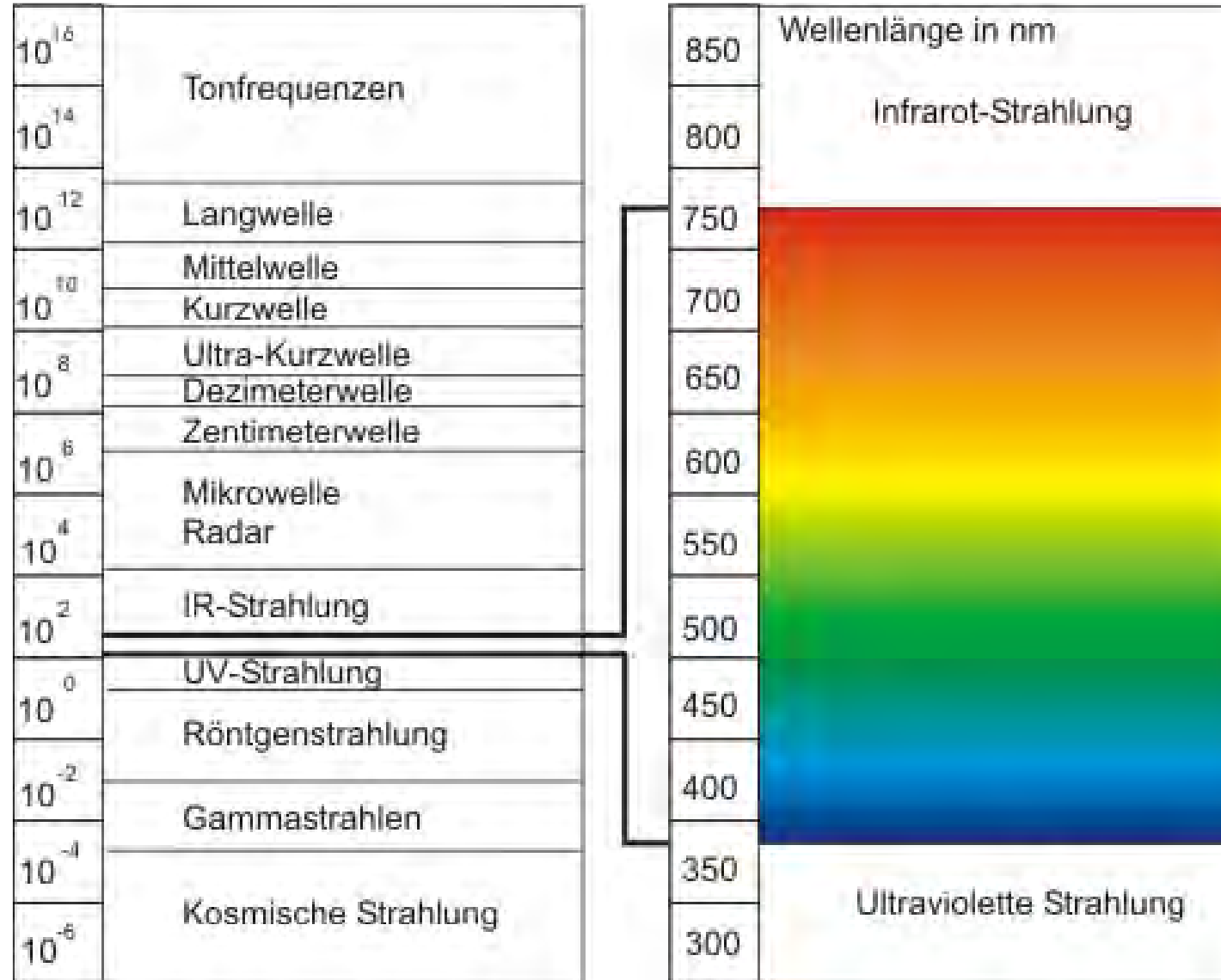
# Color, Light - Wavelengths

- Color is how we perceive beams of electromagnetic energy that fall on our retina
- We see light between 400 and 700 nanometers in wavelength ( $10^{-9}$  meters)





# Light Spectrum – Wavelengths

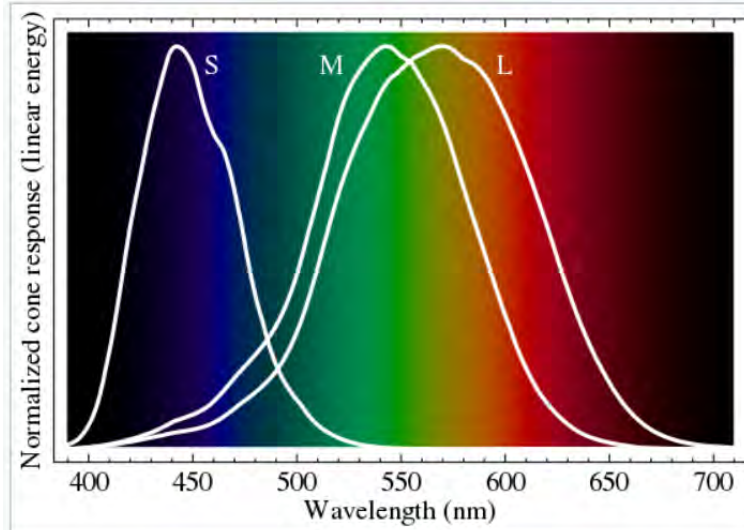


# Color

- Various models of color used:
  - Printing & graphics industry: color sample books and codes
  - Artists: tint (adding white to pure pigment), shade (adding black), tone (adding both)
  - Displays: use hardware capabilities (RGB)
  - Physicists: use optical models (wavelengths, energy measures)
- Human perception of color depends on how brain reacts to the whole visual environment, as well as brain & eye biological factors

# Tristimulus Model

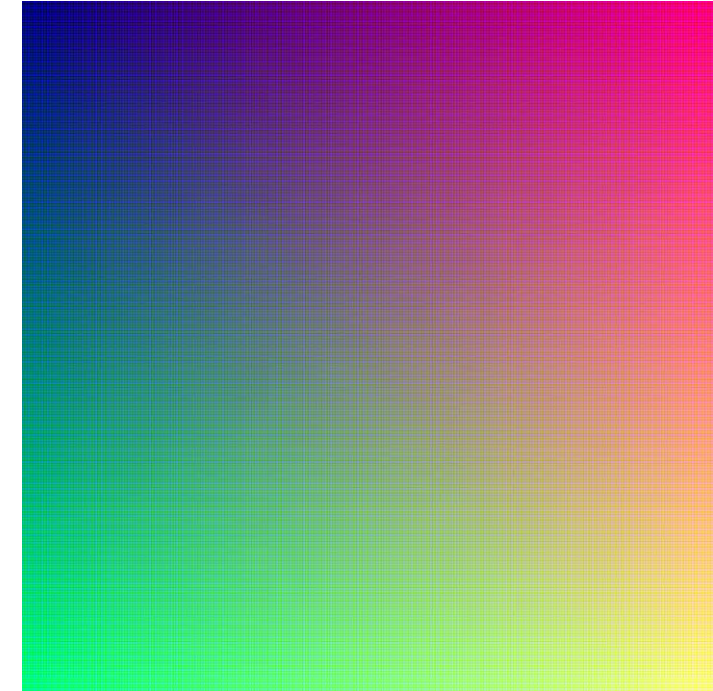
- The retina contains cones that sense either red, green, or blue light
- 6-7 million cones per eye, concentrated in central area called "fovea"
- Rods: sensors that surround fovea, and perceive weak "night" images (>100 million per eye)



Normalized response spectra of human cones, to monochromatic spectral stimuli, with wavelength given in nanometers.

# Color vision in Nonhumans

The human eye can distinguish about 10 million different colors (up to debate)



16.7 million colors

Color vision table

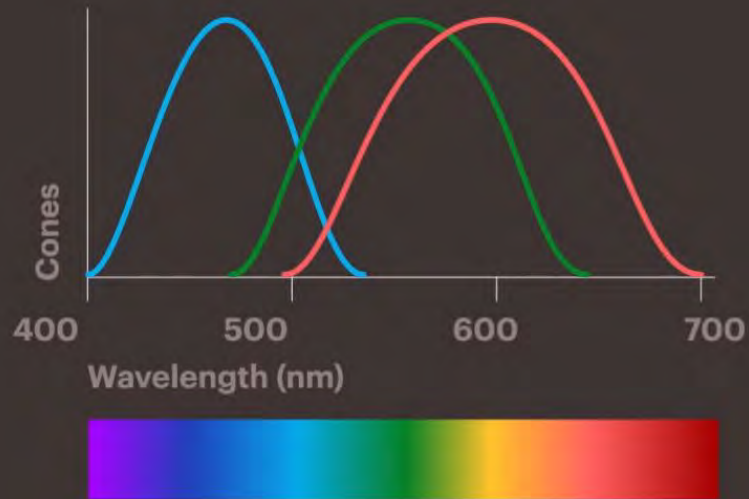
| State                | Types of cone cells | Approx. number of colors perceived | Carriers                                                                                            |
|----------------------|---------------------|------------------------------------|-----------------------------------------------------------------------------------------------------|
| <b>Monochromacy</b>  | 1                   | 200                                | Marine mammals, owl monkey, Australian sea lion, achromat primates                                  |
| <b>Dichromacy</b>    | 2                   | 40,000                             | Most terrestrial non-primate mammals, color blind primates                                          |
| <b>Trichromacy</b>   | 3                   | 10 million <sup>[54]</sup>         | Most primates, especially great apes (such as humans), marsupials, some insects (such as honeybees) |
| <b>Tetrachromacy</b> | 4                   | 100 million                        | Most reptiles, amphibians, birds and insects, rarely humans                                         |
| <b>Pentachromacy</b> | 5                   | 10 billion                         | Some insects (specific species of butterflies), some birds (pigeons for instance)                   |

**The Amazing Ways Animals See the World**

<https://www.youtube.com/watch?v=LlfKk37bkyk>

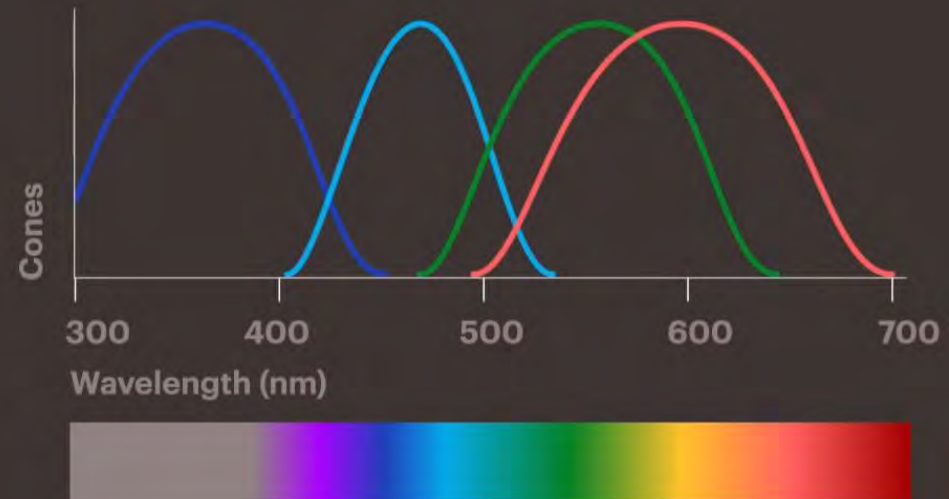


## Humans



3 types of cones

## Birds



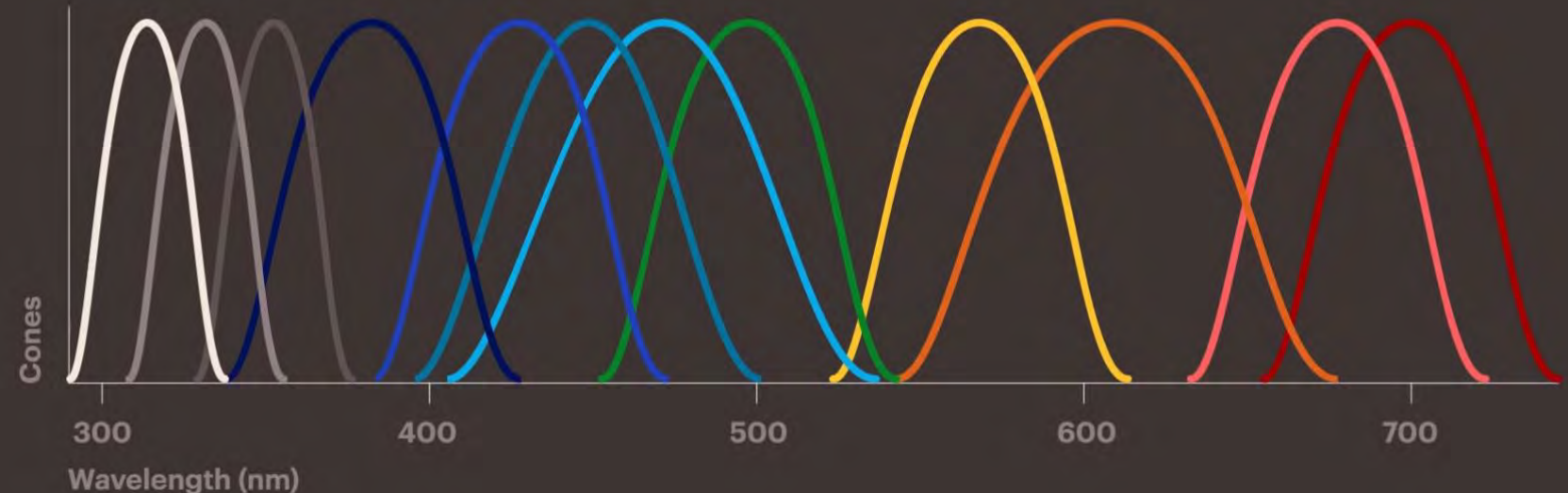
4 types of cones

## Mantis Shrimp:

- 12-16 types of cones
- independent eye movement
- Sees ultraviolet, infrared and 6 types of polarization

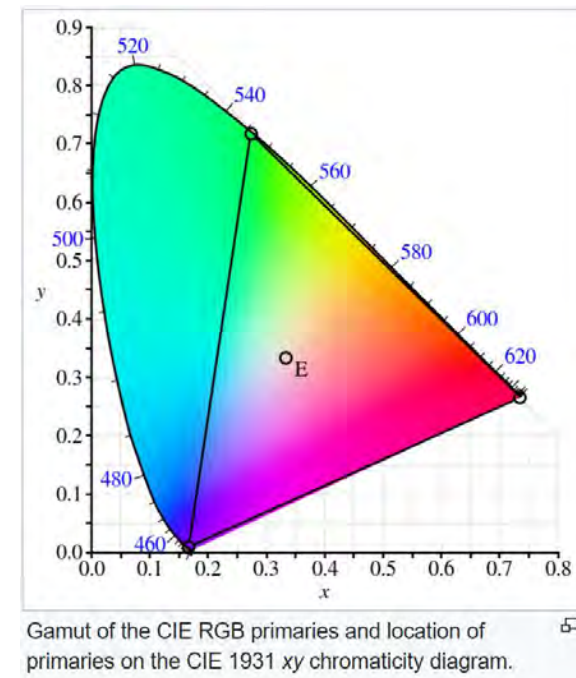


## Mantis Shrimp



# CIE XYZ Color Space (1931)

- CIE color space (CIE = Commission Internationale de l'Eclairage, International Commission on Illumination) used to calibrate other color models;  
1931 CIE XYZ
- Device-invariant representation of color
- Serves as a standard reference against which many other color spaces are defined
- Many combinations of dominant wavelength, luminance, saturation appear the same to us





# CIE Chromaticity Diagram

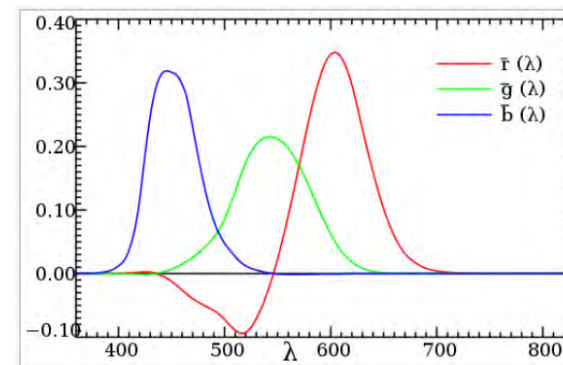
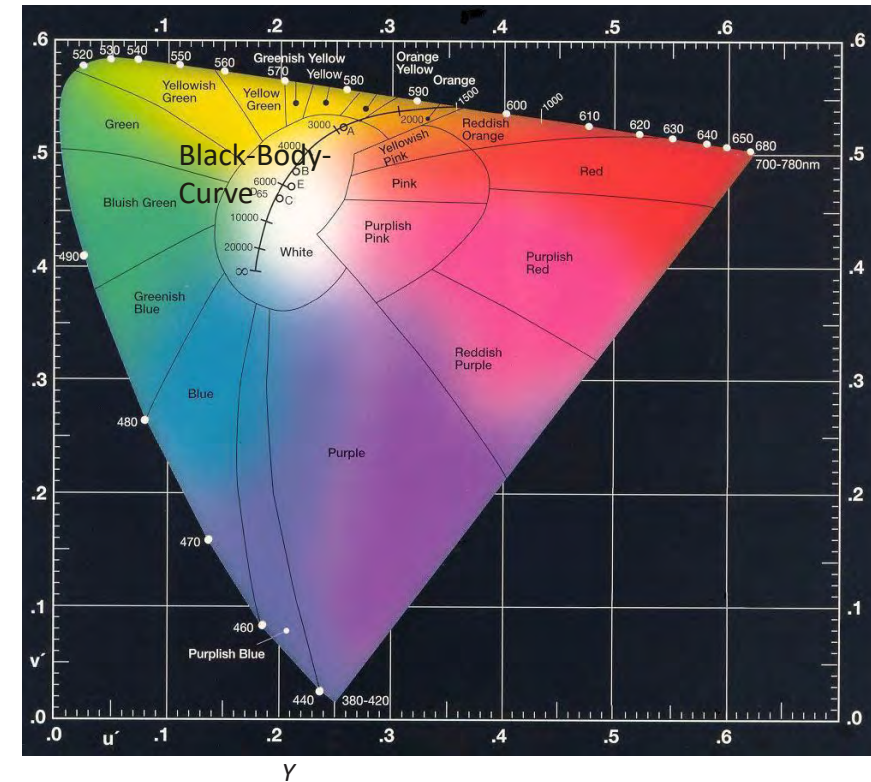
- A full plot of all human visible colors is a three-dimensional figure (cone).
- The concept of color can be divided into two parts: **luminance** and **chromaticity**
- CIE Diagram shows a **planar cut** through that cone at a specific luminance
- Color described in  $x, y$  (with  $x+y+z=1$ )
- Luminance =  $Y$
- Full color:  $(x, y, Y)$

Helligkeitssehen:

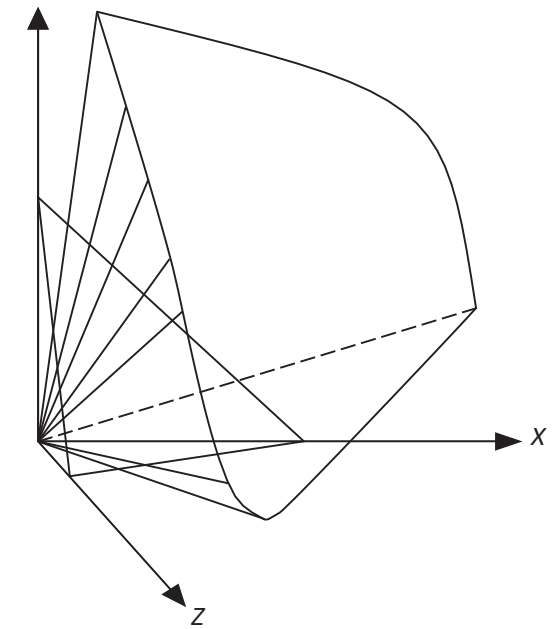
~120 Mill. "Stäbchen"/Rods

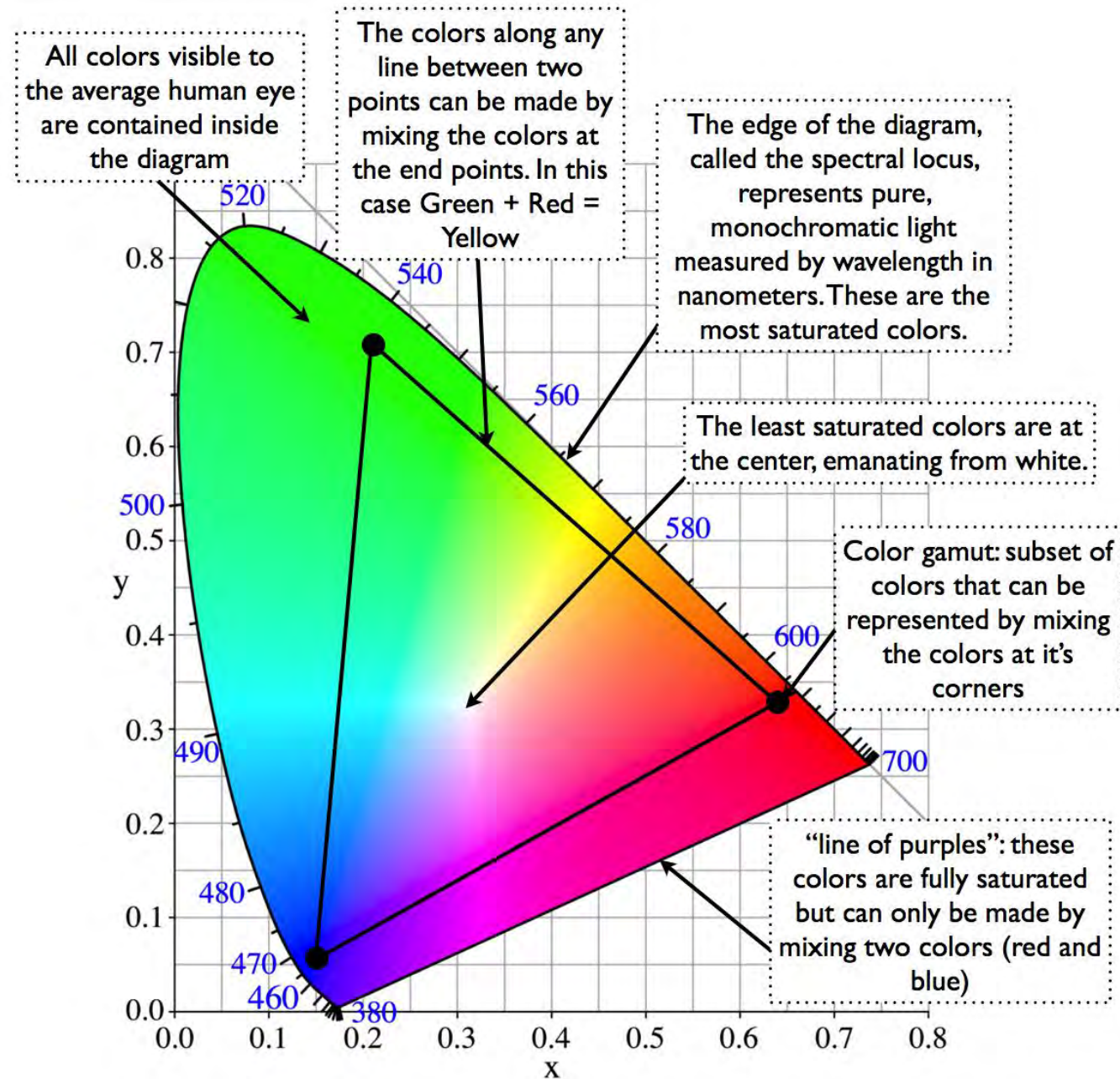
Farbsehen:

~7 Mill. "Zapfen"/Cones



Die Gewichtungen der drei Primärfarben für die reinen Spektralfarben entlang des Außenrandes des „Hufeisens“ von 360 bis 830 nm





Anatomy of a CIE Chromaticity Diagram



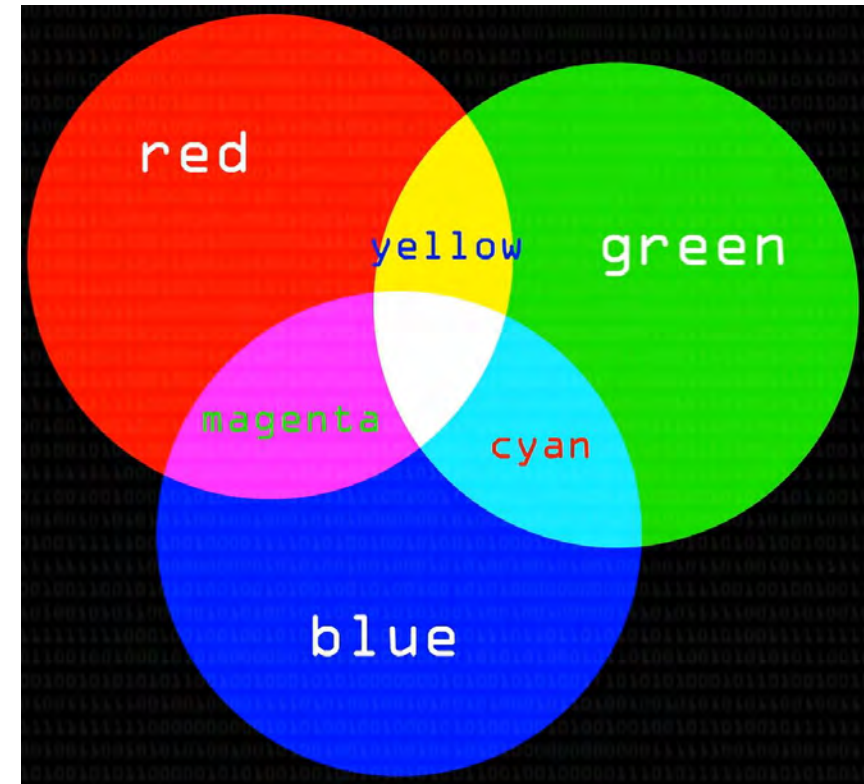
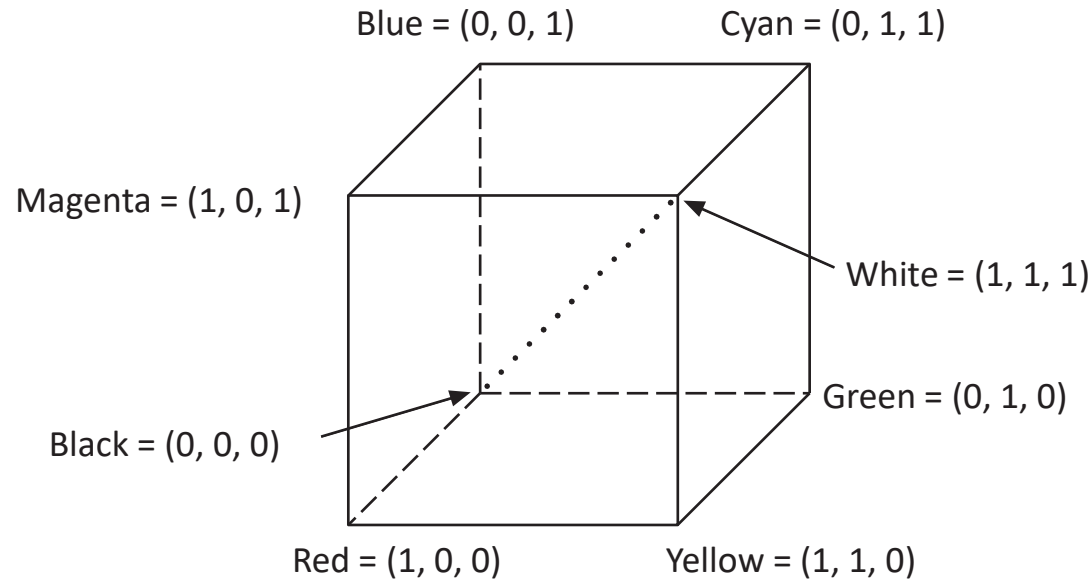
# Quick Overview of Color Spaces



# Color Models (Spaces) Overview

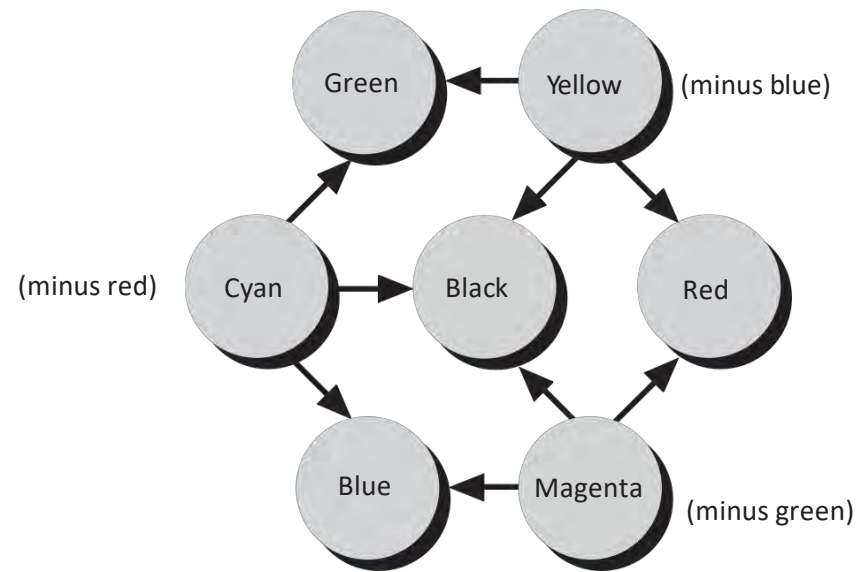
- CIE color space (Commission Internationale de l'Eclairage) used to calibrate other color models 1931 CIE XYZ, tristimulus theory
- RGB — for video display drivers
- CMYK — cyan, magenta, yellow, black, (subtractive primaries)
- HSB — hue (dominant color), saturation (amount of gray), brightness (intensity)
- YUV — used in television industry, Y (luminance), UV (color difference signals) = analog
- YCbCr = digital; Y (luminance),  $C_B$  and  $C_R$  are the blue-difference and red-difference chroma components

# The RGB Color Model

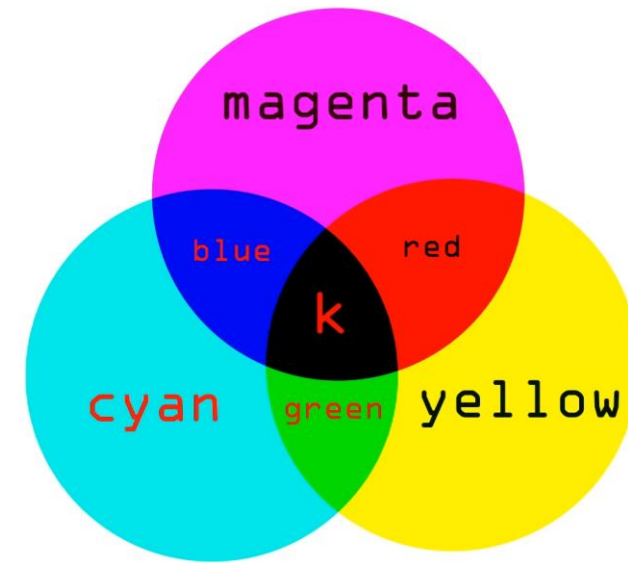


- Hardware-oriented model that turns on red, green, and blue pixels at varying intensities
- Additive: add red, green, blue together to get overall color
- Not all colors are exactly duplicable, and gamut (color range) covered by a display depends on performance of its display technology

# The CMY Color Model



$$\begin{bmatrix} R \\ G \\ B \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} C \\ M \\ Y \end{bmatrix}$$



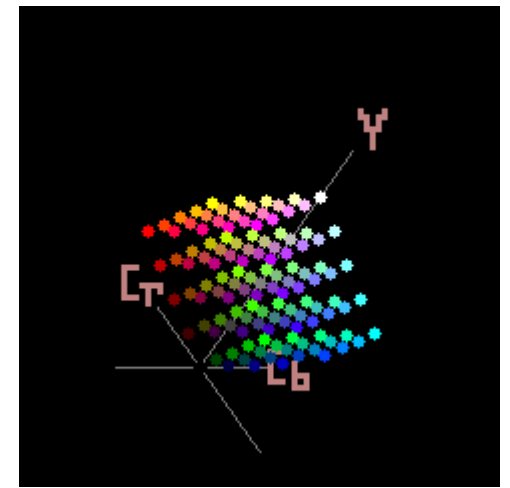
- Another related model, CMYK, uses black as 4th color:

$$K = \min(C, M, Y), C = C - K, M = M - K, Y = Y - K$$



# YCbCr / YUV

- Splits **luminance** (**Y**) from color (**chromacity**)  
**Cb** (Blue-Yellow Chrominance) und **Cr** (Red-Green Chrominance)
- Historically YUV (analog version) was used to transmit color (UV) to PAL/SECAM standard color TV sets, while black/white TVs only received a Y luminance signal and would still work. The same signal was transmitted for analog b/w and color TVs.
- Humans are very sensitive to changes in luminance, less to chromacity.
- Reducing color resolution allows great compression of images & videos (is used in JPEG)



# Conversion YCbCr <-> RGB

- Necessary in preparation for JPEG compression
- YCbCr (256 levels) can be computed directly from 8-bit RGB as follows:

$$\begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$

- During decoding YCbCr color space needs to be converted back to RGB:

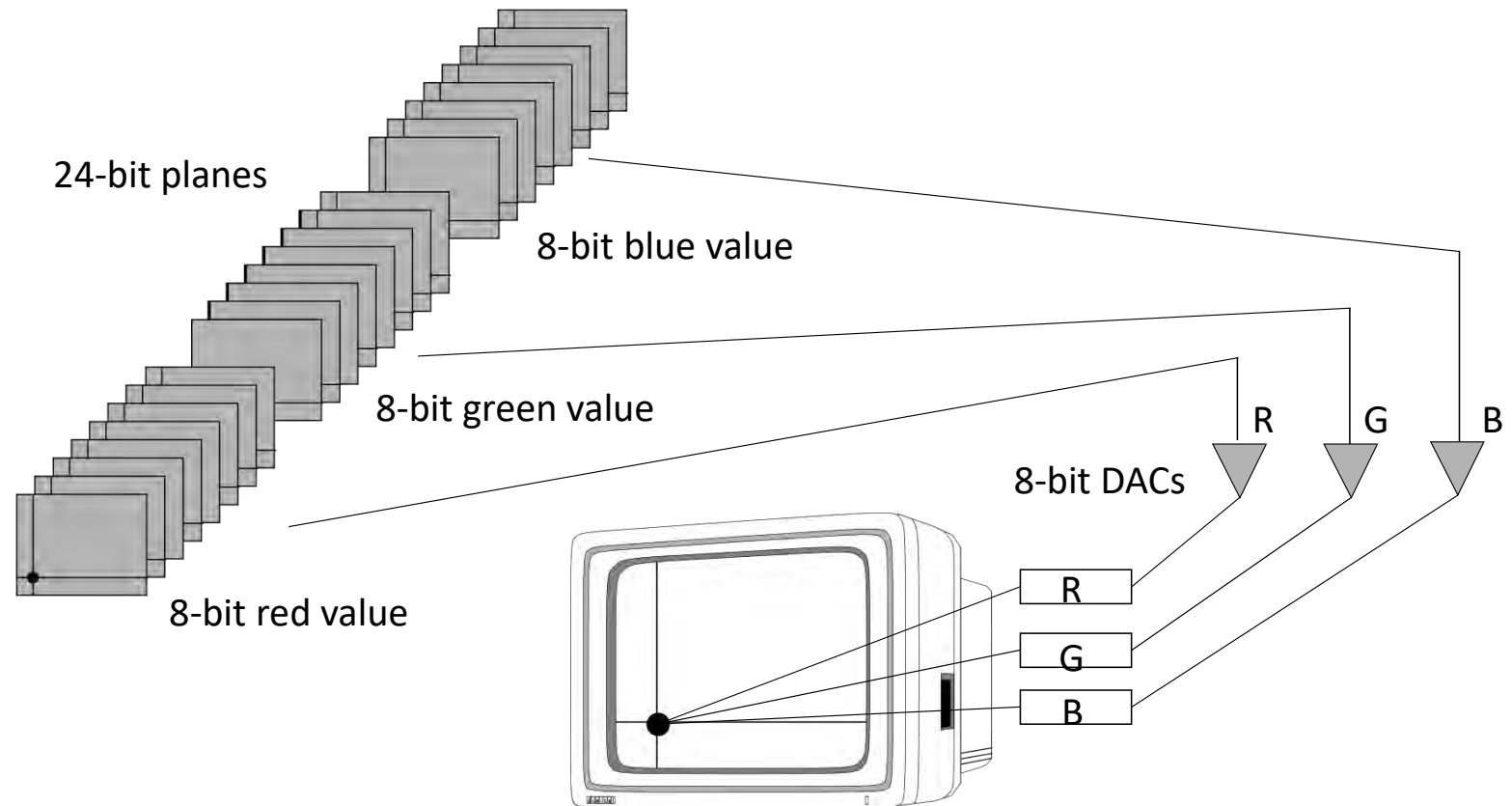
$$R = Y + 1.402 (Cr-128)$$

$$G = Y - 0.34414 (Cb-128) - 0.71414 (Cr-128)$$

$$B = Y + 1.772 (Cb-128)$$

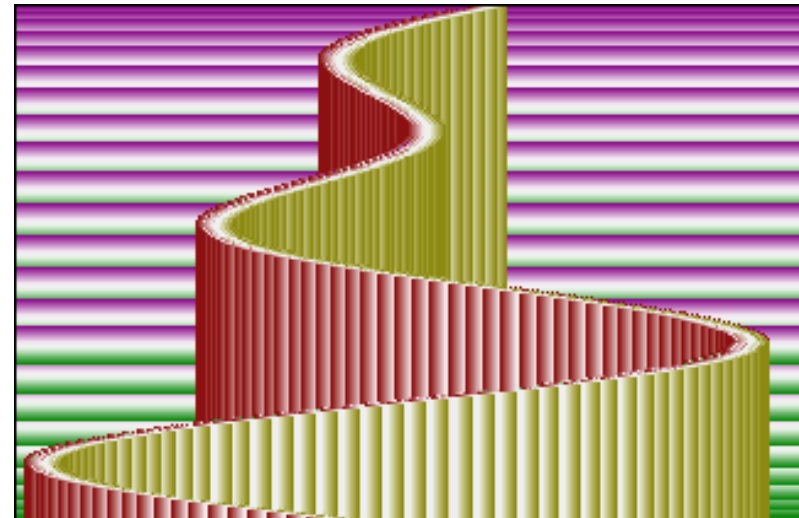
# Color Display – Color Depth

image descriptor = {  
 image width = 640  
 image height = 480  
 image depth = 24 bit  
 color model = RGB  
 encoding = YUV 8:2:2, JPEG}



# Image Representations

- Number of channels — dim. of color model plus **alpha channels**
  - e.g. 32 bit color depth = 24 bit color (8/8/8) + 8 bit alpha channel for transparency
- Channel depth — number of bits-per-pixel
- Alpha — controls transparency; masks and blends
- Pixel aspect ratio (usually square pixels nowadays)
- Interlacing (important for video)
- Compression — lossless vs. lossy
- Indexing — colors are represented by index in color maps or color lookup tables [CLUT]; map predefined or part of image







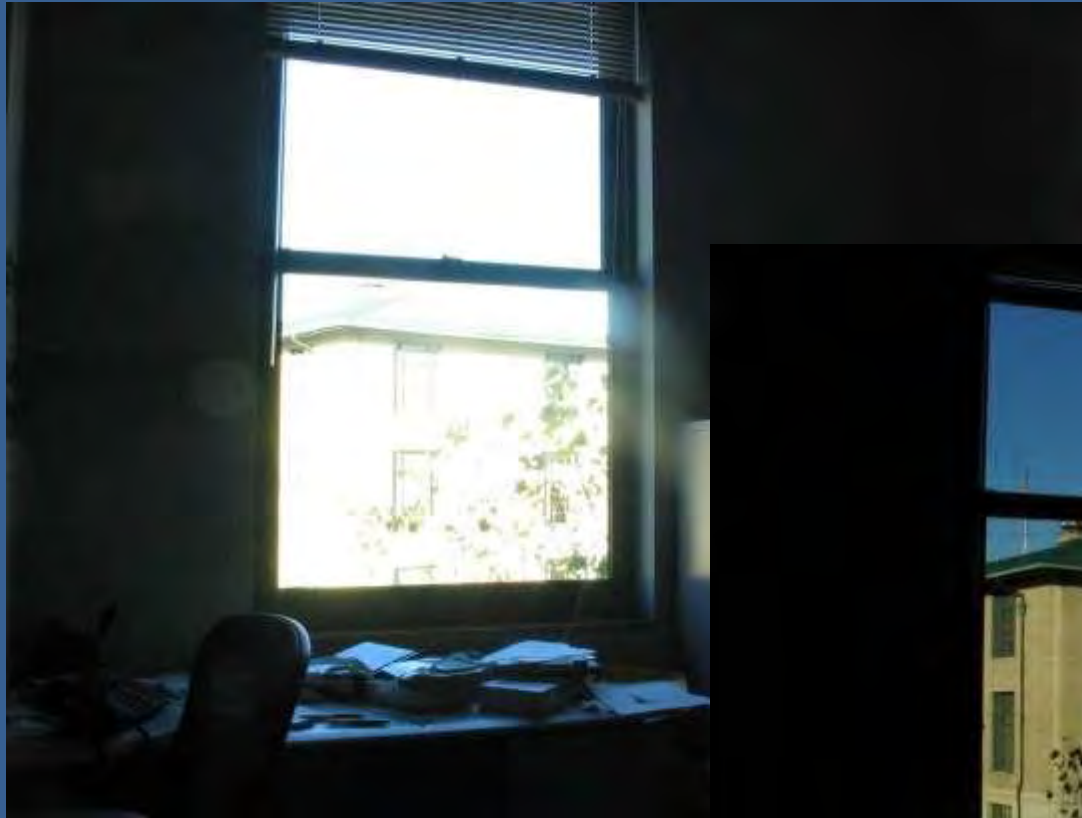
# HIGH DYNAMIC RANGE DEMYSTIFIED

Scott Janus, Principal Engineer

Slides adapted from Scott Janus, Principal Engineer at Intel; Thuong Nguyen [tnguyen@humaxdigital.com](mailto:tnguyen@humaxdigital.com); Prof. Marc Pollefeys and Dr. Gabriel Brostow at ETHZ

# Problem of Dynamic Range

## The Grandma Problem



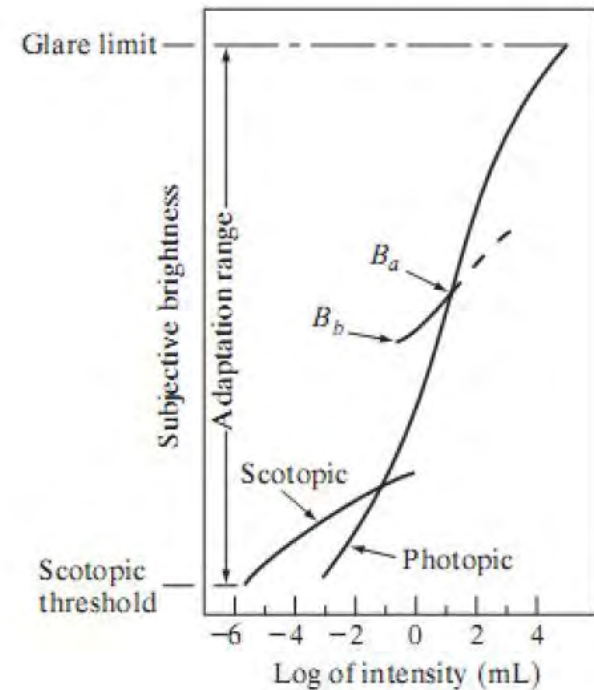
**Dynamic Range: Ratio of largest to smallest signal in a system**





# What is High Dynamic Range?

- Display does not have enough brightness to show the whole range.
- High Dynamic Range:  
Content with a **wider range of brightness and color**
  - Also an increased number of brightness and color levels
- Experiencing HDR content requires new displays!



# Color is complicated



“The Dress”

[https://en.wikipedia.org/wiki/The\\_dress](https://en.wikipedia.org/wiki/The_dress)

White and gold

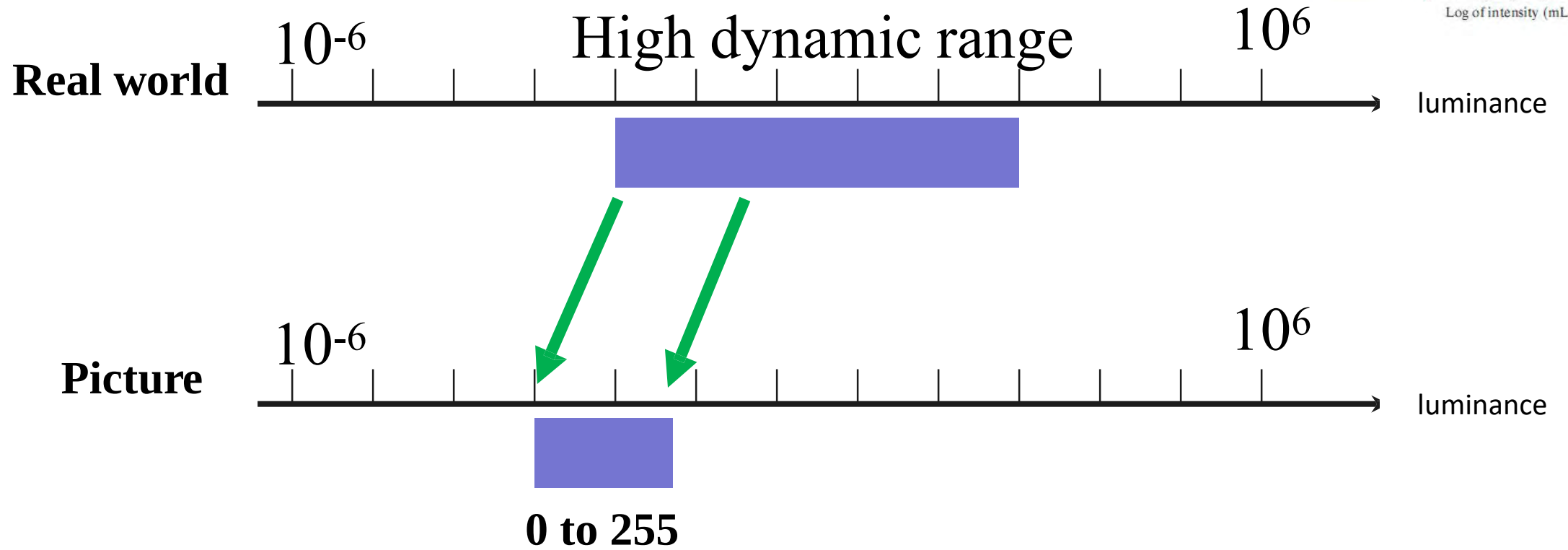
Or

Black and blue?

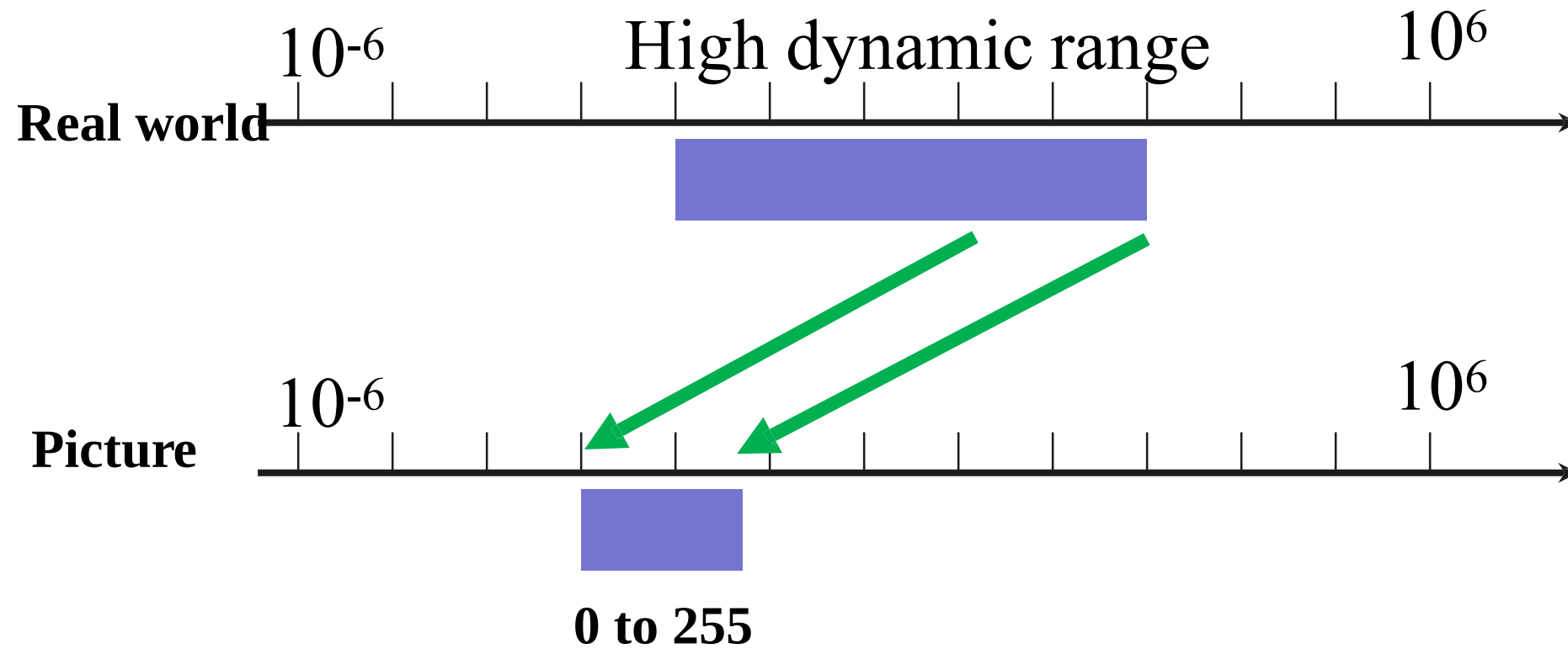


The real dress is  
Blue with black stripes

# Long Exposure

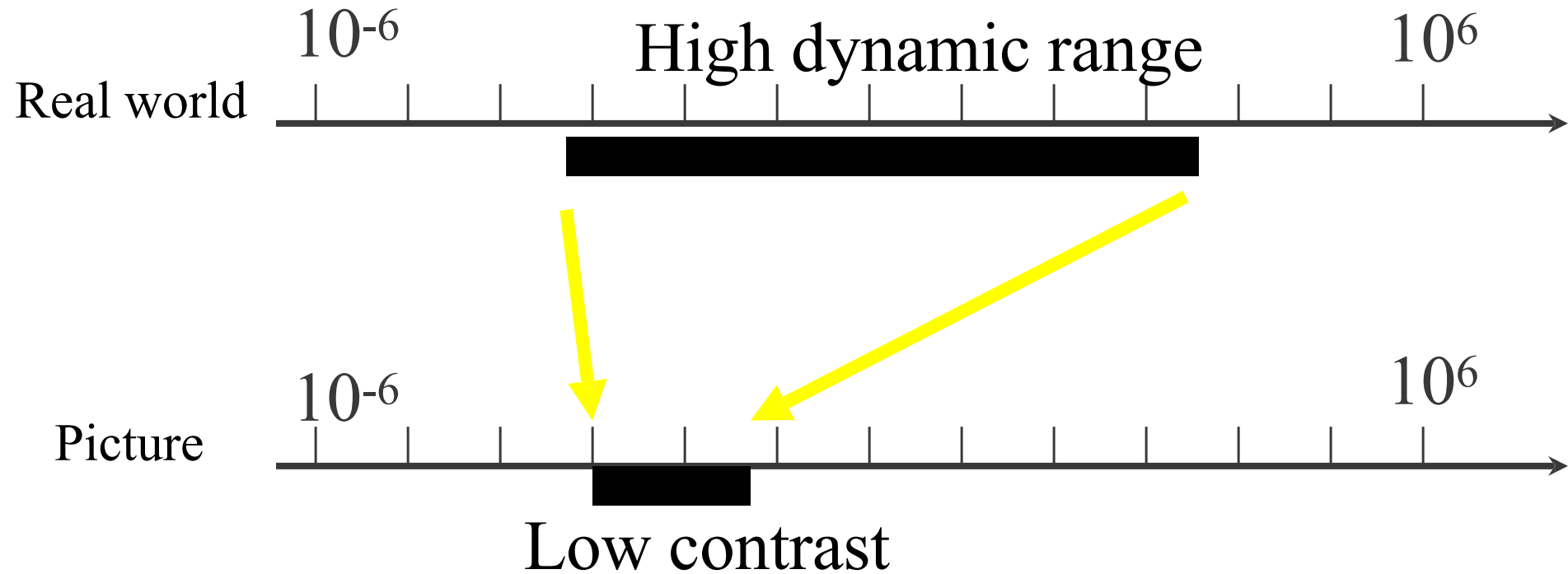


# Short Exposure



# Multiple exposure photography

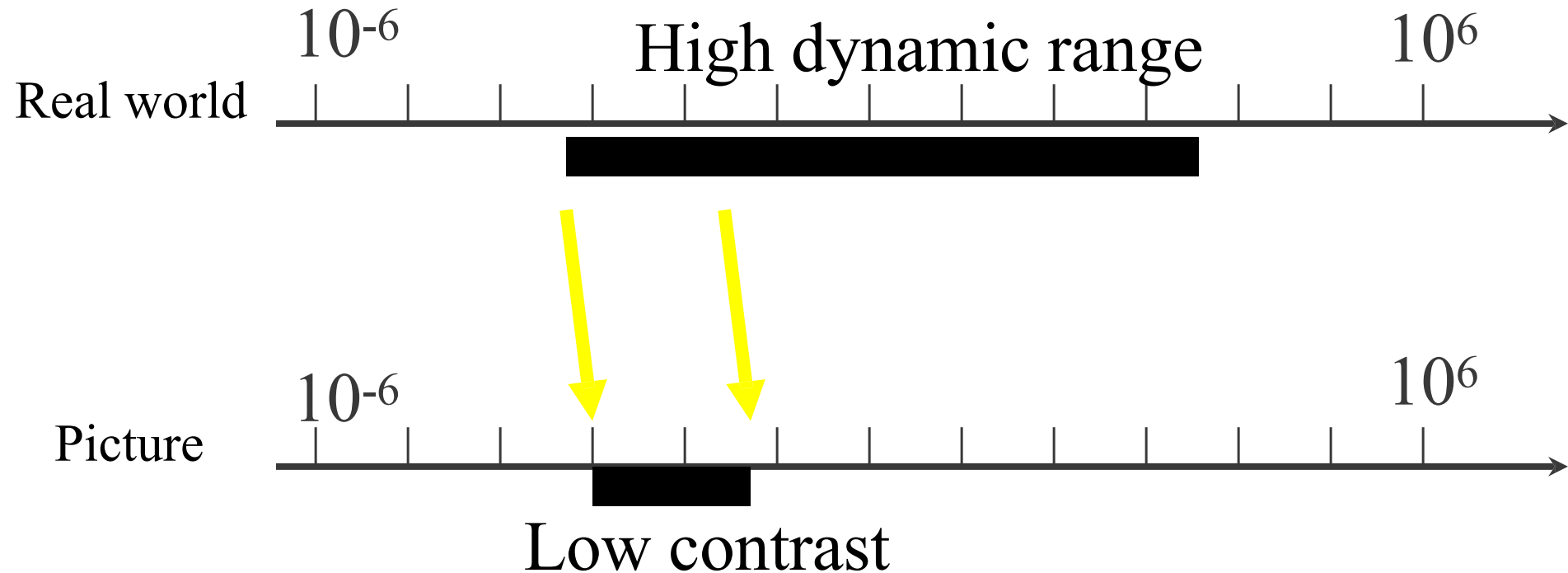
- **Sequentially measure all segments of the range**





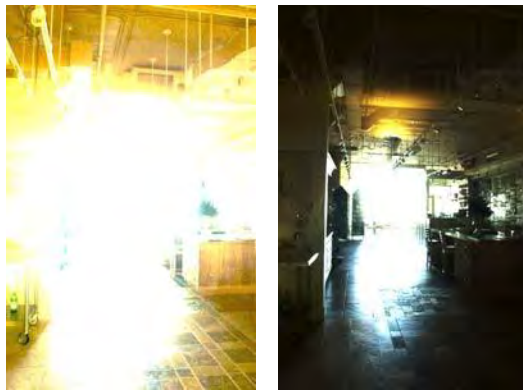
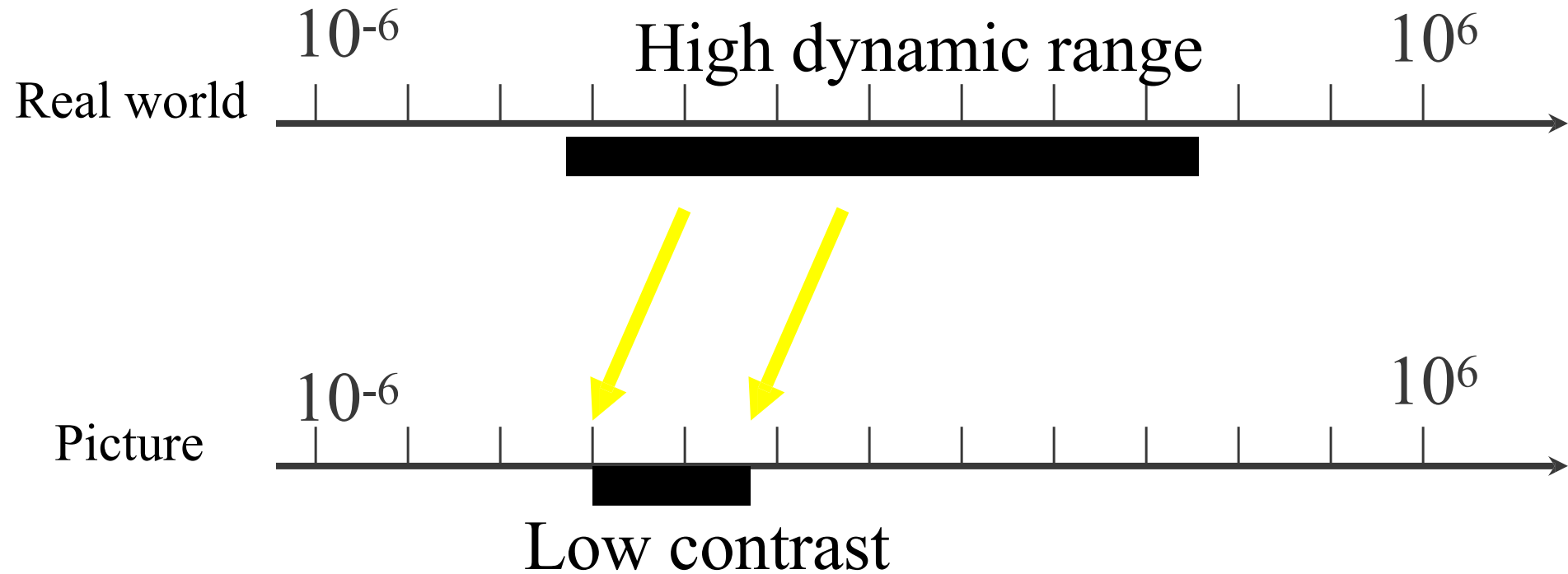
# Multiple exposure photography

- **Sequentially measure all segments of the range**



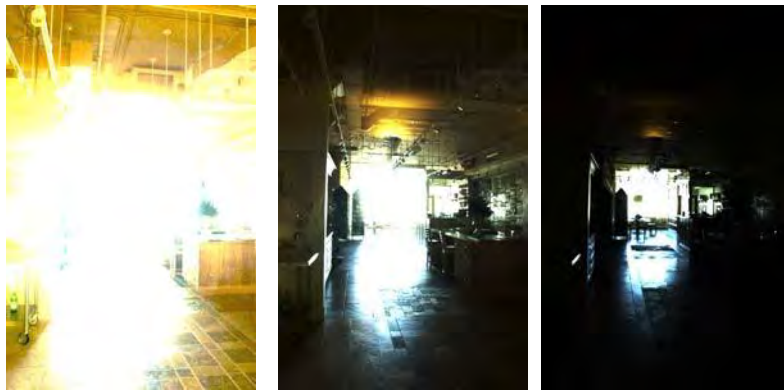
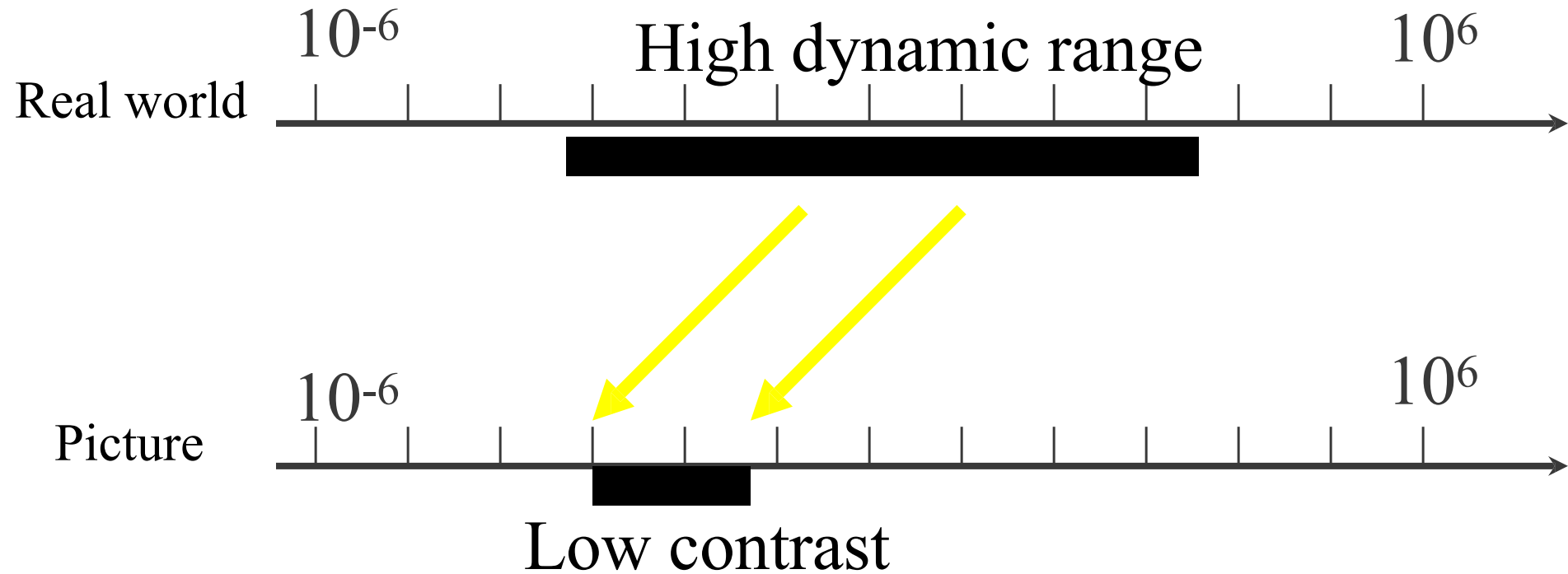
# Multiple exposure photography

- **Sequentially measure all segments of the range**



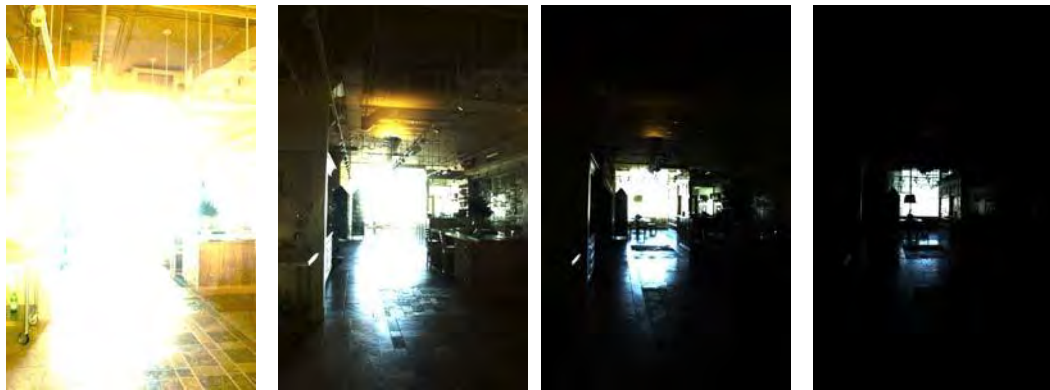
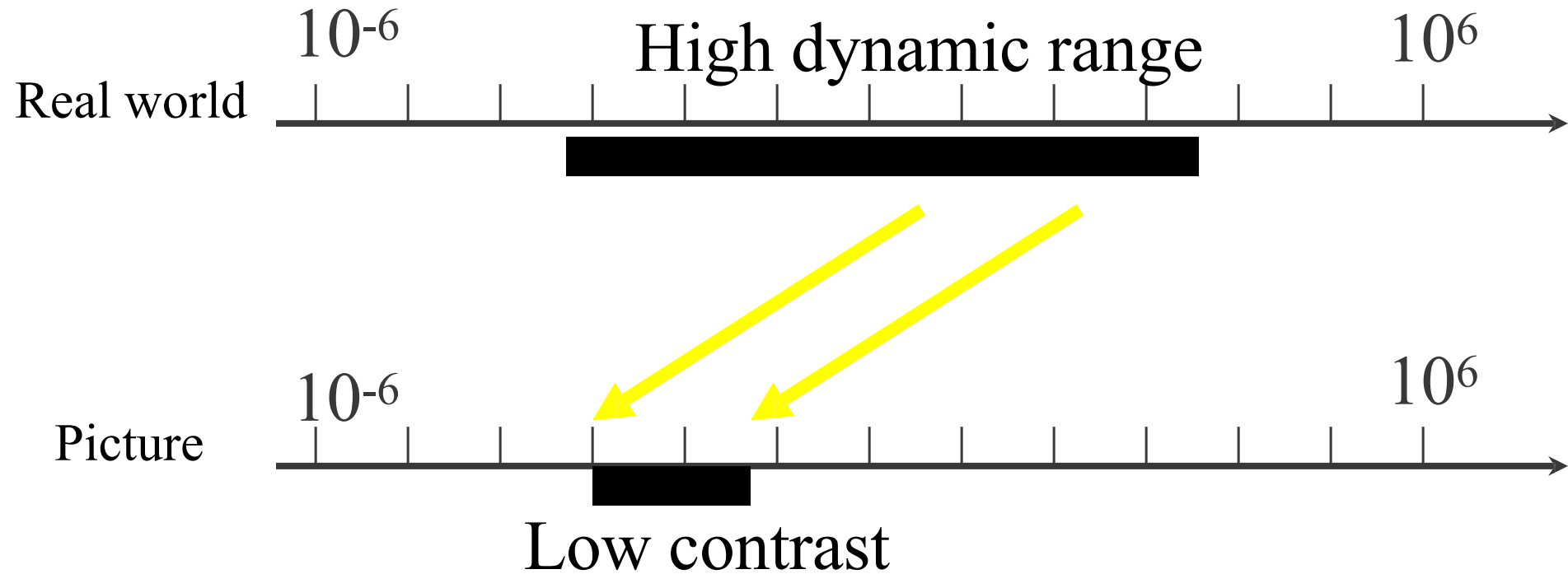
# Multiple exposure photography

- **Sequentially measure all segments of the range**



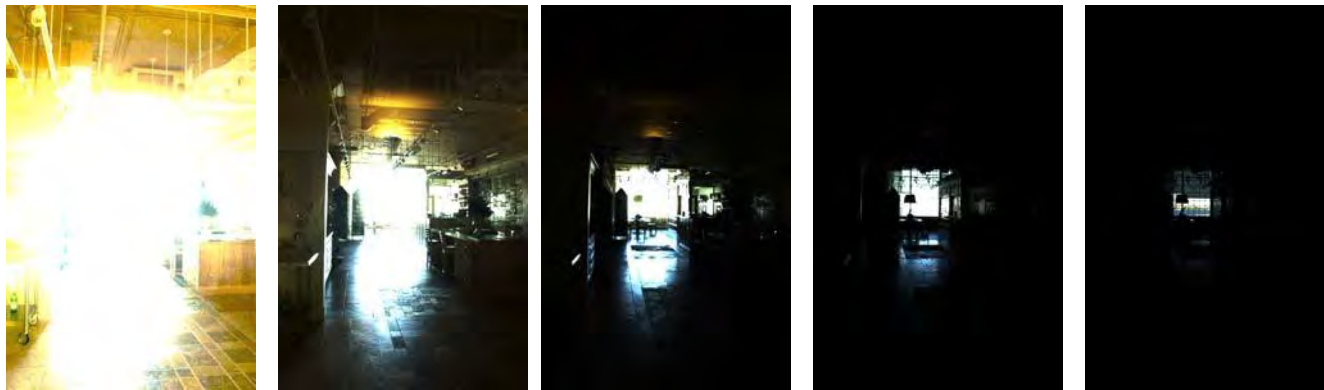
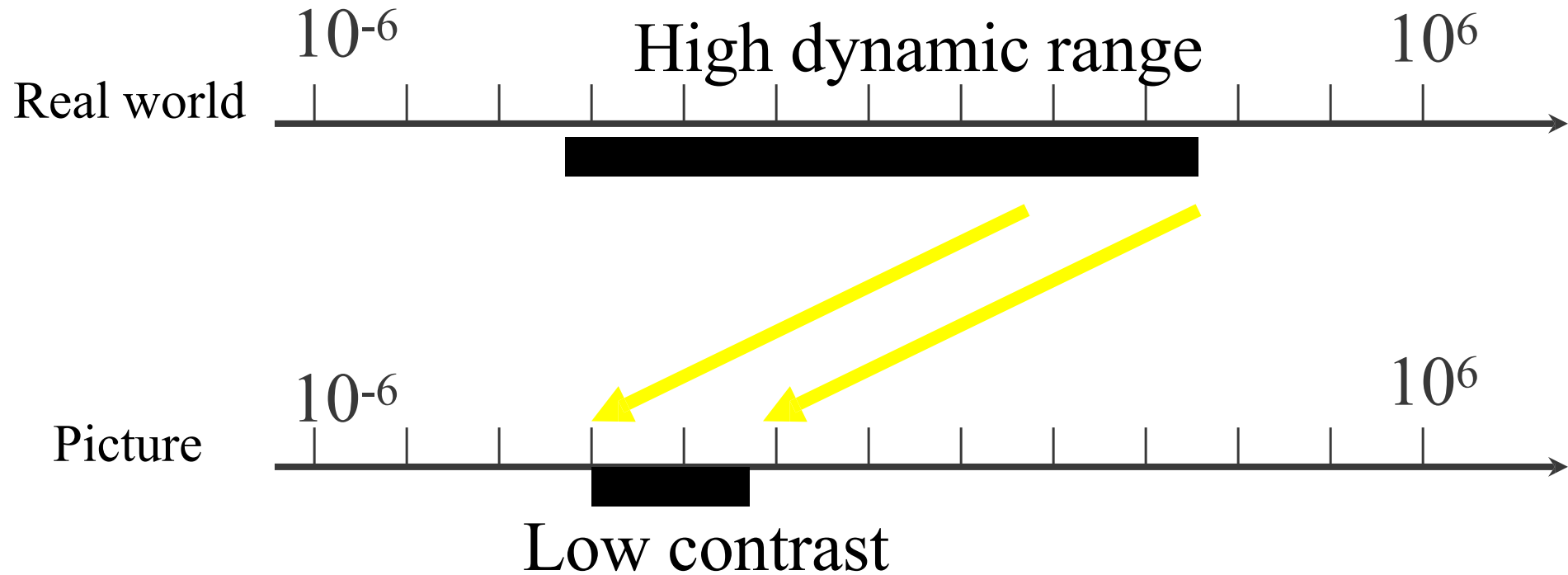
# Multiple exposure photography

- **Sequentially measure all segments of the range**



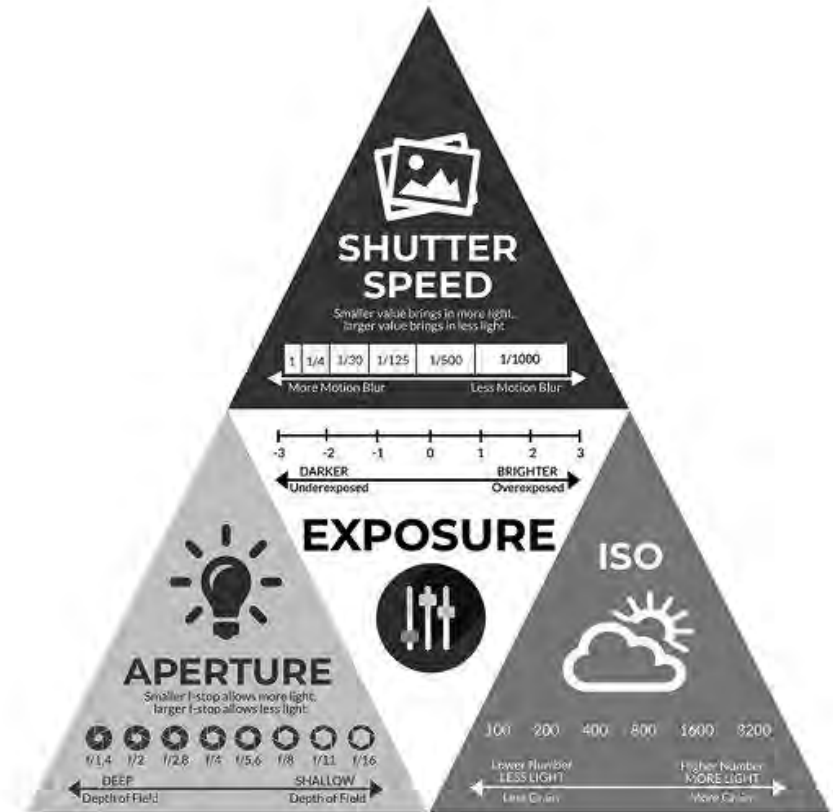
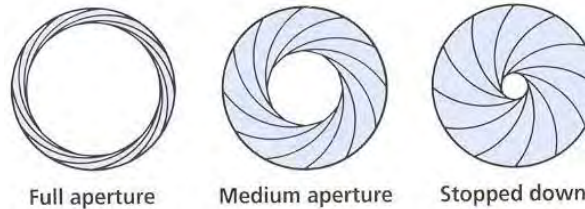
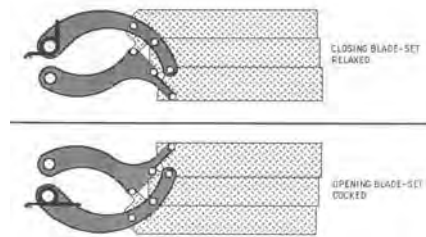
# Multiple exposure photography

- **Sequentially measure all segments of the range**



# How do we vary exposure?

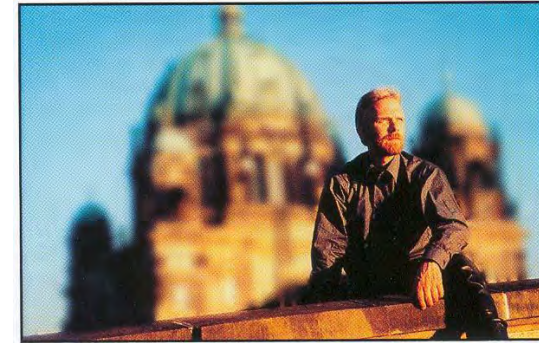
- Options:
  - Shutter speed
  - Aperture
  - Light Sensitivity (ISO)
  - [Neutral density filter]



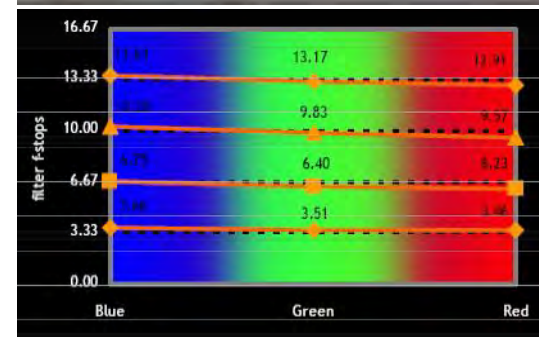


# Tradeoffs

- **Shutter speed**
  - Range:  $\sim 30$  sec to  $1/4000$ sec (6 orders of magnitude)
  - Pros: reliable, linear
  - Cons: sometimes noise for long exposure
- **Aperture**
  - Range:  $\sim f/1.4$  to  $f/22$  (2.5 orders of magnitude)
  - Cons: changes depth of field
- **Light Sensitivity (ISO)**
  - Range:  $\sim 100$  to  $1600$  (1.5 orders of magnitude)
  - Cons: noise
- **Neutral density filter**
  - Range: up to 4 densities (4 orders of magnitude) & can be stacked
  - Cons: not perfectly neutral (color shift), not very precise, need to touch camera (shake)
  - Pros: works with strobe/flash, good complement when desperate

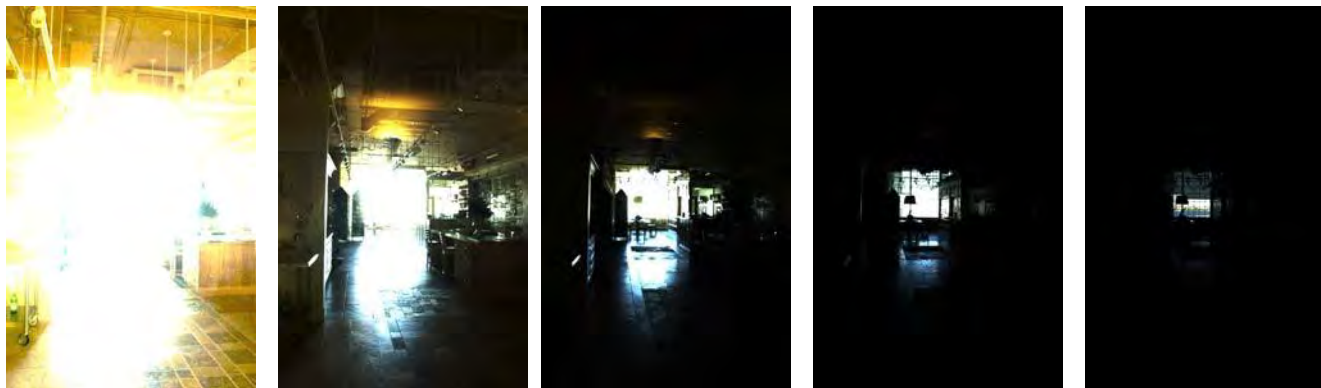


Nikon D2X  
ISO 3200



# HDR image using multiple exposure

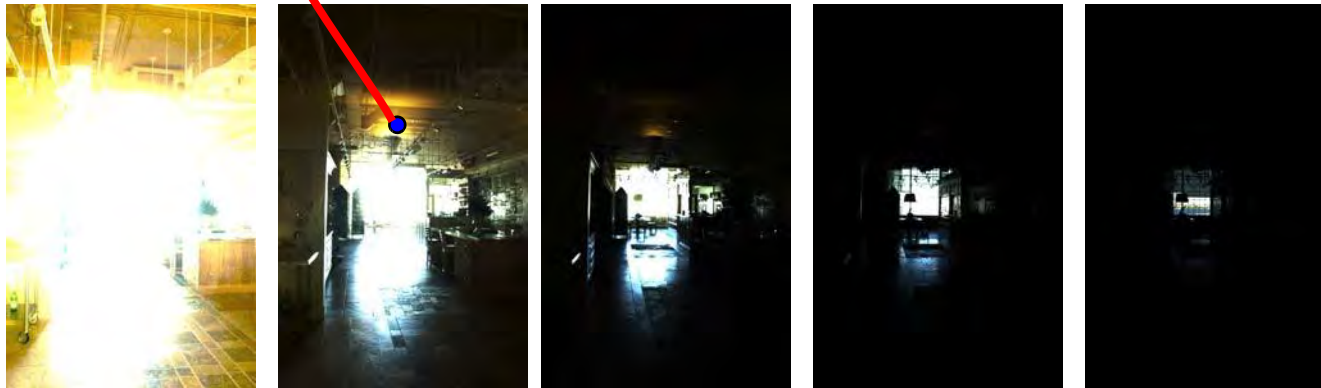
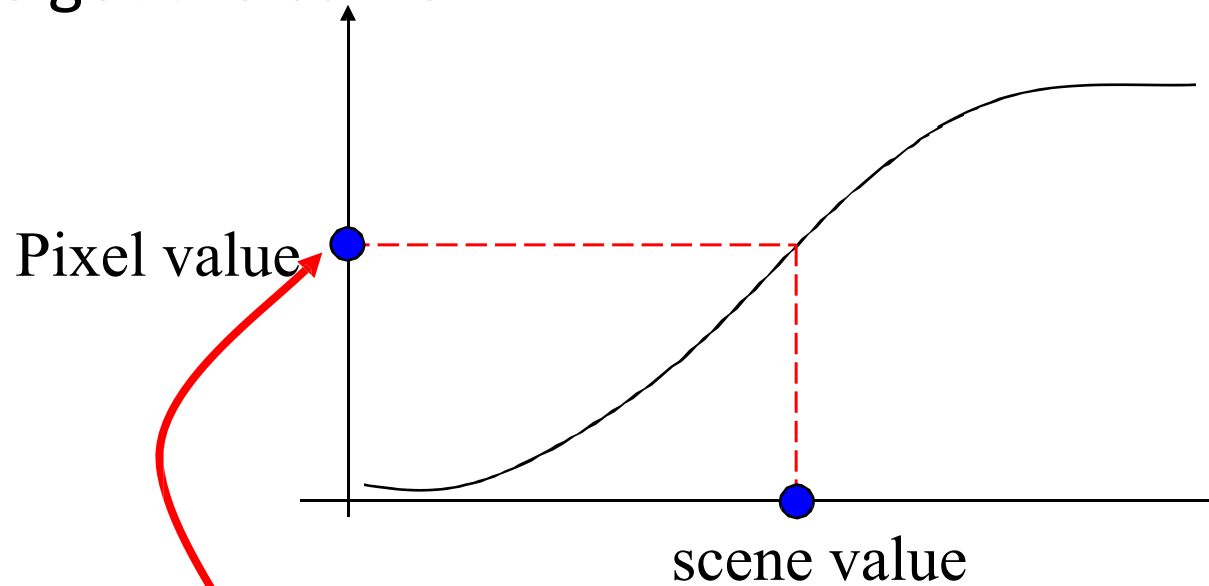
- Given  $N$  photos at different exposure
- Recover a HDR color for each pixel



# If we know the response curve

- Just look up the inverse of the response curve
- But how do we get the curve?

*A response curve is the relation between the input and output of a sensor.*



# Calibrating the response curve

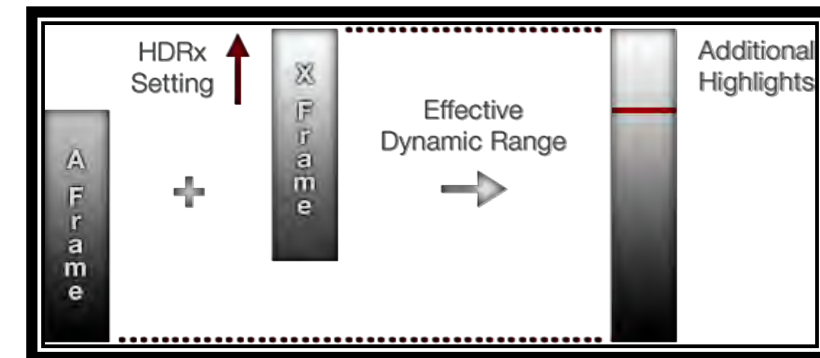
- **Two basic solutions**

- Vary scene luminance and see pixel values
  - Assumes we control and know scene luminance
- ***Vary exposure and see pixel value for one scene luminance***



- **Best of both:**

- **Vary exposure**
- Exploit the large number of pixels

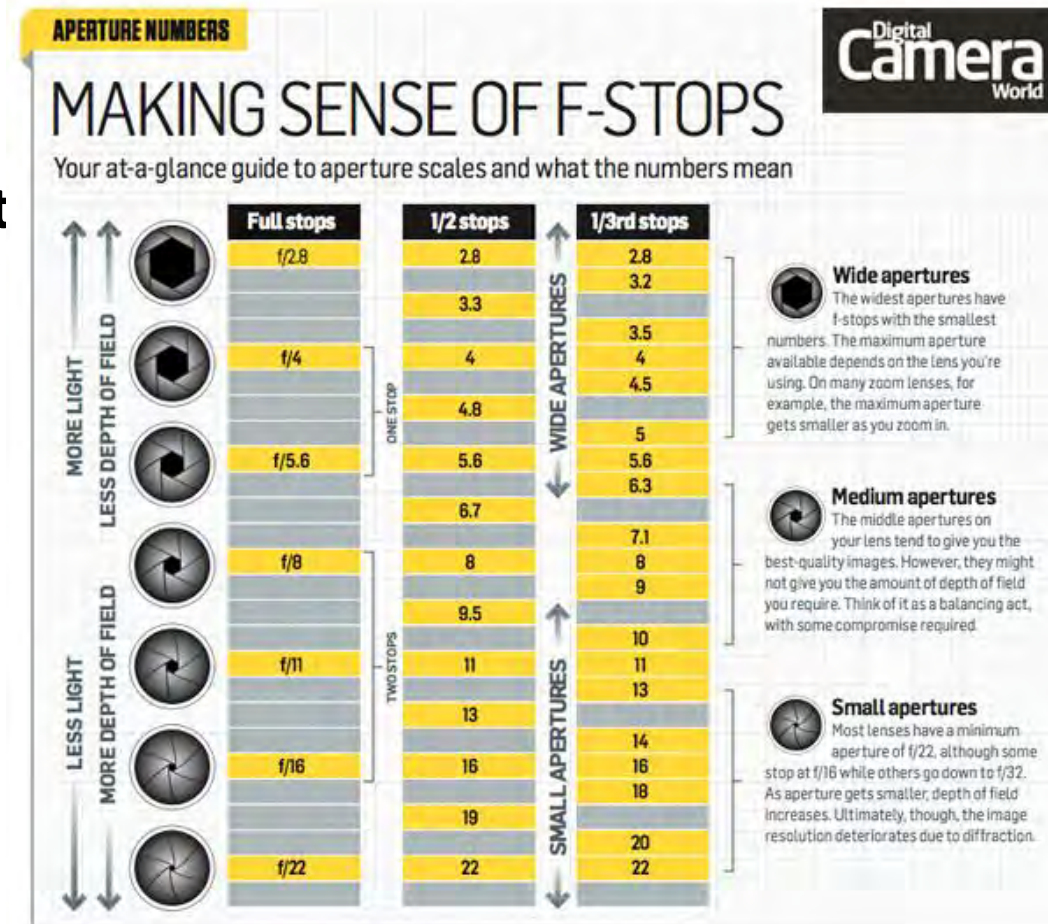


Explanation of the HDRx mode (RED film cameras) – 2 exposures captured:  
<https://www.red.com/red-101/hdrx-high-dynamic-range-video>

# F-stop number (Aperture Stop)

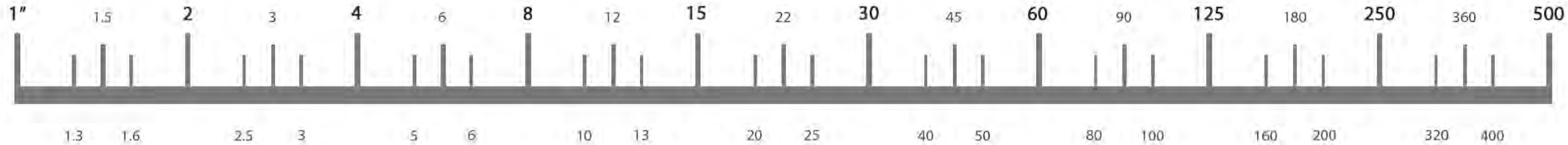
- Each stop is **twice as bright as the previous one**
- Here's the aperture scale. Each step down lets in **half** as much light:
- These are the main aperture “stops,” but most cameras and lenses today let you set some values in between, such as f/1.8 or f/3.5.
- When a camera is said to have **12 stops of dynamic range**, it means that the **brightest part of a scene can be  $2^{12}$  times brighter than the darkest part.**

| f/1.4                          | f/2.0               | f/2.8               | f/4.0                          | f/5.6                                         | f/8.0                           | f/11.0               | f/16.0               | f/22.0                                                        |
|--------------------------------|---------------------|---------------------|--------------------------------|-----------------------------------------------|---------------------------------|----------------------|----------------------|---------------------------------------------------------------|
| Very large aperture            | Large aperture      | Large aperture      | Moderate aperture              | Moderate aperture                             | Moderate aperture               | Small aperture       | Small aperture       | Very small aperture                                           |
| Lets in a huge amount of light | Half as much light  | Half as much light  | Half as much light             | Half as much light (a very “medium” aperture) | Half as much light              | Half as much light   | Half as much light   | Half as much light (by which point your photos are very dark) |
| Very thin depth of field       | Thin depth of field | Thin depth of field | Moderately thin depth of field | Moderate depth of field                       | Moderately large depth of field | Large depth of field | Large depth of field | Very large depth of field                                     |

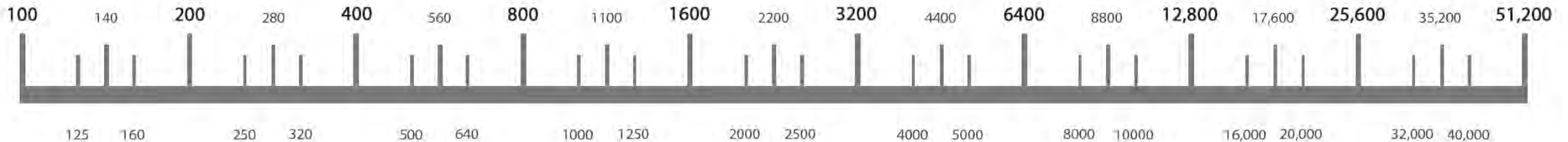




# Shutter Speed & ISO Stops



A shutter speed range of 10 stops with values from one second all the way to 1/500th of a second



Calculating ISO sensitivity stops is very easy – one just needs to divide or multiply the value by 2 to add a stop or go one stop down.

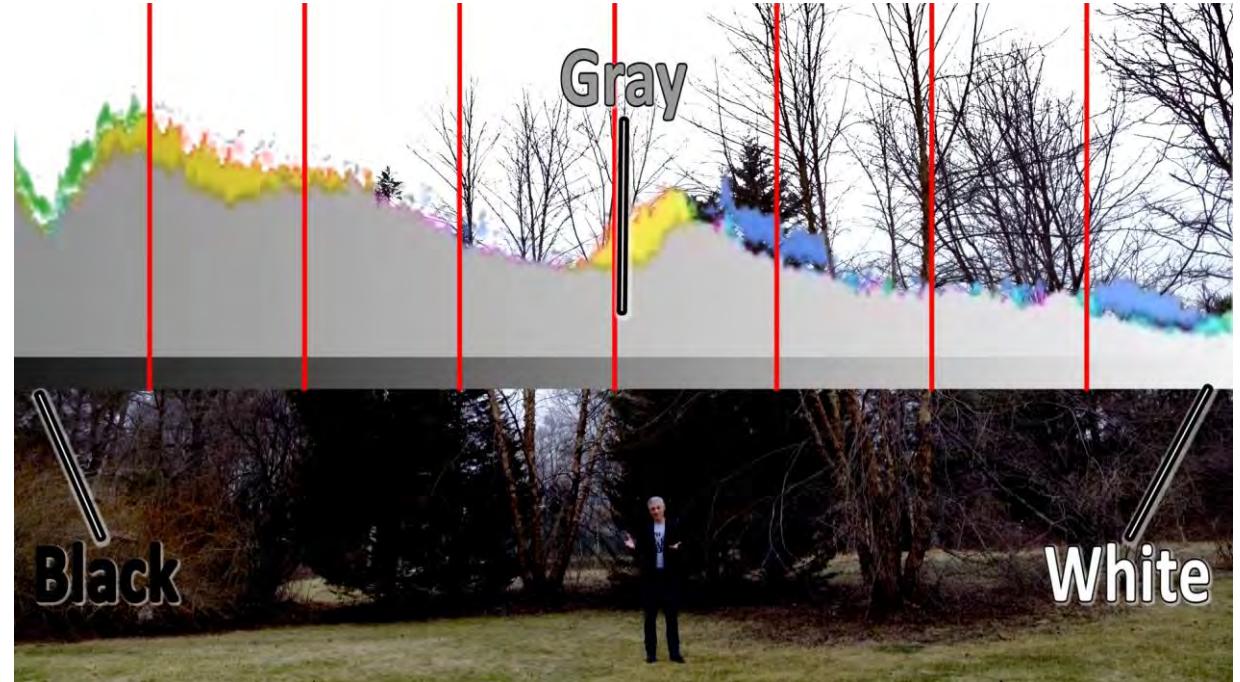
PLEASE NOTE: A digital sensor does not and cannot change its sensitivity to light. There is no setting whatsoever on a digital camera, including ISO, that can modify a sensor's sensitivity to light. ISO in digital is a level of post-capture analog amplification that occurs.



# Dynamic Range Explanations & Examples



<https://www.youtube.com/watch?v=SrXwiPH-fCg>



<https://www.youtube.com/watch?v=MfvwqcmM6FQ>

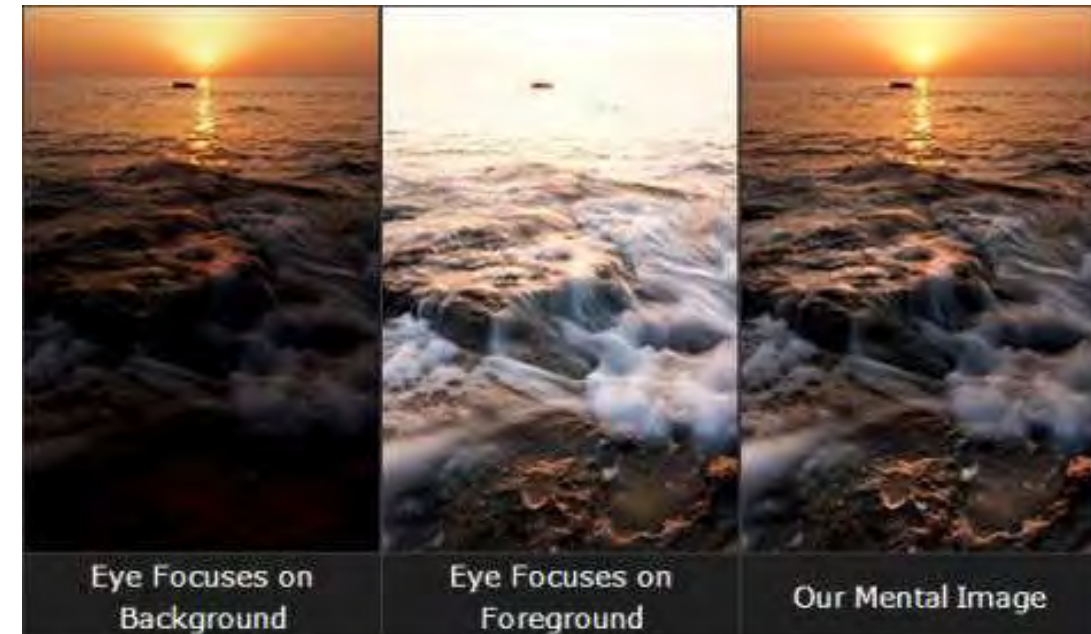
Very good detailed explanation of a film maker about Dynamic Range:

<https://www.youtube.com/watch?v=2sshGdMgJxQ>

# The Human Eye



- Human eye uses the pupil size to see darker or brighter images
  - Turn smaller to limit incoming light to see brighter scenes
  - But cannot see darker & brighter region at same time
  - Can detect a luminance range of  $10^{14}$ , or one hundred trillion (100,000,000,000,000) (**about 46.5 f-stops**), from  $10^{-6}$  cd/m<sup>2</sup>, or one millionth (0.000001 cd/m<sup>2</sup>) to  $10^8$  cd/m<sup>2</sup> or one hundred million (100,000,000 cd/m<sup>2</sup>)



# Stops - Contrast

Dynamic ranges of common devices

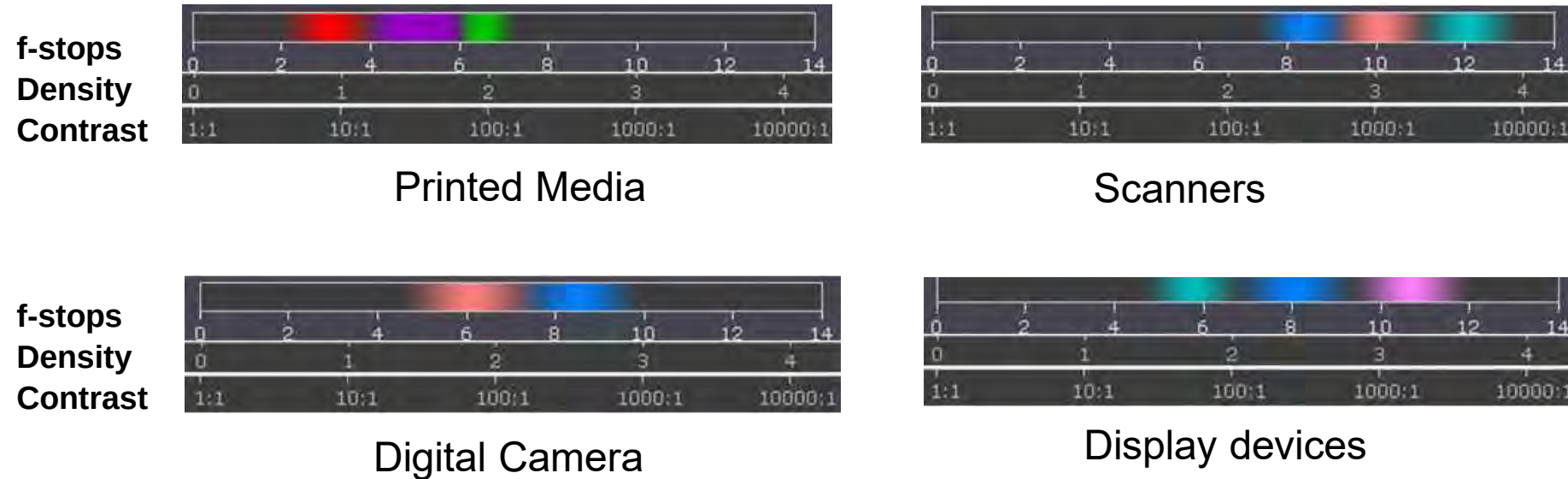
| Device                                | Stops                                       | Contrast ratio         |
|---------------------------------------|---------------------------------------------|------------------------|
| Glossy photograph paper               | 7 (7 - 7 2/3 ) <sup>[58]</sup>              | 128:1                  |
| LCD                                   | 9.5 (8 – 10.8) <sup>[citation needed]</sup> | 700:1 (250:1 – 1750:1) |
| Negative film (Kodak VISION3)         | 13 <sup>[59]</sup>                          | 8000:1                 |
| Human eye                             | 10–14 <sup>[54]</sup>                       | 1000:1 – 16000:1       |
| High-end DSLR camera (Nikon D850)     | 14.8 <sup>[60]</sup>                        | 28500:1                |
| Digital cinema camera (Red Weapon 8k) | 16.5+ <sup>[61]</sup>                       | 92000:1                |

Without adaptation



# Contrast Comparison

- Dynamic range is commonly measured on a logarithmic scale



# Bit Depth & Dynamic Range

- (bit per pixel) Quantifies how many unique colors are available
  - How many unique shades are available in grayscale image

| Bit Precision<br>of Analog/Digital<br>Converter | Contrast Ratio | Dynamic Range |         |
|-------------------------------------------------|----------------|---------------|---------|
|                                                 |                | f-stops       | Density |
| 8                                               | 256:1          | 8             | 2.4     |
| 10                                              | 1024:1         | 10            | 3.0     |
| 12                                              | 4096:1         | 12            | 3.6     |
| 14                                              | 16384:1        | 14            | 4.2     |
| 16                                              | 65536:1        | 16            | 4.8     |

- Most digital cameras use a 10 – 14 bit A/D so their theoretical maximum dynamic range is 10-14 f-stops.
- Higher precision A/D converter does not necessarily mean greater dynamic range. Total dynamic range is usually limited by noise levels.

# Measuring Luminance



## Nit (Candela/m<sup>2</sup>)

- 1 nit (cd/m<sup>2</sup>) approximately equal to the amount of light from a candle dispersed over a square meter



The real world is high dynamic range



| Object                   | Approximate Luminance (nits) |
|--------------------------|------------------------------|
| Sun                      | 1,600,000,000                |
| Arc lamp                 | 150,000,000                  |
| Maximum visual tolerance | 50,000                       |
| Cloud (sunny day)        | 35,000                       |
| 2020 UHD-TV              | 800-1000                     |
| Typical computer screen  | 100-300                      |
| White paper under lamp   | 50                           |
| Night sky                | 0.001                        |
| Threshold of vision      | 0.000003                     |



500,000  
nits



200,000  
nits



40  
nits



2,000 nits

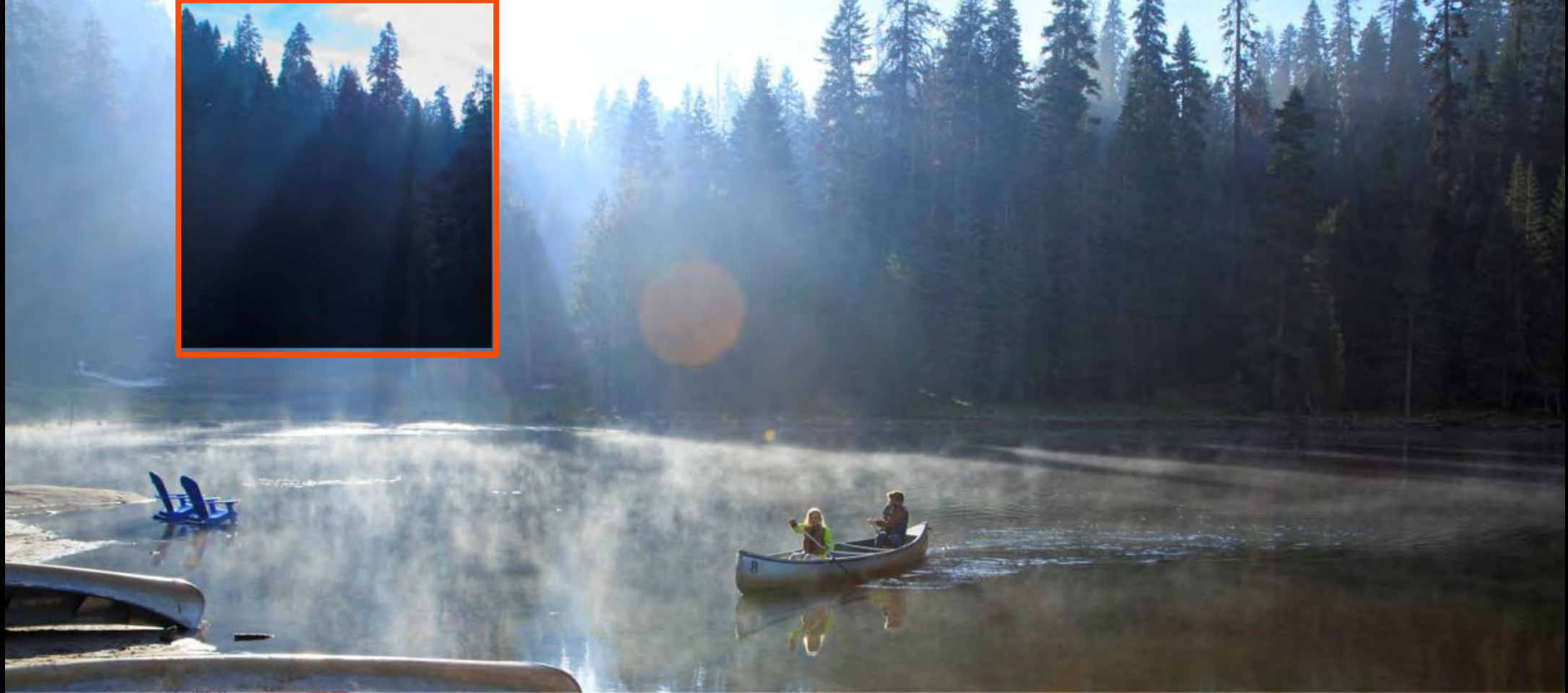














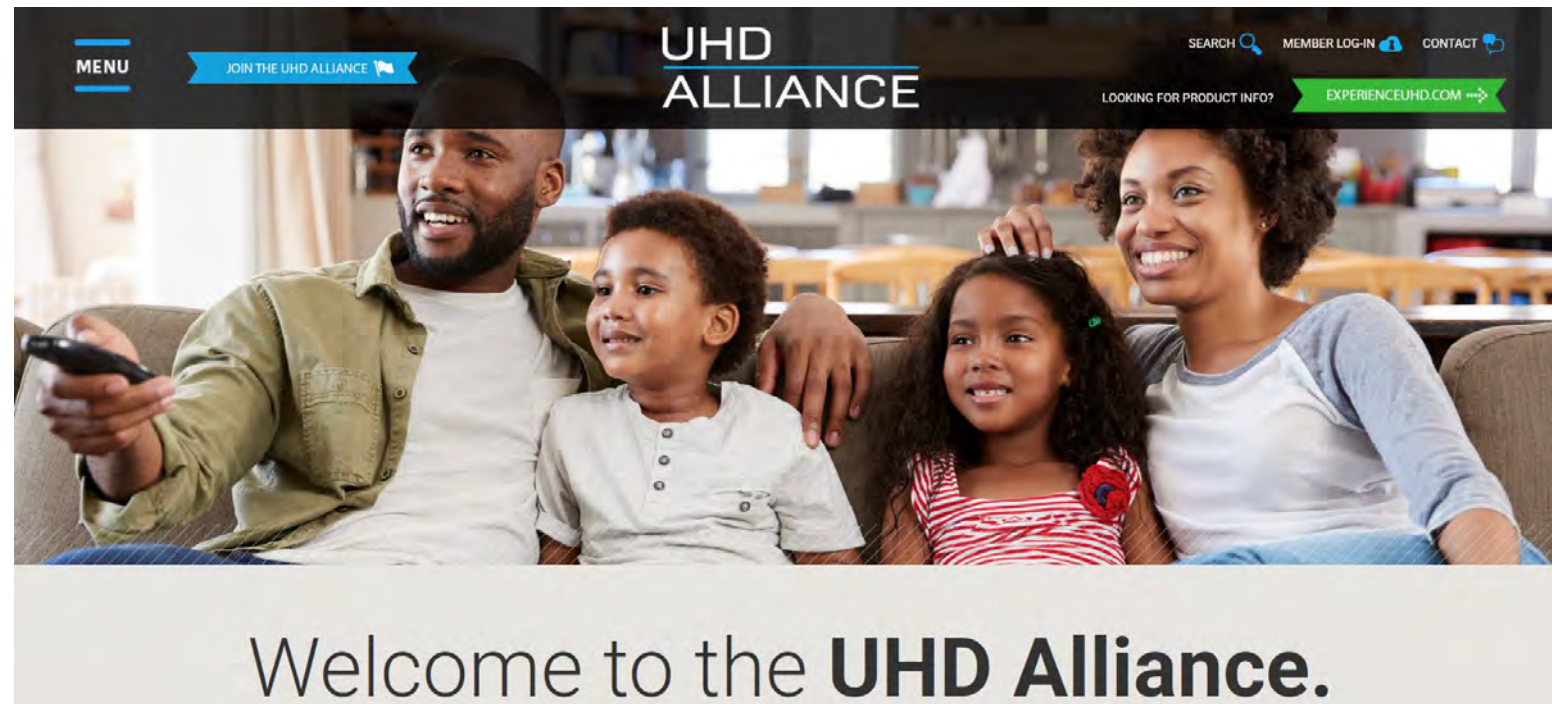
# HDR image examples





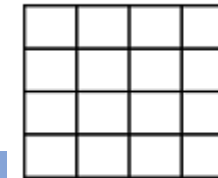
# Moving to HDR

- **UHD Alliance: with mission to create (Jan. 2015 at CES)**
  - *New standard to support video technologies including **4K**, Higher resolution, **high dynamic range**, **wider color gamut**, and **3D audio***

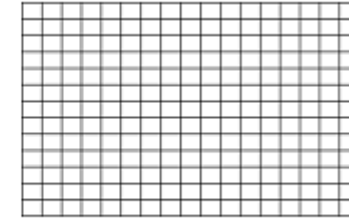


# Improving Image Quality

More pixels: SD – HD –UHD (4k, 8K)



Low resolution



High resolution

**SDTV**  
**HDTV**  
**UHDTV**



UHD vs. HD. vs SD resolution comparison

LEDTVReviews.com

**Increase spatial resolution**

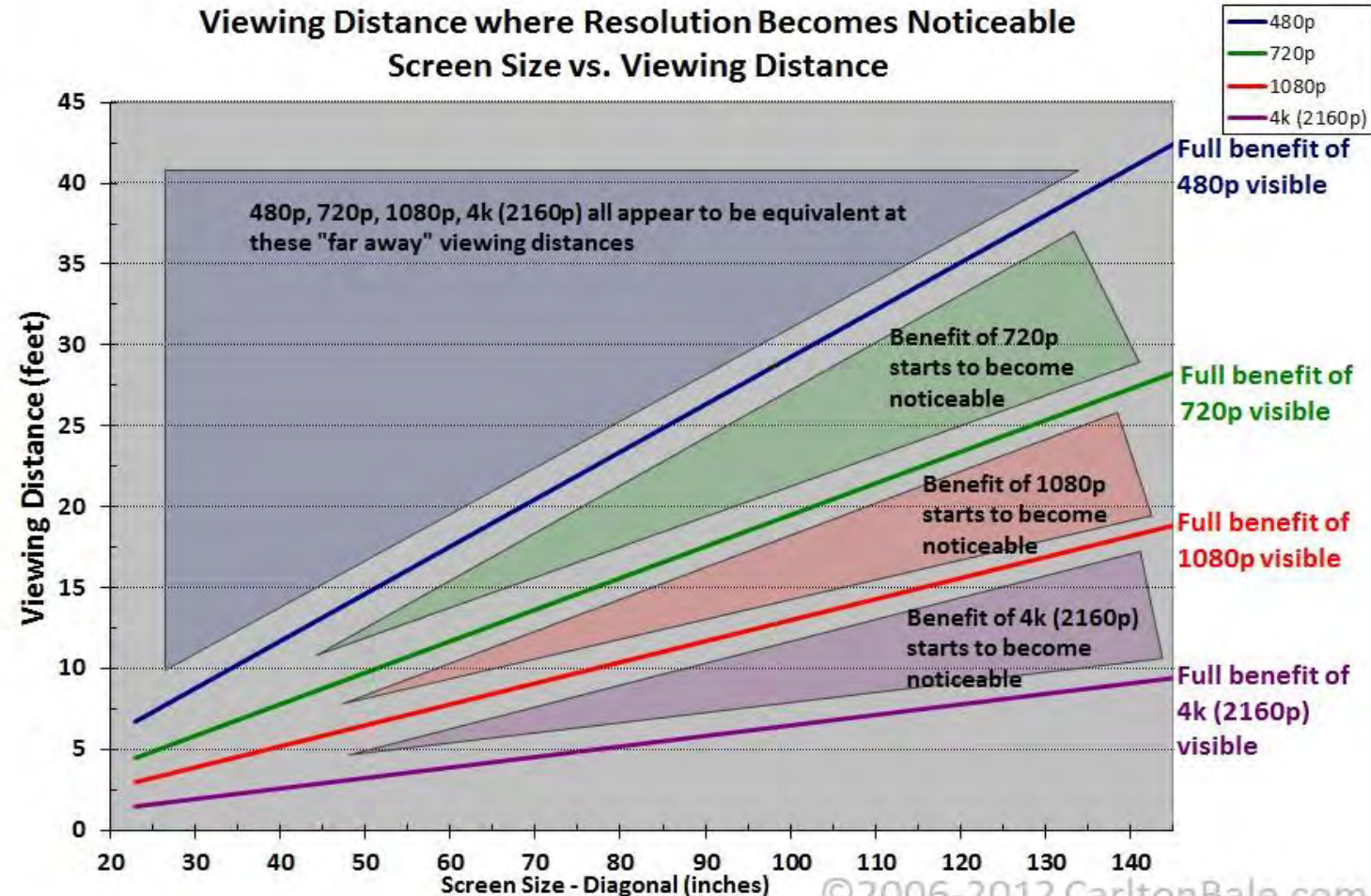
# Moving to HDR

Dramatic better  
visual quality

- (Netflix) HDR is more important for video quality than 4K
  - “We kind of ran out of more pixels to add,” Neil Hunt

Resolution is only apparent  
when you are close

**Users can notice HDR from  
any distance and screen size,  
at once**



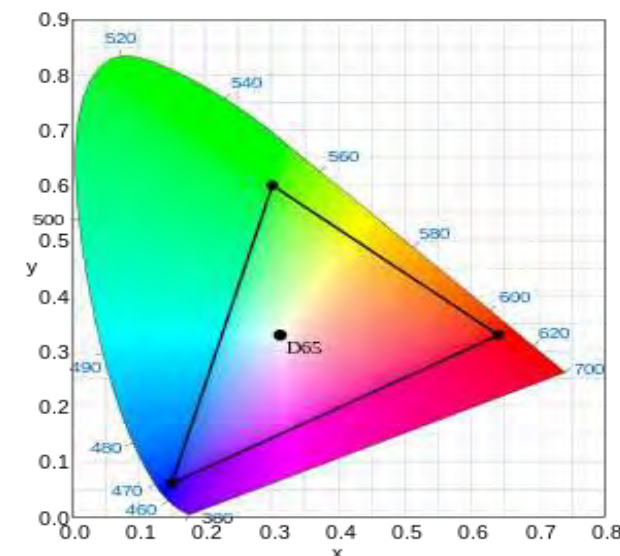


# High Dynamic Range

- Need around 10.000 nits to satisfy 90% viewer at ideal range



Current Color Space (Color Gamut)  
is not enough

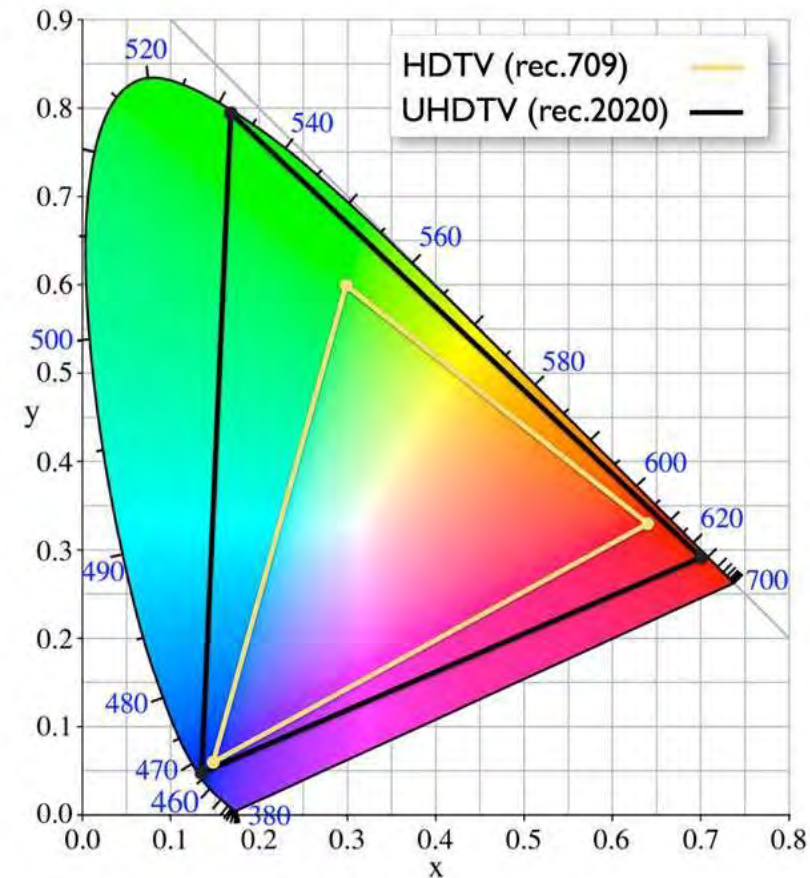


# Wider Color Gamut

- Current color gamut is limited
  - Current HDTV (BT.709) – 1990
  - Created for CRT, and HDTV
  - Around 100 nits.
  - 8 bpp (bits per pixel)

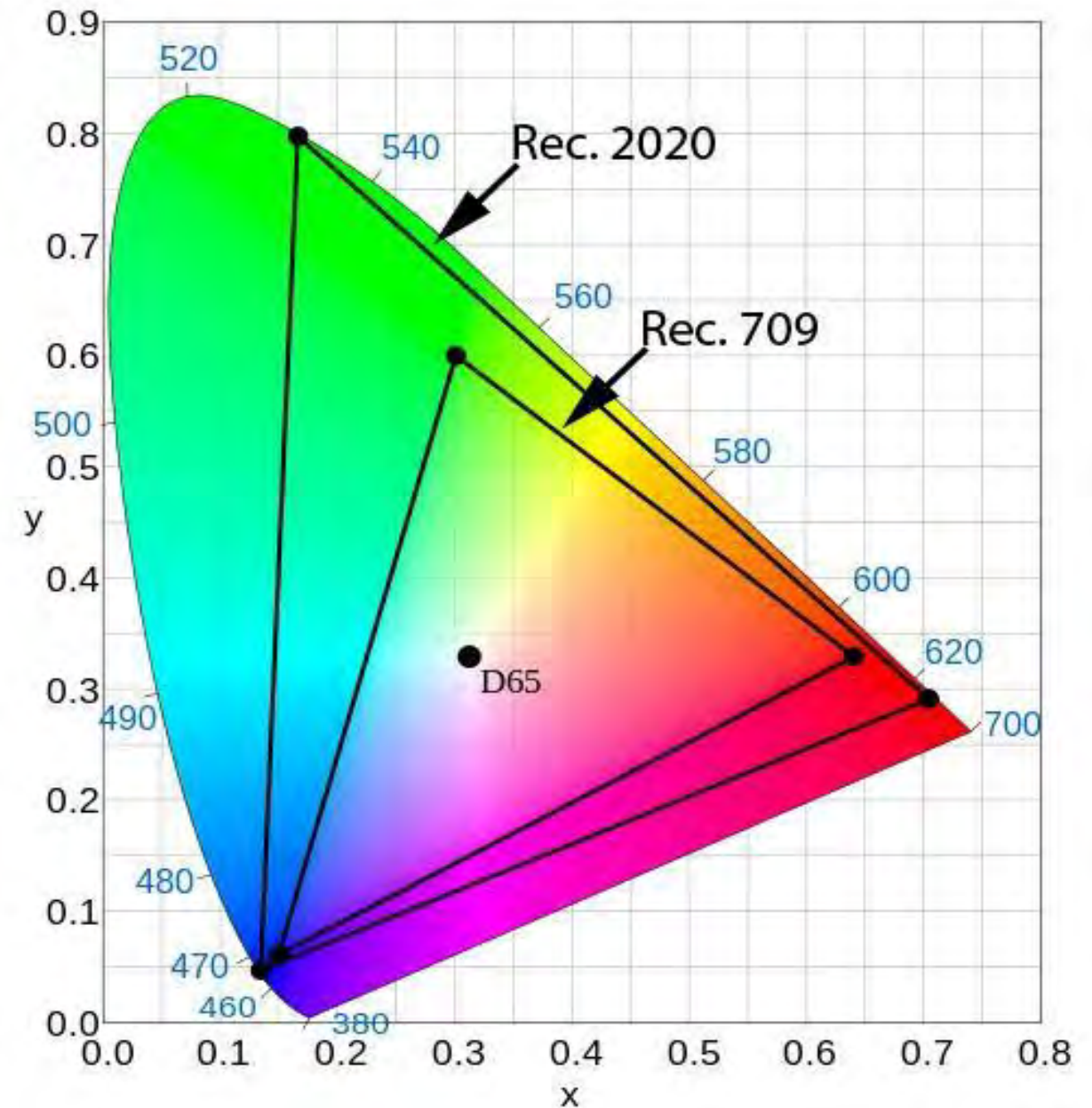
Today we have better display technology: LCD, OLED ...

More Colors: REC. 2020 for UHDTV 10, 12 bpp.

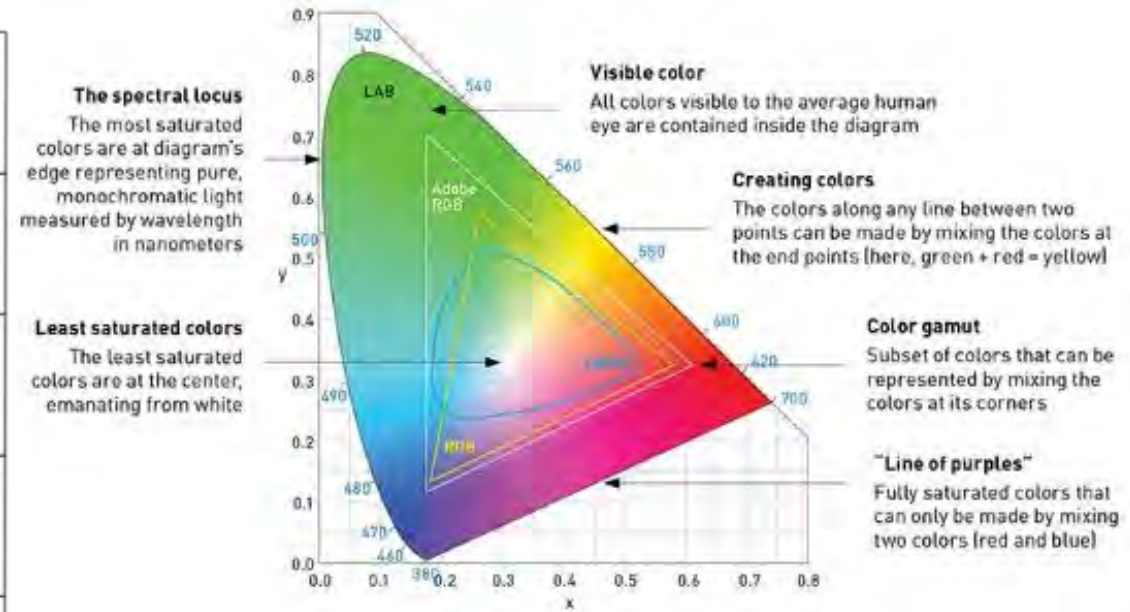
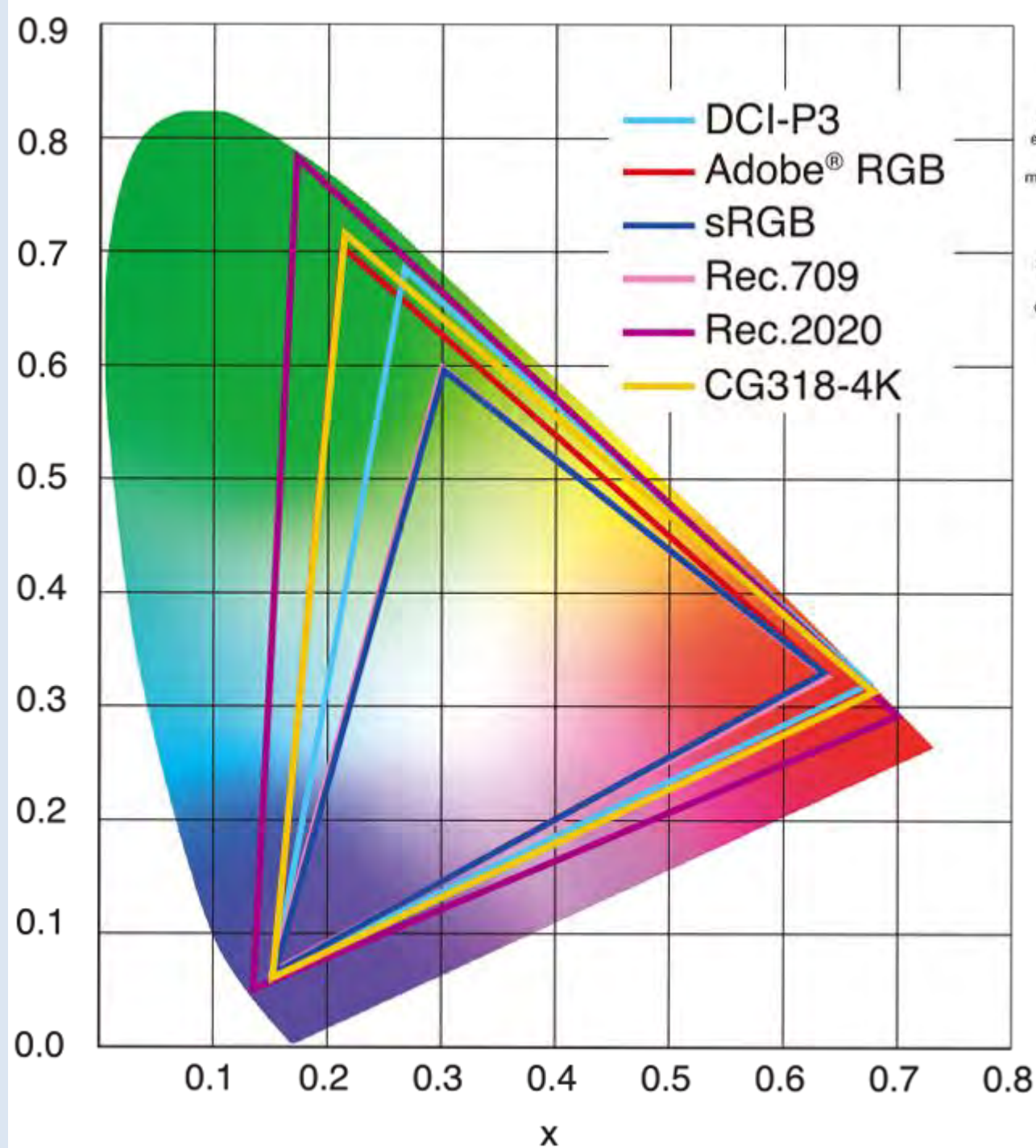


UHDTV vs HDTV Color Gamuts

- Most PCs and HDTVs use 709 gamut
- Majority of next generation content uses Bt.2020 gamut
- 2015 UHD TVs are ~85% Bt.2020







Color profiles of video and monitors

**Bt 2020 content  
displayed unmodified  
on 709 display**



**Bt 2020 content  
gamut-mapped for  
709 display**



**We must perform gamut mapping**



# Standard Dynamic Range (SDR)

- For the past 80 years, video has been graded to appear properly on a **100 nit** display
- However, no adjustments are made to comprehend displays of differing luminance
  - 30 nit laptops in low power mode
  - 600 nit HDTVs in Vivid mode
- Similarly, no adjustments to the content are made when you surf the web and play games, switching from phones to tablets to PCs.

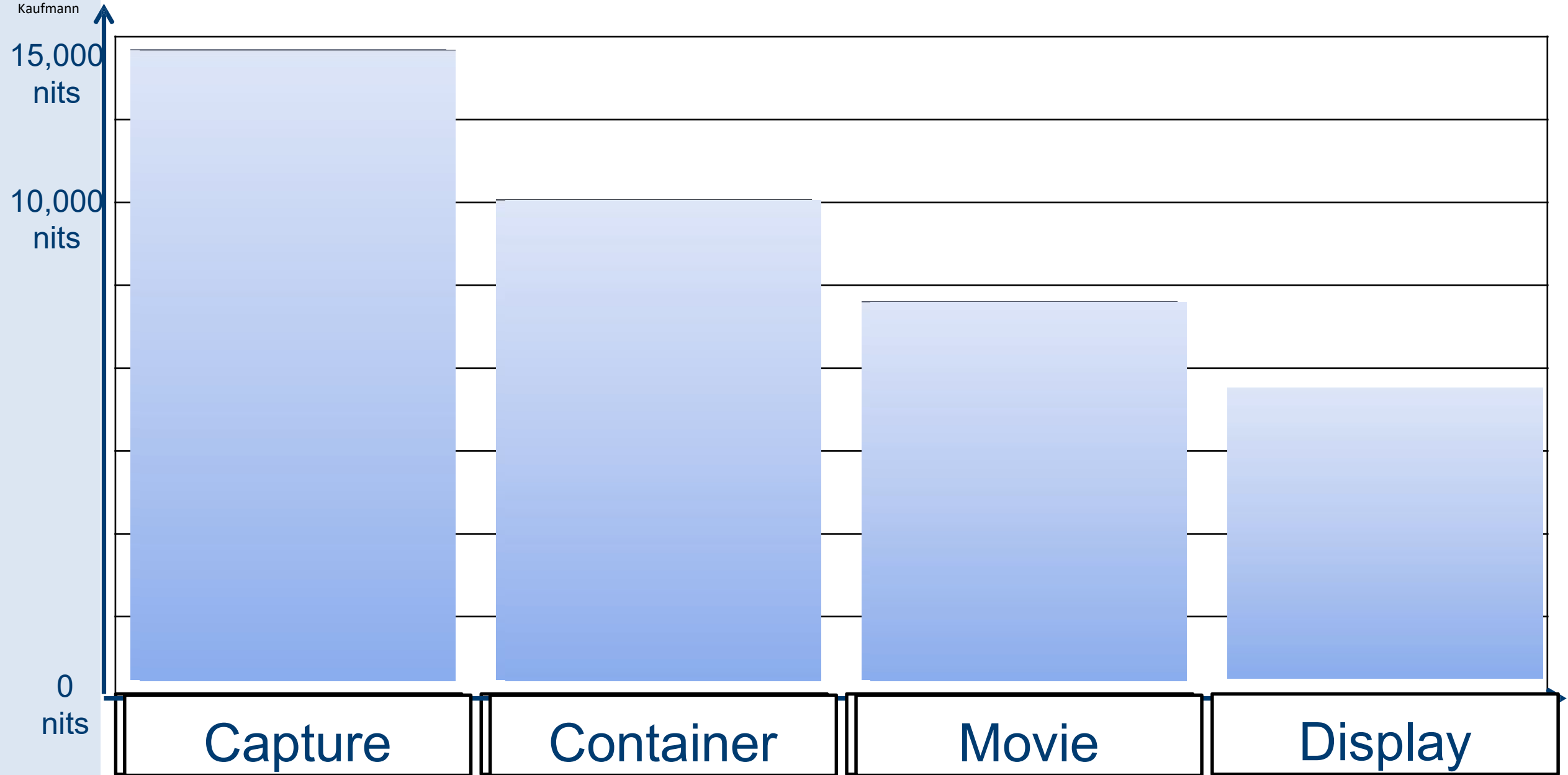
This is **wrong**, but has been **good enough**

# Things to ponder

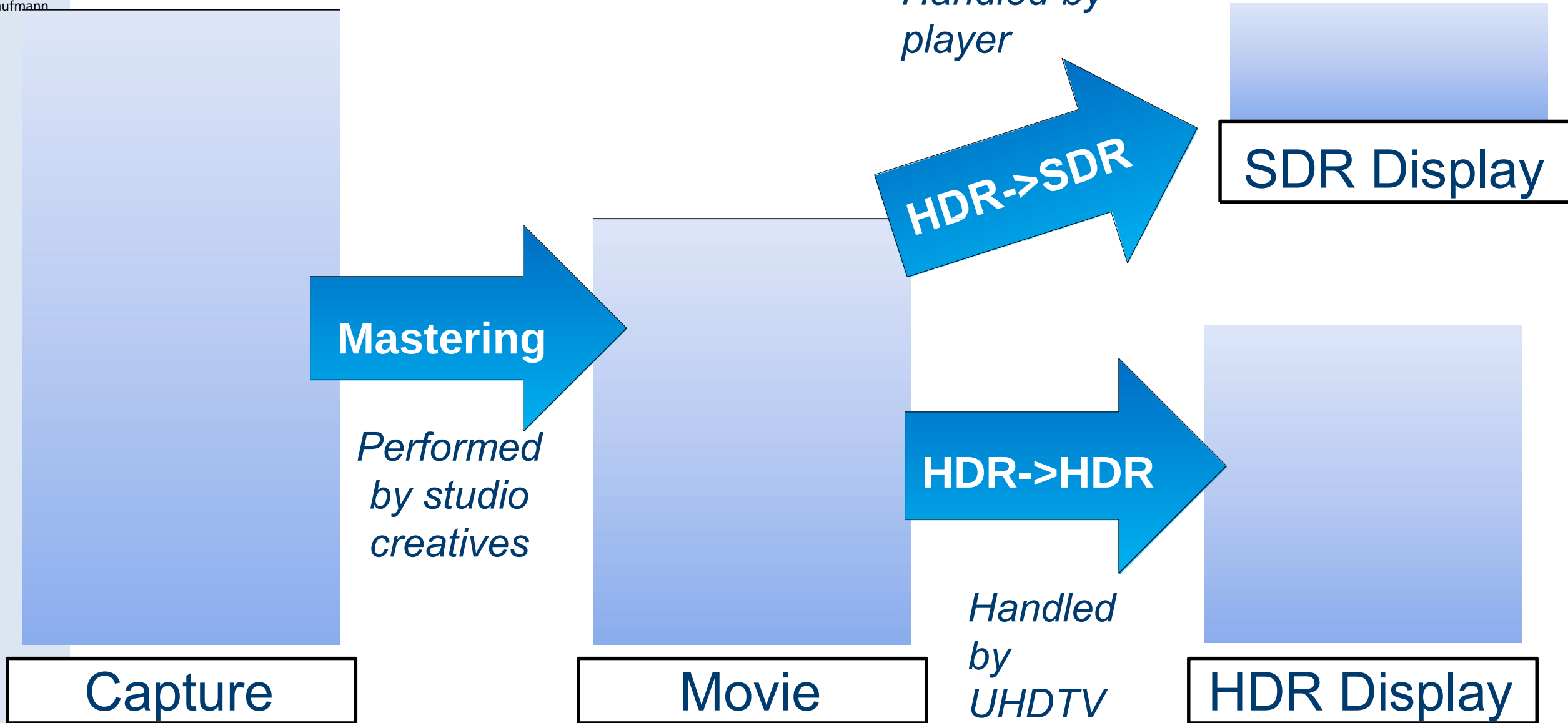
- With color, many real-world hues can be exactly reproduced on screens
- Almost every real-world situation has objects at  $>100$  nits
- So the real world is HDR, and every SDR picture you take involves HDR- $\rightarrow$ SDR conversion
- A 100-nit object on the screen almost never corresponds to a 100-nit object in the real world



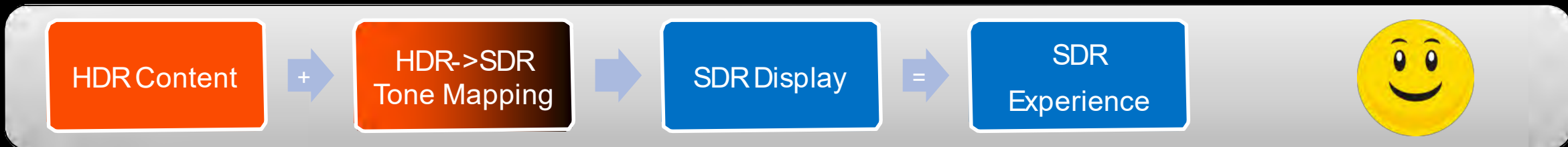
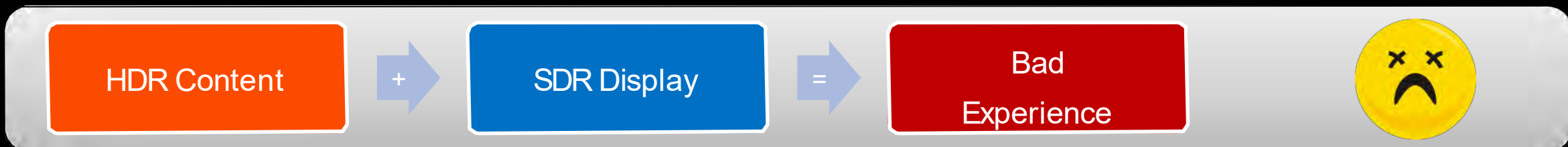
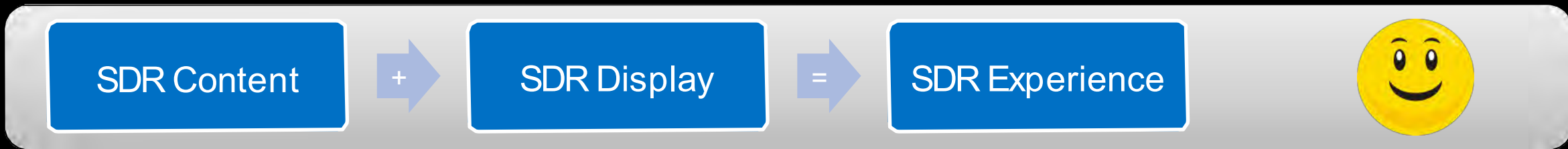
# Multiple Ranges of HDR



# HDR Conversions







# Real World

**3,000  
nits**

**300,000  
nits**



# Mastered for SDR

**100  
nits**



**Coded  
value=  
0xFF**

**100  
nits**



**Coded  
value=  
0xFF**



# Mastered for HDR

200  
nits



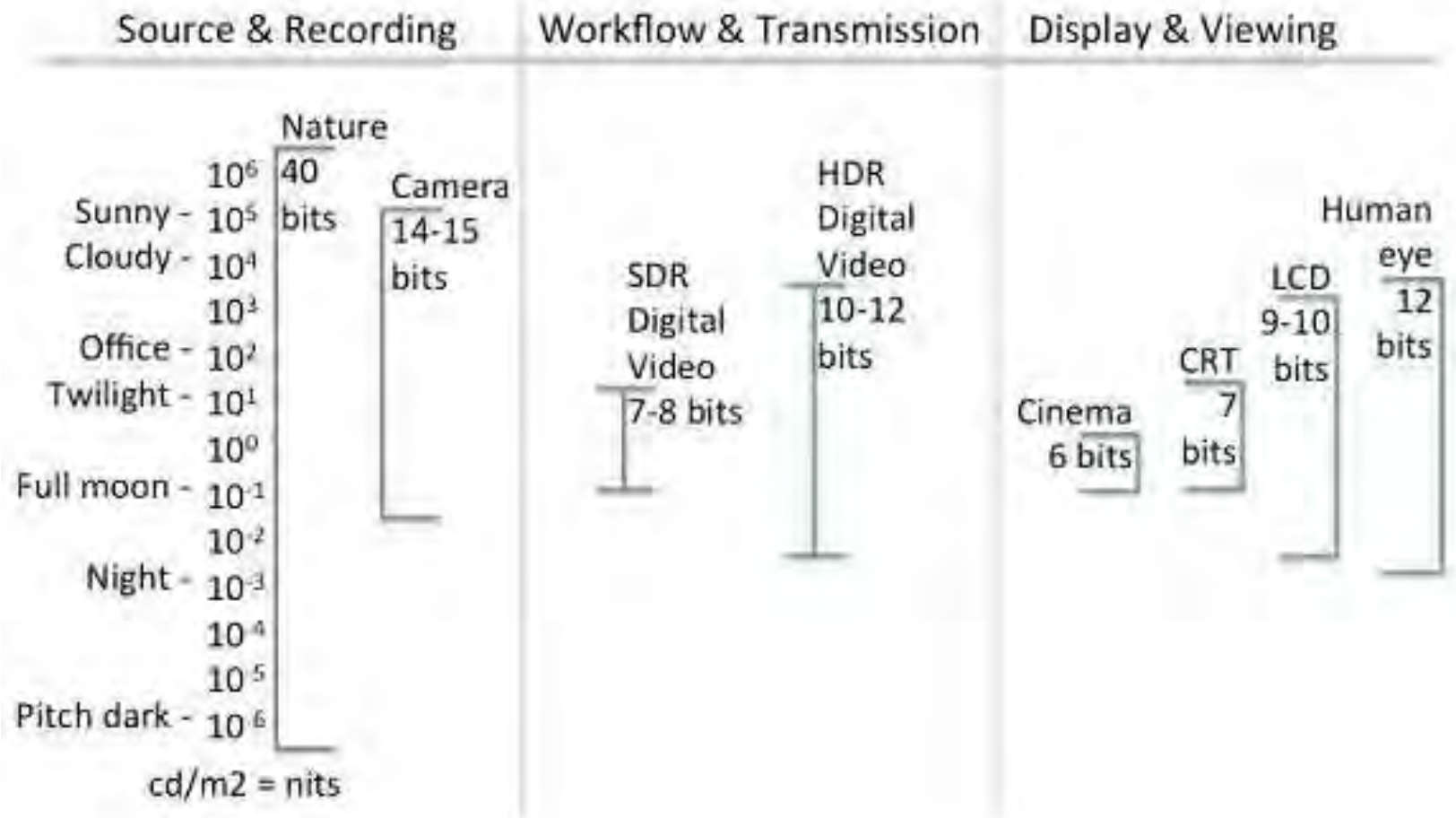
Coded  
value=  
0x23B

5,000  
nits



Coded  
value=  
0x36C

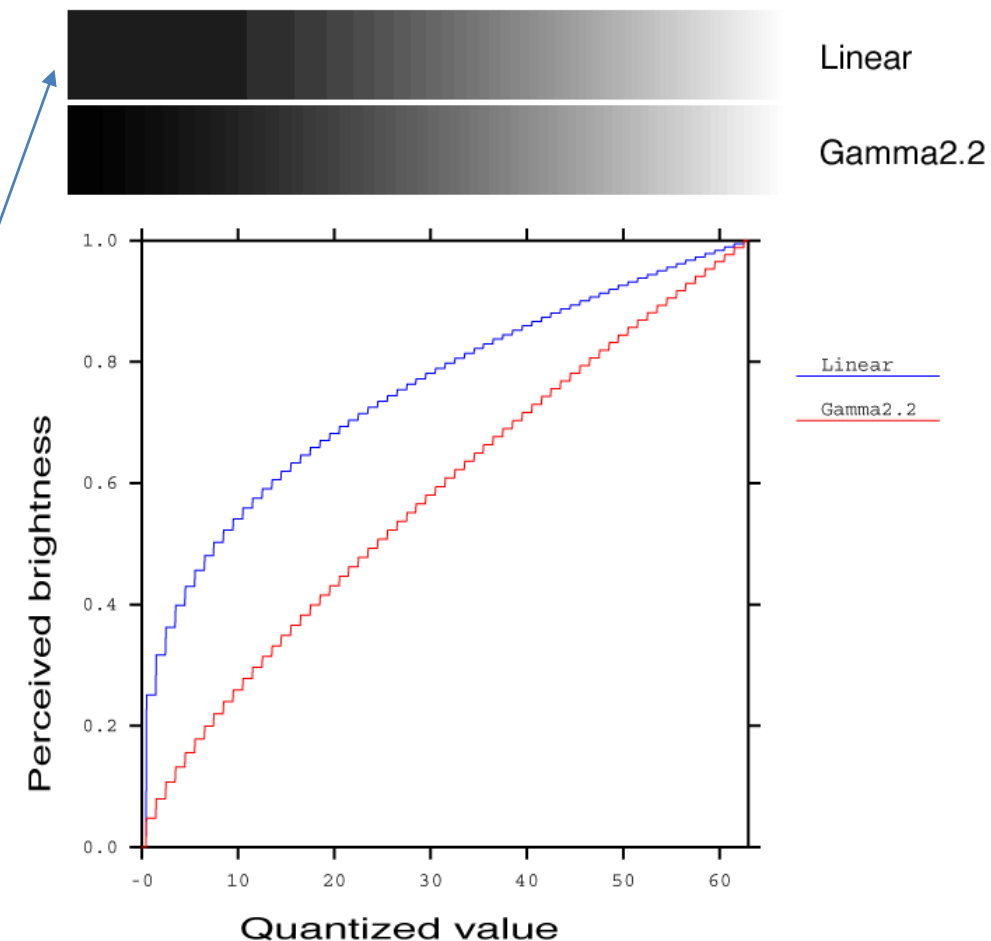
# Color and Brightness



Luminance dynamic range for various technologies

# Quantization: Linear vs non-linear light

- In the real world, luminance is determined by the number of photons
- However, brightness (the human perception of luminance) is non-linearly proportional to number of photons
  - The eye has a non-linear response to intensity. At most adaptation levels, we perceive brightness roughly as the cube root of intensity.
- If we apply a linear quantization of color values, we see more steps in darker regions than we do in the brighter regions
- Using a power law encoding with a  $g$  value of 2.2, we see a much more even distribution of quantization steps, though the behavior near black is still not ideal



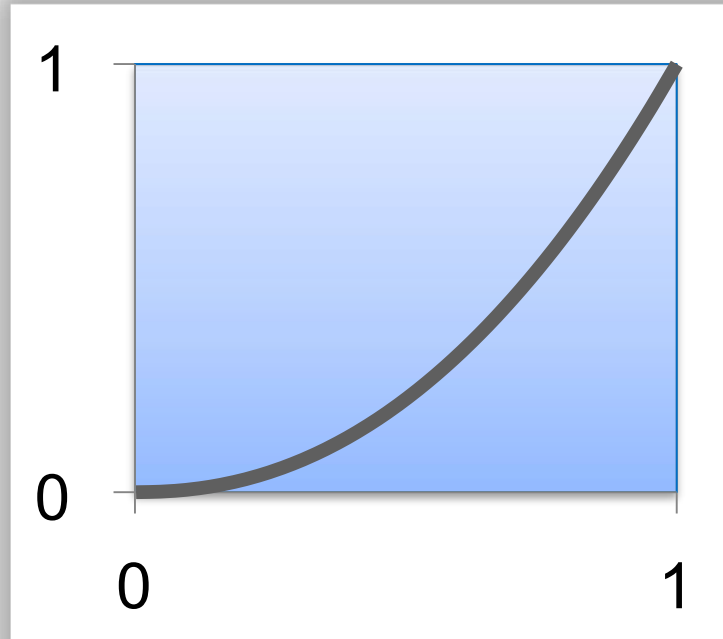
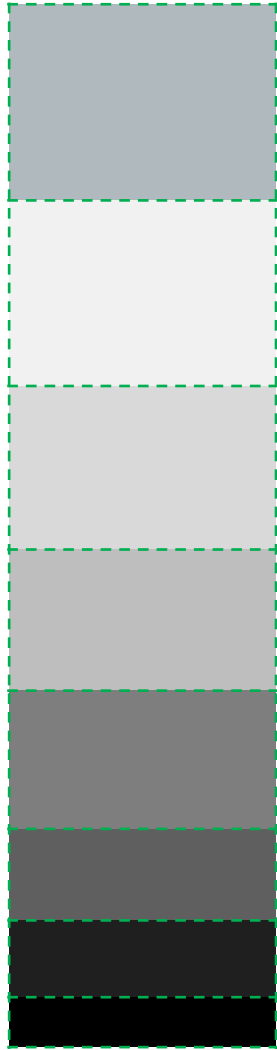


To efficiently code video signals, we apply a non-linear transform function so that we can code perceptually uniform brightness intervals

Luminance

Linear Light

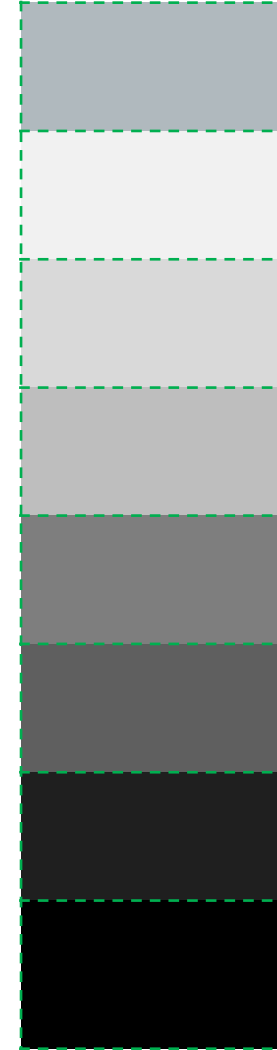
Perceptually  
non-uniform



Luma

Non-Linear Light

Perceptually  
Uniform



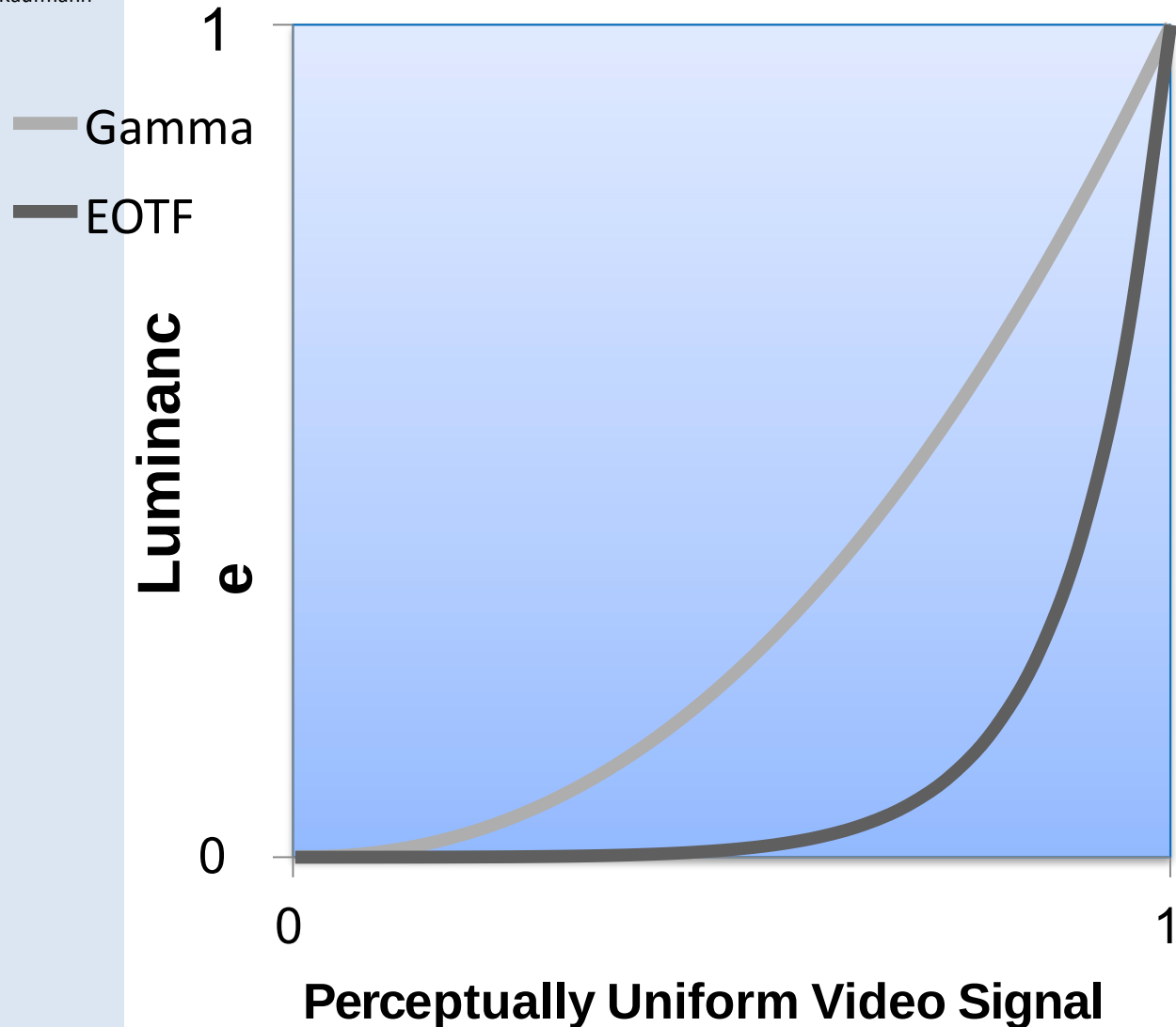
# Problems

- Although gamma is a reasonable approximation of human perception in the SDR (0-100 nit) range...
- Gamma is not a good match for human perception in the 0-10,000 nit HDR
- If you use gamma for a 10,000 nit signal, you need to use two extra bits/sample to eliminate banding

Color banding



# SMPTE 2084: HDR Electro-Optical Transfer Function (EOTF)



Electro-Optical Transfer Function (EOTF) is a correction function that **converts electrical voltages into brightness values**.

- If gamma curve is used = gamma correction, if more complex dependencies between voltage and brightness, an EOTF is used
- EOTF is used to adjust the brightness of displays or televisions to the human perception of brightness. Since this does not run linearly with voltage values, the display representation must be adapted.
- **Dependencies between brightness and voltage are different for various display technologies** and resolutions, such as SDTV, HDTV, UHDTV, HDR10 and Dolby Vision.
- Therefore several correction curves in addition to gamma correction: Perceptual Quantization (PQ) and Hybrid Log Gamma (HLG), which is described in the SMPTE's ST.2084 standard for video transmission of High Dynamic Range (HDR).

# Hybrid Log Gamma (HLG) & Perceptual Quantization

- HLG correction is comparable to gamma correction in classic television.
- Hybrid Log Gamma (HLG) and Perceptual Quantization (PQ)
  - standardized by the International Telecommunication Union (ITU) in BT.2100
  - adapting the dynamic range of the display on the UHD displays to the dynamic range of the recording devices.
  - Why: So that display corresponds to the viewer's perception of brightness.
  - Correspond to a logarithmic function
  - The brightness range corresponds to a luminance between 100 nits and 5,000 nits.
  - HLG correction curve requires a resolution of at least 10 bits and is used with High Dynamic Range HDR10 and Dolby Vision in transmission and mastering.
  - When transmitting video in SDR technology and HDR technology simultaneously in one data stream, the image content is encoded with the HLG curve so that it can be played back on both SDR and HDR displays.

# What is HDR (in this context)

- Content with a wider range of brightness and color
- Requires new monitors to experience HDR content
  - Legacy content (SDR): 8-bit color, 100 nit peak brightness, 709 gamut
    - Dynamic range of about 6 stops
  - HDR content: 10-bit color, 10,000 nit peak brightness, 2020 gamut
    - Dynamic range of about 17,6 stops

HDR is an ambiguous term

- There are several industry specs defining various flavors of HDR

# Summary

- HDR + Wide color gamut (WCG) gives us powerful new tools to create visual experiences that are clearly visible to the average viewer
- Making this all work requires lots of changes to the production pipeline and the products used to display it

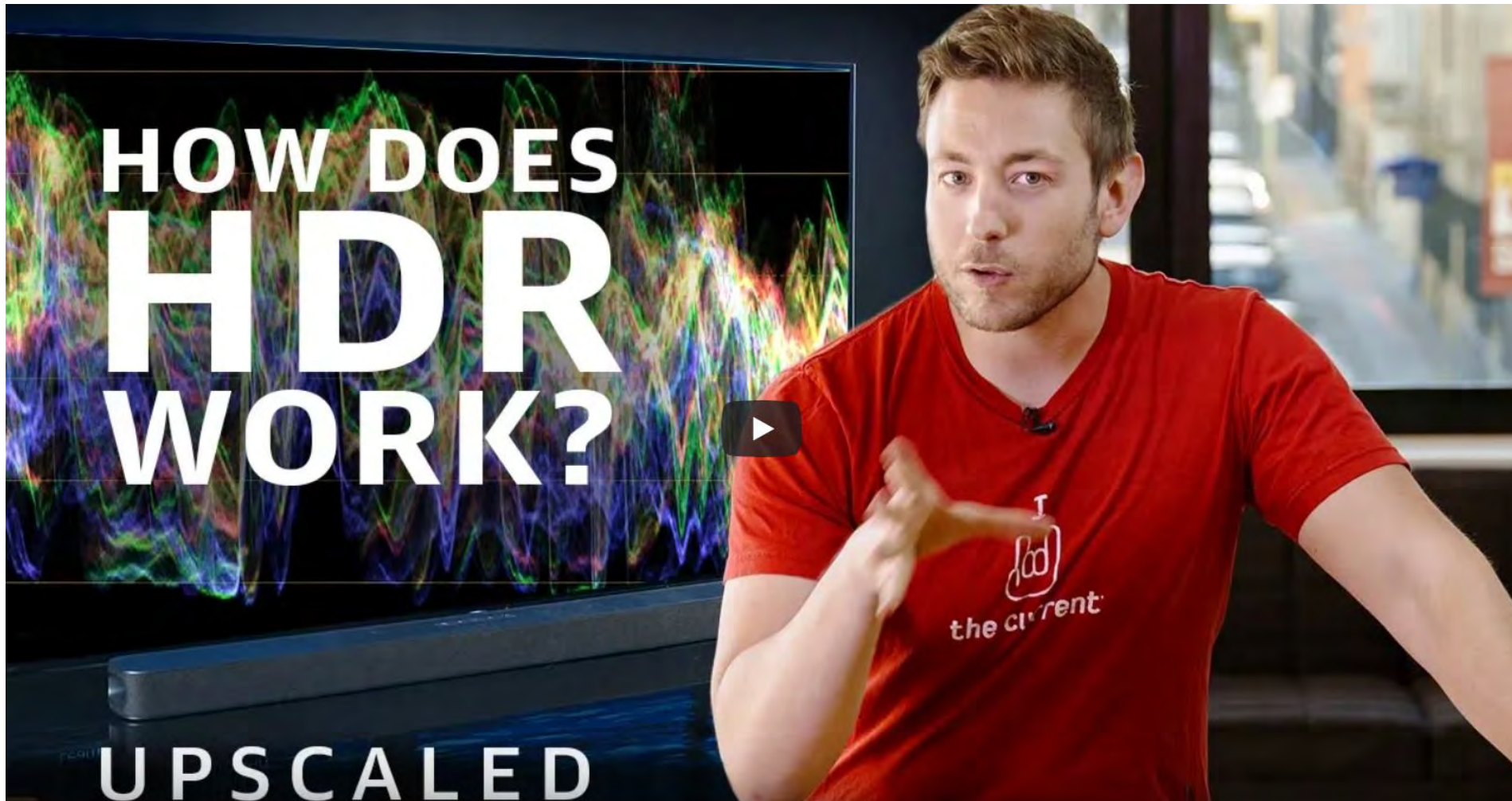


# HDR Encoding

- **Most formats are lossless**
- **Adobe DNG (digital negative)**
  - Specific for RAW files, avoid proprietary formats
- **RGBE**
  - 24 bits/pixels as usual, plus 8 bit of common exponent
  - Introduced by Greg Ward for Radiance (light simulation)
  - Enormous dynamic range
- **OpenEXR**
  - By Industrial Light + Magic, also standard in graphics hardware
  - 16bit per channel (48 bits per pixel) 10 mantissa, sign, 5 exponent
  - Fine quantization (because 10 bit mantissa), only 9.6 orders of magnitude
- **JPEG 2000**
  - Has a 16 bit mode, lossy

# HDR Technologies Summary

Luminance  
Dynamic Range  
Color Space  
Bit Depth



# HDR formats

- Summary of all HDR encoding formats (Greg Ward):  
[http://www.anywhere.com/gward/hdrenc/hdr\\_encodings.html](http://www.anywhere.com/gward/hdrenc/hdr_encodings.html)
- Greg's notes: <http://www.anywhere.com/gward/pickup/CIC13course.pdf>
- <http://www.openexr.com/>

# Some Standard Image Formats

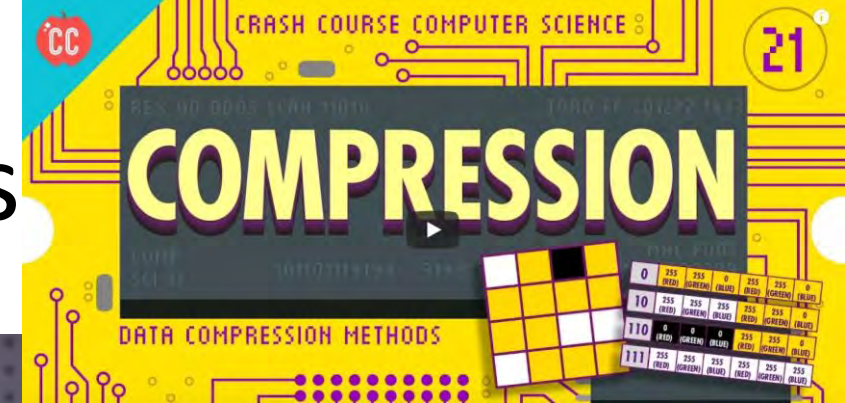
- EPS — Encapsulated PostScript
- **GIF — Graphics Interchange Format**
- PICT — may contain geometric data (lines, polygons, etc.)
- **JPEG — Joint Photographic Experts Group, lossy**
- PNG — includes transparency (alpha channel)
- and many many more....

# Image Compression



# Recap: Compression Algorithms

<https://www.youtube.com/watch?v=OtDxDvCpPL4>





# Compression Techniques - Classification

|                |                     |      |
|----------------|---------------------|------|
| Entropy coding | run-length coding   |      |
|                | Huffman coding      |      |
|                | arithmetic coding   |      |
| Source coding  | prediction          | DPCM |
|                |                     | DM   |
|                | transformation      | FFT  |
|                |                     | DCT  |
|                | vector quantization |      |
| hybrid coding  | JPEG                |      |
|                | MPEG                |      |
|                | px64                |      |
|                | DVI (RTV, PLV)      |      |

# Entropy Coding

- **lossless** encoding
- used **regardless of media's** specific **characteristics**
- data taken as simple digital sequence
- One of the main types of entropy coding
  - creates and **assigns a** unique prefix-free **code to each unique symbol** that occurs in the input.
  - then **compress data by replacing** each fixed-length **input symbol with** the corresponding variable-length prefix-free output **codeword**.
- Examples: **Run-length encoding, Huffman encoding, Arithmetic encoding**

# Source Coding & Hybrid Coding

- Usually **lossy** encoding (can also be lossless)
- **takes into account the data semantics**
- degree of compression **depends on data content**
- In source coding the particular **code** to be used **is chosen to match the source** (i.e. the relative probabilities of the symbols in the source alphabet).
- Example: Discrete Cosine Transformation (DCT) as transformation technique of the spatial domain into the two-dimensional frequency domain

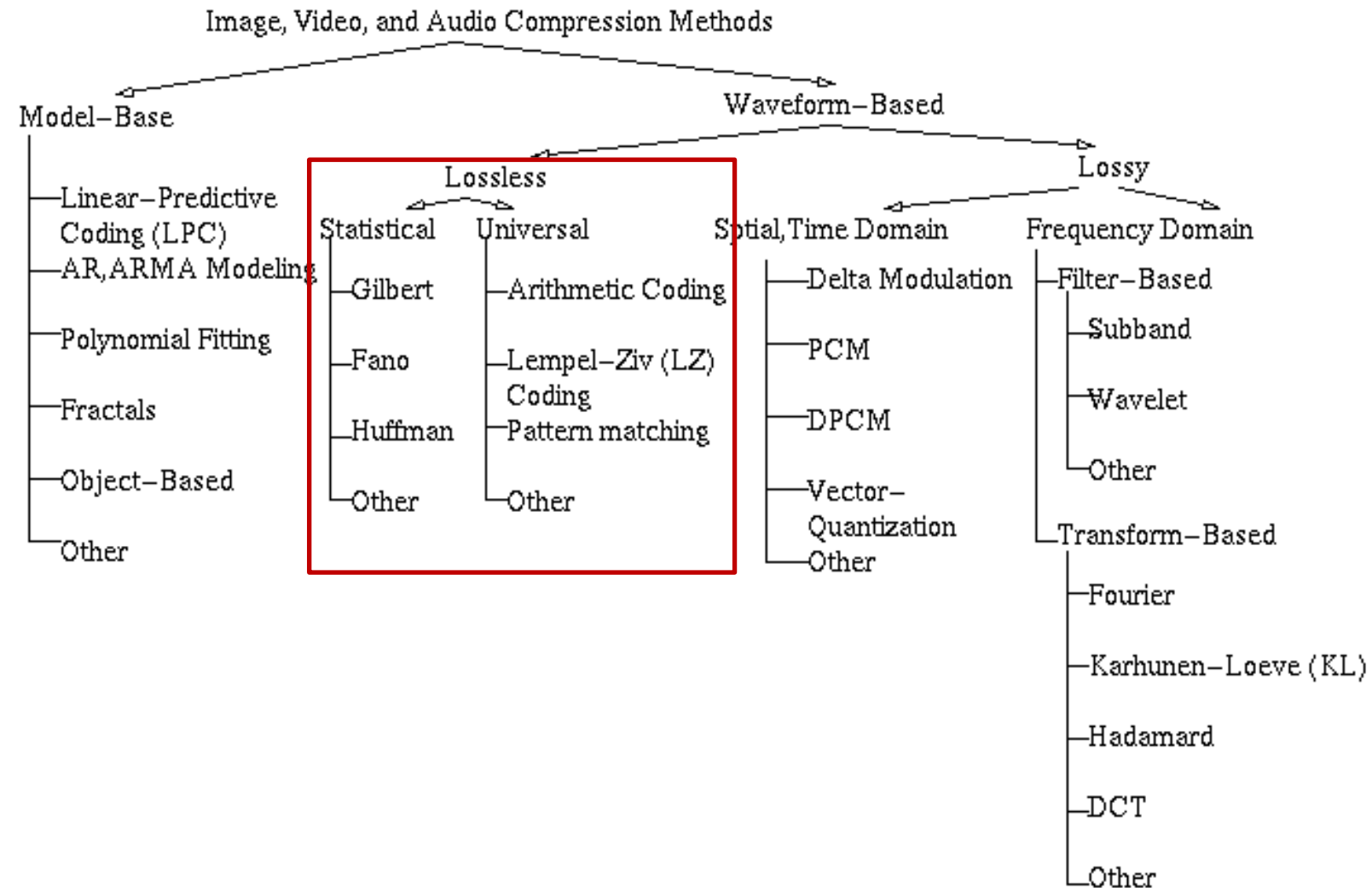
## Hybrid encoding

- Used by most multimedia systems
- Combination of entropy and source encoding
- Examples: JPEG, MPEG, H.261

# Decompression Requirements

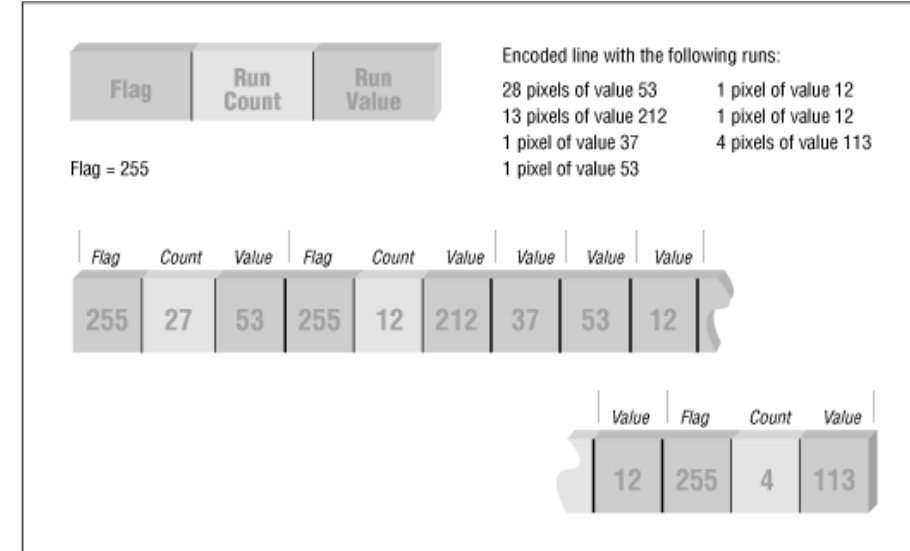
- Dialog mode applications (e.g. editing) might need **symmetric** compression  
= compression and decompression take about the same time
- Retrieval mode applications (e.g. video playback) use **asymmetric** compressions
  - compression is performed once and sample time available
  - decompression is performed frequently and needs to be done fast (in case of time-dependent media)

# Lossless Compression Algorithms



# Run-Length Encoding

- **Entropy coding** algorithm—  
content independent coding
- Replaces the sequence of the same consecutive bytes with the number of occurrences
- the number of occurrences is indicated by a *special flag (!)*
- Procedure:
  - if the same byte occurs at least 4 times count number of occurrences
- write compressed data in the format
  - the counted byte '!' number of its occurrences





# Run-length Encoding

- Example:
  - uncompressed sequence (20 Bytes):  
ABCCCCCCCCCDEFFFFGGG
  - compressed sequence (13 Bytes): ABC!9DEF!4GGG
- Variations
  - Zero suppression technique (remove zeros)
  - text compression technique (also for images, video, audio)
  - diatomic encoding (represents a pair of characters as one special character)
- Is used for older image file formats e.g. Windows Bitmap, Targa or PCX.  
In Windows \*.rle is used

# Statistical Encoding

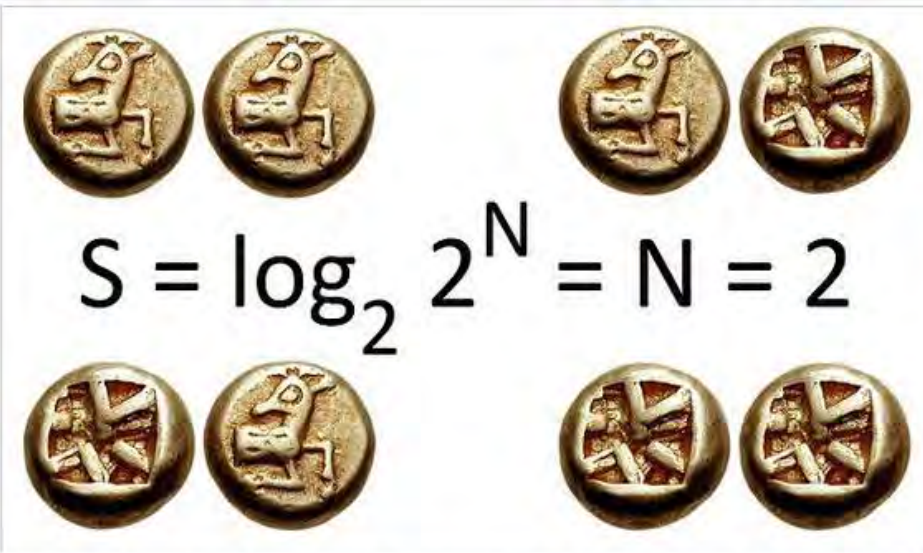
- Frequency dependent encoding—belongs to **entropy encoding**
- given is a sequence of symbols:  $S_1, S_2, S_3, \dots$  and the probability of occurrence of each symbol  $p(S_i)=p_i$
- Ex.:  $p(A)=0.16, p(B)=0.51, p(C)=0.09, p(D)=0.13, p(E)=0.11$   
first choice: encode A,B,C,D,E as 000,001,010,011,100

Q: What is the minimum number of bits per symbol?

A: The theoretical minimum average number of bits per code word is known as **entropy** ( $H$ ), according to Shannon

# Entropy

- Introduced by Claude Shannon in his 1948 paper "A Mathematical Theory of Communication" - sometimes called **Shannon entropy**
- Example: A biased coin with probability  $p$  of landing on heads and probability  $1-p$  of landing on tails. The maximum surprise is for  $p = 1/2$ , when there is no reason to expect one outcome over another, and in this case a coin flip has an entropy of **one bit**. The minimum surprise is when  $p = 0$  or  $p = 1$ , when the event is known and the entropy is zero bits.
- Shannon's experiments with human predictors show an information rate between 0.6 and 1.3 bits per character in English.
- Shannon's entropy **measures the information contained in a message** as opposed to the portion of the message that is determined (or predictable). Examples of the latter include redundancy in language structure or statistical properties relating to the occurrence frequencies of letter or word pairs, triplets etc.



Two bits of entropy: In the case of two fair coin tosses, the information entropy in bits is the base-2 logarithm of the number of possible outcomes; with two coins there are four possible outcomes, and two bits of entropy. Generally, information entropy is the average amount of information conveyed by an event, when considering all possible outcomes.

$$H(X) = - \sum_{i=1}^n P(x_i) \log P(x_i)$$

# Huffman Encoding

- Statistical encoding, depends on **occurrence frequency of single characters or sequences** of data bytes
- Characters stored with their probabilities
- Length (number of bits) of the coded characters differs
- Shortest code assigned to the most frequently occurring character
- To determine Code, it is useful to construct a **binary tree**
  - leaves are characters to be encoded
  - nodes carry **occurrence probabilities** of the characters belonging to the subtree

# Huffman Encoding — Algorithm

build a Huffman Tree:

$p(A)=0.17$ ,  $p(B)=0.51$ ,  $p(C)=0.08$ ,  $p(D)=0.14$ ,  $p(E)=0.1$

```
build ordered list of symbols (increasing probability)
```

```
do while list contains at least 2 elements
```

```
    construct tree using the first two elements in list
```

```
    add parent node for the union of these elements and compute probability
```

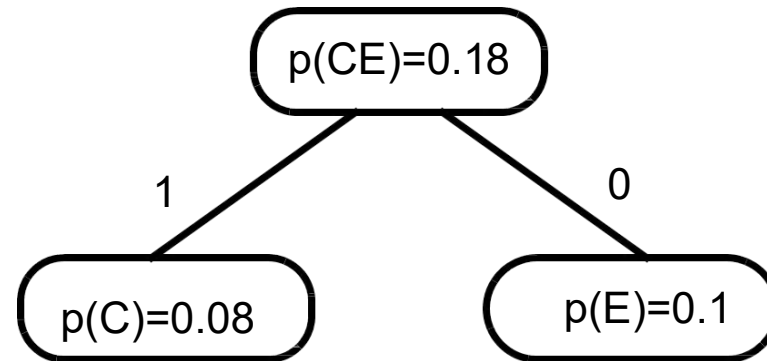
```
    mark edges by '0' and '1'
```

```
    delete the first 2 elements in list; insert parent into list
```

```
end
```

# Huffman Encoding—Example

step 1:  $p(C)=0.08$ ,  $p(E)=0.1$ ,  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(B)=0.51$





# Huffman Encoding—Example

step 2:  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(CE)=0.18$ ,  $p(B)=0.51$

build ordered list of symbols (increasing probability)

do while list contains at least 2 elements

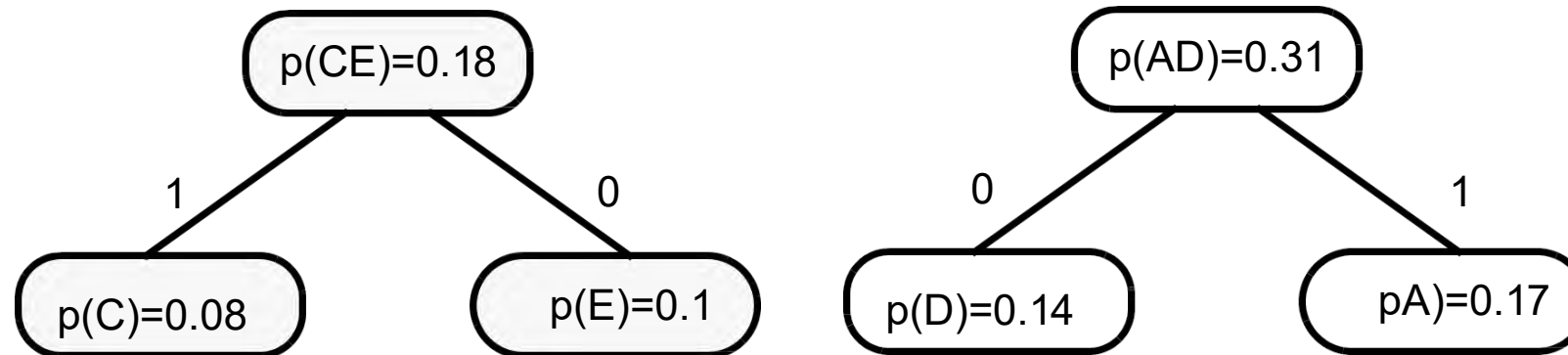
construct tree using the first two elements in list

add parent node for the union of these elements and compute probability

mark edges by '0' and '1'

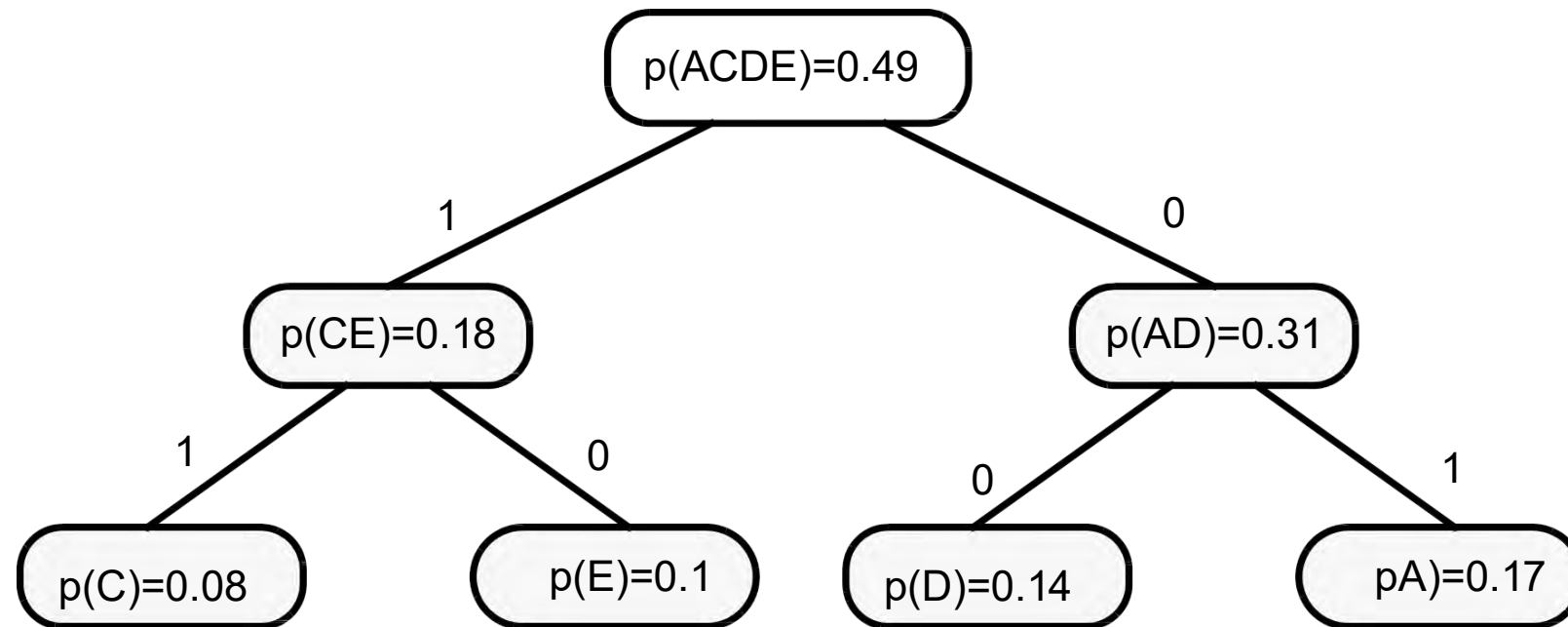
delete the first 2 elements in list;  
insert parent into list

end



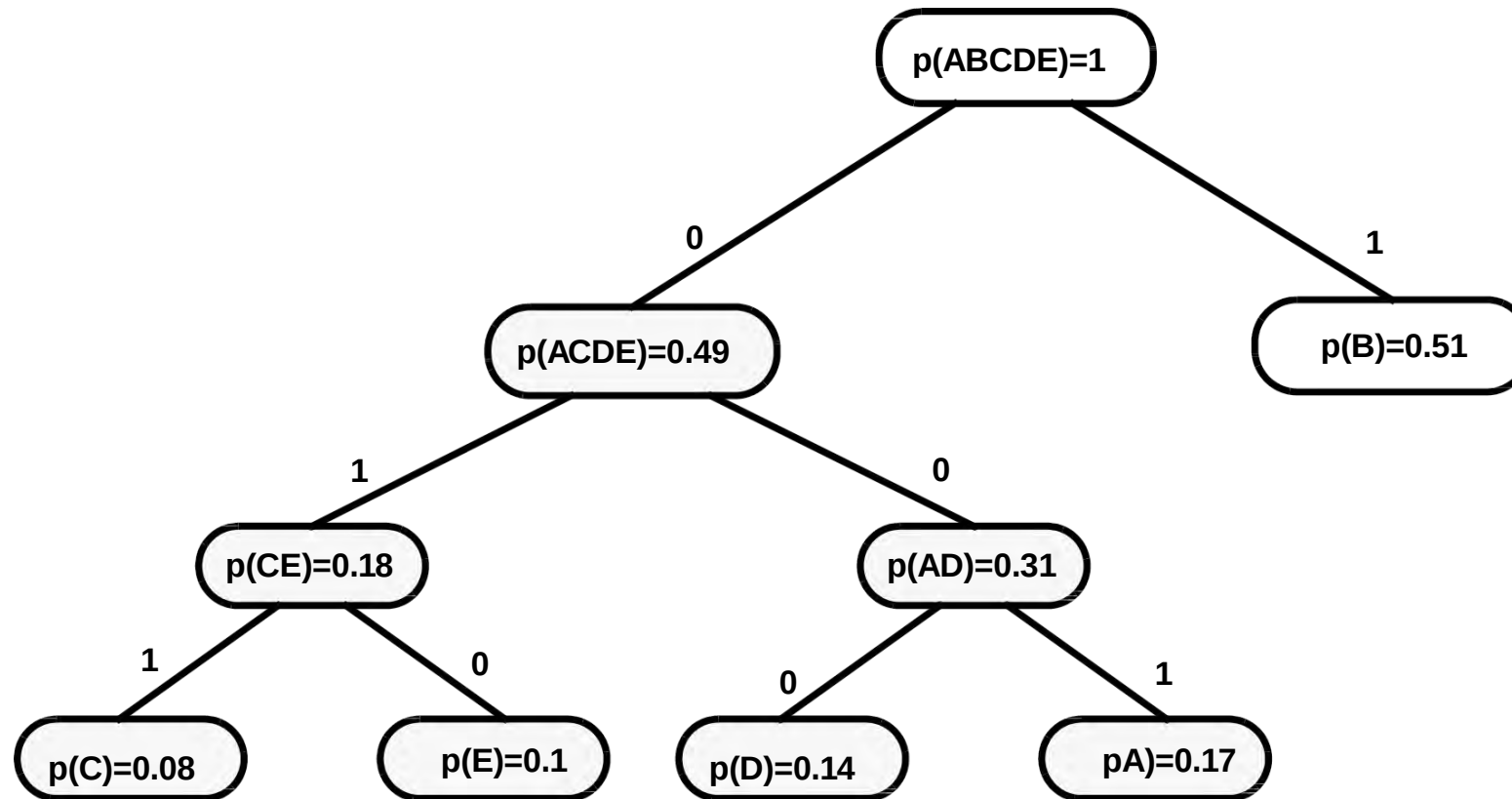
# Huffman Encoding—Example

step 3:  $p(CE)=0.18$ ,  $p(AD)=0.31$ ,  $p(B)=0.51$



# Huffman Encoding—Example

last step:  $p(ACDE)=0.49$ ,  $p(B)=0.51$



# Huffman Encoding—Example

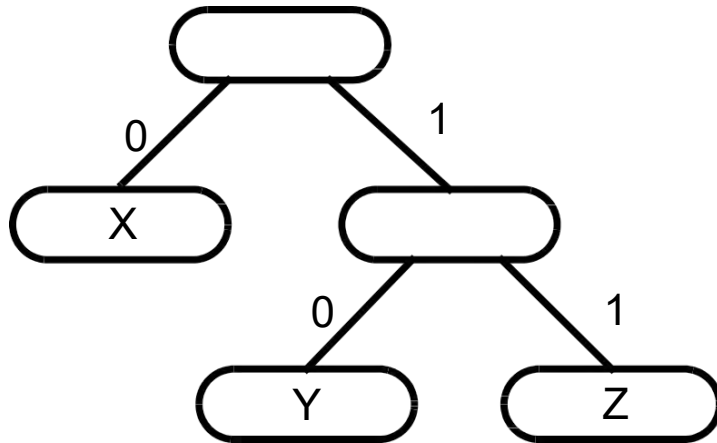
encode symbols according to tree

|   |     |
|---|-----|
| A | 001 |
| B | 1   |
| C | 011 |
| D | 000 |
| E | 010 |

$p(C)=0.08$ ,  $p(E)=0.1$ ,  $p(D)=0.14$ ,  $p(A)=0.17$ ,  $p(B)=0.51$

# Huffman Decoding—Example

given: 000101011 and Huffman Tree

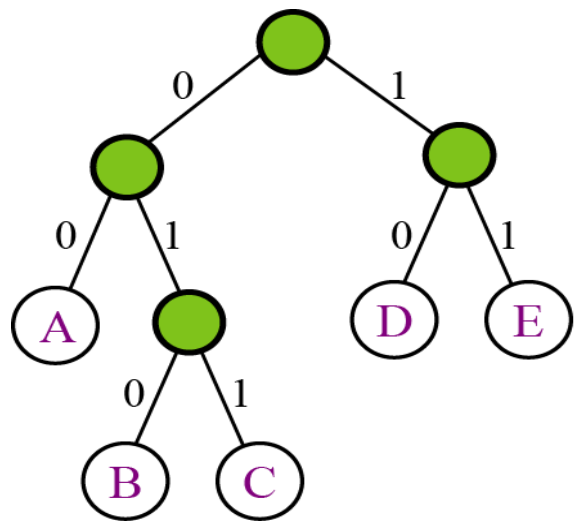


reconstruct message from bit stream using  
finite state machine

Solution: XXXYYZ

# Huffman Encoding

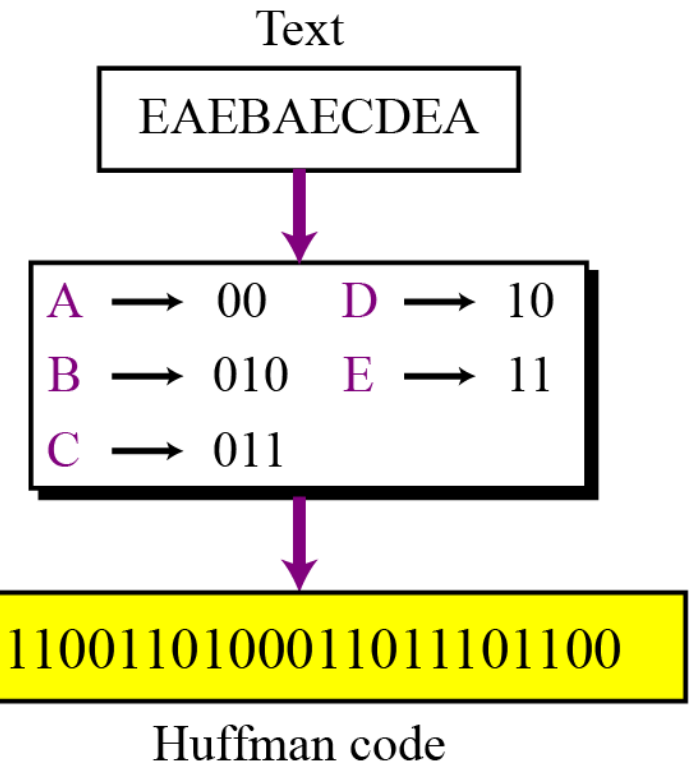
- Character code found by starting at the root and following the branches that lead to that character.
- The code itself is the bit value of each branch on the path, taken in sequence.



|        |       |
|--------|-------|
| A: 00  | D: 10 |
| B: 010 | E: 11 |
| C: 011 |       |

Code

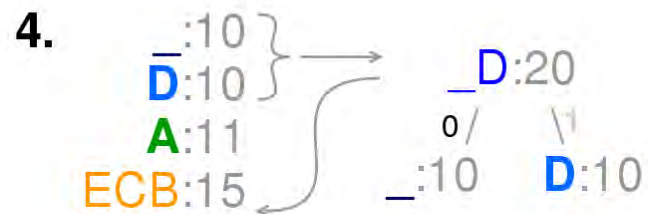
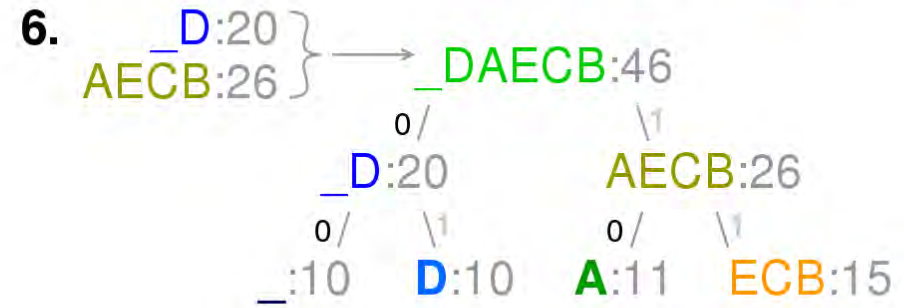
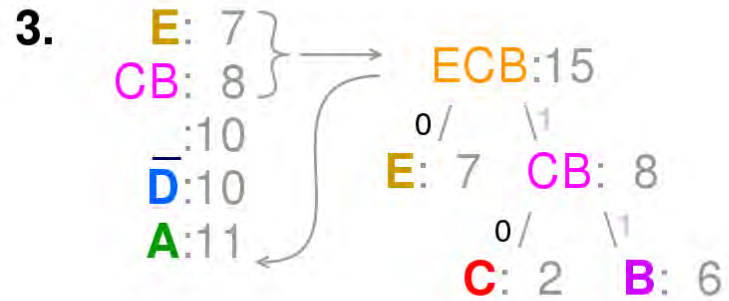
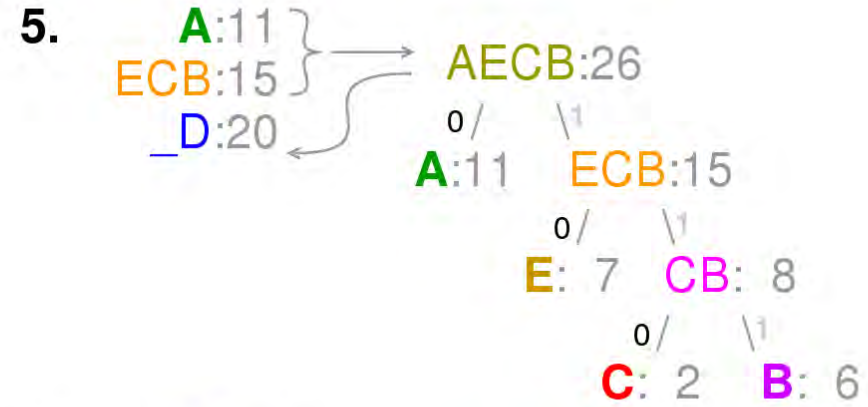
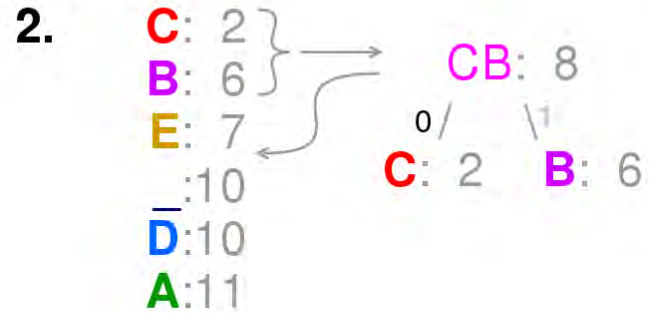
Encoder



- Decoding: reverse process



1. "A\_DEAD\_DAD\_CEDD\_A\_BAD\_BABE\_A\_BEADED\_ABACA\_BED"



8. "100001110100100011001001110110011100100100011111001001111101111110  
0010001111110100111001001011111011101000111111001"

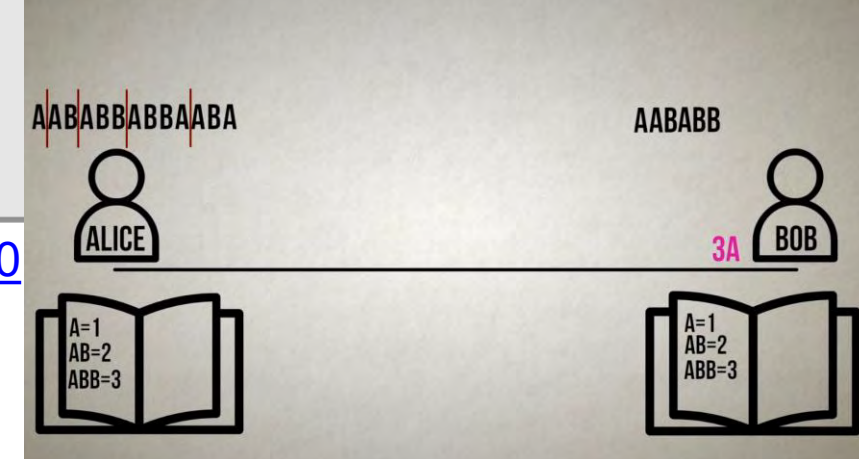
# Application of Huffman Coding for Video

- If the image information can be transformed into a bit stream, Huffman table can be used to compress data without loss
- Simple way to generate a bit stream is to code pixels individually and read them line by line
- For video: Huffman table can be used for a single sequence of images
  - for a set of scenes
  - for an entire film clip
- need Huffman table for encoding and decoding
- Is **used for MP3** encoding in the final step!

# Lempel Ziv (LZ) Dictionary-based Encoding

<https://www.youtube.com/watch?v=RV5aUr8sZD0>

- Dictionary is a table of strings
  - Sender and receiver have a copy of dictionary
  - Previously-encountered strings are substituted by their index in dictionary
- Compression - two concurrent events:
  - Building an indexed dictionary
  - Compressing a string of symbols.
- Algorithm extracts smallest substring not in the dictionary from remaining uncompressed string.
- Stores a copy of this substring in dictionary as a new entry and assigns it an index value.
- Compression occurs when substring (except for the last character) is replaced with the index found in the dictionary.
- Process inserts the index and the last character of the substring into the compressed string.



Used in GIF & ZIP

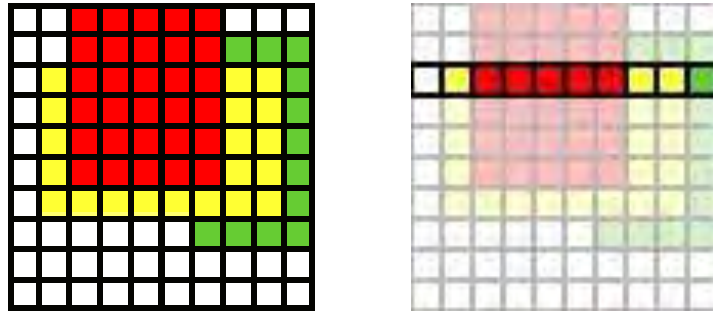
Abraham Lempel



# Lossless Image Formats



- GIF-Format (Graphics Interchange Format)
  - LZW-Compression (Lempel, Ziv, Welch): works line-wise



- 1 st line, 2nd line
- 3rd line is compressed as:
  - 1 white, 1 yellow, 5 red, 2 yellow, 1 green
- Row 4 to 6: „as row 3“
- Indexed (1-8 bit Color Lookup Table)



# CompuServ GIF (gif)



- First standardized in 1987 by CompuServ (called GIF87a)
- Updated in 1989 to include transparency, interlacing, and animation (called GIF89a)
- Use the LZW (Lempel-Ziv Welch) algorithm for compression (not free, licenses necessary)
- A maximum of **256 colors** freely selectable out of 24 bit ( $2^{24} = 16.777.216$ )
- Does not work for photographs
- Suitable for small images such as icons
- Simple animations





# Portable Network Graphics (png)



- Developed because of licensed LZW with gif
- Replacing GIF and TIFF (not JPEG!)
- 3 Types
  - True Color (3 x 16 bit/pixel)
  - Grayscale (1 x 8 bit/pixel)
  - Palette (256 colors)
- Compression similar to PKZIP (Phil Katz)
- The standard zip format provides approximately **62 percent** compression. (RAR results in a **71 percent** compression rate; 7-zip has **76 percent**)





# Differential Encoding

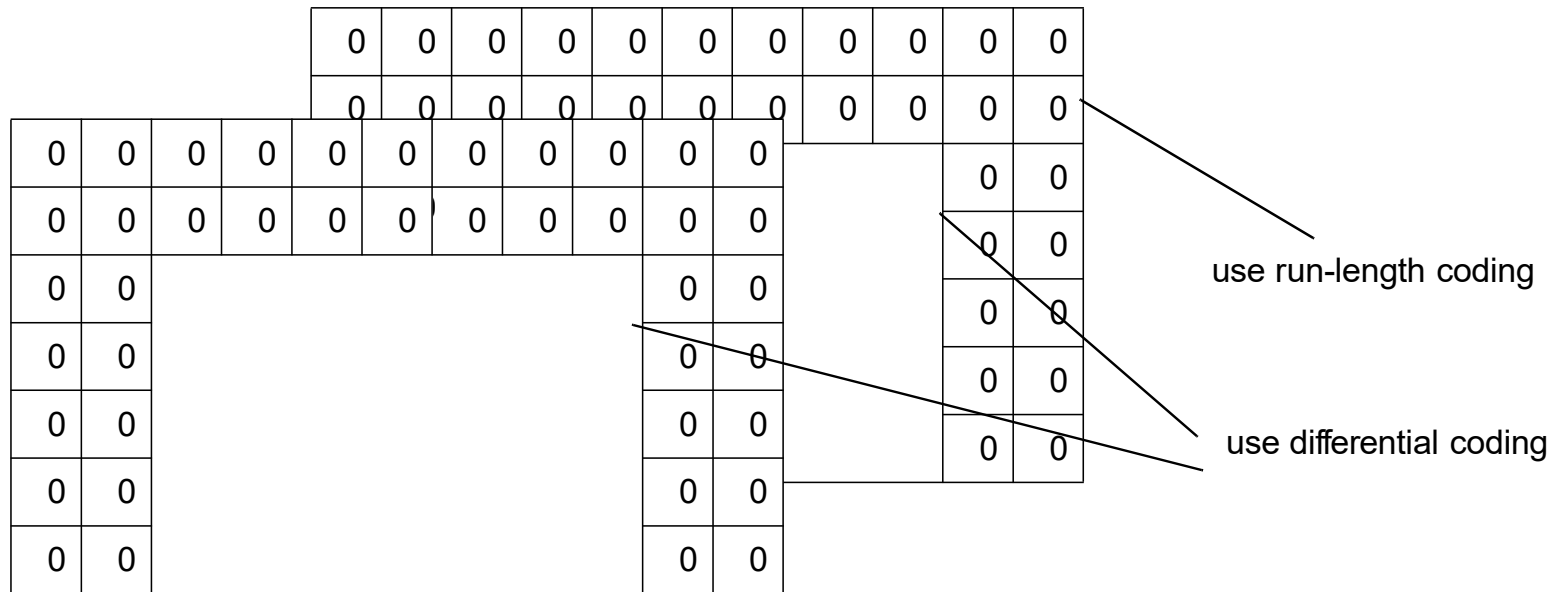
- belongs to **source coding**
- consider sequence of symbols  $S_1, S_2, S_3, \dots$  where values are not zeros, but do not vary much  $\rightarrow$  calculate difference from previous value
- Ex.: still images—calculation of differences between nearby pixels or pixel groups

|   |     |    |    |     |
|---|-----|----|----|-----|
| 0 | 0   | 0  | 0  | 0   |
| 0 | 255 | 94 | 87 | 100 |
| 0 | 0   | 0  | 0  | 255 |
| 0 | 0   | 0  | 0  | 0   |

- edges result in large values
  - similar chrominance or luminance values result in small values
- zeros can be suppressed by run-length coding

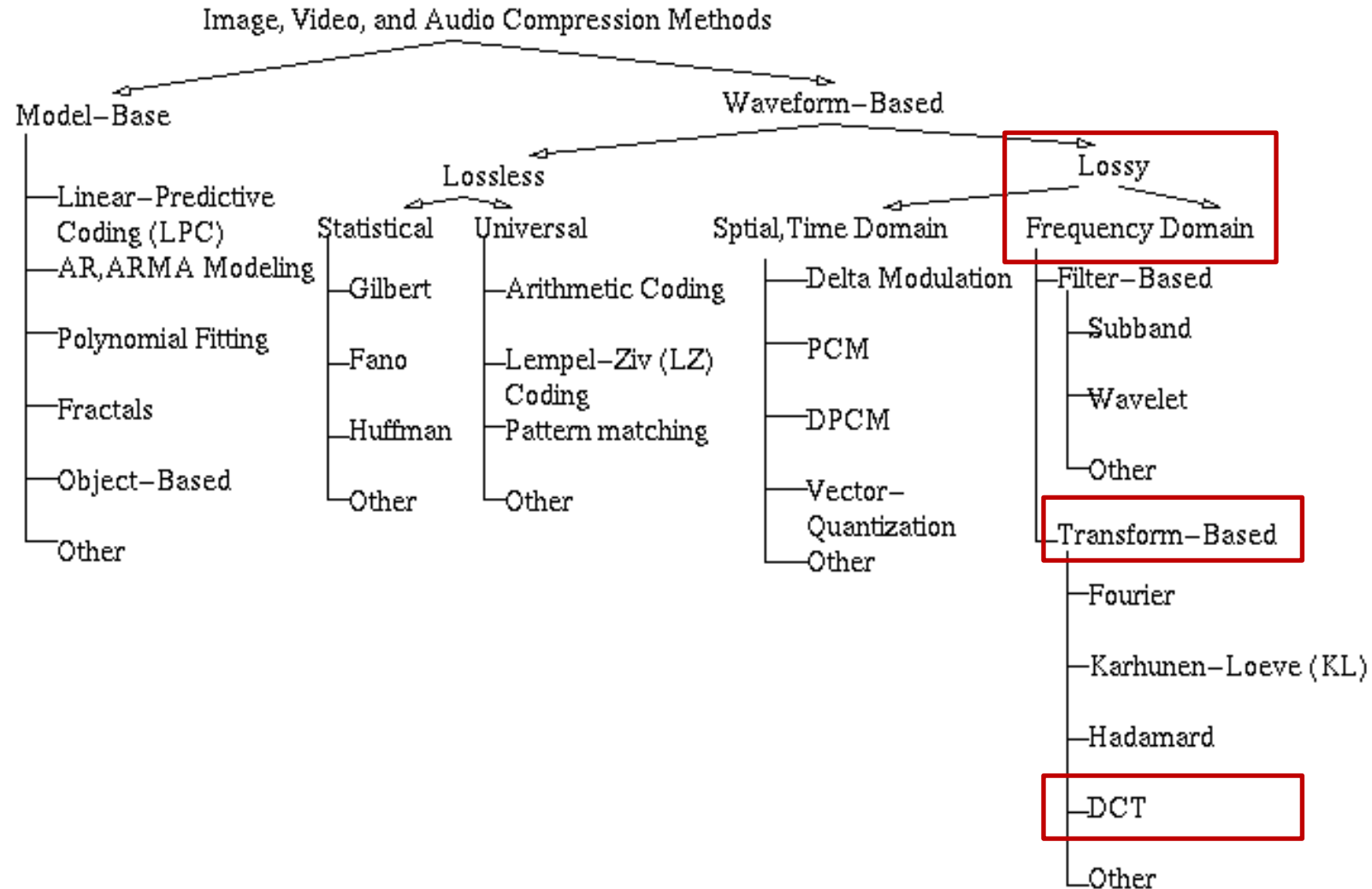
# Differential Coding in Video (Examples)

- static background (videoconferencing, newscast, etc.)

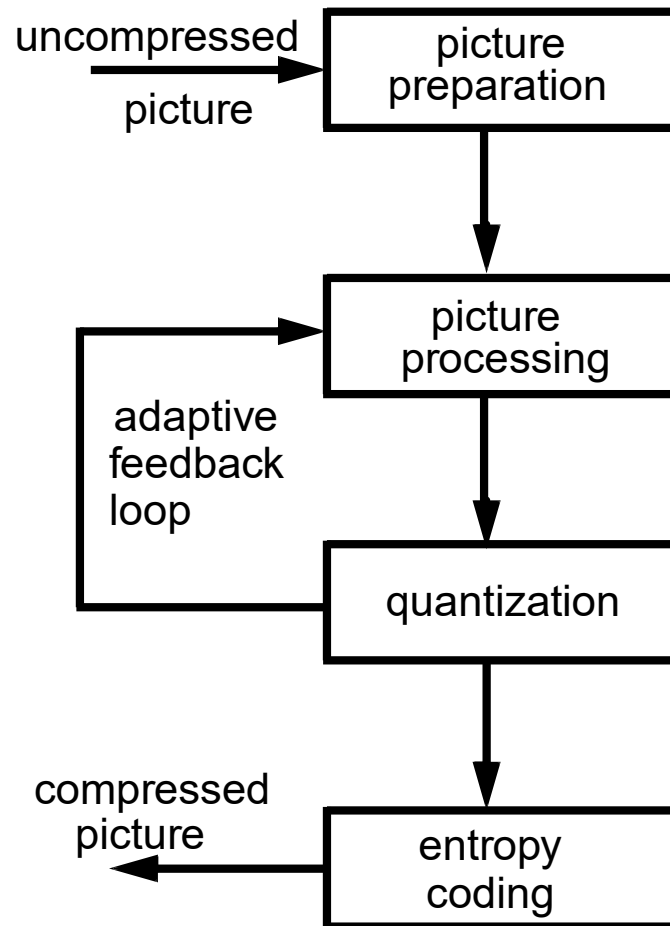


- motion compensation—8x8 blocks are compared; areas similar, only shifted, e.g., to the right (*motion vector*)

# Lossy Image Compression



# Image Compression Steps



Possible operations:

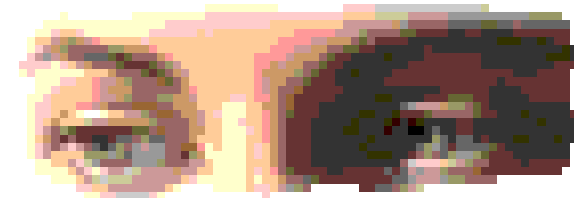
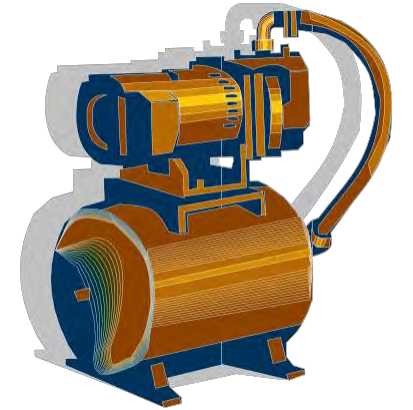
- analog-to-digital conversion
- generation of appropriate digital representation
- image division into 8 x 8 blocks
- fix number of bits/pixel

Compression Algorithm

- transformation from time to frequency domain e.g., using DCT
- motion vector application for motion video
- Typically a lossy step
- mapping of real numbers to integers (reduction in precision)  
example:  $\mu$ -law audio—from 12 bits to 8 bit quantization
- compress a sequential digital stream without loss

# Lossy Compression

- Run-length coding works well for simple images (graphics) and computer data
- However, a large, detailed picture usually is not reduced enough
- In order to reduce information further lossy compression is needed
  - Information is permanently removed
  - The trick is to remove details that are not perceived by a human observer
  - Many of these psycho-visual coding systems take advantage of a number of aspects of the human visual system



# Human Visual System



- The eye is more sensitive to brightness changes than to color changes
- The eye is not able to perceive brightness above or below certain threshold values
- The eye does not perceive little brightness or color changes. The strength of this phenomenon is dependent on the color





# Compression of Still Images



- Characteristics of human vision have been transferred to **lossy image compression techniques**
  - remove some image details to achieve a higher compression rate by suppressing higher frequencies
- Such a development is the JPEG format
  - Stands for "Joint Photographic Experts Group"
  - JPEG is the worldwide standard
  - JPEG compresses brightness and color information separately
  - Color information is more compressed (lower sensitivity)
- Color space for JPEG compression
  - RGB values are a combination of brightness and color
  - If the RGB values are separated into a luminance and a color component, the color component is more compressed



# Joint Photographic Experts Group (JPEG)

JPEG is a joint standard of ISO and ITU-T

- In June 1987, an adaptive transformation coding technique based on DCT was adopted for JPEG
- In 1992, JPEG became a ISO international standard
- Lossy compression combined with lossless techniques
- Trade-Off between file size and quality
- JPEG typically achieves 10:1 compression with little perceptible loss in image quality

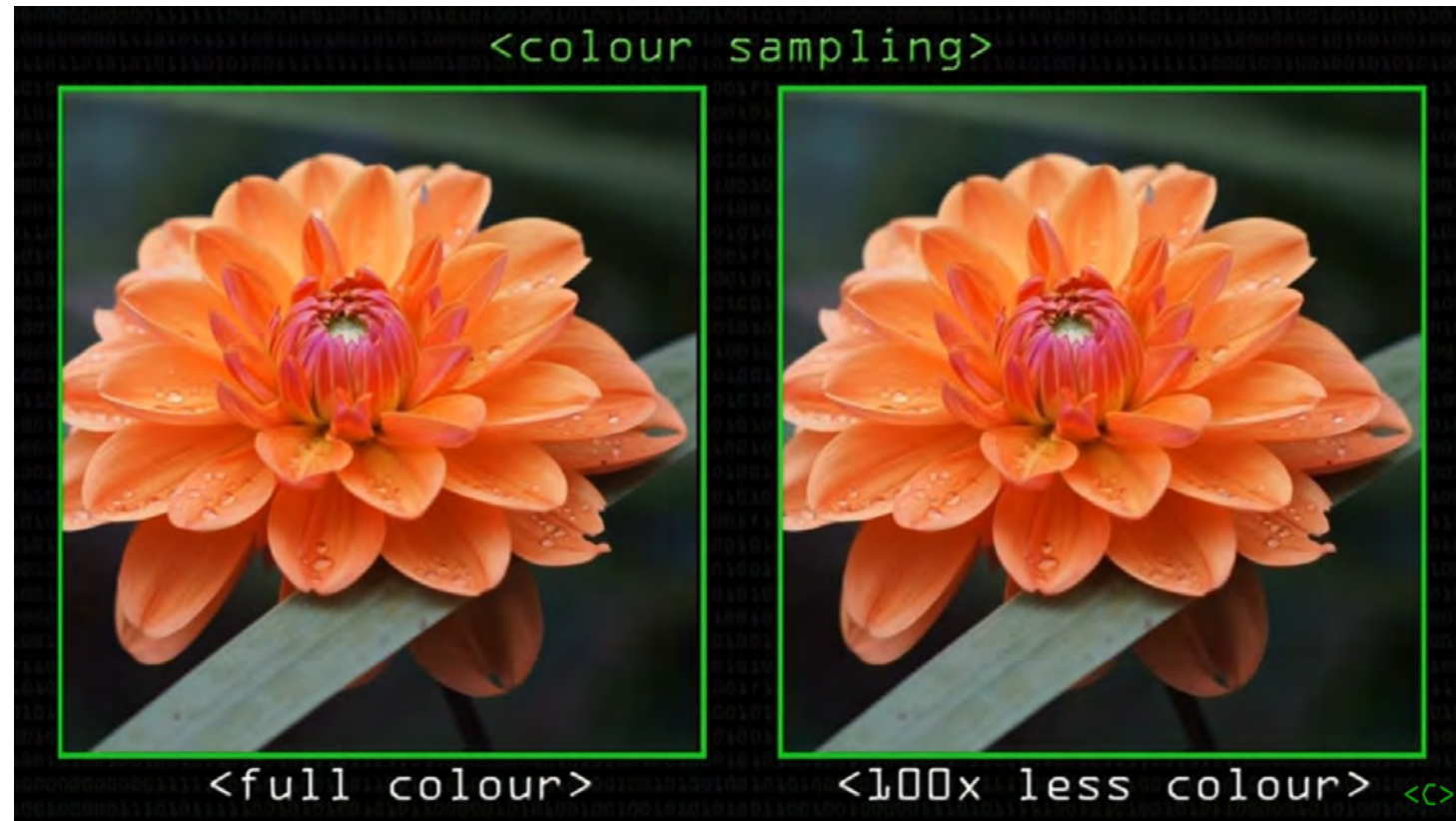
Requirements:

- to be state-of-the-art for degree of compression versus image quality,
- to be parameterizable (user can select parameters),
- to be applicable to any kind of source image, without regard to dimensions, content, image aspect ratio, pixel aspect ratio, etc.,
- to have computational requirements that are reasonable for hardware and software implementations,
- to run on as many standard platforms as possible

# Joint Photographic Experts Group (JPEG)

- to support four different modes of operation:
  - **sequential encoding—encoded in same order as scanned**
  - progressive encoding—multiple pass encoding
  - lossless encoding
  - hierarchical encoding—encoded at multiple resolutions

# How JPEG Works – Part 1



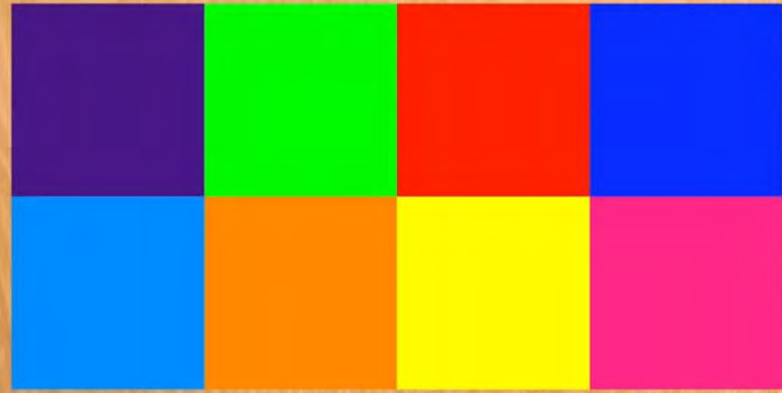
# 1<sup>st</sup> Step: Color Conversion YCbCr <-> RGB

- Necessary in preparation for JPEG compression
  - Color space conversion and Downsampling:
    - Separation of the color components from the luminance information
    - Insensitivity of human eye to rapid color changes allows coarser sampling of the color components
- > No loss of subjective quality
- > Significant data reduction

# Chroma Subsampling

**4:4:4**

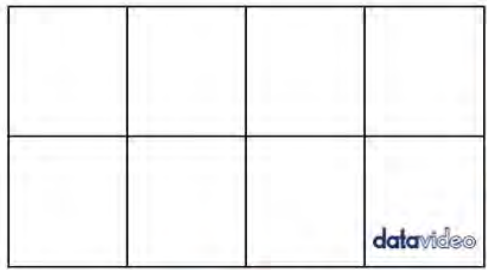
**Chroma**



Some visual examples  
<https://www.red.com/red-101/video-chroma-subsampling>

<https://www.youtube.com/watch?v=32PPzwPjDZ8>

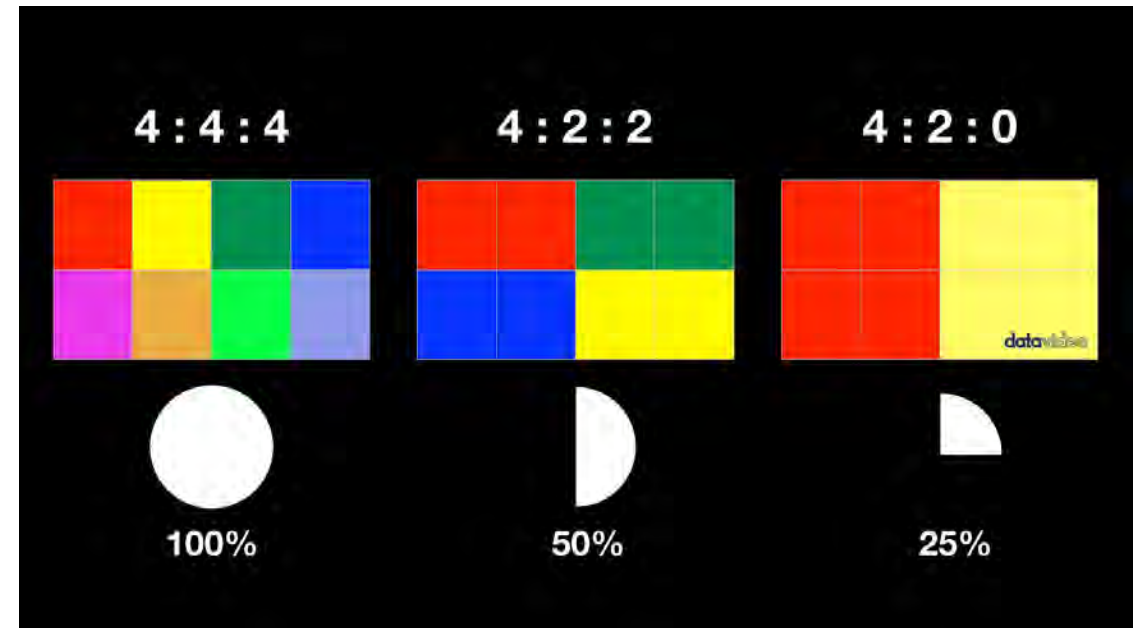
| Y                                            | Cr                                | Cb                                 |
|----------------------------------------------|-----------------------------------|------------------------------------|
| The Numbers of Across<br>Pixels <b>4</b> x 2 | First Row<br>The Numbers of Color | Second Row<br>The Numbers of Color |



Cr

Cb

**Y** **4** x 2





# 1<sup>st</sup> Step: Color Conversion YCbCr <-> RGB

- YCbCr (256 levels) can be computed directly from 8-bit RGB as follows:

$$\begin{bmatrix} Y' \\ Cb \\ Cr \end{bmatrix} = \begin{bmatrix} 0.299 & 0.587 & 0.114 \\ -0.1687 & -0.3313 & 0.5 \\ 0.5 & -0.4187 & -0.0813 \end{bmatrix} \begin{bmatrix} R \\ G \\ B \end{bmatrix} + \begin{bmatrix} 0 \\ 128 \\ 128 \end{bmatrix}$$

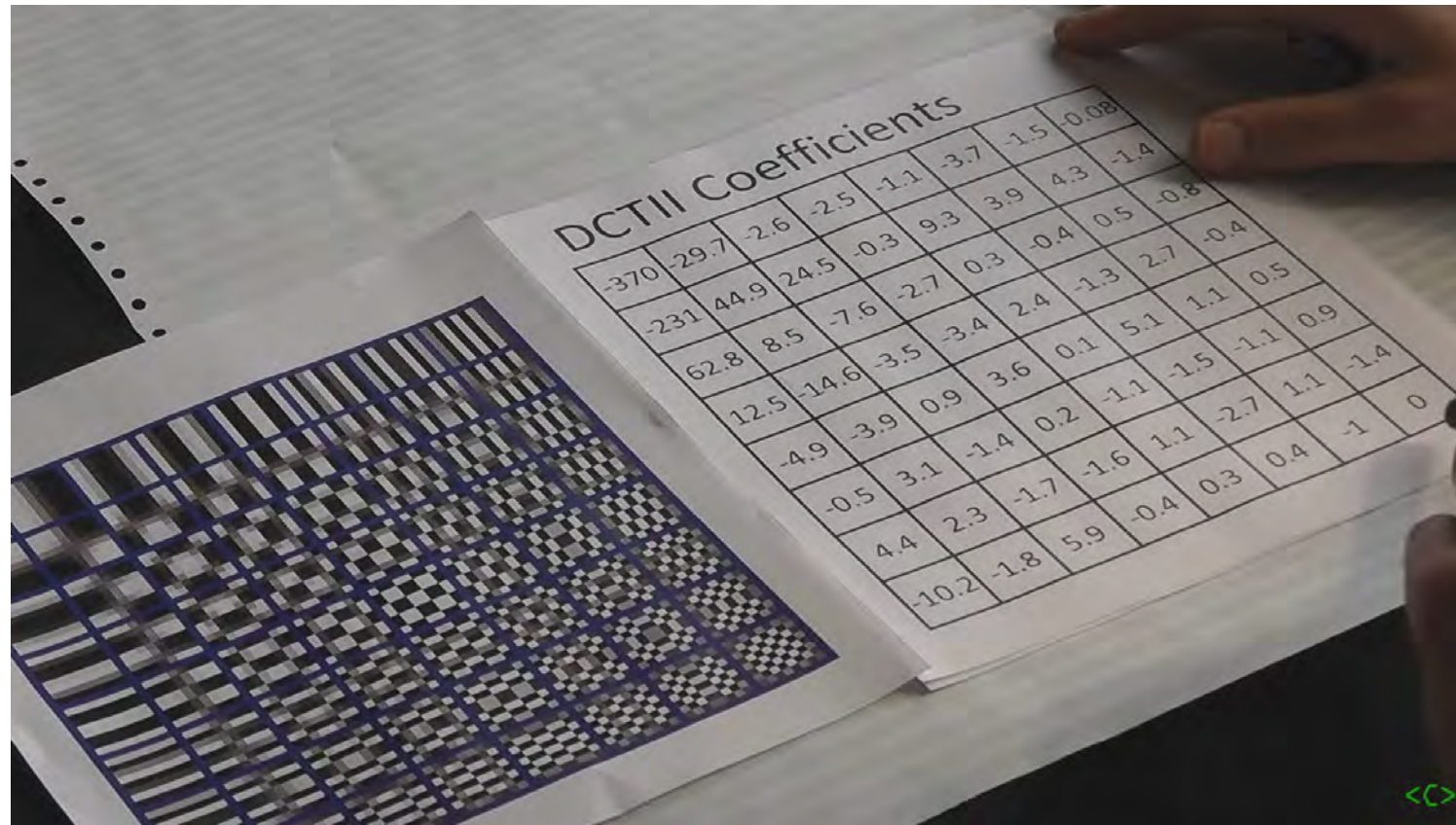
- During decoding YCbCr color space needs to be converted back to RGB:

$$R = Y + 1.402 (Cr-128)$$

$$G = Y - 0.34414 (Cb-128) - 0.71414 (Cr-128)$$

$$B = Y + 1.772 (Cb-128)$$

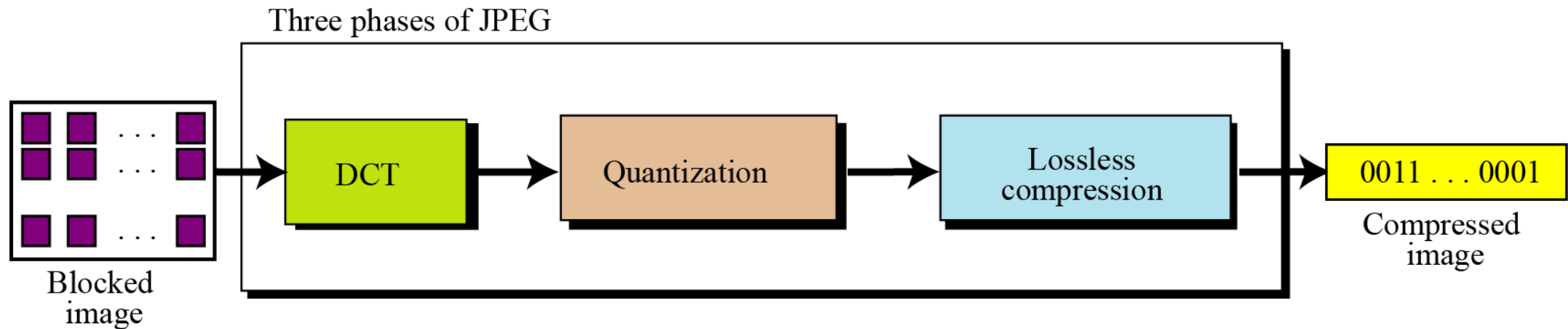
# How JPEG Works – Part 2



# JPEG Compression - Overview

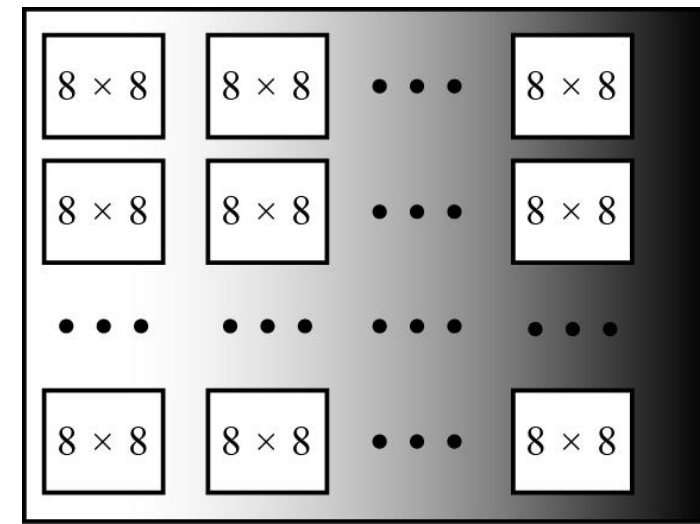


- Idea: change image into a linear (vector) set of numbers that reveals redundancies.
- Redundancies can be removed using one of the lossless compression methods



# JPEG Image Preparation

- Images – each channel – divided in data units (blocks), DCT operates on blocks to decrease the number of calculations
- Lossy mode operates on 8x8 pixel blocks (4:4:4 – no subsampling), 16×8 (4:2:2), or most commonly 16×16 (4:2:0).
- Lossless mode operates on data units of 1 pixel
- In most cases data units are processed component by component and passed to image processing
- processing order of data units
  - left-to-right, top-to-bottom
  - interleaved data ordering
- Interleaved data units of different color channels are combined to
  - **Minimum Coded Units (MCU)**



# Discrete Cosine Transform

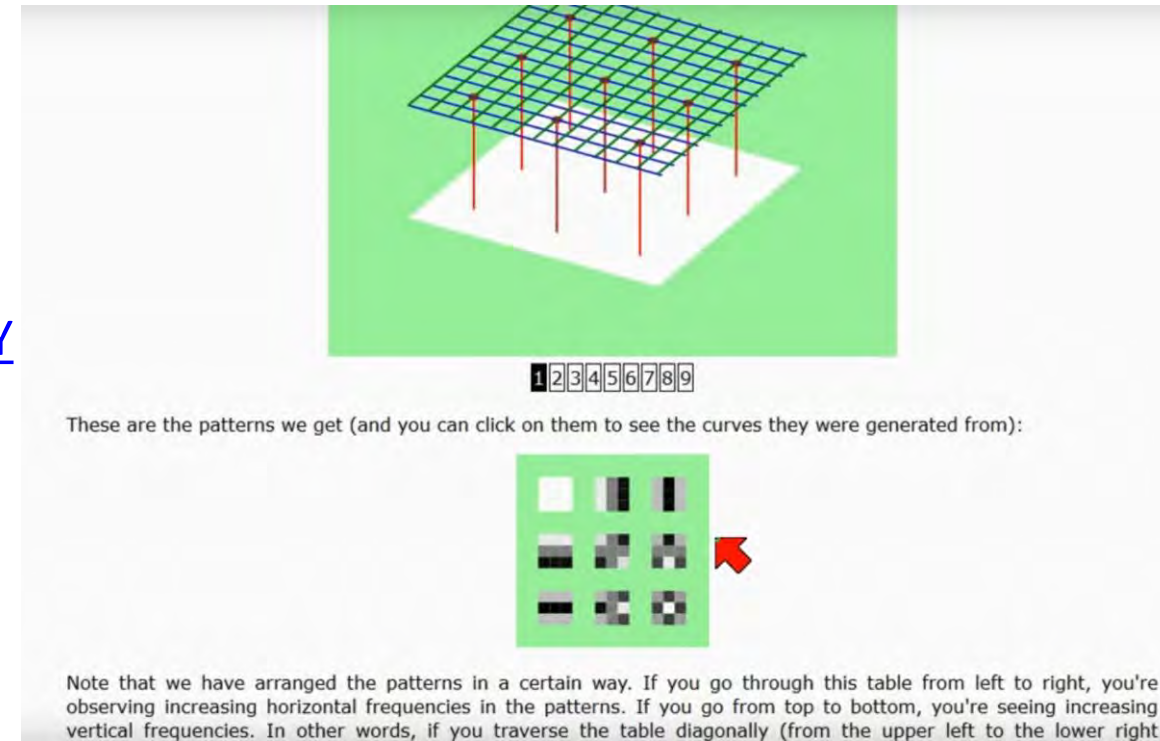
- First step: pixel values shifted into the range  $[-128, 127]$
- Forward Discrete Cosine Transformation (Forward DCT) maps values from time to frequency domain

- Interactive Website

<http://weitz.de/dct/>

Explanation:

<https://www.youtube.com/watch?v=7fhHQgu2OcY>





- DCT-based codecs use a two-dimensional version of the transform.
- The 2-D DCT of an 8 x 8 block:

$$F(u, v) = \sum_{x=0}^7 \sum_{y=0}^7 \alpha(u) \alpha(v) f(x, y) \cos\left[\frac{\pi}{8} \left(x + \frac{1}{2}\right) u\right] \cos\left[\frac{\pi}{8} \left(y + \frac{1}{2}\right) v\right]$$

$u$  is the horizontal spatial frequency, for the integers  $0 \leq u < 8$

$v$  is the vertical spatial frequency, for the integers  $0 \leq v < 8$

$$\alpha(u) = \begin{cases} \sqrt{\frac{1}{8}}, & \text{if } u = 0 \\ \sqrt{\frac{2}{8}}, & \text{otherwise} \end{cases} \quad \text{is a normalizing scale factor to make the transformation orthonormal}$$

$f(x, y)$  is the pixel value at coordinates  $(x, y)$  “space domain”

$F(u, v)$  is the DCT coefficient at coordinates  $(u, v)$  “frequency domain” (indicates how fast the information moves inside the rectangle)

◆ Note: The DCT decomposes a signal into a series of harmonic cosine functions.



# DCT Basis Functions

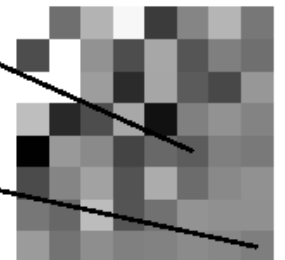
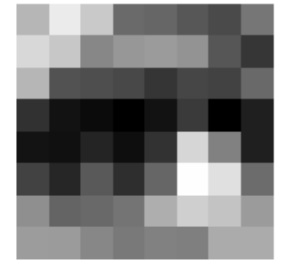
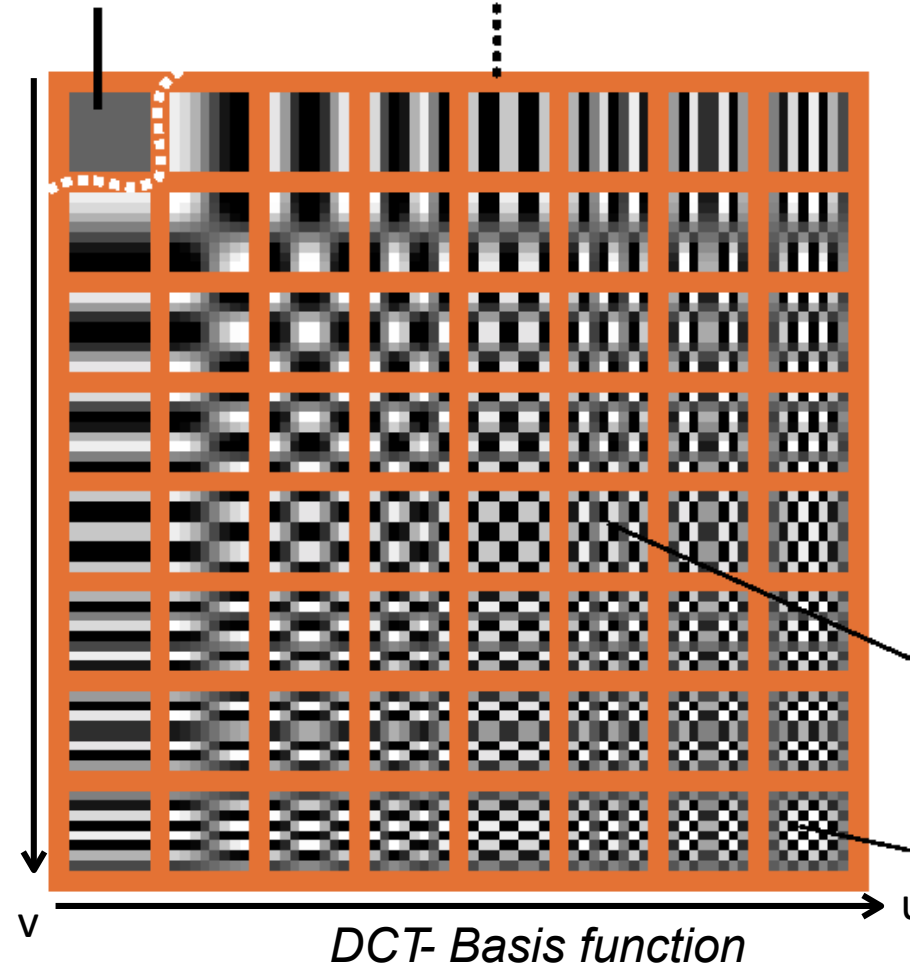


- DCT values indicate the weighting of frequency images
- The brighter the pixel, the greater the value of DCT

*Constant component*

*Alternating component*

*8 x 8 Imageblock*



# JPEG Image Processing (DCT)

- $S(u,v)$  coefficients
- $S(0,0)$ —lowest frequency in both directions is called DC coefficient
  - determines the fundamental luminance/color of the block
  - frequency = 0 in both directions
- $S(0,1), \dots, S(7,7)$  are called AC coefficients
  - frequency in one or both directions non-zero
- computing of DCT—use factoring

# Discrete Cosine Transform

Humans have only a restricted capability to perceive high frequencies.  
Images reduced from their high frequent elements may have a high degree of quality while being compressed quite considerably

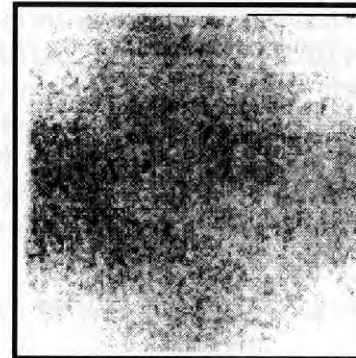
- Low frequency components of an image correspond to brief outlines of image objects
- High frequency components of an image correspond to fine structures

To decrease the high frequencies the data must be sorted by frequencies  
->DCT

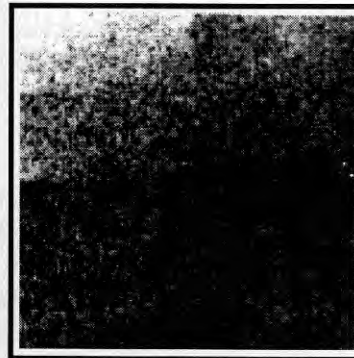
# Why DCT?



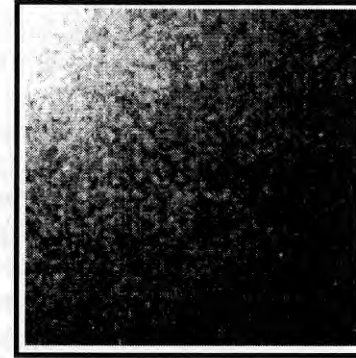
(a) Test Image



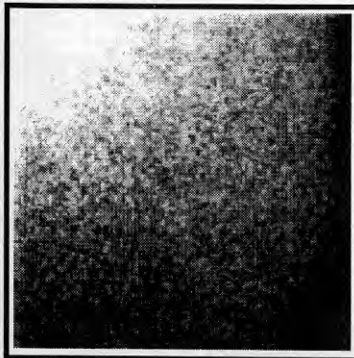
(b) Discrete Fourier transform



(c) Discrete Cosine transform



(d) Discrete Sine transform



(e) Discrete Hadamard transform



(f) Karhunen-Loeve transform

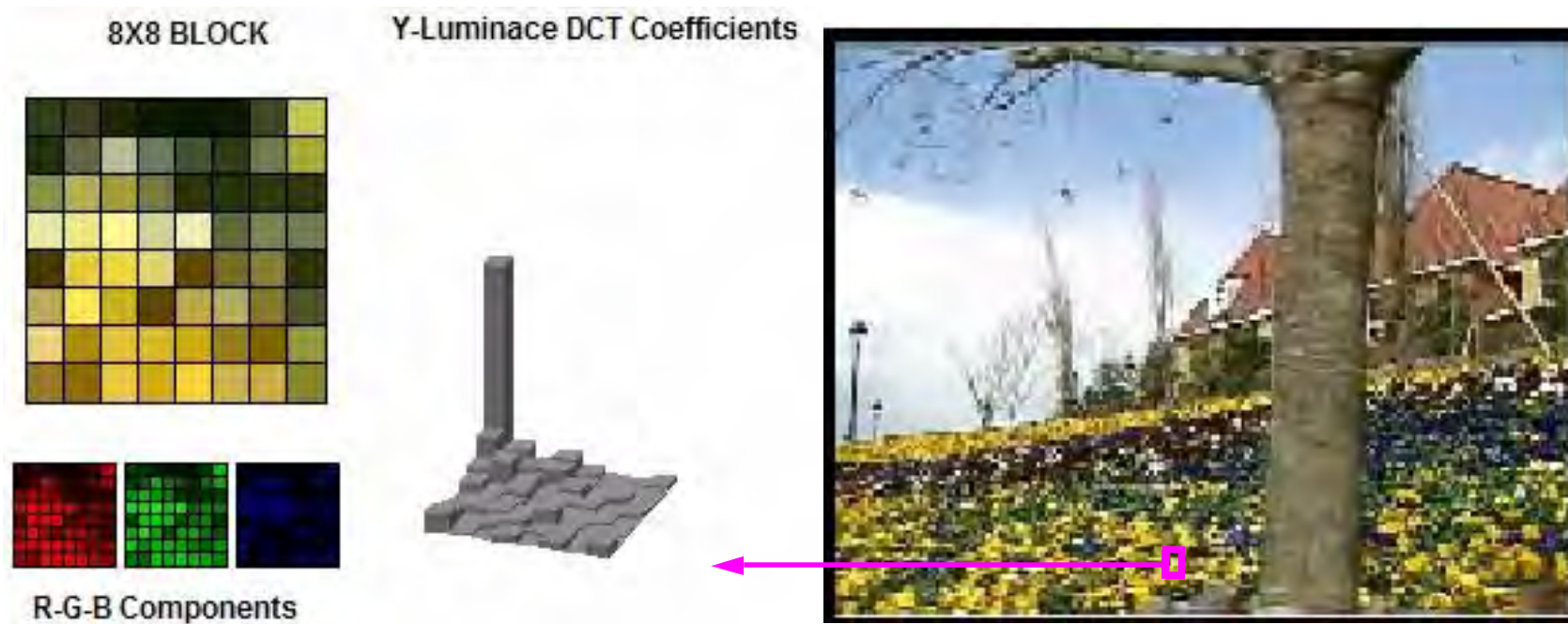
Karhunen-Loeve not used in practice for image compression because it needs to transmit the transform kernel for every block, which requires huge amount of side information.

# DCT-Coefficients





# DCT-Coefficients

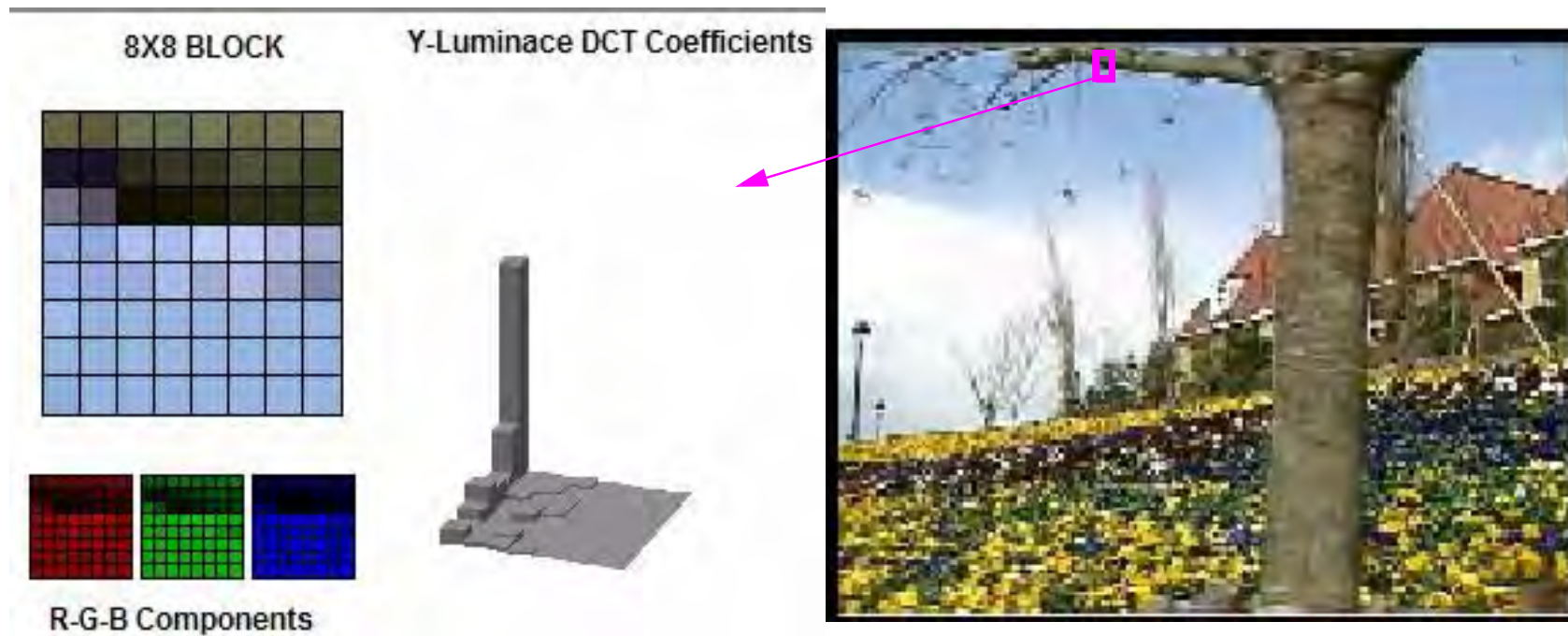




# DCT-Coefficients



# DCT-Coefficients

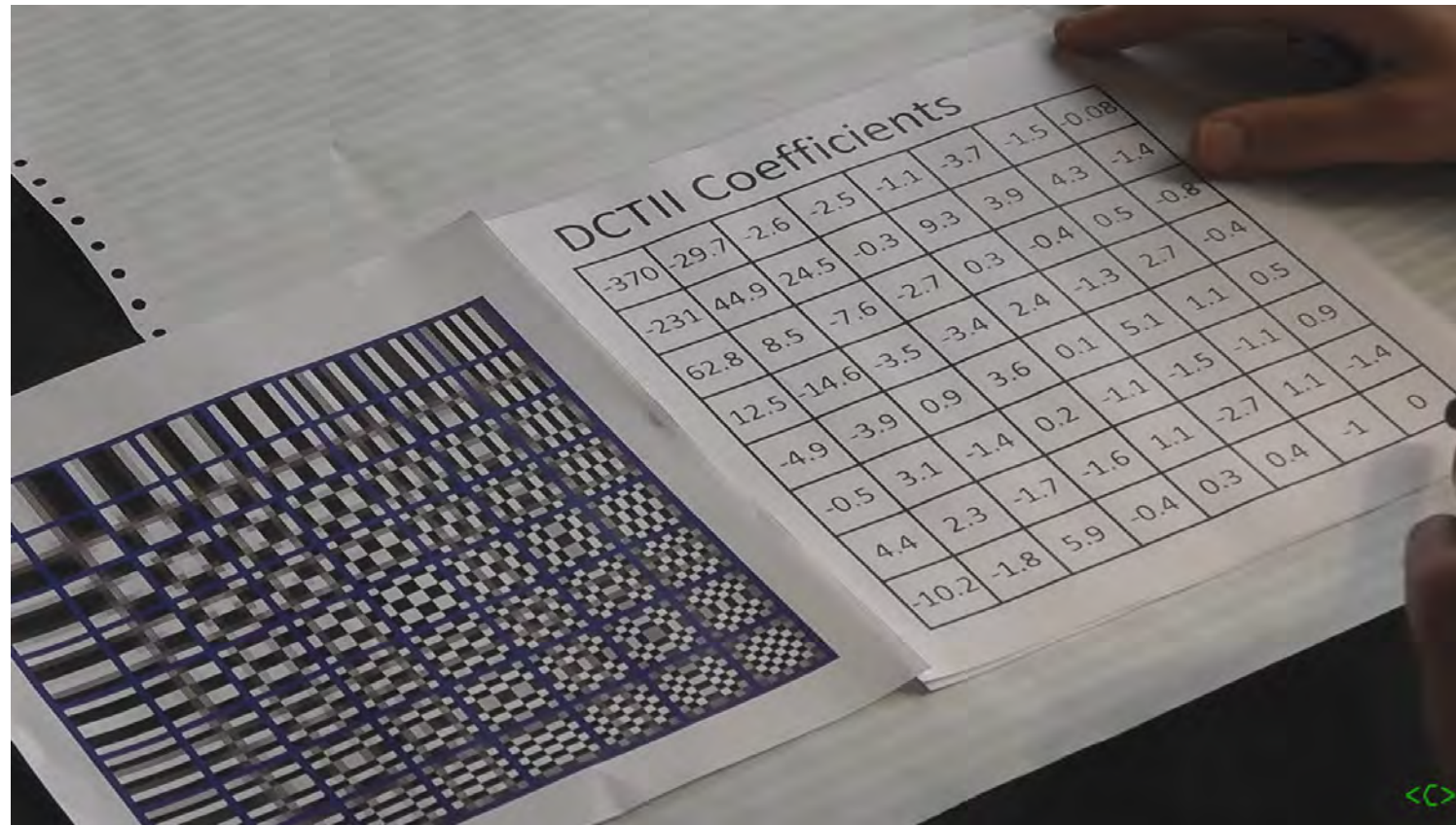


# DCT-Coefficients





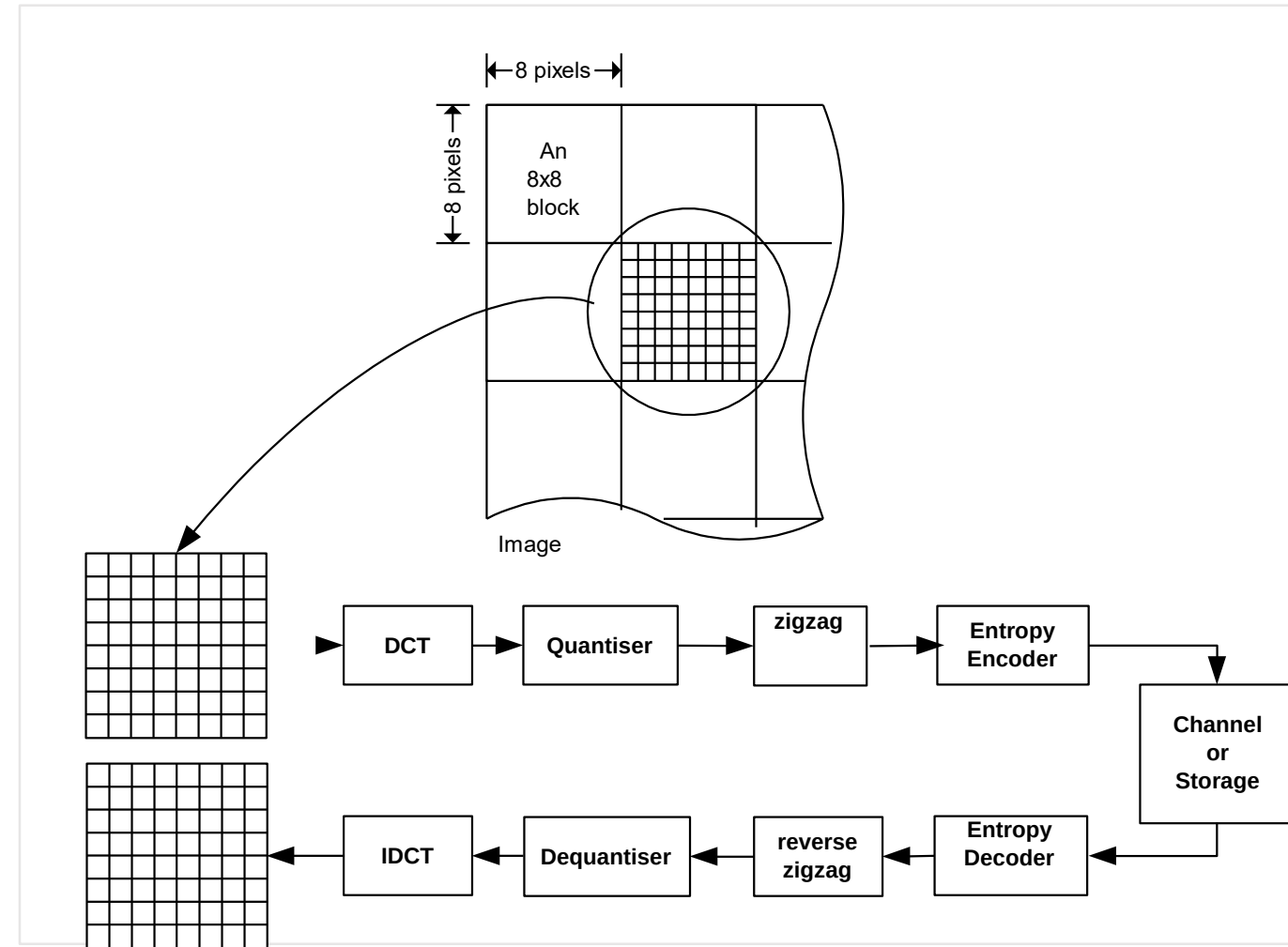
# How JPEG Works – Quantization - Part 3



# JPEG Compression



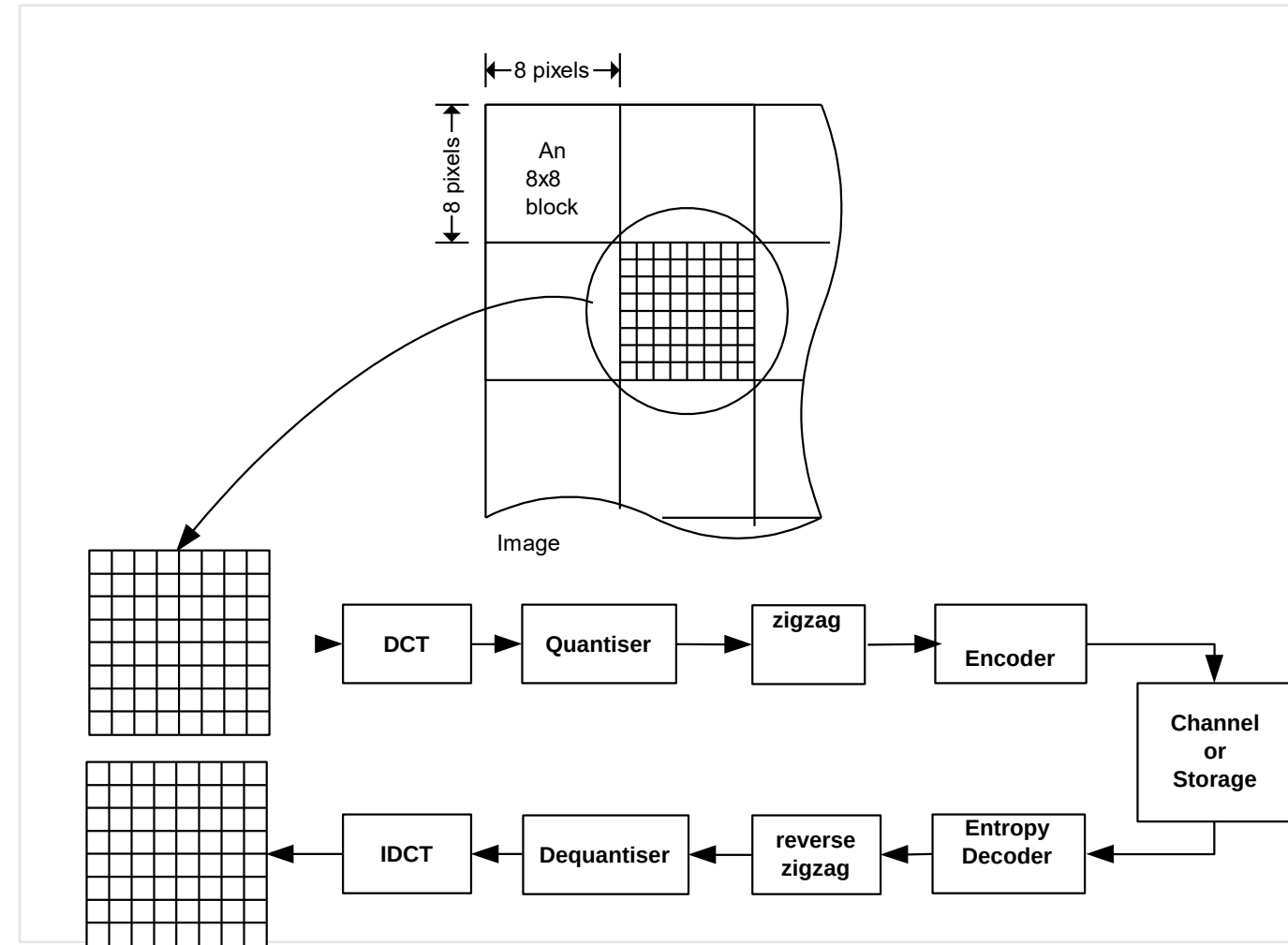
- To perform the JPEG coding, an image (in color or grey scales) is first subdivided into blocks of 8x8 pixels.
- The Discrete Cosine Transform (DCT) is then performed on each
- This generates 64 coefficients which are then quantized to reduce their magnitude.



# JPEG Compression



- The coefficients are then reordered into a one-dimensional array in a zigzag manner before further entropy encoding.
- The **compression is achieved in two stages**; the first is **during quantization** and the second **during the entropy coding** process.
- JPEG decoding is the reverse process of coding.





# JPEG Quantization

- Goal: to “throw out” bits (truncation)
- Quantization in JPEG = Process of removing high frequency data
- Uniform quantization: divide coefficient values  $S(u,v)$  by  $N$  and round result

Q: In  $S(u,v)$  how many bits should be truncated?

A: use quantization tables

- quantization tables consist of 64 elements, each value uses 8-bit:  $Q_{uv}$
- new compressed values by using tables  $Sq_{uv} = S_{uv} / Q_{uv}$
- standard defines two default tables (luminance, chroma)

# JPEG Quantization

## Luminance quantization table

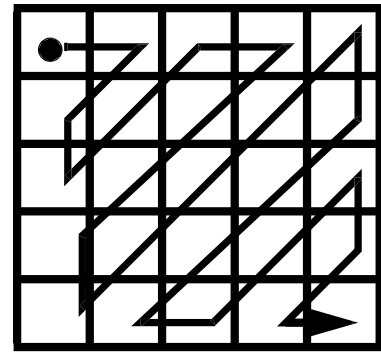
|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 16  | 11  | 12  | 14  | 12  | 10  | 16  | 14  |
| 13  | 14  | 18  | 17  | 16  | 19  | 24  | 40  |
| 26  | 24  | 22  | 22  | 24  | 49  | 35  | 37  |
| 29  | 40  | 58  | 51  | 61  | 60  | 57  | 51  |
| 56  | 55  | 64  | 72  | 92  | 78  | 64  | 68  |
| 87  | 69  | 55  | 56  | 80  | 109 | 81  | 87  |
| 95  | 98  | 103 | 104 | 103 | 62  | 77  | 113 |
| 121 | 112 | 100 | 120 | 92  | 101 | 103 | 99  |

Human eye most sensitive to low frequencies (upper left corner)

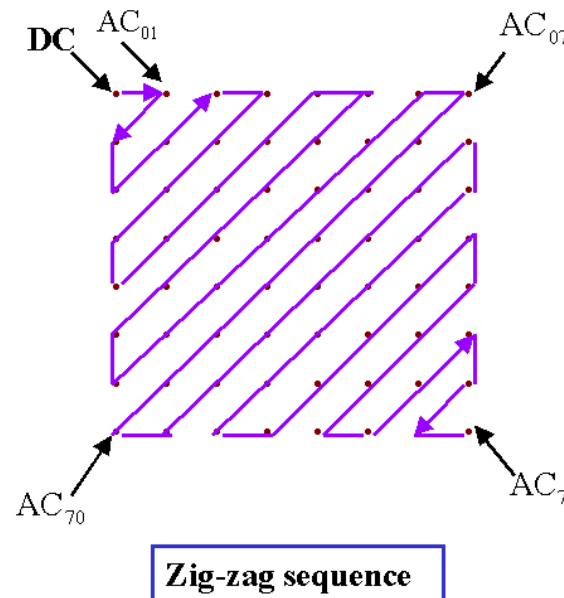
# Quantization process

- Divide DCT-coefficient value by an integer number from the quantization table
- Round the result to the nearest integer
- Quantization of all DCT-coefficients results in a *lossy* transformation – some image details given by higher frequencies are cut off.
- JPEG application provides a table with 64 entries, each used for quantization of one DCT-coefficient
  - each coefficient can be adjusted separately
- A high compression factor is achievable on the expense of image quality
  - large quantization numbers: high data reduction but information loss increases
- No default values for quantization tables are specified in JPEG

# JPEG Entropy Encoding



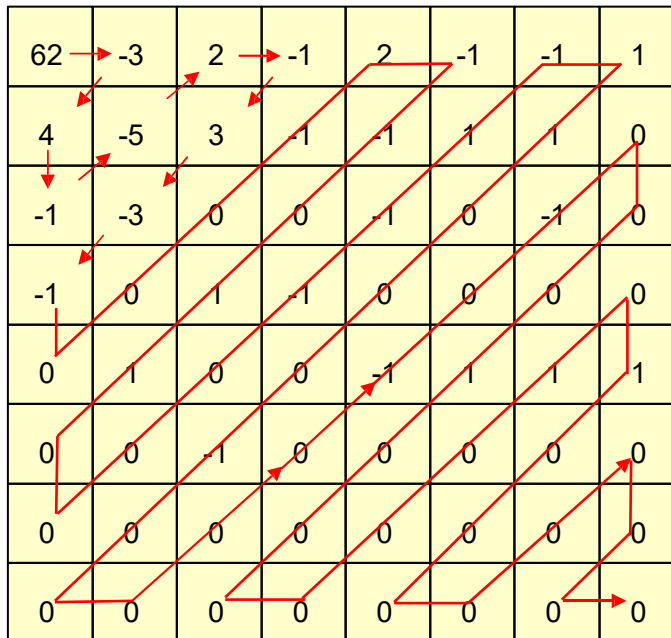
First step: map 8x8 pixel blocks into 64 element vector using zig-zag-scan



- DC coefficients processing:
  - DC coefficients determine basic color of data unit
  - DC coefficient is large, but often close to previous value  
==> encode difference

# Example

## Zig-zag ordering



*DC-coefficient*: code coefficients for one block as difference to the previous one

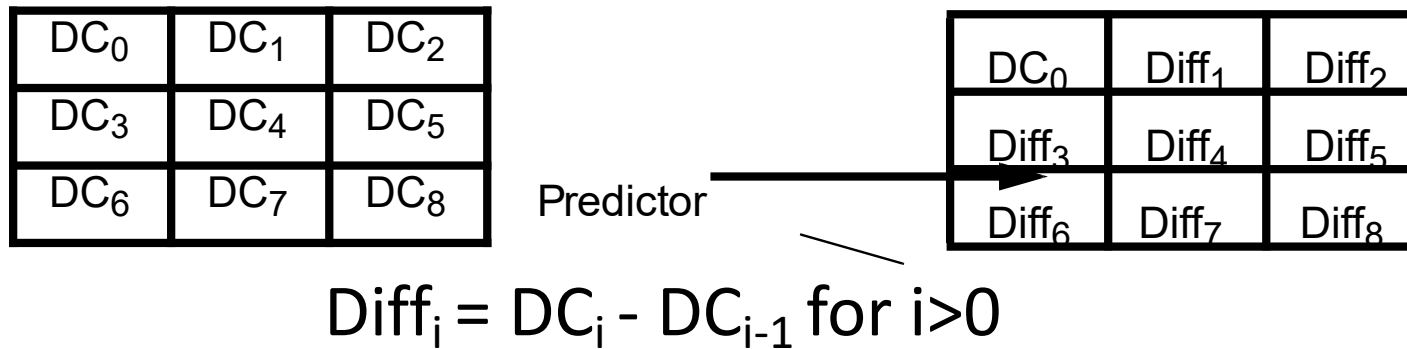
*AC-coefficients*: consider each block separately, order data using zig-zag sequence to achieve long sequences of zero- values:

[illegible]

*Entropy encoding:*

- Run-length encoding of zero values of quantized AC-coefficients
- Huffman encoding on DC- and AC-coefficients

# JPEG Entropy Encoding



- AC coefficients processing:
  - processing order of the AC coefficients using zig-zag scan (coefficients with lower frequencies are encoded first) ==>
  - sequence of similar data bytes ==> efficient entropy coding
- JPEG standard specifies Huffman or Arithmetic coding, but sequential encoding mode uses only Huffman coding.



# JPEG — Entropy Encoding

Algorithm:

1. apply run-length coding of AC coefficient of zero values
2. apply Huffman coding on DC and AC coefficients

(only DC coefficients explained in detail)

# JPEG — Entropy Encoding

- DC coefficients—Huffman coding
  - categorize DC values into DC code tables
  - difference magnitude categories for DC coefficients (12 categories)

| Diff Values      | SSSS (number of bits<br>needed to encode Diff) |
|------------------|------------------------------------------------|
| 0                | 0 bit                                          |
| -1, 1            | 1 bit                                          |
| -3, ..., 3       | 2 bits                                         |
| -7, ..., 7       | 3 bits                                         |
| .                |                                                |
| .                |                                                |
| .                |                                                |
| -2047, ..., 2047 | 11 bits                                        |

# JPEG — Entropy Encoding

- DC coefficients — Huffman encoding
  - handle SSSS as Huffman symbol, get  $p(0)$ ,  $p(1)$ , ...  $p(11)$  and create Huffman tree ==> Huffman code for SSSS
  - for each category, an additional bits field is appended to the code word to uniquely identify which difference in that category actually occurred.
  - send: (Huffman codeword, actual value)

Example: If SSSS=2 has the Huffman code 001 and Diff=-3 ==>

send 00100

Huffman code for 2      -3 in two's complement is 00 because 3=>11

Details about JPEG Entropy Coding in

<https://cnx.org/contents/IJvrSfU5@3.3:eMc9DKWD@5/JPEG-Entropy-Coding>

# JPEG Image Processing (DCT) - Decoding

- Inverse Discrete Cosine Transformation (I-DCT) maps DCT coefficients to sampled values

$$s_{xy} = \frac{1}{4} \sum_{u=0}^7 \sum_{v=0}^7 c_u c_v s_{vu} \cos \frac{(2x+1)u\pi}{16} \cos \frac{(2y+1)v\pi}{16}$$

with  $c_u, c_v = 1/\sqrt{2}$  for  $u,v=0$ ; otherwise  $c_u, c_v = 1$

- DCT and I-DCT cannot be calculated in full precision  
==> lossy compression
- JPEG does not define precision parameters, therefore various implementations exist
- many AC coefficients with small values (around zero)

# Comments

- Applications do not have to include both an encoder and decoder if the compression process agrees on a common table
- The encoded data stream has a fixed *Interchange Format*
  - encoded image data
  - chosen parameters
  - tables of the coding process
- In regular mode, the interchange format includes all of the information necessary for decoding without any previous knowledge of the coding process

# DCT Examples



- DCT sorts values from the lowest to the highest frequency
- In image blocks it is likely that the energy is concentrated in low frequencies
  - Regions with smooth colors or little details (= low spatial frequencies) have high values
  - Regions with different colors and detail (= high spatial frequencies), most values are almost zero.



8x8 Block (Luminance)

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 134 | 142 | 145 | 131 | 114 | 122 | 131 | 130 |
| 129 | 143 | 134 | 130 | 135 | 144 | 134 | 118 |
| 123 | 117 | 118 | 111 | 97  | 109 | 130 | 143 |
| 129 | 116 | 112 | 116 | 120 | 126 | 130 | 118 |
| 118 | 127 | 141 | 138 | 138 | 148 | 141 | 125 |
| 125 | 129 | 119 | 127 | 143 | 149 | 145 | 136 |
| 131 | 126 | 128 | 142 | 141 | 135 | 126 | 116 |
| 131 | 140 | 146 | 154 | 133 | 118 | 124 | 124 |

Pixelvalues

Low frequencies

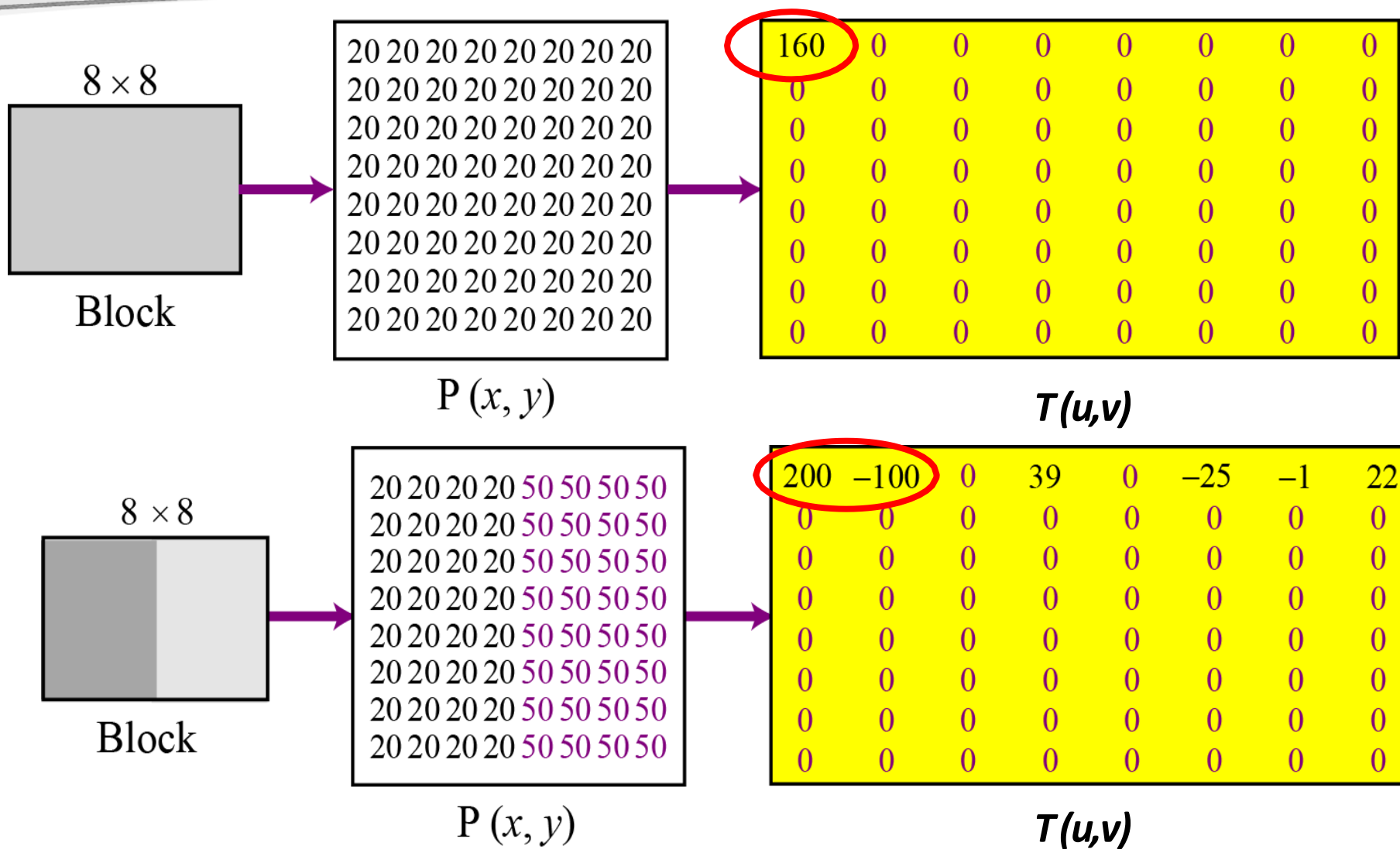
|      |     |     |     |     |    |    |    |
|------|-----|-----|-----|-----|----|----|----|
| 1037 | -1  | -6  | 1   | -12 | 8  | -4 | -4 |
| -16  | 1   | 28  | -6  | -14 | 0  | 4  | 0  |
| 19   | 32  | -7  | -19 | 2   | -1 | -4 | -3 |
| 29   | -9  | -14 | 13  | -10 | -6 | 1  | 0  |
| 4    | 14  | -6  | -13 | -2  | 7  | 1  | 2  |
| -26  | 2   | 16  | 2   | 11  | 6  | 1  | 1  |
| -10  | -11 | 27  | -18 | 4   | 1  | 0  | 0  |
| -2   | 1   | 1   | -19 | -1  | 6  | 6  | 0  |

High frequencies

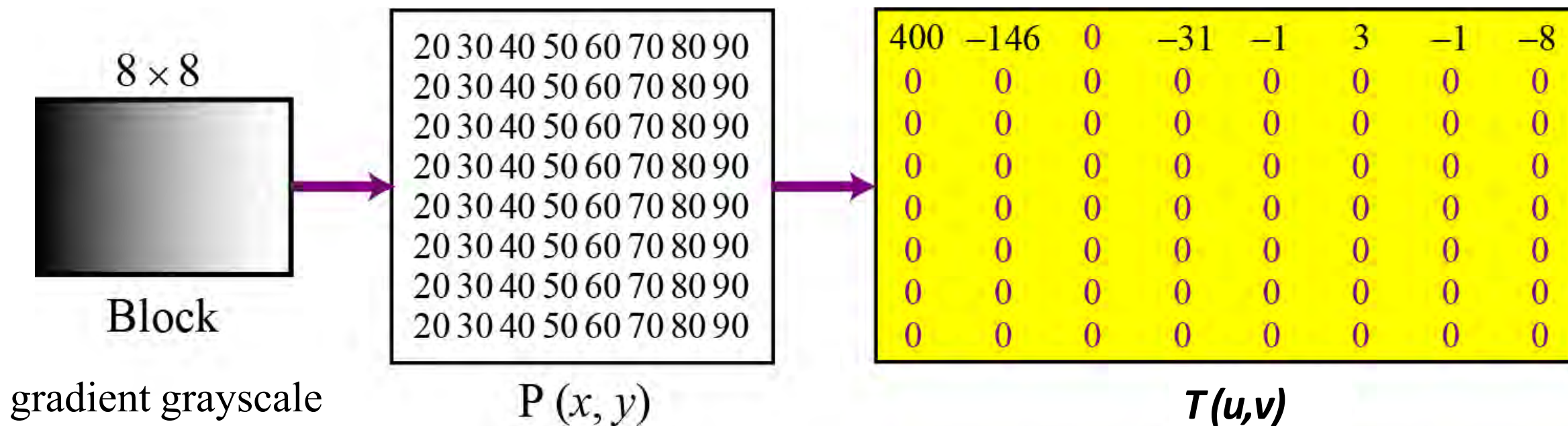
DCT- Values



# DCT Examples



# DCT Examples



# DCT Basis Functions

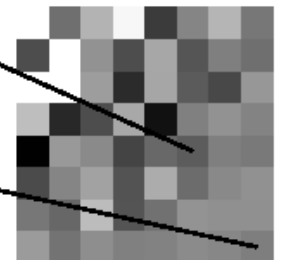
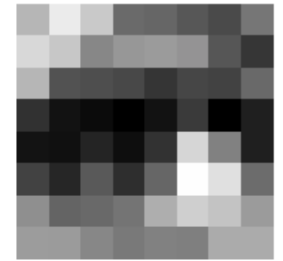
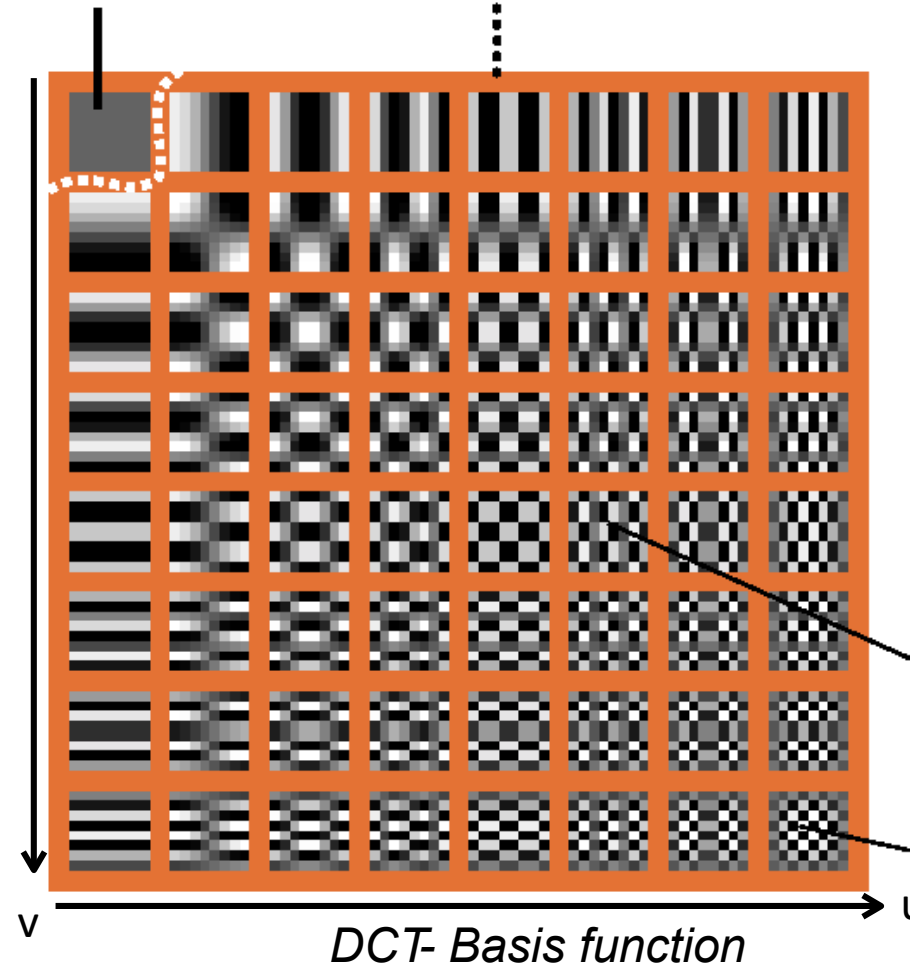


- DCT values indicate the weighting of frequency images
- The brighter the pixel, the greater the value of DCT

*Constant component*

*Alternating component*

*8 x 8 Imageblock*



# Quantization



- Compression effect: DCT values are approximated (quantized) to make them smaller and to repeat themselves
  - Each DCT value is divided by a quantization factor and then rounded
  - The larger the quantization factor, the smaller the values to be stored

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 134 | 142 | 145 | 131 | 114 | 122 | 131 | 130 |
| 129 | 143 | 134 | 130 | 135 | 144 | 134 | 118 |
| 123 | 117 | 118 | 111 | 97  | 109 | 130 | 143 |
| 129 | 116 | 112 | 116 | 120 | 126 | 130 | 118 |
| 118 | 127 | 141 | 138 | 138 | 148 | 141 | 125 |
| 125 | 129 | 119 | 127 | 143 | 149 | 145 | 136 |
| 131 | 126 | 128 | 142 | 141 | 135 | 126 | 116 |
| 131 | 140 | 146 | 154 | 133 | 118 | 124 | 124 |

*Pixelvalues*

|      |     |     |     |     |    |    |    |
|------|-----|-----|-----|-----|----|----|----|
| 1037 | -1  | -6  | 1   | -12 | 8  | -4 | -4 |
| -16  | 1   | 28  | -6  | -14 | 0  | 4  | 0  |
| 19   | 32  | -7  | -19 | 2   | -1 | -4 | -3 |
| 29   | -9  | -14 | 13  | -10 | -6 | 1  | 0  |
| 4    | 14  | -6  | -13 | -2  | 7  | 1  | 2  |
| -26  | 2   | 16  | 2   | 11  | 6  | 1  | 1  |
| -10  | -11 | 27  | -18 | 4   | 1  | 0  | 0  |
| -2   | 1   | 1   | -19 | -1  | 6  | 6  | 0  |

*DCT- Values*

|     |    |    |    |    |   |   |   |
|-----|----|----|----|----|---|---|---|
| 130 | 0  | 0  | 0  | -1 | 0 | 0 | 0 |
| -2  | 0  | 3  | 0  | -1 | 0 | 0 | 0 |
| 2   | 3  | 0  | -1 | 0  | 0 | 0 | 0 |
| 3   | -1 | -1 | 1  | 0  | 0 | 0 | 0 |
| 0   | 1  | 0  | -1 | 0  | 0 | 0 | 0 |
| -2  | 0  | 2  | 0  | 0  | 0 | 0 | 0 |
| -1  | -1 | 2  | -1 | 0  | 0 | 0 | 0 |
| 0   | 0  | 0  | -1 | 0  | 0 | 0 | 0 |

*Quantized DCT- Values*

# Quantization Matrix



- Quantization factor for each DCT value is defined using a quantization matrix
  - Disturbances in low frequency parts of the image are perceived strongly
  - Disturbances in high frequency parts of the image are less noticeable
  - Quantization matrices in JPEG standard ensures that the DCT values for low frequencies are stored more accurately than for high frequencies
  - Quantization table can be freely selected in the compression

|    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|
| 8  | 16 | 19 | 22 | 26 | 27 | 29 | 34 |
| 16 | 16 | 22 | 24 | 27 | 29 | 34 | 37 |
| 19 | 22 | 26 | 27 | 29 | 34 | 34 | 38 |
| 22 | 22 | 26 | 27 | 29 | 34 | 37 | 40 |
| 22 | 26 | 27 | 29 | 32 | 35 | 40 | 48 |
| 26 | 27 | 29 | 32 | 35 | 40 | 48 | 58 |
| 26 | 27 | 29 | 34 | 38 | 46 | 56 | 69 |
| 27 | 29 | 35 | 38 | 46 | 56 | 69 | 83 |

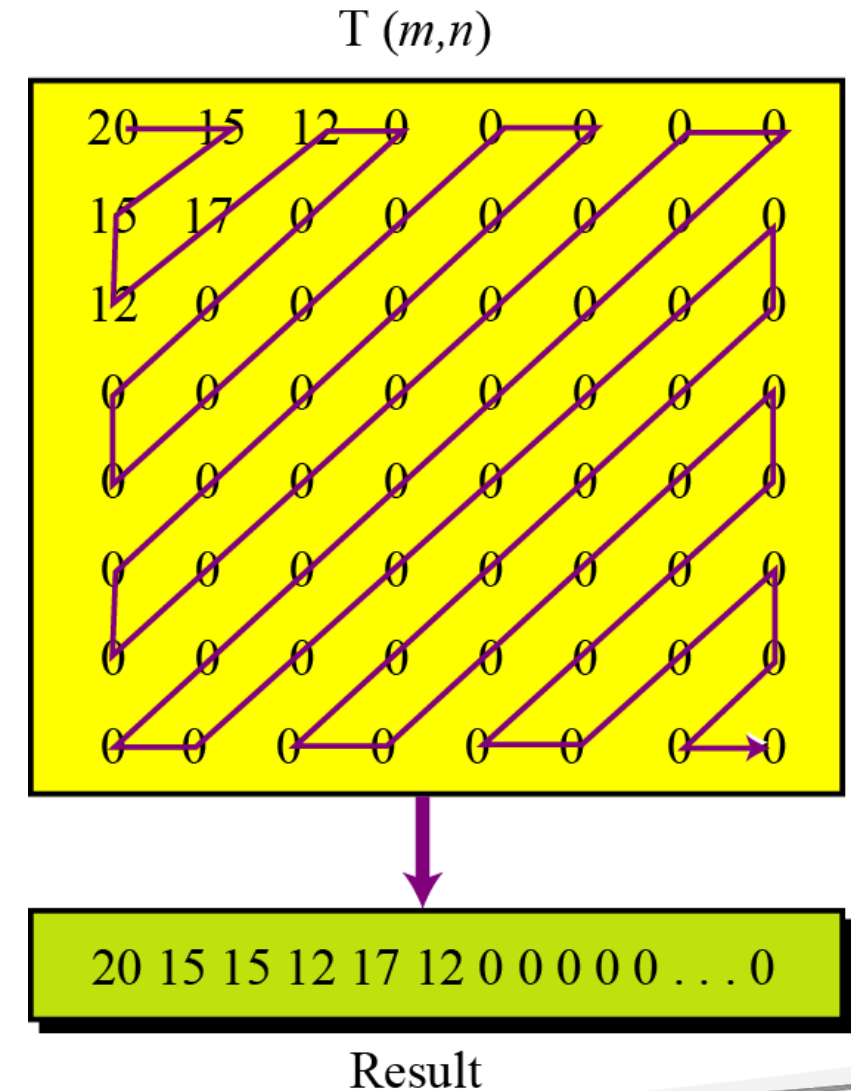
*Quantization Matrix*

|    |    |    |    |     |     |     |     |
|----|----|----|----|-----|-----|-----|-----|
| 16 | 11 | 10 | 16 | 24  | 40  | 51  | 61  |
| 12 | 12 | 14 | 19 | 26  | 58  | 60  | 55  |
| 14 | 13 | 16 | 24 | 40  | 57  | 69  | 56  |
| 14 | 17 | 22 | 29 | 51  | 87  | 80  | 62  |
| 18 | 22 | 37 | 56 | 68  | 109 | 103 | 77  |
| 24 | 35 | 55 | 64 | 81  | 104 | 113 | 92  |
| 49 | 64 | 78 | 87 | 103 | 121 | 120 | 101 |
| 72 | 92 | 95 | 98 | 112 | 100 | 103 | 99  |

*Quantization Matrix*

# Compression

- After quantization values are read from the table, and redundant 0s are removed.
- To cluster 0s together process reads the table diagonally in a zigzag fashion because if image does not have fine changes bottom right corner of table is all 0s.
- JPEG uses run-length encoding at the compression phase to compress the bit pattern resulting from the zigzag linearization.





# Quantization



## Coding

|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 134 | 142 | 145 | 131 | 114 | 122 | 131 | 130 |
| 129 | 143 | 134 | 130 | 135 | 144 | 134 | 118 |
| 123 | 117 | 118 | 111 | 97  | 109 | 130 | 143 |
| 129 | 116 | 112 | 116 | 120 | 126 | 130 | 118 |
| 118 | 127 | 141 | 138 | 138 | 148 | 141 | 125 |
| 125 | 129 | 119 | 127 | 143 | 149 | 145 | 136 |
| 131 | 126 | 128 | 142 | 141 | 135 | 126 | 116 |
| 131 | 140 | 146 | 154 | 133 | 118 | 124 | 124 |

*Pixelvalues*

|      |     |     |     |     |    |    |    |
|------|-----|-----|-----|-----|----|----|----|
| 1037 | -1  | -6  | 1   | -12 | 8  | -4 | -4 |
| -16  | 1   | 28  | -6  | -14 | 0  | 4  | 0  |
| 19   | 32  | -7  | -19 | 2   | -1 | -4 | -3 |
| 29   | -9  | -14 | 13  | -10 | -6 | 1  | 0  |
| 4    | 14  | -6  | -13 | -2  | 7  | 1  | 2  |
| -26  | 2   | 16  | 2   | 11  | 6  | 1  | 1  |
| -10  | -11 | 27  | -18 | 4   | 1  | 0  | 0  |
| -2   | 1   | 1   | -19 | -1  | 6  | 6  | 0  |

*DCT- Values*

|     |    |    |    |    |   |   |   |
|-----|----|----|----|----|---|---|---|
| 130 | 0  | 0  | 0  | -1 | 0 | 0 | 0 |
| -2  | 0  | 3  | 0  | -1 | 0 | 0 | 0 |
| 2   | 3  | 0  | -1 | 0  | 0 | 0 | 0 |
| 3   | -1 | -1 | 1  | 0  | 0 | 0 | 0 |
| 0   | 1  | 0  | -1 | 0  | 0 | 0 | 0 |
| -2  | 0  | 2  | 0  | 0  | 0 | 0 | 0 |
| -1  | -1 | 2  | -1 | 0  | 0 | 0 | 0 |
| 0   | 0  | 0  | -1 | 0  | 0 | 0 | 0 |

*Quantized DCT- Values*

## Decoding

|      |     |    |     |     |   |   |   |
|------|-----|----|-----|-----|---|---|---|
| 1040 | 0   | 0  | 0   | -9  | 0 | 0 | 0 |
| -12  | 0   | 24 | 0   | -10 | 0 | 0 | 0 |
| 14   | 24  | 0  | -10 | 0   | 0 | 0 | 0 |
| 24   | -8  | -9 | 10  | 0   | 0 | 0 | 0 |
| 0    | 9   | 0  | -10 | 0   | 0 | 0 | 0 |
| -19  | 0   | 10 | 0   | 0   | 0 | 0 | 0 |
| -9   | -10 | 21 | -12 | 0   | 0 | 0 | 0 |
| 0    | 0   | 0  | -14 | 0   | 0 | 0 | 0 |

*Reconstructed DCT- Values*

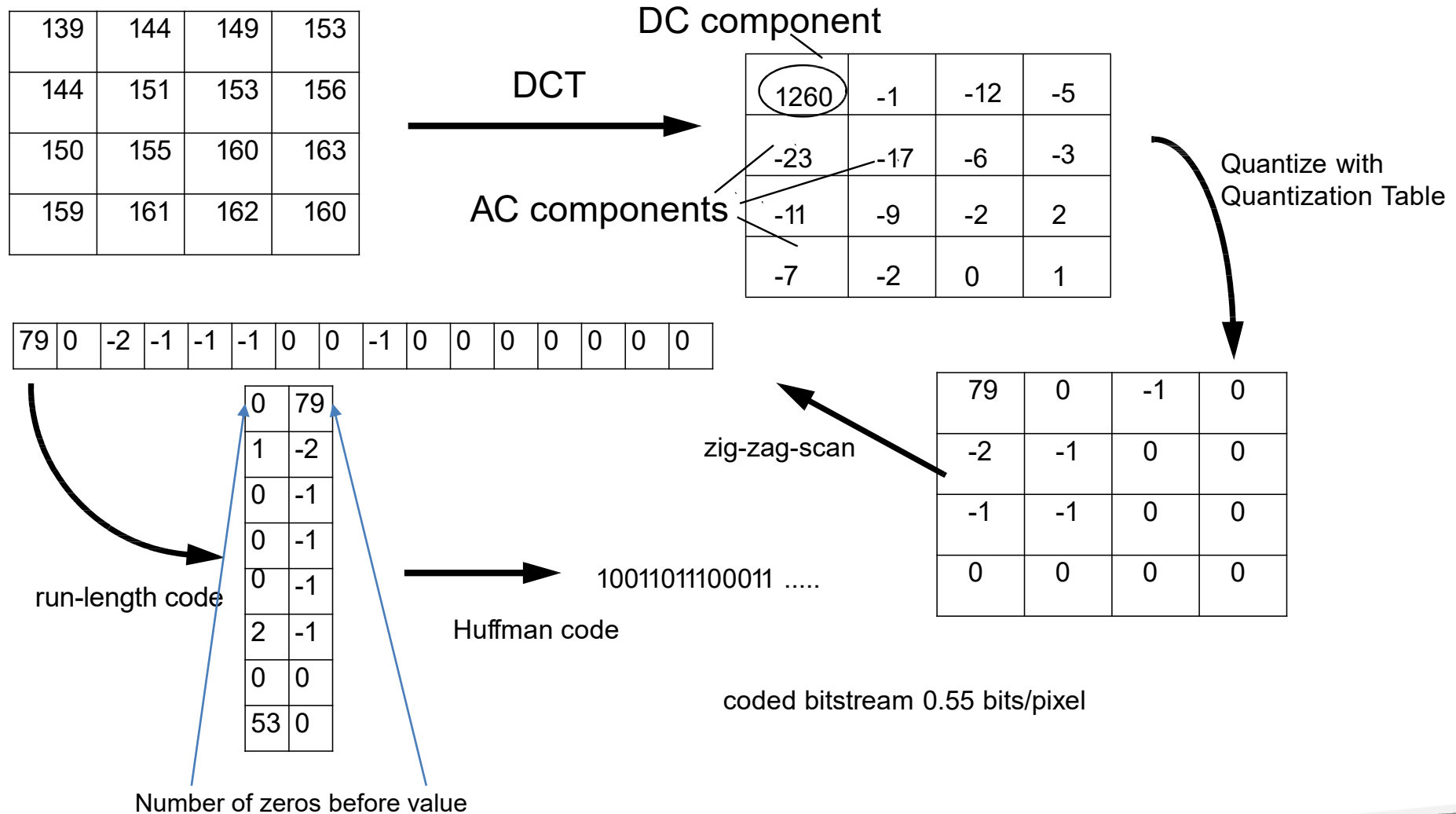
|     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|
| 136 | 141 | 138 | 125 | 119 | 125 | 132 | 134 |
| 136 | 137 | 133 | 130 | 134 | 139 | 133 | 121 |
| 121 | 125 | 122 | 112 | 107 | 117 | 130 | 138 |
| 125 | 123 | 119 | 117 | 121 | 128 | 127 | 122 |
| 123 | 129 | 135 | 139 | 140 | 139 | 136 | 131 |
| 129 | 125 | 124 | 130 | 139 | 144 | 141 | 136 |
| 129 | 130 | 134 | 139 | 140 | 135 | 127 | 120 |
| 132 | 138 | 144 | 141 | 131 | 122 | 122 | 127 |

*Reconstructed Pixels (IDCT)*

|    |    |    |    |     |    |    |    |
|----|----|----|----|-----|----|----|----|
| -2 | 1  | 7  | 6  | -5  | -3 | -1 | -4 |
| -7 | 6  | 1  | 0  | 1   | 5  | 1  | -3 |
| 2  | -8 | -4 | -1 | -10 | -8 | 0  | 5  |
| 4  | -7 | -7 | -1 | -1  | -2 | 3  | -4 |
| -5 | -2 | 6  | -1 | -2  | 9  | 5  | -6 |
| -4 | 4  | -5 | -3 | 4   | 5  | 4  | 0  |
| 2  | -4 | -6 | 3  | 1   | 0  | -1 | -4 |
| 1  | 2  | 2  | 13 | 2   | -4 | 1  | -3 |

*Errors (Difference)*

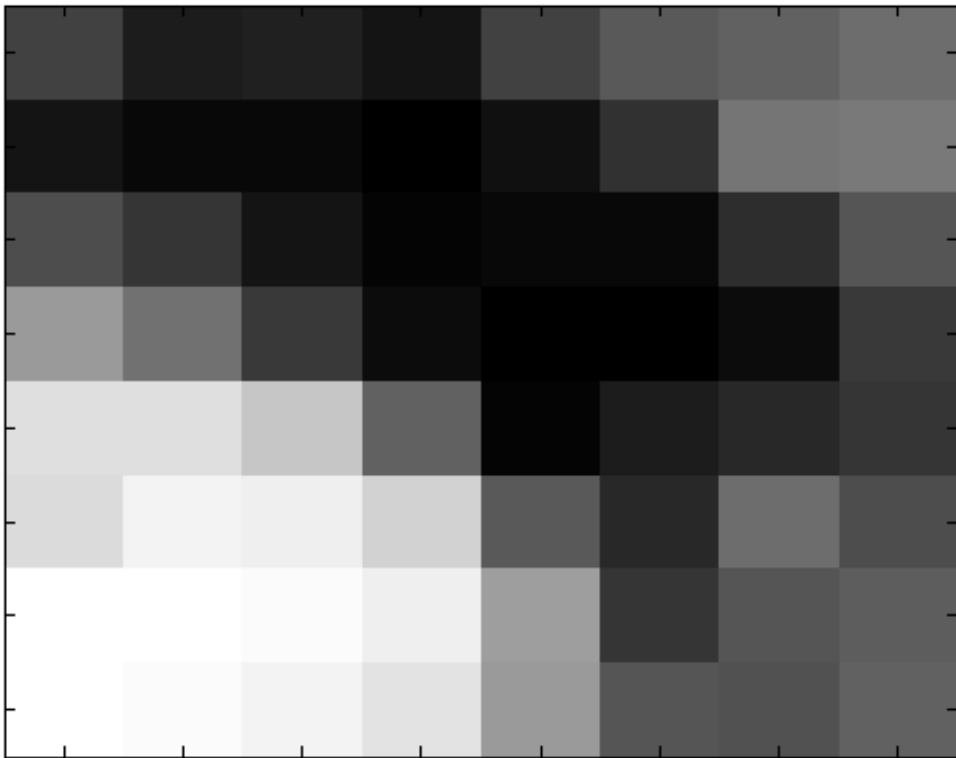
# JPEG Block Encoding (Example)



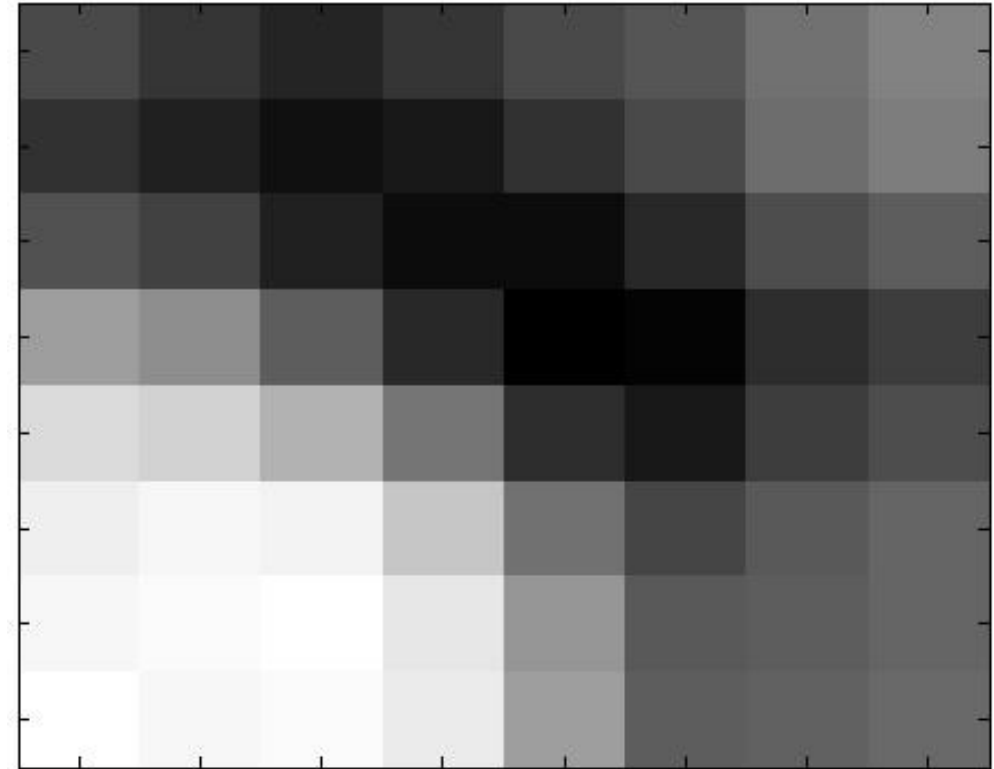
# Can You Tell the Difference?



Original



Compressed



# Image Compression



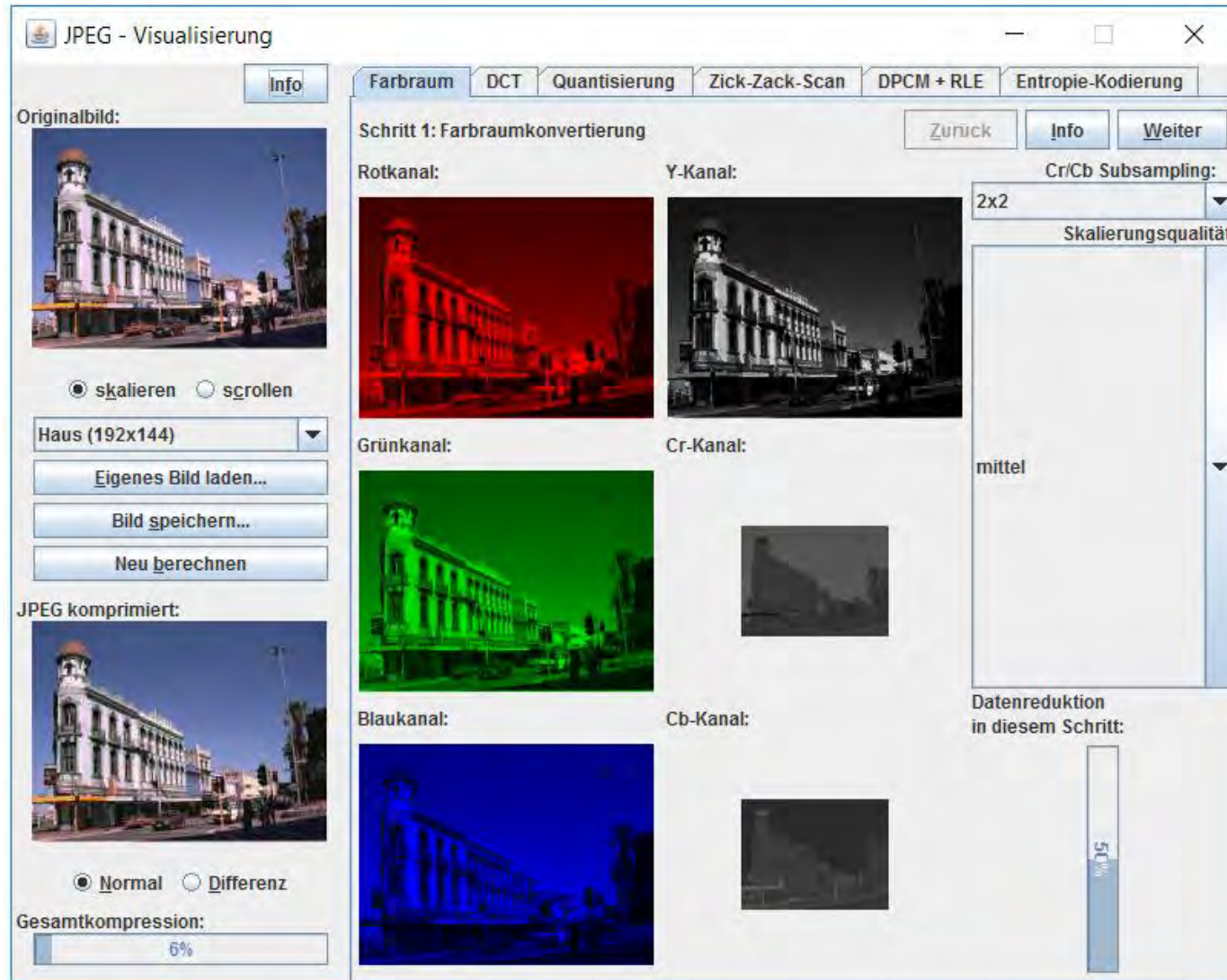
Original



Compressed



# VisJPEG-Applet





# Problems with JPEG

- Behavior in sharp transitions (e.g. fonts)
- Emergence of the 8x8 blocks at high compression rates.
- The greater the compression (quantization), the more artefacts occur (block formation)
- JPEG is made for natural images and not for artificial images (computer graphics)!





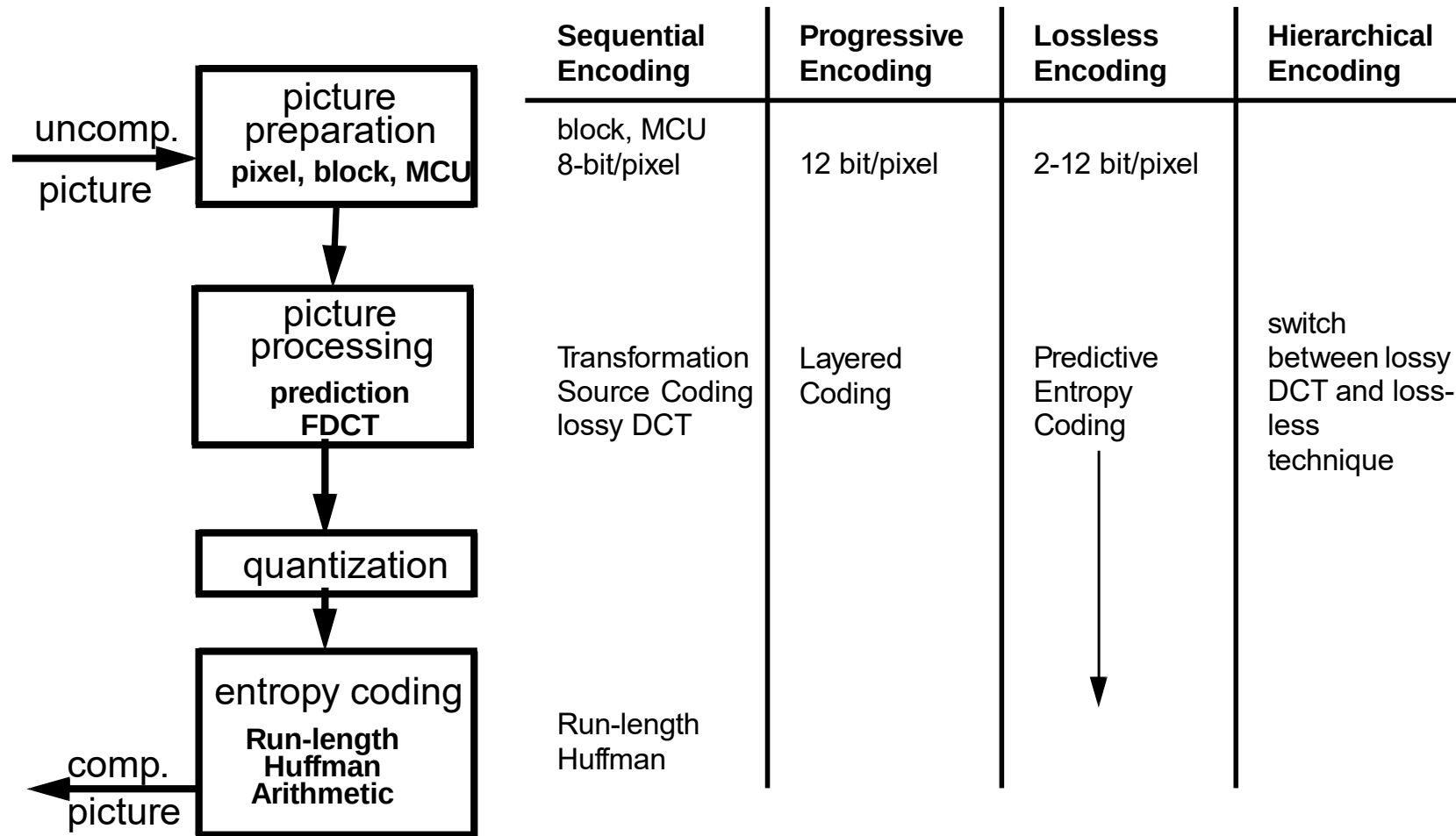


# Variations & Additions to JPEG

## Additional Material (not for Exam!)

Slides by Prof. Dr. Otto Spaniol, Dirk Thißen  
RWTH Aachen

# JPEG Processing Steps

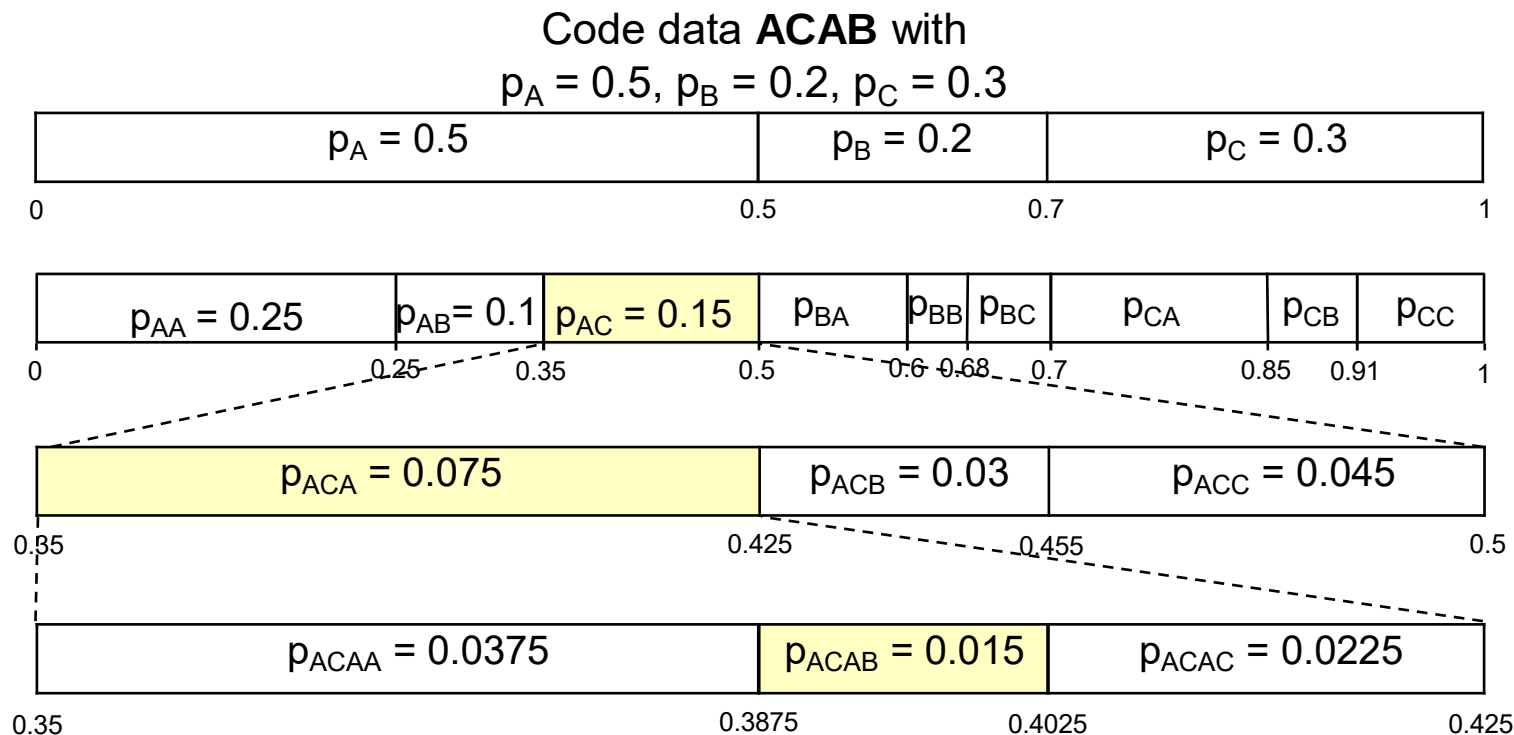


## Alternative to Huffman: Arithmetic Coding

### Characteristics:

- Achieves optimality (coding rate) as the Huffman coding
- Difference to Huffman: the entire data stream has an assigned probability, which consists of the probabilities of the contained characters. Coding a character takes place with consideration of all previous characters.
- The data are coded as an interval of real numbers between 0 and 1. Each value within the interval can be used as code word.
- The minimum length of the code is determined by the assigned probability.
- Disadvantage: the data stream can be decoded only as a whole.

## Arithmetic Coding: Example



ACAB can be coded by each binary number from the interval  $[0.3875, 0.4025)$ , rounded up to  $-\log_2(p_{ACAB}) = 6.06$  i.e. 7 bit, e.g. **0.0110010**

## Variants of Image Compression

JPEG is not a single format, but it can be chosen from a number of modes:

*Lossy sequential DCT-based mode* (baseline process)

- Presented before, but not the only method

*Expanded lossy DCT-based mode*

- Enhancement to the baseline process by adding progressive encoding

*Lossless mode*

- Low compression ratio → „perfect“ reconstruction of original image
- No DCT, but differential encoding

*Hierarchical mode*

- Accommodates images of different resolutions
- Selects its algorithms from the three other modes



## Variants: Expanded Lossy DCT-based Mode

With *sequential encoding* as in the baseline process the whole image is coded and decoded in a single run. An alternative to sequential encoding is *progressive encoding*, done in the entropy encoding step.

Two alternatives for progressive encoding are possible:

- *Spectral selection*

At first, coefficients of low frequencies are passed to entropy encoding, coefficients of higher frequencies are processed in successive runs

- *Successive approximation*

All coefficients are transferred in one run, but most-significant bits are encoded prior to less-significant bits.

12 possible coding alternatives in the expanded mode:

- Using sequential encoding, spectral selection, or successive approximation (3 variants)
- Using Huffman or Arithmetic encoding (2 variants)
- Using 8 or 12 bits for representing the samples (2 variants)

*Most popular mode*: sequential display mode with 8 bits/sample and Huffman encoding

## Expanded Lossy DCT-based Mode (Example)

**Sequential encoding:** image is coded and decoded in a single run



Step 1

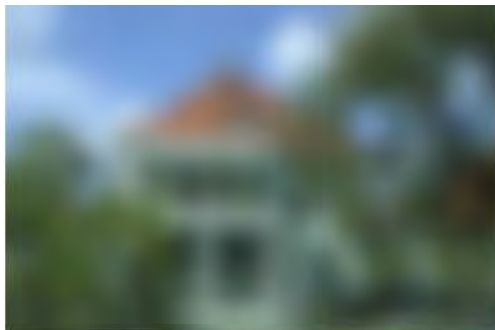


Step 2



Step 3

**Progressive encoding:** image is coded and decoded in refining steps



Step 1



Step 2



Step 3

## Variants: Lossless Mode

Lossless mode uses *differential encoding*

(Differential encoding is also known as *prediction* or *relative encoding*)

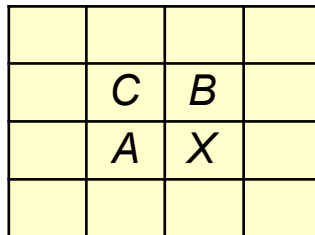
- Sequence of characters whose values are different from zero, but which do not differ much.
  - Calculate only the difference wrt. the previous value (used also for DC-coefficients)

Differential encoding for still images:

- Avoid using DCT/quantization
- Instead: calculation of differences between nearby pixels or pixel groups
  - Edges are represented by large values
  - Areas with similar luminance and chrominance are represented by small values
  - Homogenous area is represented by a large number of zeros
    - further compression with run-length encoding is possible as for DCT

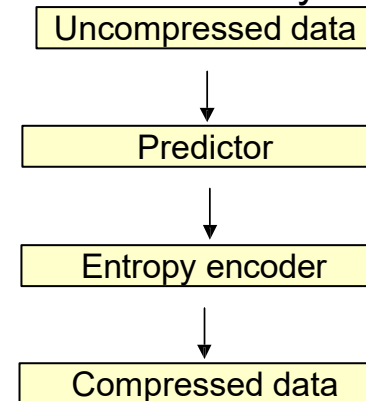
## Variants: Lossless Mode

- Uses data units of single pixels for image preparation
- Any precision between 2 and 16 bits/pixel can be used
- Image processing and quantization use a predictive technique instead of transformation encoding
- 8 predictors are specified for each pixel  $X$  by means of a combination of the already known adjacent samples  $A$ ,  $B$ , and  $C$



The actual predictor should give the best approximation of  $x$  by the already known values  $A, B, C$

| predictor | predicted values $X$ |
|-----------|----------------------|
| 0         | no prediction        |
| 1         | $A$                  |
| 2         | $B$                  |
| 3         | $C$                  |
| 4         | $A+B-C$              |
| 5         | $A+(B-C)/2$          |
| 6         | $B+(A-C)/2$          |
| 7         | $(A+B)/2$            |



The number of the chosen predictor and the difference of the prediction to the actual value are passed to entropy encoding (Huffman or Arithmetic Encoding)

### Example:

(4,0):  $X$  is exactly given by  $A+B-C$   
 (7,1):  $X$  is  $(A+B)/2+1$

## Variants: Hierarchical Mode

The *Hierarchical mode* uses either the lossy DCT-based algorithms or the lossless compression technique

The idea: encoding of an image at different resolutions

*Algorithm:*

- Image is initially sampled at a low resolution
- Subsequently, the resolution is raised and the compressed image is subtracted from the previous result
- The process is repeated until the full resolution of the image is obtained in a compressed form

*Disadvantage:*

- Requires substantially more storage capacity

*Advantage:*

- Compressed image is immediately available at different resolutions  
→ *scaling becomes cheap*

# Multimedia

## Vorlesung

Hannes Kaufmann

Research Unit Virtual and Augmented Reality  
Institute of Visual Computing & Human-Centered Technology



# How Video Streaming Works

## What are CDNs, HLS and DASH

In addition to the videos, which are linked on the next slides, this website contains a summary and details of the concepts discussed in the videos – find detailed information summarized in writing here:

<https://www.wowza.com/blog/streaming-protocols>

This helps with understanding when learning for the exam.

# What Is A CDN? How Does It Work?

<https://www.youtube.com/watch?v=RI9np1LWzqw>

## Relevant Exam Questions:

- What is a Content Delivery Network and what advantages does the usage of a CDN have?
- How does a CDN work and what is a PoP?
- What is DNS-based routing and what is Anycast?
- Why can a CDN provide content much faster?
- Why is a CDN more secure and more reliable?

# How Does a Live Streaming Platform Work?

## (YouTube live, Twitch, TikTok Live)

<https://www.youtube.com/watch?v=7AMRfNKwuYo>

### Relevant Exam Questions:

- Give an example for a transport protocol for live streaming.
- What is Adaptive-Bitrate-Streaming?
- What is meant by segmentation in transcoding for video streaming?
- What are most common live streaming formats?
- What does an HLS file consist of?
- What means latency in video streaming and why is it relevant?

# How does Netflix work? How does streaming work? Why is Netflix so fast?

<https://www.youtube.com/watch?v=G36Qbk0c2WM>

## Relevant Exam Questions:

- How does Netflix increase speed even further by working with Internet Service Providers (ISPs)?

# Live Streaming Protocol Comparison 1/2

<https://www.youtube.com/watch?v=jYYhwbF3tvM>

Details are also on the page

<https://www.wowza.com/blog/streaming-protocols>

## Relevant Exam Questions 1/2:

- Is the same protocol and codec used for recording and playing a live streamed video?
- Which protocols are used on the recording/encoder side, which on the playback side?
- What is the main difference between TCP and UDP? (see also website <https://www.wowza.com/blog/streaming-protocols>)

# Live Streaming Protocol Comparison 2/2

## Relevant Exam Questions 2/2:

- What do you know about RTSP and what are the best applications to use it?
- Can you give details about Apple HLS?
- What is MPEG-DASH? Which codecs does it support? Which latency does it have?

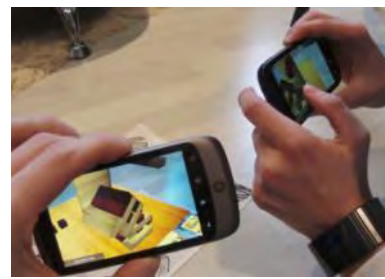
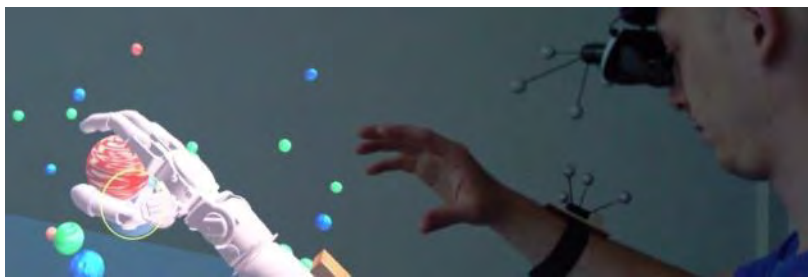


# Multimedia

## Vorbesprechung

Hannes Kaufmann

Research Unit Virtual and Augmented Reality  
Institute of Visual Computing & Human-Centered Technology



# TU WIEN: VIRTUAL & AUGMENTED REALITY

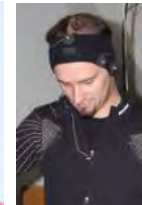
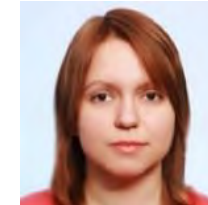


Univ.Prof. Mag. Dr.  
Hannes Kaufmann



Ao.Univ.Prof. Dr.  
Horst Eidenberger

## Research Staff



**Postdocs:** Peter Kán, Iana Podkosova, Christian Schöner, Francesco De Pace, Hugo Brument

**PhDs:** Khrystyna Vasylevska, Emanuel Vonach, Soroosh Mortezaipoor, Mohammad Ghazanfari, Matteo Bosco  
+ 3 external PhDs

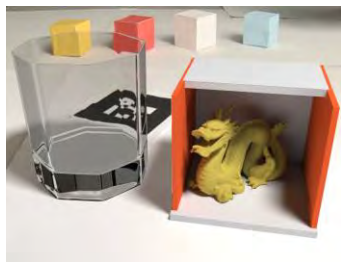
**Students:** 15 graduate and undergraduate students involved in research

# Components of Mixed Reality



Total:  
138,9  
FPS

High Quality Rendering in AR



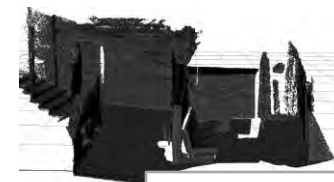
Rendering &  
Viewing

Localization  
(Tracking)

Wide Area Multi-User Optical  
Localization



ioTracker



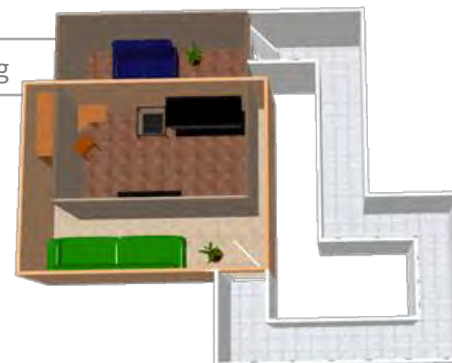
Live 3D  
Reconstruction for  
First Responders

Shared Collaborative  
Virtual Spaces Navigation



Output

Redirected Walking



Interaction

Distribution

Collaboration

Authoring

Perception

Automated 3D Model Generation

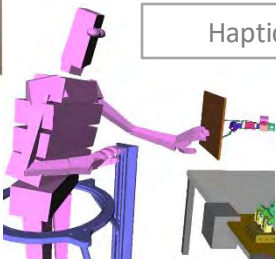


Object Detection and  
Segmentation in Live 3D  
Reconstruction

Firefighter Training with Virtualizer



Haptic Feedback



Input

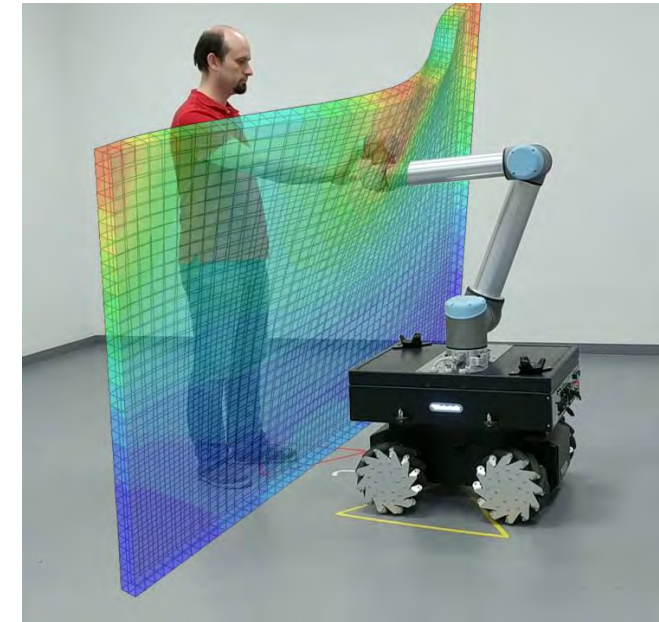
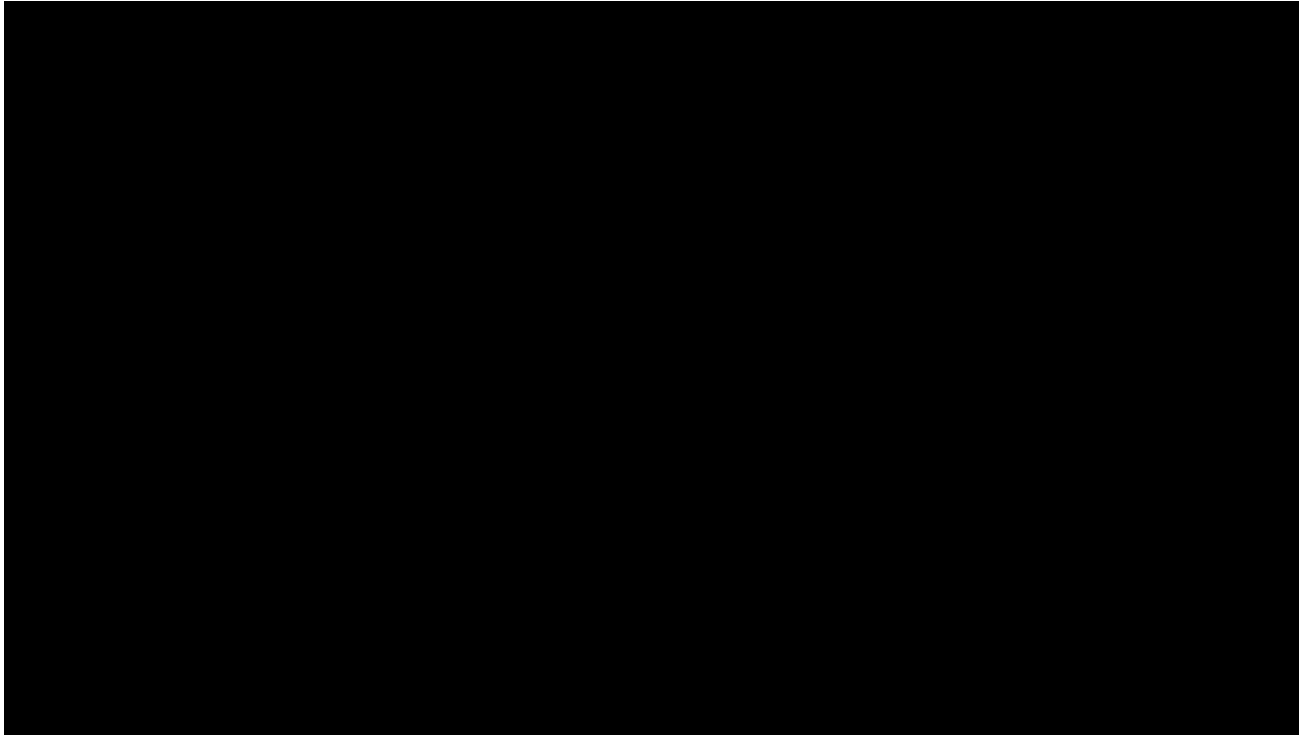
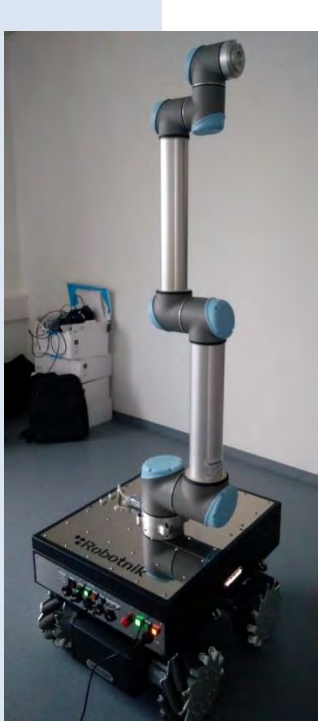


# Real Walking in Large VR - Platform Requirements

- Multi-user for **Gaming**
- Users navigate by walking
- **Low-cost** large area tracking system
- Full-body animated avatars
- Virtual elevator
- Haptic interaction
- 2 ways of using space
  - Collaboration
  - Non-shared space



# Interaction: Haptic Feedback in Large Scale VR

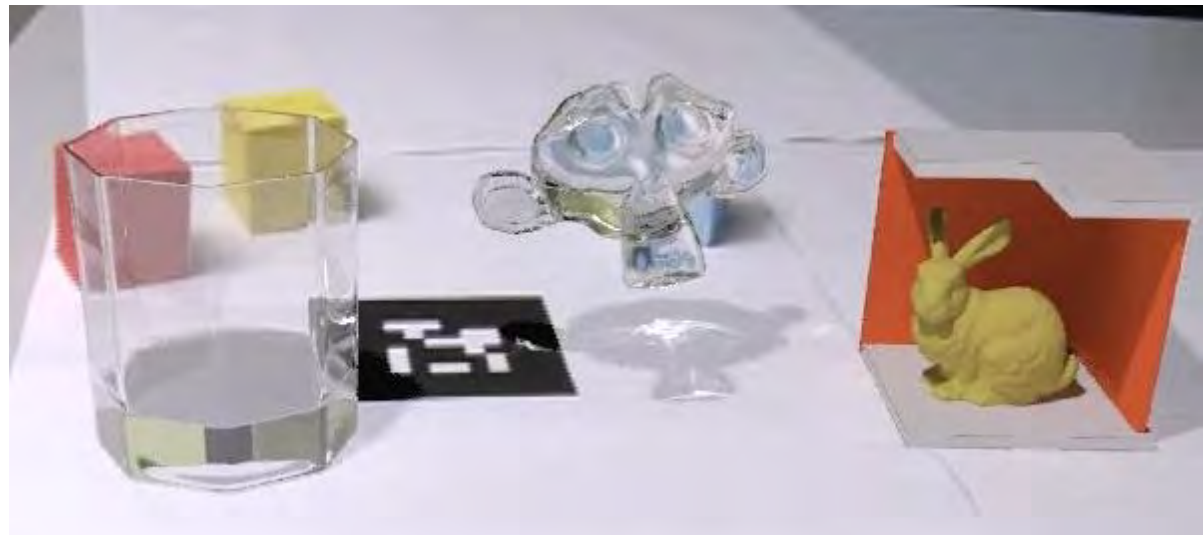
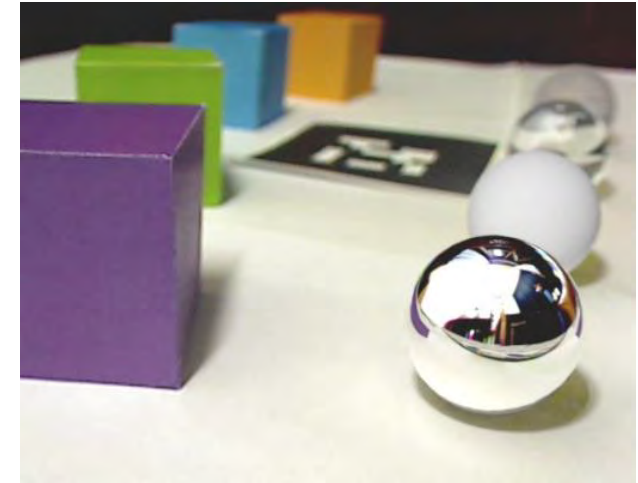


- Robot actuated physical props for passive haptic feedback
- Challenges:
- Unexpected user movements
  - Close direct interaction & multiple users in environment
  - Haptic response from differently stiff materials required



# Real-time Ray Tracing in VR/AR

- Real-time ray tracing for AR environments
  - Correctly simulate light interaction between real and virtual objects
  - Photorealistic rendering in AR





# Applications: Medical & Biosensor

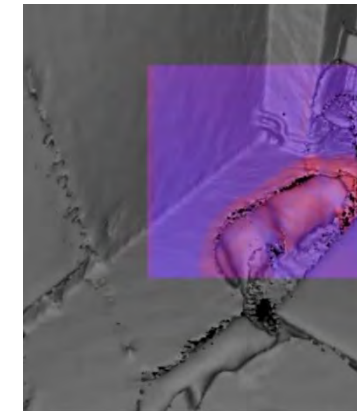
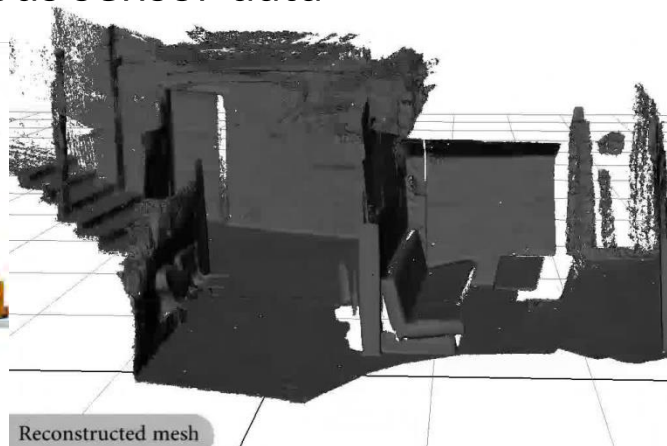
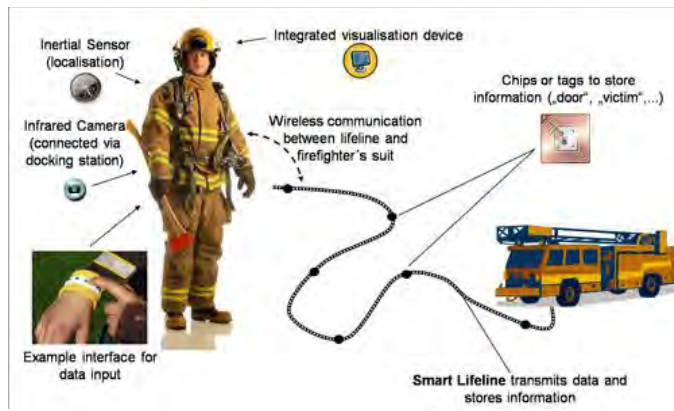
- **PLAYMANCER**

3D-Serious Game Environment  
with real-time motion capturing  
and bio-signal feedback for  
physical rehabilitation



- **ProFiTex**

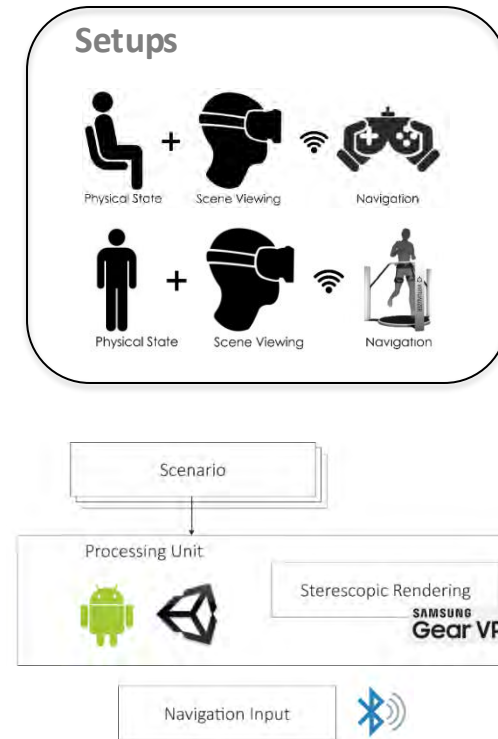
Support fire fighters with mission-relevant  
information based on various sensor data



# Applications: Training

## Virtual Simulation and On-Site Training for First Responders

- Mobile immersive virtual reality training platform
  - On-site squad leaders of disaster relief units
- Save cost and time, increase training time



A. Mossel, M. Froeschl, C. Schoenauer, A. Peer, J. Goellner, H. Kaufmann (2017), VRonSite: Towards Immersive Training of First Responder Squad Leaders in Untethered Virtual Reality, *IEEE Virtual Reality*, March 2017, Los Angeles, USA



# Current Infrastructure @ TU

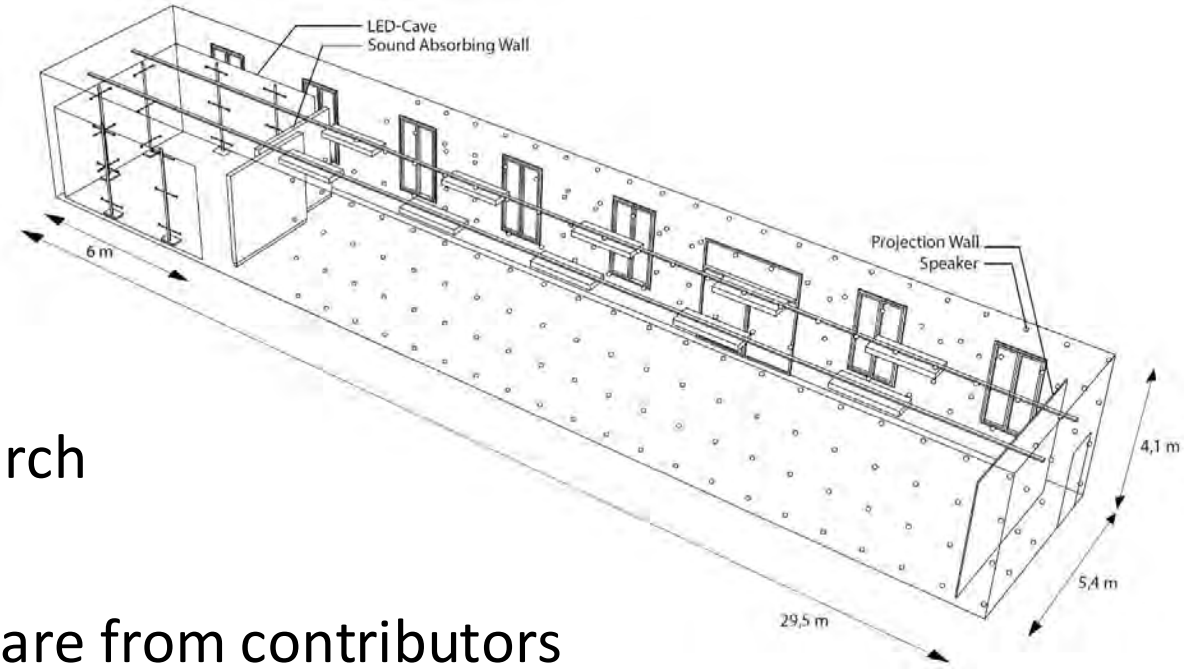
- VR Lab at Science Center (~140 m<sup>2</sup>)
  - Haptic research (SFB), large-scale VR
- Hallway Lab (~20 m<sup>2</sup>)
  - All teaching, student lab work & hand ins
- VR Lab (~20 m<sup>2</sup>)



# Infrastructure: TU Mixed Reality Lab

## ■ TU Mixed Reality Lab (2026)

- All 8 faculties contributing
- 30x5x4m room (Fav.str. 9, EG)
- Precise motion tracking in whole room
- >50 HMDs (VR & AR) for teaching and research
- LED Cave, LED Wall, Holographic displays
- Hosting Spot robot, Lidar scanners & hardware from contributors
- Splitting of room possible for multiple use-case scenarios
- Lab manager (technician/senior scientist)





# Organisational: Multimedia VO - Dates

- VU 4.0 SwS, 6.0 ECTS, LVA 193.148
- Tuesday 14:00-15:30 – starts on time! (s.t.)
- Begin: 10.3.2026 – End 19.5.2026
  
- In Presence in HS Fav 1

# VO – Dates

## Vorlesungstermine / Lecture Dates:

1. VO: 10.3. 14:00-16:00
2. VO: Inverted Classroom - Self Learning (Videos) please watch until 17.3. to understand the next lecture
3. VO: 17.3. 14:00-16:00
  - Conference & Easter Break
4. VO: 14.4. 14:00-16:00
5. VO: 21.4. 14:00-16:00
6. VO: 28.4. 14:00-16:00
7. VO: 5.5. 14:00-16:00
8. VO: 12.5. 14:00-16:00
9. VO: Inverted Classroom - Self Learning (Videos)
10. VO: 19.5. 14:00-16:00
11. VO: Inverted Classroom - Self Learning (Videos)



# Exam

- Written Exam (next 2 dates)
  - Tue 9.6.2026, 10:00 - 12:00, EI 7 HS (Gußhausstr.)
  - Tue 23.6.2026, 10:00 – 12:00, EI 7 HS (Gußhausstr.)
- Contact:
  - Tel.: 01/58801 18860
  - Email: [Hannes.kaufmann@tuwien.ac.at](mailto:Hannes.kaufmann@tuwien.ac.at)
  - Favoritenstr. 9-11; 4. Stock; Stiege 3; HC 04 15

# Grading

- Written exam for lecture part
- Grade of the practical part (group-work) results from quality of the hand-ins (defined by certain criteria) and the oral hand-in meeting.
- **To take the written exam, the practical part must be positive.**
- For a positive VU grade, the grade for the lecture part and for the practical part must be positive (>50% each).
- Final VU grade:  
( 66% lecture grade + 33% practical grade) / 2

# Content

- Multimedia Definition
- 4 Media Types:
  - Text
  - Audio: Psychoacoustics, Digital Audio, MIDI, Compression Standards (MP3)  
Spatial Audio (Binaural, Ambisonics)
  - Image: Color, HDR, Compression (e.g. JPEG)
  - Video: History (analog, TV), Compression (MPEG, H.264)
  - Other Human Senses
- Compression Standards
- Network Distribution/Streaming
- Examples

# Other Lectures offered by VR-Group

- Audio & Video Produktion (von Karyn Laudisi)
- Bachelorarbeit für Informatik und Wirtschaftsinformatik

Master:

- Virtual and Augmented Reality VO + LU, 2+4 ECTS (SS)
- Virtual Reality Advanced Topics; 188.456; 3 ECTS (in WS)
- Multimedia Interfaces VU (in WS)
- Praktikum
  - Praktikum aus Visual Computing 1 & 2
  - Projekt Medizinische Informatik
  - Project in Computer Science 1 & 2
- Diplomarbeiten

# Questions?

VO – Videos and Slides online in TUWEL course, linked in TISS.

# Multimedia

## Vorlesung

Hannes Kaufmann

Research Unit Virtual and Augmented Reality  
Institute of Visual Computing & Human-Centered Technology



# Media History Lesson

## Analog Mass Media: Radio, TV & Video Recording

### and Preparation for next Lectures

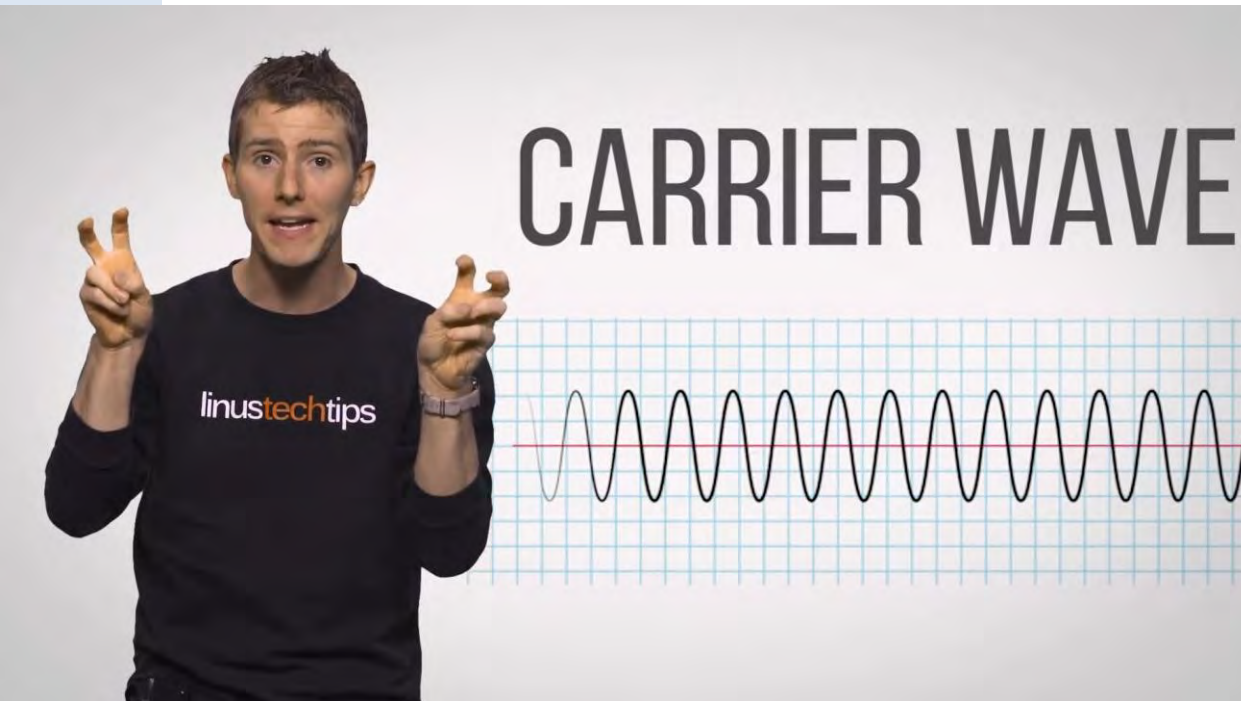
## HDR Video, Modern Codecs and Filmmaking

# Important Questions

- What is modulation?
- What is a carrier wave?
- What is the purpose of modulation?
- What are the 2 different kinds of modulation that are used for AM and FM radio?
- What is the basic functionality/components needed in a radio?

# How AM and FM (=UKW) Radio works

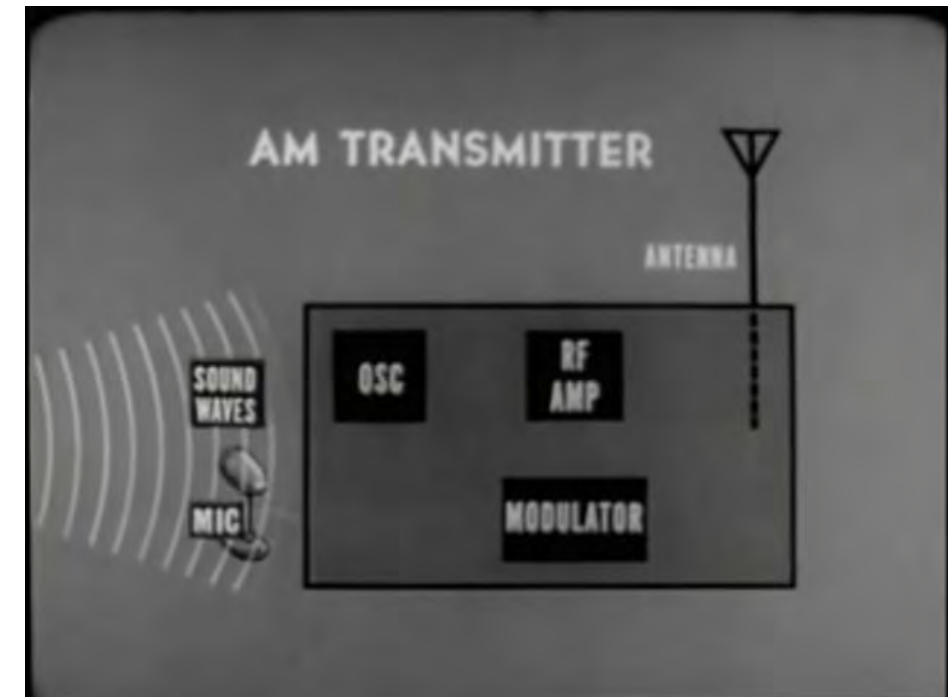
Simple fast explanation



<https://www.youtube.com/watch?v=w3M4cqAMWQs>

Very good, more detailed explanation

Highly edited version of US Army training video (TF11-3482 - Frequency Modulation Part I: Basic Principles, 1964) on the basics of AM & FM transmission.

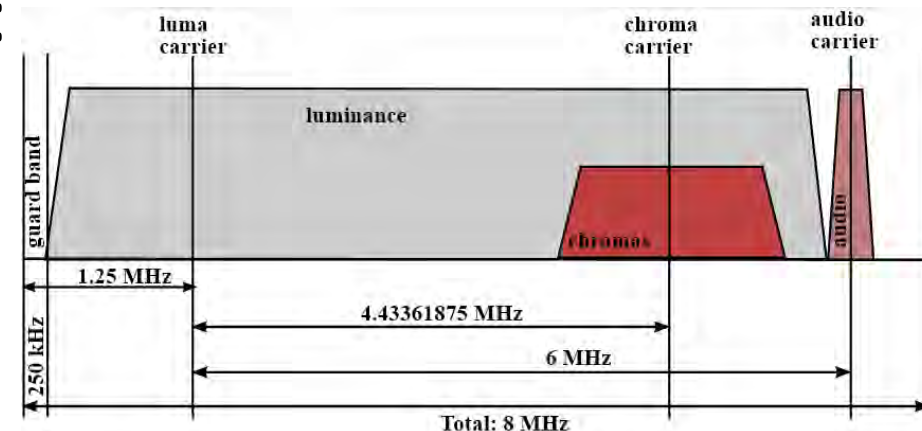


Hint: Turn on  
Captions  
(Subtitles) in  
Youtube for  
better  
understanding

<https://www.youtube.com/watch?v=D65KXwfDs3s>

# Relevant Questions

- How does a cathode ray tube TV work?
- How are colored dots on an analog TV generated?
- What is interlacing?
- How does a vacuum tube camera based on an electron beam work?
- In addition: Look up the PAL standard:
  - How many visible lines are displayed?
  - Find out what this diagram shows:





# How Was Video Invented?



# Relevant Questions

- Explain at least 2 methods that were used to store data before hard disk drives were invented.
- What are examples of portable storage before the invention of solid state storage like USB Sticks?
- How is data stored on an optical disc (e.g. CD / DVD) ?
- Look up online: How much faster is internal RAM compared to SSD storage?



# Brief History of (Digital) Storage



# In Preparation for next Lecture

- Overview of Codecs & Streaming
- Explanation of the 3 relevant HDR Video Standards & Differences

# Relevant Questions

- What is the main difference between HDR 10 and Dolby Vision?
- What kind of metadata is transmitted in Dolby Vision?
- What is Adaptive Bitrate Streaming and which standards are mentioned that use it?

# HDR Video Standards Explained



<https://www.youtube.com/watch?v=tzm2XjcyKDQ>

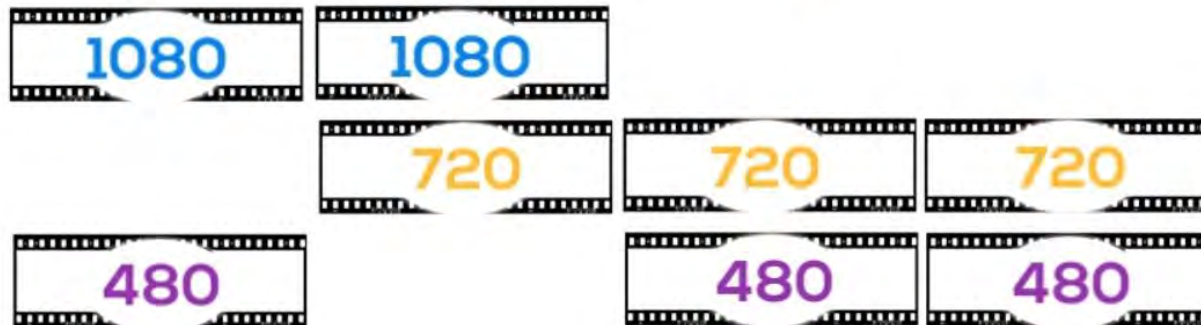
# Video Codecs Containers Today

## VIDEO FORMATS

FORMATS

MP4

## Adaptive Bitrate Segmentation



MEDIUM

SLOW

FAST

FAST





# Relevant Questions

- If you do not understand the video on the next slide, watch this one first

**“Explaining Digital Video: Formats, Codecs & Containers”**

<https://www.youtube.com/watch?v=-4NXxY4maYc>

(compression will be explained within the lecture)

What you should know after watching the video(s):

- What is a video container? (explained in the video above)
- What are common video codecs? (explained in the video above)
- What is the Profile of a Codec? What is contained in the profile?
- What are the Levels of a Codec? Give an example of what is specified in a level.
- For which purposes is the H.264 codec used?
- What are disadvantages of H.264 compared to newer codecs?
- What is “All-I” (All-Intra) video compression? (should become more clear in our next lecture)
- What is the purpose of an Intermediary Codec?
- Which two Intermediary Codecs are mentioned and what is the advantage of these intermediary codecs?
- What is the disadvantage of using ProRes and DNxHR for normal video recording?



# Modern Video Codecs & Workflow

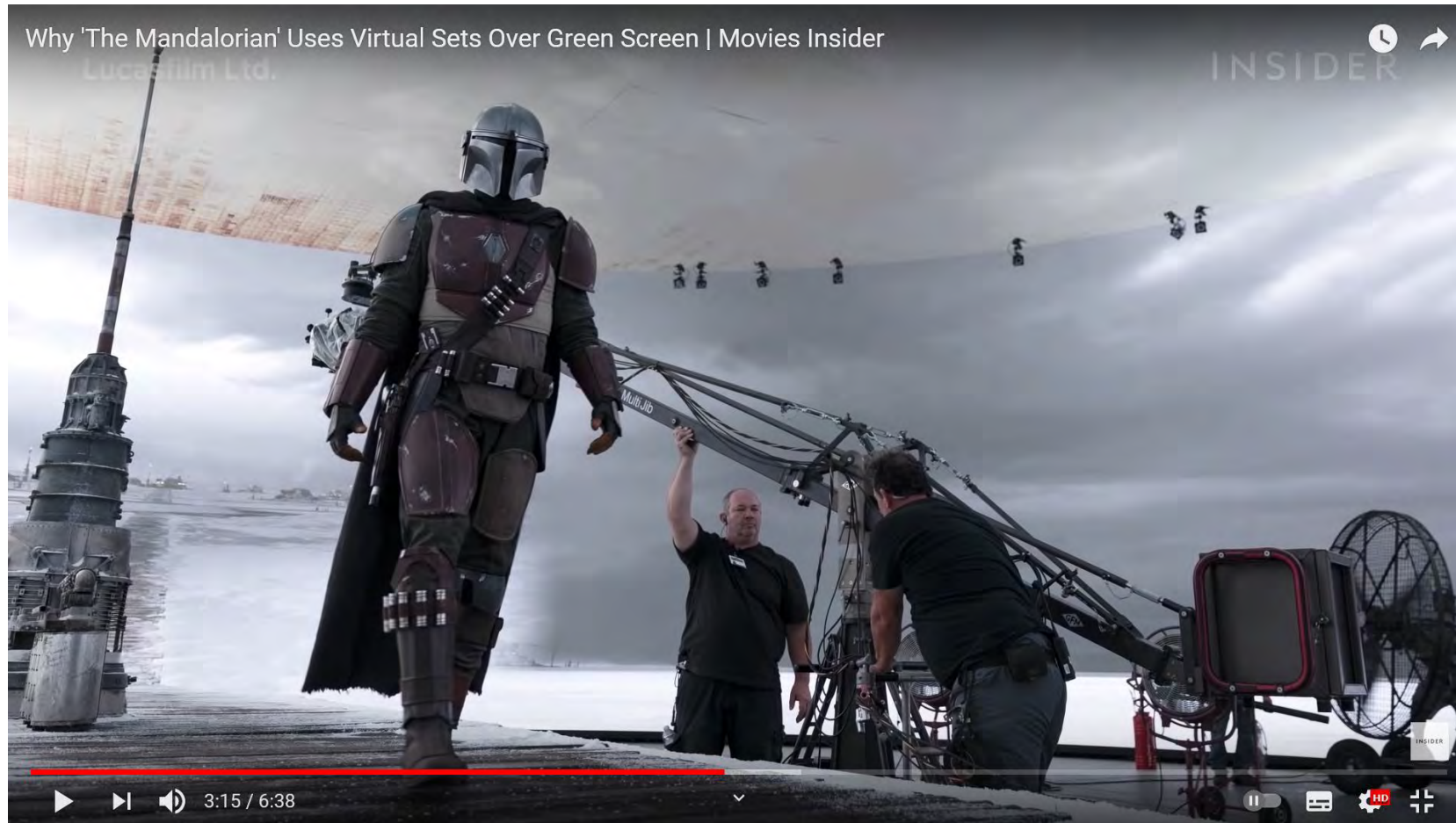


Hint: Reduce playback rate to **at least** a factor of 0.9 or 0.85

# Relevant Questions

- What is the purpose of an LED wall or green screen?
- What are the advantages of using LED walls over green screens?
- Why is it necessary to motion track cameras in a virtual set?
- What technology is used and needed in a virtual set consisting of LED screens?
- How is it possible to change the virtual set in real time?
- Optional:
  - If interested, see also the optional video on the last slide about color grading in film with LUTs
  - There is an optional video about previsualization in movies (see below) on the example of recent Marvel movies

# Modern Film Making: Virtual Set instead of Green Screen



# Optional Videos

# Relevant Questions

- What is the challenge building a tape video recorder?
- What is the innovation to solve these challenges?
- How is video and audio information stored on a video tape?
- Optional - if you are interested:

How cassette tape recorders work

<https://electronics.howstuffworks.com/gadgets/audio-music/cassette.htm>



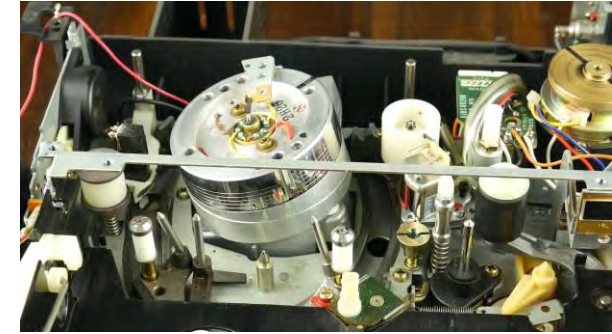
# Optional: Video Cassette Recording Explained



<https://www.youtube.com/watch?v=KfuARMCyTvg>

Hint: Reduce  
playback rate to a  
factor of 0.9

Hint: Turn on  
Captions (CC -  
Subtitles) in  
Youtube for better  
understanding.



Relevant parts (in minutes):

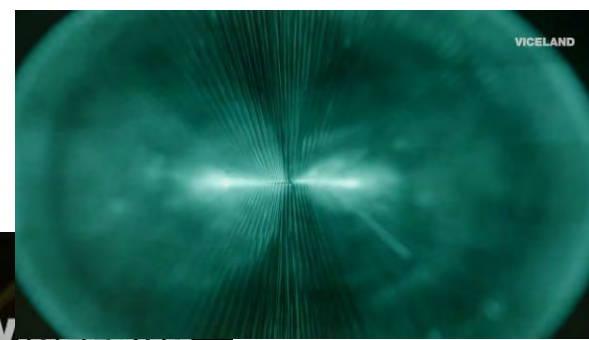
Beginning → 7:00 : Development of a consumer VCR

9:30 – 10:00 : How the signal is stored on the tape

11:40 - end : Demonstration of VHS quality & SVHS explained



# Optional: Scanimate



Hint: Turn on  
Captions (CC -  
Subtitles) in  
Youtube for better  
understanding.

# Relevant Questions

- What are 2 fields in interlaced video?
- At position 5:45min in the video: Explain how quantization is related to this statement.
  - What are disadvantages of analog signal transmission or storage?
- What is the advantage of progressive scan?
- What is the factor of pixels when scaling from Full HD to 4K or from 4K to 8K resolution?

# Optional: Home Video: Crash Course Film History



Hint: Reduce  
playback rate to  
a factor of 0.85

Hint: Turn on  
Captions (CC -  
Subtitles) in  
Youtube for better  
understanding.

<https://www.youtube.com/watch?v=VfJ-6nQAmtk>

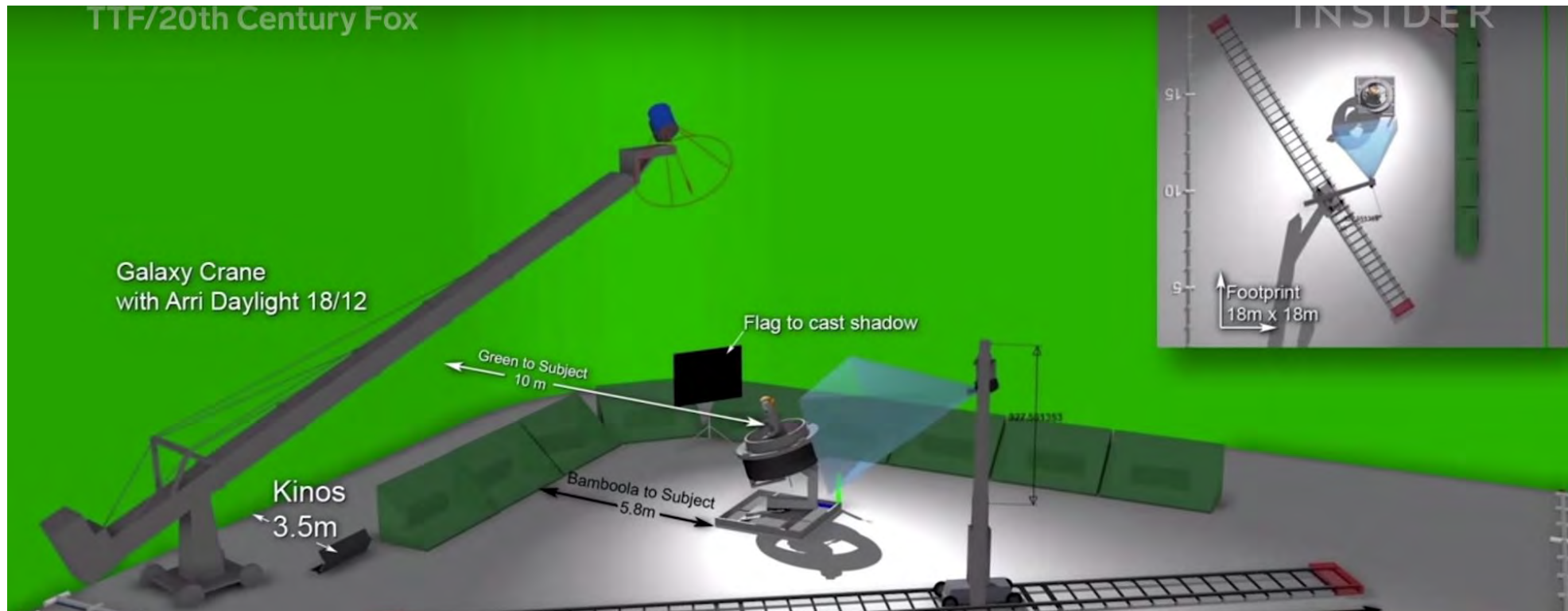


# Optional: Excellent Video about Dynamic Range – From a Filmmaker



<https://www.youtube.com/watch?v=2sshGdMgJxQ>

# Modern Film Making – The Role of Previsualization (in Marvel Movies)



<https://www.youtube.com/watch?v=bgvgi3ShcmY>

# Optional

## Color Grading Explained: LUTs (Look Up Tables)

- Read through this website  
<https://www.studiobinder.com/blog/what-is-lut/>
- The first video “LUTS: The complete video guide” gives a good introduction (expecially from 3:00 – 5:30min)
- Understand 1D LUTs and 3D LUTs